

Oracle®VirtualBox®

Programming Guide and Reference

Version 7.2.22

Copyright © 2004-2026 Oracle and/or its affiliates

http://www.virtualbox.org

Contents

1 Introduction **1**
- 1.1 Modularity: the building blocks of VirtualBox . . . 1
- 1.2 Two guises of the same “Main API”: the web service or COM/XPCOM . . . 2
- 1.3 About web services in general . . . 3
- 1.4 Running the web service . . . 4
  - 1.4.1 Command line options of vboxwebsrv . . . 4
  - 1.4.2 Authenticating at web service logon . . . 5

2 Environment-specific notes **7**
- 2.1 Using the object-oriented web service (OOWS) . . . 7
  - 2.1.1 The object-oriented web service for JAX-WS . . . 7
  - 2.1.2 The object-oriented web service for Python . . . 9
  - 2.1.3 The object-oriented web service for PHP . . . 10
- 2.2 Using the raw web service with any language . . . 10
  - 2.2.1 Raw web service example for Java with Axis . . . 10
  - 2.2.2 Raw web service example for Perl . . . 12
  - 2.2.3 Programming considerations for the raw web service . . . 12
- 2.3 Using COM/XPCOM directly . . . 16
  - 2.3.1 Python COM API . . . 17
  - 2.3.2 Common Python bindings layer . . . 17
  - 2.3.3 C++ COM API . . . 18
  - 2.3.4 Event queue processing . . . 19
  - 2.3.5 Visual Basic and Visual Basic Script (VBS) on Windows hosts . . . 20
  - 2.3.6 C bindings to the VirtualBox API . . . 20

3 Basic VirtualBox concepts; some examples **28**
- 3.1 Obtaining basic machine information. Reading attributes . . . 28
- 3.2 Changing machine settings: Sessions . . . 28
- 3.3 Launching virtual machines . . . 29
- 3.4 VirtualBox events . . . 29

4 The VirtualBox shell **31**

5 Main API changes to support the ARM64 architecture **33**
- 5.1 Overview of the Changes to the Main API . . . 33
  - 5.1.1 Changes to Classes (Interfaces) . . . 35
  - 5.1.2 Changes to Enumerations . . . 36

6 Classes (interfaces) **38**
- 6.1 IAdditionsFacility . . . 38
  - 6.1.1 Attributes . . . 38
- 6.2 IAdditionsStateChangedEvent (IEvent) . . . 38
  - 6.2.1 Attributes . . . 39
- 6.3 IAppliance . . . 39
  - 6.3.1 Attributes . . . 40
  - 6.3.2 addPasswords . . . 41
  - 6.3.3 createVFSExplorer . . . 41

6.3.4 createVirtualSystemDescriptions . . . 41
6.3.5 getMediumIdsForPasswordId . . . 41
6.3.6 getPasswordIds . . . 42
6.3.7 getWarnings . . . 42
6.3.8 importMachines . . . 42
6.3.9 interpret . . . 42
6.3.10 read . . . 42
6.3.11 write . . . 43
6.4 IAudioAdapter . . . 43
6.4.1 Attributes . . . 43
6.4.2 getProperty . . . 44
6.4.3 setProperty . . . 44
6.5 IAudioAdapterChangedEvent (IEvent) . . . 44
6.5.1 Attributes . . . 45
6.6 IAudioSettings . . . 45
6.6.1 Attributes . . . 45
6.6.2 getHostAudioDevice . . . 45
6.6.3 setHostAudioDevice . . . 45
6.7 IBandwidthControl . . . 45
6.7.1 Attributes . . . 45
6.7.2 createBandwidthGroup . . . 46
6.7.3 deleteBandwidthGroup . . . 46
6.7.4 getAllBandwidthGroups . . . 46
6.7.5 getBandwidthGroup . . . 46
6.8 IBandwidthGroup . . . 46
6.8.1 Attributes . . . 46
6.9 IBandwidthGroupChangedEvent (IEvent) . . . 47
6.9.1 Attributes . . . 47
6.10 IBooleanFormValue (IFormValue) . . . 47
6.10.1 getSelected . . . 47
6.10.2 setSelected . . . 47
6.11 ICPUChangedEvent (IEvent) . . . 47
6.11.1 Attributes . . . 48
6.12 ICPUExecutionCapChangedEvent (IEvent) . . . 48
6.12.1 Attributes . . . 48
6.13 ICPUProfile . . . 48
6.13.1 Attributes . . . 48
6.14 ICanShowWindowEvent (IVetoEvent) . . . 49
6.14.1 Attributes . . . 49
6.15 ICertificate . . . 49
6.15.1 Attributes . . . 49
6.15.2 isCurrentlyExpired . . . 51
6.15.3 queryInfo . . . 51
6.16 IChoiceFormValue (IFormValue) . . . 51
6.16.1 Attributes . . . 52
6.16.2 getSelectedIndex . . . 52
6.16.3 setSelectedIndex . . . 52
6.17 IClipboardErrorEvent (IClipboardEvent) . . . 52
6.17.1 Attributes . . . 52
6.18 IClipboardEvent (IEvent) . . . 52
6.18.1 Attributes . . . 52
6.19 IClipboardFileTransferModeChangedEvent (IEvent) . . . 53
6.19.1 Attributes . . . 53

6.20 IClipboardModeChangedEvent (IEvent) . . . 53
6.20.1 Attributes . . . 53
6.21 ICloudClient . . . 53
6.21.1 Attributes . . . 53
6.21.2 addCloudMachine . . . 54
6.21.3 cloneInstance . . . 54
6.21.4 createCloudMachine . . . 54
6.21.5 createImage . . . 54
6.21.6 deleteImage . . . 54
6.21.7 exportImage . . . 55
6.21.8 exportVM . . . 55
6.21.9 getCloudMachine . . . 55
6.21.10 getExportDescriptionForm . . . 55
6.21.11 getImageInfo . . . 55
6.21.12 getImportDescriptionForm . . . 56
6.21.13 getInstanceInfo . . . 56
6.21.14 getLaunchDescriptionForm . . . 56
6.21.15 getMetricTypeByName . . . 56
6.21.16 getSubnetSelectionForm . . . 56
6.21.17 getVnicInfo . . . 57
6.21.18 importImage . . . 57
6.21.19 importInstance . . . 57
6.21.20 launchVM . . . 57
6.21.21 listBootVolumes . . . 57
6.21.22 listImages . . . 58
6.21.23 listInstances . . . 58
6.21.24 listSourceBootVolumes . . . 58
6.21.25 listSourceInstances . . . 58
6.21.26 listVnicAttachments . . . 59
6.21.27 pauseInstance . . . 59
6.21.28 readCloudMachineList . . . 59
6.21.29 readCloudMachineStubList . . . 59
6.21.30 resetInstance . . . 59
6.21.31 setupCloudNetworkEnvironment . . . 59
6.21.32 startCloudNetworkGateway . . . 60
6.21.33 startInstance . . . 60
6.21.34 terminateInstance . . . 60
6.22 ICloudMachine . . . 60
6.22.1 Attributes . . . 60
6.22.2 createConsoleConnection . . . 62
6.22.3 deleteConsoleConnection . . . 62
6.22.4 enumerateMetricData . . . 62
6.22.5 getConsoleHistory . . . 63
6.22.6 getDetailsForm . . . 63
6.22.7 getSettingsForm . . . 63
6.22.8 listMetricNames . . . 63
6.22.9 powerDown . . . 63
6.22.10 powerUp . . . 63
6.22.11 reboot . . . 64
6.22.12 refresh . . . 64
6.22.13 remove . . . 64
6.22.14 reset . . . 64
6.22.15 shutdown . . . 64

6.22.16 terminate . . . 64
6.22.17 unregister . . . 64
6.23 ICloudNetwork . . . 64
6.23.1 Attributes . . . 64
6.24 ICloudNetworkEnvironmentInfo . . . 65
6.24.1 Attributes . . . 65
6.25 ICloudNetworkGatewayInfo . . . 65
6.25.1 Attributes . . . 65
6.26 ICloudProfile . . . 65
6.26.1 Attributes . . . 65
6.26.2 createCloudClient . . . 66
6.26.3 getProperties . . . 66
6.26.4 getProperty . . . 66
6.26.5 remove . . . 66
6.26.6 setProperties . . . 67
6.26.7 setProperty . . . 67
6.27 ICloudProfileChangedEvent (IEvent) . . . 67
6.27.1 Attributes . . . 67
6.28 ICloudProfileRegisteredEvent (IEvent) . . . 68
6.28.1 Attributes . . . 68
6.29 ICloudProvider . . . 68
6.29.1 Attributes . . . 68
6.29.2 createProfile . . . 69
6.29.3 getProfileByName . . . 69
6.29.4 getPropertyDescription . . . 69
6.29.5 importProfiles . . . 69
6.29.6 prepareUninstall . . . 69
6.29.7 restoreProfiles . . . 69
6.29.8 saveProfiles . . . 70
6.30 ICloudProviderListChangedEvent (IEvent) . . . 70
6.30.1 Attributes . . . 70
6.31 ICloudProviderManager . . . 70
6.31.1 Attributes . . . 70
6.31.2 getProviderById . . . 70
6.31.3 getProviderByName . . . 70
6.31.4 getProviderByShortName . . . 70
6.32 ICloudProviderRegisteredEvent (IEvent) . . . 71
6.32.1 Attributes . . . 71
6.33 ICloudProviderUninstallEvent (IEvent) . . . 71
6.33.1 Attributes . . . 71
6.34 IConsole . . . 71
6.34.1 Attributes . . . 71
6.34.2 addEncryptionPassword . . . 74
6.34.3 addEncryptionPasswords . . . 74
6.34.4 attachUSBDevice . . . 75
6.34.5 clearAllEncryptionPasswords . . . 75
6.34.6 createSharedFolder . . . 75
6.34.7 detachUSBDevice . . . 76
6.34.8 findUSBDeviceByAddress . . . 76
6.34.9 findUSBDeviceById . . . 76
6.34.10 getDeviceActivity . . . 76
6.34.11 getGuestEnteredACPIMode . . . 77
6.34.12 getPowerButtonHandled . . . 77

6.34.13 pause . . . 77
6.34.14 powerButton . . . 77
6.34.15 powerDown . . . 77
6.34.16 powerUp . . . 78
6.34.17 powerUpPaused . . . 78
6.34.18 removeEncryptionPassword . . . 78
6.34.19 removeSharedFolder . . . 79
6.34.20 reset . . . 79
6.34.21 resume . . . 79
6.34.22 sleepButton . . . 79
6.34.23 teleport . . . 79
6.35 ICursorPositionChangedEvent (IEvent) . . . 80
6.35.1 Attributes . . . 80
6.36 IDHCPConfig . . . 80
6.36.1 Attributes . . . 81
6.36.2 getAllOptions . . . 81
6.36.3 getOption . . . 82
6.36.4 remove . . . 82
6.36.5 removeAllOptions . . . 82
6.36.6 removeOption . . . 82
6.36.7 setOption . . . 82
6.37 IDHCPGlobalConfig (IDHCPConfig) . . . 82
6.38 IDHCPGroupCondition . . . 83
6.38.1 Attributes . . . 83
6.38.2 remove . . . 83
6.39 IDHCPGroupConfig (IDHCPConfig) . . . 83
6.39.1 Attributes . . . 83
6.39.2 addCondition . . . 84
6.39.3 removeAllConditions . . . 84
6.40 IDHCPIndividualConfig (IDHCPConfig) . . . 84
6.40.1 Attributes . . . 84
6.41 IDHCPServer . . . 85
6.41.1 Attributes . . . 85
6.41.2 findLeaseByMAC . . . 86
6.41.3 getConfig . . . 86
6.41.4 restart . . . 87
6.41.5 setConfiguration . . . 87
6.41.6 start . . . 87
6.41.7 stop . . . 87
6.42 IDataStream . . . 87
6.42.1 Attributes . . . 88
6.42.2 read . . . 88
6.43 IDirectory . . . 88
6.43.1 Attributes . . . 88
6.43.2 close . . . 89
6.43.3 list . . . 89
6.43.4 read . . . 89
6.43.5 rewind . . . 89
6.44 IDisplay . . . 89
6.44.1 Attributes . . . 90
6.44.2 attachFramebuffer . . . 90
6.44.3 createGuestScreenInfo . . . 90
6.44.4 detachFramebuffer . . . 90

6.44.5 detachScreens . . . 91
6.44.6 drawToScreen . . . 91
6.44.7 getScreenResolution . . . 91
6.44.8 getVideoModeHint . . . 92
6.44.9 invalidateAndUpdate . . . 92
6.44.10 invalidateAndUpdateScreen . . . 93
6.44.11 notifyHiDPIOutputPolicyChange . . . 93
6.44.12 notifyScaleFactorChange . . . 93
6.44.13 queryFramebuffer . . . 93
6.44.14 querySourceBitmap . . . 93
6.44.15 setScreenLayout . . . 94
6.44.16 setSeamlessMode . . . 94
6.44.17 setVideoModeHint . . . 94
6.44.18 takeScreenShot . . . 95
6.44.19 takeScreenShotToArray . . . 96
6.44.20 viewportChanged . . . 96
6.45 IDisplaySourceBitmap . . . 96
6.45.1 Attributes . . . 96
6.45.2 queryBitmapInfo . . . 97
6.46 IDnDBase . . . 97
6.46.1 Attributes . . . 97
6.46.2 addFormats . . . 97
6.46.3 isFormatSupported . . . 97
6.46.4 removeFormats . . . 98
6.47 IDnDModeChangedEvent (IEvent) . . . 98
6.47.1 Attributes . . . 98
6.48 IDnDSource (IDnDBase) . . . 98
6.48.1 dragIsPending . . . 98
6.48.2 drop . . . 99
6.48.3 receiveData . . . 99
6.49 IDnDTarget (IDnDBase) . . . 99
6.49.1 cancel . . . 99
6.49.2 drop . . . 99
6.49.3 enter . . . 100
6.49.4 leave . . . 100
6.49.5 move . . . 101
6.49.6 sendData . . . 101
6.50 IEmulatedUSB . . . 101
6.50.1 Attributes . . . 101
6.50.2 webcamAttach . . . 102
6.50.3 webcamDetach . . . 102
6.51 IEvent . . . 102
6.51.1 Attributes . . . 103
6.51.2 setProcessed . . . 103
6.51.3 waitProcessed . . . 103
6.52 IEventListener . . . 104
6.52.1 handleEvent . . . 104
6.53 IEventSource . . . 104
6.53.1 createAggregator . . . 104
6.53.2 createListener . . . 104
6.53.3 eventProcessed . . . 104
6.53.4 fireEvent . . . 105
6.53.5 getEvent . . . 105

6.53.6 registerListener . . . 105
6.53.7 unregisterListener . . . 106
6.54 IEventSourceChangedEvent (IEvent) . . . 106
6.54.1 Attributes . . . 106
6.55 IExtPack (IExtPackBase) . . . 106
6.55.1 queryObject . . . 107
6.56 IExtPackBase . . . 107
6.56.1 Attributes . . . 107
6.56.2 queryLicense . . . 109
6.57 IExtPackFile (IExtPackBase) . . . 109
6.57.1 Attributes . . . 109
6.57.2 install . . . 109
6.58 IExtPackInstalledEvent (IEvent) . . . 109
6.58.1 Attributes . . . 110
6.59 IExtPackManager . . . 110
6.59.1 Attributes . . . 110
6.59.2 cleanup . . . 110
6.59.3 find . . . 110
6.59.4 isExtPackUsable . . . 110
6.59.5 openExtPackFile . . . 111
6.59.6 queryAllPlugInsForFrontend . . . 111
6.59.7 uninstall . . . 111
6.60 IExtPackPlugIn . . . 111
6.60.1 Attributes . . . 111
6.61 IExtPackUninstalledEvent (IEvent) . . . 112
6.61.1 Attributes . . . 112
6.62 IExtraDataCanChangeEvent (IVetoEvent) . . . 112
6.62.1 Attributes . . . 112
6.63 IExtraDataChangedEvent (IEvent) . . . 113
6.63.1 Attributes . . . 113
6.64 IFile . . . 113
6.64.1 Attributes . . . 113
6.64.2 close . . . 114
6.64.3 queryInfo . . . 115
6.64.4 querySize . . . 115
6.64.5 read . . . 115
6.64.6 readAt . . . 115
6.64.7 seek . . . 115
6.64.8 setACL . . . 116
6.64.9 setSize . . . 116
6.64.10 write . . . 116
6.64.11 writeAt . . . 116
6.65 IFirmwareSettings . . . 117
6.65.1 Attributes . . . 117
6.66 IForm . . . 118
6.66.1 Attributes . . . 118
6.66.2 apply . . . 118
6.66.3 getFieldGroup . . . 118
6.67 IFormValue . . . 119
6.67.1 Attributes . . . 119
6.68 IFramebuffer . . . 119
6.68.1 Attributes . . . 119
6.68.2 getVisibleRegion . . . 121

6.68.3 notify3DEvent . . . 121
6.68.4 notifyChange . . . 121
6.68.5 notifyUpdate . . . 122
6.68.6 notifyUpdateImage . . . 122
6.68.7 processVHWACommand . . . 122
6.68.8 setVisibleRegion . . . 123
6.68.9 videoModeSupported . . . 123
6.69 IFramebufferOverlay (IFramebuffer) . . . 124
6.69.1 Attributes . . . 124
6.69.2 move . . . 124
6.70 IFsInfo . . . 124
6.70.1 Attributes . . . 125
6.71 IFsObjInfo . . . 126
6.71.1 Attributes . . . 126
6.72 IGraphicsAdapter . . . 128
6.72.1 Attributes . . . 128
6.72.2 isFeatureEnabled . . . 129
6.72.3 setFeature . . . 129
6.73 IGuest . . . 129
6.73.1 Attributes . . . 129
6.73.2 createSession . . . 131
6.73.3 findSession . . . 131
6.73.4 getAdditionsStatus . . . 131
6.73.5 getFacilityStatus . . . 132
6.73.6 internalGetStatistics . . . 132
6.73.7 setCredentials . . . 133
6.73.8 shutdown . . . 133
6.73.9 updateGuestAdditions . . . 133
6.74 IGuestAdditionsStatusChangedEvent (IEvent) . . . 134
6.74.1 Attributes . . . 134
6.75 IGuestDebugControl . . . 134
6.75.1 Attributes . . . 135
6.76 IGuestDebugControlChangedEvent (IEvent) . . . 135
6.76.1 Attributes . . . 135
6.77 IGuestDirectory (IDirectory) . . . 135
6.77.1 Attributes . . . 135
6.78 IGuestDirectoryEvent (IGuestSessionEvent) . . . 136
6.78.1 Attributes . . . 136
6.79 IGuestDirectoryReadEvent (IGuestDirectoryEvent) . . . 136
6.79.1 Attributes . . . 136
6.80 IGuestDirectoryRegisteredEvent (IGuestDirectoryEvent) . . . 136
6.80.1 Attributes . . . 137
6.81 IGuestDirectoryStateChangedEvent (IGuestDirectoryEvent) . . . 137
6.81.1 Attributes . . . 137
6.82 IGuestDnDSource (IDnDSource) . . . 137
6.82.1 Attributes . . . 137
6.83 IGuestDnDTarget (IDnDTarget) . . . 137
6.83.1 Attributes . . . 138
6.84 IGuestFile (IFile) . . . 138
6.84.1 Attributes . . . 138
6.85 IGuestFileEvent (IGuestSessionEvent) . . . 138
6.85.1 Attributes . . . 138
6.86 IGuestFileIOEvent (IGuestFileEvent) . . . 138

6.86.1 Attributes . . . 138
6.87 IGuestFileOffsetChangedEvent (IGuestFileIOEvent) . . . 139
6.87.1 Attributes . . . 139
6.88 IGuestFileReadEvent (IGuestFileIOEvent) . . . 139
6.88.1 Attributes . . . 139
6.89 IGuestFileRegisteredEvent (IGuestFileEvent) . . . 139
6.89.1 Attributes . . . 139
6.90 IGuestFileSizeChangedEvent (IGuestFileEvent) . . . 140
6.90.1 Attributes . . . 140
6.91 IGuestFileStateChangedEvent (IGuestFileEvent) . . . 140
6.91.1 Attributes . . . 140
6.92 IGuestFileWriteEvent (IGuestFileIOEvent) . . . 140
6.92.1 Attributes . . . 140
6.93 IGuestFsInfo (IFsInfo) . . . 141
6.93.1 Attributes . . . 141
6.94 IGuestFsObjInfo (IFsObjInfo) . . . 141
6.94.1 Attributes . . . 141
6.95 IGuestKeyboardEvent (IEvent) . . . 141
6.95.1 Attributes . . . 141
6.96 IGuestMonitorChangedEvent (IEvent) . . . 141
6.96.1 Attributes . . . 142
6.97 IGuestMonitorInfoChangedEvent (IEvent) . . . 142
6.97.1 Attributes . . . 142
6.98 IGuestMouseEvent (IReusableEvent) . . . 143
6.98.1 Attributes . . . 143
6.99 IGuestMultiTouchEvent (IEvent) . . . 143
6.99.1 Attributes . . . 144
6.100 IGuestOSType . . . 144
6.100.1 Attributes . . . 145
6.101 IGuestProcess (IProcess) . . . 149
6.101.1 Attributes . . . 149
6.102 IGuestProcessEvent (IGuestSessionEvent) . . . 149
6.102.1 Attributes . . . 149
6.103 IGuestProcessIOEvent (IGuestProcessEvent) . . . 149
6.103.1 Attributes . . . 150
6.104 IGuestProcessInputNotifyEvent (IGuestProcessIOEvent) . . . 150
6.104.1 Attributes . . . 150
6.105 IGuestProcessOutputEvent (IGuestProcessIOEvent) . . . 150
6.105.1 Attributes . . . 150
6.106 IGuestProcessRegisteredEvent (IGuestProcessEvent) . . . 151
6.106.1 Attributes . . . 151
6.107 IGuestProcessStateChangedEvent (IGuestProcessEvent) . . . 151
6.107.1 Attributes . . . 151
6.108 IGuestPropertyChangedEvent (IMachineEvent) . . . 151
6.108.1 Attributes . . . 151
6.109 IGuestScreenInfo . . . 152
6.109.1 Attributes . . . 152
6.110 IGuestSession . . . 153
6.110.1 Attributes . . . 153
6.110.2 close . . . 155
6.110.3 copyFromGuest . . . 155
6.110.4 copyToGuest . . . 156
6.110.5 directoryCopy . . . 156

6.110.6 directoryCopyFromGuest . . . 157
6.110.7 directoryCopyToGuest . . . 157
6.110.8 directoryCreate . . . 157
6.110.9 directoryCreateTemp . . . 158
6.110.10directoryExists . . . 158
6.110.11directoryOpen . . . 159
6.110.12directoryRemove . . . 159
6.110.13directoryRemoveRecursive . . . 159
6.110.14environmentDoesBaseVariableExist . . . 160
6.110.15environmentGetBaseVariable . . . 160
6.110.16environmentScheduleSet . . . 161
6.110.17environmentScheduleUnset . . . 161
6.110.18fileCopy . . . 161
6.110.19fileCopyFromGuest . . . 161
6.110.20fileCopyToGuest . . . 162
6.110.21fileCreateTemp . . . 162
6.110.22fileExists . . . 163
6.110.23fileOpen . . . 163
6.110.24fileOpenEx . . . 164
6.110.25fileQuerySize . . . 164
6.110.26fsObjCopyArray . . . 165
6.110.27fsObjExists . . . 165
6.110.28fsObjMove . . . 165
6.110.29fsObjMoveArray . . . 166
6.110.30fsObjQueryInfo . . . 166
6.110.31fsObjRemove . . . 166
6.110.32fsObjRemoveArray . . . 167
6.110.33fsObjRename . . . 167
6.110.34fsObjSetACL . . . 167
6.110.35fsQueryFreeSpace . . . 168
6.110.36fsQueryInfo . . . 168
6.110.37processCreate . . . 168
6.110.38processCreateEx . . . 169
6.110.39processGet . . . 170
6.110.40symlinkCreate . . . 170
6.110.41symlinkExists . . . 171
6.110.42symlinkRead . . . 171
6.110.43waitFor . . . 171
6.110.44waitForArray . . . 171
6.111 IGuestSessionEvent (IEvent) . . . 172
6.111.1 Attributes . . . 172
6.112 IGuestSessionRegisteredEvent (IGuestSessionEvent) . . . 172
6.112.1 Attributes . . . 172
6.113 IGuestSessionStateChangedEvent (IGuestSessionEvent) . . . 172
6.113.1 Attributes . . . 172
6.114 IGuestUserStateChangedEvent (IEvent) . . . 173
6.114.1 Attributes . . . 173
6.115 IHost . . . 173
6.115.1 Attributes . . . 174
6.115.2 addUSBDeviceSource . . . 176
6.115.3 createHostOnlyNetworkInterface . . . 177
6.115.4 createUSBDeviceFilter . . . 177
6.115.5 findHostDVDDrive . . . 177

6.115.6 findHostFloppyDrive . . . 177
6.115.7 findHostNetworkInterfaceById . . . 178
6.115.8 findHostNetworkInterfaceByName . . . 178
6.115.9 findHostNetworkInterfacesOfType . . . 178
6.115.10findUSBDeviceByAddress . . . 178
6.115.11findUSBDeviceById . . . 178
6.115.12generateMACAddress . . . 179
6.115.13getProcessorDescription . . . 179
6.115.14getProcessorFeature . . . 179
6.115.15getProcessorSpeed . . . 179
6.115.16insertUSBDeviceFilter . . . 179
6.115.17isExecutionEngineSupported . . . 180
6.115.18removeHostOnlyNetworkInterface . . . 180
6.115.19removeUSBDeviceFilter . . . 180
6.115.20removeUSBDeviceSource . . . 181
6.116 IHostAudioDevice . . . 181
6.116.1 Attributes . . . 181
6.116.2 getProperty . . . 182
6.117 IHostAudioDeviceChangedEvent (IEvent) . . . 182
6.117.1 Attributes . . . 182
6.118 IHostDrive . . . 183
6.118.1 Attributes . . . 183
6.119 IHostDrivePartition . . . 184
6.119.1 Attributes . . . 184
6.120 IHostNameResolutionConfigurationChangeEvent (IEvent) . . . 186
6.120.1 Attributes . . . 186
6.121 IHostNetworkInterface . . . 186
6.121.1 Attributes . . . 186
6.121.2 DHCPRediscover . . . 187
6.121.3 enableDynamicIPConfig . . . 188
6.121.4 enableStaticIPConfig . . . 188
6.121.5 enableStaticIPConfigV6 . . . 188
6.122 IHostOnlyNetwork . . . 188
6.122.1 Attributes . . . 188
6.123 IHostPCIDevicePlugEvent (IMachineEvent) . . . 189
6.123.1 Attributes . . . 189
6.124 IHostUSBDevice (IUSBDevice) . . . 190
6.124.1 Attributes . . . 190
6.125 IHostUSBDeviceFilter (IUSBDeviceFilter) . . . 190
6.125.1 Attributes . . . 190
6.126 IHostUpdateAgent (IUpdateAgent) . . . 190
6.126.1 Attributes . . . 191
6.127 IHostVideoInputDevice . . . 191
6.127.1 Attributes . . . 191
6.128 IHostX86 . . . 191
6.128.1 getProcessorCPUIDLeaf . . . 191
6.129 IInternalMachineControl . . . 192
6.129.1 authenticateExternal . . . 192
6.129.2 autoCaptureUSBDevices . . . 192
6.129.3 beginPowerUp . . . 192
6.129.4 beginPoweringDown . . . 192
6.129.5 captureUSBDevice . . . 193
6.129.6 detachAllUSBDevices . . . 193

6.129.7 detachUSBDevice . . . 193
6.129.8 ejectMedium . . . 193
6.129.9 endPowerUp . . . 194
6.129.10endPoweringDown . . . 194
6.129.11finishOnlineMergeMedium . . . 194
6.129.12lockMedia . . . 194
6.129.13onSessionEnd . . . 194
6.129.14pullGuestProperties . . . 195
6.129.15pushGuestProperty . . . 195
6.129.16reportVmStatistics . . . 195
6.129.17runUSBDeviceFilters . . . 196
6.129.18unlockMedia . . . 197
6.129.19updateState . . . 197
6.130 IInternalProgressControl . . . 197
6.130.1 notifyComplete . . . 197
6.130.2 notifyPointOfNoReturn . . . 197
6.130.3 setCurrentOperationProgress . . . 197
6.130.4 setNextOperation . . . 198
6.130.5 waitForOtherProgressCompletion . . . 198
6.131 IInternalSessionControl . . . 198
6.131.1 Attributes . . . 198
6.131.2 accessGuestProperty . . . 199
6.131.3 assignRemoteMachine . . . 199
6.131.4 cancelSaveStateWithReason . . . 199
6.131.5 enableVMMStatistics . . . 200
6.131.6 enumerateGuestProperties . . . 200
6.131.7 onAudioAdapterChange . . . 200
6.131.8 onBandwidthGroupChange . . . 201
6.131.9 onCPUChange . . . 201
6.131.10onCPUExecutionCapChange . . . 201
6.131.11onClipboardFileTransferModeChange . . . 201
6.131.12onClipboardModeChange . . . 201
6.131.13onDnDModeChange . . . 201
6.131.14onGuestDebugControlChange . . . 202
6.131.15onHostAudioDeviceChange . . . 202
6.131.16onMediumChange . . . 202
6.131.17onNetworkAdapterChange . . . 202
6.131.18onParallelPortChange . . . 203
6.131.19onRecordingScreenStateChange . . . 203
6.131.20onRecordingStateChange . . . 203
6.131.21onSerialPortChange . . . 203
6.131.22onSharedFolderChange . . . 204
6.131.23onShowWindow . . . 204
6.131.24onStorageControllerChange . . . 204
6.131.25onStorageDeviceChange . . . 205
6.131.26onUSBControllerChange . . . 205
6.131.27onUSBDeviceAttach . . . 205
6.131.28onUSBDeviceDetach . . . 206
6.131.29onVMProcessPriorityChange . . . 206
6.131.30onVRDEServerChange . . . 206
6.131.31onlineMergeMedium . . . 206
6.131.32pauseWithReason . . . 207
6.131.33reconfigureMediumAttachments . . . 207

6.131.34resumeWithReason . . . 207
6.131.35saveStateWithReason . . . 208
6.131.36uninitialize . . . 208
6.131.37updateMachineState . . . 208
6.132 IKeyboard . . . 209
6.132.1 Attributes . . . 209
6.132.2 putCAD . . . 209
6.132.3 putScancode . . . 209
6.132.4 putScancodes . . . 209
6.132.5 putUsageCode . . . 210
6.132.6 releaseKeys . . . 210
6.133 IKeyboardLedsChangedEvent (IEvent) . . . 210
6.133.1 Attributes . . . 210
6.134 ILanguageChangedEvent (IEvent) . . . 211
6.134.1 Attributes . . . 211
6.135 IMachine . . . 211
6.135.1 Attributes . . . 211
6.135.2 addEncryptionPassword . . . 223
6.135.3 addEncryptionPasswords . . . 223
6.135.4 addStorageController . . . 224
6.135.5 addUSBController . . . 224
6.135.6 adoptSavedState . . . 224
6.135.7 applyDefaults . . . 225
6.135.8 attachDevice . . . 225
6.135.9 attachDeviceWithoutMedium . . . 227
6.135.10attachHostPCIDevice . . . 228
6.135.11canShowConsoleWindow . . . 228
6.135.12changeEncryption . . . 229
6.135.13checkEncryptionPassword . . . 229
6.135.14clearAllEncryptionPasswords . . . 229
6.135.15cloneTo . . . 230
6.135.16createSharedFolder . . . 230
6.135.17deleteConfig . . . 231
6.135.18deleteGuestProperty . . . 231
6.135.19deleteSnapshot . . . 232
6.135.20deleteSnapshotAndAllChildren . . . 232
6.135.21deleteSnapshotRange . . . 233
6.135.22detachDevice . . . 234
6.135.23detachHostPCIDevice . . . 234
6.135.24discardSavedState . . . 235
6.135.25discardSettings . . . 235
6.135.26enumerateGuestProperties . . . 235
6.135.27exportTo . . . 236
6.135.28findSnapshot . . . 236
6.135.29getBootOrder . . . 236
6.135.30getCPUStatus . . . 237
6.135.31getEffectiveParavirtProvider . . . 237
6.135.32getEncryptionSettings . . . 237
6.135.33getExtraData . . . 237
6.135.34getExtraDataKeys . . . 237
6.135.35getGuestProperty . . . 238
6.135.36getGuestPropertyTimestamp . . . 238
6.135.37getGuestPropertyValue . . . 238

6.135.38getMedium . . . 238
6.135.39getMediumAttachment . . . 239
6.135.40getMediumAttachmentsOfController . . . 239
6.135.41getNetworkAdapter . . . 239
6.135.42getParallelPort . . . 240
6.135.43getSerialPort . . . 240
6.135.44getStorageControllerByInstance . . . 240
6.135.45getStorageControllerByName . . . 240
6.135.46getUSBControllerByName . . . 241
6.135.47getUSBControllerCountByType . . . 241
6.135.48hotPlugCPU . . . 241
6.135.49hotUnplugCPU . . . 241
6.135.50launchVMProcess . . . 241
6.135.51lockMachine . . . 243
6.135.52mountMedium . . . 244
6.135.53moveTo . . . 244
6.135.54nonRotationalDevice . . . 245
6.135.55passthroughDevice . . . 245
6.135.56queryLogFilename . . . 246
6.135.57querySavedGuestScreenInfo . . . 246
6.135.58querySavedScreenshotInfo . . . 246
6.135.59readLog . . . 246
6.135.60readSavedScreenshotToArray . . . 247
6.135.61readSavedThumbnailToArray . . . 247
6.135.62removeEncryptionPassword . . . 247
6.135.63removeSharedFolder . . . 248
6.135.64removeStorageController . . . 248
6.135.65removeUSBController . . . 248
6.135.66restoreSnapshot . . . 248
6.135.67saveSettings . . . 249
6.135.68saveState . . . 249
6.135.69setAutoDiscardForDevice . . . 250
6.135.70setBandwidthGroupForDevice . . . 250
6.135.71setBootOrder . . . 251
6.135.72setExtraData . . . 251
6.135.73setGuestProperty . . . 252
6.135.74setGuestPropertyValue . . . 252
6.135.75setHotPluggableForDevice . . . 253
6.135.76setNoBandwidthGroupForDevice . . . 253
6.135.77setSettingsFilePath . . . 254
6.135.78setStorageControllerBootable . . . 254
6.135.79showConsoleWindow . . . 254
6.135.80takeSnapshot . . . 255
6.135.81temporaryEjectDevice . . . 255
6.135.82unmountMedium . . . 256
6.135.83unregister . . . 256
6.136 IMachineDataChangedEvent (IMachineEvent) . . . 258
6.136.1 Attributes . . . 258
6.137 IMachineDebugger . . . 258
6.137.1 Attributes . . . 258
6.137.2 detectOS . . . 260
6.137.3 dumpGuestCore . . . 260
6.137.4 dumpGuestStack . . . 261

6.137.5 dumpHostProcessCore . . . 261
6.137.6 dumpStats . . . 261
6.137.7 getCPULoad . . . 261
6.137.8 getRegister . . . 262
6.137.9 getRegisters . . . 262
6.137.10 getStats . . . 262
6.137.11 getUVMAndVMMFunctionTable . . . 262
6.137.12 info . . . 263
6.137.13 injectNMI . . . 263
6.137.14 loadPlugIn . . . 263
6.137.15 modifyLogDestinations . . . 263
6.137.16 modifyLogFlags . . . 263
6.137.17 modifyLogGroups . . . 263
6.137.18 queryOSKernelLog . . . 264
6.137.19 readPhysicalMemory . . . 264
6.137.20 readVirtualMemory . . . 264
6.137.21 resetStats . . . 264
6.137.22 setRegister . . . 264
6.137.23 setRegisters . . . 265
6.137.24 takeGuestSample . . . 265
6.137.25 unloadPlugIn . . . 265
6.137.26 writePhysicalMemory . . . 265
6.137.27 writeVirtualMemory . . . 266
6.138 IMachineEvent (IEvent) . . . 266
6.138.1 Attributes . . . 266
6.139 IMachineGroupsChangedEvent (IMachineEvent) . . . 266
6.139.1 Attributes . . . 266
6.140 IMachineRegisteredEvent (IMachineEvent) . . . 267
6.140.1 Attributes . . . 267
6.141 IMachineStateChangedEvent (IMachineEvent) . . . 267
6.141.1 Attributes . . . 267
6.142 IManagedObjectRef . . . 267
6.142.1 getInterfaceName . . . 268
6.142.2 release . . . 268
6.143 IMedium . . . 268
6.143.1 Attributes . . . 270
6.143.2 changeEncryption . . . 274
6.143.3 checkEncryptionPassword . . . 275
6.143.4 cloneTo . . . 275
6.143.5 cloneToBase . . . 276
6.143.6 close . . . 276
6.143.7 compact . . . 277
6.143.8 createBaseStorage . . . 277
6.143.9 createDiffStorage . . . 277
6.143.10 deleteStorage . . . 278
6.143.11 getEncryptionSettings . . . 278
6.143.12 getProperties . . . 278
6.143.13 getProperty . . . 279
6.143.14 getSnapshotIds . . . 279
6.143.15 lockRead . . . 280
6.143.16 lockWrite . . . 280
6.143.17 mergeTo . . . 281
6.143.18 moveTo . . . 282

6.143.19openForIO . . . 282
6.143.20refreshState . . . 282
6.143.21reset . . . 283
6.143.22resize . . . 283
6.143.23resizeAndCloneTo . . . 283
6.143.24setIds . . . 284
6.143.25setProperties . . . 284
6.143.26setProperty . . . 285
6.144 IMediumAttachment . . . 285
6.144.1 Attributes . . . 287
6.145 IMediumChangedEvent (IEvent) . . . 289
6.145.1 Attributes . . . 289
6.146 IMediumConfigChangedEvent (IEvent) . . . 289
6.146.1 Attributes . . . 289
6.147 IMediumFormat . . . 289
6.147.1 Attributes . . . 290
6.147.2 describeFileExtensions . . . 290
6.147.3 describeProperties . . . 290
6.148 IMediumIO . . . 291
6.148.1 Attributes . . . 291
6.148.2 close . . . 292
6.148.3 convertToStream . . . 292
6.148.4 formatFAT . . . 292
6.148.5 initializePartitionTable . . . 292
6.148.6 read . . . 293
6.148.7 write . . . 293
6.149 IMediumRegisteredEvent (IEvent) . . . 293
6.149.1 Attributes . . . 293
6.150 IMouse . . . 293
6.150.1 Attributes . . . 294
6.150.2 putEventMultiTouch . . . 295
6.150.3 putEventMultiTouchString . . . 295
6.150.4 putMouseEvent . . . 296
6.150.5 putMouseEventAbsolute . . . 296
6.151 IMouseCapabilityChangedEvent (IEvent) . . . 297
6.151.1 Attributes . . . 297
6.152 IMousePointerShape . . . 298
6.152.1 Attributes . . . 298
6.153 IMousePointerShapeChangedEvent (IEvent) . . . 299
6.153.1 Attributes . . . 299
6.154 INATEngine . . . 300
6.154.1 Attributes . . . 300
6.154.2 addRedirect . . . 302
6.154.3 getNetworkSettings . . . 303
6.154.4 removeRedirect . . . 303
6.154.5 setNetworkSettings . . . 303
6.155 INATNetwork . . . 304
6.155.1 Attributes . . . 304
6.155.2 addLocalMapping . . . 306
6.155.3 addPortForwardRule . . . 306
6.155.4 removePortForwardRule . . . 306
6.155.5 start . . . 306
6.155.6 stop . . . 306

6.156 INATNetworkAlterEvent (INATNetworkChangedEvent) . . . 307
6.156.1 Attributes . . . 307
6.157 INATNetworkChangedEvent (IEvent) . . . 307
6.157.1 Attributes . . . 307
6.158 INATNetworkCreationDeletionEvent (INATNetworkAlterEvent) . . . 307
6.158.1 Attributes . . . 307
6.159 INATNetworkPortForwardEvent (INATNetworkAlterEvent) . . . 307
6.159.1 Attributes . . . 307
6.160 INATNetworkSettingEvent (INATNetworkAlterEvent) . . . 308
6.160.1 Attributes . . . 308
6.161 INATNetworkStartStopEvent (INATNetworkChangedEvent) . . . 309
6.161.1 Attributes . . . 309
6.162 INATRedirectEvent (IMachineEvent) . . . 309
6.162.1 Attributes . . . 309
6.163 INetworkAdapter . . . 310
6.163.1 Attributes . . . 310
6.163.2 getProperties . . . 312
6.163.3 getProperty . . . 313
6.163.4 setProperty . . . 313
6.164 INetworkAdapterChangedEvent (IEvent) . . . 313
6.164.1 Attributes . . . 314
6.165 INvramStore . . . 314
6.165.1 Attributes . . . 314
6.165.2 initUefiVariableStore . . . 314
6.166 IPCIAddress . . . 314
6.166.1 Attributes . . . 315
6.166.2 asLong . . . 315
6.166.3 fromLong . . . 315
6.167 IPCIDeviceAttachment . . . 315
6.167.1 Attributes . . . 315
6.168 IParallelPort . . . 316
6.168.1 Attributes . . . 316
6.169 IParallelPortChangedEvent (IEvent) . . . 317
6.169.1 Attributes . . . 317
6.170 IPerformanceCollector . . . 317
6.170.1 Attributes . . . 318
6.170.2 disableMetrics . . . 318
6.170.3 enableMetrics . . . 319
6.170.4 getMetrics . . . 319
6.170.5 queryMetricsData . . . 319
6.170.6 setupMetrics . . . 320
6.171 IPerformanceMetric . . . 321
6.171.1 Attributes . . . 321
6.172 IPlatform . . . 322
6.172.1 Attributes . . . 322
6.173 IPlatformARM . . . 323
6.173.1 getCPUProperty . . . 323
6.173.2 setCPUProperty . . . 323
6.174 IPlatformProperties . . . 323
6.174.1 Attributes . . . 323
6.174.2 getDeviceTypesForStorageBus . . . 326
6.174.3 getMaxDevicesPerPortForStorageBus . . . 326
6.174.4 getMaxInstancesOfStorageBus . . . 326

6.174.5 getMaxInstancesOfUSBControllerType . . . 327
6.174.6 getMaxNetworkAdapters . . . 327
6.174.7 getMaxNetworkAdaptersOfType . . . 327
6.174.8 getMaxPortCountForStorageBus . . . 327
6.174.9 getMinGuestRAM . . . 327
6.174.10 getMinPortCountForStorageBus . . . 328
6.174.11 getStorageBusForControllerType . . . 328
6.174.12 getStorageControllerHotplugCapable . . . 328
6.174.13 getStorageControllerTypesForBus . . . 328
6.174.14 getSupportedGfxFeaturesForType . . . 328
6.174.15 getSupportedVRAMRange . . . 328
6.175 IPlatformX86 . . . 329
6.175.1 Attributes . . . 329
6.175.2 getCPUIDLeaf . . . 329
6.175.3 getCPUIDLeafByOrdinal . . . 329
6.175.4 getCPUProperty . . . 330
6.175.5 getHWVirtExProperty . . . 330
6.175.6 removeAllCPUIDLeaves . . . 330
6.175.7 removeCPUIDLeaf . . . 330
6.175.8 setCPUIDLeaf . . . 331
6.175.9 setCPUProperty . . . 331
6.175.10 setHWVirtExProperty . . . 331
6.176 IProcess . . . 332
6.176.1 Attributes . . . 332
6.176.2 read . . . 333
6.176.3 terminate . . . 333
6.176.4 waitFor . . . 333
6.176.5 waitForArray . . . 333
6.176.6 write . . . 334
6.176.7 writeArray . . . 334
6.177 IProgress . . . 334
6.177.1 Attributes . . . 335
6.177.2 cancel . . . 337
6.177.3 waitForCompletion . . . 337
6.177.4 waitForOperationCompletion . . . 337
6.178 IProgressCreatedEvent (IProgressEvent) . . . 337
6.178.1 Attributes . . . 338
6.179 IProgressEvent (IEvent) . . . 338
6.179.1 Attributes . . . 338
6.180 IProgressPercentageChangedEvent (IProgressEvent) . . . 338
6.180.1 Attributes . . . 338
6.181 IProgressTaskCompletedEvent (IProgressEvent) . . . 338
6.181.1 Attributes . . . 338
6.182 IRangedInteger64FormValue (IFormValue) . . . 339
6.182.1 Attributes . . . 339
6.182.2 getInteger . . . 339
6.182.3 setInteger . . . 339
6.183 IRangedIntegerFormValue (IFormValue) . . . 339
6.183.1 Attributes . . . 339
6.183.2 getInteger . . . 340
6.183.3 setInteger . . . 340
6.184 IRecordingScreenSettings . . . 340
6.184.1 Attributes . . . 340

6.184.2 isFeatureEnabled . . . 343
6.185 IRecordingScreenStateChangedEvent (IRecordingStateChangedEvent) . . . 343
6.185.1 Attributes . . . 343
6.186 IRecordingSettings . . . 343
6.186.1 Attributes . . . 344
6.186.2 getScreenSettings . . . 344
6.186.3 resume . . . 344
6.186.4 start . . . 345
6.187 IRecordingStateChangedEvent (IEvent) . . . 345
6.187.1 Attributes . . . 345
6.188 IResourceStore . . . 345
6.188.1 Attributes . . . 345
6.189 IReusableEvent (IEvent) . . . 345
6.189.1 Attributes . . . 345
6.189.2 reuse . . . 346
6.190 IRuntimeErrorEvent (IEvent) . . . 346
6.190.1 Attributes . . . 346
6.191 ISerialPort . . . 347
6.191.1 Attributes . . . 347
6.192 ISerialPortChangedEvent (IEvent) . . . 348
6.192.1 Attributes . . . 348
6.193 ISession . . . 349
6.193.1 Attributes . . . 349
6.193.2 unlockMachine . . . 350
6.194 ISessionStateChangedEvent (IMachineEvent) . . . 350
6.194.1 Attributes . . . 351
6.195 ISharedFolder . . . 351
6.195.1 Attributes . . . 351
6.196 ISharedFolderChangedEvent (IEvent) . . . 352
6.196.1 Attributes . . . 353
6.197 IShowWindowEvent (IEvent) . . . 353
6.197.1 Attributes . . . 353
6.198 ISnapshot . . . 353
6.198.1 Attributes . . . 354
6.199 ISnapshotChangedEvent (ISnapshotEvent) . . . 356
6.199.1 Attributes . . . 356
6.200 ISnapshotDeletedEvent (ISnapshotEvent) . . . 356
6.200.1 Attributes . . . 356
6.201 ISnapshotEvent (IMachineEvent) . . . 356
6.201.1 Attributes . . . 356
6.202 ISnapshotRestoredEvent (ISnapshotEvent) . . . 357
6.202.1 Attributes . . . 357
6.203 ISnapshotTakenEvent (ISnapshotEvent) . . . 357
6.203.1 Attributes . . . 357
6.204 IStateChangedEvent (IEvent) . . . 357
6.204.1 Attributes . . . 357
6.205 IStorageController . . . 357
6.205.1 Attributes . . . 358
6.206 IStorageControllerChangedEvent (IEvent) . . . 359
6.206.1 Attributes . . . 359
6.207 IStorageDeviceChangedEvent (IEvent) . . . 360
6.207.1 Attributes . . . 360
6.208 IStringArray . . . 360

6.208.1 Attributes . . . 360
6.209 IStringFormValue (IFormValue) . . . 360
6.209.1 Attributes . . . 360
6.209.2 getString . . . 361
6.209.3 setString . . . 361
6.210 ISystemProperties . . . 361
6.210.1 Attributes . . . 361
6.210.2 getCPUProfiles . . . 369
6.210.3 getDefaultIoCacheSettingForStorageController . . . 369
6.210.4 getExecutionEnginesForVmCpuArchitecture . . . 369
6.211 IToken . . . 369
6.211.1 abandon . . . 369
6.211.2 dummy . . . 370
6.212 ITrustedPlatformModule . . . 370
6.212.1 Attributes . . . 370
6.213 IUSBController . . . 370
6.213.1 Attributes . . . 370
6.214 IUSBControllerChangedEvent (IEvent) . . . 371
6.214.1 Attributes . . . 371
6.215 IUSBDevice . . . 371
6.215.1 Attributes . . . 371
6.216 IUSBDeviceFilter . . . 373
6.216.1 Attributes . . . 374
6.217 IUSBDeviceFilters . . . 375
6.217.1 Attributes . . . 375
6.217.2 createDeviceFilter . . . 375
6.217.3 insertDeviceFilter . . . 376
6.217.4 removeDeviceFilter . . . 376
6.218 IUSBDeviceStateChangedEvent (IEvent) . . . 376
6.218.1 Attributes . . . 377
6.219 IUSBProxyBackend . . . 377
6.219.1 Attributes . . . 377
6.220 IUefiVariableStore . . . 377
6.220.1 Attributes . . . 377
6.220.2 addKek . . . 378
6.220.3 addSignatureToDb . . . 378
6.220.4 addSignatureToDbx . . . 378
6.220.5 addSignatureToMok . . . 378
6.220.6 addVariable . . . 379
6.220.7 changeVariable . . . 379
6.220.8 deleteVariable . . . 379
6.220.9 enrollDefaultMsSignatures . . . 379
6.220.10 enrollOraclePlatformKey . . . 379
6.220.11 enrollPlatformKey . . . 380
6.220.12 queryVariableByName . . . 380
6.220.13 queryVariables . . . 380
6.221 IUnattended . . . 380
6.221.1 Attributes . . . 381
6.221.2 constructMedia . . . 386
6.221.3 detectIsoOS . . . 387
6.221.4 done . . . 387
6.221.5 prepare . . . 387
6.221.6 reconfigureVM . . . 387

6.222 IUpdateAgent . . . 387
6.222.1 Attributes . . . 387
6.222.2 checkFor . . . 390
6.222.3 download . . . 390
6.222.4 install . . . 390
6.222.5 rollback . . . 390
6.223 IUpdateAgentAvailableEvent (IUpdateAgentEvent) . . . 390
6.223.1 Attributes . . . 390
6.224 IUpdateAgentErrorEvent (IUpdateAgentEvent) . . . 391
6.224.1 Attributes . . . 391
6.225 IUpdateAgentEvent (IEvent) . . . 391
6.225.1 Attributes . . . 392
6.226 IUpdateAgentSettingsChangedEvent (IUpdateAgentEvent) . . . 392
6.226.1 Attributes . . . 392
6.227 IUpdateAgentStateChangedEvent (IUpdateAgentEvent) . . . 392
6.227.1 Attributes . . . 392
6.228 IVBoxSVCAvailabilityChangedEvent (IEvent) . . . 392
6.228.1 Attributes . . . 393
6.229 IVBoxSVCRegistration . . . 393
6.229.1 getVirtualBox . . . 393
6.230 IVFSExplorer . . . 393
6.230.1 Attributes . . . 393
6.230.2 cd . . . 393
6.230.3 cdUp . . . 393
6.230.4 entryList . . . 394
6.230.5 exists . . . 394
6.230.6 remove . . . 394
6.230.7 update . . . 394
6.231 IVRDEServer . . . 394
6.231.1 Attributes . . . 394
6.231.2 getVRDEProperty . . . 395
6.231.3 setVRDEProperty . . . 396
6.232 IVRDEServerChangedEvent (IEvent) . . . 396
6.232.1 Attributes . . . 396
6.233 IVRDEServerInfo . . . 396
6.233.1 Attributes . . . 396
6.234 IVRDEServerInfoChangedEvent (IEvent) . . . 398
6.234.1 Attributes . . . 398
6.235 IVetoEvent (IEvent) . . . 398
6.235.1 addApproval . . . 398
6.235.2 addVeto . . . 399
6.235.3 getApprovals . . . 399
6.235.4 getVetos . . . 399
6.235.5 isApproved . . . 399
6.235.6 isVetoed . . . 399
6.236 IVirtualBox . . . 399
6.236.1 Attributes . . . 399
6.236.2 checkFirmwarePresent . . . 403
6.236.3 composeMachineFilename . . . 404
6.236.4 createAppliance . . . 404
6.236.5 createCloudNetwork . . . 404
6.236.6 createDHCPServer . . . 405
6.236.7 createHostOnlyNetwork . . . 405

6.236.8 createMachine . . . 405
6.236.9 createMedium . . . 406
6.236.10createNATNetwork . . . 407
6.236.11createSharedFolder . . . 408
6.236.12createUnattendedInstaller . . . 408
6.236.13findCloudNetworkByName . . . 408
6.236.14findDHCPServerByNetworkName . . . 408
6.236.15findHostOnlyNetworkById . . . 409
6.236.16findHostOnlyNetworkByName . . . 409
6.236.17findMachine . . . 409
6.236.18findNATNetworkByName . . . 409
6.236.19findProgressById . . . 409
6.236.20getExtraData . . . 410
6.236.21getExtraDataKeys . . . 410
6.236.22getGuestOSDescsBySubtype . . . 410
6.236.23getGuestOSSubtypesByFamilyId . . . 410
6.236.24getGuestOSType . . . 411
6.236.25getMachineStates . . . 411
6.236.26getMachinesByGroups . . . 411
6.236.27getPlatformProperties . . . 411
6.236.28getTrackedObject . . . 411
6.236.29getTrackedObjectIds . . . 412
6.236.30openMachine . . . 412
6.236.31openMedium . . . 412
6.236.32registerMachine . . . 414
6.236.33removeCloudNetwork . . . 414
6.236.34removeDHCPServer . . . 414
6.236.35removeHostOnlyNetwork . . . 414
6.236.36removeNATNetwork . . . 414
6.236.37removeSharedFolder . . . 415
6.236.38setExtraData . . . 415
6.236.39setSettingsSecret . . . 415
6.237 IVirtualBoxClient . . . 416
6.237.1 Attributes . . . 416
6.237.2 checkMachineError . . . 416
6.238 IVirtualBoxErrorInfo . . . 416
6.238.1 Attributes . . . 417
6.239 IVirtualBoxSDS . . . 418
6.239.1 deregisterVBoxSVC . . . 418
6.239.2 launchVMProcess . . . 418
6.239.3 registerVBoxSVC . . . 419
6.240 IVirtualSystemDescription . . . 420
6.240.1 Attributes . . . 420
6.240.2 addDescription . . . 420
6.240.3 getDescription . . . 420
6.240.4 getDescriptionByType . . . 422
6.240.5 getValuesByType . . . 423
6.240.6 removeDescriptionByType . . . 423
6.240.7 setFinalValues . . . 423
6.241 IVirtualSystemDescriptionForm (IForm) . . . 423
6.241.1 getVirtualSystemDescription . . . 424
6.242 IWebsessionManager . . . 424
6.242.1 getSessionObject . . . 424

6.242.2 logoff . . . 424
6.242.3 logon . . . 424

7 Enumerations (enums) 425
7.1 APICMode . . . 425
7.2 AccessMode . . . 425
7.3 AdditionsFacilityClass . . . 425
7.4 AdditionsFacilityStatus . . . 425
7.5 AdditionsFacilityType . . . 426
7.6 AdditionsRunLevelType . . . 426
7.7 AdditionsUpdateFlag . . . 426
7.8 AudioCodecType . . . 427
7.9 AudioControllerType . . . 427
7.10 AudioDeviceState . . . 427
7.11 AudioDeviceType . . . 427
7.12 AudioDirection . . . 428
7.13 AudioDriverType . . . 428
7.14 AuthType . . . 428
7.15 AutostopType . . . 428
7.16 BandwidthGroupType . . . 429
7.17 BitmapFormat . . . 429
7.18 CPUArchitecture . . . 429
7.19 CPUPropertyTypeARM . . . 429
7.20 CPUPropertyTypeX86 . . . 430
7.21 CertificateVersion . . . 431
7.22 ChipsetType . . . 431
7.23 CleanupMode . . . 431
7.24 ClipboardMode . . . 431
7.25 CloneMode . . . 432
7.26 CloneOptions . . . 432
7.27 CloudImageState . . . 432
7.28 CloudMachineState . . . 432
7.29 DHCPConfigScope . . . 433
7.30 DHCPGroupConditionType . . . 433
7.31 DHCPOption . . . 433
7.32 DHCPOptionEncoding . . . 436
7.33 DataFlags . . . 436
7.34 DataType . . . 436
7.35 DeviceActivity . . . 436
7.36 DeviceType . . . 436
7.37 DirectoryCopyFlag . . . 437
7.38 DirectoryCreateFlag . . . 437
7.39 DirectoryOpenFlag . . . 437
7.40 DirectoryRemoveRecFlag . . . 437
7.41 DirectoryStatus . . . 437
7.42 DnDAction . . . 438
7.43 DnDMode . . . 438
7.44 ExportOptions . . . 438
7.45 FileAccessMode . . . 438
7.46 FileCopyFlag . . . 439
7.47 FileOpenAction . . . 439
7.48 FileOpenExFlag . . . 439
7.49 FileSeekOrigin . . . 440
7.50 FileSharingMode . . . 440

7.51 FileStatus . . . 440
7.52 FirmwareBootMenuMode . . . 440
7.53 FirmwareType . . . 441
7.54 FormValueType . . . 441
7.55 FramebufferCapabilities . . . 441
7.56 FsObjMoveFlag . . . 441
7.57 FsObjRenameFlag . . . 442
7.58 FsObjType . . . 442
7.59 GraphicsControllerType . . . 442
7.60 GraphicsFeature . . . 442
7.61 GuestDebugIoProvider . . . 443
7.62 GuestDebugProvider . . . 443
7.63 GuestMonitorChangedEventType . . . 443
7.64 GuestMonitorStatus . . . 443
7.65 GuestMouseEventMode . . . 443
7.66 GuestSessionStatus . . . 444
7.67 GuestSessionWaitForFlag . . . 444
7.68 GuestSessionWaitResult . . . 444
7.69 GuestShutdownFlag . . . 445
7.70 GuestUserState . . . 445
7.71 HWVirtExPropertyType . . . 447
7.72 HostNetworkInterfaceMediumType . . . 447
7.73 HostNetworkInterfaceStatus . . . 448
7.74 HostNetworkInterfaceType . . . 448
7.75 ImportOptions . . . 448
7.76 IommuType . . . 448
7.77 KeyboardHIDType . . . 448
7.78 KeyboardLED . . . 449
7.79 LockType . . . 449
7.80 MachineState . . . 449
7.81 MediumFormatCapabilities . . . 452
7.82 MediumState . . . 453
7.83 MediumType . . . 453
7.84 MediumVariant . . . 454
7.85 MetricType . . . 454
7.86 MouseButtonState . . . 454
7.87 NATAliasMode . . . 455
7.88 NATProtocol . . . 455
7.89 NetworkAdapterPromiscModePolicy . . . 455
7.90 NetworkAdapterType . . . 455
7.91 NetworkAttachmentType . . . 456
7.92 ParavirtProvider . . . 456
7.93 PartitionTableType . . . 456
7.94 PartitionType . . . 456
7.95 PartitioningType . . . 461
7.96 PathStyle . . . 461
7.97 PlatformArchitecture . . . 462
7.98 PointingHIDType . . . 462
7.99 PortMode . . . 462
7.100 ProcessCreateFlag . . . 463
7.101 ProcessInputFlag . . . 463
7.102 ProcessInputStatus . . . 463
7.103 ProcessOutputFlag . . . 464

7.104 ProcessPriority . . . 464
7.105 ProcessStatus . . . 464
7.106 ProcessWaitForFlag . . . 464
7.107 ProcessWaitResult . . . 465
7.108 ProcessorFeature . . . 465
7.109 ProxyMode . . . 466
7.110 Reason . . . 466
7.111 RecordingAudioCodec . . . 466
7.112 RecordingCodecDeadline . . . 466
7.113 RecordingDestination . . . 467
7.114 RecordingFeature . . . 467
7.115 RecordingRateControlMode . . . 467
7.116 RecordingState . . . 467
7.117 RecordingVideoCodec . . . 468
7.118 RecordingVideoScalingMode . . . 468
7.119 Scope . . . 468
7.120 ScreenLayoutMode . . . 468
7.121 SessionState . . . 469
7.122 SessionType . . . 469
7.123 SettingsVersion . . . 469
7.124 SignatureType . . . 470
7.125 StorageBus . . . 470
7.126 StorageControllerType . . . 471
7.127 SymlinkPolicy . . . 471
7.128 SymlinkReadFlag . . . 471
7.129 SymlinkType . . . 472
7.130 TouchContactState . . . 472
7.131 TpmType . . . 472
7.132 TrackedObjectState . . . 472
7.133 USBConnectionSpeed . . . 473
7.134 USBControllerType . . . 473
7.135 USBDeviceFilterAction . . . 473
7.136 USBDeviceState . . . 474
7.137 UartType . . . 474
7.138 UefiVariableAttributes . . . 475
7.139 UpdateChannel . . . 475
7.140 UpdateSeverity . . . 475
7.141 UpdateState . . . 475
7.142 VBoxEventType . . . 476
7.143 VFSType . . . 479
7.144 VMExecutionEngine . . . 479
7.145 VMProcPriority . . . 480
7.146 VirtualSystemDescriptionType . . . 480
7.147 VirtualSystemDescriptionValueType . . . 482

8 Working with the Cloud 483
8.1 OCI features . . . 483
8.2 Function ICloudClient::exportVM . . . 483
8.3 Function ICloudClient::launchVM . . . 484
8.4 Function ICloudClient::getInstanceInfo . . . 484
8.5 Function ICloudClient::importInstance . . . 485

9 Host-Guest Communication Manager 486
9.1 Virtual hardware implementation . . . 486

9.2 Protocol specification . . . 486
9.2.1 Request header . . . 486
9.2.2 Connect . . . 487
9.2.3 Disconnect . . . 488
9.2.4 Call32 and Call64 . . . 488
9.2.5 Cancel . . . 489
9.3 Guest software interface . . . 489
9.3.1 The guest driver interface . . . 489
9.3.2 Guest application interface . . . 491
9.4 HGCM Service Implementation . . . 492

10 RDP Web Control **493**
10.1 RDPWeb features . . . 493
10.2 RDPWeb reference . . . 493
10.2.1 RDPWeb functions . . . 493
10.2.2 Embedding RDPWeb in an HTML page . . . 494
10.3 RDPWeb change log . . . 494
10.3.1 Version 1.2.28 . . . 494
10.3.2 Version 1.1.26 . . . 494
10.3.3 Version 1.0.24 . . . 494

11 Drag and Drop **495**
11.1 Basic concepts . . . 495
11.2 Supported formats . . . 495

12 VirtualBox external authentication modules **497**

13 Using the Java API **499**
13.1 Introduction . . . 499
13.2 Requirements . . . 499
13.3 Example . . . 500

14 License information **501**

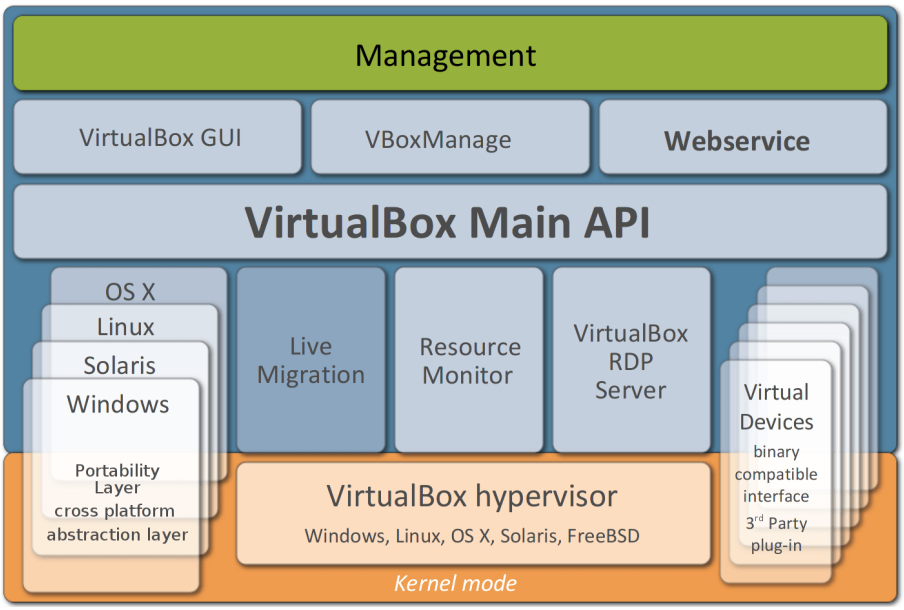
15 Main API change log **502**
15.1 Incompatible API changes with version 7.2 . . . 502
15.2 Incompatible API changes with version 7.1 . . . 502
15.3 Incompatible API changes with version 7.0 . . . 503
15.4 Incompatible API changes with version 6.1 . . . 503
15.5 Incompatible API changes with version 6.0 . . . 504
15.6 Incompatible API changes with version 5.x . . . 504
15.7 Incompatible API changes with version 5.1.28 . . . 505
15.8 Incompatible API changes with version 5.0 . . . 505
15.9 Incompatible API changes with version 4.3 . . . 508
15.10 Incompatible API changes with version 4.2 . . . 510
15.11 Incompatible API changes with version 4.1 . . . 512
15.12 Incompatible API changes with version 4.0 . . . 513
15.13 Incompatible API changes with version 3.2 . . . 516
15.14 Incompatible API changes with version 3.1 . . . 516
15.15 Incompatible API changes with version 3.0 . . . 518
15.16 Incompatible API changes with version 2.2 . . . 518
15.17 Incompatible API changes with version 2.1 . . . 519

1 Introduction

VirtualBox comes with comprehensive support for third-party developers. This Software Development Kit (SDK) contains all the documentation and interface files that are needed to write code that interacts with VirtualBox.

1.1 Modularity: the building blocks of VirtualBox

VirtualBox is cleanly separated into several layers, which can be visualized like in the picture below:



The orange area represents code that runs in kernel mode, the blue area represents userspace code.

At the bottom of the stack resides the hypervisor – the core of the virtualization engine, controlling execution of the virtual machines and making sure they do not conflict with each other or with whatever else the host computer is doing.

On top of the hypervisor, additional internal modules provide extra functionality. For example, the RDP server, which can deliver the graphical output of a VM remotely to an RDP client, is a separate module that is only loosely tacked onto the virtual graphics device.

What is primarily of interest for purposes of the SDK is the API layer block that sits on top of all the previously mentioned blocks. This API, which we call the **"Main API"**, exposes the entire feature set of the virtualization engine below. It is completely documented in this SDK Reference – see chapter 6, *Classes (interfaces)*, page 38 and chapter 7, *Enumerations (enums)*, page 425 – and available to anyone who wishes to control VirtualBox programmatically. We chose the name "Main API" to differentiate it from other programming interfaces of VirtualBox that may be publicly accessible.

With the Main API, you can create, configure, start, stop and delete virtual machines, retrieve performance statistics about running VMs, configure the VirtualBox installation in general, and

more. In fact, internally, the front-end programs `VirtualBox` and `VBoxManage` use nothing but this API as well – there are no hidden backdoors into the virtualization engine for our own front-ends. This ensures the entire Main API is both well-documented and well-tested. (The same applies to `VBoxHeadless`, which is not shown in the image.)

1.2 Two guises of the same "Main API": the web service or COM/XPCOM

There are several ways in which the Main API can be called by other code:

1. VirtualBox comes with a **web service** that maps nearly the entire Main API. The web service ships in a stand-alone executable (`vboxwebsrv`) that, when running, acts as an HTTP server, accepts SOAP connections and processes them.

   Since the entire web service API is publicly described in a web service description file (in WSDL format), you can write client programs that call the web service in any language with a toolkit that understands WSDL. These days, that includes most programming languages that are available: Java, C++, .NET, PHP, Python, Perl and probably many more.

   All of this is explained in detail in subsequent chapters of this book.

   There are two ways in which you can write client code that uses the web service:

   a) For Java as well as Python, the SDK contains easy-to-use classes that allow you to use the web service in an object-oriented, straightforward manner. We shall refer to this as the **"object-oriented web service (OOWS)"**.

      The OO bindings for Java are described in chapter 13, *Using the Java API*, page 499, those for Python in chapter 2.1.2, *The object-oriented web service for Python*, page 9.

   b) Alternatively, you can use the web service directly, without the object-oriented client layer. We shall refer to this as the **"raw web service"**.

      You will then have neither native object orientation nor full type safety, since web services are neither object-oriented nor stateful. However, in this way, you can write client code even in languages for which we do not ship object-oriented client code; all you need is a programming language with a toolkit that can parse WSDL and generate client wrapper code from it.

      We describe this further in chapter 2.2, *Using the raw web service with any language*, page 10, with samples for Java and Perl.

2. Internally, for portability and easier maintenance, the Main API is implemented using the **Component Object Model (COM),** an interprocess mechanism for software components originally introduced by Microsoft for Microsoft Windows. On a Windows host, VirtualBox will use Microsoft COM; on other hosts where COM is not present, VirtualBox ships with XPCOM, a free software implementation of COM originally created by the Mozilla project for their browsers.

   So if you are familiar with COM and the C++ programming language (or with any other programming language that can handle COM/XPCOM objects, such as Java, Visual Basic or C#), then you can use the COM/XPCOM API directly. VirtualBox comes with all the necessary files and documentation to build fully functional COM applications. For an introduction, please see chapter 2.3, *Using COM/XPCOM directly*, page 16 below.

   The VirtualBox front-ends (the graphical user interfaces as well as the command line), which are all written in C++, use COM/XPCOM to call the Main API. Technically, the web service is another front-end to this COM API, mapping almost all of it to SOAP clients.

If you are wondering which approach to choose, here are a few comparisons:

Web service	COM/XPCOM
Pro: Easy to use with Java and Python with the object-oriented web service; extensive support even with other languages (C++, .NET, PHP, Perl and others)	**Con:** Usable from languages where COM bridge available (most languages on Windows platform, Python and C++ on other hosts)
Pro: Client can be on remote machine	**Con:** Client must be on the same host where virtual machine is executed
Con: Significant overhead due to XML marshalling over the wire for each method call	**Pro:** Relatively low invocation overhead

In the following chapters, we will describe the different ways in which to program VirtualBox, starting with the method that is easiest to use and then increasing in complexity as we go along.

1.3 About web services in general

Web services are a particular type of programming interface. Whereas, with "normal" programming, a program calls an application programming interface (API) defined by another program or the operating system and both sides of the interface have to agree on the calling convention and, in most cases, use the same programming language, web services use Internet standards such as HTTP and XML to communicate.[1]

In order to successfully use a web service, a number of things are required – primarily, a web service accepting connections; service descriptions; and a client that connects to that web service. Connections to the VirtualBox web service are governed by the SOAP standard, which describes how messages are to be exchanged between a service and its clients; the service descriptions are governed by WSDL.

In the case of VirtualBox, this translates into the following three components:

1. The VirtualBox web service (the "server"): this is the `vboxwebsrv` executable shipped with VirtualBox. Once you start this executable (which acts as an HTTP server on a specific TCP/IP port), clients can connect to the web service and thus control a VirtualBox installation.

2. VirtualBox also comes with WSDL files that describe the services provided by the web service. You can find these files in the `sdk/bindings/webservice/` directory. These files are understood by the web service toolkits that are shipped with most programming languages and enable you to easily access a web service even if you don't use our object-oriented client layers. VirtualBox is shipped with pre-generated web service glue code for several languages (Python, Perl, Java).

3. A client that connects to the web service in order to control the VirtualBox installation.

   Unless you use some of the samples shipped with VirtualBox, this needs to be written by you.

[1] In some ways, web services promise to deliver the same thing as CORBA and DCOM did years ago. However, while these previous technologies relied on specific binary protocols and thus proved to be difficult to use between diverging platforms, web services circumvent these incompatibilities by using text-only standards like HTTP and XML. On the downside (and, one could say, typical of things related to XML), a lot of standards are involved before a web service can be implemented. Many of the standards invented around XML are used one way or another. As a result, web services are slow and verbose, and the details can be incredibly messy. The relevant standards here are called SOAP and WSDL, where SOAP describes the format of the messages that are exchanged (an XML document wrapped in an HTTP header), and WSDL is an XML format that describes a complete API provided by a web service. WSDL in turn uses XML Schema to describe types, which is not exactly terse either. However, as you will see from the samples provided in this chapter, the VirtualBox web service shields you from these details and is easy to use.

1.4 Running the web service

The web service ships in a stand-alone executable, `vboxwebsrv`, that, when running, acts as an HTTP server, accepts SOAP connections, remotely or from the same machine, and processes them.

> **Note:** The web service executable is not delivered with the VirtualBox SDK, but instead ships with the standard VirtualBox binary package for your specific platform. The SDK contains only platform-independent text files and documentation so thus the vboxwebsrv binary is shipped with the platform-specific packages. Therefore the information on how to run vboxwebsrv as a service is included in the VirtualBox documentation and not the SDK.

The `vboxwebsrv` program, which implements the web service, is a text-mode (console) program which, after being started, simply runs until it is interrupted with Ctrl-C or a kill command.

Once the web service is started, it acts as a front-end to the VirtualBox installation of the user account that it is running under. In other words, if the web service is run under the user account of `user1`, it will see and manipulate the virtual machines and other data represented by the VirtualBox data of that user (for example, on a Linux machine, under `/home/user1/.config/VirtualBox`; see the VirtualBox User Guide for details on where this data is stored).

1.4.1 Command line options of vboxwebsrv

The web service supports the following command line options:

- `--help` (or `-h`): print a brief summary of command line options.
- `--background` (or `-b`): run the web service as a background daemon. This option is not supported on Windows hosts.
- `--host` (or `-H`): This specifies the host to bind to and defaults to “localhost”.
- `--port` (or `-p`): This specifies which port to bind to on the host and defaults to 18083.
- `--ssl` (or `-s`): This enables SSL support.
- `--keyfile` (or `-K`): This specifies the file name containing the server private key and the certificate. This is a mandatory parameter if SSL is enabled.
- `--passwordfile` (or `-a`): This specifies the file name containing the password for the server private key. If unspecified or an empty string is specified this is interpreted as an empty password (i.e. the private key is not protected by a password). If the file name `-` is specified then then the password is read from the standard input stream, otherwise from the specified file. The user is responsible for appropriate access rights to protect the confidential password.
- `--cacert` (or `-c`): This specifies the file name containing the CA certificate appropriate for the server certificate.
- `--capath` (or `-C`): This specifies the directory containing several CA certificates appropriate for the server certificate.
- `--dhfile` (or `-D`): This specifies the file name containing the DH key. Alternatively it can contain the number of bits of the DH key to generate. If left empty, RSA is used.

- `--randfile` (or `-r`): This specifies the file name containing the seed for the random number generator. If left empty, an operating system specific source of the seed.
- `--timeout` (or `-t`): This specifies the session timeout, in seconds, and defaults to 300 (five minutes). A web service client that has logged on but makes no calls to the web service will automatically be disconnected after the number of seconds specified here, as if it had called the `IWebSessionManager::logoff()` method provided by the web service itself.

  It is normally vital that each web service client call this method, as the web service can accumulate large amounts of memory when running, especially if a web service client does not properly release managed object references. As a result, this timeout value should not be set too high, especially on machines with a high load on the web service, or the web service may eventually deny service.
- `--check-interval` (or `-i`): This specifies the interval in which the web service checks for timed-out clients, in seconds, and defaults to 5. This normally does not need to be changed.
- `--threads` (or `-T`): This specifies the maximum number or worker threads, and defaults to 100. This normally does not need to be changed.
- `--keepalive` (or `-k`): This specifies the maximum number of requests which can be sent in one web service connection, and defaults to 100. This normally does not need to be changed.
- `--authentication` (or `-A`): This specifies the desired web service authentication method. If the parameter is not specified or the empty string is specified it does not change the authentication method, otherwise it is set to the specified value. Using this parameter is a good measure against accidental misconfiguration, as the web service ensures periodically that it isn't changed.
- `--verbose` (or `-v`): Normally, the web service outputs only brief messages to the console each time a request is served. With this option, the web service prints much more detailed data about every request and the COM methods that those requests are mapped to internally, which can be useful for debugging client programs.
- `--pidfile` (or `-P`): Name of the PID file which is created when the daemon was started.
- `--logfile` (or `-F`) `<file>`: If this is specified, the web service not only prints its output to the console, but also writes it to the specified file. The file is created if it does not exist; if it does exist, new output is appended to it. This is useful if you run the web service unattended and need to debug problems after they have occurred.
- `--logrotate` (or `-R`): Number of old log files to keep, defaults to 10. Log rotation is disabled if set to 0.
- `--logsize` (or `-S`): Maximum size of log file in bytes, defaults to 100MB. Log rotation is triggered if the file grows beyond this limit.
- `--loginterval` (or `-I`): Maximum time interval to be put in a log file before rotation is triggered, in seconds, and defaults to one day.

1.4.2 Authenticating at web service logon

As opposed to the COM/XPCOM variant of the Main API, a client that wants to use the web service must first log on by calling the IWebsessionManager::logon() API that is specific to the web service. Logon is necessary for the web service to be stateful; internally, it maintains a session for each client that connects to it.

The `IWebsessionManager::logon()` API takes a user name and a password as arguments, which the web service then passes to a customizable authentication plugin that performs the actual authentication.

For testing purposes, it is recommended that you first disable authentication with the command:

```
VBoxManage setproperty websrvauthlibrary null
```

> **Warning:** This will cause all logons to succeed, regardless of user name or password. This should of course not be used in a production environment.

Generally, the mechanism by which clients are authenticated is configurable by way of the `VBoxManage` command:

```
VBoxManage setproperty websrvauthlibrary default|null|<library>
```

This way you can specify any shared object/dynamic link module that conforms with the specifications for VirtualBox external authentication modules as laid out in section **VRDE authentication** of the VirtualBox User Guide; the web service uses the same kind of modules as the VirtualBox VRDE server. For technical details on VirtualBox external authentication modules see chapter 12, *VirtualBox external authentication modules*, page 497

By default, after installation, the web service uses the VBoxAuth module that ships with VirtualBox. This module uses PAM on FreeBSD, Linux, and Solaris hosts to authenticate users. Any valid username/password combination is accepted, it does not have to be the username and password of the user running the web service daemon. If `vboxwebsrv` doesn't run as root PAM authentication can fail, because the `/etc/shadow` file, which is used by PAM, is only readable by root. On most Linux distributions PAM uses a suid root helper internally, so make sure you test this before deploying it. One can override authentication failures due to lack of read privileges of the shadow password file by setting the environment variable `VBOX_PAM_ALLOW_INACTIVE`. Please use this variable carefully and only if you fully understand what you're doing.

2 Environment-specific notes

The Main API described in chapter 6, *Classes (interfaces)*, page 38 and chapter 7, *Enumerations (enums)*, page 425 is mostly identical in all the supported programming environments which have been briefly mentioned in the introduction of this book. As a result, the Main API's general concepts described in chapter 3, *Basic VirtualBox concepts; some examples*, page 28 are the same whether you use the object-oriented web service (OOWS) for JAX-WS or a raw web service connection via, say, Perl, or whether you use C++ COM bindings.

Some things are different depending on your environment, however. These differences are explained in this chapter.

2.1 Using the object-oriented web service (OOWS)

As explained in chapter 1.2, *Two guises of the same "Main API": the web service or COM/XPCOM*, page 2, VirtualBox ships with client-side libraries for Java, Python and PHP that allow you to use the VirtualBox web service in an intuitive, object-oriented way. These libraries shield you from the client-side complications of managed object references and other implementation details that come with the VirtualBox web service. (If you are interested in these complications, have a look at chapter 2.2, *Using the raw web service with any language*, page 10).

We recommend that you start your experiments with the VirtualBox web service by using our object-oriented client libraries for JAX-WS, a web service toolkit for Java, which enables you to write code to interact with VirtualBox in the simplest manner possible.

As "interfaces", "attributes" and "methods" are COM concepts, please read the documentation in chapter 6, *Classes (interfaces)*, page 38 and chapter 7, *Enumerations (enums)*, page 425 with the following notes in mind.

The OOWS bindings attempt to map the Main API as closely as possible to the Java, Python and PHP languages. In other words, objects are objects, interfaces become classes, and you can call methods on objects as you would on local objects.

The main difference remains with attributes: to read an attribute, call a "getXXX" method, with "XXX" being the attribute name with a capitalized first letter. So when the Main API Reference says that `IMachine` has a "name" attribute (see IMachine::name), call `getName()` on an IMachine object to obtain a machine's name. Unless the attribute is marked as read-only in the documentation, there will also be a corresponding "set" method.

2.1.1 The object-oriented web service for JAX-WS

JAX-WS is a powerful toolkit by Sun Microsystems to build both server and client code with Java. It is part of Java 6 (JDK 1.6), but can also be obtained separately for Java 5 (JDK 1.5). The VirtualBox SDK comes with precompiled OOWS bindings working with both Java 5 and 6.

The following sections explain how to get the JAX-WS sample code running and explain a few common practices when using the JAX-WS object-oriented web service.

2.1.1.1 Preparations

Since JAX-WS is already integrated into Java 6, no additional preparations are needed for Java 6.

If you are using Java 5 (JDK 1.5.x), you will first need to download and install an external JAX-WS implementation, as Java 5 does not support JAX-WS out of the box; for example, you can

download one from here: https://jax-ws.dev.java.net/2.1.4/JAXWS2.1.4-20080502.jar. Then perform the installation (`java -jar JAXWS2.1.4-20080502.jar`).

2.1.1.2 Getting started: running the sample code

To run the OOWS for JAX-WS samples that we ship with the SDK, perform the following steps:

1. Open a terminal and change to the directory where the JAX-WS samples reside.[1] Examine the header of `Makefile` to see if the supplied variables (Java compiler, Java executable) and a few other details match your system settings.

2. To start the VirtualBox web service, open a second terminal and change to the directory where the VirtualBox executables are located. Then type:

   ```
   ./vboxwebsrv -v
   ```

   The web service now waits for connections and will run until you press Ctrl+C in this second terminal. The -v argument causes it to log all connections to the terminal. (See chapter 1.4, *Running the web service*, page 4 for details on how to run the web service.)

3. Back in the first terminal and still in the samples directory, to start a simple client example just type:

   ```
   make run16
   ```

   if you're on a Java 6 system; on a Java 5 system, run `make run15` instead.

   This should work on all Unix-like systems such as Linux and Solaris. For Windows systems, use commands similar to what is used in the Makefile.

   This will compile the `clienttest.java` code on the first call and then execute the resulting `clienttest` class to show the locally installed VMs (see below).

The `clienttest` sample imitates a few typical command line tasks that `VBoxManage`, VirtualBox's regular command-line front-end, would provide (see the VirtualBox User Guide for details). In particular, you can run:

- `java clienttest show vms`: show the virtual machines that are registered locally.
- `java clienttest list hostinfo`: show various information about the host this VirtualBox installation runs on.
- `java clienttest startvm <vmname|uuid>`: start the given virtual machine.

The `clienttest.java` sample code illustrates common basic practices how to use the VirtualBox OOWS for JAX-WS, which we will explain in more detail in the following chapters.

2.1.1.3 Logging on to the web service

Before a web service client can do anything useful, two objects need to be created, as can be seen in the `clienttest` constructor:

[1] In `sdk/bindings/glue/java/`.

1. An instance of IWebsessionManager, which is an interface provided by the web service to manage "web sessions" – that is, stateful connections to the web service with persistent objects upon which methods can be invoked.

   In the OOWS for JAX-WS, the IWebsessionManager class must be constructed explicitly, and a URL must be provided in the constructor that specifies where the web service (the server) awaits connections. The code in `clienttest.java` connects to "http://localhost:18083/", which is the default.

   The port number, by default 18083, must match the port number given to the `vboxwebsrv` command line; see chapter 1.4.1, *Command line options of vboxwebsrv*, page 4.

2. After that, the code calls IWebsessionManager::logon(), which is the first call that actually communicates with the server. This authenticates the client with the web service and returns an instance of IVirtualBox, the most fundamental interface of the VirtualBox web service, from which all other functionality can be derived.

   If logon doesn't work, please take another look at chapter 1.4.2, *Authenticating at web service logon*, page 5.

2.1.1.4 Object management

The current OOWS for JAX-WS has certain memory management related limitations. When you no longer need an object, call its IManagedObjectRef::release() method explicitly, which frees appropriate managed reference, as is required by the raw web service; see chapter 2.2.3.3, *Managed object references*, page 14 for details. This limitation may be reconsidered in a future version of the VirtualBox SDK.

2.1.2 The object-oriented web service for Python

VirtualBox comes with two flavors of a Python API: one for web service, discussed here, and one for the COM/XPCOM API discussed in chapter 2.3.1, *Python COM API*, page 17. The client code is mostly similar, except for the initialization part, so it is up to the application developer to choose the appropriate technology. Moreover, a common Python glue layer exists, abstracting out concrete platform access details, see chapter 2.3.2, *Common Python bindings layer*, page 17.

The minimum supported Python version is 2.6.

As indicated in chapter 1.2, *Two guises of the same "Main API": the web service or COM/XPCOM*, page 2, the COM/XPCOM API gives better performance without the SOAP overhead, and does not require a web server to be running. On the other hand, the COM/XPCOM Python API requires a suitable Python bridge for your Python installation (VirtualBox ships the most important ones for each platform[2]). On Windows, you can use the Main API from Python if the Win32 extensions package for Python[3] is installed. Versions of Python Win32 extensions earlier than 2.16 are known to have bugs, leading to issues with VirtualBox Python bindings, so please make sure to use latest available Python and Win32 extensions.

The VirtualBox OOWS for Python relies on the Python ZSI SOAP implementation (see http://pywebsvcs.sourceforge.net/zsi.html), which you will need to install locally before trying the examples. Most Linux distributions come with a package for ZSI, such as `python-zsi` in Ubuntu.

To get started, open a terminal and change to the `sdk/bindings/glue/python/sample` directory, which contains an example of a simple interactive shell able to control a VirtualBox instance. The shell is written using the API layer, thereby hiding different implementation details, so it is actually an example of code shared among XPCOM, MSCOM and web services. If you are interested in how to interact with the web services layer directly, have a look at

[2] On Mac OS X only the Python versions bundled with the OS are officially supported.
[3] See http://sourceforge.net/project/showfiles.php?group_id=78018.

`install/vboxapi/__init__.py` which contains the glue layer for all target platforms (i.e. XPCOM, MSCOM and web services).

To start the shell, run the following commands:

```
/opt/VirtualBox/vboxwebsrv -t 0 # start web service with object autocollection disabled
export VBOX_PROGRAM_PATH=/opt/VirtualBox        # your VirtualBox installation directory
export VBOX_SDK_PATH=/home/youruser/vbox-sdk    # where you've extracted the SDK
./vboxshell.py -w
```

See chapter 4, *The VirtualBox shell*, page 31 for more details on the shell's functionality. For you, as a VirtualBox application developer, the vboxshell sample could be interesting as an example of how to write code targeting both local and remote cases (COM/XPCOM and SOAP). The common part of the shell is the same – the only difference is how it interacts with the invocation layer. You can use the `connect` shell command to connect to remote VirtualBox servers; in this case you can skip starting the local web server.

2.1.3 The object-oriented web service for PHP

VirtualBox also comes with object-oriented web service (OOWS) wrappers for PHP5. These wrappers rely on the PHP SOAP Extension[4], which can be installed by configuring PHP with `--enable-soap`.

2.2 Using the raw web service with any language

The following examples show you how to use the raw web service, without the object-oriented client-side code that was described in the previous chapter.

Due to the limitations of SOAP and WSDL outlined in chapter 2.2.3.1, *Fundamental conventions*, page 12, keep the following notes in mind when reading the documentation in chapter 6, *Classes (interfaces)*, page 38 and chapter 7, *Enumerations (enums)*, page 425:

1. Any COM method call becomes a **plain function call** in the raw web service, with the object as an additional first parameter (before the "real" parameters listed in the documentation). So when the documentation says that the `IVirtualBox` interface supports the `createMachine()` method (see IVirtualBox::createMachine()), the web service operation is `IVirtualBox_createMachine(...)`, and a managed object reference to an `IVirtualBox` object must be passed as the first argument.

2. For **attributes** in interfaces, there will be at least one "get" function; there will also be a "set" function, unless the attribute is "readonly". The attribute name will be appended to the "get" or "set" prefix, with a capitalized first letter. So, the "version" readonly attribute of the `IVirtualBox` interface can be retrieved by calling `IVirtualBox_getVersion(vbox)`, with `vbox` being the VirtualBox object.

3. Whenever the API documentation says that a method (or an attribute getter) returns an **object**, it will returned a managed object reference in the web service instead. As said above, managed object references should be released if the web service client does not log off again immediately!

2.2.1 Raw web service example for Java with Axis

Axis is an older web service toolkit created by the Apache foundation. If your distribution does not have it installed, you can get a binary from http://axis.apache.org. The following examples assume that you have Axis 1.4 installed.

[4] See https://www.php.net/soap.

The VirtualBox SDK ships with an example for Axis that, again, is called `clienttest.java` and that imitates a few `VBoxManage` commands and sends them to the VirtualBox web service.

To try out the raw web service with Axis complete the following steps:

1. Create a working directory somewhere. Under your VirtualBox installation directory, find the `sdk/webservice/samples/java/axis/` directory and copy the file `clienttest.java` to your working directory.

2. Open a terminal in your working directory. Execute the following command:

   ```
   java org.apache.axis.wsdl.WSDL2Java /path/to/vboxwebService.wsdl
   ```

   The `vboxwebService.wsdl` file should be located in the `sdk/webservice/` directory.

   If this fails, your Apache Axis may not be located on your system classpath, and you may have to adjust the CLASSPATH environment variable. Something like this:

   ```
   export CLASSPATH="/path-to-axis-1_4/lib/*":$CLASSPATH
   ```

   Use the directory where the Axis JAR files are located. Mind the quotes so that your shell passes the "*" character to the java executable without expanding. Alternatively, add a corresponding `-classpath` argument to the "java" call above.

   If the command executes successfully, you should see an "org" directory with subdirectories containing Java source files in your working directory. These classes represent the interfaces that the VirtualBox web service offers, as described by the WSDL file.

   This is the bit that makes using web services so attractive to client developers: if a language's toolkit understands WSDL, it can generate large amounts of support code automatically. Clients can then easily use this support code and can be done with just a few lines of code.

3. Next, compile the `clienttest.java` source:

   ```
   javac clienttest.java
   ```

   This should yield a "clienttest.class" file.

4. To start the VirtualBox web service, open a second terminal and change to the directory where the VirtualBox executables are located. Then type:

   ```
   ./vboxwebsrv -v
   ```

   The web service now waits for connections and will run until you press Ctrl+C in this second terminal. The -v argument causes it to log all connections to the terminal. (See chapter 1.4, *Running the web service*, page 4 for details on how to run the web service.)

5. Back in the original terminal where you compiled the Java source, run the resulting binary, which will then connect to the web service:

   ```
   java clienttest
   ```

   The client sample will connect to the web service (on localhost, but the code could be changed to connect remotely if the web service was running on a different machine) and make a number of method calls. It will output the version number of your VirtualBox installation and a list of all virtual machines that are currently registered (with a bit of seemingly random data, which will be explained later).

2.2.2 Raw web service example for Perl

We also ship a small sample for Perl. It uses the SOAP::Lite perl module to communicate with the VirtualBox web service.

The `sdk/bindings/webservice/perl/lib/` directory contains a pre-generated Perl module that allows for communicating with the web service from Perl. You can generate such a module yourself using the "stubmaker" tool that comes with SOAP::Lite, but since that tool is slow as well as sometimes unreliable, we ship a working module with the SDK for your convenience.

Perform the following steps:

1. If SOAP::Lite is not yet installed on your system, you will need to install the package first. On Debian-based systems, the package is called `libsoap-lite-perl`; on Gentoo, it's `dev-perl/SOAP-Lite`.

2. Open a terminal in the `sdk/bindings/webservice/perl/samples/` directory.

3. To start the VirtualBox web service, open a second terminal and change to the directory where the VirtualBox executables are located. Then type:

   ```
   ./vboxwebsrv -v
   ```

   The web service now waits for connections and will run until you press Ctrl+C in this second terminal. The -v argument causes it to log all connections to the terminal. (See chapter 1.4, *Running the web service*, page 4 for details on how to run the web service.)

4. In the first terminal with the Perl sample, run the clienttest.pl script:

   ```
   perl -I ../lib clienttest.pl
   ```

2.2.3 Programming considerations for the raw web service

If you use the raw web service, you need to keep a number of things in mind, or you will sooner or later run into issues that are not immediately obvious. By contrast, the object-oriented client-side libraries described in chapter 2.1, *Using the object-oriented web service (OOWS)*, page 7 take care of these things automatically and thus greatly simplify using the web service.

2.2.3.1 Fundamental conventions

If you are familiar with other web services, you may find that the VirtualBox web service behaves a bit differently to accommodate for the fact that the VirtualBox web service more or less maps the VirtualBox Main COM API. The primary challenges in mapping the VirtualBox Main COM API to the web service are as follows:

- Web services, as expressed by WSDL, are not object-oriented. Even worse, they are normally stateless (or, in web services terminology, "loosely coupled"). Web service operations are entirely procedural, and one cannot normally make assumptions about the state of a web service between function calls.

  In particular, this normally means that you cannot work on objects in one method call that were created by another call.

- By contrast, the VirtualBox Main API, being expressed in COM, is object-oriented and works entirely on objects, which are grouped into public interfaces, which in turn have attributes and methods associated with them.

For the VirtualBox web service, this results in three fundamental conventions:

1. All **function names** in the VirtualBox web service consist of an interface name and a method name, joined together by an underscore. This is because there are only functions ("operations") in WSDL, but no classes, interfaces, or methods.

2. All calls to the VirtualBox web service (except for logon, see below) take a **managed object reference** as the first argument, representing the object upon which the underlying method is invoked. (Managed object references are explained in detail below; see chapter 2.2.3.3, *Managed object references*, page 14.)

   So, when one would normally code, in the pseudo-code of an object-oriented language, to invoke a method upon an object:

   ```
   IMachine machine;
   result = machine.getName();
   ```

   In the VirtualBox web service, this looks something like this (again, pseudo-code):

   ```
   IMachineRef machine;
   result = IMachine_getName(machine);
   ```

3. To make the web service stateful, and objects persistent between method calls, the VirtualBox web service introduces a **session manager** (by way of the IWebsessionManager interface), which manages object references. Any client wishing to interact with the web service must first log on to the session manager and in turn receives a managed object reference to an object that supports the IVirtualBox interface (the basic interface in the Main API).

In other words, as opposed to other web services, **the VirtualBox web service is both object-oriented and stateful.**

2.2.3.2 Example: A typical web service client session

A typical short web service session to retrieve the version number of the VirtualBox web service (to be precise, the underlying Main API version number) looks like this:

1. A client logs on to the web service by calling IWebsessionManager::logon() with a valid user name and password. See chapter 1.4.2, *Authenticating at web service logon*, page 5 for details about how authentication works.

2. On the server side, `vboxwebsrv` creates a session, which persists until the client calls IWebsessionManager::logoff() or the session times out after a configurable period of inactivity (see chapter 1.4.1, *Command line options of vboxwebsrv*, page 4).

   For the new session, the web service creates an instance of IVirtualBox. This interface is the most central one in the Main API and allows access to all other interfaces, either through attributes or method calls. For example, IVirtualBox contains a list of all virtual machines that are currently registered (as they would be listed on the left side of the VirtualBox main program).

   The web service then creates a managed object reference for this instance of IVirtualBox and returns it to the calling client, which receives it as the return value of the logon call. Something like this:

   ```
   string oVirtualBox;
   oVirtualBox = webservice.IWebsessionManager_logon("user", "pass");
   ```

(The managed object reference "oVirtualBox" is just a string consisting of digits and dashes. However, it is a string with a meaning and will be checked by the web service. For details, see below. As hinted above, IWebsessionManager::logon() is the *only* operation provided by the web service which does not take a managed object reference as the first argument!)

3. The VirtualBox Main API documentation explains that the `IVirtualBox` interface has a version attribute, which is a string. For each attribute, there is a "get" and a "set" method in COM, which maps to the corresponding operations in the web service. So, to retrieve the "version" attribute of this `IVirtualBox` object, the web service client does this:

```
string version;
version = webservice.IVirtualBox_getVersion(oVirtualBox);

print version;
```

   And it will print "7.2.22".

4. The web service client calls IWebsessionManager::logoff() with the VirtualBox managed object reference. This will clean up all allocated resources.

2.2.3.3 Managed object references

To a web service client, a managed object reference looks like a string: two 64-bit hex numbers separated by a dash. This string, however, represents a COM object that "lives" in the web service process. The two 64-bit numbers encoded in the managed object reference represent a session ID (which is the same for all objects in the same web service session, i.e. for all objects after one logon) and a unique object ID within that session.

Managed object references are created in two situations:

1. When a client logs on, by calling IWebsessionManager::logon().

   Upon logon, the websession manager creates one instance of IVirtualBox, which can be used for directly performing calls to its methods, or used as a parameter for calling some methods of IWebsessionManager. Creating Main API session objects is performed using IWebsessionManager::getSessionObject().

   (Technically, there is always only one IVirtualBox object, which is shared between all web-sessions and clients, as it is a COM singleton. However, each session receives its own managed object reference to it.)

2. Whenever a web service clients invokes an operation whose COM implementation creates COM objects.

   For example, IVirtualBox::createMachine() creates a new instance of IMachine; the COM object returned by the COM method call is then wrapped into a managed object reference by the web server, and this reference is returned to the web service client.

Internally, in the web service process, each managed object reference is simply a small data structure, containing a COM pointer to the "real" COM object, the web session ID and the object ID. This structure is allocated on creation and stored efficiently in hashes, so that the web service can look up the COM object quickly whenever a web service client wishes to make a method call. The random session ID also ensures that one web service client cannot intercept the objects of another.

Managed object references are not destroyed automatically and must be released by explicitly calling IManagedObjectRef::release(). This is important, as otherwise hundreds or thousands of managed object references (and corresponding COM objects, which can consume much more memory!) can pile up in the web service process and eventually cause it to deny service.

To reiterate: The underlying COM object, which the reference points to, is only freed if the managed object reference is released. It is therefore vital that web service clients properly clean up after the managed object references that are returned to them.

When a web service client calls IWebsessionManager::logoff(), all managed object references created during the session are automatically freed. For short-lived sessions that do not create a lot of objects, logging off may therefore be sufficient, although it is certainly not "best practice".

2.2.3.4 Some more detail about web service operation

SOAP messages Whenever a client makes a call to a web service, this involves a complicated procedure internally. These calls are remote procedure calls. Each such procedure call typically consists of two "message" being passed, where each message is a plain-text HTTP request with a standard HTTP header and a special XML document following. This XML document encodes the name of the procedure to call and the argument names and values passed to it.

To give you an idea of what such a message looks like, assuming that a web service provides a procedure called "SayHello", which takes a string "name" as an argument and returns "Hello" with a space and that name appended, the request message could look like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<SOAP-ENV:Envelope
 xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
 xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xmlns:xsd="http://www.w3.org/2001/XMLSchema"
 xmlns:test="http://test/">
<SOAP-ENV:Body>
   <test:SayHello>
    <name>Peter</name>
   </test:SayHello>
 </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

A similar message – the "response" message – would be sent back from the web service to the client, containing the return value "Hello Peter".

Most programming languages provide automatic support to generate such messages whenever code in that programming language makes such a request. In other words, these programming languages allow for writing something like this (in pseudo-C++ code):

```
webServiceClass service("localhost", 18083); // server and port
string result = service.SayHello("Peter");  // invoke remote procedure
```

and would, for these two pseudo-lines, automatically perform these steps:

1. Prepare a connection to the web service running on port 18083 of "localhost".
2. Generate a SOAP message similar to the above example for the `SayHello()` function of the web service by encoding all arguments of the remote procedure call (which could involve all kinds of type conversions and complex marshalling for arrays and structures).
3. Connect to the web service via HTTP and then send that message.
4. Wait for the web service to send a response message.
5. Decode that response message and put the return value of the remote procedure into the "result" variable.

Service descriptions in WSDL In the above explanations about SOAP, it wasn't explained how the programming language learns about how to translate function calls in its own syntax into proper SOAP messages. In other words, the programming language needs to know what operations the web service supports and what types of arguments are required for the operation's data in order to be able to properly serialize and deserialize the data to and from the web service. For example, if a web service operation expects a number in "double" floating point format for a particular parameter, the programming language cannot send it a string instead.

For this, the Web Service Definition Language (WSDL) was invented, another XML substandard that describes exactly what operations the web service supports and, for each operation, which parameters and types are needed with each request and response message. WSDL descriptions can be incredibly verbose, and one of the few good things that can be said about this standard is that it is indeed supported by most programming languages.

So, if it is said that a programming language "supports" web services, this typically means that a programming language has support for parsing WSDL files and somehow integrating the remote procedure calls into the native language syntax – for example, as shown in the Java sample in chapter 2.2.1, *Raw web service example for Java with Axis*, page 10.

For details about how programming languages support web services, please refer to the documentation that comes with the individual language. Here are a few pointers:

1. For **C++,** among many others, the gSOAP toolkit is a good option. Parts of gSOAP are also used in VirtualBox to implement the VirtualBox web service.

2. For **Java,** there are several implementations already described in this document (see chapter 2.1.1, *The object-oriented web service for JAX-WS*, page 7 and chapter 2.2.1, *Raw web service example for Java with Axis*, page 10).

3. **Perl** supports WSDL via the SOAP::Lite package. This in turn comes with a tool called `stubmaker.pl` that allows you to turn any WSDL file into a Perl package that you can import. (You can also import any WSDL file "live" by having it parsed every time the script runs, but that can take a while.) You can then code (again, assuming the above example):

   ```
   my $result = servicename->sayHello("Peter");
   ```

   A sample that uses SOAP::Lite was described in chapter 2.2.2, *Raw web service example for Perl*, page 12.

2.3 Using COM/XPCOM directly

If you do not require *remote* procedure calls such as those offered by the VirtualBox web service, and if you know Python or C++ as well as COM, you might find it preferable to program VirtualBox's Main API directly via COM.

COM stands for "Component Object Model" and is a standard originally introduced by Microsoft in the 1990s for Microsoft Windows. It allows for organizing software in an object-oriented way and across processes; code in one process may access objects that live in another process.

COM has several advantages: it is language-neutral, meaning that even though all of VirtualBox is internally written in C++, programs written in other languages can communicate with it. COM also cleanly separates interface from implementation, so that external programs do not need to know anything about the messy and complicated details of VirtualBox internals.

On a Windows host, all parts of VirtualBox will use the COM functionality that is native to Windows. On other hosts (including Linux), VirtualBox comes with a built-in implementation of XPCOM, as originally created by the Mozilla project, which we have enhanced to support interprocess communication on a level comparable to Microsoft COM. Internally, VirtualBox has an abstraction layer that allows the same VirtualBox code to work both with native COM as well as our XPCOM implementation.

2.3.1 Python COM API

On Windows, Python scripts can use COM and VirtualBox interfaces to control almost all aspects of virtual machine execution. For example, you can use the following commands to instantiate the VirtualBox object and start a VM:

```
vbox = win32com.client.Dispatch("VirtualBox.VirtualBox")
session = win32com.client.Dispatch("VirtualBox.Session")
mach = vbox.findMachine("uuid or name  of machine to start")
progress = mach.launchVMProcess(session, "gui", "")
progress.waitForCompletion(-1)
```

Also, see `sdk/bindings/glue/python/samples/vboxshell.py` for more advanced usage scenarious. However, unless you have specific requirements, we strongly recommend that you use the generic glue layer described in the next section to access MS COM objects.

2.3.2 Common Python bindings layer

As different wrappers ultimately provide access to the same underlying API, and to simplify porting and development of Python applications using the VirtualBox Main API, we developed a common glue layer that abstracts out most platform-specific details from the application and allows the developer to focus on application logic. The VirtualBox installer automatically sets up this glue layer for the system default Python installation.

See chapter 2.3.2.1, *Manual or subsequent setup*, page 18 for details on how to set up the glue layer if you want to use a different Python installation, or if the VirtualBox installer failed to detect and set it up accordingly.

The minimum supported Python version is 2.6.

In this layer, the class `VirtualBoxManager` hides most platform-specific details. It can be used to access both the local (COM) and the web service based API. The following code can be used by an application to use the glue layer.

```
# This code assumes vboxapi.py from VirtualBox distribution
# being in PYTHONPATH, or installed system-wide
from vboxapi import VirtualBoxManager

# This code initializes VirtualBox manager with default style
# and parameters
virtualBoxManager = VirtualBoxManager(None, None)

# Alternatively, one can be more verbose, and initialize
# glue with web service backend, and provide authentication
# information
virtualBoxManager = VirtualBoxManager("WEBSERVICE",
                                      {'url':'http://myhost.com::18083/',
                                      'user':'me',
                                      'password':'secret'})
```

We supply the `VirtualBoxManager` constructor with 2 arguments: style and parameters. Style defines which bindings style to use (could be “MSCOM”, “XPCOM” or “WEBSERVICE”), and if set to `None` defaults to usable platform bindings (MS COM on Windows, XPCOM on other platforms). The second argument defines parameters, passed to the platform-specific module, as we do in the second example, where we pass a username and password to be used to authenticate against the web service.

After obtaining the `VirtualBoxManager` instance, one can perform operations on the IVirtualBox class. For example, the following code will a start virtual machine by name or ID:

```
from vboxapi import VirtualBoxManager
mgr = VirtualBoxManager(None, None)
```

```
vbox = mgr.getVirtualBox()
name = "Linux"
mach = vbox.findMachine(name)
session = mgr.getSessionObject(vbox)
progress = mach.launchVMProcess(session, "gui", "")
progress.waitForCompletion(-1)
mgr.closeMachineSession(session)
```

The following code will print all registered machines and their log folders:

```
from vboxapi import VirtualBoxManager
mgr = VirtualBoxManager(None, None)
vbox = mgr.getVirtualBox()

for m in mgr.getArray(vbox, 'machines'):
    print "Machine '%s' logs in '%s'" %(m.name, m.logFolder)
```

The code above demonstrates cross-platform access to array properties (certain limitations prevent one from using `vbox.machines` to access a list of available virtual machines in the case of XPCOM), and a mechanism for uniform session creation and closure (`mgr.getSessionObject()`).

2.3.2.1 Manual or subsequent setup

If you want to use the glue layer with a different Python installation or the installer failed to set it up, then use these steps in a shell to install the necessary files:

```
# cd VBOX_INSTALL_PATH/sdk/installer
# python vboxapisetup.py install
```

Note: On Windows hosts, a Python distribution along with the win32api bindings package need to be installed as a prerequisite.

2.3.3 C++ COM API

C++ is the language that VirtualBox itself is written in, so C++ is the most direct way to use the Main API – but it is not necessarily the easiest, as using COM and XPCOM has its own set of complications.

VirtualBox ships with sample programs that demonstrate how to use the Main API to implement a number of tasks on your host platform. These samples can be found in the `sdk/bindings/xpcom/samples` directory for Linux, Mac OS X and Solaris and `sdk/bindings/mscom/samples` for Windows. The two samples are actually different, because the one for Windows uses native COM, whereas the other uses our XPCOM implementation, as described above.

Since COM and XPCOM are conceptually very similar but vary in the implementation details, we have created a "glue" layer that shields COM client code from these differences. All VirtualBox uses is this glue layer, so the same code written once works on both Windows hosts (with native COM) as well as on other hosts (with our XPCOM implementation). It is recommended to always use this glue code instead of using the COM and XPCOM APIs directly, as it is very easy to make your code completely independent from the platform it is running on.

In order to encapsulate platform differences between Microsoft COM and XPCOM, the following items should be kept in mind when using the glue layer:

1. **Attribute getters and setters.** COM has the notion of "attributes" in interfaces, which roughly compare to C++ member variables in classes. The difference is that for each attribute declared in an interface, COM automatically provides a "get" method to return the attribute's value. Unless the attribute has been marked as "readonly", a "set" attribute is also provided.

   To illustrate, the IVirtualBox interface has a "version" attribute, which is read-only and of the "wstring" type (the standard string type in COM). As a result, you can call the "get" method for this attribute to retrieve the version number of VirtualBox.

   Unfortunately, the implementation differs between COM and XPCOM. Microsoft COM names the "get" method like this: `get_Attribute()`, whereas XPCOM uses this syntax: `GetAttribute()` (and accordingly for "set" methods). To hide these differences, the VirtualBox glue code provides the `COMGETTER(attrib)` and `COMSETTER(attrib)` macros. So, `COMGETTER(version)()` (note, two pairs of brackets) expands to `get_Version()` on Windows and `GetVersion()` on other platforms.

2. **Unicode conversions.** While the rest of the modern world has pretty much settled on encoding strings in UTF-8, COM, unfortunately, uses UCS-16 encoding. This requires a lot of conversions, in particular between the VirtualBox Main API and the Qt GUI, which, like the rest of Qt, likes to use UTF-8.

   To facilitate these conversions, VirtualBox provides the `com::Bstr` and `com::Utf8Str` classes, which support all kinds of conversions back and forth.

3. **COM autopointers.** Possibly the greatest pain of using COM – reference counting – is alleviated by the `ComPtr<>` template provided by the `ptr.h` file in the glue layer.

2.3.4 Event queue processing

Both VirtualBox client programs and front-ends should periodically perform processing of the main event queue, and do that on the application's main thread. In the case of a typical GUI Windows/Mac OS application this happens automatically in the GUI's dispatch loop. However, for CLI only application, the appropriate actions have to be taken. For C++ applications, the VirtualBox SDK provided glue method

```
int EventQueue::processEventQueue(uint32_t cMsTimeout)
```

can be used for both blocking and non-blocking operations. For the Python bindings, a common layer provides the method

```
VirtualBoxManager.waitForEvents(ms)
```

with similar semantics.

Things get somewhat more complicated for situations where an application using VirtualBox cannot directly control the main event loop and the main event queue is separated from the event queue of the programming library (for example in case of Qt on Unix platforms). In such a case, the application developer is advised to use a platform/toolkit specific event injection mechanism to force event queue checks either based on periodic timer events delivered to the main thread, or by using custom platform messages to notify the main thread when events are available. See the Qt (VirtualBox) front-end as an example.

2.3.5 Visual Basic and Visual Basic Script (VBS) on Windows hosts

On Windows hosts, one can control some of the VirtualBox Main API functionality from VBS scripts, and pretty much everything from Visual Basic programs.[5]

VBS is a scripting language available in any recent Windows environment. As an example, the following VBS code will print the VirtualBox version:

```
set vb = CreateObject("VirtualBox.VirtualBox")
Wscript.Echo "VirtualBox version " & vb.version
```

See `sdk/bindings/mscom/vbs/sample/vboxinfo.vbs` for the complete sample.

Visual Basic is a popular high level language capable of accessing COM objects. The following VB code will iterate over all available virtual machines:

```
Dim vb As VirtualBox.IVirtualBox

vb = CreateObject("VirtualBox.VirtualBox")
machines = ""
For Each m In vb.Machines
   m = m & " " & m.Name
Next
```

See `sdk/bindings/mscom/vb/sample/vboxinfo.vb` for the complete sample.

2.3.6 C bindings to the VirtualBox API

The VirtualBox API was originally designed to be object oriented and leverages XPCOM or COM as the middleware to natively map the API to C++. This means that in order to use the VirtualBox API from C there needs to be some helper code to bridge the language differences and reduce the differences between platforms.

2.3.6.1 Cross-platform C bindings to the VirtualBox API

Starting with version 4.3, VirtualBox offers C bindings which allows using the same C client sources for all platforms, covering Windows, Linux, Mac OS X and Solaris. It is the preferred way to write API clients, even though the old style is still available.

2.3.6.2 Getting started

The following sections describe how to use the VirtualBox API in a C program. The necessary files are included in the SDK, in the directories `sdk/bindings/c/include` and `sdk/bindings/c/glue`.

As part of the SDK, a sample program `tstCAPIGlue.c` is provided in the directory `sdk/bindings/c/samples` which demonstrates using the C bindings to initialize the API, get handles for VirtualBox and Session objects, make calls to list and start virtual machines, monitor events, and uninitialize resources when done. The sample program is trying to illustrate all relevant concepts, so it is a great source of detail information. Among many other generally useful code sequences it contains a function which shows how to retrieve error details in C code if they are available from the API call.

The sample program `tstCAPIGlue` can be built using the provided `Makefile` and can be run without arguments.

[5] The difference results from the way VBS treats COM safearrays, which are used to keep lists in the Main API. VBS expects every array element to be a `VARIANT`, which is too strict a limitation for any high performance API. We may lift this restriction for interface APIs in a future version, or alternatively provide conversion APIs.

It uses the VBoxCAPIGlue library (source code is in directory `sdk/bindings/c/glue`, to be used in your API client code) to open the C binding layer during runtime, which is preferred to other means as it isolates the code which locates the necessary dynamic library, using a known working way which works on all platforms. If you encounter problems with this glue code in `VBoxCAPIGlue.c`, let the VirtualBox developers know, rather than inventing incompatible solutions.

The following sections document the important concepts needed to correctly use the C bindings, as it is vital for developing API client code which manages memory correctly, updates the reference counters correctly, and avoids crashes and memory leaks. Often API clients need to handle events, so the C API specifics are also described below.

2.3.6.3 VirtualBox C API initialization

Just like in C++, the API and the underlying middleware needs to be initialized before it can be used. The `VBoxCAPI_v7_2.h` header file provides the interface to the C bindings, but you can alternatively and more conveniently just include `VBoxCAPIGlue.h`, as this avoids the VirtualBox version dependent header file name and makes sure that the global variable `g_pVBoxFuncs` contains a pointer to the structure which contains the helper function pointers. Here's how to initialize the C API:

```
#include "VBoxCAPIGlue.h"
...
IVirtualBoxClient *vboxclient   = NULL;

/*
 * VBoxCGlueInit() loads the necessary dynamic library, handles errors
 * (producing an error message hinting what went wrong) and gives you
 * the pointer to the function table (g_pVBoxFuncs).
 *
 * Once you get the function table, then how and which functions
 * to use is explained below.
 *
 * g_pVBoxFuncs->pfnClientInitialize does all the necessary startup
 * action and provides us with pointers to an IVirtualBoxClient instance.
 * It should be matched by a call to g_pVBoxFuncs->pfnClientUninitialize()
 * when done.
 */

if (VBoxCGlueInit())
{
    fprintf(stderr, "s: FATAL: VBoxCGlueInit failed: %s\n",
            argv[0], g_szVBoxErrMsg);
    return EXIT_FAILURE;
}

g_pVBoxFuncs->pfnClientInitialize(NULL, &vboxclient);
if (!vboxclient)
{
    fprintf(stderr, "%s: FATAL: could not get VirtualBoxClient reference\n",
            argv[0]);
    return EXIT_FAILURE;
}
```

If `vboxclient` is still `NULL` this means the initializationi failed and the VirtualBox C API cannot be used.

It is possible to write C applications using multiple threads which all use the VirtualBox API, as long as you're initializing the C API in each thread which your application creates. This is done with `g_pVBoxFuncs->pfnClientThreadInitialize()` and likewise before the thread is terminated the API must be uninitialized with `g_pVBoxFuncs->pfnClientThreadUninitialize()`. You don't have to use these functions in worker threads created by COM/XPCOM (which you might utilize if your code uses active event handling), since everything is initialized correctly

already. On Windows the C bindings create a marshaller which supports a wide range of COM threading models, from Single-Threaded Apartments (STA) to Multithreaded Apartments (MTA), so you don't have to worry about these details unless you plan to use active event handlers. See the sample code for how to get this to work reliably (in other words think twice if passive event handling isn't the better solution after you looked at the sample code).

2.3.6.4 C API attribute and method invocation

Method invocation is straightforward. It looks very similar to the C++ mechanism since it also uses a macro which internally accesses the vtable and additionally needs to be passed a pointer to the object as the first argument to serve as the `this` pointer.

Using the C bindings all method invocations return a numeric result code of type `HRESULT` (with a few exceptions which normally are not relevant).

If an interface is specified as returning an object, a pointer to a pointer to the appropriate object must be passed as the last argument. The method will then store an object pointer in that location.

Likewise, attributes (properties) can be queried or set using method invocations, using specially named methods. For each attribute there exists a getter method, the name of which is composed of `get_` followed by the capitalized attribute name. Unless the attribute is read-only, an analogous `set_` method exists. Let's apply these rules to get the `IVirtualBox` reference, an `ISession` instance reference and read the IVirtualBox::revision attribute:

```
IVirtualBox *vbox               = NULL;
ISession *session               = NULL;
HRESULT rc;
ULONG revision;

rc = IVirtualBoxClient_get_VirtualBox(vboxclient, &vbox);

if (FAILED(rc) || !vbox)
{
    PrintErrorInfo(argv[0], "FATAL: could not get VirtualBox reference", rc);
    return EXIT_FAILURE;
}
rc = IVirtualBoxClient_get_Session(vboxclient, &session);
if (FAILED(rc) || !session)
{
    PrintErrorInfo(argv[0], "FATAL: could not get Session reference", rc);
    return EXIT_FAILURE;
}

rc = IVirtualBox_get_Revision(vbox, &revision);
if (SUCCEEDED(rc))
{
    printf("Revision: %u\n", revision);
}
```

The convenience macros for calling a method are named by prepending the method name with the interface name (using `_` as the separator).

So far only attribute getters were illustrated, but generic method calls are straightforward, too:

```
IMachine *machine = NULL;
BSTR vmname = ...;
...
/*
 * Calling IMachine::findMachine(...)
 */
rc = IVirtualBox_FindMachine(vbox, vmname, &machine);
```

As a more complicated example of a method invocation, let's call IMachine::launchVMProcess which returns an IProgress object. Note again that the method name is capitalized:

```
IProgress *progress;
...
rc = IMachine_LaunchVMProcess(
    machine,       /* this  */
    session,       /* arg 1 */
    sessionType,   /* arg 2 */
    env,           /* arg 3 */
    &progress      /* Out   */
);
```

All objects with their methods and attributes are documented in chapter 6, *Classes (interfaces)*, page 38.

2.3.6.5 String handling

When dealing with strings you have to be aware of a string's encoding and ownership.

Internally, the API uses UTF-16 encoded strings. A set of conversion functions is provided to convert other encodings to and from UTF-16. The type of a UTF-16 character is `BSTR` (or its constant counterpart `CBSTR`), which is an array type, represented by a pointer to the start of the zero-terminated string. There are functions for converting between UTF-8 and UTF-16 strings available through `g_pVBoxFuncs`:

```
int (*pfnUtf16ToUtf8)(CBSTR pwszString, char **ppszString);
int (*pfnUtf8ToUtf16)(const char *pszString, BSTR *ppwszString);
```

The ownership of a string determines who is responsible for releasing resources associated with the string. Whenever the API creates a string (essentially for output parameters), ownership is transferred to the caller. To avoid resource leaks, the caller should release resources once the string is no longer needed. There are plenty of examples in the sample code.

2.3.6.6 Array handling

Arrays are handled somewhat similarly to strings, with the additional information of the number of elements in the array. The exact details of string passing depends on the platform middleware (COM/XPCOM), and therefore the C binding offers helper functions to gloss over these differences.

Passing arrays as input parameters to API methods is usually done by the following sequence, calling a hypothetical `IArrayDemo_PassArray` API method:

```
static const ULONG aElements[] = { 1, 2, 3, 4 };
ULONG cElements = sizeof(aElements) / sizeof(aElements[0]);
SAFEARRAY *psa = NULL;
psa = g_pVBoxFuncs->pfnSafeArrayCreateVector(VT_I4, 0, cElements);
g_pVBoxFuncs->pfnSafeArrayCopyInParamHelper(psa, aElements, sizeof(aElements));
IArrayDemo_PassArray(pThis, ComSafeArrayAsInParam(psa));
g_pVBoxFuncs->pfnSafeArrayDestroy(psa);
```

Likewise, getting arrays results from output parameters is done using helper functions which manage memory allocations as part of their other functionality:

```
SAFEARRAY *psa = g_pVBoxFuncs->pfnSafeArrayOutParamAlloc();
ULONG *pData;
ULONG cElements;
IArrayDemo_ReturnArray(pThis, ComSafeArrayAsOutTypeParam(psa, ULONG));
g_pVBoxFuncs->pfnSafeArrayCopyOutParamHelper((void **)&pData, &cElements, VT_I4, psa);
g_pVBoxFuncs->pfnSafeArrayDestroy(psa);
```

This covers the necessary functionality for all array element types except interface references. These need special helpers to manage the reference counting correctly. The following code snippet gets the list of VMs, and passes the first IMachine reference to another API function (assuming that there is at least one element in the array, to simplify the example):

```
SAFEARRAY psa = g_pVBoxFuncs->pfnSafeArrayOutParamAlloc();
IMachine **machines = NULL;
ULONG machineCnt = 0;
ULONG i;
IVirtualBox_get_Machines(virtualBox, ComSafeArrayAsOutIfaceParam(machinesSA, IMachine *));
g_pVBoxFuncs->pfnSafeArrayCopyOutIfaceParamHelper((IUnknown ***)&machines, &machineCnt, machinesSA);
g_pVBoxFuncs->pfnSafeArrayDestroy(machinesSA);
/* Now "machines" contains the IMachine references, and machineCnt the
 * number of elements in the array. */
...
SAFEARRAY *psa = g_pVBoxFuncs->pfnSafeArrayCreateVector(VT_IUNKNOWN, 0, 1);
g_pVBoxFuncs->pfnSafeArrayCopyInParamHelper(psa, (void *)&machines[0], sizeof(machines[0]));
IVirtualBox_GetMachineStates(ComSafeArrayAsInParam(psa), ...);
...
g_pVBoxFuncs->pfnSafeArrayDestroy(psa);
for (i = 0; i < machineCnt; ++i)
{
    IMachine *machine = machines[i];
    IMachine_Release(machine);
}
free(machines);
```

Handling output parameters needs more special handling than input parameters, thus only for the former are there special helpers while the latter is handled through the generic array support.

2.3.6.7 Event handling

The VirtualBox API offers two types of event handling, active and passive, and consequently there is support for both with the C API binding. Active event handling (based on asynchronous callback invocation for event delivery) is more difficult, as it requires the construction of valid C++ objects in C, which is inherently platform and compiler dependent. Passive event handling is much simpler, it relies on an event loop, fetching events and triggering the necessary handlers explicitly in the API client code. Both approaches depend on an event loop to make sure that events get delivered in a timely manner but otherwise differ in what else is required to implement the respective event handler type.

The C API sample contains code for both event handling styles, and one has to modify the appropriate `#define` to select which style is actually used by the compiled program. It allows a good comparison between the two variants and the code sequences are probably worth reusing without much change in other API clients with only minor adaptions.

Active event handling needs to ensure that the following helper function is called frequently enough in the primary thread:

```
g_pVBoxFuncs->pfnProcessEventQueue(cTimeoutMS);
```

The actual event handler implementation is quite tedious, as it has to implement a complete API interface. Especially on Windows it is a lot of work to implement the complicated `IDispatch` interface, requiring loading COM type information and using it in the `IDispatch` method implementation. Overall this is quite tedious compared to passive event handling.

Passive event handling uses a similar event loop structure, which requires calling the following function in a loop, and processing the returned event appropriately:

```
rc = IEventSource_GetEvent(pEventSource, pListener, cTimeoutMS, &pEvent);
```

After processing the event it needs to be marked as processed with the following method call:

```
rc = IEventSource_EventProcessed(pEventSource, pListener, pEvent);
```

This is vital for vetoable events, as they would be stuck otherwise, waiting on whether the veto comes or not. It does not do any harm for other event types, and in the end is cheaper than checking if the event at hand is vetoable or not.

The general event handling concepts are described in the API specification (see chapter 3.4, *VirtualBox events*, page 29), including how to aggregate multiple event sources for processing in one event loop. As mentioned, the sample illustrates the practical aspects of how to use both types of event handling, active and passive, from a C application. Additional hints are in the comments documenting the helper methods in `VBoxCAPI_v7_2.h`. The code complexity of active event handling (and its inherently platform/compiler specific aspects) should be motivation to use passive event handling wherever possible.

2.3.6.8 C API uninitialization

Uninitialization is performed by `g_pVBoxFuncs->pfnClientUninitialize()`. If your program can exit in more than one place, it is a good idea to install this function as an exit handler using the C library function `atexit()` just after calling `g_pVBoxFuncs->pfnClientInitialize()` , e.g.

```
#include <stdlib.h>
#include <stdio.h>

...

/*
 * Make sure g_pVBoxFuncs->pfnClientUninitialize() is called at exit, no
 * matter if we return from the initial call to main or call exit()
 * somewhere else. Note that atexit registered functions are not
 * called upon abnormal termination, i.e. when calling abort() or
 * signal().
 */

if (atexit(g_pVBoxFuncs->pfnClientUninitialize()) != 0) {
    fprintf(stderr, "failed to register g_pVBoxFuncs->pfnClientUninitialize()\n");
    exit(EXIT_FAILURE);
}
```

Another idea would be to write your own `void myexit(int status)` function, calling `g_pVBoxFuncs->pfnClientUninitialize()` followed by the real `exit()`, and use it instead of `exit()` throughout your program and at the end of `main`.

If you expect your program to be terminated by a signal (e.g. a user types CTRL-C sending SIGINT) you might want to install a signal handler setting a flag noting that a signal was sent and then calling `g_pVBoxFuncs->pfnClientUninitialize()` later on, *not* from the handler itself.

That said, if a client program forgets to call `g_pVBoxFuncs->pfnClientUninitialize()` before it terminates, there is a mechanism in place which will eventually release references held by the client. On Windows it can take quite a while, on the order of 6-7 minutes.

2.3.6.9 Compiling and linking

A program using the C binding has to open the library during runtime using the help of the glue code provided and as shown in the example `tstCAPIGlue.c`. Compilation and linking can be achieved with a makefile fragment similar to:

```
# Where is the SDK directory?
PATH_SDK      = ../../..
CAPI_INC      = -I$(PATH_SDK)/bindings/c/include
ifdef ProgramFiles
PLATFORM_INC  = -I$(PATH_SDK)/bindings/mscom/include
PLATFORM_LIB  = $(PATH_SDK)/bindings/mscom/lib
else
PLATFORM_INC  = -I$(PATH_SDK)/bindings/xpcom/include
```

```
PLATFORM_LIB  = $(PATH_SDK)/bindings/xpcom/lib
endif
GLUE_DIR      = $(PATH_SDK)/bindings/c/glue
GLUE_INC      = -I$(GLUE_DIR)

# Compile Glue Library
VBoxCAPIGlue.o: $(GLUE_DIR)/VBoxCAPIGlue.c
    $(CC) $(CFLAGS) $(CAPI_INC) $(PLATFORM_INC) $(GLUE_INC) -o $@ -c $<

# Compile interface ID list
VirtualBox_i.o: $(PLATFORM_LIB)/VirtualBox_i.c
    $(CC) $(CFLAGS) $(CAPI_INC) $(PLATFORM_INC) $(GLUE_INC) -o $@ -c $<

# Compile program code
program.o: program.c
    $(CC) $(CFLAGS) $(CAPI_INC) $(PLATFORM_INC) $(GLUE_INC) -o $@ -c $<

# Link program.
program: program.o VBoxCAPICGlue.o VirtualBox_i.o
    $(CC) -o $@ $^ -ldl -lpthread
```

2.3.6.10 Conversion of code using legacy C bindings

This section aims to make the task of converting code using the legacy C bindings to the new style a breeze by following these key steps.

One necessary change is adjusting your Makefile to reflect the different include paths. As shown in the makefile fragment above, there are now three relevant include directories. The XPCOM include directory is still relevant for platforms where the XPCOM middleware is used but most of its include files live elsewhere now so it's good to have it last. Additionally the `VirtualBox_i.c` file needs to be compiled and linked to the program since it contains the interface IDs (IIDs) relevant for the VirtualBox API, making sure they are not replicated endlessly if the code refers to them frequently.

The C API client code should include `VBoxCAPIGlue.h` instead of `VBoxXPCOMCGlue.h` or `VBoxCAPI_v7_2.h`, as this makes sure the correct macros and internal translations are selected.

All API method calls (anything mentioning `vtbl`) should be rewritten using the convenience macros for calling methods, as these hide the internal details, are generally easier to use, and are shorter to type. You should remove as many as possible `(nsISupports **)` or similar typecasts, as the new style should use the correct type in most places, increasing the type safety in case of an error in the source code.

To gloss over the platform differences, API client code should no longer rely on XPCOM specific interface names such as `nsISupports`, `nsIException` and `nsIEventQueue`, and replace them by the platform independent interface names `IUnknown` and `IErrorInfo` for the first two respectively. Event queue handling should be replaced by using the platform independent way described in chapter 2.3.6.7, *Event handling*, page 24.

Finally adjust the string and array handling to use the new helpers, as these make sure the code works without changes with both COM and XPCOM, which are significantly different in this area. The code should be double checked to see that it uses the correct way to manage memory and is freeing it only after the last use.

2.3.6.11 Legacy C bindings to VirtualBox API for XPCOM

> **Note:** This section applies to Linux, Mac OS X and Solaris hosts only and describes deprecated use of the API from C.

Starting with version 2.2, VirtualBox offers C bindings for its API which works only on platforms using XPCOM. Refer to the old SDK documentation (included in the SDK packages for version 4.3.6 or earlier) since it still applies unchanged. The fundamental concepts are similar

(but the syntactical details are quite different) to the newer cross-platform C bindings which should be used for all new code, as the support for the old C bindings will go away in a major release after version 4.3.

3 Basic VirtualBox concepts; some examples

The following explains some basic VirtualBox concepts such as the VirtualBox object, sessions and how virtual machines are manipulated and launched using the Main API. The coding examples use a pseudo-code style closely related to the object-oriented web service (OOWS) for JAX-WS. Depending on which environment you are using, you will need to adjust the examples.

3.1 Obtaining basic machine information. Reading attributes

Any program using the Main API will first need access to the global VirtualBox object (see IVirtualBox), from which all other functionality of the API is derived. With the OOWS for JAX-WS, this is returned from the IWebsessionManager::logon() call.

To enumerate virtual machines, one would look at the "machines" array attribute in the VirtualBox object (see IVirtualBox::machines). This array contains all virtual machines currently registered with the host, each of them being an instance of IMachine. From each such instance, one can query additional information, such as the UUID, the name, memory, operating system and more by looking at the attributes; see the attributes list in IMachine documentation.

As mentioned in the preceding chapters, depending on your programming environment, attributes are mapped to corresponding "get" and (if the attribute is not read-only) "set" methods. So when the documentation says that IMachine has a "name" attribute, this means you need to code something like the following to get the machine's name:

```
IMachine machine = ...;
String name = machine.getName();
```

Boolean attribute getters can sometimes be called `isAttribute()` due to JAX-WS naming conventions.

3.2 Changing machine settings: Sessions

As described in the previous section, to read a machine's attribute, one invokes the corresponding "get" method. One would think that to change settings of a machine, it would suffice to call the corresponding "set" method – for example, to set a VM's memory to 1024 MB, one would call `setMemorySize(1024)`. Try that, and you will get an error: "The machine is not mutable."

So unfortunately, things are not that easy. VirtualBox is a complicated environment in which multiple processes compete for possibly the same resources, especially machine settings. As a result, machines must be "locked" before they can either be modified or started. This is to prevent multiple processes from making conflicting changes to a machine: it should, for example, not be allowed to change the memory size of a virtual machine while it is running. (You can't add more memory to a real computer while it is running either, at least not to an ordinary PC.) Also, two processes must not change settings at the same time, or start a machine at the same time.

These requirements are implemented in the Main API by way of "sessions", in particular, the ISession interface. Each process which talks to VirtualBox needs its own instance of ISession. In the web service, you can request the creation of such an object by calling IWebsessionManager::getSessionObject(). More complex management tasks might need multiple instances of ISession, and each call returns a new one.

This session object must then be used like a mutex semaphore in common programming environments. Before you can change machine settings, you must write-lock the machine by calling IMachine::lockMachine() with your process's session object.

After the machine has been locked, the ISession::machine attribute contains a copy of the original IMachine object upon which the session was opened, but this copy is "mutable": you can invoke "set" methods on it.

When done making the changes to the machine, you must call IMachine::saveSettings(), which will copy the changes you have made from your "mutable" machine back to the real machine and write them out to the machine settings XML file. This will make your changes permanent.

Finally, it is important to always unlock the machine again, by calling ISession::unlockMachine(). Otherwise, when the calling process exits, the machine will be moved to the "aborted" state, which can lead to loss of data.

So, as an example, the sequence to change a machine's memory to 1024 MB is something like this:

```
IWebsessionManager mgr ...;
IVirtualBox vbox = mgr.logon(user, pass);
...
IMachine machine = ...;  // read-only machine
ISession session = mgr.getSessionObject();
machine.lockMachine(session, LockType.Write); // machine is now locked for writing
IMachine mutable = session.getMachine();      // obtain the mutable machine copy
mutable.setMemorySize(1024);
mutable.saveSettings();                       // write settings to XML
session.unlockMachine();
```

3.3 Launching virtual machines

To launch a virtual machine, you call IMachine::launchVMProcess(). This instructs the VirtualBox engine to start a new process containing the virtual machine. The host system sees each virtual machine as a single process, even if the virtual machine has hundreds of its own processes running inside it. (This new VM process in turn obtains a write lock on the machine, as described above, to prevent conflicting changes from other processes; this is why opening another session will fail while the VM is running.)

Starting a machine looks something like this:

```
IWebsessionManager mgr ...;
IVirtualBox vbox = mgr.logon(user, pass);
...
IMachine machine = ...;  // read-only machine
ISession session = mgr.getSessionObject();
IProgress prog = machine.launchVMProcess(session,
                                         "gui",  // session type
                                         "");    // possibly environment setting
prog.waitForCompletion(10000);  // give the process 10 secs
if (prog.getResultCode() != 0)  // check success
    System.out.println("Cannot launch VM!")
```

The caller's session object can then be used as a sort of remote control to the VM process that was launched. It contains a "console" object (see ISession::console) with which the VM can be paused, stopped, snapshotted or other things.

3.4 VirtualBox events

In VirtualBox, "events" provide a uniform mechanism to register for and consume specific events. A VirtualBox client can register an "event listener" (represented by the IEventListener interface),

which will then get notified by the server when an event (represented by the IEvent interface) happens.

The IEvent interface is an abstract parent interface for all events that can occur in VirtualBox. The actual events that the server sends out belong to one of the specific subclasses, for example IMachineStateChangedEvent or IMediumChangedEvent.

As an example, the VirtualBox GUI waits for machine events so it can update its display when the machine state changes or machine settings are modified, even if this happens in another client. This is how the GUI can automatically refresh its display even if you manipulate a machine from a different client such as VBoxManage.

To register an event listener to listen for events, you would use code similar to the following:

```
EventSource es = console.getEventSource();
IEventListener listener = es.createListener();
VBoxEventType aTypes[] = (VBoxEventType.OnMachineStateChanged);
        // list of event types to listen for
es.registerListener(listener, aTypes, false /* active */);
        // register passive listener
IEvent ev = es.getEvent(listener, 1000);
        // wait up to one second for event to happen
if (ev != null)
{
    // downcast to specific event interface (in this case we have only registered
    // for one type, otherwise IEvent::type would tell us)
    IMachineStateChangedEvent mcse = IMachineStateChangedEvent.queryInterface(ev);
    ... // inspect and do something
    es.eventProcessed(listener, ev);
}
...
es.unregisterListener(listener);
```

A graphical user interface application would most likely want to start its own thread to wait for events and then process these in a loop.

The events mechanism was introduced with VirtualBox 3.3 and replaces various callback interfaces which were called for each event in the interface. The callback mechanism was not compatible with scripting languages, local Java bindings, and remote web services as they do not support callbacks. The new mechanism with events and event listeners works with all of these.

To simplify development of applications using events, the concept of an event aggregator was introduced. Essentially it's a mechanism to aggregate multiple event sources into one single event source, and then work with this single aggregated event source instead of the original sources. As an example, one can try out the VirtualBox Python shell's `recordDemo` option which records mouse and keyboard events using an event aggregator and displays them as separate event sources. The VirtualBox Python shell (vboxshell.py) is shipped with the SDK. The `recordDemo` code is essentially:

```
listener = console.eventSource.createListener()
agg = console.eventSource.createAggregator([console.keyboard.eventSource, console.mouse.eventSource])
agg.registerListener(listener, [ctx['global'].constants.VBoxEventType_Any], False)
registered = True
end = time.time() + dur
while  time.time() < end:
    ev = agg.getEvent(listener, 1000)
    processEent(ev)
agg.unregisterListener(listener)
```

Without using aggregators consumers would have to either poll on both sources or start multiple threads to block on those sources.

4 The VirtualBox shell

VirtualBox comes with an extensible shell which allows you to control your virtual machines from the command line. It is also a non-trivial example of how to use the VirtualBox APIs from Python for all three styles of the API (COM/XPCOM/WS).

You can easily extend this shell with your own commands. Simply create a subdirectory named `.config/VirtualBox/shexts` in your home directory (`.VirtualBox/shexts` on Windows systems and `Library/VirtualBox/shexts` on macOS systems) and put a Python file implementing your shell extension commands in this directory. This file must have an array named `commands` which contains your command definitions:

```
commands = {
'cmd1': ['Command cmd1 help', cmd1],
'cmd2': ['Command cmd2 help', cmd2]
}
```

For example, to create a command for creating hard drive images, the following code can be used:

```
def createHdd(ctx,args):
  # Show some meaningful error message on wrong input
  if (len(args) < 3):
    print "usage: createHdd sizeM location type"
    return 0

  # Get arguments
  size = int(args[1])
  loc = args[2]
  if len(args) > 3:
    format = args[3]
  else:
    # And provide some meaningful defaults
    format = "vdi"

# Call VirtualBox API, using context's fields
hdd = ctx['vb'].createMedium(format, loc, ctx['global'].constants.AccessMode_ReadWrite, \
                             ctx['global'].constants.DeviceType_HardDisk)
# Access constants using ctx['global'].constants
progress = hdd.createBaseStorage(size, (ctx['global'].constants.MediumVariant_Standard, ))
# use standard progress bar mechanism
ctx['progressBar'](progress)


# Report errors
if not hdd.id:
   print "cannot create disk (file %s exist?)" %(loc)
   return 0

# Give user some feedback on success too
print "created HDD with id: %s" %(hdd.id)

# 0 means continue execution, other values mean exit from the interpreter
return 0

commands = {
```

```
'myCreateHDD': ['Create virtual HDD, createHdd size location type', createHdd]
}
```

Just store the above text in a file named `createHdd` (or any other meaningful name) in the `shexts` directory located as described above and then start the VirtualBox shell or just issue the `reloadExts` command if the shell is already running. Your new command will now be available.

5 Main API changes to support the ARM64 architecture

VirtualBox 7.1 introduces support for running virtual machines on hosts containing an ARM CPU. Since VirtualBox has been designed and developed for virtualizing x86 hardware since its inception adding support for ARM CPUs required a variety of changes to the Main API to accommodate two different CPU architectures.

5.1 Overview of the Changes to the Main API

In order to support two different types of processor architecture in the VirtualBox Main API the key abstraction was the addition of a new interface named IPlatform which contains the details which relate to the operating environment of a VM. The IPlatform interface is distinct from the VM itself (the IMachine interface), the VM management interfaces (IVirtualBox and IConsole), as well as the global properties of the VM (the ISystemProperties interface). The contents of the IPlatform interface can be broken down into three different categories:

- properties relating to the VM which are CPU architecture-neutral (IPlatformProperties)
- properties which are CPU architecture-specific (IPlatformX86 or IPlatformARM)
- attributes of the VM which are CPU architecture-neutral such as:
  - the architecture of the VM (IPlatform::architecture)
  - the type of chipset used by the VM (IPlatform::chipsetType)
  - the type of IOMMU used by the VM (IPlatform::iommuType).

The IPlatform interface can be retrieved from the IMachine interface via the new IMachine::platform attribute. The IPlatform::architecture attribute returns the *architecture* type in a new enumeration named PlatformArchitecture. This enumeration value can be used to determine which CPU-specific properties to query, for example:

```
IPlatform platform;
result = machine.getPlatform(platform);

PlatformArchitecture_T platformArch;
result = platform.getArchitecture(platformArch);
if (platformArch == x86)
  IPlatformX86 platformX86;
  result = platform.getX86(platformX86);
else if (platformArch == ARM)
  IPlatformARM platformARM;
  result = platform.getARM(platformARM);
else
  error;
```

The value of the PlatformArchitecture enumeration can also be used when creating a new VM using IVirtualBox::createMachine.

The CPU architecture-neutral properties in (IPlatformProperties) can be found in several places:

- They can be retrieved from the IPlatform::properties attribute.
- They can be retrieved from the ISystemProperties::platform attribute.
- They can retrieved from the IVirtualBox interface using the new IVirtualBox::getPlatformProperties method.

The attributes and methods contained in IPlatformProperties were moved from the ISystemProperties interface as they can vary based on the platform of the VM:

Original Attribute	New Attribute
ISystemProperties::rawModeSupported	IPlatformProperties::rawModeSupported
ISystemProperties::exclusiveHwVirt	IPlatformProperties::exclusiveHwVirt
ISystemProperties::serialPortCount	IPlatformProperties::serialPortCount
ISystemProperties::parallelPortCount	IPlatformProperties::parallelPortCount
ISystemProperties::maxBootPosition	IPlatformProperties::maxBootPosition
ISystemProperties::supportedParavirtProviders[]	IPlatformProperties::supportedParavirtProviders[]
ISystemProperties::supportedFirmwareTypes[]	IPlatformProperties::supportedFirmwareTypes[]
ISystemProperties::supportedGuestOSTypes[]	IPlatformProperties::supportedGuestOSTypes[]
ISystemProperties::getSupportedGfxControllerTypes[]	IPlatformProperties::getSupportedGfxControllerTypes[]
ISystemProperties::supportedNetAdpPromiscModePols[]	IPlatformProperties::supportedNetAdpPromiscModePols[]
ISystemProperties::supportedNetworkAdapterTypes[]	IPlatformProperties::supportedNetworkAdapterTypes[]
ISystemProperties::supportedUartTypes[]	IPlatformProperties::supportedUartTypes[]
ISystemProperties::supportedUSBControllerTypes[]	IPlatformProperties::supportedUSBControllerTypes[]
ISystemProperties::supportedAudioControllerTypes[]	IPlatformProperties::supportedAudioControllerTypes[]
ISystemProperties::supportedBootDevices[]	IPlatformProperties::supportedBootDevices[]
ISystemProperties::supportedStorageBuses[]	IPlatformProperties::supportedStorageBuses[]
ISystemProperties::supportedStorageControllerTypes[]	IPlatformProperties::supportedStorageControllerTypes[]
ISystemProperties::supportedChipsetTypes[]	IPlatformProperties::supportedChipsetTypes[]
ISystemProperties::supportedIommuTypes[]	IPlatformProperties::supportedIommuTypes[]
ISystemProperties::supportedTpmTypes[]	IPlatformProperties::supportedTpmTypes[]

Original Method	New Method
ISystemProperties::getDeviceTypesForStorageBus	IPlatformProperties::getDeviceTypesForStorageBus
ISystemProperties::getMaxDevicesPerPortForStorageBus	IPlatformProperties::getMaxDevicesPerPortForStorageBus
ISystemProperties::getMaxInstancesOfStorageBus	IPlatformProperties::getMaxInstancesOfStorageBus
ISystemProperties::getMaxInstancesOfUSBControllerType	IPlatformProperties::getMaxInstancesOfUSBControllerType
ISystemProperties::getMaxNetworkAdapters	IPlatformProperties::getMaxNetworkAdapters
ISystemProperties::getMaxNetworkAdaptersOfType	IPlatformProperties::getMaxNetworkAdaptersOfType
ISystemProperties::getMaxPortCountForStorageBus	IPlatformProperties::getMaxPortCountForStorageBus
ISystemProperties::getMinPortCountForStorageBus	IPlatformProperties::getMinPortCountForStorageBus
ISystemProperties::getStorageBusForControllerType	IPlatformProperties::getStorageBusForControllerType
ISystemProperties::getStorageControllerHotplugCapable	IPlatformProperties::getStorageControllerHotplugCapable
ISystemProperties::getStorageControllerTypesForBus	IPlatformProperties::getStorageControllerTypesForBus

The single attribute and all of the properties contained in the new IPlatformX86 interface were moved from the IMachine interface as they are specific to the x86 CPU architecture:

Original Attribute	New Attribute
IMachine::HPETEnabled	IPlatformX86::HPETEnabled

Original Method	New Method
IMachine::getCPUIDLeaf	IPlatformX86::getCPUIDLeaf
IMachine::getCPUIDLeafByOrdinal	IPlatformX86::getCPUIDLeafByOrdinal
IMachine::getCPUProperty	IPlatformX86::getCPUProperty
IMachine::getHWVirtExProperty	IPlatformX86::getHWVirtExProperty
IMachine::removeAllCPUIDLeaves	IPlatformX86::removeAllCPUIDLeaves
IMachine::removeCPUIDLeaf	IPlatformX86::removeCPUIDLeaf
IMachine::setCPUIDLeaf	IPlatformX86::setCPUIDLeaf
IMachine::setCPUProperty	IPlatformX86::setCPUProperty
IMachine::setHWVirtExProperty	IPlatformX86::setHWVirtExProperty

An itemized breakdown of the changes made to the Main API for ARM64 support is described below.

5.1.1 Changes to Classes (Interfaces)

5.1.1.1 New Classes (Interfaces) Added

- IPlatform

  An umbrella-like interface containing a collection of properties and attributes of the software environment of the VM.

- IPlatformX86

  An interface containing the x86-specific properties of the software environment of the VM.

- IPlatformARM

  An interface containing the ARM-specific properties of the software environment of the VM.

- IPlatformProperties

  An interface containing the properties specific to the platform of the VM.

- IHostX86

  An interface which provides further classification of the x86-specific properties of the physical host machine.

5.1.1.2 Classes (Interfaces) Renamed

Original Class Name	New Class Name
IBIOSSettings	IFirmwareSettings

The IBIOSSettings interface has been renamed to the more platform-neutral IFirmwareSettings.

5.1.1.3 New Methods Added to Classes (Interfaces)

- IPlatformProperties IVirtualBox::getPlatformProperties

  The new IVirtualBox::getPlatformProperties method returns an IPlatformProperties object which contains the platform-specific properties of a VM as described above.

- IGraphicsAdapter::isFeatureEnabled

  The new IGraphicsAdapter::isFeatureEnabled method returns whether a graphics feature is enabled for a specific adapter or not.

- IGraphicsAdapter::setFeature

  The new IGraphicsAdapter::setFeature enables or disables a graphics feature for a specific adapter.

5.1.1.4 New Attributes Added to Classes (Interfaces)

- IPlatformProperties ISystemProperties::platform

  The ISystemProperties interface gets a new *platform* attribute which returns an IPlatform-Properties object which contains the platform-specific properties of a VM as described above.

- FirmwareType IFirmwareSettings::firmwareType

  The new IFirmwareSettings interface (renamed from IBIOSSettings) gets a new *firmware-Type* attribute which returns a FirmwareType enumeration which contains the VM's firmware type.

- PlatformArchitecture IHost::architecture

  The IHost interface gets a new *architecture* attribute which returns a PlatformArchitecture enumeration which contains the platform's architecture.

- IHostX86 IHost::x86

  The IHost interface gets a new *x86* attribute which returns an IHostX86 object which contains the x86-specific properties of the physical host machine.

- IPlatform IMachine::platform

  The IMachine interface gets a new *platform* attribute which returns an IPlatform object which contains the properties related to the platform of the VM.

- IFirmwareSettings IMachine::firmwareSettings

  The IMachine interface gets a new *firmwareSettings* attribute which returns an IFirmWare-Settings object which contains the properties related to the platform of the VM.

5.1.1.5 Class (Interface) Attribute Changes

Original Class Name	New Class Name
BIOSBootMenuMode IBIOSSettings::bootMenuMode	FirmwareBootMenuMode IFirmwareSettings::bootMenuMode
IGraphicsAdapter::accelerate3DEnabled	IGraphicsAdapter::setFeature and IGraphicsAdapter::isFeatureEnabled
IGraphicsAdapter::accelerate2DVideoEnabled	IGraphicsAdapter::setFeature and IGraphicsAdapter::isFeatureEnabled

The IBIOSSettings::bootMenuMode attribute has been moved to the new IFirmwareSettings interface and now returns the firmware boot menu mode in the new FirmwareBootMenuMode enumeration.

Original Class Name	New Class Name
unsigned long ISerialPort::IOBase	unsigned long ISerialPort::IOAddress

The ISerialPort::IOBase attribute has been renamed to ISerialPort::IOAddress.

5.1.2 Changes to Enumerations

5.1.2.1 New Enumerations Added

- PlatformArchitecture

  The new PlatformArchitecture enumeration contains the CPU architecture type of the VM.

- GraphicsFeature

  The new GraphicsFeature enumeration contains graphics features to be used with the host and / or virtual graphics controller.

5.1.2.2 New Enumerations Values

- CPUArchitecture
  - ARMv8_32
  - ARMv8_64

  The CPUArchitecture enumeration gets two new additions corresponding to 32-bit and 64-bit ARMv8 CPUs respectively.

- AudioControllerType
  - VirtioSound

  The AudioControllerType enumeration gets a new value named *VirtioSound*.

- ChipsetType
  - ARMv8Virtual

  The ChipsetType enumeration gets a new value named *ARMv8Virtual*.

5.1.2.3 Enumerations Renamed

Original Enumeration Name	New Enumeration Name
CPUPropertyType	CPUPropertyTypeX86

The CPUPropertyType enumeration has been renamed to CPUPropertyTypeX86.

Original Enumeration Name	New Enumeration Name
BIOSBootMenuMode	FirmwareBootMenuMode

The BIOSBootMenuMode enumeration has been renamed to FirmwareBootMenuMode.

6 Classes (interfaces)

6.1 IAdditionsFacility

> **Note:** With the web service, this interface is mapped to a structure. Attributes that return this interface will not return an object, but a complete structure containing the attributes listed below as structure members.

Structure representing a Guest Additions facility.

6.1.1 Attributes

6.1.1.1 classType (read-only)

`AdditionsFacilityClass IAdditionsFacility::classType`

The class this facility is part of.

6.1.1.2 lastUpdated (read-only)

`long long IAdditionsFacility::lastUpdated`

Timestamp of the last status update, in milliseconds since 1970-01-01 UTC.

6.1.1.3 name (read-only)

`wstring IAdditionsFacility::name`

The facility's friendly name.

6.1.1.4 status (read-only)

`AdditionsFacilityStatus IAdditionsFacility::status`

The current status.

6.1.1.5 type (read-only)

`AdditionsFacilityType IAdditionsFacility::type`

The facility's type ID.

6.2 IAdditionsStateChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a Guest Additions property changes. Interested callees should query IGuest attributes to find out what has changed.

6.2.1 Attributes

6.2.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IAdditionsStateChangedEvent::midlDoesNotLikeEmptyInterfaces
```

6.3 IAppliance

Represents a platform-independent appliance in OVF format. An instance of this is returned by IVirtualBox::createAppliance(), which can then be used to import and export virtual machines within an appliance with VirtualBox.

The OVF standard suggests two different physical file formats:

1. If the appliance is distributed as a set of files, there must be at least one XML descriptor file that conforms to the OVF standard and carries an `.ovf` file extension. If this descriptor file references other files such as disk images, as OVF appliances typically do, those additional files must be in the same directory as the descriptor file.

2. If the appliance is distributed as a single file, it must be in TAR format and have the `.ova` file extension. This TAR file must then contain at least the OVF descriptor files and optionally other files.

   At this time, VirtualBox does not not yet support the packed (TAR) variant; support will be added with a later version.

Importing an OVF appliance into VirtualBox as instances of IMachine involves the following sequence of API calls:

1. Call IVirtualBox::createAppliance(). This will create an empty IAppliance object.

2. On the new object, call read() with the full path of the OVF file you would like to import. So long as this file is syntactically valid, this will succeed and fill the appliance object with the parsed data from the OVF file.

3. Next, call interpret(), which analyzes the OVF data and sets up the contents of the IAppliance attributes accordingly. These can be inspected by a VirtualBox front-end such as the GUI, and the suggestions can be displayed to the user. In particular, the virtualSystemDescriptions[] array contains instances of IVirtualSystemDescription which represent the virtual systems (machines) in the OVF, which in turn describe the virtual hardware prescribed by the OVF (network and hardware adapters, virtual disk images, memory size and so on). The GUI can then give the user the option to confirm and/or change these suggestions.

4. If desired, call IVirtualSystemDescription::setFinalValues() for each virtual system (machine) to override the suggestions made by the interpret() routine.

5. Finally, call importMachines() to create virtual machines in VirtualBox as instances of IMachine that match the information in the virtual system descriptions. After this call succeeded, the UUIDs of the machines created can be found in the machines[] array attribute.

Exporting VirtualBox machines into an OVF appliance involves the following steps:

1. As with importing, first call IVirtualBox::createAppliance() to create an empty IAppliance object.

2. For each machine you would like to export, call IMachine::exportTo() with the IAppliance object you just created. Each such call creates one instance of IVirtualSystemDescription inside the appliance.

3. If desired, call IVirtualSystemDescription::setFinalValues() for each virtual system (machine) to override the suggestions made by the IMachine::exportTo() routine.

4. Finally, call write() with a path specification to have the OVF file written.

6.3.1 Attributes

6.3.1.1 path (read-only)

```
wstring IAppliance::path
```

Path to the main file of the OVF appliance, which is either the `.ovf` or the `.ova` file passed to read() (for import) or write() (for export). This attribute is empty until one of these methods has been called.

6.3.1.2 disks (read-only)

```
wstring IAppliance::disks[]
```

Array of virtual disk definitions. One such description exists for each disk definition in the OVF; each string array item represents one such piece of disk information, with the information fields separated by tab (\\t) characters.

The caller should be prepared for additional fields being appended to this string in future versions of VirtualBox and therefore check for the number of tabs in the strings returned.

In the current version, the following eight fields are returned per string in the array:

1. Disk ID (unique string identifier given to disk)

2. Capacity (unsigned integer indicating the maximum capacity of the disk)

3. Populated size (optional unsigned integer indicating the current size of the disk; can be approximate; -1 if unspecified)

4. Format (string identifying the disk format, typically "http://www.vmware.com/specifications/vmdk.html#sparse")

5. Reference (where to find the disk image, typically a file name; if empty, then the disk should be created on import)

6. Image size (optional unsigned integer indicating the size of the image, which need not necessarily be the same as the values specified above, since the image may be compressed or sparse; -1 if not specified)

7. Chunk size (optional unsigned integer if the image is split into chunks; presently unsupported and always -1)

8. Compression (optional string equaling "gzip" if the image is gzip-compressed)

6.3.1.3 virtualSystemDescriptions (read-only)

```
IVirtualSystemDescription IAppliance::virtualSystemDescriptions[]
```

Array of virtual system descriptions. One such description is created for each virtual system (machine) found in the OVF. This array is empty until either interpret() (for import) or IMachine::exportTo() (for export) has been called.

6.3.1.4 machines (read-only)

```
wstring IAppliance::machines[]
```

Contains the UUIDs of the machines created from the information in this appliances. This is only relevant for the import case, and will only contain data after a call to importMachines() succeeded.

6.3.1.5 certificate (read-only)

```
ICertificate IAppliance::certificate
```

The X.509 signing certificate, if the imported OVF was signed, `null` if not signed. This is available after calling read().

6.3.2 addPasswords

```
void IAppliance::addPasswords(
            [in] wstring identifiers[],
            [in] wstring passwords[])
```

identifiers List of identifiers.

passwords List of matching passwords.

Adds a list of passwords required to import or export encrypted virtual machines.

6.3.3 createVFSExplorer

```
IVFSExplorer IAppliance::createVFSExplorer(
            [in] wstring URI)
```

URI The URI describing the file system to use.

Returns a IVFSExplorer object for the given URI.

6.3.4 createVirtualSystemDescriptions

```
unsigned long IAppliance::createVirtualSystemDescriptions(
            [in] unsigned long requested)
```

requested Requested number of new virtual system description objects

Creates a number of IVirtualSystemDescription objects and store them in the virtualSystemDescriptions[] array.

6.3.5 getMediumIdsForPasswordId

```
uuid[] IAppliance::getMediumIdsForPasswordId(
            [in] wstring passwordId)
```

passwordId The password identifier to get the medium identifiers for.

Returns a list of medium identifiers which use the given password identifier.

6.3.6 getPasswordIds

```
wstring[] IAppliance::getPasswordIds()
```

Returns a list of password identifiers which must be supplied to import or export encrypted virtual machines.

6.3.7 getWarnings

```
wstring[] IAppliance::getWarnings()
```

Returns textual warnings which occurred during execution of interpret().

6.3.8 importMachines

```
IProgress IAppliance::importMachines(
            [in] ImportOptions options[])
```

options Options for the importing operation.

Imports the appliance into VirtualBox by creating instances of IMachine and other interfaces that match the information contained in the appliance as closely as possible, as represented by the import instructions in the virtualSystemDescriptions[] array.

Calling this method is the final step of importing an appliance into VirtualBox; see IAppliance for an overview.

Since importing the appliance will most probably involve copying and converting disk images, which can take a long time, this method operates asynchronously and returns an IProgress object to allow the caller to monitor the progress.

After the import succeeded, the UUIDs of the IMachine instances created can be retrieved from the machines[] array attribute.

6.3.9 interpret

```
void IAppliance::interpret()
```

Interprets the OVF data that was read when the appliance was constructed. After calling this method, one can inspect the virtualSystemDescriptions[] array attribute, which will then contain one IVirtualSystemDescription for each virtual machine found in the appliance.

Calling this method is the second step of importing an appliance into VirtualBox; see IAppliance for an overview.

After calling this method, one should call getWarnings() to find out if problems were encountered during the processing which might later lead to errors.

6.3.10 read

```
IProgress IAppliance::read(
            [in] wstring file)
```

file Name of appliance file to open (either with an `.ovf` or `.ova` extension, depending on whether the appliance is distributed as a set of files or as a single file, respectively).

Reads an OVF file into the appliance object.

This method succeeds if the OVF is syntactically valid and, by itself, without errors. The mere fact that this method returns successfully does not mean that VirtualBox supports all features requested by the appliance; this can only be examined after a call to interpret().

6.3.11 write

```
IProgress IAppliance::write(
           [in] wstring format,
           [in] ExportOptions options[],
           [in] wstring path)
```

format Output format, as a string. Currently supported formats are “ovf-0.9”, “ovf-1.0”, “ovf-2.0” and “opc-1.0”; future versions of VirtualBox may support additional formats. The “opc-1.0” format is for creating tarballs for the Oracle Public Cloud.

options Options for the exporting operation.

path Name of appliance file to create. There are certain restrictions with regard to the file name suffix. If the format parameter is “opc-1.0” a `.tar.gz` suffix is required. Otherwise the suffix must either be `.ovf` or `.ova`, depending on whether the appliance is distributed as a set of files or as a single file, respectively.

Writes the contents of the appliance exports into a new OVF file.

Calling this method is the final step of exporting an appliance from VirtualBox; see IAppliance for an overview.

Since exporting the appliance will most probably involve copying and converting disk images, which can take a long time, this method operates asynchronously and returns an IProgress object to allow the caller to monitor the progress.

6.4 IAudioAdapter

The IAudioAdapter interface represents the virtual audio adapter of the virtual machine. Used in IAudioSettings::adapter.

6.4.1 Attributes

6.4.1.1 enabled (read/write)

```
boolean IAudioAdapter::enabled
```

Flag whether the audio adapter is present in the guest system. If disabled, the virtual guest hardware will not contain any audio adapter. Can only be changed when the VM is not running.

6.4.1.2 enabledIn (read/write)

```
boolean IAudioAdapter::enabledIn
```

Flag whether the audio adapter is enabled for audio input. Only relevant if the adapter is enabled.

6.4.1.3 enabledOut (read/write)

```
boolean IAudioAdapter::enabledOut
```

Flag whether the audio adapter is enabled for audio output. Only relevant if the adapter is enabled.

6.4.1.4 audioController (read/write)

```
AudioControllerType IAudioAdapter::audioController
```

The emulated audio controller.

6.4.1.5 audioCodec (read/write)

`AudioCodecType IAudioAdapter::audioCodec`

The exact variant of audio codec hardware presented to the guest. For HDA and SB16, only one variant is available, but for AC'97, there are several.

6.4.1.6 audioDriver (read/write)

`AudioDriverType IAudioAdapter::audioDriver`

Audio driver the adapter is connected to. This setting can only be changed when the VM is not running.

6.4.1.7 propertiesList (read-only)

`wstring IAudioAdapter::propertiesList[]`

Array of names of tunable properties, which can be supported by audio driver.

6.4.2 getProperty

```
wstring IAudioAdapter::getProperty(
            [in] wstring key)
```

key Name of the key to get.

Returns an audio specific property string.
If the requested data `key` does not exist, this function will succeed and return an empty string in the `value` argument.

6.4.3 setProperty

```
void IAudioAdapter::setProperty(
            [in] wstring key,
            [in] wstring value)
```

key Name of the key to set.

value Value to assign to the key.

Sets an audio specific property string.
If you pass `null` or empty string as a key `value`, the given `key` will be deleted.

6.5 IAudioAdapterChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a property of the audio adapter changes. Interested callees should use IAudioAdapter methods and attributes to find out what has changed.

6.5.1 Attributes

6.5.1.1 audioAdapter (read-only)

```
IAudioAdapter IAudioAdapterChangedEvent::audioAdapter
```

Audio adapter that is subject to change.

6.6 IAudioSettings

The IAudioSettings interface represents the audio settings for a virtual machine.

6.6.1 Attributes

6.6.1.1 adapter (read-only)

```
IAudioAdapter IAudioSettings::adapter
```

Associated audio adapter, always present.

6.6.2 getHostAudioDevice

```
IHostAudioDevice IAudioSettings::getHostAudioDevice(
            [in] AudioDirection usage)
```

usage Usage to retrieve audio device for.

Returns the machine's current host audio device for the specified usage.
If this method fails, the following error codes may be reported:

- E_NOTIMPL: This method is not implemented yet.

6.6.3 setHostAudioDevice

```
void IAudioSettings::setHostAudioDevice(
            [in] IHostAudioDevice device,
            [in] AudioDirection usage)
```

device Sets the host audio device for the specified usage.

usage Usage to set audio device for.

Sets the machine's current host audio device for the specified usage.
If this method fails, the following error codes may be reported:

- E_NOTIMPL: This method is not implemented yet.

6.7 IBandwidthControl

Controls the bandwidth groups of one machine used to cap I/O done by a VM. This includes network and disk I/O.

6.7.1 Attributes

6.7.1.1 numGroups (read-only)

```
unsigned long IBandwidthControl::numGroups
```

The current number of existing bandwidth groups managed.

6.7.2 createBandwidthGroup

```
void IBandwidthControl::createBandwidthGroup(
            [in] wstring name,
            [in] BandwidthGroupType type,
            [in] long long maxBytesPerSec)
```

name Name of the bandwidth group.

type The type of the bandwidth group (network or disk).

maxBytesPerSec The maximum number of bytes which can be transferred by all entities attached to this group during one second.

Creates a new bandwidth group.

6.7.3 deleteBandwidthGroup

```
void IBandwidthControl::deleteBandwidthGroup(
            [in] wstring name)
```

name Name of the bandwidth group to delete.

Deletes a new bandwidth group.

6.7.4 getAllBandwidthGroups

```
IBandwidthGroup[] IBandwidthControl::getAllBandwidthGroups()
```

Get all managed bandwidth groups.

6.7.5 getBandwidthGroup

```
IBandwidthGroup IBandwidthControl::getBandwidthGroup(
            [in] wstring name)
```

name Name of the bandwidth group to get.

Get a bandwidth group by name.

6.8 IBandwidthGroup

Represents one bandwidth group.

6.8.1 Attributes

6.8.1.1 name (read-only)

```
wstring IBandwidthGroup::name
```

Name of the group.

6.8.1.2 type (read-only)

```
BandwidthGroupType IBandwidthGroup::type
```

Type of the group.

6.8.1.3 reference (read-only)

```
unsigned long IBandwidthGroup::reference
```

How many devices/medium attachments use this group.

6.8.1.4 maxBytesPerSec (read/write)

```
long long IBandwidthGroup::maxBytesPerSec
```

The maximum number of bytes which can be transferred by all entities attached to this group during one second.

6.9 IBandwidthGroupChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when one of the bandwidth groups changed

6.9.1 Attributes

6.9.1.1 bandwidthGroup (read-only)

```
IBandwidthGroup IBandwidthGroupChangedEvent::bandwidthGroup
```

The changed bandwidth group.

6.10 IBooleanFormValue (IFormValue)

> **Note:** This interface extends IFormValue and therefore supports all its methods and attributes as well.

6.10.1 getSelected

```
boolean IBooleanFormValue::getSelected()
```

6.10.2 setSelected

```
IProgress IBooleanFormValue::setSelected(
            [in] boolean selected)
```

selected

6.11 ICPUChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a CPU changes.

6.11.1 Attributes

6.11.1.1 CPU (read-only)

```
unsigned long ICPUChangedEvent::CPU
```

The CPU which changed.

6.11.1.2 add (read-only)

```
boolean ICPUChangedEvent::add
```

Flag whether the CPU was added or removed.

6.12 ICPUExecutionCapChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the CPU execution cap changes.

6.12.1 Attributes

6.12.1.1 executionCap (read-only)

```
unsigned long ICPUExecutionCapChangedEvent::executionCap
```

The new CPU execution cap value. (1-100)

6.13 ICPUProfile

CPU profile. Immutable.

6.13.1 Attributes

6.13.1.1 name (read-only)

```
wstring ICPUProfile::name
```

The name.

6.13.1.2 fullName (read-only)

```
wstring ICPUProfile::fullName
```

The full name.

6.13.1.3 architecture (read-only)

```
CPUArchitecture ICPUProfile::architecture
```

The CPU architecture.

6.14 ICanShowWindowEvent (IVetoEvent)

> **Note:** This interface extends IVetoEvent and therefore supports all its methods and attributes as well.

Notification when a call to IMachine::canShowConsoleWindow() is made by a front-end to check if a subsequent call to IMachine::showConsoleWindow() can succeed.

The callee should give an answer appropriate to the current machine state using event veto. This answer must remain valid at least until the next machine state change.

6.14.1 Attributes

6.14.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean ICanShowWindowEvent::midlDoesNotLikeEmptyInterfaces
```

6.15 ICertificate

X.509 certificate details.

6.15.1 Attributes

6.15.1.1 versionNumber (read-only)

```
CertificateVersion ICertificate::versionNumber
```

Certificate version number.

6.15.1.2 serialNumber (read-only)

```
wstring ICertificate::serialNumber
```

Certificate serial number.

6.15.1.3 signatureAlgorithmOID (read-only)

```
wstring ICertificate::signatureAlgorithmOID
```

The dotted OID of the signature algorithm.

6.15.1.4 signatureAlgorithmName (read-only)

```
wstring ICertificate::signatureAlgorithmName
```

The signature algorithm name if known (if known).

6.15.1.5 issuerName (read-only)

```
wstring ICertificate::issuerName[]
```

Issuer name. Each member of the array is on the format COMPONENT=NAME, e.g. “C=DE”, “ST=Example”, “L=For Instance”, “O=Beispiel GmbH”, “CN=beispiel.example.org”.

6.15.1.6 subjectName (read-only)

```
wstring ICertificate::subjectName[]
```

Subject name. Same format as issuerName.

6.15.1.7 friendlyName (read-only)

```
wstring ICertificate::friendlyName
```

Friendly subject name or similar.

6.15.1.8 validityPeriodNotBefore (read-only)

```
wstring ICertificate::validityPeriodNotBefore
```

Certificate not valid before ISO timestamp.

6.15.1.9 validityPeriodNotAfter (read-only)

```
wstring ICertificate::validityPeriodNotAfter
```

Certificate not valid after ISO timestamp.

6.15.1.10 publicKeyAlgorithmOID (read-only)

```
wstring ICertificate::publicKeyAlgorithmOID
```

The dotted OID of the public key algorithm.

6.15.1.11 publicKeyAlgorithm (read-only)

```
wstring ICertificate::publicKeyAlgorithm
```

The public key algorithm name (if known).

6.15.1.12 subjectPublicKey (read-only)

```
octet ICertificate::subjectPublicKey[]
```

The raw public key bytes.

6.15.1.13 issuerUniqueIdentifier (read-only)

```
wstring ICertificate::issuerUniqueIdentifier
```

Unique identifier of the issuer (empty string if not present).

6.15.1.14 subjectUniqueIdentifier (read-only)

```
wstring ICertificate::subjectUniqueIdentifier
```

Unique identifier of this certificate (empty string if not present).

6.15.1.15 certificateAuthority (read-only)

```
boolean ICertificate::certificateAuthority
```

Whether this certificate is a certificate authority. Will return E_FAIL if this attribute is not present.

6.15.1.16 keyUsage (read-only)

```
unsigned long ICertificate::keyUsage
```

Key usage mask. Will return 0 if not present.

6.15.1.17 extendedKeyUsage (read-only)

```
wstring ICertificate::extendedKeyUsage[]
```

Array of dotted extended key usage OIDs. Empty array if not present.

6.15.1.18 rawCertData (read-only)

```
octet ICertificate::rawCertData[]
```

The raw certificate bytes.

6.15.1.19 selfSigned (read-only)

```
boolean ICertificate::selfSigned
```

Set if self signed certificate.

6.15.1.20 trusted (read-only)

```
boolean ICertificate::trusted
```

Set if the certificate is trusted (by the parent object).

6.15.1.21 expired (read-only)

```
boolean ICertificate::expired
```

Set if the certificate has expired (relevant to the parent object)/

6.15.2 isCurrentlyExpired

```
boolean ICertificate::isCurrentlyExpired()
```

Tests if the certificate has expired at the present time according to the X.509 validity of the certificate.

6.15.3 queryInfo

```
wstring ICertificate::queryInfo(
           [in] long what)
```

what

Way to extend the interface.

6.16 IChoiceFormValue (IFormValue)

> **Note:** This interface extends IFormValue and therefore supports all its methods and attributes as well.

6.16.1 Attributes

6.16.1.1 values (read-only)

```
wstring IChoiceFormValue::values[]
```

6.16.2 getSelectedIndex

```
long IChoiceFormValue::getSelectedIndex()
```

6.16.3 setSelectedIndex

```
IProgress IChoiceFormValue::setSelectedIndex(
            [in] long index)
```

index

6.17 IClipboardErrorEvent (IClipboardEvent)

> **Note:** This interface extends IClipboardEvent and therefore supports all its methods and attributes as well.

Notification when a clipboard error occurred.

6.17.1 Attributes

6.17.1.1 msg (read-only)

```
wstring IClipboardErrorEvent::msg
```

Error message in human readable format.

6.17.1.2 rcError (read-only)

```
long IClipboardErrorEvent::rcError
```

IPRT-style error code.

6.18 IClipboardEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Abstract base interface for clipboard events.

6.18.1 Attributes

6.18.1.1 id (read-only)

```
wstring IClipboardEvent::id
```

Name of the clipboard this event belongs to.
For Windows or macOS hosts / guests, this string is empty. For X11-based hosts / guests, this can hold either “PRIMARY” or “SECONDARY”.

6.19 IClipboardFileTransferModeChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the shared clipboard file transfer mode changes.

6.19.1 Attributes

6.19.1.1 enabled (read-only)

```
boolean IClipboardFileTransferModeChangedEvent::enabled
```

Whether file transfers are allowed or not.

6.20 IClipboardModeChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the shared clipboard mode changes.

6.20.1 Attributes

6.20.1.1 clipboardMode (read-only)

```
ClipboardMode IClipboardModeChangedEvent::clipboardMode
```

The new clipboard mode.

6.21 ICloudClient

6.21.1 Attributes

6.21.1.1 cloudMachineList (read-only)

```
ICloudMachine ICloudClient::cloudMachineList[]
```

See readCloudMachineList().

6.21.1.2 cloudMachineStubList (read-only)

```
ICloudMachine ICloudClient::cloudMachineStubList[]
```

See readCloudMachineStubList().

6.21.2 addCloudMachine

```
IProgress ICloudClient::addCloudMachine(
            [in] wstring instanceId,
            [out] ICloudMachine machine)
```

instanceId ID of an instance to be added as a cloud machine.

machine Object that represents the newly registered cloud machine.

Adopt a running instance and register it as cloud machine. This is kinda like adding a local .vbox file as a local VM.

6.21.3 cloneInstance

```
IProgress ICloudClient::cloneInstance(
            [in] wstring uid,
            [in] wstring newName,
            [out] ICloudMachine clone)
```

uid The id of instance in the Cloud that should be cloned. Prerequisites: The instance must be in the list of VirtualBox cloud machines. VirtualBox operates only with those machines which are already known.

newName Name for a cloud machine.

clone Object that represents a cloned cloud machine.

Clone cloud virtual machine.

6.21.4 createCloudMachine

```
IProgress ICloudClient::createCloudMachine(
            [in] IVirtualSystemDescription description,
            [out] ICloudMachine machine)
```

description Virtual system description object which describes the machine and all required parameters.

machine Object that represents the newly created cloud machine.

This is transitional method that combines launchVM() and addCloudMachine().

6.21.5 createImage

```
IProgress ICloudClient::createImage(
            [in] wstring parameters[])
```

parameters Each parameter in the array must be in the form “name=value”.

Create an image in the Cloud.

6.21.6 deleteImage

```
IProgress ICloudClient::deleteImage(
            [in] wstring uid)
```

uid The id of image in the Cloud.

Delete an existing image with passed id from the Cloud.

6.21.7 exportImage

```
IProgress ICloudClient::exportImage(
            [in] IMedium image,
            [in] wstring parameters[])
```

image Reference to the existing VBox image.

parameters Each parameter in the array must be in the form “name=value”.

Export an existing VBox image in the Cloud.

6.21.8 exportVM

```
void ICloudClient::exportVM(
            [in] IVirtualSystemDescription description,
            [in] IProgress progress)
```

description Virtual system description object which describes the machine and all required parameters.

progress Progress object to track the operation completion.

Export local VM into the cloud, creating a custom image.

6.21.9 getCloudMachine

```
ICloudMachine ICloudClient::getCloudMachine(
            [in] uuid id)
```

id UUID of a cloud machine.

Create an object that represents a cloud machine with the specified UUID. Note that the operation is synchronous. The returned machine is initially in inaccessible state and requires a refresh to get its data from the cloud.

6.21.10 getExportDescriptionForm

```
IProgress ICloudClient::getExportDescriptionForm(
            [in] IVirtualSystemDescription description,
            [out] IVirtualSystemDescriptionForm form)
```

description Virtual system description to be edited.

form An IForm instance for editing the virtual system description.

Returns a form for editing the virtual system description for exporting a local VM into a cloud custom image.

6.21.11 getImageInfo

```
IProgress ICloudClient::getImageInfo(
            [in] wstring uid,
            [out] IStringArray infoArray)
```

uid The id of image in the Cloud.

infoArray An array where the image settings or properties is returned. Each parameter in the array must be in the form “name=value”.

Returns the information about an image in the Cloud.

6.21.12 getImportDescriptionForm

```
IProgress ICloudClient::getImportDescriptionForm(
            [in] IVirtualSystemDescription description,
            [out] IVirtualSystemDescriptionForm form)
```

description Virtual system description to be edited.

form An IForm instance for editing the virtual system description.

Returns a form for editing the virtual system description for import from cloud.

6.21.13 getInstanceInfo

```
IProgress ICloudClient::getInstanceInfo(
            [in] wstring uid,
            [in] IVirtualSystemDescription description)
```

uid The id of instance in the Cloud.

description VirtualSystemDescription object which is describing a machine

Returns the information about an instance in the Cloud.

6.21.14 getLaunchDescriptionForm

```
IProgress ICloudClient::getLaunchDescriptionForm(
            [in] IVirtualSystemDescription description,
            [out] IVirtualSystemDescriptionForm form)
```

description Virtual system description to be edited.

form An IForm instance for editing the virtual system description.

6.21.15 getMetricTypeByName

```
MetricType ICloudClient::getMetricTypeByName(
            [in] wstring metricName)
```

metricName Metric name (see ICloudMachine::listMetricNames())

Tries to find an appropriate metric type for the passed metric name.

6.21.16 getSubnetSelectionForm

```
IProgress ICloudClient::getSubnetSelectionForm(
            [in] IVirtualSystemDescription description,
            [out] IVirtualSystemDescriptionForm form)
```

description Virtual system description to be edited.

form An IForm instance for editing the virtual system description.

6.21.17 getVnicInfo

```
IProgress ICloudClient::getVnicInfo(
            [in] wstring uid,
            [out] IStringArray infoArray)
```

uid The id of vnic in the Cloud.

infoArray An array where the Vnic settings/properties is returned. Each parameter in the array must be in the form “name=value”.

Returns the information about Vnic in the Cloud.

6.21.18 importImage

```
IProgress ICloudClient::importImage(
            [in] wstring uid,
            [in] wstring parameters[])
```

uid the id of image in the Cloud.

parameters Each parameter in the array must be in the form “name=value”.

Import an existing image in the Cloud to the local host.

6.21.19 importInstance

```
void ICloudClient::importInstance(
            [in] IVirtualSystemDescription description,
            [in] IProgress progress)
```

description VirtualSystemDescription object which is describing a machine and all required parameters.

progress Progress object to track the operation completion.

Import an existing cloud instance to the local host. All needed parameters are passed in the description (VSD).

6.21.20 launchVM

```
IProgress ICloudClient::launchVM(
            [in] IVirtualSystemDescription description)
```

description Virtual system description object which describes the machine and all required parameters.

6.21.21 listBootVolumes

```
IProgress ICloudClient::listBootVolumes(
            [out] IStringArray returnNames,
            [out] IStringArray returnIds)
```

returnNames Boot volume names.

returnIds Boot volume ids.

Returns the list of boot volumes in the Cloud.

6.21.22 listImages

```
IProgress ICloudClient::listImages(
           [in] CloudImageState imageState[],
           [out] IStringArray returnNames,
           [out] IStringArray returnIds)
```

imageState State of each image.

returnNames Images names.

returnIds Images ids.

Returns the list of the images in the Cloud.

6.21.23 listInstances

```
IProgress ICloudClient::listInstances(
           [in] CloudMachineState machineState[],
           [out] IStringArray returnNames,
           [out] IStringArray returnIds)
```

machineState State of each VM.

returnNames VM names.

returnIds VM ids.

Returns the list of the instances in the Cloud.

6.21.24 listSourceBootVolumes

```
IProgress ICloudClient::listSourceBootVolumes(
           [out] IStringArray returnNames,
           [out] IStringArray returnIds)
```

returnNames Boot volume names.

returnIds Boot volume ids.

Returns the list of boot volumes in the cloud that can be added/adopted as VirtualBox cloud machines.

6.21.25 listSourceInstances

```
IProgress ICloudClient::listSourceInstances(
           [out] IStringArray returnNames,
           [out] IStringArray returnIds)
```

returnNames Instance names.

returnIds Instance idss.

Returns the list of instances in the cloud that can be added/adopted as VirtualBox cloud machines.

6.21.26 listVnicAttachments

```
IProgress ICloudClient::listVnicAttachments(
            [in] wstring parameters[],
            [out] IStringArray returnVnicAttachmentIds,
            [out] IStringArray returnVnicIds)
```

parameters Each parameter in the array must be in the form “name=value”.

returnVnicAttachmentIds VM ids.

returnVnicIds VM ids.

Returns the list of the Vnic attachements in the Cloud.

6.21.27 pauseInstance

```
IProgress ICloudClient::pauseInstance(
            [in] wstring uid)
```

uid The id of instance in the Cloud.

Pause an existing instance with passed id.

6.21.28 readCloudMachineList

```
IProgress ICloudClient::readCloudMachineList()
```

Make the list of cloud machines available via cloudMachineList[] attribute.

6.21.29 readCloudMachineStubList

```
IProgress ICloudClient::readCloudMachineStubList()
```

Make the list of cloud machine stubs available via cloudMachineStubList[] attribute. Like with getCloudMachine(), the returned machines are initially inaccessible and require a refresh to get their data from the cloud.

6.21.30 resetInstance

```
IProgress ICloudClient::resetInstance(
            [in] wstring uid)
```

uid The id of instance in the Cloud.

Reset an existing instance with passed id.

6.21.31 setupCloudNetworkEnvironment

```
IProgress ICloudClient::setupCloudNetworkEnvironment(
            [in] wstring tunnelNetworkName,
            [in] wstring tunnelNetworkRange,
            [in] wstring gatewayOsName,
            [in] wstring gatewayOsVersion,
            [in] wstring gatewayShape,
            [out] ICloudNetworkEnvironmentInfo networkEnvironmentInfo)
```

tunnelNetworkName The name of tunnelling network to be created in the Cloud. If this parameter is empty the default value “VirtualBox Tunneling Network” is assumed.

tunnelNetworkRange The IP address range of tunnelling network to be created in the Cloud. If this parameter is empty the default value “10.0.0.0/16” is assumed.

gatewayOsName The name of the operating system to be used for cloud gateway instances. The default value is “Oracle Linux”.

gatewayOsVersion The version of the operating system to be used for cloud gateway instances. The default value is “7.8”.

gatewayShape The shape of cloud gateway instance. The default value is “VM.Standard2.1”.

networkEnvironmentInfo Information about the created network environment.

6.21.32 startCloudNetworkGateway

```
IProgress ICloudClient::startCloudNetworkGateway(
            [in] ICloudNetwork network,
            [in] wstring sshPublicKey,
            [out] ICloudNetworkGatewayInfo gatewayInfo)
```

network The id of image in the Cloud.

sshPublicKey The id of image in the Cloud.

gatewayInfo Information about the started gateway.

6.21.33 startInstance

```
IProgress ICloudClient::startInstance(
            [in] wstring uid)
```

uid The id of instance in the Cloud.

Start an existing instance with passed id.

6.21.34 terminateInstance

```
IProgress ICloudClient::terminateInstance(
            [in] wstring uid)
```

uid the id of instance in the Cloud.

Terminate an existing instance with passed id.

6.22 ICloudMachine

Virtual virtual machine (sic) in the cloud.
Reading object attributes returns cached values, use refresh() to refresh them.

6.22.1 Attributes

6.22.1.1 id (read-only)

```
uuid ICloudMachine::id
```

UUID of the cloud machine within VirtualBox.

6.22.1.2 cloudId (read-only)

`wstring ICloudMachine::cloudId`

UUID of the cloud machine within Cloud

6.22.1.3 accessible (read-only)

`boolean ICloudMachine::accessible`

Whether this virtual machine is currently accessible or not. TBD...

6.22.1.4 accessError (read-only)

`IVirtualBoxErrorInfo ICloudMachine::accessError`

Error information describing the reason of machine inaccessibility.
Reading this property is only valid after the last call to accessible returned `false` (i.e. the machine is currently inaccessible). Otherwise, a `null` IVirtualBoxErrorInfo object will be returned.

6.22.1.5 name (read-only)

`wstring ICloudMachine::name`

Convenience shortcut to retrieve the name of the cloud machine. The name is part of the machine settings that are hidden behind the settings form (see getSettingsForm()).

6.22.1.6 OSTypeId (read-only)

`wstring ICloudMachine::OSTypeId`

Convenience shortcut to retrieve the OS Type id of the cloud machine. It is part of the machine settings that are hidden behind the settings form (see getSettingsForm()).

6.22.1.7 state (read-only)

`CloudMachineState ICloudMachine::state`

Machine state.

6.22.1.8 consoleConnectionFingerprint (read-only)

`wstring ICloudMachine::consoleConnectionFingerprint`

The fingerprint of the ssh key that is authorized to access the machine's console connection.

6.22.1.9 serialConsoleCommand (read-only)

`wstring ICloudMachine::serialConsoleCommand`

The shell command to establish ssh connection to the cloud machine serial console.

6.22.1.10 serialConsoleCommandWindows (read-only)

`wstring ICloudMachine::serialConsoleCommandWindows`

The PowerShell command to establish ssh connection to the cloud machine serial console using PuTTY's plink.

6.22.1.11 VNCConsoleCommand (read-only)

```
wstring ICloudMachine::VNCConsoleCommand
```

The shell command to establish ssh port forwarding for the VNC connection to the cloud machine console.

6.22.1.12 VNCConsoleCommandWindows (read-only)

```
wstring ICloudMachine::VNCConsoleCommandWindows
```

The PowerShell command to establish ssh port forwarding for the VNC connection to the cloud machine console using PuTTY's plink.

6.22.2 createConsoleConnection

```
IProgress ICloudMachine::createConsoleConnection(
            [in] wstring sshPublicKey)
```

sshPublicKey SSH public key authorized to connect to the console.

6.22.3 deleteConsoleConnection

```
IProgress ICloudMachine::deleteConsoleConnection()
```

6.22.4 enumerateMetricData

```
IProgress ICloudMachine::enumerateMetricData(
            [in] MetricType metricType,
            [in] unsigned long pointsNumber,
            [out] IStringArray values,
            [out] IStringArray timestamps,
            [out] IStringArray unit)
```

metricType The type of the requested metric. Standard usage to get an appropriate MetricType: - call listMetricNames() function; returns the types in string representation. - call ICloudClient::getMetricTypeByName() function; converts the string representation into MetricType.

pointsNumber History metric point numbers start at 1 and can go backwards as long as history exists. Points are counted from the current time to the past. If user only wants the last actual value he passes the value "1". If 2 values need to be returned (last and second to last), the user passes "2", etc. and etc.

values The values of the metric returned.

timestamps The timestamps of the metric returned. The array entries match the corresponding entries in the `array`. Time format is represented in RFC2822.

unit The measurement unit as "byte", "percentage" and etc... The array entries match the corresponding entries in the `array`.

Returns the interested metric history points.

6.22.5 getConsoleHistory

```
IProgress ICloudMachine::getConsoleHistory(
            [out] IDataStream stream)
```

stream Data stream object for reading the console history. For now we are abusing/repurposing this interface from the media conversion API to avoid marshalling a huge string through xpcom.

Get the backlog of the machine's console. If you have ever seen console teletypes, this is like looking at the last few meters of the paper it spewed.

6.22.6 getDetailsForm

```
IForm ICloudMachine::getDetailsForm()
```

Obtain a form with the current settings for this cloud machine. The form is not editable.

6.22.7 getSettingsForm

```
IProgress ICloudMachine::getSettingsForm(
            [out] IForm form)
```

form A form with the cloud machine settings.

Obtain a form with settings for this cloud machine. The form is editable.

6.22.8 listMetricNames

```
IProgress ICloudMachine::listMetricNames(
            [out] IStringArray metricNames)
```

metricNames List of metrics names. May be empty if there are no metrics available on the instance or if the corresponding service is not running on the instance.

Returns the metrics available for the instances in the Cloud. In instance, CPU utilization, memory utilization, disk read I/O. Not all instances can have such metrics because a special service must be running on the instance. Converting a string representation to a MetricType is done using ICloudClient::getMetricTypeByName().

6.22.9 powerDown

```
IProgress ICloudMachine::powerDown()
```

Initiates the power down procedure to stop the virtual machine execution.

The completion of the power down procedure is tracked using the returned IProgress object. After the operation is complete, the machine will go to the PoweredOff state.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine must be Running, to be powered down.

6.22.10 powerUp

```
IProgress ICloudMachine::powerUp()
```

Start cloud virtual machine execution.

6.22.11 reboot

`IProgress ICloudMachine::reboot()`

Reboot cloud virtual machine.

6.22.12 refresh

`IProgress ICloudMachine::refresh()`

Refresh information by reading it from the cloud.

6.22.13 remove

`IProgress ICloudMachine::remove()`

Unregister this cloud machine and delete all its cloud artifacts.

6.22.14 reset

`IProgress ICloudMachine::reset()`

Forcing reset or hard reset of a cloud virtual machine.

6.22.15 shutdown

`IProgress ICloudMachine::shutdown()`

Shutdown cloud virtual machine.

6.22.16 terminate

`IProgress ICloudMachine::terminate()`

Terminate cloud virtual machine.

6.22.17 unregister

`IProgress ICloudMachine::unregister()`

Unregister this cloud machine, but leave the cloud artifacts intact.

6.23 ICloudNetwork

6.23.1 Attributes

6.23.1.1 networkName (read/write)

`wstring ICloudNetwork::networkName`

TBD: User-friendly, descriptive name of cloud subnet. For example, domain names of subnet and vcn, separated by dot.

6.23.1.2 enabled (read/write)

`boolean ICloudNetwork::enabled`

6.23.1.3 provider (read/write)

```
wstring ICloudNetwork::provider
```

Cloud provider short name.

6.23.1.4 profile (read/write)

```
wstring ICloudNetwork::profile
```

Cloud profile name.

6.23.1.5 networkId (read/write)

```
wstring ICloudNetwork::networkId
```

Cloud network id.

6.24 ICloudNetworkEnvironmentInfo

6.24.1 Attributes

6.24.1.1 tunnelNetworkId (read-only)

```
wstring ICloudNetworkEnvironmentInfo::tunnelNetworkId
```

6.25 ICloudNetworkGatewayInfo

6.25.1 Attributes

6.25.1.1 publicIP (read-only)

```
wstring ICloudNetworkGatewayInfo::publicIP
```

6.25.1.2 secondaryPublicIP (read-only)

```
wstring ICloudNetworkGatewayInfo::secondaryPublicIP
```

6.25.1.3 macAddress (read-only)

```
wstring ICloudNetworkGatewayInfo::macAddress
```

6.25.1.4 instanceId (read-only)

```
wstring ICloudNetworkGatewayInfo::instanceId
```

6.26 ICloudProfile

6.26.1 Attributes

6.26.1.1 name (read/write)

```
wstring ICloudProfile::name
```

Returns the profile name.

6.26.1.2 providerId (read-only)

```
uuid ICloudProfile::providerId
```

Returns provider identifier tied with this profile.

6.26.2 createCloudClient

```
ICloudClient ICloudProfile::createCloudClient()
```

Creates a cloud client for this cloud profile.

6.26.3 getProperties

```
wstring[] ICloudProfile::getProperties(
            [in] wstring names,
            [out] wstring returnNames[])
```

names Names of properties to get.

returnNames Names of returned properties.

Returns values for a group of properties in one call.
The names of the properties to get are specified using the `names` argument which is a list of comma-separated property names or an empty string if all properties are to be returned.

> **Note:** Currently the value of this argument is ignored and the method always returns all existing properties.

The method returns two arrays, the array of property names corresponding to the `names` argument and the current values of these properties. Both arrays have the same number of elements with each element at the given index in the first array corresponds to an element at the same index in the second array.

6.26.4 getProperty

```
wstring ICloudProfile::getProperty(
            [in] wstring name)
```

name Name of the property to get.

Returns the value of the cloud profile property with the given name.
If the requested data `name` does not exist, this function will succeed and return an empty string in the `value` argument.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG: name` is `null` or empty.

6.26.5 remove

```
void ICloudProfile::remove()
```

Deletes a profile.

6.26.6 setProperties

```
void ICloudProfile::setProperties(
            [in] wstring names[],
            [in] wstring values[])
```

names Names of properties.

values Values of set properties.

Updates profile, changing/adding/removing properties.

The names of the properties to set are passed in the `names` array along with the new values for them in the `values` array. Both arrays have the same number of elements with each element at the given index in the first array corresponding to an element at the same index in the second array.

If there is at least one property name in `names` that is not valid, the method will fail before changing the values of any other properties from the `names` array.

Using this method over setProperty() is preferred if you need to set several properties at once since it is more efficient.

Setting the property value to `null` or an empty string is equivalent to deleting the existing value.

6.26.7 setProperty

```
void ICloudProfile::setProperty(
            [in] wstring name,
            [in] wstring value)
```

name Name of the property to set.

value Property value to set.

Sets the value of the cloud profile property with the given name.

Setting the property value to `null` or an empty string is equivalent to deleting the existing value.

If this method fails, the following error codes may be reported:

- `E_INVALIDARG: name is null or empty.`

6.27 ICloudProfileChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

6.27.1 Attributes

6.27.1.1 providerId (read-only)

```
uuid ICloudProfileChangedEvent::providerId
```

6.27.1.2 name (read-only)

```
wstring ICloudProfileChangedEvent::name
```

6.28 ICloudProfileRegisteredEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

6.28.1 Attributes

6.28.1.1 providerId (read-only)

```
uuid ICloudProfileRegisteredEvent::providerId
```

6.28.1.2 name (read-only)

```
wstring ICloudProfileRegisteredEvent::name
```

6.28.1.3 registered (read-only)

```
boolean ICloudProfileRegisteredEvent::registered
```

6.29 ICloudProvider

6.29.1 Attributes

6.29.1.1 name (read-only)

```
wstring ICloudProvider::name
```

Returns the long name of the provider. Includes vendor and precise product name spelled out in the preferred way.

6.29.1.2 shortName (read-only)

```
wstring ICloudProvider::shortName
```

Returns the short name of the provider. Less than 8 ASCII chars, using acronyms. No vendor name, but can contain a hint if it's a 3rd party implementation for this cloud provider, to keep it unique.

6.29.1.3 id (read-only)

```
uuid ICloudProvider::id
```

Returns the UUID of this cloud provider.

6.29.1.4 profiles (read-only)

```
ICloudProfile ICloudProvider::profiles[]
```

Returns all profiles for this cloud provider.

6.29.1.5 profileNames (read-only)

```
wstring ICloudProvider::profileNames[]
```

Returns all profile names for this cloud provider.

6.29.1.6 supportedPropertyNames (read-only)

```
wstring ICloudProvider::supportedPropertyNames[]
```

Returns the supported property names.

6.29.2 createProfile

```
void ICloudProvider::createProfile(
           [in] wstring profileName,
           [in] wstring names[],
           [in] wstring values[])
```

profileName The profile name. Must not exist, otherwise an error is raised.

names Names of properties.

values Values of set properties.

Creates a new profile.

6.29.3 getProfileByName

```
ICloudProfile ICloudProvider::getProfileByName(
           [in] wstring profileName)
```

profileName

6.29.4 getPropertyDescription

```
wstring ICloudProvider::getPropertyDescription(
           [in] wstring name)
```

name Property name.

6.29.5 importProfiles

```
void ICloudProvider::importProfiles()
```

Import the profiles from the original source

6.29.6 prepareUninstall

```
void ICloudProvider::prepareUninstall()
```

The caller requests the cloud provider to cease operation. Should return an error if this is currently not possible (due to ongoing cloud activity, possibly by a different API client). However, this must not wait for the completion for a larger amount of time (ideally stays below a second of execution time). If this succeeds it should leave the cloud provider in a state which does not allow starting new operations.

6.29.7 restoreProfiles

```
void ICloudProvider::restoreProfiles()
```

Restores the old local profiles if they exist

6.29.8 saveProfiles

```
void ICloudProvider::saveProfiles()
```

Saves the local profiles

6.30 ICloudProviderListChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Each individual provider that is installed or uninstalled is reported as an ICloudProviderRegisteredEvent. When the batch is done this event is sent and listens can get the new list.

6.30.1 Attributes

6.30.1.1 registered (read-only)

```
boolean ICloudProviderListChangedEvent::registered
```

6.31 ICloudProviderManager

6.31.1 Attributes

6.31.1.1 providers (read-only)

```
ICloudProvider ICloudProviderManager::providers[]
```

Returns all supported cloud providers.

6.31.2 getProviderById

```
ICloudProvider ICloudProviderManager::getProviderById(
           [in] uuid providerId)
```

providerId

6.31.3 getProviderByName

```
ICloudProvider ICloudProviderManager::getProviderByName(
           [in] wstring providerName)
```

providerName

6.31.4 getProviderByShortName

```
ICloudProvider ICloudProviderManager::getProviderByShortName(
           [in] wstring providerName)
```

providerName

6.32 ICloudProviderRegisteredEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

A cloud provider was installed or uninstalled. See also ICloudProviderListChangedEvent.

6.32.1 Attributes

6.32.1.1 id (read-only)

```
uuid ICloudProviderRegisteredEvent::id
```

6.32.1.2 registered (read-only)

```
boolean ICloudProviderRegisteredEvent::registered
```

6.33 ICloudProviderUninstallEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

A cloud provider is about to be uninstalled. Unlike ICloudProviderRegisteredEvent this one is waitable and is sent *before* an attempt is made to uninstall a provider. Listens should release references to the provider and related objects if they have any, or the uninstallation of the provider is going to fail because it's still in use.

6.33.1 Attributes

6.33.1.1 id (read-only)

```
uuid ICloudProviderUninstallEvent::id
```

6.34 IConsole

The IConsole interface represents an interface to control virtual machine execution.

A console object gets created when a machine has been locked for a particular session (client process) using IMachine::lockMachine() or IMachine::launchVMProcess(). The console object can then be found in the session's ISession::console attribute.

Methods of the IConsole interface allow the caller to query the current virtual machine execution state, pause the machine or power it down, save the machine state or take a snapshot, attach and detach removable media and so on.

See also: ISession

6.34.1 Attributes

6.34.1.1 machine (read-only)

```
IMachine IConsole::machine
```

Machine object for this console session.

Note: This is a convenience property, it has the same value as ISession::machine of the corresponding session object.

6.34.1.2 state (read-only)

`MachineState IConsole::state`

Current execution state of the machine.

Note: This property always returns the same value as the corresponding property of the IMachine object for this console session. For the process that owns (executes) the VM, this is the preferable way of querying the VM state, because no IPC calls are made.

6.34.1.3 guest (read-only)

`IGuest IConsole::guest`

Guest object.

6.34.1.4 keyboard (read-only)

`IKeyboard IConsole::keyboard`

Virtual keyboard object.

Note: If the machine is not running, any attempt to use the returned object will result in an error.

6.34.1.5 mouse (read-only)

`IMouse IConsole::mouse`

Virtual mouse object.

Note: If the machine is not running, any attempt to use the returned object will result in an error.

6.34.1.6 display (read-only)

`IDisplay IConsole::display`

Virtual display object.

Note: If the machine is not running, any attempt to use the returned object will result in an error.

6.34.1.7 debugger (read-only)

IMachineDebugger IConsole::debugger

Debugging interface.

6.34.1.8 USBDevices (read-only)

IUSBDevice IConsole::USBDevices[]

Collection of USB devices currently attached to the virtual USB controller.

> **Note:** The collection is empty if the machine is not running.

6.34.1.9 remoteUSBDevices (read-only)

IHostUSBDevice IConsole::remoteUSBDevices[]

List of USB devices currently attached to the remote VRDE client. Once a new device is physically attached to the remote host computer, it appears in this list and remains there until detached.

6.34.1.10 sharedFolders (read-only)

ISharedFolder IConsole::sharedFolders[]

Collection of shared folders for the current session. These folders are called transient shared folders because they are available to the guest OS running inside the associated virtual machine only for the duration of the session (as opposed to IMachine::sharedFolders[] which represent permanent shared folders). When the session is closed (e.g. the machine is powered down), these folders are automatically discarded.

New shared folders are added to the collection using createSharedFolder(). Existing shared folders can be removed using removeSharedFolder().

6.34.1.11 VRDEServerInfo (read-only)

IVRDEServerInfo IConsole::VRDEServerInfo

Interface that provides information on Remote Desktop Extension (VRDE) connection.

6.34.1.12 eventSource (read-only)

IEventSource IConsole::eventSource

Event source for console events.

6.34.1.13 attachedPCIDevices (read-only)

IPCIDeviceAttachment IConsole::attachedPCIDevices[]

Array of PCI devices attached to this machine.

6.34.1.14 useHostClipboard (read/write)

```
boolean IConsole::useHostClipboard
```

Whether the guest clipboard should be connected to the host one or whether it should only be allowed access to the VRDE clipboard. This setting may not affect existing guest clipboard connections which are already connected to the host clipboard.

6.34.1.15 emulatedUSB (read-only)

```
IEmulatedUSB IConsole::emulatedUSB
```

Interface that manages emulated USB devices.

6.34.2 addEncryptionPassword

```
void IConsole::addEncryptionPassword(
            [in] wstring id,
            [in] wstring password,
            [in] boolean clearOnSuspend)
```

id The identifier used for the password. Must match the identifier used when the encrypted medium was created.

password The password.

clearOnSuspend Flag whether to clear the password on VM suspend (due to a suspending host for example). The password must be supplied again before the VM can resume.

Adds a password used for encryption/decryption.
If this method fails, the following error codes may be reported:

- VBOX_E_PASSWORD_INCORRECT: The password provided wasn't correct for at least one disk using the provided ID.

6.34.3 addEncryptionPasswords

```
void IConsole::addEncryptionPasswords(
            [in] wstring ids[],
            [in] wstring passwords[],
            [in] boolean clearOnSuspend)
```

ids List of identifiers for the passwords. Must match the identifier used when the encrypted medium was created.

passwords List of passwords.

clearOnSuspend Flag whether to clear the given passwords on VM suspend (due to a suspending host for example). The passwords must be supplied again before the VM can resume.

Adds a password used for encryption/decryption.
If this method fails, the following error codes may be reported:

- VBOX_E_PASSWORD_INCORRECT: The password provided wasn't correct for at least one disk using the provided ID.

6.34.4 attachUSBDevice

```
void IConsole::attachUSBDevice(
            [in] uuid id,
            [in] wstring captureFilename)
```

id UUID of the host USB device to attach.

captureFilename Filename to capture the USB traffic to.

Attaches a host USB device with the given UUID to the USB controller of the virtual machine.
The device needs to be in one of the following states: Busy, Available or Held, otherwise an error is immediately returned.
When the device state is Busy, an error may also be returned if the host computer refuses to release it for some reason.
See also: IUSBDeviceFilters::deviceFilters[], USBDeviceState
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine state neither Running nor Paused.
- VBOX_E_PDM_ERROR: Virtual machine does not have a USB controller.

6.34.5 clearAllEncryptionPasswords

```
void IConsole::clearAllEncryptionPasswords()
```

Clears all provided supplied encryption passwords.

6.34.6 createSharedFolder

```
void IConsole::createSharedFolder(
            [in] wstring name,
            [in] wstring hostPath,
            [in] boolean writable,
            [in] boolean automount,
            [in] wstring autoMountPoint)
```

name Unique logical name of the shared folder.

hostPath Full path to the shared folder in the host file system.

writable Whether the share is writable or readonly

automount Whether the share gets automatically mounted by the guest or not.

autoMountPoint Where the guest should automatically mount the folder, if possible. For Windows and OS/2 guests this should be a drive letter, while other guests it should be a absolute directory.

Creates a transient new shared folder by associating the given logical name with the given host path, adds it to the collection of shared folders and starts sharing it. Refer to the description of ISharedFolder to read more about logical names.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine is in the Saved or AbortedSaved state or currently changing state.
- VBOX_E_FILE_ERROR: Shared folder already exists or not accessible.

6.34.7 detachUSBDevice

```
IUSBDevice IConsole::detachUSBDevice(
           [in] uuid id)
```

id UUID of the USB device to detach.

Detaches a USB device with the given UUID from the USB controller of the virtual machine.
After this method succeeds, the VirtualBox server re-initiates all USB filters as if the device were just physically attached to the host, but filters of this machine are ignored to avoid a possible automatic re-attachment.
See also: IUSBDeviceFilters::deviceFilters[], USBDeviceState
If this method fails, the following error codes may be reported:

- VBOX_E_PDM_ERROR: Virtual machine does not have a USB controller.
- E_INVALIDARG: USB device not attached to this virtual machine.

6.34.8 findUSBDeviceByAddress

```
IUSBDevice IConsole::findUSBDeviceByAddress(
           [in] wstring name)
```

name Address of the USB device (as assigned by the host) to search for.

Searches for a USB device with the given host address.
See also: IUSBDevice::address
If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: Given name does not correspond to any USB device.

6.34.9 findUSBDeviceById

```
IUSBDevice IConsole::findUSBDeviceById(
           [in] uuid id)
```

id UUID of the USB device to search for.

Searches for a USB device with the given UUID.
See also: IUSBDevice::id
If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: Given id does not correspond to any USB device.

6.34.10 getDeviceActivity

```
DeviceActivity[] IConsole::getDeviceActivity(
           [in] DeviceType type[])
```

type

Gets the current activity type of given devices or device groups.
If this method fails, the following error codes may be reported:

- E_INVALIDARG: Invalid device type.

6.34.11 getGuestEnteredACPIMode

```
boolean IConsole::getGuestEnteredACPIMode()
```

Checks if the guest entered the ACPI mode G0 (working) or G1 (sleeping). If this method returns `false`, the guest will most likely not respond to external ACPI events.
If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine not in Running state.

6.34.12 getPowerButtonHandled

```
boolean IConsole::getPowerButtonHandled()
```

Checks if the last power button event was handled by guest.
If this method fails, the following error codes may be reported:

- `VBOX_E_PDM_ERROR`: Checking if the event was handled by the guest OS failed.

6.34.13 pause

```
void IConsole::pause()
```

Pauses the virtual machine execution.
If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine not in Running state.
- `VBOX_E_VM_ERROR`: Virtual machine error in suspend operation.

6.34.14 powerButton

```
void IConsole::powerButton()
```

Sends the ACPI power button event to the guest.
If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine not in Running state.
- `VBOX_E_PDM_ERROR`: Controlled power off failed.

6.34.15 powerDown

```
IProgress IConsole::powerDown()
```

Initiates the power down procedure to stop the virtual machine execution.
The completion of the power down procedure is tracked using the returned IProgress object. After the operation is complete, the machine will go to the PoweredOff state.
If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine must be Running, Paused or Stuck to be powered down.

6.34.16 powerUp

```
IProgress IConsole::powerUp()
```

Starts the virtual machine execution using the current machine state (that is, its current execution state, current settings and current storage devices).

> **Note:** This method is only useful for front-ends that want to actually execute virtual machines in their own process (like the VirtualBox or VBoxSDL front-ends). Unless you are intending to write such a front-end, do not call this method. If you simply want to start virtual machine execution using one of the existing front-ends (for example the VirtualBox GUI or headless server), use IMachine::launchVMProcess() instead; these front-ends will power up the machine automatically for you.

If the machine is powered off or aborted, the execution will start from the beginning (as if the real hardware were just powered on).

If the machine is in the Saved state or the AbortedSaved state it will continue its execution from the point where the state was saved.

If the machine IMachine::teleporterEnabled property is enabled on the machine being powered up, the machine will wait for an incoming teleportation in the TeleportingIn state. The returned progress object will have at least three operations where the last three are defined as: (1) powering up and starting TCP server, (2) waiting for incoming teleportations, and (3) perform teleportation. These operations will be reflected as the last three operations of the progress objected returned by IMachine::launchVMProcess() as well.

See also: IMachine::saveState()

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine already running.
- `VBOX_E_HOST_ERROR`: Host interface does not exist or name not set.
- `VBOX_E_FILE_ERROR`: Invalid saved state file.

6.34.17 powerUpPaused

```
IProgress IConsole::powerUpPaused()
```

Identical to powerUp except that the VM will enter the Paused state, instead of Running.

See also: powerUp()

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine already running.
- `VBOX_E_HOST_ERROR`: Host interface does not exist or name not set.
- `VBOX_E_FILE_ERROR`: Invalid saved state file.

6.34.18 removeEncryptionPassword

```
void IConsole::removeEncryptionPassword(
          [in] wstring id)
```

id The identifier used for the password. Must match the identifier used when the encrypted medium was created.

Removes a password used for hard disk encryption/decryption from the running VM. As soon as the medium requiring this password is accessed the VM is paused with an error and the password must be provided again.

6.34.19 removeSharedFolder

```
void IConsole::removeSharedFolder(
           [in] wstring name)
```

name Logical name of the shared folder to remove.

Removes a transient shared folder with the given name previously created by createSharedFolder() from the collection of shared folders and stops sharing it.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine is in the Saved or AbortedSaved state or currently changing state.
- VBOX_E_FILE_ERROR: Shared folder does not exists.

6.34.20 reset

```
void IConsole::reset()
```

Resets the virtual machine.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine not in Running state.
- VBOX_E_VM_ERROR: Virtual machine error in reset operation.

6.34.21 resume

```
void IConsole::resume()
```

Resumes the virtual machine execution.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine not in Paused state.
- VBOX_E_VM_ERROR: Virtual machine error in resume operation.

6.34.22 sleepButton

```
void IConsole::sleepButton()
```

Sends the ACPI sleep button event to the guest.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine not in Running state.
- VBOX_E_PDM_ERROR: Sending sleep button event failed.

6.34.23 teleport

```
IProgress IConsole::teleport(
           [in] wstring hostname,
           [in] unsigned long tcpport,
           [in] wstring password,
           [in] unsigned long maxDowntime)
```

hostname The name or IP of the host to teleport to.

tcpport The TCP port to connect to (1..65535).

password The password.

maxDowntime The maximum allowed downtime given as milliseconds. 0 is not a valid value. Recommended value: 250 ms.

The higher the value is, the greater the chance for a successful teleportation. A small value may easily result in the teleportation process taking hours and eventually fail.

> **Note:** The current implementation treats this a guideline, not as an absolute rule.

Teleport the VM to a different host machine or process.
@todo Explain the details.
If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` Virtual machine not running or paused.

6.35 ICursorPositionChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

The guest reports cursor position data.

6.35.1 Attributes

6.35.1.1 hasData (read-only)

```
boolean ICursorPositionChangedEvent::hasData
```

Event contains valid data. If not set, switch back to using the host cursor.

6.35.1.2 x (read-only)

```
unsigned long ICursorPositionChangedEvent::x
```

Reported X position

6.35.1.3 y (read-only)

```
unsigned long ICursorPositionChangedEvent::y
```

Reported Y position

6.36 IDHCPConfig

The DHCP server has several configuration levels: global, group and individual MAC. This interface implements the settings common to each level.

6.36.1 Attributes

6.36.1.1 scope (read-only)

```
DHCPConfigScope IDHCPConfig::scope
```

Indicates the kind of config this is (mostly for IDHCPIndividualConfig).

6.36.1.2 minLeaseTime (read/write)

```
unsigned long IDHCPConfig::minLeaseTime
```

The minimum lease time in seconds, ignored if zero.

6.36.1.3 defaultLeaseTime (read/write)

```
unsigned long IDHCPConfig::defaultLeaseTime
```

The default lease time in seconds, ignored if zero.

6.36.1.4 maxLeaseTime (read/write)

```
unsigned long IDHCPConfig::maxLeaseTime
```

The maximum lease time in seconds, ignored if zero.

6.36.1.5 forcedOptions (read/write)

```
DHCPOption IDHCPConfig::forcedOptions[]
```

List of DHCP options which should be forced upon the clients in this config scope when they are available, whether the clients asks for them or not.

6.36.1.6 suppressedOptions (read/write)

```
DHCPOption IDHCPConfig::suppressedOptions[]
```

List of DHCP options which should not be sent to the clients in this config scope. This is intended for cases where one client or a group of clients shouldn't see one or more (typically global) options.

6.36.2 getAllOptions

```
wstring[] IDHCPConfig::getAllOptions(
            [out] DHCPOption options[],
            [out] DHCPOptionEncoding encodings[])
```

options Array containing the DHCP option numbers.

encodings Array of value encodings that runs parallel to `options`.

Gets all DHCP options and their values

6.36.3 getOption

```
wstring IDHCPConfig::getOption(
            [in] DHCPOption option,
            [out] DHCPOptionEncoding encoding)
```

option The DHCP option being sought.

encoding The value encoding.

Gets the value of a single DHCP option.

6.36.4 remove

```
void IDHCPConfig::remove()
```

Remove this group or individual configuration. Will of course not work on global configurations.

6.36.5 removeAllOptions

```
void IDHCPConfig::removeAllOptions()
```

Removes all the options.
One exception here is the DhcpOpt_SubnetMask option in the global scope that is linked to the IDHCPServer::networkMask attribute and therefore cannot be removed.

6.36.6 removeOption

```
void IDHCPConfig::removeOption(
            [in] DHCPOption option)
```

option

Removes the given DHCP option.

6.36.7 setOption

```
void IDHCPConfig::setOption(
            [in] DHCPOption option,
            [in] DHCPOptionEncoding encoding,
            [in] wstring value)
```

option The DHCP option.

encoding The value encoding.

value The DHCP option value. The exact format depends on the DHCP `option` value and `encoding`, see see DHCPOption for the Normal format.

Sets a DHCP option.

6.37 IDHCPGlobalConfig (IDHCPConfig)

> **Note:** This interface extends IDHCPConfig and therefore supports all its methods and attributes as well.

The global DHCP server configuration, see IDHCPServer::globalConfig.

6.38 IDHCPGroupCondition

6.38.1 Attributes

6.38.1.1 inclusive (read/write)

```
boolean IDHCPGroupCondition::inclusive
```

Whether this is an inclusive or exclusive group membership condition

6.38.1.2 type (read/write)

```
DHCPGroupConditionType IDHCPGroupCondition::type
```

Defines how the value is interpreted.

6.38.1.3 value (read/write)

```
wstring IDHCPGroupCondition::value
```

The condition value.

6.38.2 remove

```
void IDHCPGroupCondition::remove()
```

Remove this condition from the group.

6.39 IDHCPGroupConfig (IDHCPConfig)

> **Note:** This interface extends IDHCPConfig and therefore supports all its methods and attributes as well.

A configuration that applies to a group of NICs.

6.39.1 Attributes

6.39.1.1 name (read/write)

```
wstring IDHCPGroupConfig::name
```

The group name.

6.39.1.2 conditions (read-only)

```
IDHCPGroupCondition IDHCPGroupConfig::conditions[]
```

Group membership conditions.
Add new conditions by calling addCondition() and use IDHCPGroupCondition::remove() to remove.

6.39.2 addCondition

```
IDHCPGroupCondition IDHCPGroupConfig::addCondition(
            [in] boolean inclusive,
            [in] DHCPGroupConditionType type,
            [in] wstring value)
```

inclusive

type

value

Adds a new condition.

6.39.3 removeAllConditions

```
void IDHCPGroupConfig::removeAllConditions()
```

Removes all conditions.

6.40 IDHCPIndividualConfig (IDHCPConfig)

> **Note:** This interface extends IDHCPConfig and therefore supports all its methods and attributes as well.

Configuration for a single NIC, either given directly by MAC address or by VM + adaptor slot number.

6.40.1 Attributes

6.40.1.1 MACAddress (read-only)

```
wstring IDHCPIndividualConfig::MACAddress
```

The MAC address. If a MachineNIC config, this will be queried via the VM ID.

6.40.1.2 machineId (read-only)

```
uuid IDHCPIndividualConfig::machineId
```

The virtual machine ID if a MachineNIC config, null UUID for MAC.

6.40.1.3 slot (read-only)

```
unsigned long IDHCPIndividualConfig::slot
```

The NIC slot number of the VM if a MachineNIC config.

6.40.1.4 fixedAddress (read/write)

```
wstring IDHCPIndividualConfig::fixedAddress
```

Fixed IPv4 address assignment, dynamic if empty.

6.41 IDHCPServer

The IDHCPServer interface represents the VirtualBox DHCP server configuration.
To enumerate all the DHCP servers on the host, use the IVirtualBox::DHCPServers[] attribute.

6.41.1 Attributes

6.41.1.1 eventSource (read-only)

`IEventSource IDHCPServer::eventSource`

6.41.1.2 enabled (read/write)

`boolean IDHCPServer::enabled`

specifies if the DHCP server is enabled

6.41.1.3 IPAddress (read-only)

`wstring IDHCPServer::IPAddress`

specifies server IP

6.41.1.4 networkMask (read-only)

`wstring IDHCPServer::networkMask`

specifies server network mask

6.41.1.5 networkName (read-only)

`wstring IDHCPServer::networkName`

specifies internal network name the server is used for

6.41.1.6 lowerIP (read-only)

`wstring IDHCPServer::lowerIP`

specifies from IP address in server address range

6.41.1.7 upperIP (read-only)

`wstring IDHCPServer::upperIP`

specifies to IP address in server address range

6.41.1.8 globalConfig (read-only)

`IDHCPGlobalConfig IDHCPServer::globalConfig`

Global configuration that applies to all clients.

6.41.1.9 groupConfigs (read-only)

`IDHCPGroupConfig IDHCPServer::groupConfigs[]`

Configuration groups that applies to selected clients, selection is flexible.

6.41.1.10 individualConfigs (read-only)

```
IDHCPIndividualConfig IDHCPServer::individualConfigs[]
```

Individual NIC configurations either by MAC address or VM + NIC number.

6.41.2 findLeaseByMAC

```
void IDHCPServer::findLeaseByMAC(
            [in] wstring mac,
            [in] long type,
            [out] wstring address,
            [out] wstring state,
            [out] long long issued,
            [out] long long expire)
```

mac The MAC address to look up.

type Reserved, MBZ.

address The assigned address.

state The lease state.

issued Timestamp of when the lease was issued, in seconds since 1970-01-01 UTC.

expire Timestamp of when the lease expires/expired, in seconds since 1970-01-01 UTC.

Queries the persistent lease database by MAC address.
This is handy if the host wants to connect to a server running inside a VM on a host only network.
If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: If MAC address not in the database.
- VBOX_E_FILE_ERROR: If not able to read the lease database file.

6.41.3 getConfig

```
IDHCPConfig IDHCPServer::getConfig(
            [in] DHCPConfigScope scope,
            [in] wstring name,
            [in] unsigned long slot,
            [in] boolean mayAdd)
```

scope The kind of configuration being sought or added.

name Meaning depends on the scope: - Ignored when the scope is Global. - A VM name or UUID for MachineNIC. - A MAC address for MAC. - A group name for Group.

slot The NIC slot when scope is set to MachineNIC, must be zero for all other scope values.

mayAdd Set to TRUE if the configuration should be added if not found. If set to FALSE the method will fail with VBOX_E_OBJECT_NOT_FOUND.

Gets or adds a configuration.

6.41.4 restart

```
void IDHCPServer::restart()
```

Restart running DHCP server process.
If this method fails, the following error codes may be reported:

- E_FAIL: Failed to restart the process.

6.41.5 setConfiguration

```
void IDHCPServer::setConfiguration(
            [in] wstring IPAddress,
            [in] wstring networkMask,
            [in] wstring FromIPAddress,
            [in] wstring ToIPAddress)
```

IPAddress server IP address

networkMask server network mask

FromIPAddress server From IP address for address range

ToIPAddress server To IP address for address range

configures the server
If this method fails, the following error codes may be reported:

- E_INVALIDARG: invalid configuration supplied

6.41.6 start

```
void IDHCPServer::start(
            [in] wstring trunkName,
            [in] wstring trunkType)
```

trunkName Name of internal network trunk.

trunkType Type of internal network trunk.

Starts DHCP server process.
If this method fails, the following error codes may be reported:

- E_FAIL: Failed to start the process.

6.41.7 stop

```
void IDHCPServer::stop()
```

Stops DHCP server process.
If this method fails, the following error codes may be reported:

- E_FAIL: Failed to stop the process.

6.42 IDataStream

The IDataStream interface is used to retrieve a data stream. It is returned by IMediumIO::convertToStream().

6.42.1 Attributes

6.42.1.1 readSize (read-only)

```
unsigned long IDataStream::readSize
```

Recommended size of a read. Requesting a larger read may be possible in certain situations, but it is not guaranteed.

6.42.2 read

```
octet[] IDataStream::read(
            [in] unsigned long size,
            [in] unsigned long timeoutMS)
```

size How many bytes to try read.

timeoutMS Timeout (in ms) for limiting the wait time for data to be available. Pass 0 for an infinite timeout.

Read data from the stream.
If this method fails, the following error codes may be reported:

- VBOX_E_TIMEOUT: Waiting time has expired.

6.43 IDirectory

Abstract parent interface for directories handled by VirtualBox.

6.43.1 Attributes

6.43.1.1 directoryName (read-only)

```
wstring IDirectory::directoryName
```

The path specified when opening the directory.

6.43.1.2 eventSource (read-only)

```
IEventSource IDirectory::eventSource
```

Event source for directory events.

6.43.1.3 filter (read-only)

```
wstring IDirectory::filter
```

Directory listing filter to (specified when opening the directory).

6.43.1.4 id (read-only)

```
unsigned long IDirectory::id
```

The ID VirtualBox internally assigned to the open directory.

6.43.1.5 status (read-only)

```
DirectoryStatus IDirectory::status
```

Current directory status.

6.43.2 close

```
void IDirectory::close()
```

Closes this directory. After closing operations like reading the next directory entry will not be possible anymore.

6.43.3 list

```
IFsObjInfo[] IDirectory::list(
           [in] unsigned long maxEntries)
```

maxEntries Maximum entries to return per call. The guest might decide to return less than the given maximum, depending on the guest OS.

Lists directory entries of this directory.
If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: No more directory entries to read.
- VBOX_E_NOT_SUPPORTED: Method not supported by installed Guest Additions.

6.43.4 read

```
IFsObjInfo IDirectory::read()
```

Reads the next directory entry of this directory.
If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: No more directory entries to read.

6.43.5 rewind

```
void IDirectory::rewind()
```

Rewinds the directory reading.

6.44 IDisplay

The IDisplay interface represents the virtual machine's display.

The object implementing this interface is contained in each IConsole::display attribute and represents the visual output of the virtual machine.

The virtual display supports pluggable output targets represented by the IFramebuffer interface. Examples of the output target are a window on the host computer or an RDP session's display on a remote computer.

6.44.1 Attributes

6.44.1.1 guestScreenLayout (read-only)

```
IGuestScreenInfo IDisplay::guestScreenLayout[]
```

Layout of the guest screens.

6.44.2 attachFramebuffer

```
uuid IDisplay::attachFramebuffer(
            [in] unsigned long screenId,
            [in] IFramebuffer framebuffer)
```

screenId

framebuffer

Sets the graphics update target for a screen.

6.44.3 createGuestScreenInfo

```
IGuestScreenInfo IDisplay::createGuestScreenInfo(
            [in] unsigned long display,
            [in] GuestMonitorStatus status,
            [in] boolean primary,
            [in] boolean changeOrigin,
            [in] long originX,
            [in] long originY,
            [in] unsigned long width,
            [in] unsigned long height,
            [in] unsigned long bitsPerPixel)
```

display The number of the guest display.

status `True`, if this guest screen is enabled, `False` otherwise.

primary Whether this guest monitor must be primary.

changeOrigin `True`, if the origin of the guest screen should be changed, `False` otherwise.

originX The X origin of the guest screen.

originY The Y origin of the guest screen.

width The width of the guest screen.

height The height of the guest screen.

bitsPerPixel The number of bits per pixel of the guest screen.

Make a IGuestScreenInfo object with the provided parameters.

6.44.4 detachFramebuffer

```
void IDisplay::detachFramebuffer(
            [in] unsigned long screenId,
            [in] uuid id)
```

screenId

id

Removes the graphics updates target for a screen.

6.44.5 detachScreens

```
void IDisplay::detachScreens(
            [in] long screenIds[])
```

screenIds

Unplugs monitors from the virtual graphics card.

6.44.6 drawToScreen

> **Note:** This method is not supported in the web service.

```
void IDisplay::drawToScreen(
            [in] unsigned long screenId,
            [in] [ptr] octet address,
            [in] unsigned long x,
            [in] unsigned long y,
            [in] unsigned long width,
            [in] unsigned long height)
```

screenId Monitor to take the screenshot from.

address Address to store the screenshot to

x Relative to the screen top left corner.

y Relative to the screen top left corner.

width Desired image width.

height Desired image height.

Draws a 32-bpp image of the specified size from the given buffer to the given point on the VM display.

If this method fails, the following error codes may be reported:

- `E_NOTIMPL:` Feature not implemented.
- `VBOX_E_IPRT_ERROR:` Could not draw to screen.

6.44.7 getScreenResolution

```
void IDisplay::getScreenResolution(
            [in] unsigned long screenId,
            [out] unsigned long width,
            [out] unsigned long height,
            [out] unsigned long bitsPerPixel,
            [out] long xOrigin,
            [out] long yOrigin,
            [out] GuestMonitorStatus guestMonitorStatus)
```

screenId

width

height

bitsPerPixel

xOrigin

yOrigin

guestMonitorStatus

Queries certain attributes such as display width, height, color depth and the X and Y origin for a given guest screen.

The parameters `xOrigin` and `yOrigin` return the X and Y coordinates of the framebuffer's origin.

All return parameters are optional.

6.44.8 getVideoModeHint

```
void IDisplay::getVideoModeHint(
           [in] unsigned long display,
           [out] boolean enabled,
           [out] boolean changeOrigin,
           [out] long originX,
           [out] long originY,
           [out] unsigned long width,
           [out] unsigned long height,
           [out] unsigned long bitsPerPixel)
```

display The number of the guest output to query.

enabled `True` if a monitor is connected, `False` otherwise.

changeOrigin `True`, if the position of the guest screen was specified, `False` otherwise.

originX The X origin of the guest screen.

originY The Y origin of the guest screen.

width The width of the monitor preferred mode.

height The height of the monitor preferred mode.

bitsPerPixel The number of bits per pixel of the monitor preferred mode.

Queries the monitor information for a given guest output. See `setVideoModeHint`. If no monitor information has been set yet by a front-end the preferred mode values returned will be zero.

@todo Rename this to `getMonitorInfo` for 7.0.

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` The `display` value is higher than the number of outputs.

6.44.9 invalidateAndUpdate

```
void IDisplay::invalidateAndUpdate()
```

Does a full invalidation of the VM display and instructs the VM to update it.

If this method fails, the following error codes may be reported:

- `VBOX_E_IPRT_ERROR:` Could not invalidate and update screen.

6.44.10 invalidateAndUpdateScreen

```
void IDisplay::invalidateAndUpdateScreen(
           [in] unsigned long screenId)
```

screenId The guest screen to redraw.

Redraw the specified VM screen.

6.44.11 notifyHiDPIOutputPolicyChange

```
void IDisplay::notifyHiDPIOutputPolicyChange(
           [in] boolean fUnscaledHiDPI)
```

fUnscaledHiDPI

Notify OpenGL HGCM host service about HiDPI monitor scaling policy change.

6.44.12 notifyScaleFactorChange

```
void IDisplay::notifyScaleFactorChange(
           [in] unsigned long screenId,
           [in] unsigned long u32ScaleFactorWMultiplied,
           [in] unsigned long u32ScaleFactorHMultiplied)
```

screenId

u32ScaleFactorWMultiplied

u32ScaleFactorHMultiplied

Notify OpenGL HGCM host service about graphics content scaling factor change.

6.44.13 queryFramebuffer

```
IFramebuffer IDisplay::queryFramebuffer(
           [in] unsigned long screenId)
```

screenId

Queries the graphics updates targets for a screen.

6.44.14 querySourceBitmap

Note: This method is not supported in the web service.

```
void IDisplay::querySourceBitmap(
           [in] unsigned long screenId,
           [out] IDisplaySourceBitmap displaySourceBitmap)
```

screenId

displaySourceBitmap

Obtains the guest screen bitmap parameters.

6.44.15 setScreenLayout

```
void IDisplay::setScreenLayout(
            [in] ScreenLayoutMode screenLayoutMode,
            [in] IGuestScreenInfo guestScreenInfo[])
```

screenLayoutMode

guestScreenInfo

Set video modes for the guest screens.

6.44.16 setSeamlessMode

```
void IDisplay::setSeamlessMode(
            [in] boolean enabled)
```

enabled

Enables or disables seamless guest display rendering (seamless desktop integration) mode.

> **Note:** Calling this method has no effect if IGuest::getFacilityStatus() with facility `Seamless` does not return `Active`.

6.44.17 setVideoModeHint

```
void IDisplay::setVideoModeHint(
            [in] unsigned long display,
            [in] boolean enabled,
            [in] boolean changeOrigin,
            [in] long originX,
            [in] long originY,
            [in] unsigned long width,
            [in] unsigned long height,
            [in] unsigned long bitsPerPixel,
            [in] boolean notify)
```

display The number of the guest output to change.

enabled `True` if a monitor is connected, `False` otherwise.

changeOrigin `True`, if the position of the guest screen is specified, `False` otherwise.

originX The X origin of the guest screen.

originY The Y origin of the guest screen.

width The width of the guest screen.

height The height of the guest screen.

bitsPerPixel The number of bits per pixel of the guest screen.

notify Whether the guest should be notified of the change. Normally this is wished, but it might not be when re-setting monitor information from the last session (no hotplug happened, as it is still the same virtual monitor). Might also be useful if several monitors are to be changed at once, but this would not reflect physical hardware well, and we also have `setScreenLayout` for that.

Changes the monitor information reported by a given output of the guest graphics device. This information can be read by the guest if suitable drivers and driver tools are available, including but not limited to those in the Guest Additions. The guest will receive monitor hotplug notification when the monitor information is changed, and the information itself will be available to the guest until the next change. The information should not be resent if the guest does not resize in response. The guest might have chosen to ignore the change, or the resize might happen later when a suitable driver is started.

Specifying `0` for either `width`, `height` or `bitsPerPixel` parameters means that the corresponding values should be taken from the current video mode (i.e. left unchanged).

@todo Rename this to `setMonitorInfo` for 7.0.

If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: The `display` value is higher then the number of outputs.

6.44.18 takeScreenShot

> **Note:** This method is not supported in the web service.

```
void IDisplay::takeScreenShot(
           [in] unsigned long screenId,
           [in] [ptr] octet address,
           [in] unsigned long width,
           [in] unsigned long height,
           [in] BitmapFormat bitmapFormat)
```

screenId

address

width

height

bitmapFormat

Takes a screen shot of the requested size and format and copies it to the buffer allocated by the caller and pointed to by `address`. The buffer size must be enough for a 32 bits per pixel bitmap, i.e. width * height * 4 bytes.

> **Note:** This API can be used only locally by a VM process through the COM/XPCOM C++ API as it requires pointer support. It is not available for scripting languages, Java or any webservice clients. Unless you are writing a new VM frontend use takeScreenShotToArray().

If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: Current screen configuration does not support taking a screen shot (typically because it is 0x0 in size).

6.44.19 takeScreenShotToArray

```
octet[] IDisplay::takeScreenShotToArray(
            [in] unsigned long screenId,
            [in] unsigned long width,
            [in] unsigned long height,
            [in] BitmapFormat bitmapFormat)
```

screenId The guest monitor to take screenshot from.

width Desired image width.

height Desired image height.

bitmapFormat The requested format.

Takes a guest screen shot of the requested size and format and returns it as an array of bytes.

6.44.20 viewportChanged

```
void IDisplay::viewportChanged(
            [in] unsigned long screenId,
            [in] unsigned long x,
            [in] unsigned long y,
            [in] unsigned long width,
            [in] unsigned long height)
```

screenId Monitor to take the screenshot from.

x Framebuffer x offset.

y Framebuffer y offset.

width Viewport width.

height Viewport height.

Signals that framebuffer window viewport has changed.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: The specified viewport data is invalid.

6.45 IDisplaySourceBitmap

Note: This interface is not supported in the web service.

6.45.1 Attributes

6.45.1.1 screenId (read-only)

```
unsigned long IDisplaySourceBitmap::screenId
```

6.45.2 queryBitmapInfo

Note: This method is not supported in the web service.

```
void IDisplaySourceBitmap::queryBitmapInfo(
            [out] [ptr] octet address,
            [out] unsigned long width,
            [out] unsigned long height,
            [out] unsigned long bitsPerPixel,
            [out] unsigned long bytesPerLine,
            [out] BitmapFormat bitmapFormat)
```

address

width

height

bitsPerPixel

bytesPerLine

bitmapFormat

Information about the screen bitmap.

6.46 IDnDBase

Base abstract interface for drag'n drop.

6.46.1 Attributes

6.46.1.1 formats (read-only)

```
wstring IDnDBase::formats[]
```

Returns all supported drag'n drop formats.

6.46.2 addFormats

```
void IDnDBase::addFormats(
            [in] wstring formats[])
```

formats Collection of formats to add.

Adds MIME / Content-type formats to the supported formats.

6.46.3 isFormatSupported

```
boolean IDnDBase::isFormatSupported(
            [in] wstring format)
```

format Format to check for.

Checks if a specific drag'n drop MIME / Content-type format is supported.

6.46.4 removeFormats

```
void IDnDBase::removeFormats(
            [in] wstring formats[])
```

formats Collection of formats to remove.

Removes MIME / Content-type formats from the supported formats.

6.47 IDnDModeChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the drag'n drop mode changes.

6.47.1 Attributes

6.47.1.1 dndMode (read-only)

```
DnDMode IDnDModeChangedEvent::dndMode
```

The new drag'n drop mode.

6.48 IDnDSource (IDnDBase)

> **Note:** This interface extends IDnDBase and therefore supports all its methods and attributes as well.

Abstract interface for handling drag'n drop sources.

6.48.1 dragIsPending

```
DnDAction IDnDSource::dragIsPending(
            [in] unsigned long screenId,
            [out] wstring formats[],
            [out] DnDAction allowedActions[])
```

screenId The screen ID where the drag and drop event occurred.

formats On return the supported mime types.

allowedActions On return the actions which are allowed.

Ask the source if there is any drag and drop operation pending. If no drag and drop operation is pending currently, DnDAction_Ignore is returned.
If this method fails, the following error codes may be reported:

- VBOX_E_VM_ERROR: VMM device is not available.

6.48.2 drop

```
IProgress IDnDSource::drop(
            [in] wstring format,
            [in] DnDAction action)
```

format The mime type the data must be in.

action The action to use.

Informs the source that a drop event occurred for a pending drag and drop operation.
If this method fails, the following error codes may be reported:

- VBOX_E_VM_ERROR: VMM device is not available.

6.48.3 receiveData

```
octet[] IDnDSource::receiveData()
```

Receive the data of a previously drag and drop event from the source.
If this method fails, the following error codes may be reported:

- VBOX_E_VM_ERROR: VMM device is not available.

6.49 IDnDTarget (IDnDBase)

Note: This interface extends IDnDBase and therefore supports all its methods and attributes as well.

Abstract interface for handling drag'n drop targets.

6.49.1 cancel

```
boolean IDnDTarget::cancel()
```

Requests cancelling the current operation. The target can veto the request in case the operation is not cancelable at the moment.
If this method fails, the following error codes may be reported:

- VBOX_E_VM_ERROR: VMM device is not available.

6.49.2 drop

```
DnDAction IDnDTarget::drop(
            [in] unsigned long screenId,
            [in] unsigned long x,
            [in] unsigned long y,
            [in] DnDAction defaultAction,
            [in] DnDAction allowedActions[],
            [in] wstring formats[],
            [out] wstring format)
```

screenId The screen ID where the Drag and Drop event occurred.

x X-position of the event.

y Y-position of the event.

defaultAction The default action to use.

allowedActions The actions which are allowed.

formats The supported MIME types.

format The resulting format of this event.

Informs the target about a drop event.
If this method fails, the following error codes may be reported:

- VBOX_E_VM_ERROR: VMM device is not available.

6.49.3 enter

```
DnDAction IDnDTarget::enter(
            [in] unsigned long screenId,
            [in] unsigned long y,
            [in] unsigned long x,
            [in] DnDAction defaultAction,
            [in] DnDAction allowedActions[],
            [in] wstring formats[])
```

screenId The screen ID where the drag and drop event occurred.

y Y-position of the event.

x X-position of the event.

defaultAction The default action to use.

allowedActions The actions which are allowed.

formats The supported MIME types.

Informs the target about a drag and drop enter event.
If this method fails, the following error codes may be reported:

- VBOX_E_VM_ERROR: VMM device is not available.

6.49.4 leave

```
void IDnDTarget::leave(
            [in] unsigned long screenId)
```

screenId The screen ID where the drag and drop event occurred.

Informs the target about a drag and drop leave event.
If this method fails, the following error codes may be reported:

- VBOX_E_VM_ERROR: VMM device is not available.

6.49.5 move

```
DnDAction IDnDTarget::move(
            [in] unsigned long screenId,
            [in] unsigned long x,
            [in] unsigned long y,
            [in] DnDAction defaultAction,
            [in] DnDAction allowedActions[],
            [in] wstring formats[])
```

screenId The screen ID where the drag and drop event occurred.

x X-position of the event.

y Y-position of the event.

defaultAction The default action to use.

allowedActions The actions which are allowed.

formats The supported MIME types.

Informs the target about a drag and drop move event.
If this method fails, the following error codes may be reported:

- VBOX_E_VM_ERROR: VMM device is not available.

6.49.6 sendData

```
IProgress IDnDTarget::sendData(
            [in] unsigned long screenId,
            [in] wstring format,
            [in] octet data[])
```

screenId The screen ID where the drag and drop event occurred.

format The MIME type the data is in.

data The actual data.

Initiates sending data to the target.
If this method fails, the following error codes may be reported:

- VBOX_E_VM_ERROR: VMM device is not available.

6.50 IEmulatedUSB

Manages emulated USB devices.

6.50.1 Attributes

6.50.1.1 webcams (read-only)

```
wstring IEmulatedUSB::webcams[]
```

Lists attached virtual webcams.

6.50.2 webcamAttach

```
void IEmulatedUSB::webcamAttach(
           [in] wstring path,
           [in] wstring settings)
```

path The host path of the capture device to use.

settings Optional settings.

Attaches the emulated USB webcam to the VM, which will use a host video capture device.

6.50.3 webcamDetach

```
void IEmulatedUSB::webcamDetach(
           [in] wstring path)
```

path The host path of the capture device to detach.

Detaches the emulated USB webcam from the VM

6.51 IEvent

Abstract parent interface for VirtualBox events. Actual events will typically implement a more specific interface which derives from this (see below).

Introduction to VirtualBox events

Generally speaking, an event (represented by this interface) signals that something happened, while an event listener (see IEventListener) represents an entity that is interested in certain events. In order for this to work with unidirectional protocols (i.e. web services), the concepts of passive and active listener are used.

Event consumers can register themselves as listeners, providing an array of events they are interested in (see IEventSource::registerListener()). When an event triggers, the listener is notified about the event. The exact mechanism of the notification depends on whether the listener was registered as an active or passive listener:

- An active listener is very similar to a callback: it is a function invoked by the API. As opposed to the callbacks that were used in the API before VirtualBox 4.0 however, events are now objects with an interface hierarchy.
- Passive listeners are somewhat trickier to implement, but do not require a client function to be callable, which is not an option with scripting languages or web service clients. Internally the IEventSource implementation maintains an event queue for each passive listener, and newly arrived events are put in this queue. When the listener calls IEventSource::getEvent(), first element from its internal event queue is returned. When the client completes processing of an event, the IEventSource::eventProcessed() function must be called, acknowledging that the event was processed. It supports implementing waitable events. On passive listener unregistration, all events from its queue are auto-acknowledged.

Waitable events are useful in situations where the event generator wants to track delivery or a party wants to wait until all listeners have completed the event. A typical example would be a vetoable event (see IVetoEvent) where a listeners might veto a certain action, and thus the event producer has to make sure that all listeners have processed the event and not vetoed before taking the action.

A given event may have both passive and active listeners at the same time.

Using events

Any VirtualBox object capable of producing externally visible events provides an `eventSource` read-only attribute, which is of the type IEventSource. This event source object is notified by VirtualBox once something has happened, so consumers may register event listeners with this event source. To register a listener, an object implementing the IEventListener interface must be provided. For active listeners, such an object is typically created by the consumer, while for passive listeners IEventSource::createListener() should be used. Please note that a listener created with IEventSource::createListener() must not be used as an active listener.

Once created, the listener must be registered to listen for the desired events (see IEventSource::registerListener()), providing an array of VBoxEventType enums. Those elements can either be the individual event IDs or wildcards matching multiple event IDs.

After registration, the callback's IEventListener::handleEvent() method is called automatically when the event is triggered, while passive listeners have to call IEventSource::getEvent() and IEventSource::eventProcessed() in an event processing loop.

The IEvent interface is an abstract parent interface for all such VirtualBox events coming in. As a result, the standard use pattern inside IEventListener::handleEvent() or the event processing loop is to check the type attribute of the event and then cast to the appropriate specific interface using `QueryInterface()`.

6.51.1 Attributes

6.51.1.1 type (read-only)

```
VBoxEventType IEvent::type
```

Event type.

6.51.1.2 source (read-only)

```
IEventSource IEvent::source
```

Source of this event.

6.51.1.3 waitable (read-only)

```
boolean IEvent::waitable
```

If we can wait for this event being processed. If false, waitProcessed() returns immediately, and setProcessed() doesn't make sense. Non-waitable events are generally better performing, as no additional overhead associated with waitability imposed. Waitable events are needed when one need to be able to wait for particular event processed, for example for vetoable changes, or if event refers to some resource which need to be kept immutable until all consumers confirmed events.

6.51.2 setProcessed

```
void IEvent::setProcessed()
```

Internal method called by the system when all listeners of a particular event have called IEventSource::eventProcessed(). This should not be called by client code.

6.51.3 waitProcessed

```
boolean IEvent::waitProcessed(
           [in] long timeout)
```

timeout Maximum time to wait for event processing, in ms; 0 = no wait, -1 = indefinite wait.

Wait until time outs, or this event is processed. Event must be waitable for this operation to have described semantics, for non-waitable returns true immediately.

6.52 IEventListener

Event listener. An event listener can work in either active or passive mode, depending on the way it was registered. See IEvent for an introduction to VirtualBox event handling.

6.52.1 handleEvent

```
void IEventListener::handleEvent(
            [in] IEvent event)
```

event Event available.

Handle event callback for active listeners. It is not called for passive listeners. After calling handleEvent() on all active listeners and having received acknowledgement from all passive listeners via IEventSource::eventProcessed(), the event is marked as processed and IEvent::waitProcessed() will return immediately.

6.53 IEventSource

Event source. Generally, any object which could generate events can be an event source, or aggregate one. To simplify using one-way protocols such as webservices running on top of HTTP(S), an event source can work with listeners in either active or passive mode. In active mode it is up to the IEventSource implementation to call IEventListener::handleEvent(), in passive mode the event source keeps track of pending events for each listener and returns available events on demand.

See IEvent for an introduction to VirtualBox event handling.

6.53.1 createAggregator

```
IEventSource IEventSource::createAggregator(
            [in] IEventSource subordinates[])
```

subordinates Subordinate event source this one aggregates.

Creates an aggregator event source, collecting events from multiple sources. This way a single listener can listen for events coming from multiple sources, using a single blocking getEvent() on the returned aggregator.

6.53.2 createListener

```
IEventListener IEventSource::createListener()
```

Creates a new listener object, useful for passive mode.

6.53.3 eventProcessed

```
void IEventSource::eventProcessed(
            [in] IEventListener listener,
            [in] IEvent event)
```

listener Which listener processed event.

event Which event.

Must be called for waitable events after a particular listener finished its event processing. When all listeners of a particular event have called this method, the system will then call IEvent::setProcessed().

6.53.4 fireEvent

```
boolean IEventSource::fireEvent(
            [in] IEvent event,
            [in] long timeout)
```

event Event to deliver.

timeout Maximum time to wait for event processing (if event is waitable), in ms; 0 = no wait, -1 = indefinite wait.

Fire an event for this source.

6.53.5 getEvent

```
IEvent IEventSource::getEvent(
            [in] IEventListener listener,
            [in] long timeout)
```

listener Which listener to get data for.

timeout Maximum time to wait for events, in ms; 0 = no wait, -1 = indefinite wait.

Get events from this peer's event queue (for passive mode). Calling this method regularly is required for passive event listeners to avoid system overload; see registerListener() for details.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: Listener is not registered, or autounregistered.

6.53.6 registerListener

```
void IEventSource::registerListener(
            [in] IEventListener listener,
            [in] VBoxEventType interesting[],
            [in] boolean active)
```

listener Listener to register.

interesting Event types listener is interested in. One can use wildcards like - Any to specify wildcards, matching more than one event.

active Which mode this listener is operating in. In active mode, IEventListener::handleEvent() is called directly. In passive mode, an internal event queue is created for this this IEventListener. For each event coming in, it is added to queues for all interested registered passive listeners. It is then up to the external code to call the listener's IEventListener::handleEvent() method. When done with an event, the external code must call eventProcessed().

Register an event listener.

Note: To avoid system overload, the VirtualBox server process checks if passive event listeners call getEvent() frequently enough. In the current implementation, if more than 500 pending events are detected for a passive event listener, it is forcefully unregistered by the system, and further getEvent() calls will return VBOX_E_OBJECT_NOT_FOUND.

6.53.7 unregisterListener

```
void IEventSource::unregisterListener(
           [in] IEventListener listener)
```

listener Listener to unregister.

Unregister an event listener. If listener is passive, and some waitable events are still in queue they are marked as processed automatically.

6.54 IEventSourceChangedEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when an event source state changes (listener added or removed).

6.54.1 Attributes

6.54.1.1 listener (read-only)

```
IEventListener IEventSourceChangedEvent::listener
```

Event listener which has changed.

6.54.1.2 add (read-only)

```
boolean IEventSourceChangedEvent::add
```

Flag whether listener was added or removed.

6.55 IExtPack (IExtPackBase)

Note: This interface is not supported in the web service.

Note: This interface extends IExtPackBase and therefore supports all its methods and attributes as well.

Interface for querying information about an extension pack as well as accessing COM objects within it.

6.55.1 queryObject

```
$unknown IExtPack::queryObject(
           [in] wstring objUuid)
```

objUuid The object ID. What exactly this is

Queries the IUnknown interface to an object in the extension pack main module. This allows plug-ins and others to talk directly to an extension pack.

6.56 IExtPackBase

Note: This interface is not supported in the web service.

Interface for querying information about an extension pack as well as accessing COM objects within it.

6.56.1 Attributes

6.56.1.1 name (read-only)

```
wstring IExtPackBase::name
```

The extension pack name. This is unique.

6.56.1.2 description (read-only)

```
wstring IExtPackBase::description
```

The extension pack description.

6.56.1.3 version (read-only)

```
wstring IExtPackBase::version
```

The extension pack version string. This is restricted to the dotted version number and optionally a build indicator. No tree revision or tag will be included in the string as those things are available as separate properties. An optional publisher tag may be present like for IVirtualBox::version.

Examples: “1.2.3”, “1.2.3_BETA1” and “1.2.3_RC2”.

6.56.1.4 revision (read-only)

```
unsigned long IExtPackBase::revision
```

The extension pack internal revision number.

6.56.1.5 edition (read-only)

```
wstring IExtPackBase::edition
```

Edition indicator. This is usually empty.

Can for instance be used to help distinguishing between two editions of the same extension pack where only the license, service contract or something differs.

6.56.1.6 VRDEModule (read-only)

```
wstring IExtPackBase::VRDEModule
```

The name of the VRDE module if the extension pack sports one.

6.56.1.7 CryptoModule (read-only)

```
wstring IExtPackBase::CryptoModule
```

The name of the crypto module if the extension pack sports one. This module is required for full VM encryption.

6.56.1.8 plugIns (read-only)

```
IExtPackPlugIn IExtPackBase::plugIns[]
```

> **Note:** This attribute is not supported in the web service.

Plug-ins provided by this extension pack.

6.56.1.9 usable (read-only)

```
boolean IExtPackBase::usable
```

Indicates whether the extension pack is usable or not.
There are a number of reasons why an extension pack might be unusable, typical examples would be broken installation/file or that it is incompatible with the current VirtualBox version.

6.56.1.10 whyUnusable (read-only)

```
wstring IExtPackBase::whyUnusable
```

String indicating why the extension pack is not usable. This is an empty string if usable and always a non-empty string if not usable.

6.56.1.11 showLicense (read-only)

```
boolean IExtPackBase::showLicense
```

Whether to show the license before installation

6.56.1.12 license (read-only)

```
wstring IExtPackBase::license
```

The default HTML license text for the extension pack. Same as calling queryLicense with preferredLocale and preferredLanguage as empty strings and format set to html.

6.56.2 queryLicense

```
wstring IExtPackBase::queryLicense(
            [in] wstring preferredLocale,
            [in] wstring preferredLanguage,
            [in] wstring format)
```

preferredLocale The preferred license locale. Pass an empty string to get the default license.

preferredLanguage The preferred license language. Pass an empty string to get the default language for the locale.

format The license format: html, rtf or txt. If a license is present there will always be an HTML of it, the rich text format (RTF) and plain text (txt) versions are optional. If

Full feature version of the license attribute.

6.57 IExtPackFile (IExtPackBase)

Note: This interface is not supported in the web service.

Note: This interface extends IExtPackBase and therefore supports all its methods and attributes as well.

Extension pack file (aka tarball, .vbox-extpack) representation returned by IExtPackManager::openExtPackFile(). This provides the base extension pack information with the addition of the file name.

6.57.1 Attributes

6.57.1.1 filePath (read-only)

```
wstring IExtPackFile::filePath
```

The path to the extension pack file.

6.57.2 install

```
IProgress IExtPackFile::install(
            [in] boolean replace,
            [in] wstring displayInfo)
```

replace Set this to automatically uninstall any existing extension pack with the same name as the one being installed.

displayInfo Platform specific display information. Reserved for future hacks.

Install the extension pack.

6.58 IExtPackInstalledEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

Extension pack installed event.

6.58.1 Attributes

6.58.1.1 name (read-only)

```
wstring IExtPackInstalledEvent::name
```

Name of the extension pack.

6.59 IExtPackManager

> **Note:** This interface is not supported in the web service.

Interface for managing VirtualBox Extension Packs.
@todo Describe extension packs, how they are managed and how to create one.

6.59.1 Attributes

6.59.1.1 installedExtPacks (read-only)

```
IExtPack IExtPackManager::installedExtPacks[]
```

> **Note:** This attribute is not supported in the web service.

List of the installed extension packs.

6.59.2 cleanup

```
void IExtPackManager::cleanup()
```

Cleans up failed installs and uninstalls

6.59.3 find

> **Note:** This method is not supported in the web service.

```
IExtPack IExtPackManager::find(
           [in] wstring name)
```

name The name of the extension pack to locate.

Returns the extension pack with the specified name if found.
If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND`: No extension pack matching `name` was found.

6.59.4 isExtPackUsable

```
boolean IExtPackManager::isExtPackUsable(
           [in] wstring name)
```

name The name of the extension pack to check for.

Check if the given extension pack is loaded and usable.

6.59.5 openExtPackFile

> **Note:** This method is not supported in the web service.

```
IExtPackFile IExtPackManager::openExtPackFile(
           [in] wstring path)
```

path The path of the extension pack tarball. This can optionally be followed by a "::SHA-256=hex-digit" of the tarball.

Attempts to open an extension pack file in preparation for installation.

6.59.6 queryAllPlugInsForFrontend

```
wstring[] IExtPackManager::queryAllPlugInsForFrontend(
           [in] wstring frontendName)
```

frontendName The name of the frontend or component.

Gets the path to all the plug-in modules for a given frontend.
This is a convenience method that is intended to simplify the plug-in loading process for a frontend.

6.59.7 uninstall

```
IProgress IExtPackManager::uninstall(
           [in] wstring name,
           [in] boolean forcedRemoval,
           [in] wstring displayInfo)
```

name The name of the extension pack to uninstall.

forcedRemoval Forced removal of the extension pack. This means that the uninstall hook will not be called.

displayInfo Platform specific display information. Reserved for future hacks.

Uninstalls an extension pack, removing all related files.

6.60 IExtPackPlugIn

> **Note:** This interface is not supported in the web service.

Interface for keeping information about a plug-in that ships with an extension pack.

6.60.1 Attributes

6.60.1.1 name (read-only)

```
wstring IExtPackPlugIn::name
```

The plug-in name.

6.60.1.2 description (read-only)

`wstring IExtPackPlugIn::description`

The plug-in description.

6.60.1.3 frontend (read-only)

`wstring IExtPackPlugIn::frontend`

The name of the frontend or component name this plug-in plugs into.

6.60.1.4 modulePath (read-only)

`wstring IExtPackPlugIn::modulePath`

The module path.

6.61 IExtPackUninstalledEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Extension pack uninstalled event.

6.61.1 Attributes

6.61.1.1 name (read-only)

`wstring IExtPackUninstalledEvent::name`

Name of the extension pack.

6.62 IExtraDataCanChangeEvent (IVetoEvent)

> **Note:** This interface extends IVetoEvent and therefore supports all its methods and attributes as well.

Notification when someone tries to change extra data for either the given machine or (if `null`) global extra data. This gives the chance to veto against changes.

6.62.1 Attributes

6.62.1.1 machineId (read-only)

`uuid IExtraDataCanChangeEvent::machineId`

ID of the machine this event relates to. Null for global extra data changes.

6.62.1.2 key (read-only)

`wstring IExtraDataCanChangeEvent::key`

Extra data key that has changed.

6.62.1.3 value (read-only)

`wstring IExtraDataCanChangeEvent::value`

Extra data value for the given key.

6.63 IExtraDataChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when machine specific or global extra data has changed.

6.63.1 Attributes

6.63.1.1 machineId (read-only)

`uuid IExtraDataChangedEvent::machineId`

ID of the machine this event relates to. Null for global extra data changes.

6.63.1.2 key (read-only)

`wstring IExtraDataChangedEvent::key`

Extra data key that has changed.

6.63.1.3 value (read-only)

`wstring IExtraDataChangedEvent::value`

Extra data value for the given key.

6.64 IFile

Abstract parent interface for files handled by VirtualBox.

6.64.1 Attributes

6.64.1.1 eventSource (read-only)

`IEventSource IFile::eventSource`

Event source for file events.

6.64.1.2 id (read-only)

`unsigned long IFile::id`

The ID VirtualBox internally assigned to the open file.

6.64.1.3 initialSize (read-only)

`long long IFile::initialSize`

The initial size in bytes when opened.

6.64.1.4 offset (read-only)

```
long long IFile::offset
```

The current file position.

The file current position always applies to the read() method, which updates it upon return. Same goes for the write() method except when accessMode is AppendOnly or AppendRead, where it will always write to the end of the file and will leave this attribute unchanged.

The seek() is used to change this attribute without transferring any file data like read and write does.

> **Note:** This will not always be correct with older Guest Additions (version 5.2.30 and earlier, as well as versions 6.0.0 thru 6.0.8) after a calling readAt() or writeAt(), or after calling write() on a file in append mode. The correct file offset can be obtained using seek().

6.64.1.5 status (read-only)

```
FileStatus IFile::status
```

Current file status.

6.64.1.6 filename (read-only)

```
wstring IFile::filename
```

Full path of the actual file name of this file.

6.64.1.7 creationMode (read-only)

```
unsigned long IFile::creationMode
```

The UNIX-style creation mode specified when opening the file.

6.64.1.8 openAction (read-only)

```
FileOpenAction IFile::openAction
```

The opening action specified when opening the file.

6.64.1.9 accessMode (read-only)

```
FileAccessMode IFile::accessMode
```

The file access mode.

6.64.2 close

```
void IFile::close()
```

Closes this file. After closing operations like reading data, writing data or querying information will not be possible anymore.

6.64.3 queryInfo

```
IFsObjInfo IFile::queryInfo()
```

Queries information about this file.

6.64.4 querySize

```
long long IFile::querySize()
```

Queries the current file size.

6.64.5 read

```
octet[] IFile::read(
            [in] unsigned long toRead,
            [in] unsigned long timeoutMS)
```

toRead Number of bytes to read.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Reads data from this file.
The file current position (offset) is updated on success.

6.64.6 readAt

```
octet[] IFile::readAt(
            [in] long long offset,
            [in] unsigned long toRead,
            [in] unsigned long timeoutMS)
```

offset Offset in bytes to start reading.

toRead Number of bytes to read.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Reads data from an offset of this file.
The file current position (offset) is updated on success.

6.64.7 seek

```
long long IFile::seek(
            [in] long long offset,
            [in] FileSeekOrigin whence)
```

offset Offset to seek relative to the position specified by `whence`.

whence One of the FileSeekOrigin seek starting points.

Changes the current file position of this file.
The file current position always applies to the read() method. Same for the write() method it except when the accessMode is AppendOnly or AppendRead.

6.64.8 setACL

```
void IFile::setACL(
            [in] wstring acl,
            [in] unsigned long mode)
```

acl The ACL specification string. To-be-defined.

mode UNIX-style mode mask to use if `acl` is empty. As mention in IGuestSession::directoryCreate() this is realized on a best effort basis and the exact behavior depends on the Guest OS.

Sets the ACL of this file.
If this method fails, the following error codes may be reported:

- `E_NOTIMPL`: The method is not implemented yet.

6.64.9 setSize

```
void IFile::setSize(
            [in] long long size)
```

size The new file size.

Changes the file size.

6.64.10 write

```
unsigned long IFile::write(
            [in] octet data[],
            [in] unsigned long timeoutMS)
```

data Array of bytes to write. The size of the array also specifies how much to write.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Writes bytes to this file.
The file current position (offset) is updated on success.

6.64.11 writeAt

```
unsigned long IFile::writeAt(
            [in] long long offset,
            [in] octet data[],
            [in] unsigned long timeoutMS)
```

offset Offset in bytes to start writing. If the file was opened with the accessMode set to AppendOnly or AppendRead, the offset is ignored and the write always goes to the end of the file.

data Array of bytes to write. The size of the array also specifies how much to write.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Writes bytes at a certain offset to this file.
The file current position (offset) is updated on success.

6.65 IFirmwareSettings

The IFirmwareSettings interface represents firmware settings of the virtual machine. This is used only in the IMachine::firmwareSettings attribute.

6.65.1 Attributes

6.65.1.1 firmwareType (read/write)

`FirmwareType IFirmwareSettings::firmwareType`

Type of firmware (such as legacy BIOS or EFI), used for initial bootstrap in this VM.

6.65.1.2 logoFadeIn (read/write)

`boolean IFirmwareSettings::logoFadeIn`

Fade in flag for BIOS logo animation.

6.65.1.3 logoFadeOut (read/write)

`boolean IFirmwareSettings::logoFadeOut`

Fade out flag for BIOS logo animation.

6.65.1.4 logoDisplayTime (read/write)

`unsigned long IFirmwareSettings::logoDisplayTime`

BIOS logo display time in milliseconds (0 = default).

6.65.1.5 logoImagePath (read/write)

`wstring IFirmwareSettings::logoImagePath`

Local file system path for external BIOS splash image. Empty string means the default image is shown on boot.

6.65.1.6 bootMenuMode (read/write)

`FirmwareBootMenuMode IFirmwareSettings::bootMenuMode`

Mode of the firmware boot device menu.

6.65.1.7 ACPIEnabled (read/write)

`boolean IFirmwareSettings::ACPIEnabled`

ACPI support flag.

6.65.1.8 IOAPICEnabled (read/write)

`boolean IFirmwareSettings::IOAPICEnabled`

I/O-APIC support flag. If set, VirtualBox will provide an I/O-APIC and support IRQs above 15.

6.65.1.9 APICMode (read/write)

```
APICMode IFirmwareSettings::APICMode
```

APIC mode to set up by the firmware.

6.65.1.10 timeOffset (read/write)

```
long long IFirmwareSettings::timeOffset
```

Offset in milliseconds from the host system time. This allows for guests running with a different system date/time than the host. It is equivalent to setting the system date/time in the BIOS except it is not an absolute value but a relative one. Guest Additions time synchronization honors this offset.

6.65.1.11 PXEDebugEnabled (read/write)

```
boolean IFirmwareSettings::PXEDebugEnabled
```

PXE debug logging flag. If set, VirtualBox will write extensive PXE trace information to the release log.

6.65.1.12 SMBIOSUuidLittleEndian (read/write)

```
boolean IFirmwareSettings::SMBIOSUuidLittleEndian
```

Flag to control whether the SMBIOS system UUID is presented in little endian form to the guest as mandated by the SMBIOS spec chapter 7.2.1. Before VirtualBox version 6.1 it was always presented in big endian form and to retain the old behavior this flag was introduced so it can be changed. VMs created with VBox 6.1 will default to true for this flag.

6.65.1.13 AutoSerialNumGen (read/write)

```
boolean IFirmwareSettings::AutoSerialNumGen
```

Flag for enabling automatic VM serial number generation.

6.66 IForm

6.66.1 Attributes

6.66.1.1 values (read-only)

```
IFormValue IForm::values[]
```

6.66.2 apply

```
IProgress IForm::apply()
```

6.66.3 getFieldGroup

```
wstring[] IForm::getFieldGroup(
           [in] wstring field)
```

field

6.67 IFormValue

6.67.1 Attributes

6.67.1.1 type (read-only)

```
FormValueType IFormValue::type
```

6.67.1.2 generation (read-only)

```
long IFormValue::generation
```

6.67.1.3 enabled (read-only)

```
boolean IFormValue::enabled
```

6.67.1.4 visible (read-only)

```
boolean IFormValue::visible
```

6.67.1.5 label (read-only)

```
wstring IFormValue::label
```

6.67.1.6 description (read-only)

```
wstring IFormValue::description
```

6.67.1.7 help (read-only)

```
wstring IFormValue::help
```

6.68 IFramebuffer

6.68.1 Attributes

6.68.1.1 width (read-only)

```
unsigned long IFramebuffer::width
```

Frame buffer width, in pixels.

6.68.1.2 height (read-only)

```
unsigned long IFramebuffer::height
```

Frame buffer height, in pixels.

6.68.1.3 bitsPerPixel (read-only)

```
unsigned long IFramebuffer::bitsPerPixel
```

Color depth, in bits per pixel.

6.68.1.4 bytesPerLine (read-only)

`unsigned long IFramebuffer::bytesPerLine`

Scan line size, in bytes.

6.68.1.5 pixelFormat (read-only)

`BitmapFormat IFramebuffer::pixelFormat`

Frame buffer pixel format. It's one of the values defined by BitmapFormat.

> **Note:** This attribute must never (and will never) return Opaque – the format of the frame buffer must be always known.

6.68.1.6 heightReduction (read-only)

`unsigned long IFramebuffer::heightReduction`

Hint from the frame buffer about how much of the standard screen height it wants to use for itself. This information is exposed to the guest through the VESA BIOS and VMMDev interface so that it can use it for determining its video mode table. It is not guaranteed that the guest respects the value.

6.68.1.7 overlay (read-only)

`IFramebufferOverlay IFramebuffer::overlay`

An alpha-blended overlay which is superposed over the frame buffer. The initial purpose is to allow the display of icons providing information about the VM state, including disk activity, in front ends which do not have other means of doing that. The overlay is designed to controlled exclusively by IDisplay. It has no locking of its own, and any changes made to it are not guaranteed to be visible until the affected portion of IFramebuffer is updated. The overlay can be created lazily the first time it is requested. This attribute can also return `null` to signal that the overlay is not implemented.

6.68.1.8 winId (read-only)

`long long IFramebuffer::winId`

Platform-dependent identifier of the window where context of this frame buffer is drawn, or zero if there's no such window.

6.68.1.9 capabilities (read-only)

`FramebufferCapabilities IFramebuffer::capabilities[]`

Capabilities of the framebuffer instance.
For the meaning of individual capability flags see FramebufferCapabilities.

6.68.2 getVisibleRegion

> **Note:** This method is not supported in the web service.

```
unsigned long IFramebuffer::getVisibleRegion(
            [in] [ptr] octet rectangles,
            [in] unsigned long count)
```

rectangles Pointer to the RTRECT array to receive region data.

count Number of RTRECT elements in the rectangles array.

Returns the visible region of this frame buffer.

If the rectangles parameter is null then the value of the count parameter is ignored and the number of elements necessary to describe the current visible region is returned in countCopied.

If rectangles is not null but count is less than the required number of elements to store region data, the method will report a failure. If count is equal or greater than the required number of elements, then the actual number of elements copied to the provided array will be returned in countCopied.

> **Note:** The address of the provided array must be in the process space of this IFramebuffer object.

> **Note:** Method not yet implemented.

6.68.3 notify3DEvent

```
void IFramebuffer::notify3DEvent(
            [in] unsigned long type,
            [in] octet data[])
```

type event type. VBOX3D_NOTIFY_TYPE_* in VBoxVideo3D.h

data event-specific data, depends on the supplied event type

Notifies framebuffer about 3D backend event.

6.68.4 notifyChange

```
void IFramebuffer::notifyChange(
            [in] unsigned long screenId,
            [in] unsigned long xOrigin,
            [in] unsigned long yOrigin,
            [in] unsigned long width,
            [in] unsigned long height)
```

screenId Logical guest screen number.

xOrigin Location of the screen in the guest.

yOrigin Location of the screen in the guest.

width Width of the guest display, in pixels.

height Height of the guest display, in pixels.

Requests a size change.

6.68.5 notifyUpdate

```
void IFramebuffer::notifyUpdate(
            [in] unsigned long x,
            [in] unsigned long y,
            [in] unsigned long width,
            [in] unsigned long height)
```

x X position of update.

y Y position of update.

width Width of update.

height Height of update.

Informs about an update. Gets called by the display object where this buffer is registered.

6.68.6 notifyUpdateImage

```
void IFramebuffer::notifyUpdateImage(
            [in] unsigned long x,
            [in] unsigned long y,
            [in] unsigned long width,
            [in] unsigned long height,
            [in] octet image[])
```

x X position of update.

y Y position of update.

width Width of update.

height Height of update.

image Array with 32BPP image data.

Informs about an update and provides 32bpp bitmap.

6.68.7 processVHWACommand

> **Note:** This method is not supported in the web service.

```
void IFramebuffer::processVHWACommand(
            [in] [ptr] octet command,
            [in] long enmCmd,
            [in] boolean fromGuest)
```

command Pointer to VBOXVHWACMD containing the command to execute.

enmCmd The validated VBOXVHWACMD::enmCmd value from the command.

fromGuest Set when the command origins from the guest, clear if host.

Posts a Video HW Acceleration Command to the frame buffer for processing. The commands used for 2D video acceleration (DDraw surface creation/destroying, blitting, scaling, color conversion, overlaying, etc.) are posted from quest to the host to be processed by the host hardware.

> **Note:** The address of the provided command must be in the process space of this IFramebuffer object.

6.68.8 setVisibleRegion

> **Note:** This method is not supported in the web service.

```
void IFramebuffer::setVisibleRegion(
            [in] [ptr] octet rectangles,
            [in] unsigned long count)
```

rectangles Pointer to the RTRECT array.

count Number of RTRECT elements in the rectangles array.

Suggests a new visible region to this frame buffer. This region represents the area of the VM display which is a union of regions of all top-level windows of the guest operating system running inside the VM (if the Guest Additions for this system support this functionality). This information may be used by the frontends to implement the seamless desktop integration feature.

> **Note:** The address of the provided array must be in the process space of this IFramebuffer object.

> **Note:** The IFramebuffer implementation must make a copy of the provided array of rectangles.

> **Note:** Method not yet implemented.

6.68.9 videoModeSupported

```
boolean IFramebuffer::videoModeSupported(
            [in] unsigned long width,
            [in] unsigned long height,
            [in] unsigned long bpp)
```

width

height

bpp

Returns whether the frame buffer implementation is willing to support a given video mode. In case it is not able to render the video mode (or for some reason not willing), it should return false. Usually this method is called when the guest asks the VMM device whether a given video mode is supported so the information returned is directly exposed to the guest. It is important that this method returns very quickly.

6.69 IFramebufferOverlay (IFramebuffer)

> **Note:** This interface extends IFramebuffer and therefore supports all its methods and attributes as well.

The IFramebufferOverlay interface represents an alpha blended overlay for displaying status icons above an IFramebuffer. It is always created not visible, so that it must be explicitly shown. It only covers a portion of the IFramebuffer, determined by its width, height and co-ordinates. It is always in packed pixel little-endian 32bit ARGB (in that order) format, and may be written to directly. Do re-read the width though, after setting it, as it may be adjusted (increased) to make it more suitable for the front end.

6.69.1 Attributes

6.69.1.1 x (read-only)

```
unsigned long IFramebufferOverlay::x
```

X position of the overlay, relative to the frame buffer.

6.69.1.2 y (read-only)

```
unsigned long IFramebufferOverlay::y
```

Y position of the overlay, relative to the frame buffer.

6.69.1.3 visible (read/write)

```
boolean IFramebufferOverlay::visible
```

Whether the overlay is currently visible.

6.69.1.4 alpha (read/write)

```
unsigned long IFramebufferOverlay::alpha
```

The global alpha value for the overlay. This may or may not be supported by a given front end.

6.69.2 move

```
void IFramebufferOverlay::move(
           [in] unsigned long x,
           [in] unsigned long y)
```

x

y

Changes the overlay's position relative to the IFramebuffer.

6.70 IFsInfo

Abstract parent interface for VirtualBox file system information. This can be information about a host or guest file system, for example.

6.70.1 Attributes

6.70.1.1 freeSize (read-only)

```
long long IFsInfo::freeSize
```

Remaining free space (in bytes) of the filesystem.

6.70.1.2 totalSize (read-only)

```
long long IFsInfo::totalSize
```

Total space (in bytes) of the filesystem.

6.70.1.3 blockSize (read-only)

```
unsigned long IFsInfo::blockSize
```

Block size (in bytes) of the filesystem.

6.70.1.4 sectorSize (read-only)

```
unsigned long IFsInfo::sectorSize
```

Sector size (in bytes) of the filesystem.

6.70.1.5 serialNumber (read-only)

```
unsigned long IFsInfo::serialNumber
```

Serial number of the filesystem.

6.70.1.6 isRemote (read-only)

```
boolean IFsInfo::isRemote
```

`TRUE` if the filesystem is remote, `FALSE` if the filesystem is local.

6.70.1.7 isCaseSensitive (read-only)

```
boolean IFsInfo::isCaseSensitive
```

`TRUE` if the filesystem is case sensitive, `FALSE` if the filesystem is case insensitive.

6.70.1.8 isReadOnly (read-only)

```
boolean IFsInfo::isReadOnly
```

`TRUE` if the filesystem is mounted read only, `FALSE` if the filesystem is mounted read write.

6.70.1.9 isCompressed (read-only)

```
boolean IFsInfo::isCompressed
```

`TRUE` if the filesystem is compressed, `FALSE` if it isn't or we don't know.

6.70.1.10 supportsFileCompression (read-only)

```
boolean IFsInfo::supportsFileCompression
```

TRUE if the filesystem compresses of individual files, FALSE if it doesn't or we don't know.

6.70.1.11 maxComponent (read-only)

```
unsigned long IFsInfo::maxComponent
```

The maximum size of a filesystem object name.

6.70.1.12 type (read-only)

```
wstring IFsInfo::type
```

Name of the filesystem.

6.70.1.13 label (read-only)

```
wstring IFsInfo::label
```

Label of the filesystem.

6.70.1.14 mountPoint (read-only)

```
wstring IFsInfo::mountPoint
```

Mount point of the filesystem.

6.71 IFsObjInfo

Abstract parent interface for VirtualBox file system object information. This can be information about a file or a directory, for example.

6.71.1 Attributes

6.71.1.1 name (read-only)

```
wstring IFsObjInfo::name
```

The object's name.

6.71.1.2 type (read-only)

```
FsObjType IFsObjInfo::type
```

The object type. See FsObjType for more.

6.71.1.3 fileAttributes (read-only)

```
wstring IFsObjInfo::fileAttributes
```

File attributes.

6.71.1.4 objectSize (read-only)

```
long long IFsObjInfo::objectSize
```

The logical size (st_size). For normal files this is the size of the file. For symbolic links, this is the length of the path name contained in the symbolic link. For other objects this fields needs to be specified.

6.71.1.5 allocatedSize (read-only)

```
long long IFsObjInfo::allocatedSize
```

Disk allocation size (st_blocks * DEV_BSIZE).

6.71.1.6 accessTime (read-only)

```
long long IFsObjInfo::accessTime
```

Time of last access (st_atime).

6.71.1.7 birthTime (read-only)

```
long long IFsObjInfo::birthTime
```

Time of file birth (st_birthtime).

6.71.1.8 changeTime (read-only)

```
long long IFsObjInfo::changeTime
```

Time of last status change (st_ctime).

6.71.1.9 modificationTime (read-only)

```
long long IFsObjInfo::modificationTime
```

Time of last data modification (st_mtime).

6.71.1.10 UID (read-only)

```
long IFsObjInfo::UID
```

The user owning the filesystem object (st_uid). This is -1 if not available.

6.71.1.11 userName (read-only)

```
wstring IFsObjInfo::userName
```

The user name.

6.71.1.12 GID (read-only)

```
long IFsObjInfo::GID
```

The group the filesystem object is assigned (st_gid). This is -1 if not available.

6.71.1.13 groupName (read-only)

```
wstring IFsObjInfo::groupName
```

The group name.

6.71.1.14 nodeId (read-only)

```
long long IFsObjInfo::nodeId
```

The unique identifier (within the filesystem) of this filesystem object (st_ino). This is zero if not availalbe.

6.71.1.15 nodeIdDevice (read-only)

```
unsigned long IFsObjInfo::nodeIdDevice
```

The device number of the device which this filesystem object resides on (st_dev).

6.71.1.16 hardLinks (read-only)

```
unsigned long IFsObjInfo::hardLinks
```

Number of hard links to this filesystem object (st_nlink).

6.71.1.17 deviceNumber (read-only)

```
unsigned long IFsObjInfo::deviceNumber
```

The device number of a character or block device type object (st_rdev).

6.71.1.18 generationId (read-only)

```
unsigned long IFsObjInfo::generationId
```

The current generation number (st_gen).

6.71.1.19 userFlags (read-only)

```
unsigned long IFsObjInfo::userFlags
```

User flags (st_flags).

6.72 IGraphicsAdapter

The IGraphicsAdapter interface represents the graphics adapter of the virtual machine.

6.72.1 Attributes

6.72.1.1 graphicsControllerType (read/write)

```
GraphicsControllerType IGraphicsAdapter::graphicsControllerType
```

Graphics controller type.

6.72.1.2 VRAMSize (read/write)

```
unsigned long IGraphicsAdapter::VRAMSize
```

Video memory size in megabytes.

6.72.1.3 monitorCount (read/write)

```
unsigned long IGraphicsAdapter::monitorCount
```

Number of virtual monitors.

> **Note:** Only effective on Windows XP and later guests with Guest Additions installed.

6.72.2 isFeatureEnabled

```
boolean IGraphicsAdapter::isFeatureEnabled(
            [in] GraphicsFeature feature)
```

feature Feature to check for.

Returns whether a particular feature is enabled for this adapter or not.

6.72.3 setFeature

```
void IGraphicsAdapter::setFeature(
            [in] GraphicsFeature feature,
            [in] boolean enabled)
```

feature Graphics controller feature to set.

enabled Whether to enable or disable the feature.

Sets a graphics controller feature.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: The feature is not supported on the current platform.

6.73 IGuest

The IGuest interface represents information about the operating system running inside the virtual machine. Used in IConsole::guest.

IGuest provides information about the guest operating system, whether Guest Additions are installed and other OS-specific virtual machine properties.

6.73.1 Attributes

6.73.1.1 OSTypeId (read-only)

```
wstring IGuest::OSTypeId
```

Identifier of the Guest OS type as reported by the Guest Additions. You may use IVirtualBox::getGuestOSType() to obtain an IGuestOSType object representing details about the given Guest OS type.

> **Note:** If Guest Additions are not installed, this value will be the same as IMachine::OSTypeId.

6.73.1.2 additionsRunLevel (read-only)

`AdditionsRunLevelType IGuest::additionsRunLevel`

Current run level of the installed Guest Additions.

6.73.1.3 additionsVersion (read-only)

`wstring IGuest::additionsVersion`

Version of the installed Guest Additions in the same format as IVirtualBox::version.

6.73.1.4 additionsRevision (read-only)

`unsigned long IGuest::additionsRevision`

The internal build revision number of the installed Guest Additions.
See also IVirtualBox::revision.

6.73.1.5 dnDSource (read-only)

`IGuestDnDSource IGuest::dnDSource`

Retrieves the drag'n drop source implementation for the guest side, that is, handling and retrieving drag'n drop data from the guest.

6.73.1.6 dnDTarget (read-only)

`IGuestDnDTarget IGuest::dnDTarget`

Retrieves the drag'n drop source implementation for the host side. This will allow the host to handle and initiate a drag'n drop operation to copy data from the host to the guest.

6.73.1.7 eventSource (read-only)

`IEventSource IGuest::eventSource`

Event source for guest events.

6.73.1.8 facilities (read-only)

`IAdditionsFacility IGuest::facilities[]`

Returns a collection of current known facilities. Only returns facilities where a status is known, e.g. facilities with an unknown status will not be returned.

6.73.1.9 sessions (read-only)

`IGuestSession IGuest::sessions[]`

Returns a collection of all opened guest sessions.

6.73.1.10 memoryBalloonSize (read/write)

`unsigned long IGuest::memoryBalloonSize`

Guest system memory balloon size in megabytes (transient property).

6.73.1.11 statisticsUpdateInterval (read/write)

```
unsigned long IGuest::statisticsUpdateInterval
```

Interval to update guest statistics in seconds.

6.73.2 createSession

```
IGuestSession IGuest::createSession(
            [in] wstring user,
            [in] wstring password,
            [in] wstring domain,
            [in] wstring sessionName)
```

user User name this session will be using to control the guest; has to exist and have the appropriate rights to execute programs in the VM. Must not be empty.

password Password of the user account to be used. Empty passwords are allowed.

domain Domain name of the user account to be used if the guest is part of a domain. Optional. This feature is not implemented yet.

sessionName The session's friendly name. Optional, can be empty.

Creates a new guest session for controlling the guest. The new session will be started asynchronously, meaning on return of this function it is not guaranteed that the guest session is in a started and/or usable state. To wait for successful startup, use the IGuestSession::waitFor() call.

A guest session represents one impersonated user account in the guest, so every operation will use the same credentials specified when creating the session object via createSession(). Anonymous sessions, that is, sessions without specifying a valid user account in the guest are not allowed reasons of security.

There can be a maximum of 32 sessions at once per VM. An error will be returned if this has been reached.

For more information please consult IGuestSession

If this method fails, the following error codes may be reported:

- VBOX_E_IPRT_ERROR: Error creating guest session.
- VBOX_E_MAXIMUM_REACHED: The maximum of concurrent guest sessions has been reached.

6.73.3 findSession

```
IGuestSession[] IGuest::findSession(
            [in] wstring sessionName)
```

sessionName The session's friendly name to find. Wildcards like ? and * are allowed.

Finds guest sessions by their friendly name and returns an interface array with all found guest sessions.

6.73.4 getAdditionsStatus

```
boolean IGuest::getAdditionsStatus(
            [in] AdditionsRunLevelType level)
```

level Status level to check

Retrieve the current status of a certain Guest Additions run level.

If this method fails, the following error codes may be reported:

- VBOX_E_NOT_SUPPORTED: Wrong status level specified.

6.73.5 getFacilityStatus

```
AdditionsFacilityStatus IGuest::getFacilityStatus(
           [in] AdditionsFacilityType facility,
           [out] long long timestamp)
```

facility Facility to check status for.

timestamp Timestamp (in ms) of last status update seen by the host.

Get the current status of a Guest Additions facility.

6.73.6 internalGetStatistics

```
void IGuest::internalGetStatistics(
           [out] unsigned long cpuUser,
           [out] unsigned long cpuKernel,
           [out] unsigned long cpuIdle,
           [out] unsigned long memTotal,
           [out] unsigned long memFree,
           [out] unsigned long memBalloon,
           [out] unsigned long memShared,
           [out] unsigned long memCache,
           [out] unsigned long pagedTotal,
           [out] unsigned long memAllocTotal,
           [out] unsigned long memFreeTotal,
           [out] unsigned long memBalloonTotal,
           [out] unsigned long memSharedTotal)
```

cpuUser Percentage of processor time spent in user mode as seen by the guest.

cpuKernel Percentage of processor time spent in kernel mode as seen by the guest.

cpuIdle Percentage of processor time spent idling as seen by the guest.

memTotal Total amount of physical guest RAM.

memFree Free amount of physical guest RAM.

memBalloon Amount of ballooned physical guest RAM.

memShared Amount of shared physical guest RAM.

memCache Total amount of guest (disk) cache memory.

pagedTotal Total amount of space in the page file.

memAllocTotal Total amount of memory allocated by the hypervisor.

memFreeTotal Total amount of free memory available in the hypervisor.

memBalloonTotal Total amount of memory ballooned by the hypervisor.

memSharedTotal Total amount of shared memory in the hypervisor.

Internal method; do not use as it might change at any time.

6.73.7 setCredentials

```
void IGuest::setCredentials(
            [in] wstring userName,
            [in] wstring password,
            [in] wstring domain,
            [in] boolean allowInteractiveLogon)
```

userName User name string, can be empty

password Password string, can be empty

domain Domain name (guest logon scheme specific), can be empty

allowInteractiveLogon Flag whether the guest should alternatively allow the user to interactively specify different credentials. This flag might not be supported by all versions of the Additions.

Store login credentials that can be queried by guest operating systems with Additions installed. The credentials are transient to the session and the guest may also choose to erase them. Note that the caller cannot determine whether the guest operating system has queried or made use of the credentials.

If this method fails, the following error codes may be reported:

- VBOX_E_VM_ERROR: VMM device is not available.

6.73.8 shutdown

```
void IGuest::shutdown(
            [in] GuestShutdownFlag flags[])
```

flags GuestShutdownFlag flags.

Shuts down (and optionally halts and/or reboots) the guest. Needs supported Guest Additions installed.

If this method fails, the following error codes may be reported:

- VBOX_E_NOT_SUPPORTED: Guest OS is not supported for shutting down, or the already installed Guest Additions are not supported.
- VBOX_E_IPRT_ERROR: Error while shutting down.

6.73.9 updateGuestAdditions

```
IProgress IGuest::updateGuestAdditions(
            [in] wstring source,
            [in] wstring arguments[],
            [in] AdditionsUpdateFlag flags[])
```

source Path to the Guest Additions .ISO file to use for the update.

arguments Optional command line arguments to use for the Guest Additions installer. Useful for retrofitting features which weren't installed before in the guest.

flags AdditionsUpdateFlag flags.

Automatically updates already installed Guest Additions in a VM.
At the moment only Windows and Linux guests are supported.
Because the VirtualBox Guest Additions drivers are not WHQL-certified yet there might be warning dialogs during the actual Guest Additions update. These need to be confirmed manually in order to continue the installation process. This applies to Windows 2000 and Windows XP guests and therefore these guests can't be updated in a fully automated fashion without user interaction. However, to start a Guest Additions update for the mentioned Windows versions anyway, the flag AdditionsUpdateFlag_WaitForUpdateStartOnly can be specified. See AdditionsUpdateFlag for more information.
The guest needs to be restarted in order to make use of the updated Guest Additions.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: Guest OS is not supported for automated Guest Additions updates or the already installed Guest Additions are not ready yet.
- `VBOX_E_IPRT_ERROR`: Error while updating.

6.74 IGuestAdditionsStatusChangedEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

The guest addition status changed.

6.74.1 Attributes

6.74.1.1 facility (read-only)

```
AdditionsFacilityType IGuestAdditionsStatusChangedEvent::facility
```

Facility this event relates to.

6.74.1.2 status (read-only)

```
AdditionsFacilityStatus IGuestAdditionsStatusChangedEvent::status
```

The new facility status.

6.74.1.3 runLevel (read-only)

```
AdditionsRunLevelType IGuestAdditionsStatusChangedEvent::runLevel
```

The new run level.

6.74.1.4 timestamp (read-only)

```
long long IGuestAdditionsStatusChangedEvent::timestamp
```

The millisecond timestamp associated with the event.

6.75 IGuestDebugControl

Controls the guest debug settings of one virtual machine.

6.75.1 Attributes

6.75.1.1 debugProvider (read/write)

`GuestDebugProvider IGuestDebugControl::debugProvider`

The currently active debug provider.

6.75.1.2 debugIoProvider (read/write)

`GuestDebugIoProvider IGuestDebugControl::debugIoProvider`

The I/O backend for the selected debug provider.

6.75.1.3 debugAddress (read/write)

`wstring IGuestDebugControl::debugAddress`

The address to connect to or listen on, depending on the type.

6.75.1.4 debugPort (read/write)

`unsigned long IGuestDebugControl::debugPort`

The port to listen on or connect to, depending on the selected I/O provider. Might be ignored by some providers.

6.76 IGuestDebugControlChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a property of the guest debug settings changes. Interested callees should use IGuestDebugControl methods and attributes to find out what has changed.

6.76.1 Attributes

6.76.1.1 guestDebugControl (read-only)

`IGuestDebugControl IGuestDebugControlChangedEvent::guestDebugControl`

Guest debug control object that is subject to change.

6.77 IGuestDirectory (IDirectory)

> **Note:** This interface extends IDirectory and therefore supports all its methods and attributes as well.

Implementation of the IDirectory object for directories in the guest.

6.77.1 Attributes

6.77.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

`boolean IGuestDirectory::midlDoesNotLikeEmptyInterfaces`

6.78 IGuestDirectoryEvent (IGuestSessionEvent)

> **Note:** This interface extends IGuestSessionEvent and therefore supports all its methods and attributes as well.

Base abstract interface for all guest directory events.

6.78.1 Attributes

6.78.1.1 directory (read-only)

`IGuestDirectory IGuestDirectoryEvent::directory`

Guest directory object which is related to this event.

6.79 IGuestDirectoryReadEvent (IGuestDirectoryEvent)

> **Note:** This interface extends IGuestDirectoryEvent and therefore supports all its methods and attributes as well.

Notification when a directory entry has been read.

6.79.1 Attributes

6.79.1.1 path (read-only)

`wstring IGuestDirectoryReadEvent::path`

Path of the directory entry.

6.79.1.2 objInfo (read-only)

`IFsObjInfo IGuestDirectoryReadEvent::objInfo`

Object information of the directory entry. Also see IFsObjInfo.

6.79.1.3 owner (read-only)

`wstring IGuestDirectoryReadEvent::owner`

Resolved owner name of the directory entry.

6.79.1.4 groups (read-only)

`wstring IGuestDirectoryReadEvent::groups`

Resolved user groups of the directory entry.

6.80 IGuestDirectoryRegisteredEvent (IGuestDirectoryEvent)

> **Note:** This interface extends IGuestDirectoryEvent and therefore supports all its methods and attributes as well.

Notification when a guest directory was registered or unregistered.

6.80.1 Attributes

6.80.1.1 registered (read-only)

`boolean IGuestDirectoryRegisteredEvent::registered`

If `true`, the guest directory was registered, otherwise it was unregistered.

6.81 IGuestDirectoryStateChangedEvent (IGuestDirectoryEvent)

> **Note:** This interface extends IGuestDirectoryEvent and therefore supports all its methods and attributes as well.

Notification when a guest directory changed its state.

6.81.1 Attributes

6.81.1.1 status (read-only)

`DirectoryStatus IGuestDirectoryStateChangedEvent::status`

New guest directory status.

6.81.1.2 error (read-only)

`IVirtualBoxErrorInfo IGuestDirectoryStateChangedEvent::error`

Error information in case of new session status is indicating an error.
The attribute IVirtualBoxErrorInfo::resultDetail will contain the runtime (IPRT) error code from the guest. See include/iprt/err.h and include/VBox/err.h for details.

6.82 IGuestDnDSource (IDnDSource)

> **Note:** This interface extends IDnDSource and therefore supports all its methods and attributes as well.

Implementation of the IDnDSource object for source drag'n drop operations on the guest.

6.82.1 Attributes

6.82.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

`boolean IGuestDnDSource::midlDoesNotLikeEmptyInterfaces`

6.83 IGuestDnDTarget (IDnDTarget)

> **Note:** This interface extends IDnDTarget and therefore supports all its methods and attributes as well.

Implementation of the IDnDTarget object for target drag'n drop operations on the guest.

6.83.1 Attributes

6.83.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IGuestDnDTarget::midlDoesNotLikeEmptyInterfaces
```

6.84 IGuestFile (IFile)

> **Note:** This interface extends IFile and therefore supports all its methods and attributes as well.

Implementation of the IFile object for files in the guest.

6.84.1 Attributes

6.84.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IGuestFile::midlDoesNotLikeEmptyInterfaces
```

6.85 IGuestFileEvent (IGuestSessionEvent)

> **Note:** This interface extends IGuestSessionEvent and therefore supports all its methods and attributes as well.

Base abstract interface for all guest file events.

6.85.1 Attributes

6.85.1.1 file (read-only)

```
IGuestFile IGuestFileEvent::file
```

Guest file object which is related to this event.

6.86 IGuestFileIOEvent (IGuestFileEvent)

> **Note:** This interface extends IGuestFileEvent and therefore supports all its methods and attributes as well.

Base abstract interface for all guest file input/output (IO) events.

6.86.1 Attributes

6.86.1.1 offset (read-only)

```
long long IGuestFileIOEvent::offset
```

Current offset (in bytes).

6.86.1.2 processed (read-only)

`unsigned long IGuestFileIOEvent::processed`

Processed input or output (in bytes).

6.87 IGuestFileOffsetChangedEvent (IGuestFileIOEvent)

> **Note:** This interface extends IGuestFileIOEvent and therefore supports all its methods and attributes as well.

Notification when a guest file changed its current offset via IFile::seek().

6.87.1 Attributes

6.87.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

`boolean IGuestFileOffsetChangedEvent::midlDoesNotLikeEmptyInterfaces`

6.88 IGuestFileReadEvent (IGuestFileIOEvent)

> **Note:** This interface extends IGuestFileIOEvent and therefore supports all its methods and attributes as well.

Notification when data has been read from a guest file.

6.88.1 Attributes

6.88.1.1 data (read-only)

`octet IGuestFileReadEvent::data[]`

Actual data read.

6.89 IGuestFileRegisteredEvent (IGuestFileEvent)

> **Note:** This interface extends IGuestFileEvent and therefore supports all its methods and attributes as well.

Notification when a guest file was registered or unregistered.

6.89.1 Attributes

6.89.1.1 registered (read-only)

`boolean IGuestFileRegisteredEvent::registered`

If `true`, the guest file was registered, otherwise it was unregistered.

6.90 IGuestFileSizeChangedEvent (IGuestFileEvent)

> **Note:** This interface extends IGuestFileEvent and therefore supports all its methods and attributes as well.

Notification when a guest file changed its size via IFile::setSize().

6.90.1 Attributes

6.90.1.1 newSize (read-only)

```
long long IGuestFileSizeChangedEvent::newSize
```

6.91 IGuestFileStateChangedEvent (IGuestFileEvent)

> **Note:** This interface extends IGuestFileEvent and therefore supports all its methods and attributes as well.

Notification when a guest file changed its state.

6.91.1 Attributes

6.91.1.1 status (read-only)

```
FileStatus IGuestFileStateChangedEvent::status
```

New guest file status.

6.91.1.2 error (read-only)

```
IVirtualBoxErrorInfo IGuestFileStateChangedEvent::error
```

Error information in case of new session status is indicating an error.
The attribute IVirtualBoxErrorInfo::resultDetail will contain the runtime (IPRT) error code from the guest. See include/iprt/err.h and include/VBox/err.h for details.

6.92 IGuestFileWriteEvent (IGuestFileIOEvent)

> **Note:** This interface extends IGuestFileIOEvent and therefore supports all its methods and attributes as well.

Notification when data has been written to a guest file.

6.92.1 Attributes

6.92.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IGuestFileWriteEvent::midlDoesNotLikeEmptyInterfaces
```

6.93 IGuestFsInfo (IFsInfo)

> **Note:** This interface extends IFsInfo and therefore supports all its methods and attributes as well.

Represents the guest implementation of the IFsInfo object.

6.93.1 Attributes

6.93.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IGuestFsInfo::midlDoesNotLikeEmptyInterfaces
```

6.94 IGuestFsObjInfo (IFsObjInfo)

> **Note:** This interface extends IFsObjInfo and therefore supports all its methods and attributes as well.

Represents the guest implementation of the IFsObjInfo object.

6.94.1 Attributes

6.94.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IGuestFsObjInfo::midlDoesNotLikeEmptyInterfaces
```

6.95 IGuestKeyboardEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when guest keyboard event happens.

6.95.1 Attributes

6.95.1.1 scancodes (read-only)

```
long IGuestKeyboardEvent::scancodes[]
```

Array of scancodes.

6.96 IGuestMonitorChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the guest enables one of its monitors.

6.96.1 Attributes

6.96.1.1 changeType (read-only)

`GuestMonitorChangedEventType IGuestMonitorChangedEvent::changeType`

What was changed for this guest monitor.

6.96.1.2 screenId (read-only)

`unsigned long IGuestMonitorChangedEvent::screenId`

The monitor which was changed.

6.96.1.3 originX (read-only)

`unsigned long IGuestMonitorChangedEvent::originX`

Physical X origin relative to the primary screen. Valid for Enabled and NewOrigin.

6.96.1.4 originY (read-only)

`unsigned long IGuestMonitorChangedEvent::originY`

Physical Y origin relative to the primary screen. Valid for Enabled and NewOrigin.

6.96.1.5 width (read-only)

`unsigned long IGuestMonitorChangedEvent::width`

Width of the screen. Valid for Enabled.

6.96.1.6 height (read-only)

`unsigned long IGuestMonitorChangedEvent::height`

Height of the screen. Valid for Enabled.

6.97 IGuestMonitorInfoChangedEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

The guest reports cursor position data.

6.97.1 Attributes

6.97.1.1 output (read-only)

`unsigned long IGuestMonitorInfoChangedEvent::output`

The virtual display output on which the monitor has changed.

6.98 IGuestMouseEvent (IReusableEvent)

> **Note:** This interface extends IReusableEvent and therefore supports all its methods and attributes as well.

Notification when guest mouse event happens.

6.98.1 Attributes

6.98.1.1 mode (read-only)

`GuestMouseEventMode IGuestMouseEvent::mode`

If this event is relative, absolute or multi-touch.

6.98.1.2 x (read-only)

`long IGuestMouseEvent::x`

New X position, or X delta.

6.98.1.3 y (read-only)

`long IGuestMouseEvent::y`

New Y position, or Y delta.

6.98.1.4 z (read-only)

`long IGuestMouseEvent::z`

Z delta.

6.98.1.5 w (read-only)

`long IGuestMouseEvent::w`

W delta.

6.98.1.6 buttons (read-only)

`long IGuestMouseEvent::buttons`

Button state bitmask.

6.99 IGuestMultiTouchEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when guest touch screen event happens.

6.99.1 Attributes

6.99.1.1 contactCount (read-only)

```
long IGuestMultiTouchEvent::contactCount
```

Number of contacts in the event.

6.99.1.2 xPositions (read-only)

```
short IGuestMultiTouchEvent::xPositions[]
```

X positions.

6.99.1.3 yPositions (read-only)

```
short IGuestMultiTouchEvent::yPositions[]
```

Y positions.

6.99.1.4 contactIds (read-only)

```
unsigned short IGuestMultiTouchEvent::contactIds[]
```

Contact identifiers.

6.99.1.5 contactFlags (read-only)

```
unsigned short IGuestMultiTouchEvent::contactFlags[]
```

Contact state. Bit 0: in contact. Bit 1: in range.

6.99.1.6 isTouchScreen (read-only)

```
boolean IGuestMultiTouchEvent::isTouchScreen
```

Distinguishes between touchscreen and touchpad events.

6.99.1.7 scanTime (read-only)

```
unsigned long IGuestMultiTouchEvent::scanTime
```

Timestamp of the event in milliseconds. Only relative time between events is important.

6.100 IGuestOSType

Note: With the web service, this interface is mapped to a structure. Attributes that return this interface will not return an object, but a complete structure containing the attributes listed below as structure members.

Specifies a guest OS type and its recommendations.

6.100.1 Attributes

6.100.1.1 familyId (read-only)

```
wstring IGuestOSType::familyId
```

Guest OS family identifier string.

6.100.1.2 familyDescription (read-only)

```
wstring IGuestOSType::familyDescription
```

Human readable description of the guest OS family.

6.100.1.3 id (read-only)

```
wstring IGuestOSType::id
```

Guest OS identifier string.

6.100.1.4 subtype (read-only)

```
wstring IGuestOSType::subtype
```

Guest OS subtype string.

6.100.1.5 description (read-only)

```
wstring IGuestOSType::description
```

Human readable description of the guest OS.

6.100.1.6 is64Bit (read-only)

```
boolean IGuestOSType::is64Bit
```

Returns `true` if the given OS is 64-bit

6.100.1.7 platformArchitecture (read-only)

```
PlatformArchitecture IGuestOSType::platformArchitecture
```

Returns the guest OS platform architecture.

6.100.1.8 recommendedIOAPIC (read-only)

```
boolean IGuestOSType::recommendedIOAPIC
```

Returns `true` if I/O-APIC recommended for this OS type. Only applies to x86-based platforms.

6.100.1.9 recommendedVirtEx (read-only)

```
boolean IGuestOSType::recommendedVirtEx
```

Returns `true` if VT-x or AMD-V recommended for this OS type. Only applies to x86-based platforms.

6.100.1.10 recommendedRAM (read-only)

`unsigned long IGuestOSType::recommendedRAM`

Recommended RAM size in Megabytes.

6.100.1.11 recommendedGraphicsController (read-only)

`GraphicsControllerType IGuestOSType::recommendedGraphicsController`

Recommended graphics controller type.

6.100.1.12 recommendedVRAM (read-only)

`unsigned long IGuestOSType::recommendedVRAM`

Recommended video RAM size in Megabytes.

6.100.1.13 recommended3DAcceleration (read-only)

`boolean IGuestOSType::recommended3DAcceleration`

Returns `true` if 3D acceleration is recommended for this OS type.

6.100.1.14 recommendedHDD (read-only)

`long long IGuestOSType::recommendedHDD`

Recommended hard disk size in bytes.

6.100.1.15 adapterType (read-only)

`NetworkAdapterType IGuestOSType::adapterType`

Returns recommended network adapter for this OS type.

6.100.1.16 recommendedPAE (read-only)

`boolean IGuestOSType::recommendedPAE`

Returns `true` if using PAE is recommended for this OS type. Only applies to x86-based platforms.

6.100.1.17 recommendedDVDStorageController (read-only)

`StorageControllerType IGuestOSType::recommendedDVDStorageController`

Recommended storage controller type for DVD/CD drives.

6.100.1.18 recommendedDVDStorageBus (read-only)

`StorageBus IGuestOSType::recommendedDVDStorageBus`

Recommended storage bus type for DVD/CD drives.

6.100.1.19 recommendedHDStorageController (read-only)

`StorageControllerType IGuestOSType::recommendedHDStorageController`

Recommended storage controller type for HD drives.

6.100.1.20 recommendedHDStorageBus (read-only)

`StorageBus IGuestOSType::recommendedHDStorageBus`

Recommended storage bus type for HD drives.

6.100.1.21 recommendedFirmware (read-only)

`FirmwareType IGuestOSType::recommendedFirmware`

Recommended firmware type.

6.100.1.22 recommendedUSBHID (read-only)

`boolean IGuestOSType::recommendedUSBHID`

Returns `true` if using USB Human Interface Devices, such as keyboard and mouse recommended.

6.100.1.23 recommendedHPET (read-only)

`boolean IGuestOSType::recommendedHPET`

Returns `true` if using HPET is recommended for this OS type. Only applies to x86-based platforms.

6.100.1.24 recommendedUSBTablet (read-only)

`boolean IGuestOSType::recommendedUSBTablet`

Returns `true` if using a USB Tablet is recommended.

6.100.1.25 recommendedRTCUseUTC (read-only)

`boolean IGuestOSType::recommendedRTCUseUTC`

Returns `true` if the RTC of this VM should be set to UTC.

6.100.1.26 recommendedChipset (read-only)

`ChipsetType IGuestOSType::recommendedChipset`

Recommended chipset type.

6.100.1.27 recommendedIommuType (read-only)

`IommuType IGuestOSType::recommendedIommuType`

Recommended IOMMU type.

6.100.1.28 recommendedAudioController (read-only)

`AudioControllerType IGuestOSType::recommendedAudioController`

Recommended audio controller type.

6.100.1.29 recommendedAudioCodec (read-only)

`AudioCodecType IGuestOSType::recommendedAudioCodec`

Recommended audio codec type.

6.100.1.30 recommendedFloppy (read-only)

`boolean IGuestOSType::recommendedFloppy`

Returns `true` a floppy drive is recommended for this OS type.

6.100.1.31 recommendedUSB (read-only)

`boolean IGuestOSType::recommendedUSB`

Returns `true` a USB controller is recommended for this OS type.

6.100.1.32 recommendedUSB3 (read-only)

`boolean IGuestOSType::recommendedUSB3`

Returns `true` an xHCI (USB 3) controller is recommended for this OS type.

6.100.1.33 recommendedTFReset (read-only)

`boolean IGuestOSType::recommendedTFReset`

Returns `true` if using VCPU reset on triple fault is recommended for this OS type.

6.100.1.34 recommendedX2APIC (read-only)

`boolean IGuestOSType::recommendedX2APIC`

Returns `true` if X2APIC is recommended for this OS type. Only applies to x86-based platforms.

6.100.1.35 recommendedCPUCount (read-only)

`unsigned long IGuestOSType::recommendedCPUCount`

Number of vCPUs recommended for this OS type.

6.100.1.36 recommendedTpmType (read-only)

`TpmType IGuestOSType::recommendedTpmType`

Returns the recommended trusted platform module type for this OS type.

6.100.1.37 recommendedSecureBoot (read-only)

`boolean IGuestOSType::recommendedSecureBoot`

Returns `true` if EFI secure boot is recommended for this OS type.

6.100.1.38 recommendedWDDMGraphics (read-only)

`boolean IGuestOSType::recommendedWDDMGraphics`

Returns `true` if this OS usually has a WDDM graphics driver from guest additions.

6.100.1.39 guestAdditionsInstallPackageName (read-only)

`wstring IGuestOSType::guestAdditionsInstallPackageName`

The guest additions install package name as a string.

6.101 IGuestProcess (IProcess)

> **Note:** This interface extends IProcess and therefore supports all its methods and attributes as well.

Implementation of the IProcess object for processes the host has started in the guest.

6.101.1 Attributes

6.101.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

`boolean IGuestProcess::midlDoesNotLikeEmptyInterfaces`

6.102 IGuestProcessEvent (IGuestSessionEvent)

> **Note:** This interface extends IGuestSessionEvent and therefore supports all its methods and attributes as well.

Base abstract interface for all guest process events.

6.102.1 Attributes

6.102.1.1 process (read-only)

`IGuestProcess IGuestProcessEvent::process`

Guest process object which is related to this event.

6.102.1.2 pid (read-only)

`unsigned long IGuestProcessEvent::pid`

Guest process ID (PID).

6.103 IGuestProcessIOEvent (IGuestProcessEvent)

> **Note:** This interface extends IGuestProcessEvent and therefore supports all its methods and attributes as well.

Base abstract interface for all guest process input/output (IO) events.

6.103.1 Attributes

6.103.1.1 handle (read-only)

```
unsigned long IGuestProcessIOEvent::handle
```

Input/output (IO) handle involved in this event. Usually 0 is stdin, 1 is stdout and 2 is stderr.

6.103.1.2 processed (read-only)

```
unsigned long IGuestProcessIOEvent::processed
```

Processed input or output (in bytes).

6.104 IGuestProcessInputNotifyEvent (IGuestProcessIOEvent)

> **Note:** This interface extends IGuestProcessIOEvent and therefore supports all its methods and attributes as well.

Notification when a guest process' stdin became available.

> **Note:** This event is right now not implemented!

6.104.1 Attributes

6.104.1.1 status (read-only)

```
ProcessInputStatus IGuestProcessInputNotifyEvent::status
```

Current process input status.

6.105 IGuestProcessOutputEvent (IGuestProcessIOEvent)

> **Note:** This interface extends IGuestProcessIOEvent and therefore supports all its methods and attributes as well.

Notification when there is guest process output available for reading.

6.105.1 Attributes

6.105.1.1 data (read-only)

```
octet IGuestProcessOutputEvent::data[]
```

Actual output data.

6.106 IGuestProcessRegisteredEvent (IGuestProcessEvent)

Note: This interface extends IGuestProcessEvent and therefore supports all its methods and attributes as well.

Notification when a guest process was registered or unregistered.

6.106.1 Attributes

6.106.1.1 registered (read-only)

```
boolean IGuestProcessRegisteredEvent::registered
```

If `true`, the guest process was registered, otherwise it was unregistered.

6.107 IGuestProcessStateChangedEvent (IGuestProcessEvent)

Note: This interface extends IGuestProcessEvent and therefore supports all its methods and attributes as well.

Notification when a guest process changed its state.

6.107.1 Attributes

6.107.1.1 status (read-only)

```
ProcessStatus IGuestProcessStateChangedEvent::status
```

New guest process status.

6.107.1.2 error (read-only)

```
IVirtualBoxErrorInfo IGuestProcessStateChangedEvent::error
```

Error information in case of new session status is indicating an error.

The attribute IVirtualBoxErrorInfo::resultDetail will contain the runtime (IPRT) error code from the guest. See include/iprt/err.h and include/VBox/err.h for details.

6.108 IGuestPropertyChangedEvent (IMachineEvent)

Note: This interface extends IMachineEvent and therefore supports all its methods and attributes as well.

Notification when a guest property has changed.

6.108.1 Attributes

6.108.1.1 name (read-only)

```
wstring IGuestPropertyChangedEvent::name
```

The name of the property that has changed.

6.108.1.2 value (read-only)

```
wstring IGuestPropertyChangedEvent::value
```

The new property value.

6.108.1.3 flags (read-only)

```
wstring IGuestPropertyChangedEvent::flags
```

The new property flags.

6.108.1.4 fWasDeleted (read-only)

```
boolean IGuestPropertyChangedEvent::fWasDeleted
```

A flag which indicates that property was deleted.

6.109 IGuestScreenInfo

6.109.1 Attributes

6.109.1.1 screenId (read-only)

```
unsigned long IGuestScreenInfo::screenId
```

6.109.1.2 guestMonitorStatus (read-only)

```
GuestMonitorStatus IGuestScreenInfo::guestMonitorStatus
```

6.109.1.3 primary (read-only)

```
boolean IGuestScreenInfo::primary
```

6.109.1.4 origin (read-only)

```
boolean IGuestScreenInfo::origin
```

6.109.1.5 originX (read-only)

```
long IGuestScreenInfo::originX
```

6.109.1.6 originY (read-only)

```
long IGuestScreenInfo::originY
```

6.109.1.7 width (read-only)

```
unsigned long IGuestScreenInfo::width
```

6.109.1.8 height (read-only)

```
unsigned long IGuestScreenInfo::height
```

6.109.1.9 bitsPerPixel (read-only)

```
unsigned long IGuestScreenInfo::bitsPerPixel
```

6.109.1.10 extendedInfo (read-only)

```
wstring IGuestScreenInfo::extendedInfo
```

6.110 IGuestSession

A guest session represents one impersonated user account in the guest, so every operation will use the same credentials specified when creating the session object via IGuest::createSession().

There can be a maximum of 32 sessions at once per VM, whereas session 0 always is reserved for the root session (the root session is part of that limit).

This root session is controlling all other guest sessions and also is responsible for actions which require system level privileges.

Each guest session keeps track of the guest directories and files that it opened as well as guest processes it has created. To work on guest files or directories a guest session offers methods to open or create such objects (see fileOpen() or directoryOpen() for instance). Similarly, there a methods for creating guest processes.

There can be up to 2048 objects (guest processes, files and directories) a time per guest session. Exceeding the limit will result in an error (see the corresponding functions for more).

When done with either of these objects, including the guest session itself, use the appropriate close() method to let the object do its cleanup work.

Closing a session via close() will try to close all the mentioned objects above unless these objects are still used by a client.

A set of environment variables changes is associated with each session (environmentChanges[]). These are applied to the base environment of the impersonated guest user when creating a new guest process. For additional flexibility the processCreate() and processCreateEx() methods allows you to specify individual environment changes for each process you create. With newer guest addition versions, the base environment is also made available via environmentBase[]. (One reason for why we record changes to a base environment instead of working directly on an environment block is that we need to be compatible with older Guest Additions. Another reason is that this way it is always possible to undo all the changes you've scheduled.)

6.110.1 Attributes

6.110.1.1 user (read-only)

```
wstring IGuestSession::user
```

Returns the user name used by this session to impersonate users in the guest.

6.110.1.2 domain (read-only)

```
wstring IGuestSession::domain
```

Returns the domain name used by this session to impersonate users in the guest.

6.110.1.3 name (read-only)

```
wstring IGuestSession::name
```

Returns the session's friendly name.

6.110.1.4 id (read-only)

```
unsigned long IGuestSession::id
```

Returns the internal session ID.

6.110.1.5 timeout (read/write)

```
unsigned long IGuestSession::timeout
```

Gets or sets the session timeout (in ms).

6.110.1.6 protocolVersion (read-only)

```
unsigned long IGuestSession::protocolVersion
```

Returns the protocol version which is used by this session to communicate with the guest.

6.110.1.7 status (read-only)

```
GuestSessionStatus IGuestSession::status
```

Returns the current session status.

6.110.1.8 environmentChanges (read/write)

```
wstring IGuestSession::environmentChanges[]
```

The set of scheduled environment changes to the base environment of the session. They are in putenv format, i.e. "VAR=VALUE" for setting and "VAR" for unsetting. One entry per variable (change). The changes are applied when creating new guest processes.

This is writable, so to undo all the scheduled changes, assign it an empty array.

6.110.1.9 environmentBase (read-only)

```
wstring IGuestSession::environmentBase[]
```

The base environment of the session. They are on the "VAR=VALUE" form, one array entry per variable.

Access fails with VBOX_E_NOT_SUPPORTED if the Guest Additions do not support the session base environment feature. Support for this was introduced with protocol version XXXX.

Access fails with VBOX_E_INVALID_OBJECT_STATE if the Guest Additions has yet to report the session base environment.

6.110.1.10 processes (read-only)

```
IGuestProcess IGuestSession::processes[]
```

Returns all current guest processes.

6.110.1.11 pathStyle (read-only)

```
PathStyle IGuestSession::pathStyle
```

The style of paths used by the guest. Handy for giving the right kind of path specifications to fileOpen() and similar methods.

6.110.1.12 currentDirectory (read/write)

```
wstring IGuestSession::currentDirectory
```

Gets or sets the current directory of the session. Guest path style.

6.110.1.13 userHome (read-only)

```
wstring IGuestSession::userHome
```

Returns the user's home / profile directory. Guest path style.

6.110.1.14 userDocuments (read-only)

```
wstring IGuestSession::userDocuments
```

Returns the user's documents directory. Guest path style.

6.110.1.15 mountPoints (read-only)

```
wstring IGuestSession::mountPoints[]
```

Returns all currently accessible (disk-based) mount points. On Windows guests, this returns the currently mapped disk drives.

Access fails with VBOX_E_NOT_SUPPORTED if the Guest Additions do not support this feature.

6.110.1.16 directories (read-only)

```
IGuestDirectory IGuestSession::directories[]
```

Returns all currently opened guest directories.

6.110.1.17 files (read-only)

```
IGuestFile IGuestSession::files[]
```

Returns all currently opened guest files.

6.110.1.18 eventSource (read-only)

```
IEventSource IGuestSession::eventSource
```

Event source for guest session events.

6.110.2 close

```
void IGuestSession::close()
```

Closes this session. All opened guest directories, files and processes which are not referenced by clients anymore will be closed. Guest processes which fall into this category and still are running in the guest will be terminated automatically.

6.110.3 copyFromGuest

```
IProgress IGuestSession::copyFromGuest(
            [in] wstring sources[],
            [in] wstring filters[],
            [in] wstring flags[],
            [in] wstring destination)
```

sources Paths to directories and/or files on the guest side that should be copied to the host. If the path ends with a path delimiter, only the directory's content is being copied. Guest path style.

filters Array of source filters. This uses the DOS/NT style wildcard characters '?' and '*'.

flags Array of comma-separated list of source flags.

The following flags are available:

`CopyIntoExisting`Allow copying into an existing destination directory.`NoReplace`Do not replace any existing destination files on the destination.`FollowLinks`Follows (and handles) (symbolic) links.`Update`Only copy when the source file is newer than the destination file or when the destination file is missing.

destination Where to put the sources on the host. Host path style.

Copies directories and/or files from guest to the host.
This function requires several parallel arrays to be supplied, one set for each source.

6.110.4 copyToGuest

```
IProgress IGuestSession::copyToGuest(
            [in] wstring sources[],
            [in] wstring filters[],
            [in] wstring flags[],
            [in] wstring destination)
```

sources Paths to directories and/or files on the host side that should be copied to the guest. If the path ends with a path delimiter, only the directory's content is being copied. Host path style.

filters Array of source filters. This uses the DOS/NT style wildcard characters '?' and '*'.

flags Array of comma-separated list of source flags.

The following flags are available:

`CopyIntoExisting`Allow copying into an existing destination directory.`NoReplace`Do not replace any existing destination files on the destination.`FollowLinks`Follows (and handles) (symbolic) links.`Update`Only copy when the source file is newer than the destination file or when the destination file is missing.

destination Where to put the sources on the guest. Guest path style.

Copies directories and/or files from host to the guest.
This function requires several parallel arrays to be supplied, one set for each source.

6.110.5 directoryCopy

```
IProgress IGuestSession::directoryCopy(
            [in] wstring source,
            [in] wstring destination,
            [in] DirectoryCopyFlag flags[])
```

source The path to the directory to copy (in the guest). Guest path style.

destination The path to the target directory (in the guest). Unless the CopyIntoExisting flag is given, the directory shall not already exist. Guest path style.

flags Zero or more DirectoryCopyFlag values.

Recursively copies a directory from one guest location to another.
If this method fails, the following error codes may be reported:

- `E_NOTIMPL:` Not yet implemented.

6.110.6 directoryCopyFromGuest

```
IProgress IGuestSession::directoryCopyFromGuest(
            [in] wstring source,
            [in] wstring destination,
            [in] DirectoryCopyFlag flags[])
```

source Path to the directory on the guest side that should be copied to the host. Guest path style.

destination Where to put the directory on the host. Unless the CopyIntoExisting flag is given, the directory shall not already exist. Host path style.

flags Zero or more DirectoryCopyFlag values.

Recursively copies a directory from the guest to the host.

6.110.7 directoryCopyToGuest

```
IProgress IGuestSession::directoryCopyToGuest(
            [in] wstring source,
            [in] wstring destination,
            [in] DirectoryCopyFlag flags[])
```

source Path to the directory on the host side that should be copied to the guest. Host path style.

destination Where to put the file in the guest. Unless the CopyIntoExisting flag is given, the directory shall not already exist. Guest style path.

flags Zero or more DirectoryCopyFlag values.

Recursively copies a directory from the host to the guest.

6.110.8 directoryCreate

```
void IGuestSession::directoryCreate(
            [in] wstring path,
            [in] unsigned long mode,
            [in] DirectoryCreateFlag flags[])
```

path Path to the directory directory to be created. Guest path style.

mode The UNIX-style access mode mask to create the directory with. Whether/how all three access groups and associated access rights are realized is guest OS dependent. The API does the best it can on each OS.

flags Zero or more DirectoryCreateFlag flags.

Creates a directory in the guest.
If this method fails, the following error codes may be reported:

- `VBOX_E_IPRT_ERROR`: Error while creating the directory.

6.110.9 directoryCreateTemp

```
wstring IGuestSession::directoryCreateTemp(
            [in] wstring templateName,
            [in] unsigned long mode,
            [in] wstring path,
            [in] boolean secure)
```

templateName Template for the name of the directory to create. This must contain at least one 'X' character. The first group of consecutive 'X' characters in the template will be replaced by a random alphanumeric string to produce a unique name.

mode The UNIX-style access mode mask to create the directory with. Whether/how all three access groups and associated access rights are realized is guest OS dependent. The API does the best it can on each OS.

This parameter is ignored if the `secure` parameter is set to `true`.

Note: It is strongly recommended to use 0700.

path The path to the directory in which the temporary directory should be created. Guest path style.

secure Whether to fail if the directory can not be securely created. Currently this means that another unprivileged user cannot manipulate the path specified or remove the temporary directory after it has been created. Also causes the mode specified to be ignored. May not be supported on all guest types.

Creates a temporary directory in the guest.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED:` The operation is not possible as requested on this particular guest type.
- `E_INVALIDARG:` Invalid argument. This includes an incorrectly formatted template, or a non-absolute path.
- `VBOX_E_IPRT_ERROR:` The temporary directory could not be created. Possible reasons include a non-existing path or an insecure path when the secure option was requested.

6.110.10 directoryExists

```
boolean IGuestSession::directoryExists(
            [in] wstring path,
            [in] boolean followSymlinks)
```

path Path to the directory to check if exists. Guest path style.

followSymlinks If `true`, symbolic links in the final component will be followed and the existence of the symlink target made the question for this method. If `false`, a symbolic link in the final component will make the method return `false` (because a symlink isn't a directory).

Checks whether a directory exists in the guest or not.
If this method fails, the following error codes may be reported:

- `VBOX_E_IPRT_ERROR:` Error while checking existence of the directory specified.

6.110.11 directoryOpen

```
IGuestDirectory IGuestSession::directoryOpen(
            [in] wstring path,
            [in] wstring filter,
            [in] DirectoryOpenFlag flags[])
```

path Path to the directory to open. Guest path style.

filter Optional directory listing filter to apply. This uses the DOS/NT style wildcard characters '?' and '*'.

flags Zero or more DirectoryOpenFlag flags.

Opens a directory in the guest and creates a IGuestDirectory object that can be used for further operations.

> **Note:** This method follows symbolic links by default at the moment, this may change in the future.

> **Note:** One idiosyncrasy of the current implementation is that you will NOT get VBOX_E_OBJECT_NOT_FOUND returned here if the directory doesn't exist. Instead the read function will fail with VBOX_E_IPRT_ERROR. This will be fixed soon.

If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND`: Directory to open was not found.
- `VBOX_E_IPRT_ERROR`: Error while opening the directory.
- `VBOX_E_MAXIMUM_REACHED`: The maximum of concurrent guest directories has been reached.

6.110.12 directoryRemove

```
void IGuestSession::directoryRemove(
            [in] wstring path)
```

path Path to the directory that should be removed. Guest path style.

Removes a guest directory if empty.

> **Note:** Symbolic links in the final component will not be followed, instead an not-a-directory error is reported.

6.110.13 directoryRemoveRecursive

```
IProgress IGuestSession::directoryRemoveRecursive(
            [in] wstring path,
            [in] DirectoryRemoveRecFlag flags[])
```

path Path of the directory that is to be removed recursively. Guest path style.

flags Zero or more DirectoryRemoveRecFlag flags.

> **Note:** WARNING! SPECIFYING ContentAndDir IS MANDATORY AT THE MOMENT!!

Removes a guest directory recursively.

> **Note:** WARNING!! THE FLAGS ARE NOT CURRENTLY IMPLEMENTED. THE IMPLEMENTATION WORKS AS IF FLAGS WAS SET TO ContentAndDir.

> **Note:** If the final path component is a symbolic link, this method will fail as it can only be applied to directories.

6.110.14 environmentDoesBaseVariableExist

```
boolean IGuestSession::environmentDoesBaseVariableExist(
           [in] wstring name)
```

name Name of the environment variable to look for. This cannot be empty nor can it contain any equal signs.

Checks if the given environment variable exists in the session's base environment (environmentBase[]).
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: If the Guest Additions do not support the session base environment feature. Support for this was introduced with protocol version XXXX.
- `VBOX_E_INVALID_OBJECT_STATE`: If the Guest Additions has yet to report the session base environment.

6.110.15 environmentGetBaseVariable

```
wstring IGuestSession::environmentGetBaseVariable(
           [in] wstring name)
```

name Name of the environment variable to get.This cannot be empty nor can it contain any equal signs.

Gets an environment variable from the session's base environment (environmentBase[]).
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: If the Guest Additions do not support the session base environment feature. Support for this was introduced with protocol version XXXX.
- `VBOX_E_INVALID_OBJECT_STATE`: If the Guest Additions has yet to report the session base environment.

6.110.16 environmentScheduleSet

```
void IGuestSession::environmentScheduleSet(
            [in] wstring name,
            [in] wstring value)
```

name Name of the environment variable to set. This cannot be empty nor can it contain any equal signs.

value Value to set the session environment variable to.

Schedules setting an environment variable when creating the next guest process. This affects the environmentChanges[] attribute.

6.110.17 environmentScheduleUnset

```
void IGuestSession::environmentScheduleUnset(
            [in] wstring name)
```

name Name of the environment variable to unset. This cannot be empty nor can it contain any equal signs.

Schedules unsetting (removing) an environment variable when creating the next guest process. This affects the environmentChanges[] attribute.

6.110.18 fileCopy

```
IProgress IGuestSession::fileCopy(
            [in] wstring source,
            [in] wstring destination,
            [in] FileCopyFlag flags[])
```

source The path to the file to copy (in the guest). Guest path style.

destination The path to the target file (in the guest). This cannot be a directory. Guest path style.

flags Zero or more FileCopyFlag values.

Copies a file from one guest location to another.

Note: Will overwrite the destination file unless NoReplace is specified.

If this method fails, the following error codes may be reported:

- `E_NOTIMPL`: Not yet implemented.

6.110.19 fileCopyFromGuest

```
IProgress IGuestSession::fileCopyFromGuest(
            [in] wstring source,
            [in] wstring destination,
            [in] FileCopyFlag flags[])
```

source Path to the file on the guest side that should be copied to the host. Guest path style.

destination Where to put the file on the host (file, not directory). Host path style.

flags Zero or more FileCopyFlag values.

Copies a file from the guest to the host.

> **Note:** Will overwrite the destination file unless NoReplace is specified.

If this method fails, the following error codes may be reported:

- VBOX_E_IPRT_ERROR: Error starting the copy operation.

6.110.20 fileCopyToGuest

```
IProgress IGuestSession::fileCopyToGuest(
            [in] wstring source,
            [in] wstring destination,
            [in] FileCopyFlag flags[])
```

source Path to the file on the host side that should be copied to the guest. Host path style.

destination Where to put the file in the guest (file, not directory). Guest style path.

flags Zero or more FileCopyFlag values.

Copies a file from the host to the guest.

> **Note:** Will overwrite the destination file unless NoReplace is specified.

If this method fails, the following error codes may be reported:

- VBOX_E_IPRT_ERROR: Error starting the copy operation.

6.110.21 fileCreateTemp

```
IGuestFile IGuestSession::fileCreateTemp(
            [in] wstring templateName,
            [in] unsigned long mode,
            [in] wstring path,
            [in] boolean secure)
```

templateName Template for the name of the file to create. This must contain at least one 'X' character. The first group of consecutive 'X' characters in the template will be replaced by a random alphanumeric string to produce a unique name.

mode The UNIX-style access mode mask to create the file with. Whether/how all three access groups and associated access rights are realized is guest OS dependent. The API does the best it can on each OS.

This parameter is ignore if the `secure` parameter is set to `true`.

> **Note:** It is strongly recommended to use 0600.

path The path to the directory in which the temporary file should be created.

secure Whether to fail if the file can not be securely created. Currently this means that another unprivileged user cannot manipulate the path specified or remove the temporary file after it has been created. Also causes the mode specified to be ignored. May not be supported on all guest types.

Creates a temporary file in the guest.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: The operation is not possible as requested on this particular guest OS.
- `E_INVALIDARG`: Invalid argument. This includes an incorrectly formatted template, or a non-absolute path.
- `VBOX_E_IPRT_ERROR`: The temporary file could not be created. Possible reasons include a non-existing path or an insecure path when the secure option was requested.

6.110.22 fileExists

```
boolean IGuestSession::fileExists(
            [in] wstring path,
            [in] boolean followSymlinks)
```

path Path to the alleged regular file. Guest path style.

followSymlinks If `true`, symbolic links in the final component will be followed and the existence of the symlink target made the question for this method. If `false`, a symbolic link in the final component will make the method return `false` (because a symlink isn't a regular file).

Checks whether a regular file exists in the guest or not.
If this method fails, the following error codes may be reported:

- `VBOX_E_IPRT_ERROR`: Error while checking existence of the file specified.

6.110.23 fileOpen

```
IGuestFile IGuestSession::fileOpen(
            [in] wstring path,
            [in] FileAccessMode accessMode,
            [in] FileOpenAction openAction,
            [in] unsigned long creationMode)
```

path Path to file to open. Guest path style.

accessMode The file access mode (read, write and/or append). See FileAccessMode for details.

openAction What action to take depending on whether the file exists or not. See FileOpenAction for details.

creationMode The UNIX-style access mode mask to create the file with if `openAction` requested the file to be created (otherwise ignored). Whether/how all three access groups and associated access rights are realized is guest OS dependent. The API does the best it can on each OS.

Opens a file and creates a IGuestFile object that can be used for further operations.
If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND`: File to open was not found.
- `VBOX_E_IPRT_ERROR`: Error while opening the file.
- `VBOX_E_MAXIMUM_REACHED`: The maximum of concurrent guest files has been reached.

6.110.24 fileOpenEx

```
IGuestFile IGuestSession::fileOpenEx(
           [in] wstring path,
           [in] FileAccessMode accessMode,
           [in] FileOpenAction openAction,
           [in] FileSharingMode sharingMode,
           [in] unsigned long creationMode,
           [in] FileOpenExFlag flags[])
```

path Path to file to open. Guest path style.

accessMode The file access mode (read, write and/or append). See FileAccessMode for details.

openAction What action to take depending on whether the file exists or not. See FileOpenAction for details.

sharingMode The file sharing mode in the guest. This parameter is currently ignore for all guest OSes. It will in the future be implemented for Windows, OS/2 and maybe Solaris guests only, the others will ignore it. Use All.

creationMode The UNIX-style access mode mask to create the file with if `openAction` requested the file to be created (otherwise ignored). Whether/how all three access groups and associated access rights are realized is guest OS dependent. The API does the best it can on each OS.

flags Zero or more FileOpenExFlag values.

Opens a file and creates a IGuestFile object that can be used for further operations, extended version.

If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND`: File to open was not found.
- `VBOX_E_IPRT_ERROR`: Error while opening the file.

6.110.25 fileQuerySize

```
long long IGuestSession::fileQuerySize(
           [in] wstring path,
           [in] boolean followSymlinks)
```

path Path to the file which size is requested. Guest path style.

followSymlinks It `true`, symbolic links in the final path component will be followed to their target, and the size of the target is returned. If `false`, symbolic links in the final path component will make the method call fail (symlink is not a regular file).

Queries the size of a regular file in the guest.

If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND`: File to was not found.
- `VBOX_E_IPRT_ERROR`: Error querying file size.

6.110.26 fsObjCopyArray

```
IProgress IGuestSession::fsObjCopyArray(
            [in] wstring source[],
            [in] wstring destination,
            [in] FileCopyFlag flags[])
```

source Array of paths to the file system objects to copy. Guest style path.

destination Where to copy the file system objects to (directory). Guest path style.

flags Zero or more FileCopyFlag values.

Copies file system objects (files, directories, symlinks, etc) from one guest location to another.
If this method fails, the following error codes may be reported:

- `E_NOTIMPL`: Not yet implemented.

6.110.27 fsObjExists

```
boolean IGuestSession::fsObjExists(
            [in] wstring path,
            [in] boolean followSymlinks)
```

path Path to the file system object to check the existence of. Guest path style.

followSymlinks If `true`, symbolic links in the final component will be followed and the method will instead check if the target exists. If `false`, symbolic links in the final component will satisfy the method and it will return `true` in `exists`.

Checks whether a file system object (file, directory, etc) exists in the guest or not.
If this method fails, the following error codes may be reported:

- `VBOX_E_IPRT_ERROR`: Error while checking existence of the file specified.

6.110.28 fsObjMove

```
IProgress IGuestSession::fsObjMove(
            [in] wstring source,
            [in] wstring destination,
            [in] FsObjMoveFlag flags[])
```

source Path to the file to move. Guest path style.

destination Where to move the file to (file, not directory). Guest path style.

flags Zero or more FsObjMoveFlag values.

Moves a file system object (file, directory, symlink, etc) from one guest location to another.
This differs from fsObjRename() in that it can move across file system boundaries. In that case it will perform a copy and then delete the original. For directories, this can take a while and is subject to races.
If this method fails, the following error codes may be reported:

- `E_NOTIMPL`: Not yet implemented.

6.110.29 fsObjMoveArray

```
IProgress IGuestSession::fsObjMoveArray(
            [in] wstring source[],
            [in] wstring destination,
            [in] FsObjMoveFlag flags[])
```

source Array of paths to the file system objects to move. Guest style path.

destination Where to move the file system objects to (directory). Guest path style.

flags Zero or more FsObjMoveFlag values.

Moves file system objects (files, directories, symlinks, etc) from one guest location to another.
If this method fails, the following error codes may be reported:

- E_NOTIMPL: Not yet implemented.

6.110.30 fsObjQueryInfo

```
IGuestFsObjInfo IGuestSession::fsObjQueryInfo(
            [in] wstring path,
            [in] boolean followSymlinks)
```

path Path to the file system object to gather information about. Guest path style.

followSymlinks Information about symbolic links is returned if `false`. Otherwise, symbolic links are followed and the returned information concerns itself with the symlink target if `true`.

Queries information about a file system object (file, directory, etc) in the guest.
If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: The file system object was not found.
- VBOX_E_IPRT_ERROR: Error while querying information.

6.110.31 fsObjRemove

```
void IGuestSession::fsObjRemove(
            [in] wstring path)
```

path Path to the file system object to remove. Guest style path.

Removes a file system object (file, symlink, etc) in the guest. Will not work on directories, use directoryRemove() to remove directories.

> **Note:** This method will remove symbolic links in the final path component, not follow them.

If this method fails, the following error codes may be reported:

- E_NOTIMPL: The method has not been implemented yet.
- VBOX_E_OBJECT_NOT_FOUND: The file system object was not found.
- VBOX_E_IPRT_ERROR: For most other errors. We know this is unhelpful, will fix shortly...

6.110.32 fsObjRemoveArray

```
IProgress IGuestSession::fsObjRemoveArray(
            [in] wstring path[])
```

path Array of paths to the file system objects to remove. Guest style path.

Removes multiple file system objects (files, directories, symlinks, etc) in the guest. Use with caution.

> **Note:** This method is not implemented yet and will return E_NOTIMPL.

> **Note:** This method will remove symbolic links in the final path component, not follow them.

If this method fails, the following error codes may be reported:

- `E_NOTIMPL:` The method has not been implemented yet.

6.110.33 fsObjRename

```
void IGuestSession::fsObjRename(
            [in] wstring oldPath,
            [in] wstring newPath,
            [in] FsObjRenameFlag flags[])
```

oldPath The current path to the object. Guest path style.

newPath The new path to the object. Guest path style.

flags Zero or more FsObjRenameFlag values.

Renames a file system object (file, directory, symlink, etc) in the guest.
If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND:` The file system object was not found.
- `VBOX_E_IPRT_ERROR:` For most other errors. We know this is unhelpful, will fix shortly...

6.110.34 fsObjSetACL

```
void IGuestSession::fsObjSetACL(
            [in] wstring path,
            [in] boolean followSymlinks,
            [in] wstring acl,
            [in] unsigned long mode)
```

path Full path of the file system object which ACL to set.

followSymlinks If `true` symbolic links in the final component will be followed, otherwise, if `false`, the method will work directly on a symbolic link in the final component.

acl The ACL specification string. To-be-defined.

mode UNIX-style mode mask to use if `acl` is empty. As mention in directoryCreate() this is realized on a best effort basis and the exact behavior depends on the Guest OS.

Sets the access control list (ACL) of a file system object (file, directory, etc) in the guest.
If this method fails, the following error codes may be reported:

- `E_NOTIMPL:` The method is not implemented yet.

6.110.35 fsQueryFreeSpace

```
long long IGuestSession::fsQueryFreeSpace(
            [in] wstring path)
```

path Full path to return the free space for.

Returns the free space (in bytes) of a given path.

6.110.36 fsQueryInfo

```
IGuestFsInfo IGuestSession::fsQueryInfo(
            [in] wstring path)
```

path Full path to return file system information for.

Returns file system information for a given path.

6.110.37 processCreate

```
IGuestProcess IGuestSession::processCreate(
            [in] wstring executable,
            [in] wstring arguments[],
            [in] wstring cwd,
            [in] wstring environmentChanges[],
            [in] ProcessCreateFlag flags[],
            [in] unsigned long timeoutMS)
```

executable Full path to the file to execute in the guest. The file has to exist in the guest VM with executable right to the session user in order to succeed. If empty/null, the first entry in the `arguments` array will be used instead (i.e. argv[0]).

arguments Array of arguments passed to the new process.

> **Note:** Starting with VirtualBox 5.0 this array starts with argument 0 instead of argument 1 as in previous versions. Whether the zeroth argument can be passed to the guest depends on the VBoxService version running there. If you depend on this, check that the protocolVersion is 3 or higher.

cwd Path to the directory in which to execute in the guest. The directory has to exist in the guest VM with search rights to the session user in order to succeed. If empty/null, the session user's default (typically 'home') directory is used. If not a full path, it is interpreted relative to the default directory; e.g. 'work' means '$HOME/work' (according to the guest's '$HOME'- like concept).

environmentChanges Set of environment changes to complement environmentChanges[]. Takes precedence over the session ones. The changes are in putenv format, i.e. "VAR=VALUE" for setting and "VAR" for unsetting.

The changes are applied to the base environment of the impersonated guest user (environmentBase[]) when creating the process. (This is done on the guest side of things in order to be compatible with older Guest Additions. That is one of the motivations for not passing in the whole environment here.)

flags Process creation flags; see ProcessCreateFlag for more information.

timeoutMS Timeout (in ms) for limiting the guest process' running time. Pass 0 for an infinite timeout. On timeout the guest process will be killed and its status will be put to an appropriate value. See ProcessStatus for more information.

Creates a new process running in the guest. The new process will be started asynchronously, meaning on return of this function it is not be guaranteed that the guest process is in a started state. To wait for successful startup, use the IProcess::waitFor() call.

> **Note:** Starting at VirtualBox 4.2 guest process execution by is default limited to serve up to 255 guest processes at a time. If all 255 guest processes are active and running, creating a new guest process will result in an error.
> If ProcessCreateFlag_WaitForStdOut and/or ProcessCreateFlag_WaitForStdErr are set, the guest process will not enter the terminated state until all data from the specified streams have been read.

If this method fails, the following error codes may be reported:

- `VBOX_E_IPRT_ERROR`: Error creating guest process.
- `VBOX_E_MAXIMUM_REACHED`: The maximum of concurrent guest processes has been reached.

6.110.38 processCreateEx

```
IGuestProcess IGuestSession::processCreateEx(
            [in] wstring executable,
            [in] wstring arguments[],
            [in] wstring cwd,
            [in] wstring environmentChanges[],
            [in] ProcessCreateFlag flags[],
            [in] unsigned long timeoutMS,
            [in] ProcessPriority priority,
            [in] long affinity[])
```

executable Full path to the file to execute in the guest. The file has to exist in the guest VM with executable right to the session user in order to succeed. If empty/null, the first entry in the `arguments` array will be used instead (i.e. argv[0]).

arguments Array of arguments passed to the new process.

> **Note:** Starting with VirtualBox 5.0 this array starts with argument 0 instead of argument 1 as in previous versions. Whether the zeroth argument can be passed to the guest depends on the VBoxService version running there. If you depend on this, check that the protocolVersion is 3 or higher.

cwd Path to the directory in which to execute in the guest. The directory has to exist in the guest VM with search rights to the session user in order to succeed. If empty/null, the session user's default (typically 'home') directory is used. If not a full path, it is interpreted relative to the default directory; e.g. 'work' means '$HOME/work' (according to the guest's '$HOME'- like concept).

environmentChanges Set of environment changes to complement environmentChanges[]. Takes precedence over the session ones. The changes are in putenv format, i.e. "VAR=VALUE" for setting and "VAR" for unsetting.

The changes are applied to the base environment of the impersonated guest user (environmentBase[]) when creating the process. (This is done on the guest side of things in order to be compatible with older Guest Additions. That is one of the motivations for not passing in the whole environment here.)

flags Process creation flags, see ProcessCreateFlag for detailed description of available flags.

timeoutMS Timeout (in ms) for limiting the guest process' running time. Pass 0 for an infinite timeout. On timeout the guest process will be killed and its status will be put to an appropriate value. See ProcessStatus for more information.

priority Process priority to use for execution, see ProcessPriority for available priority levels.

> **Note:** This is silently ignored if not supported by Guest Additions.

affinity Processor affinity to set for the new process. This is a list of guest CPU numbers the process is allowed to run on.

> **Note:** This is silently ignored if the guest does not support setting the affinity of processes, or if the Guest Additions do not implemet this feature.

Creates a new process running in the guest with the extended options for setting the process priority and affinity.
See processCreate() for more information.

6.110.39 processGet

```
IGuestProcess IGuestSession::processGet(
            [in] unsigned long pid)
```

pid Process ID (PID) to get guest process for.

Gets a certain guest process by its process ID (PID).

6.110.40 symlinkCreate

```
void IGuestSession::symlinkCreate(
            [in] wstring symlink,
            [in] wstring target,
            [in] SymlinkType type)
```

symlink Path to the symbolic link that should be created. Guest path style.

target The path to the symbolic link target. If not an absolute, this will be relative to the `symlink` location at access time. Guest path style.

type The symbolic link type (mainly for Windows). See SymlinkType for more information.

Creates a symbolic link in the guest.
If this method fails, the following error codes may be reported:

- `E_NOTIMPL`: The method is not implemented yet.

6.110.41 symlinkExists

```
boolean IGuestSession::symlinkExists(
            [in] wstring symlink)
```

symlink Path to the alleged symbolic link. Guest path style.

Checks whether a symbolic link exists in the guest.
If this method fails, the following error codes may be reported:

- E_NOTIMPL: The method is not implemented yet.

6.110.42 symlinkRead

```
wstring IGuestSession::symlinkRead(
            [in] wstring symlink,
            [in] SymlinkReadFlag flags[])
```

symlink Path to the symbolic link to read.

flags Zero or more SymlinkReadFlag values.

Reads the target value of a symbolic link in the guest.
If this method fails, the following error codes may be reported:

- E_NOTIMPL: The method is not implemented yet.

6.110.43 waitFor

```
GuestSessionWaitResult IGuestSession::waitFor(
            [in] unsigned long waitFor,
            [in] unsigned long timeoutMS)
```

waitFor Specifies what to wait for; see GuestSessionWaitForFlag for more information.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Waits for one or more events to happen.

Note: Deprecated and will be removed. Use waitForArray() instead.

6.110.44 waitForArray

```
GuestSessionWaitResult IGuestSession::waitForArray(
            [in] GuestSessionWaitForFlag waitFor[],
            [in] unsigned long timeoutMS)
```

waitFor Specifies what to wait for; see GuestSessionWaitForFlag for more information.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Waits for one or more events to happen. Scriptable version of waitFor().

6.111 IGuestSessionEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Base abstract interface for all guest session events.

6.111.1 Attributes

6.111.1.1 session (read-only)

`IGuestSession IGuestSessionEvent::session`

Guest session that is subject to change.

6.112 IGuestSessionRegisteredEvent (IGuestSessionEvent)

> **Note:** This interface extends IGuestSessionEvent and therefore supports all its methods and attributes as well.

Notification when a guest session was registered or unregistered.

6.112.1 Attributes

6.112.1.1 registered (read-only)

`boolean IGuestSessionRegisteredEvent::registered`

If `true`, the guest session was registered, otherwise it was unregistered.

6.113 IGuestSessionStateChangedEvent (IGuestSessionEvent)

> **Note:** This interface extends IGuestSessionEvent and therefore supports all its methods and attributes as well.

Notification when a guest session changed its state.

6.113.1 Attributes

6.113.1.1 id (read-only)

`unsigned long IGuestSessionStateChangedEvent::id`

Session ID of guest session which was changed.

6.113.1.2 status (read-only)

`GuestSessionStatus IGuestSessionStateChangedEvent::status`

New session status.

6.113.1.3 error (read-only)

`IVirtualBoxErrorInfo IGuestSessionStateChangedEvent::error`

Error information in case of new session status is indicating an error.

The attribute IVirtualBoxErrorInfo::resultDetail will contain the runtime (IPRT) error code from the guest. See include/iprt/err.h and include/VBox/err.h for details.

6.114 IGuestUserStateChangedEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a guest user changed its state.

6.114.1 Attributes

6.114.1.1 name (read-only)

`wstring IGuestUserStateChangedEvent::name`

Name of the guest user whose state changed.

6.114.1.2 domain (read-only)

`wstring IGuestUserStateChangedEvent::domain`

Name of the FQDN (fully qualified domain name) this user is bound to. Optional.

6.114.1.3 state (read-only)

`GuestUserState IGuestUserStateChangedEvent::state`

What was changed for this guest user. See GuestUserState for more information.

6.114.1.4 stateDetails (read-only)

`wstring IGuestUserStateChangedEvent::stateDetails`

Optional state details, depending on the state attribute.

6.115 IHost

The IHost interface represents the physical machine that this VirtualBox installation runs on.

An object implementing this interface is returned by the IVirtualBox::host attribute. This interface contains read-only information about the host's physical hardware (such as what processors and disks are available, what the host operating system is, and so on) and also allows for manipulating some of the host's hardware, such as global USB device filters and host interface networking.

6.115.1 Attributes

6.115.1.1 architecture (read-only)

`PlatformArchitecture IHost::architecture`

Platform architecture type.

6.115.1.2 x86 (read-only)

`IHostX86 IHost::x86`

Associated object for x86 host specifics.

6.115.1.3 DVDDrives (read-only)

`IMedium IHost::DVDDrives[]`

List of DVD drives available on the host.

6.115.1.4 floppyDrives (read-only)

`IMedium IHost::floppyDrives[]`

List of floppy drives available on the host.

6.115.1.5 audioDevices (read-only)

`IHostAudioDevice IHost::audioDevices[]`

List of audio devices currently available on the host.

6.115.1.6 USBDevices (read-only)

`IHostUSBDevice IHost::USBDevices[]`

List of USB devices currently attached to the host. Once a new device is physically attached to the host computer, it appears in this list and remains there until detached.

> **Note:** If USB functionality is not available in the given edition of VirtualBox, this method will set the result code to `E_NOTIMPL`.

6.115.1.7 USBDeviceFilters (read-only)

`IHostUSBDeviceFilter IHost::USBDeviceFilters[]`

List of USB device filters in action. When a new device is physically attached to the host computer, filters from this list are applied to it (in order they are stored in the list). The first matched filter will determine the action performed on the device.

Unless the device is ignored by these filters, filters of all currently running virtual machines (IUSBDeviceFilters::deviceFilters[]) are applied to it.

> **Note:** If USB functionality is not available in the given edition of VirtualBox, this method will set the result code to `E_NOTIMPL`.

See also: IHostUSBDeviceFilter, USBDeviceState

6.115.1.8 networkInterfaces (read-only)

`IHostNetworkInterface IHost::networkInterfaces[]`

List of host network interfaces currently defined on the host.

6.115.1.9 nameServers (read-only)

`wstring IHost::nameServers[]`

The list of nameservers registered in host's name resolving system.

6.115.1.10 domainName (read-only)

`wstring IHost::domainName`

Domain name used for name resolving.

6.115.1.11 searchStrings (read-only)

`wstring IHost::searchStrings[]`

Search string registered for name resolving.

6.115.1.12 processorCount (read-only)

`unsigned long IHost::processorCount`

Number of (logical) CPUs installed in the host system.

6.115.1.13 processorOnlineCount (read-only)

`unsigned long IHost::processorOnlineCount`

Number of (logical) CPUs online in the host system.

6.115.1.14 processorCoreCount (read-only)

`unsigned long IHost::processorCoreCount`

Number of physical processor cores installed in the host system.

6.115.1.15 processorOnlineCoreCount (read-only)

`unsigned long IHost::processorOnlineCoreCount`

Number of physical processor cores online in the host system.

6.115.1.16 hostDrives (read-only)

`IHostDrive IHost::hostDrives[]`

List of the host drive available to use in the VirtualBox.

6.115.1.17 memorySize (read-only)

`unsigned long IHost::memorySize`

Amount of system memory in megabytes installed in the host system.

6.115.1.18 memoryAvailable (read-only)

```
unsigned long IHost::memoryAvailable
```

Available system memory in the host system.

6.115.1.19 operatingSystem (read-only)

```
wstring IHost::operatingSystem
```

Name of the host system's operating system.

6.115.1.20 OSVersion (read-only)

```
wstring IHost::OSVersion
```

Host operating system's version string.

6.115.1.21 UTCTime (read-only)

```
long long IHost::UTCTime
```

Returns the current host time in milliseconds since 1970-01-01 UTC.

6.115.1.22 videoInputDevices (read-only)

```
IHostVideoInputDevice IHost::videoInputDevices[]
```

List of currently available host video capture devices.

6.115.1.23 updateHost (read-only)

```
IUpdateAgent IHost::updateHost
```

Checks for new VirtualBox host versions.

6.115.1.24 updateExtPack (read-only)

```
IUpdateAgent IHost::updateExtPack
```

Checks for new VirtualBox Extension Pack versions.

6.115.1.25 updateGuestAdditions (read-only)

```
IUpdateAgent IHost::updateGuestAdditions
```

Checks for new Guest Additions versions.

6.115.2 addUSBDeviceSource

```
void IHost::addUSBDeviceSource(
            [in] wstring backend,
            [in] wstring id,
            [in] wstring address,
            [in] wstring propertyNames[],
            [in] wstring propertyValues[])
```

backend The backend to use as the new device source.

id Unique ID to identify the source.

address Address to use, the format is dependent on the backend. For USB/IP backends for example the notation is host[:port].

propertyNames Array of property names for more detailed configuration. Not used at the moment.

propertyValues Array of property values for more detailed configuration. Not used at the moment.

Adds a new USB device source.

6.115.3 createHostOnlyNetworkInterface

```
IProgress IHost::createHostOnlyNetworkInterface(
          [out] IHostNetworkInterface hostInterface)
```

hostInterface Created host interface object.

Creates a new adapter for Host Only Networking.
If this method fails, the following error codes may be reported:

- E_INVALIDARG: Host network interface name already exists.

6.115.4 createUSBDeviceFilter

```
IHostUSBDeviceFilter IHost::createUSBDeviceFilter(
          [in] wstring name)
```

name Filter name. See IUSBDeviceFilter::name for more information.

Creates a new USB device filter. All attributes except the filter name are set to empty (any match), *active* is false (the filter is not active).
The created filter can be added to the list of filters using insertUSBDeviceFilter().
See also: USBDeviceFilters[]

6.115.5 findHostDVDDrive

```
IMedium IHost::findHostDVDDrive(
          [in] wstring name)
```

name Name of the host drive to search for

Searches for a host DVD drive with the given name.
If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: Given name does not correspond to any host drive.

6.115.6 findHostFloppyDrive

```
IMedium IHost::findHostFloppyDrive(
          [in] wstring name)
```

name Name of the host floppy drive to search for

Searches for a host floppy drive with the given name.
If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: Given name does not correspond to any host floppy drive.

6.115.7 findHostNetworkInterfaceById

```
IHostNetworkInterface IHost::findHostNetworkInterfaceById(
            [in] uuid id)
```

id GUID of the host network interface to search for.

Searches through all host network interfaces for an interface with the given GUID.

> **Note:** The method returns an error if the given GUID does not correspond to any host network interface.

6.115.8 findHostNetworkInterfaceByName

```
IHostNetworkInterface IHost::findHostNetworkInterfaceByName(
            [in] wstring name)
```

name Name of the host network interface to search for.

Searches through all host network interfaces for an interface with the given `name`.

> **Note:** The method returns an error if the given `name` does not correspond to any host network interface.

6.115.9 findHostNetworkInterfacesOfType

```
IHostNetworkInterface[] IHost::findHostNetworkInterfacesOfType(
            [in] HostNetworkInterfaceType type)
```

type type of the host network interfaces to search for.

Searches through all host network interfaces and returns a list of interfaces of the specified type

6.115.10 findUSBDeviceByAddress

```
IHostUSBDevice IHost::findUSBDeviceByAddress(
            [in] wstring name)
```

name Address of the USB device (as assigned by the host) to search for.

Searches for a USB device with the given host address.
See also: IUSBDevice::address
If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND`: Given `name` does not correspond to any USB device.

6.115.11 findUSBDeviceById

```
IHostUSBDevice IHost::findUSBDeviceById(
            [in] uuid id)
```

id UUID of the USB device to search for.

Searches for a USB device with the given UUID.
See also: IUSBDevice::id
If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND`: Given `id` does not correspond to any USB device.

6.115.12 generateMACAddress

```
wstring IHost::generateMACAddress()
```

Generates a valid Ethernet MAC address, 12 hexadecimal characters.

6.115.13 getProcessorDescription

```
wstring IHost::getProcessorDescription(
            [in] unsigned long cpuId)
```

cpuId Identifier of the CPU.

> **Note:** The current implementation might not necessarily return the description for this exact CPU.

Query the model string of a specified host CPU.

6.115.14 getProcessorFeature

```
boolean IHost::getProcessorFeature(
            [in] ProcessorFeature feature)
```

feature CPU Feature identifier.

Query whether a CPU feature is supported or not.

6.115.15 getProcessorSpeed

```
unsigned long IHost::getProcessorSpeed(
            [in] unsigned long cpuId)
```

cpuId Identifier of the CPU.

Query the (approximate) maximum speed of a specified host CPU in Megahertz.

6.115.16 insertUSBDeviceFilter

```
void IHost::insertUSBDeviceFilter(
            [in] unsigned long position,
            [in] IHostUSBDeviceFilter filter)
```

position Position to insert the filter to.

filter USB device filter to insert.

Inserts the given USB device to the specified position in the list of filters.
Positions are numbered starting from `0`. If the specified position is equal to or greater than the number of elements in the list, the filter is added at the end of the collection.

> **Note:** Duplicates are not allowed, so an attempt to insert a filter already in the list is an error.

> **Note:** If USB functionality is not available in the given edition of VirtualBox, this method will set the result code to E_NOTIMPL.

See also: USBDeviceFilters[]
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_OBJECT_STATE: USB device filter is not created within this VirtualBox instance.
- E_INVALIDARG: USB device filter already in list.

6.115.17 isExecutionEngineSupported

```
boolean IHost::isExecutionEngineSupported(
            [in] CPUArchitecture cpuArchitecture,
            [in] VMExecutionEngine executionEngine)
```

cpuArchitecture The guest CPU architecture to check the execution engine for.

executionEngine The execution engine to check.

Returns whether the given execution engine is supported on the host. This is primarily to check whether the native API is supported (think of Hyper-V not being available due to not running in a Hyper-V root partition).

6.115.18 removeHostOnlyNetworkInterface

```
IProgress IHost::removeHostOnlyNetworkInterface(
            [in] uuid id)
```

id Adapter GUID.

Removes the given Host Only Networking interface.
If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: No host network interface matching id found.

6.115.19 removeUSBDeviceFilter

```
void IHost::removeUSBDeviceFilter(
            [in] unsigned long position)
```

position Position to remove the filter from.

Removes a USB device filter from the specified position in the list of filters.
Positions are numbered starting from 0. Specifying a position equal to or greater than the number of elements in the list will produce an error.

> **Note:** If USB functionality is not available in the given edition of VirtualBox, this method will set the result code to E_NOTIMPL.

See also: USBDeviceFilters[]
If this method fails, the following error codes may be reported:

- E_INVALIDARG: USB device filter list empty or invalid position.

6.115.20 removeUSBDeviceSource

```
void IHost::removeUSBDeviceSource(
            [in] wstring id)
```

id The identifier used when the source was added.

Removes a previously added USB device source.

6.116 IHostAudioDevice

Represents an audio device provided by the host OS.

6.116.1 Attributes

6.116.1.1 id (read-only)

```
uuid IHostAudioDevice::id
```

Unique device ID.

6.116.1.2 name (read/write)

```
wstring IHostAudioDevice::name
```

Friendly name of the device.

6.116.1.3 type (read/write)

```
AudioDeviceType IHostAudioDevice::type
```

Device type.

6.116.1.4 usage (read/write)

```
AudioDirection IHostAudioDevice::usage
```

Usage type of the device.

6.116.1.5 defaultIn (read/write)

```
boolean IHostAudioDevice::defaultIn
```

Whether this device is being marked as the default input device by the host OS.

6.116.1.6 defaultOut (read/write)

```
boolean IHostAudioDevice::defaultOut
```

Whether this device is being marked as the default output device by the host OS.

6.116.1.7 isHotPlug (read/write)

```
boolean IHostAudioDevice::isHotPlug
```

Whether this device is being marked as a hotplug device, i.e. can be removed from the system.

6.116.1.8 state (read/write)

`AudioDeviceState IHostAudioDevice::state`

Current device state.

6.116.2 getProperty

```
wstring IHostAudioDevice::getProperty(
            [in] wstring key)
```

key Name of the key to get.

Returns an audio specific property string.
If the requested data `key` does not exist, this function will succeed and return an empty string in the `value` argument.

6.117 IHostAudioDeviceChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a host audio device state has changed.

6.117.1 Attributes

6.117.1.1 device (read-only)

`IHostAudioDevice IHostAudioDeviceChangedEvent::device`

Host audio device that has changed.

6.117.1.2 new (read-only)

`boolean IHostAudioDeviceChangedEvent::new`

`true` if the host device is a newly detected device, `false` if not.

6.117.1.3 state (read-only)

`AudioDeviceState IHostAudioDeviceChangedEvent::state`

New audio device state.

6.117.1.4 error (read-only)

`IVirtualBoxErrorInfo IHostAudioDeviceChangedEvent::error`

`null` on success or an error message object on failure.

6.118 IHostDrive

The IHostDrive interface represents the drive of the physical machine. It is not a complete medium description and, therefore, it is not IMedium based. The interface is used to get information about a host drive and its partitioning.

> **Note:** The object operates in limited mode if the user cannot open the drive and parse the partition table. In limited mode on the drivePath and model attributes can be accessed, the rest will fail with E_ACCESSDENIED.

6.118.1 Attributes

6.118.1.1 drivePath (read-only)

```
wstring IHostDrive::drivePath
```

The path of the drive. Platform dependent.

6.118.1.2 partitioningType (read-only)

```
PartitioningType IHostDrive::partitioningType
```

The scheme of the partitions the disk has.

6.118.1.3 uuid (read-only)

```
uuid IHostDrive::uuid
```

The GUID of the disk.

6.118.1.4 sectorSize (read-only)

```
unsigned long IHostDrive::sectorSize
```

The size of the sector in bytes.

6.118.1.5 size (read-only)

```
long long IHostDrive::size
```

The size of the disk in bytes.

6.118.1.6 model (read-only)

```
wstring IHostDrive::model
```

The model string of the drive if available.

6.118.1.7 partitions (read-only)

```
IHostDrivePartition IHostDrive::partitions[]
```

List of partitions available on the host drive.

6.119 IHostDrivePartition

> **Note:** With the web service, this interface is mapped to a structure. Attributes that return this interface will not return an object, but a complete structure containing the attributes listed below as structure members.

The IHostDrivePartition interface represents the partition of the host drive. To enumerate all available drives partitions in the host, use the IHost::hostDrives[] attribute.

6.119.1 Attributes

6.119.1.1 number (read-only)

```
unsigned long IHostDrivePartition::number
```

The number of the partition. Represents the system number of the partition, e.g. /dev/sdX on Linux, where X is the number returned.

6.119.1.2 size (read-only)

```
long long IHostDrivePartition::size
```

The partition size in bytes.

6.119.1.3 start (read-only)

```
long long IHostDrivePartition::start
```

The start byte offset of this partition in bytes relative to the beginning of the hard disk.

6.119.1.4 type (read-only)

```
PartitionType IHostDrivePartition::type
```

A translation of typeMBR and typeUuid when possible, otherwise set to Unknown.

6.119.1.5 active (read-only)

```
boolean IHostDrivePartition::active
```

The partition is bootable when TRUE.

6.119.1.6 typeMBR (read-only)

```
unsigned long IHostDrivePartition::typeMBR
```

The raw MBR partition type, 0 for non-MBR disks.

6.119.1.7 startCylinder (read-only)

```
unsigned long IHostDrivePartition::startCylinder
```

The cylinder (0..1023) of the first sector in the partition on an MBR disk, zero for not an MBR disk.

6.119.1.8 startHead (read-only)

```
unsigned long IHostDrivePartition::startHead
```

The head (0..255) of the first sector in the partition on an MBR disk, zero for not an MBR disk.

6.119.1.9 startSector (read-only)

```
unsigned long IHostDrivePartition::startSector
```

The sector (0..63) of the first sector in the partition on an MBR disk, zero for not an MBR disk.

6.119.1.10 endCylinder (read-only)

```
unsigned long IHostDrivePartition::endCylinder
```

The cylinder (0..1023) of the last sector (inclusive) in the partition on an MBR disk, zero for not an MBR disk.

6.119.1.11 endHead (read-only)

```
unsigned long IHostDrivePartition::endHead
```

The head (0..255) of the last sector (inclusive) in the partition on an MBR disk, zero for not an MBR disk.

6.119.1.12 endSector (read-only)

```
unsigned long IHostDrivePartition::endSector
```

The sector (1..63) of the last sector (inclusive) in the partition on an MBR disk, zero for not an MBR disk.

6.119.1.13 typeUuid (read-only)

```
uuid IHostDrivePartition::typeUuid
```

The partition type when GUID partitioning scheme is used, NULL UUID value for not a GPT disks.

6.119.1.14 uuid (read-only)

```
uuid IHostDrivePartition::uuid
```

The GUID of the partition when GUID partitioning scheme is used, NULL UUID value for not a GPT disks.

6.119.1.15 name (read-only)

```
wstring IHostDrivePartition::name
```

The name of the partition if GPT partitioning is used, empty if not a GPT disk.

6.120 IHostNameResolutionConfigurationChangeEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

6.120.1 Attributes

6.120.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IHostNameResolutionConfigurationChangeEvent::midlDoesNotLikeEmptyInterfaces
```

6.121 IHostNetworkInterface

Represents one of host's network interfaces. IP V6 address and network mask are strings of 32 hexadecimal digits grouped by four. Groups are separated by colons. For example, fe80:0000:0000:0000:021e:c2ff:fed2:b030.

6.121.1 Attributes

6.121.1.1 name (read-only)

```
wstring IHostNetworkInterface::name
```

Returns the host network interface name.

6.121.1.2 shortName (read-only)

```
wstring IHostNetworkInterface::shortName
```

Returns the host network interface short name.

6.121.1.3 id (read-only)

```
uuid IHostNetworkInterface::id
```

Returns the interface UUID.

6.121.1.4 networkName (read-only)

```
wstring IHostNetworkInterface::networkName
```

Returns the name of a virtual network the interface gets attached to.

6.121.1.5 DHCPEnabled (read-only)

```
boolean IHostNetworkInterface::DHCPEnabled
```

Specifies whether the DHCP is enabled for the interface.

6.121.1.6 IPAddress (read-only)

```
wstring IHostNetworkInterface::IPAddress
```

Returns the IP V4 address of the interface.

6.121.1.7 networkMask (read-only)

```
wstring IHostNetworkInterface::networkMask
```

Returns the network mask of the interface.

6.121.1.8 IPV6Supported (read-only)

```
boolean IHostNetworkInterface::IPV6Supported
```

Specifies whether the IP V6 is supported/enabled for the interface.

6.121.1.9 IPV6Address (read-only)

```
wstring IHostNetworkInterface::IPV6Address
```

Returns the IP V6 address of the interface.

6.121.1.10 IPV6NetworkMaskPrefixLength (read-only)

```
unsigned long IHostNetworkInterface::IPV6NetworkMaskPrefixLength
```

Returns the length IP V6 network mask prefix of the interface.

6.121.1.11 hardwareAddress (read-only)

```
wstring IHostNetworkInterface::hardwareAddress
```

Returns the hardware address. For Ethernet it is MAC address.

6.121.1.12 mediumType (read-only)

```
HostNetworkInterfaceMediumType IHostNetworkInterface::mediumType
```

Type of protocol encapsulation used.

6.121.1.13 status (read-only)

```
HostNetworkInterfaceStatus IHostNetworkInterface::status
```

Status of the interface.

6.121.1.14 interfaceType (read-only)

```
HostNetworkInterfaceType IHostNetworkInterface::interfaceType
```

specifies the host interface type.

6.121.1.15 wireless (read-only)

```
boolean IHostNetworkInterface::wireless
```

Specifies whether the interface is wireless.

6.121.2 DHCPRediscover

```
void IHostNetworkInterface::DHCPRediscover()
```

refreshes the IP configuration for DHCP-enabled interface.

6.121.3 enableDynamicIPConfig

```
void IHostNetworkInterface::enableDynamicIPConfig()
```

enables the dynamic IP configuration.

6.121.4 enableStaticIPConfig

```
void IHostNetworkInterface::enableStaticIPConfig(
           [in] wstring IPAddress,
           [in] wstring networkMask)
```

IPAddress IP address.

networkMask network mask.

sets and enables the static IP V4 configuration for the given interface.

6.121.5 enableStaticIPConfigV6

```
void IHostNetworkInterface::enableStaticIPConfigV6(
           [in] wstring IPV6Address,
           [in] unsigned long IPV6NetworkMaskPrefixLength)
```

IPV6Address IP address.

IPV6NetworkMaskPrefixLength network mask.

sets and enables the static IP V6 configuration for the given interface.

6.122 IHostOnlyNetwork

6.122.1 Attributes

6.122.1.1 networkName (read/write)

```
wstring IHostOnlyNetwork::networkName
```

TBD: User-friendly, descriptive name of host-only network. For example, “Host-only network 192.168.56.0”.

6.122.1.2 enabled (read/write)

```
boolean IHostOnlyNetwork::enabled
```

6.122.1.3 networkMask (read/write)

```
wstring IHostOnlyNetwork::networkMask
```

specifies network mask

6.122.1.4 hostIP (read-only)

```
wstring IHostOnlyNetwork::hostIP
```

host IP address, which usually is the lower IP address of DHCP range.

6.122.1.5 lowerIP (read/write)

```
wstring IHostOnlyNetwork::lowerIP
```

specifies from IP address in DHCP address range

6.122.1.6 upperIP (read/write)

```
wstring IHostOnlyNetwork::upperIP
```

specifies to IP address in DHCP address range

6.122.1.7 id (read/write)

```
uuid IHostOnlyNetwork::id
```

Host-only network ID.

6.123 IHostPCIDevicePlugEvent (IMachineEvent)

> **Note:** This interface extends IMachineEvent and therefore supports all its methods and attributes as well.

Notification when host PCI device is plugged/unplugged. Plugging usually takes place on VM startup, unplug - when IMachine::detachHostPCIDevice() is called.
See also: IMachine::detachHostPCIDevice()

6.123.1 Attributes

6.123.1.1 plugged (read-only)

```
boolean IHostPCIDevicePlugEvent::plugged
```

If device successfully plugged or unplugged.

6.123.1.2 success (read-only)

```
boolean IHostPCIDevicePlugEvent::success
```

If operation was successful, if false - 'message' attribute may be of interest.

6.123.1.3 attachment (read-only)

```
IPCIDeviceAttachment IHostPCIDevicePlugEvent::attachment
```

Attachment info for this device.

6.123.1.4 message (read-only)

```
wstring IHostPCIDevicePlugEvent::message
```

Optional error message.

6.124 IHostUSBDevice (IUSBDevice)

> **Note:** This interface extends IUSBDevice and therefore supports all its methods and attributes as well.

The IHostUSBDevice interface represents a physical USB device attached to the host computer.
Besides properties inherited from IUSBDevice, this interface adds the state property that holds the current state of the USB device.
See also: IHost::USBDevices[], IHost::USBDeviceFilters[]

6.124.1 Attributes

6.124.1.1 state (read-only)

`USBDeviceState IHostUSBDevice::state`

Current state of the device.

6.125 IHostUSBDeviceFilter (IUSBDeviceFilter)

> **Note:** This interface extends IUSBDeviceFilter and therefore supports all its methods and attributes as well.

The IHostUSBDeviceFilter interface represents a global filter for a physical USB device used by the host computer. Used indirectly in IHost::USBDeviceFilters[].
Using filters of this type, the host computer determines the initial state of the USB device after it is physically attached to the host's USB controller.

> **Note:** The IUSBDeviceFilter::remote attribute is ignored by this type of filters, because it makes sense only for machine USB filters.

See also: IHost::USBDeviceFilters[]

6.125.1 Attributes

6.125.1.1 action (read/write)

`USBDeviceFilterAction IHostUSBDeviceFilter::action`

Action performed by the host when an attached USB device matches this filter.

6.126 IHostUpdateAgent (IUpdateAgent)

> **Note:** This interface extends IUpdateAgent and therefore supports all its methods and attributes as well.

Implementation of the IUpdateAgent object for VirtualBox host updates.

> **Note:** If no proxy is explicitly set, the proxy settings from ISystemProperties will be used.

6.126.1 Attributes

6.126.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IHostUpdateAgent::midlDoesNotLikeEmptyInterfaces
```

6.127 IHostVideoInputDevice

Represents one of host's video capture devices, for example a webcam.

6.127.1 Attributes

6.127.1.1 name (read-only)

```
wstring IHostVideoInputDevice::name
```

User friendly name.

6.127.1.2 path (read-only)

```
wstring IHostVideoInputDevice::path
```

The host path of the device.

6.127.1.3 alias (read-only)

```
wstring IHostVideoInputDevice::alias
```

An alias which can be used for IEmulatedUSB::webcamAttach()

6.128 IHostX86

The x86-specific interface for the physical host machine.

6.128.1 getProcessorCPUIDLeaf

```
void IHostX86::getProcessorCPUIDLeaf(
            [in] unsigned long cpuId,
            [in] unsigned long leaf,
            [in] unsigned long subLeaf,
            [out] unsigned long valEax,
            [out] unsigned long valEbx,
            [out] unsigned long valEcx,
            [out] unsigned long valEdx)
```

cpuId Identifier of the CPU. The CPU must be online.

> **Note:** The current implementation might not necessarily return the description for this exact CPU.

leaf CPUID leaf index (eax).

subLeaf CPUID leaf sub index (ecx). This currently only applies to cache information on Intel CPUs. Use 0 if retrieving values for IPlatformX86::setCPUIDLeaf().

valEax CPUID leaf value for register eax.

valEbx CPUID leaf value for register ebx.

valEcx CPUID leaf value for register ecx.

valEdx CPUID leaf value for register edx.

Returns the CPU cpuid information for the specified leaf.

6.129 IInternalMachineControl

Note: This interface is not supported in the web service.

6.129.1 authenticateExternal

```
void IInternalMachineControl::authenticateExternal(
           [in] wstring authParams[],
           [out] wstring result)
```

authParams The auth parameters, credentials, etc.

result The authentification result.

Verify credentials using the external auth library.

6.129.2 autoCaptureUSBDevices

```
void IInternalMachineControl::autoCaptureUSBDevices()
```

Requests a capture all matching USB devices attached to the host. When the request is completed, the VM process will get a IInternalSessionControl::onUSBDeviceAttach() notification per every captured device.

6.129.3 beginPowerUp

```
void IInternalMachineControl::beginPowerUp(
           [in] IProgress progress)
```

progress

Tells VBoxSVC that IConsole::powerUp() is under ways and gives it the progress object that should be part of any pending IMachine::launchVMProcess() operations. The progress object may be called back to reflect an early cancelation, so some care have to be taken with respect to any cancelation callbacks. The console object will call endPowerUp() to signal the completion of the progress object.

6.129.4 beginPoweringDown

```
void IInternalMachineControl::beginPoweringDown(
           [out] IProgress progress)
```

progress Progress object created by VBoxSVC to wait until the VM is powered down.

Called by the VM process to inform the server it wants to stop the VM execution and power down.

6.129.5 captureUSBDevice

```
void IInternalMachineControl::captureUSBDevice(
            [in] uuid id,
            [in] wstring captureFilename)
```

id

captureFilename

Requests a capture of the given host USB device. When the request is completed, the VM process will get a IInternalSessionControl::onUSBDeviceAttach() notification.

6.129.6 detachAllUSBDevices

```
void IInternalMachineControl::detachAllUSBDevices(
            [in] boolean done)
```

done

Notification that a VM that is being powered down. The done parameter indicates whether which stage of the power down we're at. When `done = false` the VM is announcing its intentions, while when `done = true` the VM is reporting what it has done.

> **Note:** In the `done = true` case, the server must run its own filters and filters of all VMs but this one on all detach devices as if they were just attached to the host computer.

6.129.7 detachUSBDevice

```
void IInternalMachineControl::detachUSBDevice(
            [in] uuid id,
            [in] boolean done)
```

id

done

Notification that a VM is going to detach (`done = false`) or has already detached (`done = true`) the given USB device. When the `done = true` request is completed, the VM process will get a IInternalSessionControl::onUSBDeviceDetach() notification.

> **Note:** In the `done = true` case, the server must run its own filters and filters of all VMs but this one on the detached device as if it were just attached to the host computer.

6.129.8 ejectMedium

```
IMediumAttachment IInternalMachineControl::ejectMedium(
            [in] IMediumAttachment attachment)
```

attachment The medium attachment where the eject happened.

Tells VBoxSVC that the guest has ejected the medium associated with the medium attachment.

6.129.9 endPowerUp

```
void IInternalMachineControl::endPowerUp(
           [in] long result)
```

result

Tells VBoxSVC that IConsole::powerUp() has completed. This method may query status information from the progress object it received in beginPowerUp() and copy it over to any in-progress IMachine::launchVMProcess() call in order to complete that progress object.

6.129.10 endPoweringDown

```
void IInternalMachineControl::endPoweringDown(
           [in] long result,
           [in] wstring errMsg)
```

result S_OK to indicate success.

errMsg human readable error message in case of failure.

Called by the VM process to inform the server that powering down previously requested by #beginPoweringDown is either successfully finished or there was a failure.

If this method fails, the following error codes may be reported:

- VBOX_E_FILE_ERROR: Settings file not accessible.
- VBOX_E_XML_ERROR: Could not parse the settings file.

6.129.11 finishOnlineMergeMedium

```
void IInternalMachineControl::finishOnlineMergeMedium()
```

Gets called by IInternalSessionControl::onlineMergeMedium(). All necessary state information is available at the called object.

6.129.12 lockMedia

```
void IInternalMachineControl::lockMedia()
```

Locks all media attached to the machine for writing and parents of attached differencing media (if any) for reading. This operation is atomic so that if it fails no media is actually locked.

This method is intended to be called when the machine is in Starting or Restoring state. The locked media will be automatically unlocked when the machine is powered off or crashed.

6.129.13 onSessionEnd

```
IProgress IInternalMachineControl::onSessionEnd(
           [in] ISession session)
```

session Session that is being closed

Triggered by the given session object when the session is about to close normally.

6.129.14 pullGuestProperties

```
void IInternalMachineControl::pullGuestProperties(
            [out] wstring names[],
            [out] wstring values[],
            [out] long long timestamps[],
            [out] wstring flags[])
```

names The names of the properties returned.

values The values of the properties returned. The array entries match the corresponding entries in the name array.

timestamps The timestamps of the properties returned. The array entries match the corresponding entries in the name array.

flags The flags of the properties returned. The array entries match the corresponding entries in the name array.

Get the list of the guest properties matching a set of patterns along with their values, timestamps and flags and give responsibility for managing properties to the console.

6.129.15 pushGuestProperty

```
void IInternalMachineControl::pushGuestProperty(
            [in] wstring name,
            [in] wstring value,
            [in] long long timestamp,
            [in] wstring flags,
            [in] boolean fWasDeleted)
```

name The name of the property to be updated.

value The value of the property.

timestamp The timestamp of the property.

flags The flags of the property.

fWasDeleted The flag which indicates if property was deleted.

Update a single guest property in IMachine.

6.129.16 reportVmStatistics

```
void IInternalMachineControl::reportVmStatistics(
            [in] unsigned long validStats,
            [in] unsigned long cpuUser,
            [in] unsigned long cpuKernel,
            [in] unsigned long cpuIdle,
            [in] unsigned long memTotal,
            [in] unsigned long memFree,
            [in] unsigned long memBalloon,
            [in] unsigned long memShared,
            [in] unsigned long memCache,
            [in] unsigned long pagedTotal,
            [in] unsigned long memAllocTotal,
            [in] unsigned long memFreeTotal,
            [in] unsigned long memBalloonTotal,
```

```
            [in] unsigned long memSharedTotal,
            [in] unsigned long vmNetRx,
            [in] unsigned long vmNetTx)
```

validStats Mask defining which parameters are valid. For example: 0x11 means that cpuIdle and XXX are valid. Other parameters should be ignored.

cpuUser Percentage of processor time spent in user mode as seen by the guest.

cpuKernel Percentage of processor time spent in kernel mode as seen by the guest.

cpuIdle Percentage of processor time spent idling as seen by the guest.

memTotal Total amount of physical guest RAM.

memFree Free amount of physical guest RAM.

memBalloon Amount of ballooned physical guest RAM.

memShared Amount of shared physical guest RAM.

memCache Total amount of guest (disk) cache memory.

pagedTotal Total amount of space in the page file.

memAllocTotal Total amount of memory allocated by the hypervisor.

memFreeTotal Total amount of free memory available in the hypervisor.

memBalloonTotal Total amount of memory ballooned by the hypervisor.

memSharedTotal Total amount of shared memory in the hypervisor.

vmNetRx Network receive rate for VM.

vmNetTx Network transmit rate for VM.

Passes statistics collected by VM (including guest statistics) to VBoxSVC.

6.129.17 runUSBDeviceFilters

```
void IInternalMachineControl::runUSBDeviceFilters(
            [in] IUSBDevice device,
            [out] boolean matched,
            [out] unsigned long maskedInterfaces)
```

device

matched

maskedInterfaces

Asks the server to run USB devices filters of the associated machine against the given USB device and tell if there is a match.

> **Note:** Intended to be used only for remote USB devices. Local ones don't require to call this method (this is done implicitly by the Host and USBProxyService).

6.129.18 unlockMedia

```
void IInternalMachineControl::unlockMedia()
```

Unlocks all media previously locked using lockMedia().

This method is intended to be used with teleportation so that it is possible to teleport between processes on the same machine.

6.129.19 updateState

```
void IInternalMachineControl::updateState(
            [in] MachineState state)
```

state

Updates the VM state.

> **Note:** This operation will also update the settings file with the correct information about the saved state file and delete this file from disk when appropriate.

6.130 IInternalProgressControl

> **Note:** This interface is not supported in the web service.

6.130.1 notifyComplete

```
void IInternalProgressControl::notifyComplete(
            [in] long resultCode,
            [in] IVirtualBoxErrorInfo errorInfo)
```

resultCode

errorInfo

Internal method, not to be called externally.

6.130.2 notifyPointOfNoReturn

```
void IInternalProgressControl::notifyPointOfNoReturn()
```

Internal method, not to be called externally.

6.130.3 setCurrentOperationProgress

```
void IInternalProgressControl::setCurrentOperationProgress(
            [in] unsigned long percent)
```

percent

Internal method, not to be called externally.

6.130.4 setNextOperation

```
void IInternalProgressControl::setNextOperation(
           [in] wstring nextOperationDescription,
           [in] unsigned long nextOperationsWeight)
```

nextOperationDescription

nextOperationsWeight

Internal method, not to be called externally.

6.130.5 waitForOtherProgressCompletion

```
void IInternalProgressControl::waitForOtherProgressCompletion(
           [in] IProgress progressOther,
           [in] unsigned long timeoutMS)
```

progressOther Other progress object to be “cloned”.

timeoutMS Timeout (in ms). Pass 0 for an infinite timeout.

Internal method, not to be called externally.
Waits until the other task is completed (including all sub-operations) and forward all changes from the other progress to this progress. This means sub-operation number, description, percent and so on.
The caller is responsible for having at least the same count of sub-operations in this progress object as there are in the other progress object.
If the other progress object supports cancel and this object gets any cancel request (when here enabled as well), it will be forwarded to the other progress object.
Error information is automatically preserved (by transferring it to the current thread’s error information). If the caller wants to set it as the completion state of this progress it needs to be done separately.
If this method fails, the following error codes may be reported:

- VBOX_E_TIMEOUT: Waiting time has expired.

6.131 IInternalSessionControl

Note: This interface is not supported in the web service.

6.131.1 Attributes

6.131.1.1 PID (read-only)

```
unsigned long IInternalSessionControl::PID
```

PID of the process that has created this Session object.

6.131.1.2 remoteConsole (read-only)

```
IConsole IInternalSessionControl::remoteConsole
```

Returns the console object suitable for remote control.

6.131.1.3 nominalState (read-only)

```
MachineState IInternalSessionControl::nominalState
```

Returns suitable machine state for the VM execution state. Useful for choosing a sensible machine state after a complex operation which failed or otherwise resulted in an unclear situation.

6.131.2 accessGuestProperty

```
void IInternalSessionControl::accessGuestProperty(
            [in] wstring name,
            [in] wstring value,
            [in] wstring flags,
            [in] unsigned long accessMode,
            [out] wstring retValue,
            [out] long long retTimestamp,
            [out] wstring retFlags)
```

name Name of guest property.

value Value of guest property.

flags Flags of guest property.

accessMode 0 = get, 1 = set, 2 = delete.

retValue When getting: Value of guest property.

retTimestamp When getting: Timestamp of guest property.

retFlags When getting: Flags of guest property.

Called by IMachine::getGuestProperty() and by IMachine::setGuestProperty() in order to read and modify guest properties.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Machine session is not open.
- VBOX_E_INVALID_OBJECT_STATE: Session type is not direct.

6.131.3 assignRemoteMachine

```
void IInternalSessionControl::assignRemoteMachine(
            [in] IMachine machine,
            [in] IConsole console)
```

machine

console

Assigns the machine and the (remote) console object associated with this remote-type session.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Session state prevents operation.

6.131.4 cancelSaveStateWithReason

```
void IInternalSessionControl::cancelSaveStateWithReason()
```

Internal method for cancelling a VM save state. See also: saveStateWithReason()

6.131.5 enableVMMStatistics

```
void IInternalSessionControl::enableVMMStatistics(
            [in] boolean enable)
```

enable True enables statistics collection.

Enables or disables collection of VMM RAM statistics.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Machine session is not open.
- VBOX_E_INVALID_OBJECT_STATE: Session type is not direct.

6.131.6 enumerateGuestProperties

```
void IInternalSessionControl::enumerateGuestProperties(
            [in] wstring patterns,
            [out] wstring keys[],
            [out] wstring values[],
            [out] long long timestamps[],
            [out] wstring flags[])
```

patterns The patterns to match the properties against as a comma-separated string. If this is empty, all properties currently set will be returned.

keys The key names of the properties returned.

values The values of the properties returned. The array entries match the corresponding entries in the key array.

timestamps The timestamps of the properties returned. The array entries match the corresponding entries in the key array.

flags The flags of the properties returned. The array entries match the corresponding entries in the key array.

Return a list of the guest properties matching a set of patterns along with their values, timestamps and flags.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Machine session is not open.
- VBOX_E_INVALID_OBJECT_STATE: Session type is not direct.

6.131.7 onAudioAdapterChange

```
void IInternalSessionControl::onAudioAdapterChange(
            [in] IAudioAdapter audioAdapter)
```

audioAdapter

Triggerd when settings of the audio adapter of the associated virtual machine have changed.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Session state prevents operation.
- VBOX_E_INVALID_OBJECT_STATE: Session type prevents operation.

6.131.8 onBandwidthGroupChange

```
void IInternalSessionControl::onBandwidthGroupChange(
            [in] IBandwidthGroup bandwidthGroup)
```

bandwidthGroup The bandwidth group which changed.

Notification when one of the bandwidth groups change.

6.131.9 onCPUChange

```
void IInternalSessionControl::onCPUChange(
            [in] unsigned long cpu,
            [in] boolean add)
```

cpu The CPU which changed

add Flag whether the CPU was added or removed

Notification when a CPU changes.

6.131.10 onCPUExecutionCapChange

```
void IInternalSessionControl::onCPUExecutionCapChange(
            [in] unsigned long executionCap)
```

executionCap The new CPU execution cap value. (1-100)

Notification when the CPU execution cap changes.

6.131.11 onClipboardFileTransferModeChange

```
void IInternalSessionControl::onClipboardFileTransferModeChange(
            [in] boolean enabled)
```

enabled Flag whether clipboard file transfers are allowed or not.

Notification when the shared clipboard file transfers mode changes.

6.131.12 onClipboardModeChange

```
void IInternalSessionControl::onClipboardModeChange(
            [in] ClipboardMode clipboardMode)
```

clipboardMode The new shared clipboard mode.

Notification when the shared clipboard mode changes.

6.131.13 onDnDModeChange

```
void IInternalSessionControl::onDnDModeChange(
            [in] DnDMode dndMode)
```

dndMode The new mode for drag'n drop.

Notification when the drag'n drop mode changes.

6.131.14 onGuestDebugControlChange

```
void IInternalSessionControl::onGuestDebugControlChange(
            [in] IGuestDebugControl guestDebugControl)
```

guestDebugControl

Triggered when settings of the guest debug settings of the associated virtual machine have changed.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Session state prevents operation.
- VBOX_E_INVALID_OBJECT_STATE: Session type prevents operation.

6.131.15 onHostAudioDeviceChange

```
void IInternalSessionControl::onHostAudioDeviceChange(
            [in] IHostAudioDevice device,
            [in] boolean isNew,
            [in] AudioDeviceState state,
            [in] IVirtualBoxErrorInfo errorInfo)
```

device Device the state has been changed for.

isNew Whether this is a newly detected device or not.

state The new device state.

errorInfo Error information.

Triggerd when the state of a host audio device has changed.

6.131.16 onMediumChange

```
void IInternalSessionControl::onMediumChange(
            [in] IMediumAttachment mediumAttachment,
            [in] boolean force)
```

mediumAttachment The medium attachment which changed.

force If the medium change was forced.

Triggered when attached media of the associated virtual machine have changed.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Session state prevents operation.
- VBOX_E_INVALID_OBJECT_STATE: Session type prevents operation.

6.131.17 onNetworkAdapterChange

```
void IInternalSessionControl::onNetworkAdapterChange(
            [in] INetworkAdapter networkAdapter,
            [in] boolean changeAdapter)
```

networkAdapter

changeAdapter

Triggered when settings of a network adapter of the associated virtual machine have changed. If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Session state prevents operation.
- VBOX_E_INVALID_OBJECT_STATE: Session type prevents operation.

6.131.18 onParallelPortChange

```
void IInternalSessionControl::onParallelPortChange(
            [in] IParallelPort parallelPort)
```

parallelPort

Triggered when settings of a parallel port of the associated virtual machine have changed. If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Session state prevents operation.
- VBOX_E_INVALID_OBJECT_STATE: Session type prevents operation.

6.131.19 onRecordingScreenStateChange

```
void IInternalSessionControl::onRecordingScreenStateChange(
            [in] RecordingState state,
            [in] unsigned long screen)
```

state Whether recording for that screen was enabled or disabled.

screen Number of screen for which the event has happened.

Triggered when a recording screen state got changed.

6.131.20 onRecordingStateChange

```
void IInternalSessionControl::onRecordingStateChange(
            [in] RecordingState state,
            [out] IProgress progress)
```

state Whether recording was enabled or disabled.

progress Progress object.

Triggered when a recording state got changed.

6.131.21 onSerialPortChange

```
void IInternalSessionControl::onSerialPortChange(
            [in] ISerialPort serialPort)
```

serialPort

Triggered when settings of a serial port of the associated virtual machine have changed. If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Session state prevents operation.
- VBOX_E_INVALID_OBJECT_STATE: Session type prevents operation.

6.131.22 onSharedFolderChange

```
void IInternalSessionControl::onSharedFolderChange(
            [in] boolean global)
```

global

Triggered when a permanent (global or machine) shared folder has been created or removed.

> **Note:** We don't pass shared folder parameters in this notification because the order in which parallel notifications are delivered is not defined, therefore it could happen that these parameters were outdated by the time of processing this notification.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Session state prevents operation.
- `VBOX_E_INVALID_OBJECT_STATE`: Session type prevents operation.

6.131.23 onShowWindow

```
void IInternalSessionControl::onShowWindow(
            [in] boolean check,
            [out] boolean canShow,
            [out] long long winId)
```

check

canShow

winId

Called by IMachine::canShowConsoleWindow() and by IMachine::showConsoleWindow() in order to notify console listeners ICanShowWindowEvent and IShowWindowEvent.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_OBJECT_STATE`: Session type prevents operation.

6.131.24 onStorageControllerChange

```
void IInternalSessionControl::onStorageControllerChange(
            [in] uuid machineId,
            [in] wstring controllerName)
```

machineId The id of the machine containing the storage controller.

controllerName The name of the storage controller.

Triggered when settings of a storage controller of the associated virtual machine have changed.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Session state prevents operation.
- `VBOX_E_INVALID_OBJECT_STATE`: Session type prevents operation.

6.131.25 onStorageDeviceChange

```
void IInternalSessionControl::onStorageDeviceChange(
            [in] IMediumAttachment mediumAttachment,
            [in] boolean remove,
            [in] boolean silent)
```

mediumAttachment The medium attachment which changed.

remove TRUE if the device is removed, FALSE if it was added.

silent TRUE if the device is is silently reconfigured without notifying the guest about it.

Triggered when attached storage devices of the associated virtual machine have changed.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Session state prevents operation.
- VBOX_E_INVALID_OBJECT_STATE: Session type prevents operation.

6.131.26 onUSBControllerChange

```
void IInternalSessionControl::onUSBControllerChange()
```

Triggered when settings of the USB controller object of the associated virtual machine have changed.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Session state prevents operation.
- VBOX_E_INVALID_OBJECT_STATE: Session type prevents operation.

6.131.27 onUSBDeviceAttach

```
void IInternalSessionControl::onUSBDeviceAttach(
            [in] IUSBDevice device,
            [in] IVirtualBoxErrorInfo error,
            [in] unsigned long maskedInterfaces,
            [in] wstring captureFilename)
```

device

error

maskedInterfaces

captureFilename

Triggered when a request to capture a USB device (as a result of matched USB filters or direct call to IConsole::attachUSBDevice()) has completed. A nullerror object means success, otherwise it describes a failure.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Session state prevents operation.
- VBOX_E_INVALID_OBJECT_STATE: Session type prevents operation.

6.131.28 onUSBDeviceDetach

```
void IInternalSessionControl::onUSBDeviceDetach(
            [in] uuid id,
            [in] IVirtualBoxErrorInfo error)
```

id

error

Triggered when a request to release the USB device (as a result of machine termination or direct call to IConsole::detachUSBDevice()) has completed. A `nullerror` object means success, otherwise it describes a failure.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` Session state prevents operation.
- `VBOX_E_INVALID_OBJECT_STATE:` Session type prevents operation.

6.131.29 onVMProcessPriorityChange

```
void IInternalSessionControl::onVMProcessPriorityChange(
            [in] VMProcPriority priority)
```

priority The priority which set.

Triggered when process priority of the associated virtual machine have changed.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` Session state prevents operation.
- `VBOX_E_INVALID_OBJECT_STATE:` Session type prevents operation.
- `VBOX_E_VM_ERROR:` Error from underlying level. See additional error info.

6.131.30 onVRDEServerChange

```
void IInternalSessionControl::onVRDEServerChange(
            [in] boolean restart)
```

restart Flag whether the server must be restarted

Triggered when settings of the VRDE server object of the associated virtual machine have changed.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` Session state prevents operation.
- `VBOX_E_INVALID_OBJECT_STATE:` Session type prevents operation.

6.131.31 onlineMergeMedium

```
void IInternalSessionControl::onlineMergeMedium(
            [in] IMediumAttachment mediumAttachment,
            [in] unsigned long sourceIdx,
            [in] unsigned long targetIdx,
            [in] IProgress progress)
```

mediumAttachment The medium attachment to identify the medium chain.

sourceIdx The index of the source image in the chain. Redundant, but drastically reduces IPC.

targetIdx The index of the target image in the chain. Redundant, but drastically reduces IPC.

progress Progress object for this operation.

Triggers online merging of a hard disk. Used internally when deleting a snapshot while a VM referring to the same hard disk chain is running.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Machine session is not open.
- VBOX_E_INVALID_OBJECT_STATE: Session type is not direct.

6.131.32 pauseWithReason

```
void IInternalSessionControl::pauseWithReason(
            [in] Reason reason)
```

reason Specify the best matching reason code please.

Internal method for triggering a VM pause with a specified reason code. The reason code can be interpreted by device/drivers and thus it might behave slightly differently than a normal VM pause.
See also: IConsole::pause()
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine not in Running state.
- VBOX_E_VM_ERROR: Virtual machine error in suspend operation.

6.131.33 reconfigureMediumAttachments

```
void IInternalSessionControl::reconfigureMediumAttachments(
            [in] IMediumAttachment attachments[])
```

attachments Array containing the medium attachments which need to be reconfigured.

Reconfigure all specified medium attachments in one go, making sure the current state corresponds to the specified medium.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Machine session is not open.
- VBOX_E_INVALID_OBJECT_STATE: Session type is not direct.

6.131.34 resumeWithReason

```
void IInternalSessionControl::resumeWithReason(
            [in] Reason reason)
```

reason Specify the best matching reason code please.

Internal method for triggering a VM resume with a specified reason code. The reason code can be interpreted by device/drivers and thus it might behave slightly differently than a normal VM resume.
See also: IConsole::resume()
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine not in Paused state.
- VBOX_E_VM_ERROR: Virtual machine error in resume operation.

6.131.35 saveStateWithReason

```
boolean IInternalSessionControl::saveStateWithReason(
            [in] Reason reason,
            [in] IProgress progress,
            [in] ISnapshot snapshot,
            [in] wstring stateFilePath,
            [in] boolean pauseVM)
```

reason Specify the best matching reason code please.

progress Progress object to track the operation completion.

snapshot Snapshot object for which this save state operation is executed.

stateFilePath File path the VM process must save the execution state to.

pauseVM The VM should be paused before saving state. It is automatically unpaused on error in the "vanilla save state" case.

Internal method for triggering a VM save state with a specified reason code. The reason code can be interpreted by device/drivers and thus it might behave slightly differently than a normal VM save state.

This call is fully synchronous, and the caller is expected to have set the machine state appropriately (and has to set the follow-up machine state if this call failed).

See also: IMachine::saveState()

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine state is not one of the expected values.
- `VBOX_E_FILE_ERROR`: Failed to create directory for saved state file.

6.131.36 uninitialize

```
void IInternalSessionControl::uninitialize()
```

Uninitializes (closes) this session. Used by VirtualBox to close the corresponding remote session when the direct session dies or gets closed.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Session state prevents operation.

6.131.37 updateMachineState

```
void IInternalSessionControl::updateMachineState(
            [in] MachineState machineState)
```

machineState

Updates the machine state in the VM process. Must be called only in certain cases (see the method implementation).

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Session state prevents operation.
- `VBOX_E_INVALID_OBJECT_STATE`: Session type prevents operation.

6.132 IKeyboard

The IKeyboard interface represents the virtual machine's keyboard. Used in IConsole::keyboard.
Use this interface to send keystrokes or the Ctrl-Alt-Del sequence to the virtual machine.

6.132.1 Attributes

6.132.1.1 keyboardLEDs (read-only)

```
KeyboardLED IKeyboard::keyboardLEDs[]
```

Current status of the guest keyboard LEDs.

6.132.1.2 eventSource (read-only)

```
IEventSource IKeyboard::eventSource
```

Event source for keyboard events.

6.132.2 putCAD

```
void IKeyboard::putCAD()
```

Sends the Ctrl-Alt-Del sequence to the keyboard. This function is nothing special, it is just a convenience function calling putScancodes() with the proper scancodes.
If this method fails, the following error codes may be reported:

- VBOX_E_IPRT_ERROR: Could not send all scan codes to virtual keyboard.

6.132.3 putScancode

```
void IKeyboard::putScancode(
           [in] long scancode)
```

scancode

Sends a scancode to the keyboard.
If this method fails, the following error codes may be reported:

- VBOX_E_IPRT_ERROR: Could not send scan code to virtual keyboard.

6.132.4 putScancodes

```
unsigned long IKeyboard::putScancodes(
           [in] long scancodes[])
```

scancodes

Sends an array of scancodes to the keyboard.
If this method fails, the following error codes may be reported:

- VBOX_E_IPRT_ERROR: Could not send all scan codes to virtual keyboard.

6.132.5 putUsageCode

```
void IKeyboard::putUsageCode(
            [in] long usageCode,
            [in] long usagePage,
            [in] boolean keyRelease)
```

usageCode

usagePage

keyRelease

Sends a USB HID usage code and page to the keyboard. The keyRelease flag is set when the key is being released.

If this method fails, the following error codes may be reported:

- VBOX_E_IPRT_ERROR: Could not send usage code to virtual keyboard.

6.132.6 releaseKeys

```
void IKeyboard::releaseKeys()
```

Causes the virtual keyboard to release any keys which are currently pressed. Useful when host and guest keyboard may be out of sync.

If this method fails, the following error codes may be reported:

- VBOX_E_IPRT_ERROR: Could not release some or all keys.

6.133 IKeyboardLedsChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the guest OS executes the KBD_CMD_SET_LEDS command to alter the state of the keyboard LEDs.

6.133.1 Attributes

6.133.1.1 numLock (read-only)

```
boolean IKeyboardLedsChangedEvent::numLock
```

NumLock status.

6.133.1.2 capsLock (read-only)

```
boolean IKeyboardLedsChangedEvent::capsLock
```

CapsLock status.

6.133.1.3 scrollLock (read-only)

```
boolean IKeyboardLedsChangedEvent::scrollLock
```

ScrollLock status.

6.134 ILanguageChangedEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

6.134.1 Attributes

6.134.1.1 languageId (read-only)

```
wstring ILanguageChangedEvent::languageId
```

6.135 IMachine

The IMachine interface represents a virtual machine, or guest, created in VirtualBox.

This interface is used in two contexts. First of all, a collection of objects implementing this interface is stored in the IVirtualBox::machines[] attribute which lists all the virtual machines that are currently registered with this VirtualBox installation. Also, once a session has been opened for the given virtual machine (e.g. the virtual machine is running), the machine object associated with the open session can be queried from the session object; see ISession for details.

The main role of this interface is to expose the settings of the virtual machine and provide methods to change various aspects of the virtual machine's configuration. For machine objects stored in the IVirtualBox::machines[] collection, all attributes are read-only unless explicitly stated otherwise in individual attribute and method descriptions.

In order to change a machine setting, a session for this machine must be opened using one of the lockMachine() or launchVMProcess() methods. After the machine has been successfully locked for a session, a mutable machine object needs to be queried from the session object and then the desired settings changes can be applied to the returned object using IMachine attributes and methods. See the ISession interface description for more information about sessions.

Note that IMachine does not provide methods to control virtual machine execution (such as start the machine, or power it down) – these methods are grouped in a separate interface called IConsole.

See also: ISession, IConsole

6.135.1 Attributes

6.135.1.1 parent (read-only)

```
IVirtualBox IMachine::parent
```

Associated parent object.

6.135.1.2 icon (read/write)

```
octet IMachine::icon[]
```

Overridden VM Icon details.

6.135.1.3 accessible (read-only)

```
boolean IMachine::accessible
```

Whether this virtual machine is currently accessible or not.

A machine is always deemed accessible unless it is registered *and* its settings file cannot be read or parsed (either because the file itself is unavailable or has invalid XML contents).

Every time this property is read, the accessibility state of this machine is re-evaluated. If the returned value is `false`, the accessError property may be used to get the detailed error information describing the reason of inaccessibility, including XML error messages.

When the machine is inaccessible, only the following properties can be used on it:

- parent
- id
- settingsFilePath
- accessible
- accessError

An attempt to access any other property or method will return an error.

The only possible action you can perform on an inaccessible machine is to unregister it using the unregister() call (or, to check for the accessibility state once more by querying this property).

> **Note:** In the current implementation, once this property returns `true`, the machine will never become inaccessible later, even if its settings file cannot be successfully read/written any more (at least, until the VirtualBox server is restarted). This limitation may be removed in future releases.

6.135.1.4 accessError (read-only)

`IVirtualBoxErrorInfo IMachine::accessError`

Error information describing the reason of machine inaccessibility.

Reading this property is only valid after the last call to accessible returned `false` (i.e. the machine is currently inaccessible). Otherwise, a `null` IVirtualBoxErrorInfo object will be returned.

6.135.1.5 name (read/write)

`wstring IMachine::name`

Name of the virtual machine.

Besides being used for human-readable identification purposes everywhere in VirtualBox, the virtual machine name is also used as a name of the machine's settings file and as a name of the subdirectory this settings file resides in. Thus, every time you change the value of this property, the settings file will be renamed once you call saveSettings() to confirm the change. The containing subdirectory will be also renamed, but only if it has exactly the same name as the settings file itself prior to changing this property (for backward compatibility with previous API releases). The above implies the following limitations:

- The machine name cannot be empty.
- The machine name can contain only characters that are valid file name characters according to the rules of the file system used to store VirtualBox configuration.
- You cannot have two or more machines with the same name if they use the same subdirectory for storing the machine settings files.

- You cannot change the name of the machine if it is running, or if any file in the directory containing the settings file is being used by another running machine or by any other process in the host operating system at a time when saveSettings() is called.

If any of the above limitations are hit, saveSettings() will return an appropriate error message explaining the exact reason and the changes you made to this machine will not be saved.

Starting with VirtualBox 4.0, a ".vbox" extension of the settings file is recommended, but not enforced. (Previous versions always used a generic ".xml" extension.)

6.135.1.6 description (read/write)

```
wstring IMachine::description
```

Description of the virtual machine.

The description attribute can contain any text and is typically used to describe the hardware and software configuration of the virtual machine in detail (i.e. network settings, versions of the installed software and so on).

6.135.1.7 id (read-only)

```
uuid IMachine::id
```

UUID of the virtual machine.

6.135.1.8 groups (read/write)

```
wstring IMachine::groups[]
```

Array of machine group names of which this machine is a member. `""` and `"/"` are synonyms for the toplevel group. Each group is only listed once, however they are listed in no particular order and there is no guarantee that there are no gaps in the group hierarchy (i.e. `"/group"`, `"/group/subgroup/subsubgroup"` is a valid result).

6.135.1.9 OSTypeId (read/write)

```
wstring IMachine::OSTypeId
```

User-defined identifier of the Guest OS type. You may use IVirtualBox::getGuestOSType() to obtain an IGuestOSType object representing details about the given Guest OS type. All Guest OS types are considered valid, even those which are not known to IVirtualBox::getGuestOSType().

> **Note:** This value may differ from the value returned by IGuest::OSTypeId if Guest Additions are installed to the guest OS.

6.135.1.10 hardwareVersion (read/write)

```
wstring IMachine::hardwareVersion
```

Hardware version identifier. Internal use only for now.

6.135.1.11 hardwareUUID (read/write)

```
uuid IMachine::hardwareUUID
```

The UUID presented to the guest via memory tables, hardware and guest properties. For most VMs this is the same as the `id`, but for VMs which have been cloned or teleported it may be the same as the source VM. The latter is because the guest shouldn't notice that it was cloned or teleported.

6.135.1.12 CPUCount (read/write)

```
unsigned long IMachine::CPUCount
```

Number of virtual CPUs in the VM.

6.135.1.13 CPUHotPlugEnabled (read/write)

```
boolean IMachine::CPUHotPlugEnabled
```

This setting determines whether VirtualBox allows CPU hotplugging for this machine.

6.135.1.14 CPUExecutionCap (read/write)

```
unsigned long IMachine::CPUExecutionCap
```

Means to limit the number of CPU cycles a guest can use. The unit is percentage of host CPU cycles per second. The valid range is 1 - 100. 100 (the default) implies no limit.

6.135.1.15 CPUIDPortabilityLevel (read/write)

```
unsigned long IMachine::CPUIDPortabilityLevel
```

Virtual CPUID portability level. The higher the number the fewer vendor-specific or recent CPU features are reported to the guest (via the CPUID instruction). The default level of zero (0) means that all virtualized features supported by the host are passed through to the guest. While the value of three (3) is currently the level suppressing the most features.

Exactly which of the CPUID features are left out by the VMM at which level is subject to change with each major version.

6.135.1.16 memorySize (read/write)

```
unsigned long IMachine::memorySize
```

System memory size in megabytes.

6.135.1.17 memoryBalloonSize (read/write)

```
unsigned long IMachine::memoryBalloonSize
```

Memory balloon size in megabytes.

6.135.1.18 pageFusionEnabled (read/write)

```
boolean IMachine::pageFusionEnabled
```

This setting determines whether VirtualBox allows page fusion for this machine (64-bit hosts only).

6.135.1.19 graphicsAdapter (read-only)

`IGraphicsAdapter IMachine::graphicsAdapter`

Graphics adapter object.

6.135.1.20 platform (read-only)

`IPlatform IMachine::platform`

Associated platform object.

6.135.1.21 firmwareSettings (read-only)

`IFirmwareSettings IMachine::firmwareSettings`

Object containing all firmware settings.

6.135.1.22 trustedPlatformModule (read-only)

`ITrustedPlatformModule IMachine::trustedPlatformModule`

Object containing all TPM settings.

6.135.1.23 nonVolatileStore (read-only)

`INvramStore IMachine::nonVolatileStore`

Object to manipulate data in the non volatile storage file.

6.135.1.24 recordingSettings (read-only)

`IRecordingSettings IMachine::recordingSettings`

Object containing all recording settings.

6.135.1.25 pointingHIDType (read/write)

`PointingHIDType IMachine::pointingHIDType`

Type of pointing HID (such as mouse or tablet) used in this VM. The default is typically "PS2Mouse" but can vary depending on the requirements of the guest operating system.

6.135.1.26 keyboardHIDType (read/write)

`KeyboardHIDType IMachine::keyboardHIDType`

Type of keyboard HID used in this VM. The default is typically "PS2Keyboard" but can vary depending on the requirements of the guest operating system.

6.135.1.27 snapshotFolder (read/write)

`wstring IMachine::snapshotFolder`

Full path to the directory used to store snapshot data (differencing media and saved state files) of this machine.

The initial value of this property is <path_to_settings_file>/<machine_uuid>.

Currently, it is an error to try to change this property on a machine that has snapshots (because this would require to move possibly large files to a different location). A separate method will be available for this purpose later.

> **Note:** Setting this property to `null` or to an empty string will restore the initial value.

> **Note:** When setting this property, the specified path can be absolute (full path) or relative to the directory where the machine settings file is located. When reading this property, a full path is always returned.

> **Note:** The specified path may not exist, it will be created when necessary.

6.135.1.28 VRDEServer (read-only)

`IVRDEServer IMachine::VRDEServer`

VirtualBox Remote Desktop Extension (VRDE) server object.

6.135.1.29 emulatedUSBCardReaderEnabled (read/write)

`boolean IMachine::emulatedUSBCardReaderEnabled`

6.135.1.30 mediumAttachments (read-only)

`IMediumAttachment IMachine::mediumAttachments[]`

Array of media attached to this machine.

6.135.1.31 USBControllers (read-only)

`IUSBController IMachine::USBControllers[]`

Array of USB controllers attached to this machine.

> **Note:** If USB functionality is not available in the given edition of VirtualBox, this method will set the result code to `E_NOTIMPL`.

6.135.1.32 USBDeviceFilters (read-only)

`IUSBDeviceFilters IMachine::USBDeviceFilters`

Associated USB device filters object.

> **Note:** If USB functionality is not available in the given edition of VirtualBox, this method will set the result code to `E_NOTIMPL`.

6.135.1.33 audioSettings (read-only)

`IAudioSettings IMachine::audioSettings`

The machine's audio settings.

6.135.1.34 storageControllers (read-only)

`IStorageController IMachine::storageControllers[]`

Array of storage controllers attached to this machine.

6.135.1.35 settingsFilePath (read-only)

`wstring IMachine::settingsFilePath`

Full name of the file containing machine settings data.

6.135.1.36 settingsAuxFilePath (read-only)

`wstring IMachine::settingsAuxFilePath`

Full name of the file containing auxiliary machine settings data.

6.135.1.37 settingsModified (read-only)

`boolean IMachine::settingsModified`

Whether the settings of this machine have been modified (but neither yet saved nor discarded).

> **Note:** Reading this property is only valid on instances returned by ISession::machine and on new machines created by IVirtualBox::createMachine() or opened by IVirtualBox::openMachine() but not yet registered, or on unregistered machines after calling unregister(). For all other cases, the settings can never be modified.

> **Note:** For newly created unregistered machines, the value of this property is always `true` until saveSettings() is called (no matter if any machine settings have been changed after the creation or not). For opened machines the value is set to `false` (and then follows to normal rules).

6.135.1.38 sessionState (read-only)

`SessionState IMachine::sessionState`

Current session state for this machine.

6.135.1.39 sessionName (read-only)

`wstring IMachine::sessionName`

Name of the session. If sessionState is Spawning or Locked, this attribute contains the same value as passed to the launchVMProcess() method in the `name` parameter. If the session was established with lockMachine(), it is the name of the session (if set, otherwise empty string). If sessionState is SessionClosed, the value of this attribute is an empty string.

6.135.1.40 sessionPID (read-only)

`unsigned long IMachine::sessionPID`

Identifier of the session process. This attribute contains the platform-dependent identifier of the process whose session was used with lockMachine() call. The returned value is only valid if sessionState is Locked or Unlocking by the time this property is read.

6.135.1.41 state (read-only)

`MachineState IMachine::state`

Current execution state of this machine.

6.135.1.42 lastStateChange (read-only)

`long long IMachine::lastStateChange`

Timestamp of the last execution state change, in milliseconds since 1970-01-01 UTC.

6.135.1.43 stateFilePath (read-only)

`wstring IMachine::stateFilePath`

Full path to the file that stores the execution state of the machine when it is in either the Saved or AbortedSaved state.

> **Note:** When the machine is not in the Saved or AbortedSaved state, this attribute is an empty string.

6.135.1.44 logFolder (read-only)

`wstring IMachine::logFolder`

Full path to the folder that stores a set of rotated log files recorded during machine execution. The most recent log file is named `VBox.log`, the previous log file is named `VBox.log.1` and so on (up to `VBox.log.3` in the current version).

6.135.1.45 currentSnapshot (read-only)

`ISnapshot IMachine::currentSnapshot`

Current snapshot of this machine. This is `null` if the machine currently has no snapshots. If it is not `null`, then it was set by one of takeSnapshot(), deleteSnapshot() or restoreSnapshot(), depending on which was called last. See ISnapshot for details.

6.135.1.46 snapshotCount (read-only)

`unsigned long IMachine::snapshotCount`

Number of snapshots taken on this machine. Zero means the machine doesn't have any snapshots.

6.135.1.47 currentStateModified (read-only)

`boolean IMachine::currentStateModified`

Returns `true` if the current state of the machine is not identical to the state stored in the current snapshot.

The current state is identical to the current snapshot only directly after one of the following calls are made:

- restoreSnapshot()
- takeSnapshot() (issued on a "powered off" or "saved" machine, for which settingsModified returns `false`)

The current state remains identical until one of the following happens:

- settings of the machine are changed
- the saved state is deleted
- the current snapshot is deleted
- an attempt to execute the machine is made

Note: For machines that don't have snapshots, this property is always `false`.

6.135.1.48 sharedFolders (read-only)

`ISharedFolder IMachine::sharedFolders[]`

Collection of shared folders for this machine (permanent shared folders). These folders are shared automatically at machine startup and available only to the guest OS installed within this machine.

New shared folders are added to the collection using createSharedFolder(). Existing shared folders can be removed using removeSharedFolder().

6.135.1.49 clipboardMode (read/write)

`ClipboardMode IMachine::clipboardMode`

Synchronization mode between the host OS clipboard and the guest OS clipboard.

6.135.1.50 clipboardFileTransfersEnabled (read/write)

```
boolean IMachine::clipboardFileTransfersEnabled
```

Sets or retrieves whether clipboard file transfers are allowed or not.
When set to `true`, clipboard file transfers between supported host and guest OSes are allowed.

6.135.1.51 dnDMode (read/write)

```
DnDMode IMachine::dnDMode
```

Sets or retrieves the current drag'n drop mode.

6.135.1.52 teleporterEnabled (read/write)

```
boolean IMachine::teleporterEnabled
```

When set to `true`, the virtual machine becomes a target teleporter the next time it is powered on. This can only set to `true` when the VM is in the `PoweredOff` or `Aborted` state.

6.135.1.53 teleporterPort (read/write)

```
unsigned long IMachine::teleporterPort
```

The TCP port the target teleporter will listen for incoming teleportations on.
0 means the port is automatically selected upon power on. The actual value can be read from this property while the machine is waiting for incoming teleportations.

6.135.1.54 teleporterAddress (read/write)

```
wstring IMachine::teleporterAddress
```

The address the target teleporter will listen on. If set to an empty string, it will listen on all addresses.

6.135.1.55 teleporterPassword (read/write)

```
wstring IMachine::teleporterPassword
```

The password to check for on the target teleporter. This is just a very basic measure to prevent simple hacks and operators accidentally beaming a virtual machine to the wrong place.
Note that you SET a plain text password while reading back a HASHED password. Setting a hashed password is currently not supported.

6.135.1.56 paravirtProvider (read/write)

```
ParavirtProvider IMachine::paravirtProvider
```

The paravirtualized guest interface provider.

6.135.1.57 IOCacheEnabled (read/write)

```
boolean IMachine::IOCacheEnabled
```

When set to `true`, the builtin I/O cache of the virtual machine will be enabled.

6.135.1.58 IOCacheSize (read/write)

```
unsigned long IMachine::IOCacheSize
```

Maximum size of the I/O cache in MB.

6.135.1.59 PCIDeviceAssignments (read-only)

```
IPCIDeviceAttachment IMachine::PCIDeviceAssignments[]
```

Array of PCI devices assigned to this machine, to get list of all PCI devices attached to the machine use IConsole::attachedPCIDevices[] attribute, as this attribute is intended to list only devices additional to what described in virtual hardware config. Usually, this list keeps host's physical devices assigned to the particular machine.

6.135.1.60 bandwidthControl (read-only)

```
IBandwidthControl IMachine::bandwidthControl
```

Bandwidth control manager.

6.135.1.61 tracingEnabled (read/write)

```
boolean IMachine::tracingEnabled
```

Enables the tracing facility in the VMM (including PDM devices + drivers). The VMM will consume about 0.5MB of more memory when enabled and there may be some extra overhead from tracepoints that are always enabled.

6.135.1.62 tracingConfig (read/write)

```
wstring IMachine::tracingConfig
```

Tracepoint configuration to apply at startup when tracingEnabled is true. The string specifies a space separated of tracepoint group names to enable. The special group 'all' enables all tracepoints. Check DBGFR3TracingConfig for more details on available tracepoint groups and such.

Note that on hosts supporting DTrace (or similar), a lot of the tracepoints may be implemented exclusively as DTrace probes. So, the effect of the same config may differ between Solaris and Windows for example.

6.135.1.63 allowTracingToAccessVM (read/write)

```
boolean IMachine::allowTracingToAccessVM
```

Enables tracepoints in PDM devices and drivers to use the VMCPU or VM structures when firing off trace points. This is especially useful with DTrace tracepoints, as it allows you to use the VMCPU or VM pointer to obtain useful information such as guest register state.

This is disabled by default because devices and drivers normally has no business accessing the VMCPU or VM structures, and are therefore unable to get any pointers to these.

6.135.1.64 autostartEnabled (read/write)

```
boolean IMachine::autostartEnabled
```

Enables autostart of the VM during system boot.

6.135.1.65 autostartDelay (read/write)

```
unsigned long IMachine::autostartDelay
```

Number of seconds to wait until the VM should be started during system boot.

6.135.1.66 autostopType (read/write)

```
AutostopType IMachine::autostopType
```

Action type to do when the system is shutting down.

6.135.1.67 defaultFrontend (read/write)

```
wstring IMachine::defaultFrontend
```

Selects which VM frontend should be used by default when launching this VM through the launchVMProcess() method. Empty or `null` strings do not define a particular default, it is up to launchVMProcess() to select one. See the description of launchVMProcess() for the valid frontend types.

This per-VM setting overrides the default defined by ISystemProperties::defaultFrontend attribute, and is overridden by a frontend type passed to launchVMProcess().

6.135.1.68 USBProxyAvailable (read-only)

```
boolean IMachine::USBProxyAvailable
```

Returns whether there is a USB proxy available.

6.135.1.69 VMProcessPriority (read/write)

```
VMProcPriority IMachine::VMProcessPriority
```

Sets the priority of the VM process. It is a VM setting which can be changed both before starting the VM and at runtime.

The default value is 'Default', which selects the default process priority.

6.135.1.70 VMExecutionEngine (read/write)

```
VMExecutionEngine IMachine::VMExecutionEngine
```

The configured VM execution engine. It is a VM setting which can be changed both before starting the VM and when a VM is saved.

The default value is 'Default', which selects the default aka best available execution engine available on the host.

6.135.1.71 paravirtDebug (read/write)

```
wstring IMachine::paravirtDebug
```

Debug parameters for the paravirtualized guest interface provider.

6.135.1.72 CPUProfile (read/write)

```
wstring IMachine::CPUProfile
```

Experimental feature to select the guest CPU profile. The default is "host", which indicates the host CPU. All other names are subject to change.

Use the ISystemProperties::getCPUProfiles() method to get currently available CPU profiles.

6.135.1.73 stateKeyId (read-only)

```
wstring IMachine::stateKeyId
```

Key Id of the password used for encrypting the state file. Internal use only for now.

6.135.1.74 stateKeyStore (read-only)

```
wstring IMachine::stateKeyStore
```

Key store used for encrypting the state file. Internal use only for now.

6.135.1.75 logKeyId (read-only)

```
wstring IMachine::logKeyId
```

Key Id of the password used for encrypting the log files. Internal use only for now.

6.135.1.76 logKeyStore (read-only)

```
wstring IMachine::logKeyStore
```

Key store used for encrypting the log files. Internal use only for now.

6.135.1.77 guestDebugControl (read-only)

```
IGuestDebugControl IMachine::guestDebugControl
```

Guest debugging configuration.

6.135.2 addEncryptionPassword

```
void IMachine::addEncryptionPassword(
            [in] wstring id,
            [in] wstring password)
```

id The identifier used for the password. Must match the identifier used when the encrypted VM was created.

password The password.

Adds a password used for encryption. Updates the accessibility state if password used the VM encryption.

If this method fails, the following error codes may be reported:

- VBOX_E_PASSWORD_INCORRECT: The password provided wasn't correct for the VM using the provided ID.

6.135.3 addEncryptionPasswords

```
void IMachine::addEncryptionPasswords(
            [in] wstring ids[],
            [in] wstring passwords[])
```

ids List of identifiers for the passwords. Must match the identifier used when the encrypted VM was created.

passwords List of passwords.

Adds passwords used for encryption. Updates the accessibility state if the list contains password used the VM encryption.

If this method fails, the following error codes may be reported:

- VBOX_E_PASSWORD_INCORRECT: The password provided wasn't correct for the VM using the provided ID.
- E_INVALIDARG: Id and passwords arrays have different size.

6.135.4 addStorageController

```
IStorageController IMachine::addStorageController(
            [in] wstring name,
            [in] StorageBus connectionType)
```

name

connectionType

Adds a new storage controller (SCSI, SAS or SATA controller) to the machine and returns it as an instance of IStorageController.

name identifies the controller for subsequent calls such as getStorageControllerByName(), getStorageControllerByInstance(), removeStorageController(), attachDevice() or mountMedium().

After the controller has been added, you can set its exact type by setting the IStorageController::controllerType.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_IN_USE: A storage controller with given name exists already.
- E_INVALIDARG: Invalid controllerType.

6.135.5 addUSBController

```
IUSBController IMachine::addUSBController(
            [in] wstring name,
            [in] USBControllerType type)
```

name

type

Adds a new USB controller to the machine and returns it as an instance of IUSBController.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_IN_USE: A USB controller with given type exists already.
- E_INVALIDARG: Invalid controllerType.

6.135.6 adoptSavedState

```
void IMachine::adoptSavedState(
            [in] wstring savedStateFile)
```

savedStateFile Path to the saved state file to adopt.

Associates the given saved state file to the virtual machine.

On success, the machine will go to the Saved state. The next time it is powered up it will be restored from the adopted saved state and continue execution from the place where the saved state file was created.

The specified saved state file path may be absolute or relative to the folder the VM normally saves the state to (usually, snapshotFolder).

> **Note:** It's a caller's responsibility to make sure the given saved state file is compatible with the settings of this virtual machine that represent its virtual hardware (memory size, storage disk configuration etc.). If there is a mismatch, the behavior of the virtual machine is undefined.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` Virtual machine state neither PoweredOff nor Aborted.

6.135.7 applyDefaults

```
void IMachine::applyDefaults(
           [in] wstring flags)
```

flags Additional flags, to be defined later.

Applies the defaults for the configured guest OS type. This is primarily for getting sane settings straight after creating a new VM, but it can also be applied later.

> **Note:** This is primarily a shortcut, centralizing the tedious job of getting the recommended settings and translating them into settings updates. The settings are made at the end of the call, but not saved.

If this method fails, the following error codes may be reported:

- `E_FAIL:` General error.
- `VBOX_E_INVALID_VM_STATE:` The machine is in invalid state.
- `VBOX_E_OBJECT_IN_USE:` Some of the applied objects already exist. The method has been called to already configured machine.

6.135.8 attachDevice

```
void IMachine::attachDevice(
           [in] wstring name,
           [in] long controllerPort,
           [in] long device,
           [in] DeviceType type,
           [in] IMedium medium)
```

name Name of the storage controller to attach the device to.

controllerPort Port to attach the device to. For an IDE controller, 0 specifies the primary controller and 1 specifies the secondary controller. For a SCSI controller, this must range from 0 to 15; for a SATA controller, from 0 to 29; for an SAS controller, from 0 to 7.

device Device slot in the given port to attach the device to. This is only relevant for IDE controllers, for which 0 specifies the master device and 1 specifies the slave device. For all other controller types, this must be 0.

type Device type of the attached device. For media opened by IVirtualBox::openMedium(), this must match the device type specified there.

medium Medium to mount or `null` for an empty drive.

Attaches a device and optionally mounts a medium to the given storage controller (IStorageController, identified by `name`), at the indicated port and device.

This method is intended for managing storage devices in general while a machine is powered off. It can be used to attach and detach fixed and removable media. The following kind of media can be attached to a machine:

- For fixed and removable media, you can pass in a medium that was previously opened using IVirtualBox::openMedium().
- Only for storage devices supporting removable media (such as DVDs and floppies), you can also specify a null pointer to indicate an empty drive or one of the medium objects listed in the IHost::DVDDrives[] and IHost::floppyDrives[] arrays to indicate a host drive. For removable devices, you can also use mountMedium() to change the media while the machine is running.

In a VM's default configuration of virtual machines, the secondary master of the IDE controller is used for a CD/DVD drive.

After calling this returns successfully, a new instance of IMediumAttachment will appear in the machine's list of medium attachments (see mediumAttachments[]).

See IMedium and IMediumAttachment for more information about attaching media.

The specified device slot must not have a device attached to it, or this method will fail.

> **Note:** You cannot attach a device to a newly created machine until this machine's settings are saved to disk using saveSettings().

> **Note:** If the medium is being attached indirectly, a new differencing medium will implicitly be created for it and attached instead. If the changes made to the machine settings (including this indirect attachment) are later cancelled using discardSettings(), this implicitly created differencing medium will implicitly be deleted.

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` SATA device, SATA port, IDE port or IDE slot out of range, or file or UUID not found.
- `VBOX_E_INVALID_OBJECT_STATE:` Machine must be registered before media can be attached.
- `VBOX_E_INVALID_VM_STATE:` Invalid machine state.
- `VBOX_E_OBJECT_IN_USE:` A medium is already attached to this or another virtual machine.

6.135.9 attachDeviceWithoutMedium

```
void IMachine::attachDeviceWithoutMedium(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device,
            [in] DeviceType type)
```

name Name of the storage controller to attach the device to.

controllerPort Port to attach the device to. For an IDE controller, 0 specifies the primary controller and 1 specifies the secondary controller. For a SCSI controller, this must range from 0 to 15; for a SATA controller, from 0 to 29; for an SAS controller, from 0 to 7.

device Device slot in the given port to attach the device to. This is only relevant for IDE controllers, for which 0 specifies the master device and 1 specifies the slave device. For all other controller types, this must be 0.

type Device type of the attached device. For media opened by IVirtualBox::openMedium(), this must match the device type specified there.

Attaches a device and optionally mounts a medium to the given storage controller (IStorageController, identified by `name`), at the indicated port and device.

This method is intended for managing storage devices in general while a machine is powered off. It can be used to attach and detach fixed and removable media. The following kind of media can be attached to a machine:

- For fixed and removable media, you can pass in a medium that was previously opened using IVirtualBox::openMedium().
- Only for storage devices supporting removable media (such as DVDs and floppies) with an empty drive or one of the medium objects listed in the IHost::DVDDrives[] and IHost::floppyDrives[] arrays to indicate a host drive. For removable devices, you can also use mountMedium() to change the media while the machine is running.

In a VM's default configuration of virtual machines, the secondary master of the IDE controller is used for a CD/DVD drive. IMediumAttachment will appear in the machine's list of medium attachments (see mediumAttachments[]).

See IMedium and IMediumAttachment for more information about attaching media.

The specified device slot must not have a device attached to it, or this method will fail.

> **Note:** You cannot attach a device to a newly created machine until this machine's settings are saved to disk using saveSettings().

> **Note:** If the medium is being attached indirectly, a new differencing medium will implicitly be created for it and attached instead. If the changes made to the machine settings (including this indirect attachment) are later cancelled using discardSettings(), this implicitly created differencing medium will implicitly be deleted.

If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: SATA device, SATA port, IDE port or IDE slot out of range, or file or UUID not found.

- VBOX_E_INVALID_OBJECT_STATE: Machine must be registered before media can be attached.
- VBOX_E_INVALID_VM_STATE: Invalid machine state.
- VBOX_E_OBJECT_IN_USE: A medium is already attached to this or another virtual machine.

6.135.10 attachHostPCIDevice

```
void IMachine::attachHostPCIDevice(
            [in] long hostAddress,
            [in] long desiredGuestAddress,
            [in] boolean tryToUnbind)
```

hostAddress Address of the host PCI device.

desiredGuestAddress Desired position of this device on guest PCI bus.

tryToUnbind If VMM shall try to unbind existing drivers from the device before attaching it to the guest.

Attaches host PCI device with the given (host) PCI address to the PCI bus of the virtual machine. Please note, that this operation is two phase, as real attachment will happen when VM will start, and most information will be delivered as IHostPCIDevicePlugEvent on IVirtualBox event source.

See also: IHostPCIDevicePlugEvent

If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine state is not stopped (PCI hotplug not yet implemented).
- VBOX_E_PDM_ERROR: Virtual machine does not have a PCI controller allowing attachment of physical devices.
- VBOX_E_NOT_SUPPORTED: Hardware or host OS doesn't allow PCI device passthrough.

6.135.11 canShowConsoleWindow

```
boolean IMachine::canShowConsoleWindow()
```

Returns `true` if the VM console process can activate the console window and bring it to foreground on the desktop of the host PC.

> **Note:** This method will fail if a session for this machine is not currently open.

If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Machine session is not open.

6.135.12 changeEncryption

```
IProgress IMachine::changeEncryption(
            [in] wstring currentPassword,
            [in] wstring cipher,
            [in] wstring newPassword,
            [in] wstring newPasswordId,
            [in] boolean force)
```

currentPassword The current password the VM is protected with. Use an empty string to indicate that the VM isn't encrypted.

cipher The cipher to use for encryption. An empty string indicates no encryption for the result.

newPassword The new password the VM should be protected with. An empty password and password ID will result in the VM being encrypted with the current password.

newPasswordId The ID of the new password when unlocking the VM.

force Force reencryption of the data if just password is changed. Otherwise, if data already encrypted and cipher doesn't changed only the password is changed.

Starts encryption of this VM. This means that the stored data of the VM is encrypted.
Please note that the results can be either returned straight away, or later as the result of the background operation via the object returned via the `progress` parameter.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: Encryption is not supported for various reasons e.g. unsupported cipher.

6.135.13 checkEncryptionPassword

```
void IMachine::checkEncryptionPassword(
            [in] wstring password)
```

password The password to check.

Checks whether the supplied password is correct for the VM.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: Encryption is not configured for this VM.
- `VBOX_E_PASSWORD_INCORRECT`: The given password is incorrect.

6.135.14 clearAllEncryptionPasswords

```
void IMachine::clearAllEncryptionPasswords()
```

Clears all provided VM passwords. The passwords can be removed only if the VM is powered off. Removing the passwords causes the VM goes to the inaccessible state and the password must be provided again.

6.135.15 cloneTo

```
IProgress IMachine::cloneTo(
            [in] IMachine target,
            [in] CloneMode mode,
            [in] CloneOptions options[])
```

target Target machine object.

mode Which states should be cloned.

options Options for the cloning operation.

Creates a clone of this machine, either as a full clone (which means creating independent copies of the hard disk media, save states and so on), or as a linked clone (which uses its own differencing media, sharing the parent media with the source machine).

The target machine object must have been created previously with IVirtualBox::createMachine(), and all the settings will be transferred except the VM name and the hardware UUID. You can set the VM name and the new hardware UUID when creating the target machine. The network MAC addresses are newly created for all enabled network adapters. You can change that behaviour with the options parameter. The operation is performed asynchronously, so the machine object will be not be usable until the `progress` object signals completion.

If this method fails, the following error codes may be reported:

- `E_INVALIDARG: target` is `null`.

6.135.16 createSharedFolder

```
void IMachine::createSharedFolder(
            [in] wstring name,
            [in] wstring hostPath,
            [in] boolean writable,
            [in] boolean automount,
            [in] wstring autoMountPoint)
```

name Unique logical name of the shared folder.

hostPath Full path to the shared folder in the host file system.

writable Whether the share is writable or read-only.

automount Whether the share gets automatically mounted by the guest or not.

autoMountPoint Where the guest should automatically mount the folder, if possible. For Windows and OS/2 guests this should be a drive letter, while other guests it should be a absolute directory.

Creates a new permanent shared folder by associating the given logical name with the given host path, adds it to the collection of shared folders and starts sharing it. Refer to the description of ISharedFolder to read more about logical names.

If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_IN_USE`: Shared folder already exists.
- `VBOX_E_FILE_ERROR`: Shared folder `hostPath` not accessible.

6.135.17 deleteConfig

```
IProgress IMachine::deleteConfig(
            [in] IMedium media[])
```

media List of media to be closed and whose storage files will be deleted.

Deletes the files associated with this machine from disk. If medium objects are passed in with the `aMedia` argument, they are closed and, if closing was successful, their storage files are deleted as well. For convenience, this array of media files can be the same as the one returned from a previous unregister() call.

This method must only be called on machines which are either write-locked (i.e. on instances returned by ISession::machine) or on unregistered machines (i.e. not yet registered machines created by IVirtualBox::createMachine() or opened by IVirtualBox::openMachine(), or after having called unregister()).

The following files will be deleted by this method:

- If unregister() had been previously called with a `cleanupMode` argument other than "UnregisterOnly", this will delete all saved state files that the machine had in use; possibly one if the machine was in either the "Saved" or "AbortedSaved" state and one for each online snapshot that the machine had.
- On each medium object passed in the `aMedia` array, this will call IMedium::close(). If that succeeds, this will attempt to delete the medium's storage on disk. Since the IMedium::close() call will fail if the medium is still in use, e.g. because it is still attached to a second machine; in that case the storage will not be deleted.
- Finally, the machine's own XML file will be deleted.

Since deleting large disk image files can be a time-consuming I/O operation, this method operates asynchronously and returns an IProgress object to allow the caller to monitor the progress. There will be one sub-operation for each file that is being deleted (saved state or medium storage file).

> **Note:** settingsModified will return `true` after this method successfully returns.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Machine is registered but not write-locked.
- `VBOX_E_IPRT_ERROR`: Could not delete the settings file.

6.135.18 deleteGuestProperty

```
void IMachine::deleteGuestProperty(
            [in] wstring name)
```

name The name of the property to delete.

Deletes an entry from the machine's guest property store.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Machine session is not open.

6.135.19 deleteSnapshot

```
IProgress IMachine::deleteSnapshot(
            [in] uuid id)
```

id UUID of the snapshot to delete.

Starts deleting the specified snapshot asynchronously. See ISnapshot for an introduction to snapshots.

The execution state and settings of the associated machine stored in the snapshot will be deleted. The contents of all differencing media of this snapshot will be merged with the contents of their dependent child media to keep the medium chain valid (in other words, all changes represented by media being deleted will be propagated to their child medium). After that, this snapshot's differencing medium will be deleted. The parent of this snapshot will become a new parent for all its child snapshots.

If the deleted snapshot is the current one, its parent snapshot will become a new current snapshot. The current machine state is not directly affected in this case, except that currently attached differencing media based on media of the deleted snapshot will be also merged as described above.

If the deleted snapshot is the first or current snapshot, then the respective IMachine attributes will be adjusted. Deleting the current snapshot will also implicitly call saveSettings() to make all current machine settings permanent.

Deleting a snapshot has the following preconditions:

- Child media of all normal media of the deleted snapshot must be accessible (see IMedium::state) for this operation to succeed. If only one running VM refers to all images which participates in merging the operation can be performed while the VM is running. Otherwise all virtual machines whose media are directly or indirectly based on the media of deleted snapshot must be powered off. In any case, online snapshot deleting usually is slower than the same operation without any running VM.
- You cannot delete the snapshot if a medium attached to it has more than one child medium (differencing images) because otherwise merging would be impossible. This might be the case if there is more than one child snapshot or differencing images were created for other reason (e.g. implicitly because of multiple machine attachments).

The virtual machine's state is changed to "DeletingSnapshot", "DeletingSnapshotOnline" or "DeletingSnapshotPaused" while this operation is in progress.

> **Note:** Merging medium contents can be very time and disk space consuming, if these media are big in size and have many children. However, if the snapshot being deleted is the last (head) snapshot on the branch, the operation will be rather quick.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: The running virtual machine prevents deleting this snapshot. This happens only in very specific situations, usually snapshots can be deleted without trouble while a VM is running. The error message text explains the reason for the failure.

6.135.20 deleteSnapshotAndAllChildren

```
IProgress IMachine::deleteSnapshotAndAllChildren(
            [in] uuid id)
```

id UUID of the snapshot to delete, including all its children.

Starts deleting the specified snapshot and all its children asynchronously. See ISnapshot for an introduction to snapshots. The conditions and many details are the same as with deleteSnapshot().

This operation is very fast if the snapshot subtree does not include the current state. It is still significantly faster than deleting the snapshots one by one if the current state is in the subtree and there are more than one snapshots from current state to the snapshot which marks the subtree, since it eliminates the incremental image merging.

> **Note:** This API method is right now not implemented!

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` The running virtual machine prevents deleting this snapshot. This happens only in very specific situations, usually snapshots can be deleted without trouble while a VM is running. The error message text explains the reason for the failure.
- `E_NOTIMPL:` The method is not implemented yet.

6.135.21 deleteSnapshotRange

```
IProgress IMachine::deleteSnapshotRange(
          [in] uuid startId,
          [in] uuid endId)
```

startId UUID of the first snapshot to delete.

endId UUID of the last snapshot to delete.

Starts deleting the specified snapshot range. This is limited to linear snapshot lists, which means there may not be any other child snapshots other than the direct sequence between the start and end snapshot. If the start and end snapshot point to the same snapshot this method is completely equivalent to deleteSnapshot(). See ISnapshot for an introduction to snapshots. The conditions and many details are the same as with deleteSnapshot().

This operation is generally faster than deleting snapshots one by one and often also needs less extra disk space before freeing up disk space by deleting the removed disk images corresponding to the snapshot.

> **Note:** This API method is right now not implemented!

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` The running virtual machine prevents deleting this snapshot. This happens only in very specific situations, usually snapshots can be deleted without trouble while a VM is running. The error message text explains the reason for the failure.
- `E_NOTIMPL:` The method is not implemented yet.

6.135.22 detachDevice

```
void IMachine::detachDevice(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device)
```

name Name of the storage controller to detach the medium from.

controllerPort Port number to detach the medium from.

device Device slot number to detach the medium from.

Detaches the device attached to a device slot of the specified bus.

Detaching the device from the virtual machine is deferred. This means that the medium remains associated with the machine when this method returns and gets actually de-associated only after a successful saveSettings() call. See IMedium for more detailed information about attaching media.

> **Note:** You cannot detach a device from a running machine.

> **Note:** Detaching differencing media implicitly created by attachDevice() for the indirect attachment using this method will **not** implicitly delete them. The IMedium::deleteStorage() operation should be explicitly performed by the caller after the medium is successfully detached and the settings are saved with saveSettings(), if it is the desired action.

If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Attempt to detach medium from a running virtual machine.
- VBOX_E_OBJECT_NOT_FOUND: No medium attached to given slot/bus.
- VBOX_E_NOT_SUPPORTED: Medium format does not support storage deletion (only for implicitly created differencing media, should not happen).

6.135.23 detachHostPCIDevice

```
void IMachine::detachHostPCIDevice(
            [in] long hostAddress)
```

hostAddress Address of the host PCI device.

Detach host PCI device from the virtual machine. Also HostPCIDevicePlugEvent on IVirtualBox event source will be delivered. As currently we don't support hot device unplug, IHostPCIDevicePlugEvent event is delivered immediately.

See also: IHostPCIDevicePlugEvent

If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine state is not stopped (PCI hotplug not yet implemented).
- VBOX_E_OBJECT_NOT_FOUND: This host device is not attached to this machine.
- VBOX_E_PDM_ERROR: Virtual machine does not have a PCI controller allowing attachment of physical devices.
- VBOX_E_NOT_SUPPORTED: Hardware or host OS doesn't allow PCI device passthrough.

6.135.24 discardSavedState

```
void IMachine::discardSavedState(
            [in] boolean fRemoveFile)
```

fRemoveFile Whether to also remove the saved state file.

Forcibly resets the machine to "Powered Off" state if it is currently in the "Saved" state previously created by saveState()) or in the "AbortedSaved" state. The next time the machine is powered up a clean boot will occur.

> **Note:** This operation is equivalent to resetting or powering off the machine without doing a proper shutdown of the guest operating system; as with resetting a running phyiscal computer, it can can lead to data loss.

If `fRemoveFile` is `true`, the file in the machine directory into which the machine state was saved is also deleted. If this is `false`, then the state can be recovered and later re-inserted into a machine using adoptSavedState(). The location of the file can be found in the stateFilePath attribute.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine not in either the Saved or AbortedSaved state.

6.135.25 discardSettings

```
void IMachine::discardSettings()
```

Discards any changes to the machine settings made since the session has been opened or since the last call to saveSettings() or discardSettings().

> **Note:** Calling this method is only valid on instances returned by ISession::machine and on new machines created by IVirtualBox::createMachine() or opened by IVirtualBox::openMachine() but not yet registered, or on unregistered machines after calling unregister().

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine is not mutable.

6.135.26 enumerateGuestProperties

```
void IMachine::enumerateGuestProperties(
            [in] wstring patterns,
            [out] wstring names[],
            [out] wstring values[],
            [out] long long timestamps[],
            [out] wstring flags[])
```

patterns The patterns to match the properties against, separated by '|' characters. If this is empty or `null`, all properties will match.

names The names of the properties returned.

values The values of the properties returned. The array entries match the corresponding entries in the `name` array.

timestamps The timestamps of the properties returned. The array entries match the corresponding entries in the name array.

flags The flags of the properties returned. The array entries match the corresponding entries in the name array.

Return a list of the guest properties matching a set of patterns along with their values, timestamps and flags.

6.135.27 exportTo

```
IVirtualSystemDescription IMachine::exportTo(
            [in] IAppliance appliance,
            [in] wstring location)
```

appliance Appliance to export this machine to.

location The target location.

Exports the machine to an OVF appliance. See IAppliance for the steps required to export VirtualBox machines to OVF.

6.135.28 findSnapshot

```
ISnapshot IMachine::findSnapshot(
            [in] wstring nameOrId)
```

nameOrId What to search for. Name or UUID of the snapshot to find

Returns a snapshot of this machine with the given name or UUID.

Returns a snapshot of this machine with the given UUID. A null argument can be used to obtain the first snapshot taken on this machine. To traverse the whole tree of snapshots starting from the root, inspect the root snapshot's ISnapshot::children[] attribute and recurse over those children.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: Virtual machine has no snapshots or snapshot not found.

6.135.29 getBootOrder

```
DeviceType IMachine::getBootOrder(
            [in] unsigned long position)
```

position Position in the boot order (1 to the total number of devices the machine can boot from, as returned by IPlatformProperties::maxBootPosition).

Returns the device type that occupies the specified position in the boot order.

@todo [remove?] If the machine can have more than one device of the returned type (such as hard disks), then a separate method should be used to retrieve the individual device that occupies the given position.

If here are no devices at the given position, then Null is returned.

@todo getHardDiskBootOrder(), getNetworkBootOrder()

If this method fails, the following error codes may be reported:

- E_INVALIDARG: Boot position out of range.

6.135.30 getCPUStatus

```
boolean IMachine::getCPUStatus(
           [in] unsigned long cpu)
```

cpu The CPU id to check for.

Returns the current status of the given CPU.

6.135.31 getEffectiveParavirtProvider

```
ParavirtProvider IMachine::getEffectiveParavirtProvider()
```

Returns the effective paravirtualization provider for this VM.

6.135.32 getEncryptionSettings

```
void IMachine::getEncryptionSettings(
           [out] wstring cipher,
           [out] wstring passwordId)
```

cipher The cipher used for encryption.

passwordId The ID of the password when unlocking the VM.

Returns the encryption settings for this VM.
If this method fails, the following error codes may be reported:

- VBOX_E_NOT_SUPPORTED: Encryption is not configured for this VM.

6.135.33 getExtraData

```
wstring IMachine::getExtraData(
           [in] wstring key)
```

key Name of the data key to get.

Returns associated machine-specific extra data.
If the requested data key does not exist, this function will succeed and return an empty string in the value argument.
If this method fails, the following error codes may be reported:

- VBOX_E_FILE_ERROR: Settings file not accessible.
- VBOX_E_XML_ERROR: Could not parse the settings file.

6.135.34 getExtraDataKeys

```
wstring[] IMachine::getExtraDataKeys()
```

Returns an array representing the machine-specific extra data keys which currently have values defined.

6.135.35 getGuestProperty

```
void IMachine::getGuestProperty(
            [in] wstring name,
            [out] wstring value,
            [out] long long timestamp,
            [out] wstring flags)
```

name The name of the property to read.

value The value of the property. If the property does not exist then this will be empty.

timestamp The time at which the property was last modified, as seen by the server process.

flags Additional property parameters, passed as a comma-separated list of "name=value" type entries.

Reads an entry from the machine's guest property store.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Machine session is not open.

6.135.36 getGuestPropertyTimestamp

```
long long IMachine::getGuestPropertyTimestamp(
            [in] wstring property)
```

property The name of the property to read.

Reads a property timestamp from the machine's guest property store.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Machine session is not open.

6.135.37 getGuestPropertyValue

```
wstring IMachine::getGuestPropertyValue(
            [in] wstring property)
```

property The name of the property to read.

Reads a value from the machine's guest property store.
If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Machine session is not open.

6.135.38 getMedium

```
IMedium IMachine::getMedium(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device)
```

name Name of the storage controller the medium is attached to.

controllerPort Port to query.

device Device slot in the given port to query.

Returns the virtual medium attached to a device slot of the specified bus.

Note that if the medium was indirectly attached by mountMedium() to the given device slot then this method will return not the same object as passed to the mountMedium() call. See IMedium for more detailed information about mounting a medium.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: No medium attached to given slot/bus.

6.135.39 getMediumAttachment

```
IMediumAttachment IMachine::getMediumAttachment(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device)
```

name

controllerPort

device

Returns a medium attachment which corresponds to the controller with the given name, on the given port and device slot.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: No attachment exists for the given controller/port/device combination.

6.135.40 getMediumAttachmentsOfController

```
IMediumAttachment[] IMachine::getMediumAttachmentsOfController(
            [in] wstring name)
```

name

Returns an array of medium attachments which are attached to the controller with the given name.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: A storage controller with given name doesn't exist.

6.135.41 getNetworkAdapter

```
INetworkAdapter IMachine::getNetworkAdapter(
            [in] unsigned long slot)
```

slot

Returns the network adapter associated with the given slot. Slots are numbered sequentially, starting with zero. The total number of adapters per machine is defined by the IPlatformProperties::getMaxNetworkAdapters() property, so the maximum slot number is one less than that property's value.

If this method fails, the following error codes may be reported:

- E_INVALIDARG: Invalid slot number.

6.135.42 getParallelPort

```
IParallelPort IMachine::getParallelPort(
           [in] unsigned long slot)
```

slot

Returns the parallel port associated with the given slot. Slots are numbered sequentially, starting with zero. The total number of parallel ports per machine is defined by the IPlatformProperties::parallelPortCount property, so the maximum slot number is one less than that property's value.

If this method fails, the following error codes may be reported:

- E_INVALIDARG: Invalid slot number.

6.135.43 getSerialPort

```
ISerialPort IMachine::getSerialPort(
           [in] unsigned long slot)
```

slot

Returns the serial port associated with the given slot. Slots are numbered sequentially, starting with zero. The total number of serial ports per machine is defined by the IPlatformProperties::serialPortCount property, so the maximum slot number is one less than that property's value.

If this method fails, the following error codes may be reported:

- E_INVALIDARG: Invalid slot number.

6.135.44 getStorageControllerByInstance

```
IStorageController IMachine::getStorageControllerByInstance(
           [in] StorageBus connectionType,
           [in] unsigned long instance)
```

connectionType

instance

Returns a storage controller of a specific storage bus with the given instance number.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: A storage controller with given instance number doesn't exist.

6.135.45 getStorageControllerByName

```
IStorageController IMachine::getStorageControllerByName(
           [in] wstring name)
```

name

Returns a storage controller with the given name.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: A storage controller with given name doesn't exist.

6.135.46 getUSBControllerByName

```
IUSBController IMachine::getUSBControllerByName(
            [in] wstring name)
```

name

Returns a USB controller with the given type.
If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: A USB controller with given name doesn't exist.

6.135.47 getUSBControllerCountByType

```
unsigned long IMachine::getUSBControllerCountByType(
            [in] USBControllerType type)
```

type

Returns the number of USB controllers of the given type attached to the VM.

6.135.48 hotPlugCPU

```
void IMachine::hotPlugCPU(
            [in] unsigned long cpu)
```

cpu The CPU id to insert.

Plugs a CPU into the machine.

6.135.49 hotUnplugCPU

```
void IMachine::hotUnplugCPU(
            [in] unsigned long cpu)
```

cpu The CPU id to remove.

Removes a CPU from the machine.

6.135.50 launchVMProcess

```
IProgress IMachine::launchVMProcess(
            [in] ISession session,
            [in] wstring name,
            [in] wstring environmentChanges[])
```

session Client session object to which the VM process will be connected (this must be in "Unlocked" state).

name Front-end to use for the new VM process. The following are currently supported:

- "gui": VirtualBox Qt GUI front-end
- "headless": VBoxHeadless (VRDE Server) front-end
- "sdl": VirtualBox SDL front-end
- "emergencystop": reserved value, used for aborting the currently running VM or session owner. In this case the session parameter may be null (if it is non-null it isn't used in any way), and the progress return value will be always null. The operation completes immediately.

- `""`: use the per-VM default frontend if set, otherwise the global default defined in the system properties. If neither are set, the API will launch a `"gui"` session, which may fail if there is no windowing environment available. See defaultFrontend and ISystemProperties::defaultFrontend.

environmentChanges The list of putenv-style changes to the VM process environment.

Spawns a new process that will execute the virtual machine and obtains a shared lock on the machine for the calling session.

If launching the VM succeeds, the new VM process will create its own session and write-lock the machine for it, preventing conflicting changes from other processes. If the machine is already locked (because it is already running or because another session has a write lock), launching the VM process will therefore fail. Reversely, future attempts to obtain a write lock will also fail while the machine is running.

The caller's session object remains separate from the session opened by the new VM process. It receives its own IConsole object which can be used to control machine execution, but it cannot be used to change all VM settings which would be available after a lockMachine() call.

The caller must eventually release the session's shared lock by calling ISession::unlockMachine() on the local session object once this call has returned. However, the session's state (see ISession::state) will not return to "Unlocked" until the remote session has also unlocked the machine (i.e. the machine has stopped running).

Launching a VM process can take some time (a new VM is started in a new process, for which memory and other resources need to be set up). Because of this, an IProgress object is returned to allow the caller to wait for this asynchronous operation to be completed. Until then, the caller's session object remains in the "Unlocked" state, and its ISession::machine and ISession::console attributes cannot be accessed. It is recommended to use IProgress::waitForCompletion() or similar calls to wait for completion. Completion is signalled when the VM is powered on. If launching the VM fails, error messages can be queried via the progress object, if available.

The progress object will have at least 2 sub-operations. The first operation covers the period up to the new VM process calls powerUp. The subsequent operations mirror the IConsole::powerUp() progress object. Because IConsole::powerUp() may require some extra sub-operations, the IProgress::operationCount may change at the completion of operation.

For details on the teleportation progress operation, see IConsole::powerUp().

The `environmentChanges` argument is a list of strings where every string contains environment variable in the putenv style, i.e. "VAR=VALUE" for setting/replacing and "VAR" for unsetting. These environment variables will be applied to the environment of the VirtualBox server process. If an environment variable exists both in the server process and in this list, the value from this list takes precedence over the server's variable. If the value of the environment variable is omitted, this variable will be removed from the resulting environment. If the list is empty, the server environment is inherited by the started process as is.

If this method fails, the following error codes may be reported:

- `E_UNEXPECTED`: Virtual machine not registered.
- `E_INVALIDARG`: Invalid session type `type`.
- `VBOX_E_OBJECT_NOT_FOUND`: No machine matching `machineId` found.
- `VBOX_E_INVALID_OBJECT_STATE`: Session already open or being opened.
- `VBOX_E_IPRT_ERROR`: Launching process for machine failed.
- `VBOX_E_VM_ERROR`: Failed to assign machine to session.

6.135.51 lockMachine

```
void IMachine::lockMachine(
            [in] ISession session,
            [in] LockType lockType)
```

session Session object for which the machine will be locked.

lockType If set to `Write`, then attempt to acquire an exclusive write lock or fail. If set to `Shared`, then either acquire an exclusive write lock or establish a link to an existing session.

Locks the machine for the given session to enable the caller to make changes to the machine or start the VM or control VM execution.

There are two ways to lock a machine for such uses:

- If you want to make changes to the machine settings, you must obtain an exclusive write lock on the machine by setting `lockType` to `Write`.

  This will only succeed if no other process has locked the machine to prevent conflicting changes. Only after an exclusive write lock has been obtained using this method, one can change all VM settings or execute the VM in the process space of the session object. (Note that the latter is only of interest if you actually want to write a new front-end for virtual machines; but this API gets called internally by the existing front-ends such as VBoxHeadless and the VirtualBox GUI to acquire a write lock on the machine that they are running.)

  On success, write-locking the machine for a session creates a second copy of the IMachine object. It is this second object upon which changes can be made; in VirtualBox terminology, the second copy is "mutable". It is only this second, mutable machine object upon which you can call methods that change the machine state. After having called this method, you can obtain this second, mutable machine object using the ISession::machine attribute.

- If you only want to check the machine state or control machine execution without actually changing machine settings (e.g. to get access to VM statistics or take a snapshot or save the machine state), then set the `lockType` argument to `Shared`.

  If no other session has obtained a lock, you will obtain an exclusive write lock as described above. However, if another session has already obtained such a lock, then a link to that existing session will be established which allows you to control that existing session.

  To find out which type of lock was obtained, you can inspect ISession::type, which will have been set to either `WriteLock` or `Shared`.

In either case, you can get access to the IConsole object which controls VM execution.

Also in all of the above cases, one must always call ISession::unlockMachine() to release the lock on the machine, or the machine's state will eventually be set to "Aborted".

To change settings on a machine, the following sequence is typically performed:

1. Call this method to obtain an exclusive write lock for the current session.
2. Obtain a mutable IMachine object from ISession::machine.
3. Change the settings of the machine by invoking IMachine methods.
4. Call saveSettings().
5. Release the write lock by calling ISession::unlockMachine().

If this method fails, the following error codes may be reported:

- `E_UNEXPECTED:` Virtual machine not registered.

- E_ACCESSDENIED: Process not started by launchVMProcess().
- VBOX_E_INVALID_OBJECT_STATE: Session already open or being opened.
- VBOX_E_VM_ERROR: Failed to assign machine to session.

6.135.52 mountMedium

```
void IMachine::mountMedium(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device,
            [in] IMedium medium,
            [in] boolean force)
```

name Name of the storage controller to attach the medium to.

controllerPort Port to attach the medium to.

device Device slot in the given port to attach the medium to.

medium Medium to mount or null for an empty drive.

force Allows to force unmount/mount of a medium which is locked by the device slot in the given port to attach the medium to.

Mounts a medium (IMedium, identified by the given UUID id) to the given storage controller (IStorageController, identified by name), at the indicated port and device. The device must already exist; see attachDevice() for how to attach a new device.

This method is intended only for managing removable media, where the device is fixed but media is changeable at runtime (such as DVDs and floppies). It cannot be used for fixed media such as hard disks.

The controllerPort and device parameters specify the device slot and have have the same meaning as with attachDevice().

The specified device slot can have a medium mounted, which will be unmounted first. Specifying a zero UUID (or an empty string) for medium does just an unmount.

See IMedium for more detailed information about attaching media.

If this method fails, the following error codes may be reported:

- E_INVALIDARG: SATA device, SATA port, IDE port or IDE slot out of range.
- VBOX_E_INVALID_OBJECT_STATE: Attempt to attach medium to an unregistered virtual machine.
- VBOX_E_INVALID_VM_STATE: Invalid machine state.
- VBOX_E_OBJECT_IN_USE: Medium already attached to this or another virtual machine.

6.135.53 moveTo

```
IProgress IMachine::moveTo(
            [in] wstring folder,
            [in] wstring type)
```

folder Target folder where machine is moved. May be the same folder where the VM already is located or the empty string, in which case the machine is kept in this location and the disk images and other files which are stored elsewhere are moved.

type Type of moving. Possible values: basic - Only the files which belong solely to this machine are moved from the original machine's folder to a new folder.

Move machine on to new place/folder

6.135.54 nonRotationalDevice

```
void IMachine::nonRotationalDevice(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device,
            [in] boolean nonRotational)
```

name Name of the storage controller.

controllerPort Storage controller port.

device Device slot in the given port.

nonRotational New value for the non-rotational device flag.

Sets a flag in the device information which indicates that the medium is not based on rotational technology, i.e. that the access times are more or less independent of the position on the medium. This may or may not be supported by a particular drive, and is silently ignored in the latter case. At the moment only hard disks (which is a misnomer in this context) accept this setting. Changing the setting while the VM is running is forbidden. The device must already exist; see attachDevice() for how to attach a new device.

The `controllerPort` and `device` parameters specify the device slot and have have the same meaning as with attachDevice().

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` SATA device, SATA port, IDE port or IDE slot out of range.
- `VBOX_E_INVALID_OBJECT_STATE:` Attempt to modify an unregistered virtual machine.
- `VBOX_E_INVALID_VM_STATE:` Invalid machine state.

6.135.55 passthroughDevice

```
void IMachine::passthroughDevice(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device,
            [in] boolean passthrough)
```

name Name of the storage controller.

controllerPort Storage controller port.

device Device slot in the given port.

passthrough New value for the passthrough setting.

Sets the passthrough mode of an existing DVD device. Changing the setting while the VM is running is forbidden. The setting is only used if at VM start the device is configured as a host DVD drive, in all other cases it is ignored. The device must already exist; see attachDevice() for how to attach a new device.

The `controllerPort` and `device` parameters specify the device slot and have have the same meaning as with attachDevice().

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` SATA device, SATA port, IDE port or IDE slot out of range.
- `VBOX_E_INVALID_OBJECT_STATE:` Attempt to modify an unregistered virtual machine.
- `VBOX_E_INVALID_VM_STATE:` Invalid machine state.

6.135.56 queryLogFilename

```
wstring IMachine::queryLogFilename(
            [in] unsigned long idx)
```

idx Which log file name to query. 0=current log file.

Queries for the VM log file name of an given index. Returns an empty string if a log file with that index doesn't exists.

6.135.57 querySavedGuestScreenInfo

```
void IMachine::querySavedGuestScreenInfo(
            [in] unsigned long screenId,
            [out] unsigned long originX,
            [out] unsigned long originY,
            [out] unsigned long width,
            [out] unsigned long height,
            [out] boolean enabled)
```

screenId Saved guest screen to query info from.

originX The X position of the guest monitor top left corner.

originY The Y position of the guest monitor top left corner.

width Guest width at the time of the saved state was taken.

height Guest height at the time of the saved state was taken.

enabled Whether the monitor is enabled in the guest.

Returns the guest dimensions from the saved state.

6.135.58 querySavedScreenshotInfo

```
BitmapFormat[] IMachine::querySavedScreenshotInfo(
            [in] unsigned long screenId,
            [out] unsigned long width,
            [out] unsigned long height)
```

screenId Saved guest screen to query info from.

width Image width.

height Image height.

Returns available formats and size of the screenshot from saved state.

6.135.59 readLog

```
octet[] IMachine::readLog(
            [in] unsigned long idx,
            [in] long long offset,
            [in] long long size)
```

idx Which log file to read. 0=current log file.

offset Offset in the log file.

size Chunk size to read in the log file.

Reads the VM log file. The chunk size is limited, so even if you ask for a big piece there might be less data returned.

6.135.60 readSavedScreenshotToArray

```
octet[] IMachine::readSavedScreenshotToArray(
            [in] unsigned long screenId,
            [in] BitmapFormat bitmapFormat,
            [out] unsigned long width,
            [out] unsigned long height)
```

screenId Saved guest screen to read from.

bitmapFormat The requested format.

width Image width.

height Image height.

Screenshot in requested format is retrieved to an array of bytes.

6.135.61 readSavedThumbnailToArray

```
octet[] IMachine::readSavedThumbnailToArray(
            [in] unsigned long screenId,
            [in] BitmapFormat bitmapFormat,
            [out] unsigned long width,
            [out] unsigned long height)
```

screenId Saved guest screen to read from.

bitmapFormat The requested format.

width Bitmap width.

height Bitmap height.

Thumbnail is retrieved to an array of bytes in the requested format.

6.135.62 removeEncryptionPassword

```
void IMachine::removeEncryptionPassword(
            [in] wstring id)
```

id The identifier used for the password. Must match the identifier used when the encrypted VM was created.

Removes a password used for the VM encryption/decryption. The password can be removed only if the VM is powered off. Removing the password causes the VM goes to the inaccessible state and the password must be provided again.

> **Note:** If machine becomes inaccessible all passwords are purged. One has to add required passwords again using either addEncryptionPassword() or addEncryptionPasswords() methods.

If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: The VM is not in proper state e.g powered on.

6.135.63 removeSharedFolder

```
void IMachine::removeSharedFolder(
            [in] wstring name)
```

name Logical name of the shared folder to remove.

Removes the permanent shared folder with the given name previously created by createSharedFolder() from the collection of shared folders and stops sharing it.

If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine is not mutable.
- VBOX_E_OBJECT_NOT_FOUND: Shared folder name does not exist.

6.135.64 removeStorageController

```
void IMachine::removeStorageController(
            [in] wstring name)
```

name

Removes a storage controller from the machine with all devices attached to it.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: A storage controller with given name doesn't exist.
- VBOX_E_NOT_SUPPORTED: Medium format does not support storage deletion (only for implicitly created differencing media, should not happen).

6.135.65 removeUSBController

```
void IMachine::removeUSBController(
            [in] wstring name)
```

name

Removes a USB controller from the machine.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: A USB controller with given type doesn't exist.

6.135.66 restoreSnapshot

```
IProgress IMachine::restoreSnapshot(
            [in] ISnapshot snapshot)
```

snapshot The snapshot to restore the VM state from.

Starts resetting the machine's current state to the state contained in the given snapshot, asynchronously. All current settings of the machine will be reset and changes stored in differencing media will be lost. See ISnapshot for an introduction to snapshots.

After this operation is successfully completed, new empty differencing media are created for all normal media of the machine.

If the given snapshot is an online snapshot, the machine will go to the Saved state, so that the next time it is powered on, the execution state will be restored from the state of the snapshot.

Note: The machine must not be running, otherwise the operation will fail.

Note: If the machine is in the Saved state prior to this operation, the saved state file will be implicitly deleted (as if discardSavedState() were called).

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` Virtual machine is running.

6.135.67 saveSettings

```
void IMachine::saveSettings()
```

Saves any changes to machine settings made since the session has been opened or a new machine has been created, or since the last call to saveSettings() or discardSettings(). For registered machines, new settings become visible to all other VirtualBox clients after successful invocation of this method.

Note: The method sends IMachineDataChangedEvent notification event after the configuration has been successfully saved (only for registered machines).

Note: Calling this method is only valid on instances returned by ISession::machine and on new machines created by IVirtualBox::createMachine() but not yet registered, or on unregistered machines after calling unregister().

If this method fails, the following error codes may be reported:

- `VBOX_E_FILE_ERROR:` Settings file not accessible.
- `VBOX_E_XML_ERROR:` Could not parse the settings file.
- `E_ACCESSDENIED:` Modification request refused.

6.135.68 saveState

```
IProgress IMachine::saveState()
```

Saves the current execution state of a running virtual machine and stops its execution.

After this operation completes, the machine will go to the Saved state. The next time it is powered up this state will be restored and the machine will continue its execution from the place where it was saved.

This operation differs from taking a snapshot to the effect that it doesn't create new differencing media. Also, once the machine is powered up from the state saved using this method, the saved state is deleted, so it will be impossible to return to this state later.

Note: On success, this method implicitly calls saveSettings() to save all current machine settings (including runtime changes to the DVD medium, etc.). Together with the impossibility to change any VM settings when it is in the Saved state, this guarantees adequate hardware configuration of the machine when it is restored from the saved state file.

Note: The machine must be in the Running or Paused state, otherwise the operation will fail.

See also: takeSnapshot()
If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` Virtual machine state neither Running nor Paused.
- `VBOX_E_FILE_ERROR:` Failed to create directory for saved state file.

6.135.69 setAutoDiscardForDevice

```
void IMachine::setAutoDiscardForDevice(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device,
            [in] boolean discard)
```

name Name of the storage controller.

controllerPort Storage controller port.

device Device slot in the given port.

discard New value for the discard device flag.

Sets a flag in the device information which indicates that the medium supports discarding unused blocks (called trimming for SATA or unmap for SCSI devices) .This may or may not be supported by a particular drive, and is silently ignored in the latter case. At the moment only hard disks (which is a misnomer in this context) accept this setting. Changing the setting while the VM is running is forbidden. The device must already exist; see attachDevice() for how to attach a new device.
The `controllerPort` and `device` parameters specify the device slot and have have the same meaning as with attachDevice().
If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` SATA device, SATA port, SCSI port out of range.
- `VBOX_E_INVALID_OBJECT_STATE:` Attempt to modify an unregistered virtual machine.
- `VBOX_E_INVALID_VM_STATE:` Invalid machine state.

6.135.70 setBandwidthGroupForDevice

```
void IMachine::setBandwidthGroupForDevice(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device,
            [in] IBandwidthGroup bandwidthGroup)
```

name Name of the storage controller.

controllerPort Storage controller port.

device Device slot in the given port.

bandwidthGroup New value for the bandwidth group or `null` for no group.

Sets the bandwidth group of an existing storage device. The device must already exist; see attachDevice() for how to attach a new device.

The `controllerPort` and `device` parameters specify the device slot and have have the same meaning as with attachDevice().

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` SATA device, SATA port, IDE port or IDE slot out of range.
- `VBOX_E_INVALID_OBJECT_STATE:` Attempt to modify an unregistered virtual machine.
- `VBOX_E_INVALID_VM_STATE:` Invalid machine state.

6.135.71 setBootOrder

```
void IMachine::setBootOrder(
            [in] unsigned long position,
            [in] DeviceType device)
```

position Position in the boot order (1 to the total number of devices the machine can boot from, as returned by IPlatformProperties::maxBootPosition).

device The type of the device used to boot at the given position.

Puts the given device to the specified position in the boot order.

To indicate that no device is associated with the given position, Null should be used.

@todo setHardDiskBootOrder(), setNetworkBootOrder()

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` Boot `position` out of range.
- `E_NOTIMPL:` Booting from USB `device` currently not supported.

6.135.72 setExtraData

```
void IMachine::setExtraData(
            [in] wstring key,
            [in] wstring value)
```

key Name of the data key to set.

value Value to assign to the key.

Sets associated machine-specific extra data.

If you pass `null` or an empty string as a key `value`, the given `key` will be deleted.

> **Note:** Key must contain printable (non-control) UTF-8 characters only.

> **Note:** Before performing the actual data change, this method will ask all registered event listeners using the IExtraDataCanChangeEvent notification for a permission. If one of the listeners refuses the new value, the change will not be performed.

> **Note:** On success, the IExtraDataChangedEvent notification is called to inform all registered listeners about a successful data change.

Note: This method can be called outside the machine session and therefore it's a caller's responsibility to handle possible race conditions when several clients change the same key at the same time.

If this method fails, the following error codes may be reported:

- VBOX_E_FILE_ERROR: Settings file not accessible.
- VBOX_E_XML_ERROR: Could not parse the settings file.
- E_INVALIDARG: Key contains invalid characters.

6.135.73 setGuestProperty

```
void IMachine::setGuestProperty(
            [in] wstring property,
            [in] wstring value,
            [in] wstring flags)
```

property The name of the property to set, change or delete.

value The new value of the property to set, change or delete. If the property does not yet exist and value is non-empty, it will be created. If the value is null or empty, the property will be deleted if it exists.

flags Additional property parameters, passed as a comma-separated list of "name=value" type entries.

Sets, changes or deletes an entry in the machine's guest property store.
If this method fails, the following error codes may be reported:

- E_ACCESSDENIED: Property cannot be changed.
- E_INVALIDARG: Invalid flags.
- VBOX_E_INVALID_VM_STATE: Virtual machine is not mutable or session not open.
- VBOX_E_INVALID_OBJECT_STATE: Cannot set transient property when machine not running.

6.135.74 setGuestPropertyValue

```
void IMachine::setGuestPropertyValue(
            [in] wstring property,
            [in] wstring value)
```

property The name of the property to set or change.

value The new value of the property to set or change. If the property does not yet exist and value is non-empty, it will be created.

Sets or changes a value in the machine's guest property store. The flags field will be left unchanged or created empty for a new property.
If this method fails, the following error codes may be reported:

- E_ACCESSDENIED: Property cannot be changed.
- VBOX_E_INVALID_VM_STATE: Virtual machine is not mutable or session not open.
- VBOX_E_INVALID_OBJECT_STATE: Cannot set transient property when machine not running.

6.135.75 setHotPluggableForDevice

```
void IMachine::setHotPluggableForDevice(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device,
            [in] boolean hotPluggable)
```

name Name of the storage controller.

controllerPort Storage controller port.

device Device slot in the given port.

hotPluggable New value for the hot-pluggable device flag.

Sets a flag in the device information which indicates that the attached device is hot pluggable or not. This may or may not be supported by a particular controller and/or drive, and is silently ignored in the latter case. Changing the setting while the VM is running is forbidden. The device must already exist; see attachDevice() for how to attach a new device.

The `controllerPort` and `device` parameters specify the device slot and have have the same meaning as with attachDevice().

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` SATA device, SATA port, IDE port or IDE slot out of range.
- `VBOX_E_INVALID_OBJECT_STATE:` Attempt to modify an unregistered virtual machine.
- `VBOX_E_INVALID_VM_STATE:` Invalid machine state.
- `VBOX_E_NOT_SUPPORTED:` Controller doesn't support hot plugging.

6.135.76 setNoBandwidthGroupForDevice

```
void IMachine::setNoBandwidthGroupForDevice(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device)
```

name Name of the storage controller.

controllerPort Storage controller port.

device Device slot in the given port.

Sets no bandwidth group for an existing storage device. The device must already exist; see attachDevice() for how to attach a new device. The `controllerPort` and `device` parameters specify the device slot and have have the same meaning as with attachDevice().

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` SATA device, SATA port, IDE port or IDE slot out of range.
- `VBOX_E_INVALID_OBJECT_STATE:` Attempt to modify an unregistered virtual machine.
- `VBOX_E_INVALID_VM_STATE:` Invalid machine state.

6.135.77 setSettingsFilePath

```
IProgress IMachine::setSettingsFilePath(
            [in] wstring settingsFilePath)
```

settingsFilePath New settings file path, will be used to determine the new location for the attached media if it is in the same directory or below as the original settings file.

Currently, it is an error to change this property on any machine. Later this will allow setting a new path for the settings file, with automatic relocation of all files (including snapshots and disk images) which are inside the base directory. This operation is only allowed when there are no pending unsaved settings.

> **Note:** Setting this property to `null` or to an empty string is forbidden. When setting this property, the specified path must be absolute. The specified path may not exist, it will be created when necessary.

If this method fails, the following error codes may be reported:

- `E_NOTIMPL:` The operation is not implemented yet.

6.135.78 setStorageControllerBootable

```
void IMachine::setStorageControllerBootable(
            [in] wstring name,
            [in] boolean bootable)
```

name

bootable

Sets the bootable flag of the storage controller with the given name.
If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND:` A storage controller with given name doesn't exist.
- `VBOX_E_OBJECT_IN_USE:` Another storage controller is marked as bootable already.

6.135.79 showConsoleWindow

```
long long IMachine::showConsoleWindow()
```

Activates the console window and brings it to foreground on the desktop of the host PC. Many modern window managers on many platforms implement some sort of focus stealing prevention logic, so that it may be impossible to activate a window without the help of the currently active application. In this case, this method will return a non-zero identifier that represents the top-level window of the VM console process. The caller, if it represents a currently active process, is responsible to use this identifier (in a platform-dependent manner) to perform actual window activation.

> **Note:** This method will fail if a session for this machine is not currently open.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` Machine session is not open.

6.135.80 takeSnapshot

```
IProgress IMachine::takeSnapshot(
            [in] wstring name,
            [in] wstring description,
            [in] boolean pause,
            [out] uuid id)
```

name Short name for the snapshot.

description Optional description of the snapshot.

pause Whether the VM should be paused while taking the snapshot. Only relevant when the VM is running, and distinguishes between online (`true`) and live (`false`) snapshots. When the VM is not running the result is always an offline snapshot.

id UUID of the snapshot which will be created. Useful for follow-up operations after the snapshot has been created.

Saves the current execution state and all settings of the machine and creates differencing images for all normal (non-independent) media. See ISnapshot for an introduction to snapshots.

This method can be called for a PoweredOff, Saved (see saveState()), AbortedSaved, Running or Paused virtual machine. When the machine is PoweredOff, an offline snapshot is created. When the machine is Running a live snapshot is created, and an online snapshot is created when Paused.

The taken snapshot is always based on the current snapshot of the associated virtual machine and becomes a new current snapshot.

> **Note:** This method implicitly calls saveSettings() to save all current machine settings before taking an offline snapshot.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: Virtual machine currently changing state.

6.135.81 temporaryEjectDevice

```
void IMachine::temporaryEjectDevice(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device,
            [in] boolean temporaryEject)
```

name Name of the storage controller.

controllerPort Storage controller port.

device Device slot in the given port.

temporaryEject New value for the eject behavior.

Sets the behavior for guest-triggered medium eject. In some situations it is desirable that such ejects update the VM configuration, and in others the eject should keep the VM configuration. The device must already exist; see attachDevice() for how to attach a new device.

The `controllerPort` and `device` parameters specify the device slot and have have the same meaning as with attachDevice().

If this method fails, the following error codes may be reported:

- E_INVALIDARG: SATA device, SATA port, IDE port or IDE slot out of range.
- VBOX_E_INVALID_OBJECT_STATE: Attempt to modify an unregistered virtual machine.
- VBOX_E_INVALID_VM_STATE: Invalid machine state.

6.135.82 unmountMedium

```
void IMachine::unmountMedium(
            [in] wstring name,
            [in] long controllerPort,
            [in] long device,
            [in] boolean force)
```

name Name of the storage controller to unmount the medium from.

controllerPort Port to unmount the medium from.

device Device slot in the given port to unmount the medium from.

force Allows to force unmount of a medium which is locked by the device slot in the given port medium is attached to.

Unmounts any currently mounted medium (IMedium, identified by the given UUID id) to the given storage controller (IStorageController, identified by name), at the indicated port and device. The device must already exist;

This method is intended only for managing removable media, where the device is fixed but media is changeable at runtime (such as DVDs and floppies). It cannot be used for fixed media such as hard disks.

The controllerPort and device parameters specify the device slot and have have the same meaning as with attachDevice().

The specified device slot must have a medium mounted, which will be unmounted. If there is no mounted medium it will do nothing. See IMedium for more detailed information about attaching/unmounting media.

If this method fails, the following error codes may be reported:

- E_INVALIDARG: SATA device, SATA port, IDE port or IDE slot out of range.
- VBOX_E_INVALID_OBJECT_STATE: Attempt to unmount medium that is not removable - not DVD or floppy.
- VBOX_E_INVALID_VM_STATE: Invalid machine state.
- VBOX_E_OBJECT_IN_USE: Medium already attached to this or another virtual machine.
- VBOX_E_OBJECT_NOT_FOUND: Medium not attached to specified port, device, controller.

6.135.83 unregister

```
IMedium[] IMachine::unregister(
            [in] CleanupMode cleanupMode)
```

cleanupMode How to clean up after the machine has been unregistered.

Unregisters a machine previously registered with IVirtualBox::registerMachine() and optionally do additional cleanup before the machine is unregistered.

This method does not delete any files. It only changes the machine configuration and the list of registered machines in the VirtualBox object. To delete the files which belonged to the

machine, including the XML file of the machine itself, call deleteConfig(), optionally with the array of IMedium objects which was returned from this method.

How thoroughly this method cleans up the machine configuration before unregistering the machine depends on the `cleanupMode` argument.

- With "UnregisterOnly", the machine will only be unregistered, but no additional cleanup will be performed. The call will fail if the machine has any snapshots or any media attached (see IMediumAttachment). It is the responsibility of the caller to delete all such configuration in this mode. In this mode, the API behaves like the former `IVirtualBox`::unregisterMachine() API which it replaces.
- With "DetachAllReturnNone", the call will succeed even if the machine is in "Saved" state or if it has snapshots or media attached. All media attached to the current machine state or in snapshots will be detached. No medium objects will be returned; all of the machine's media will remain open.
- With "DetachAllReturnHardDisksOnly", the call will behave like with "DetachAllReturnNone", except that all the hard disk medium objects which were detached from the machine will be returned as an array. This allows for quickly passing them to the deleteConfig() API for closing and deletion.
- With "Full", the call will behave like with "DetachAllReturnHardDisksOnly", except that all media will be returned in the array, including removable media like DVDs and floppies. This might be useful if the user wants to inspect in detail which media were attached to the machine. Be careful when passing the media array to deleteConfig() in that case because users will typically want to preserve ISO and RAW image files.

A typical implementation will use "DetachAllReturnHardDisksOnly" and then pass the resulting IMedium array to deleteConfig(). This way, the machine is completely deleted with all its saved states and hard disk images, but images for removable drives (such as ISO and RAW files) will remain on disk.

This API does not verify whether the media files returned in the array are still attached to other machines (i.e. shared between several machines). If such a shared image is passed to deleteConfig() however, closing the image will fail there and the image will be silently skipped.

This API may, however, move media from this machine's media registry to other media registries (see IMedium for details on media registries). For machines created with VirtualBox 4.0 or later, if media from this machine's media registry are also attached to another machine (shared attachments), each such medium will be moved to another machine's registry. This is because without this machine's media registry, the other machine cannot find its media any more and would become inaccessible.

This API implicitly calls saveSettings() to save all current machine settings before unregistering it. It may also silently call saveSettings() on other machines if media are moved to other machines' media registries.

After successful method invocation, the IMachineRegisteredEvent event is fired.

The call will fail if the machine is currently locked (see ISession).

> **Note:** If the given machine is inaccessible (see accessible), it will be unregistered and fully uninitialized right afterwards. As a result, the returned machine object will be unusable and an attempt to call **any** method will return the "Object not ready" error.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_OBJECT_STATE:` Machine is currently locked for a session.

6.136 IMachineDataChangedEvent (IMachineEvent)

> **Note:** This interface extends IMachineEvent and therefore supports all its methods and attributes as well.

Any of the settings of the given machine has changed.

6.136.1 Attributes

6.136.1.1 temporary (read-only)

```
boolean IMachineDataChangedEvent::temporary
```

`true` if the settings change is temporary. All permanent settings changes will trigger an event, and only temporary settings changes for running VMs will trigger an event. Note: sending events for temporary changes is NOT IMPLEMENTED.

6.137 IMachineDebugger

6.137.1 Attributes

6.137.1.1 singleStep (read/write)

```
boolean IMachineDebugger::singleStep
```

Switch for enabling single-stepping.

6.137.1.2 executeAllInIEM (read/write)

```
boolean IMachineDebugger::executeAllInIEM
```

Whether to execute all the code in the instruction interpreter.

When `true` the VM execution engine changes to VMExecutionEngine::Interpreter (or VMExecutionEngine::Recompiler when recompiledIEMExecution is set to `true`).

This is mainly for temporarily testing the interpreter and its recompiler.

See also IMachine::VMExecutionEngine for how to permanently configure a VM to run in VMExecutionEngine::Interpreter mode.

6.137.1.3 logEnabled (read/write)

```
boolean IMachineDebugger::logEnabled
```

Switch for enabling and disabling the debug logger.

6.137.1.4 logDbgFlags (read-only)

```
wstring IMachineDebugger::logDbgFlags
```

The debug logger flags.

6.137.1.5 logDbgGroups (read-only)

```
wstring IMachineDebugger::logDbgGroups
```

The debug logger's group settings.

6.137.1.6 logDbgDestinations (read-only)

```
wstring IMachineDebugger::logDbgDestinations
```

The debug logger's destination settings.

6.137.1.7 logRelFlags (read-only)

```
wstring IMachineDebugger::logRelFlags
```

The release logger flags.

6.137.1.8 logRelGroups (read-only)

```
wstring IMachineDebugger::logRelGroups
```

The release logger's group settings.

6.137.1.9 logRelDestinations (read-only)

```
wstring IMachineDebugger::logRelDestinations
```

The release logger's destination settings.

6.137.1.10 executionEngine (read-only)

```
VMExecutionEngine IMachineDebugger::executionEngine
```

Gets the main execution engine of the VM.

6.137.1.11 HWVirtExNestedPagingEnabled (read-only)

```
boolean IMachineDebugger::HWVirtExNestedPagingEnabled
```

Flag indicating whether the VM is currently making use of the nested paging CPU hardware virtualization extension.

6.137.1.12 HWVirtExVPIDEnabled (read-only)

```
boolean IMachineDebugger::HWVirtExVPIDEnabled
```

Flag indicating whether the VM is currently making use of the VPID VT-x extension.

6.137.1.13 HWVirtExUXEnabled (read-only)

```
boolean IMachineDebugger::HWVirtExUXEnabled
```

Flag indicating whether the VM is currently making use of the unrestricted execution feature of VT-x.

6.137.1.14 OSName (read-only)

```
wstring IMachineDebugger::OSName
```

Query the guest OS kernel name as detected by the DBGF.
This feature is not implemented in the 4.0.0 release but may show up in a dot release.

6.137.1.15 OSVersion (read-only)

`wstring IMachineDebugger::OSVersion`

Query the guest OS kernel version string as detected by the DBGF.
This feature is not implemented in the 4.0.0 release but may show up in a dot release.

6.137.1.16 PAEEnabled (read-only)

`boolean IMachineDebugger::PAEEnabled`

Flag indicating whether the VM is currently making use of the Physical Address Extension CPU feature.

6.137.1.17 virtualTimeRate (read/write)

`unsigned long IMachineDebugger::virtualTimeRate`

The rate at which the virtual time runs expressed as a percentage. The accepted range is 2% to 20000%.

6.137.1.18 uptime (read-only)

`long long IMachineDebugger::uptime`

VM uptime in milliseconds, i.e. time in which it could have been executing guest code. Excludes the time when the VM was paused.

6.137.1.19 recompiledIEMExecution (read/write)

`boolean IMachineDebugger::recompiledIEMExecution`

Whether to use the recompiler when executing in the interpreter.
This is used together with executeAllInIEM to temporarily change the main VM execution engine to VMExecutionEngine::Recompiler.
If the VM is configured to run in VMExecutionEngine::Interpreter via IMachine::VMExecutionEngine, setting this to `true` temporarily changes it to VMExecutionEngine::Recompiler.
This mainly for testing the instruction interpreter in recompiler mode.
See also IMachine::VMExecutionEngine for how to permanently configure a VM to run in VMExecutionEngine::Recompiler mode.

6.137.2 detectOS

`wstring IMachineDebugger::detectOS()`

Tries to (re-)detect the guest OS kernel.
This feature is not implemented in the 4.0.0 release but may show up in a dot release.

6.137.3 dumpGuestCore

```
void IMachineDebugger::dumpGuestCore(
           [in] wstring filename,
           [in] wstring compression)
```

filename The name of the output file. The file must not exist.

compression Reserved for future compression method indicator.

Takes a core dump of the guest.
See include/VBox/dbgfcorefmt.h for details on the file format.

6.137.4 dumpGuestStack

```
wstring IMachineDebugger::dumpGuestStack(
            [in] unsigned long cpuId)
```

cpuId The identifier of the Virtual CPU.

Produce a simple stack dump using the current guest state.
This feature is not implemented in the 4.0.0 release but may show up in a dot release.

6.137.5 dumpHostProcessCore

```
void IMachineDebugger::dumpHostProcessCore(
            [in] wstring filename,
            [in] wstring compression)
```

filename The name of the output file. The file must not exist.

compression Reserved for future compression method indicator.

Takes a core dump of the VM process on the host.
This feature is not implemented in the 4.0.0 release but it may show up in a dot release.

6.137.6 dumpStats

```
void IMachineDebugger::dumpStats(
            [in] wstring pattern)
```

pattern The selection pattern. A bit similar to filename globbing. Wildcards are '*' and '?', where the asterisk matches zero or more characters and question mark matches exactly one character. Multiple pattern can be joined by putting '|' between them.

Dumps VM statistics.

6.137.7 getCPULoad

```
long long IMachineDebugger::getCPULoad(
            [in] unsigned long cpuId,
            [out] unsigned long pctExecuting,
            [out] unsigned long pctHalted,
            [out] unsigned long pctOther)
```

cpuId The ID of the virtual CPU to retrieve stats for, pass 0x7fffffff or higher for the average accross all CPUs.

pctExecuting Percentage of the interval that the CPU(s) spend executing guest code.

pctHalted Percentage of the interval that the CPU(s) spend halted.

pctOther Percentage of the interval that the CPU(s) preempted by the host scheduler, on virtualization overhead and on other tasks.

Get the load percentages (as observed by the VMM) for all virtual CPUs or a specific one.

6.137.8 getRegister

```
wstring IMachineDebugger::getRegister(
            [in] unsigned long cpuId,
            [in] wstring name)
```

cpuId The identifier of the Virtual CPU.

name The register name, case is ignored.

Gets one register.

6.137.9 getRegisters

```
void IMachineDebugger::getRegisters(
            [in] unsigned long cpuId,
            [out] wstring names[],
            [out] wstring values[])
```

cpuId The identifier of the Virtual CPU.

names Array containing the lowercase register names.

values Array parallel to the names holding the register values as if the register was returned by getRegister().

Gets all the registers for the given CPU.

6.137.10 getStats

```
wstring IMachineDebugger::getStats(
            [in] wstring pattern,
            [in] boolean withDescriptions)
```

pattern The selection pattern. A bit similar to filename globbing. Wildchars are '*' and '?', where the asterisk matches zero or more characters and question mark matches exactly one character. Multiple pattern can be joined by putting '|' between them.

withDescriptions Whether to include the descriptions.

Get the VM statistics in a XMLish format.

6.137.11 getUVMAndVMMFunctionTable

```
long long IMachineDebugger::getUVMAndVMMFunctionTable(
            [in] long long magicVersion,
            [out] long long VMMFunctionTable)
```

magicVersion The VMMR3VTABLE_MAGIC_VERSION value of the caller. The method will fail if this is not compatible with the VMM function table.

VMMFunctionTable The VMM function table address.

Gets the user-mode VM handle, with a reference, and the VMM function table. The VM handle must be passed to VMR3ReleaseUVM when done. This can only be called from within the VM process, remote calls will always fail.

6.137.12 info

```
wstring IMachineDebugger::info(
            [in] wstring name,
            [in] wstring args)
```

name The name of the info item.

args Arguments to the info dumper.

Interfaces with the info dumpers (DBGFInfo).
This feature is not implemented in the 4.0.0 release but it may show up in a dot release.

6.137.13 injectNMI

```
void IMachineDebugger::injectNMI()
```

Inject an NMI into a running VT-x/AMD-V VM.

6.137.14 loadPlugIn

```
wstring IMachineDebugger::loadPlugIn(
            [in] wstring name)
```

name The plug-in name or DLL. Special name 'all' loads all installed plug-ins.

Loads a DBGF plug-in.

6.137.15 modifyLogDestinations

```
void IMachineDebugger::modifyLogDestinations(
            [in] wstring settings)
```

settings The destination settings string. See iprt/log.h for details. To target the release logger, prefix the string with "release:".

Modifies the debug or release logger destinations.

6.137.16 modifyLogFlags

```
void IMachineDebugger::modifyLogFlags(
            [in] wstring settings)
```

settings The flags settings string. See iprt/log.h for details. To target the release logger, prefix the string with "release:".

Modifies the debug or release logger flags.

6.137.17 modifyLogGroups

```
void IMachineDebugger::modifyLogGroups(
            [in] wstring settings)
```

settings The group settings string. See iprt/log.h for details. To target the release logger, prefix the string with "release:".

Modifies the group settings of the debug or release logger.

6.137.18 queryOSKernelLog

```
wstring IMachineDebugger::queryOSKernelLog(
            [in] unsigned long maxMessages)
```

maxMessages Max number of messages to return, counting from the end of the log. If 0, there is no limit.

Tries to get the kernel log (dmesg) of the guest OS.

6.137.19 readPhysicalMemory

```
octet[] IMachineDebugger::readPhysicalMemory(
            [in] long long address,
            [in] unsigned long size)
```

address The guest physical address.

size The number of bytes to read.

Reads guest physical memory, no side effects (MMIO++).
This feature is not implemented in the 4.0.0 release but may show up in a dot release.

6.137.20 readVirtualMemory

```
octet[] IMachineDebugger::readVirtualMemory(
            [in] unsigned long cpuId,
            [in] long long address,
            [in] unsigned long size)
```

cpuId The identifier of the Virtual CPU.

address The guest virtual address.

size The number of bytes to read.

Reads guest virtual memory, no side effects (MMIO++).
This feature is not implemented in the 4.0.0 release but may show up in a dot release.

6.137.21 resetStats

```
void IMachineDebugger::resetStats(
            [in] wstring pattern)
```

pattern The selection pattern. A bit similar to filename globbing. Wildcards are '*' and '?', where the asterisk matches zero or more characters and question mark matches exactly one character. Multiple pattern can be joined by putting '|' between them.

Reset VM statistics.

6.137.22 setRegister

```
void IMachineDebugger::setRegister(
            [in] unsigned long cpuId,
            [in] wstring name,
            [in] wstring value)
```

cpuId The identifier of the Virtual CPU.

name The register name, case is ignored.

value The new register value. Hexadecimal, decimal and octal formats are supported in addition to any special formats returned by the getters.

Gets one register.
This feature is not implemented in the 4.0.0 release but may show up in a dot release.

6.137.23 setRegisters

```
void IMachineDebugger::setRegisters(
            [in] unsigned long cpuId,
            [in] wstring names[],
            [in] wstring values[])
```

cpuId The identifier of the Virtual CPU.

names Array containing the register names, case ignored.

values Array parallel to the names holding the register values. See setRegister() for formatting guidelines.

Sets zero or more registers atomically.
This feature is not implemented in the 4.0.0 release but may show up in a dot release.

6.137.24 takeGuestSample

```
IProgress IMachineDebugger::takeGuestSample(
            [in] wstring filename,
            [in] unsigned long usInterval,
            [in] long long usSampleTime)
```

filename The file to dump the report to.

usInterval The sample interval.

usSampleTime The number of microseconds to sample.

Creates a sample report of the guest and emulated device activity.

6.137.25 unloadPlugIn

```
void IMachineDebugger::unloadPlugIn(
            [in] wstring name)
```

name The plug-in name or DLL. Special name 'all' unloads all plug-ins.

Unloads a DBGF plug-in.

6.137.26 writePhysicalMemory

```
void IMachineDebugger::writePhysicalMemory(
            [in] long long address,
            [in] unsigned long size,
            [in] octet bytes[])
```

address The guest physical address.

size The number of bytes to read.

bytes The bytes to write.

Writes guest physical memory, access handles (MMIO++) are ignored.
This feature is not implemented in the 4.0.0 release but may show up in a dot release.

6.137.27 writeVirtualMemory

```
void IMachineDebugger::writeVirtualMemory(
           [in] unsigned long cpuId,
           [in] long long address,
           [in] unsigned long size,
           [in] octet bytes[])
```

cpuId The identifier of the Virtual CPU.

address The guest virtual address.

size The number of bytes to read.

bytes The bytes to write.

Writes guest virtual memory, access handles (MMIO++) are ignored.
This feature is not implemented in the 4.0.0 release but may show up in a dot release.

6.138 IMachineEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Base abstract interface for all machine events.

6.138.1 Attributes

6.138.1.1 machineId (read-only)

```
uuid IMachineEvent::machineId
```

ID of the machine this event relates to.

6.139 IMachineGroupsChangedEvent (IMachineEvent)

> **Note:** This interface extends IMachineEvent and therefore supports all its methods and attributes as well.

The groups of the machine have changed.

6.139.1 Attributes

6.139.1.1 dummy (read-only)

```
boolean IMachineGroupsChangedEvent::dummy
```

Dummy state because you can't have an event without attributes apparently.

6.140 IMachineRegisteredEvent (IMachineEvent)

Note: This interface extends IMachineEvent and therefore supports all its methods and attributes as well.

The given machine was registered or unregistered within this VirtualBox installation.

6.140.1 Attributes

6.140.1.1 registered (read-only)

```
boolean IMachineRegisteredEvent::registered
```

If `true`, the machine was registered, otherwise it was unregistered.

6.141 IMachineStateChangedEvent (IMachineEvent)

Note: This interface extends IMachineEvent and therefore supports all its methods and attributes as well.

Machine state change event.

6.141.1 Attributes

6.141.1.1 state (read-only)

```
MachineState IMachineStateChangedEvent::state
```

New execution state.

6.142 IManagedObjectRef

Note: This interface is supported in the web service only, not in COM/XPCOM.

Managed object reference.

Only within the webservice, a managed object reference (which is really an opaque number) allows a webservice client to address an object that lives in the address space of the webservice server.

Behind each managed object reference, there is a COM object that lives in the webservice server's address space. The COM object is not freed until the managed object reference is released, either by an explicit call to release() or by logging off from the webservice (IWebsessionManager::logoff()), which releases all objects created during the webservice session.

Whenever a method call of the VirtualBox API returns a COM object, the webservice representation of that method will instead return a managed object reference, which can then be used to invoke methods on that object.

6.142.1 getInterfaceName

```
wstring IManagedObjectRef::getInterfaceName()
```

Returns the name of the interface that this managed object represents, for example, "IMachine", as a string.

6.142.2 release

```
void IManagedObjectRef::release()
```

Releases this managed object reference and frees the resources that were allocated for it in the webservice server process. After calling this method, the identifier of the reference can no longer be used.

6.143 IMedium

The IMedium interface represents virtual storage for a machine's hard disks, CD/DVD or floppy drives. It will typically represent a disk image on the host, for example a VDI or VMDK file representing a virtual hard disk, or an ISO or RAW file representing virtual removable media, but can also point to a network location (e.g. for iSCSI targets).

Instances of IMedium are connected to virtual machines by way of medium attachments, which link the storage medium to a particular device slot of a storage controller of the virtual machine. In the VirtualBox API, virtual storage is therefore always represented by the following chain of object links:

- IMachine::storageControllers[] contains an array of storage controllers (IDE, SATA, SCSI, SAS or a floppy controller; these are instances of IStorageController).
- IMachine::mediumAttachments[] contains an array of medium attachments (instances of IMediumAttachment created by IMachine::attachDevice()), each containing a storage controller from the above array, a port/device specification, and an instance of IMedium representing the medium storage (image file).

  For removable media, the storage medium is optional; a medium attachment with no medium represents a CD/DVD or floppy drive with no medium inserted. By contrast, hard disk attachments will always have an IMedium object attached.
- Each IMedium in turn points to a storage unit (such as a file on the host computer or a network resource) that holds actual data. This location is represented by the location attribute.

Existing media are opened using IVirtualBox::openMedium(); new hard disk media can be created with the VirtualBox API using the IVirtualBox::createMedium() method. Differencing hard disks (see below) are usually implicitly created by VirtualBox as needed, but may also be created explicitly using createDiffStorage(). VirtualBox cannot create CD/DVD or floppy images (ISO and RAW files); these should be created with external tools and then opened from within VirtualBox.

Only for CD/DVDs and floppies, an IMedium instance can also represent a host drive. In that case the id attribute contains the UUID of one of the drives in IHost::DVDDrives[] or IHost::floppyDrives[].

Media registries

When a medium has been opened or created using one of the aforementioned APIs, it becomes "known" to VirtualBox. Known media can be attached to virtual machines and re-found through IVirtualBox::openMedium(). They also appear in the global IVirtualBox::hardDisks[], IVirtualBox::DVDImages[] and IVirtualBox::floppyImages[] arrays.

Prior to VirtualBox 4.0, opening a medium added it to a global media registry in the VirtualBox.xml file, which was shared between all machines and made transporting machines and their media from one host to another difficult.

Starting with VirtualBox 4.0, media are only added to a registry when they are *attached* to a machine using IMachine::attachDevice(). For backwards compatibility, which registry a medium is added to depends on which VirtualBox version created a machine:

- If the medium has first been attached to a machine which was created by VirtualBox 4.0 or later, it is added to that machine's media registry in the machine XML settings file. This way all information about a machine's media attachments is contained in a single file and can be transported easily.
- For older media attachments (i.e. if the medium was first attached to a machine which was created with a VirtualBox version before 4.0), media continue to be registered in the global VirtualBox settings file, for backwards compatibility.

See IVirtualBox::openMedium() for more information.

Media are removed from media registries by the close(), deleteStorage() and mergeTo() methods.

Accessibility checks

VirtualBox defers media accessibility checks until the refreshState() method is called explicitly on a medium. This is done to make the VirtualBox object ready for serving requests as fast as possible and let the end-user application decide if it needs to check media accessibility right away or not.

As a result, when VirtualBox starts up (e.g. the VirtualBox object gets created for the first time), all known media are in the "Inaccessible" state, but the value of the lastAccessError attribute is an empty string because no actual accessibility check has been made yet.

After calling refreshState(), a medium is considered *accessible* if its storage unit can be read. In that case, the state attribute has a value of "Created". If the storage unit cannot be read (for example, because it is located on a disconnected network resource, or was accidentally deleted outside VirtualBox), the medium is considered *inaccessible*, which is indicated by the "Inaccessible" state. The exact reason why the medium is inaccessible can be obtained by reading the lastAccessError attribute.

Medium types

There are five types of medium behavior which are stored in the type attribute (see MediumType) and which define the medium's behavior with attachments and snapshots.

All media can be also divided in two groups: *base* media and *differencing* media. A base medium contains all sectors of the medium data in its own storage and therefore can be used independently. In contrast, a differencing medium is a "delta" to some other medium and contains only those sectors which differ from that other medium, which is then called a *parent*. The differencing medium is said to be *linked to* that parent. The parent may be itself a differencing medium, thus forming a chain of linked media. The last element in that chain must always be a base medium. Note that several differencing media may be linked to the same parent medium.

Differencing media can be distinguished from base media by querying the parent attribute: base media do not have parents they would depend on, so the value of this attribute is always `null` for them. Using this attribute, it is possible to walk up the medium tree (from the child medium to its parent). It is also possible to walk down the tree using the children[] attribute.

Note that the type of all differencing media is "normal"; all other values are meaningless for them. Base media may be of any type.

Automatic composition of the file name part

Another extension to the location attribute is that there is a possibility to cause VirtualBox to compose a unique value for the file name part of the location using the UUID of the hard disk. This applies only to hard disks in NotCreated state, e.g. before the storage unit is created, and works as follows. You set the value of the location attribute to a location specification which only

contains the path specification but not the file name part and ends with either a forward slash or a backslash character. In response, VirtualBox will generate a new UUID for the hard disk and compose the file name using the following pattern:

```
<path>/{<uuid>}.<ext>
```

where <path> is the supplied path specification, <uuid> is the newly generated UUID and <ext> is the default extension for the storage format of this hard disk. After that, you may call any of the methods that create a new hard disk storage unit and they will use the generated UUID and file name.

6.143.1 Attributes

6.143.1.1 id (read-only)

`uuid IMedium::id`

UUID of the medium. For a newly created medium, this value is a randomly generated UUID.

> **Note:** For media in one of MediumState_NotCreated, MediumState_Creating or MediumState_Deleting states, the value of this property is undefined and will most likely be an empty UUID.

6.143.1.2 description (read/write)

`wstring IMedium::description`

Optional description of the medium. For a newly created medium the value of this attribute is an empty string.

Medium types that don't support this attribute will return E_NOTIMPL in attempt to get or set this attribute's value.

> **Note:** For some storage types, reading this attribute may return an outdated (last known) value when state is Inaccessible or LockedWrite because the value of this attribute is stored within the storage unit itself. Also note that changing the attribute value is not possible in such case, as well as when the medium is the LockedRead state.

6.143.1.3 state (read-only)

`MediumState IMedium::state`

Returns the current medium state, which is the last state set by the accessibility check performed by refreshState(). If that method has not yet been called on the medium, the state is "Inaccessible"; as opposed to truly inaccessible media, the value of lastAccessError will be an empty string in that case.

> **Note:** As of version 3.1, this no longer performs an accessibility check automatically; call refreshState() for that.

6.143.1.4 variant (read-only)

```
MediumVariant IMedium::variant[]
```

Returns the storage format variant information for this medium as an array of the flags described at MediumVariant. Before refreshState() is called this method returns an undefined value.

6.143.1.5 location (read/write)

```
wstring IMedium::location
```

Location of the storage unit holding medium data.

The format of the location string is medium type specific. For medium types using regular files in a host's file system, the location string is the full file name.

6.143.1.6 name (read-only)

```
wstring IMedium::name
```

Name of the storage unit holding medium data.

The returned string is a short version of the location attribute that is suitable for representing the medium in situations where the full location specification is too long (such as lists and comboboxes in GUI frontends). This string is also used by frontends to sort the media list alphabetically when needed.

For example, for locations that are regular files in the host's file system, the value of this attribute is just the file name (+ extension), without the path specification.

Note that as opposed to the location attribute, the name attribute will not necessary be unique for a list of media of the given type and format.

6.143.1.7 deviceType (read-only)

```
DeviceType IMedium::deviceType
```

Kind of device (DVD/Floppy/HardDisk) which is applicable to this medium.

6.143.1.8 hostDrive (read-only)

```
boolean IMedium::hostDrive
```

True if this corresponds to a drive on the host.

6.143.1.9 size (read-only)

```
long long IMedium::size
```

Physical size of the storage unit used to hold medium data (in bytes).

> **Note:** For media whose state is Inaccessible, the value of this property is the last known size. For NotCreated media, the returned value is zero.

6.143.1.10 format (read-only)

`wstring IMedium::format`

Storage format of this medium.
The value of this attribute is a string that specifies a backend used to store medium data. The storage format is defined when you create a new medium or automatically detected when you open an existing medium, and cannot be changed later.
The list of all storage formats supported by this VirtualBox installation can be obtained using ISystemProperties::mediumFormats[].

6.143.1.11 mediumFormat (read-only)

`IMediumFormat IMedium::mediumFormat`

Storage medium format object corresponding to this medium.
The value of this attribute is a reference to the medium format object that specifies the backend properties used to store medium data. The storage format is defined when you create a new medium or automatically detected when you open an existing medium, and cannot be changed later.

> **Note:** `null` is returned if there is no associated medium format object. This can e.g. happen for medium objects representing host drives and other special medium objects.

6.143.1.12 type (read/write)

`MediumType IMedium::type`

Type (role) of this medium.
The following constraints apply when changing the value of this attribute:

- If a medium is attached to a virtual machine (either in the current state or in one of the snapshots), its type cannot be changed.
- As long as the medium has children, its type cannot be set to Writethrough.
- The type of all differencing media is Normal and cannot be changed.

The type of a newly created or opened medium is set to Normal, except for DVD and floppy media, which have a type of Writethrough.

6.143.1.13 allowedTypes (read-only)

`MediumType IMedium::allowedTypes[]`

Returns which medium types can selected for this medium.

6.143.1.14 parent (read-only)

`IMedium IMedium::parent`

Parent of this medium (the medium this medium is directly based on).
Only differencing media have parents. For base (non-differencing) media, `null` is returned.

6.143.1.15 children (read-only)

`IMedium IMedium::children[]`

Children of this medium (all differencing media directly based on this medium). A `null` array is returned if this medium does not have any children.

6.143.1.16 base (read-only)

`IMedium IMedium::base`

Base medium of this medium.

If this is a differencing medium, its base medium is the medium the given medium branch starts from. For all other types of media, this property returns the medium object itself (i.e. the same object this property is read on).

6.143.1.17 readOnly (read-only)

`boolean IMedium::readOnly`

Returns `true` if this medium is read-only and `false` otherwise.

A medium is considered to be read-only when its contents cannot be modified without breaking the integrity of other parties that depend on this medium such as its child media or snapshots of virtual machines where this medium is attached to these machines. If there are no children and no such snapshots then there is no dependency and the medium is not read-only.

The value of this attribute can be used to determine the kind of the attachment that will take place when attaching this medium to a virtual machine. If the value is `false` then the medium will be attached directly. If the value is `true` then the medium will be attached indirectly by creating a new differencing child medium for that. See the interface description for more information.

Note that all Immutable media are always read-only while all Writethrough media are always not.

> **Note:** The read-only condition represented by this attribute is related to the medium type and usage, not to the current medium state and not to the read-only state of the storage unit.

6.143.1.18 logicalSize (read-only)

`long long IMedium::logicalSize`

Logical size of this medium (in bytes), as reported to the guest OS running inside the virtual machine this medium is attached to. The logical size is defined when the medium is created and cannot be changed later.

> **Note:** For media whose state is state is Inaccessible, the value of this property is the last known logical size. For NotCreated media, the returned value is zero.

6.143.1.19 autoReset (read/write)

```
boolean IMedium::autoReset
```

Whether this differencing medium will be automatically reset each time a virtual machine it is attached to is powered up. This attribute is automatically set to `true` for the last differencing image of an "immutable" medium (see MediumType).

See reset() for more information about resetting differencing media.

> **Note:** Reading this property on a base (non-differencing) medium will always `false`. Changing the value of this property in this case is not supported.

6.143.1.20 lastAccessError (read-only)

```
wstring IMedium::lastAccessError
```

Text message that represents the result of the last accessibility check performed by refreshState().

An empty string is returned if the last accessibility check was successful or has not yet been called. As a result, if state is "Inaccessible" and this attribute is empty, then refreshState() has yet to be called; this is the default value of media after VirtualBox initialization. A non-empty string indicates a failure and should normally describe a reason of the failure (for example, a file read error).

6.143.1.21 machineIds (read-only)

```
uuid IMedium::machineIds[]
```

Array of UUIDs of all machines this medium is attached to.

A `null` array is returned if this medium is not attached to any machine or to any machine's snapshot.

> **Note:** The returned array will include a machine even if this medium is not attached to that machine in the current state but attached to it in one of the machine's snapshots. See getSnapshotIds() for details.

6.143.2 changeEncryption

```
IProgress IMedium::changeEncryption(
            [in] wstring currentPassword,
            [in] wstring cipher,
            [in] wstring newPassword,
            [in] wstring newPasswordId)
```

currentPassword The current password the medium is protected with. Use an empty string to indicate that the medium isn't encrypted.

cipher The cipher to use for encryption. An empty string indicates no encryption for the result.

newPassword The new password the medium should be protected with. An empty password and password ID will result in the medium being encrypted with the current password.

newPasswordId The ID of the new password when unlocking the medium.

Starts encryption of this medium. This means that the stored data in the medium is encrypted.
This medium will be placed to LockedWrite state.
Please note that the results can be either returned straight away, or later as the result of the background operation via the object returned via the `progress` parameter.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED:` Encryption is not supported for this medium because it is attached to more than one VM or has children.

6.143.3 checkEncryptionPassword

```
void IMedium::checkEncryptionPassword(
           [in] wstring password)
```

password The password to check.

Checks whether the supplied password is correct for the medium.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED:` Encryption is not configured for this medium.
- `VBOX_E_PASSWORD_INCORRECT:` The given password is incorrect.

6.143.4 cloneTo

```
IProgress IMedium::cloneTo(
           [in] IMedium target,
           [in] MediumVariant variant[],
           [in] IMedium parent)
```

target Target medium.

variant Exact image variant which should be created (as a combination of MediumVariant flags).

parent Parent of the cloned medium.

Starts creating a clone of this medium in the format and at the location defined by the `target` argument.
The target medium must be either in NotCreated state (i.e. must not have an existing storage unit) or in Created state (i.e. created and not locked, and big enough to hold the data or else the copy will be partial). Upon successful completion, the cloned medium will contain exactly the same sector data as the medium being cloned, except that in the first case a new UUID for the clone will be randomly generated, and in the second case the UUID will remain unchanged.
The `parent` argument defines which medium will be the parent of the clone. Passing a `null` reference indicates that the clone will be a base image, i.e. completely independent. It is possible to specify an arbitrary medium for this parameter, including the parent of the medium which is being cloned. Even cloning to a child of the source medium is possible. Note that when cloning to an existing image, the `parent` argument is ignored.
After the returned progress object reports that the operation is successfully complete, the target medium gets remembered by this VirtualBox installation and may be attached to virtual machines.

> **Note:** This medium will be placed to LockedRead state for the duration of this operation.

If this method fails, the following error codes may be reported:

- `E_NOTIMPL:` The specified cloning variant is not supported at the moment.

6.143.5 cloneToBase

```
IProgress IMedium::cloneToBase(
            [in] IMedium target,
            [in] MediumVariant variant[])
```

target Target medium.

variant MediumVariant flags).

Starts creating a clone of this medium in the format and at the location defined by the `target` argument.

The target medium must be either in NotCreated state (i.e. must not have an existing storage unit) or in Created state (i.e. created and not locked, and big enough to hold the data or else the copy will be partial). Upon successful completion, the cloned medium will contain exactly the same sector data as the medium being cloned, except that in the first case a new UUID for the clone will be randomly generated, and in the second case the UUID will remain unchanged.

The `parent` argument defines which medium will be the parent of the clone. In this case the clone will be a base image, i.e. completely independent. It is possible to specify an arbitrary medium for this parameter, including the parent of the medium which is being cloned. Even cloning to a child of the source medium is possible. Note that when cloning to an existing image, the `parent` argument is ignored.

After the returned progress object reports that the operation is successfully complete, the target medium gets remembered by this VirtualBox installation and may be attached to virtual machines.

> **Note:** This medium will be placed to LockedRead state for the duration of this operation.

If this method fails, the following error codes may be reported:

- `E_NOTIMPL:` The specified cloning variant is not supported at the moment.

6.143.6 close

```
void IMedium::close()
```

Closes this medium.

The medium must not be attached to any known virtual machine and must not have any known child media, otherwise the operation will fail.

When the medium is successfully closed, it is removed from the list of registered media, but its storage unit is not deleted. In particular, this means that this medium can later be opened again using the IVirtualBox::openMedium() call.

Note that after this method successfully returns, the given medium object becomes uninitialized. This means that any attempt to call any of its methods or attributes will fail with the `"Object not ready" (E_ACCESSDENIED)` error.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_OBJECT_STATE:` Invalid medium state (other than not created, created or inaccessible).
- `VBOX_E_OBJECT_IN_USE:` Medium attached to virtual machine.
- `VBOX_E_FILE_ERROR:` Settings file not accessible.
- `VBOX_E_XML_ERROR:` Could not parse the settings file.

6.143.7 compact

```
IProgress IMedium::compact()
```

Starts compacting of this medium. This means that the medium is transformed into a possibly more compact storage representation. This potentially creates temporary images, which can require a substantial amount of additional disk space.

This medium will be placed to LockedWrite state and all its parent media (if any) will be placed to LockedRead state for the duration of this operation.

Please note that the results can be either returned straight away, or later as the result of the background operation via the object returned via the `progress` parameter.

If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: Medium format does not support compacting (but potentially needs it).

6.143.8 createBaseStorage

```
IProgress IMedium::createBaseStorage(
            [in] long long logicalSize,
            [in] MediumVariant variant[])
```

logicalSize Maximum logical size of the medium in bytes.

variant Exact image variant which should be created (as a combination of MediumVariant flags).

Starts creating a hard disk storage unit (fixed/dynamic, according to the variant flags) in the background. The previous storage unit created for this object, if any, must first be deleted using deleteStorage(), otherwise the operation will fail.

Before the operation starts, the medium is placed in Creating state. If the create operation fails, the medium will be placed back in NotCreated state.

After the returned progress object reports that the operation has successfully completed, the medium state will be set to Created, the medium will be remembered by this VirtualBox installation and may be attached to virtual machines.

If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: The variant of storage creation operation is not supported. See IMediumFormat::capabilities[].

6.143.9 createDiffStorage

```
IProgress IMedium::createDiffStorage(
            [in] IMedium target,
            [in] MediumVariant variant[])
```

target Target medium.

variant Exact image variant which should be created (as a combination of MediumVariant flags).

Starts creating an empty differencing storage unit based on this medium in the format and at the location defined by the `target` argument.

The target medium must be in NotCreated state (i.e. must not have an existing storage unit). Upon successful completion, this operation will set the type of the target medium to Normal and create a storage unit necessary to represent the differencing medium data in the given format (according to the storage format of the target object).

After the returned progress object reports that the operation is successfully complete, the target medium gets remembered by this VirtualBox installation and may be attached to virtual machines.

Note: The medium will be set to LockedRead state for the duration of this operation.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_IN_USE: Medium not in NotCreated state.

6.143.10 deleteStorage

```
IProgress IMedium::deleteStorage()
```

Starts deleting the storage unit of this medium.

The medium must not be attached to any known virtual machine and must not have any known child media, otherwise the operation will fail. It will also fail if there is no storage unit to delete or if deletion is already in progress, or if the medium is being in use (locked for read or for write) or inaccessible. Therefore, the only valid state for this operation to succeed is Created.

Before the operation starts, the medium is placed in Deleting state and gets removed from the list of remembered hard disks (media registry). If the delete operation fails, the medium will be remembered again and placed back to Created state.

After the returned progress object reports that the operation is complete, the medium state will be set to NotCreated and you will be able to use one of the storage creation methods to create it again.

See also: close()

Note: If the deletion operation fails, it is not guaranteed that the storage unit still exists. You may check the state value to answer this question.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_IN_USE: Medium is attached to a virtual machine.
- VBOX_E_NOT_SUPPORTED: Storage deletion is not allowed because neither of storage creation operations are supported. See IMediumFormat::capabilities[].

6.143.11 getEncryptionSettings

```
wstring IMedium::getEncryptionSettings(
            [out] wstring cipher)
```

cipher The cipher used for encryption.

Returns the encryption settings for this medium.

If this method fails, the following error codes may be reported:

- VBOX_E_NOT_SUPPORTED: Encryption is not configured for this medium.

6.143.12 getProperties

```
wstring[] IMedium::getProperties(
            [in] wstring names,
            [out] wstring returnNames[])
```

names Names of properties to get.

returnNames Names of returned properties.

Returns values for a group of properties in one call.

The names of the properties to get are specified using the `names` argument which is a list of comma-separated property names or an empty string if all properties are to be returned.

> **Note:** Currently the value of this argument is ignored and the method always returns all existing properties.

The list of all properties supported by the given medium format can be obtained with IMediumFormat::describeProperties().

The method returns two arrays, the array of property names corresponding to the `names` argument and the current values of these properties. Both arrays have the same number of elements with each element at the given index in the first array corresponds to an element at the same index in the second array.

For properties that do not have assigned values, an empty string is returned at the appropriate index in the `returnValues` array.

6.143.13 getProperty

```
wstring IMedium::getProperty(
            [in] wstring name)
```

name Name of the property to get.

Returns the value of the custom medium property with the given name.

The list of all properties supported by the given medium format can be obtained with IMediumFormat::describeProperties().

> **Note:** If this method returns an empty string in `value`, the requested property is supported but currently not assigned any value.

If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND`: Requested property does not exist (not supported by the format).
- `E_INVALIDARG`: `name` is `null` or empty.

6.143.14 getSnapshotIds

```
uuid[] IMedium::getSnapshotIds(
            [in] uuid machineId)
```

machineId UUID of the machine to query.

Returns an array of UUIDs of all snapshots of the given machine where this medium is attached to.

If the medium is attached to the machine in the current state, then the first element in the array will always be the ID of the queried machine (i.e. the value equal to the `machineId` argument), followed by snapshot IDs (if any).

If the medium is not attached to the machine in the current state, then the array will contain only snapshot IDs.

The returned array may be `null` if this medium is not attached to the given machine at all, neither in the current state nor in one of the snapshots.

6.143.15 lockRead

`IToken IMedium::lockRead()`

Locks this medium for reading.

A read lock is shared: many clients can simultaneously lock the same medium for reading unless it is already locked for writing (see lockWrite()) in which case an error is returned.

When the medium is locked for reading, it cannot be modified from within VirtualBox. This means that any method that changes the properties of this medium or contents of the storage unit will return an error (unless explicitly stated otherwise). That includes an attempt to start a virtual machine that wants to write to the medium.

When the virtual machine is started up, it locks for reading all media it uses in read-only mode. If some medium cannot be locked for reading, the startup procedure will fail. A medium is typically locked for reading while it is used by a running virtual machine but has a depending differencing image that receives the actual write operations. This way one base medium can have multiple child differencing images which can be written to simultaneously. Read-only media such as DVD and floppy images are also locked for reading only (so they can be in use by multiple machines simultaneously).

A medium is also locked for reading when it is the source of a write operation such as cloneTo() or mergeTo().

The medium locked for reading must be unlocked by abandoning the returned token object, see IToken. Calls to lockRead() can be nested and the lock is actually released when all callers have abandoned the token.

This method sets the medium state (see state) to “LockedRead” on success. The medium’s previous state must be one of “Created”, “Inaccessible” or “LockedRead”.

Locking an inaccessible medium is not an error; this method performs a logical lock that prevents modifications of this medium through the VirtualBox API, not a physical file-system lock of the underlying storage unit.

This method returns the current state of the medium *before* the operation.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_OBJECT_STATE`: Invalid medium state (e.g. not created, locked, inaccessible, creating, deleting).

6.143.16 lockWrite

`IToken IMedium::lockWrite()`

Locks this medium for writing.

A write lock, as opposed to lockRead(), is exclusive: there may be only one client holding a write lock, and there may be no read locks while the write lock is held. As a result, read-locking fails if a write lock is held, and write-locking fails if either a read or another write lock is held.

When a medium is locked for writing, it cannot be modified from within VirtualBox, and it is not guaranteed that the values of its properties are up-to-date. Any method that changes the properties of this medium or contents of the storage unit will return an error (unless explicitly stated otherwise).

When a virtual machine is started up, it locks for writing all media it uses to write data to. If any medium could not be locked for writing, the startup procedure will fail. If a medium has differencing images, then while the machine is running, only the last (“leaf”) differencing image is locked for writing, whereas its parents are locked for reading only.

A medium is also locked for writing when it is the target of a write operation such as cloneTo() or mergeTo().

The medium locked for writing must be unlocked by abandoning the returned token object, see IToken. Write locks *cannot* be nested.

This method sets the medium state (see state) to “LockedWrite” on success. The medium’s previous state must be either “Created” or “Inaccessible”.

Locking an inaccessible medium is not an error; this method performs a logical lock that prevents modifications of this medium through the VirtualBox API, not a physical file-system lock of the underlying storage unit.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_OBJECT_STATE`: Invalid medium state (e.g. not created, locked, inaccessible, creating, deleting).

6.143.17 mergeTo

```
IProgress IMedium::mergeTo(
            [in] IMedium target)
```

target Target medium.

Starts merging the contents of this medium and all intermediate differencing media in the chain to the given target medium.

The target medium must be either a descendant of this medium or its ancestor (otherwise this method will immediately return a failure). It follows that there are two logical directions of the merge operation: from ancestor to descendant (*forward merge*) and from descendant to ancestor (*backward merge*). Let us consider the following medium chain:

```
Base <- Diff_1 <- Diff_2
```

Here, calling this method on the `Base` medium object with `Diff_2` as an argument will be a forward merge; calling it on `Diff_2` with `Base` as an argument will be a backward merge. Note that in both cases the contents of the resulting medium will be the same, the only difference is the medium object that takes the result of the merge operation. In case of the forward merge in the above example, the result will be written to `Diff_2`; in case of the backward merge, the result will be written to `Base`. In other words, the result of the operation is always stored in the target medium.

Upon successful operation completion, the storage units of all media in the chain between this (source) medium and the target medium, including the source medium itself, will be automatically deleted and the relevant medium objects (including this medium) will become uninitialized. This means that any attempt to call any of their methods or attributes will fail with the `"Object not ready" (E_ACCESSDENIED)` error. Applied to the above example, the forward merge of `Base` to `Diff_2` will delete and uninitialize both `Base` and `Diff_1` media. Note that `Diff_2` in this case will become a base medium itself since it will no longer be based on any other medium.

Considering the above, all of the following conditions must be met in order for the merge operation to succeed:

- Neither this (source) medium nor any intermediate differencing medium in the chain between it and the target medium is attached to any virtual machine.
- Neither the source medium nor the target medium is an Immutable medium.
- The part of the medium tree from the source medium to the target medium is a linear chain, i.e. all medium in this chain have exactly one child which is the next medium in this chain. The only exception from this rule is the target medium in the forward merge operation; it is allowed to have any number of child media because the merge operation will not change its logical contents (as it is seen by the guest OS or by children).
- None of the involved media are in LockedRead or LockedWrite state.

Note: This (source) medium and all intermediates will be placed to Deleting state and the target medium will be placed to LockedWrite state and for the duration of this operation.

6.143.18 moveTo

```
IProgress IMedium::moveTo(
          [in] wstring location)
```

location New location.

Changes the location of this medium. Some medium types may support changing the storage unit location by simply changing the value of the associated property. In this case the operation is performed immediately, and `progress` is returning a `null` reference. Otherwise on success there is a progress object returned, which signals progress and completion of the operation. This distinction is necessary because for some formats the operation is very fast, while for others it can be very slow (moving the image file by copying all data), and in the former case it'd be a waste of resources to create a progress object which will immediately signal completion.

When setting a location for a medium which corresponds to a/several regular file(s) in the host's file system, the given file name may be either relative to the VirtualBox home folder or absolute. Note that if the given location specification does not contain the file extension part then a proper default extension will be automatically appended by the implementation depending on the medium type.

If this method fails, the following error codes may be reported:

- `E_NOTIMPL`: The operation is not implemented yet.
- `VBOX_E_NOT_SUPPORTED`: Medium format does not support changing the location.

6.143.19 openForIO

```
IMediumIO IMedium::openForIO(
          [in] boolean writable,
          [in] wstring password)
```

writable Set this to open the medium for both reading and writing. When not set the medium is opened readonly.

password Password for accessing an encrypted medium. Must be empty if not encrypted.

Open the medium for I/O.

6.143.20 refreshState

```
MediumState IMedium::refreshState()
```

If the current medium state (see MediumState) is one of "Created", "Inaccessible" or "LockedRead", then this performs an accessibility check on the medium and sets the value of the state attribute accordingly; that value is also returned for convenience.

For all other state values, this does not perform a refresh but returns the state only.

The refresh, if performed, may take a long time (several seconds or even minutes, depending on the storage unit location and format) because it performs an accessibility check of the storage unit. This check may cause a significant delay if the storage unit of the given medium is, for example, a file located on a network share which is not currently accessible due to connectivity

problems. In that case, the call will not return until a timeout interval defined by the host OS for this operation expires. For this reason, it is recommended to never read this attribute on the main UI thread to avoid making the UI unresponsive.

If the last known state of the medium is "Created" and the accessibility check fails, then the state would be set to "Inaccessible", and lastAccessError may be used to get more details about the failure. If the state of the medium is "LockedRead", then it remains the same, and a non-empty value of lastAccessError will indicate a failed accessibility check in this case.

Note that not all medium states are applicable to all medium types.

6.143.21 reset

```
IProgress IMedium::reset()
```

Starts erasing the contents of this differencing medium.

This operation will reset the differencing medium to its initial state when it does not contain any sector data and any read operation is redirected to its parent medium. This automatically gets called during VM power-up for every medium whose autoReset attribute is `true`.

The medium will be write-locked for the duration of this operation (see lockWrite()).

If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED:` This is not a differencing medium.
- `VBOX_E_INVALID_OBJECT_STATE:` Medium is not in Created or Inaccessible state.

6.143.22 resize

```
IProgress IMedium::resize(
            [in] long long logicalSize)
```

logicalSize New nominal capacity of the medium in bytes.

Starts resizing this medium. This means that the nominal size of the medium is set to the new value. Both increasing and decreasing the size is possible, and there are no safety checks, since VirtualBox does not make any assumptions about the medium contents.

Resizing usually needs additional disk space, and possibly also some temporary disk space. Note that resize does not create a full temporary copy of the medium, so the additional disk space requirement is usually much lower than using the clone operation.

This medium will be placed to LockedWrite state for the duration of this operation.

Please note that the results can be either returned straight away, or later as the result of the background operation via the object returned via the `progress` parameter.

If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED:` Medium format does not support resizing.

6.143.23 resizeAndCloneTo

```
IProgress IMedium::resizeAndCloneTo(
            [in] IMedium target,
            [in] long long logicalSize,
            [in] MediumVariant variant[],
            [in] IMedium parent)
```

target Target medium.

logicalSize New nominal capacity of the medium in bytes.

variant Exact image variant which should be created (as a combination of MediumVariant flags).

parent Parent of the cloned medium.

This is a helper function that combines the functionality of cloneTo() and resize(). The target medium will take the contents of the calling medium.

If this method fails, the following error codes may be reported:

- `E_NOTIMPL:` The specified cloning variant is not supported at the moment.

6.143.24 setIds

```
void IMedium::setIds(
            [in] boolean setImageId,
            [in] uuid imageId,
            [in] boolean setParentId,
            [in] uuid parentId)
```

setImageId Select whether a new image UUID is set or not.

imageId New UUID for the image. If an empty string is passed, then a new UUID is automatically created, provided that `setImageId` is `true`. Specifying a zero UUID is not allowed.

setParentId Select whether a new parent UUID is set or not.

parentId New parent UUID for the image. If an empty string is passed, then a new UUID is automatically created, provided `setParentId` is `true`. A zero UUID is valid.

Changes the UUID and parent UUID for a hard disk medium.

6.143.25 setProperties

```
void IMedium::setProperties(
            [in] wstring names[],
            [in] wstring values[])
```

names Names of properties to set.

values Values of properties to set.

Sets values for a group of properties in one call.

The names of the properties to set are passed in the `names` array along with the new values for them in the `values` array. Both arrays have the same number of elements with each element at the given index in the first array corresponding to an element at the same index in the second array.

If there is at least one property name in `names` that is not valid, the method will fail before changing the values of any other properties from the `names` array.

Using this method over setProperty() is preferred if you need to set several properties at once since it is more efficient.

The list of all properties supported by the given medium format can be obtained with IMediumFormat::describeProperties().

Setting the property value to `null` or an empty string is equivalent to deleting the existing value. A default value (if it is defined for this property) will be used by the format backend in this case.

6.143.26 setProperty

```
void IMedium::setProperty(
            [in] wstring name,
            [in] wstring value)
```

name Name of the property to set.

value Property value to set.

Sets the value of the custom medium property with the given name.

The list of all properties supported by the given medium format can be obtained with IMediumFormat::describeProperties().

> **Note:** Setting the property value to `null` or an empty string is equivalent to deleting the existing value. A default value (if it is defined for this property) will be used by the format backend in this case.

If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND`: Requested property does not exist (not supported by the format).
- `E_INVALIDARG`: `name` is `null` or empty.

6.144 IMediumAttachment

> **Note:** With the web service, this interface is mapped to a structure. Attributes that return this interface will not return an object, but a complete structure containing the attributes listed below as structure members.

The IMediumAttachment interface links storage media to virtual machines. For each medium (IMedium) which has been attached to a storage controller (IStorageController) of a machine (IMachine) via the IMachine::attachDevice() method, one instance of IMediumAttachment is added to the machine's IMachine::mediumAttachments[] array attribute.

Each medium attachment specifies the storage controller as well as a port and device number and the IMedium instance representing a virtual hard disk or floppy or DVD image.

For removable media (DVDs or floppies), there are two additional options. For one, the IMedium instance can be `null` to represent an empty drive with no media inserted (see IMachine::mountMedium()); secondly, the medium can be one of the pseudo-media for host drives listed in IHost::DVDDrives[] or IHost::floppyDrives[].

Attaching Hard Disks

Hard disks are attached to virtual machines using the IMachine::attachDevice() method and detached using the IMachine::detachDevice() method. Depending on a medium's type (see IMedium::type), hard disks are attached either *directly* or *indirectly*.

When a hard disk is being attached directly, it is associated with the virtual machine and used for hard disk operations when the machine is running. When a hard disk is being attached indirectly, a new differencing hard disk linked to it is implicitly created and this differencing hard disk is associated with the machine and used for hard disk operations. This also means that if IMachine::attachDevice() performs a direct attachment then the same hard disk will be returned in response to the subsequent IMachine::getMedium() call; however if an indirect attachment is performed then IMachine::getMedium() will return the implicitly created differencing hard disk, not the original one passed to IMachine::attachDevice(). In detail:

- **Normal base** hard disks that do not have children (i.e. differencing hard disks linked to them) and that are not already attached to virtual machines in snapshots are attached **directly**. Otherwise, they are attached **indirectly** because having dependent children or being part of the snapshot makes it impossible to modify hard disk contents without breaking the integrity of the dependent party. The IMedium::readOnly attribute allows to quickly determine the kind of the attachment for the given hard disk. Note that if a normal base hard disk is to be indirectly attached to a virtual machine with snapshots then a special procedure called *smart attachment* is performed (see below).

- **Normal differencing** hard disks are like normal base hard disks: they are attached **directly** if they do not have children and are not attached to virtual machines in snapshots, and **indirectly** otherwise. Note that the smart attachment procedure is never performed for differencing hard disks.

- **Immutable** hard disks are always attached **indirectly** because they are designed to be non-writable. If an immutable hard disk is attached to a virtual machine with snapshots then a special procedure called smart attachment is performed (see below).

- **Writethrough** hard disks are always attached **directly**, also as designed. This also means that writethrough hard disks cannot have other hard disks linked to them at all.

- **Shareable** hard disks are always attached **directly**, also as designed. This also means that shareable hard disks cannot have other hard disks linked to them at all. They behave almost like writethrough hard disks, except that shareable hard disks can be attached to several virtual machines which are running, allowing concurrent accesses. You need special cluster software running in the virtual machines to make use of such disks.

Note that the same hard disk, regardless of its type, may be attached to more than one virtual machine at a time. In this case, the machine that is started first gains exclusive access to the hard disk and attempts to start other machines having this hard disk attached will fail until the first machine is powered down.

Detaching hard disks is performed in a *deferred* fashion. This means that the given hard disk remains associated with the given machine after a successful IMachine::detachDevice() call until IMachine::saveSettings() is called to save all changes to machine settings to disk. This deferring is necessary to guarantee that the hard disk configuration may be restored at any time by a call to IMachine::discardSettings() before the settings are saved (committed).

Note that if IMachine::discardSettings() is called after indirectly attaching some hard disks to the machine but before a call to IMachine::saveSettings() is made, it will implicitly delete all differencing hard disks implicitly created by IMachine::attachDevice() for these indirect attachments. Such implicitly created hard disks will also be immediately deleted when detached explicitly using the IMachine::detachDevice() call if it is made before IMachine::saveSettings(). This implicit deletion is safe because newly created differencing hard disks do not contain any user data.

However, keep in mind that detaching differencing hard disks that were implicitly created by IMachine::attachDevice() before the last IMachine::saveSettings() call will **not** implicitly delete them as they may already contain some data (for example, as a result of virtual machine execution). If these hard disks are no more necessary, the caller can always delete them explicitly using IMedium::deleteStorage() after they are actually de-associated from this machine by the IMachine::saveSettings() call.

Smart Attachment

When normal base or immutable hard disks are indirectly attached to a virtual machine then some additional steps are performed to make sure the virtual machine will have the most recent "view" of the hard disk being attached. These steps include walking through the machine's snapshots starting from the current one and going through ancestors up to the first snapshot. Hard disks attached to the virtual machine in all of the encountered snapshots are checked

whether they are descendants of the given normal base or immutable hard disk. The first found child (which is the differencing hard disk) will be used instead of the normal base or immutable hard disk as a parent for creating a new differencing hard disk that will be actually attached to the machine. And only if no descendants are found or if the virtual machine does not have any snapshots then the normal base or immutable hard disk will be used itself as a parent for this differencing hard disk.

It is easier to explain what smart attachment does using the following example:

```
BEFORE attaching B.vdi:       AFTER attaching B.vdi:

Snapshot 1 (B.vdi)            Snapshot 1 (B.vdi)
 Snapshot 2 (D1->B.vdi)        Snapshot 2 (D1->B.vdi)
  Snapshot 3 (D2->D1.vdi)       Snapshot 3 (D2->D1.vdi)
   Snapshot 4 (none)             Snapshot 4 (none)
    CurState   (none)             CurState   (D3->D2.vdi)

                              NOT
                                 ...
                                  CurState   (D3->B.vdi)
```

The first column is the virtual machine configuration before the base hard disk `B.vdi` is attached, the second column shows the machine after this hard disk is attached. Constructs like `D1->B.vdi` and similar mean that the hard disk that is actually attached to the machine is a differencing hard disk, `D1.vdi`, which is linked to (based on) another hard disk, `B.vdi`.

As we can see from the example, the hard disk `B.vdi` was detached from the machine before taking Snapshot 4. Later, after Snapshot 4 was taken, the user decides to attach `B.vdi` again. `B.vdi` has dependent child hard disks (`D1.vdi`, `D2.vdi`), therefore it cannot be attached directly and needs an indirect attachment (i.e. implicit creation of a new differencing hard disk). Due to the smart attachment procedure, the new differencing hard disk (`D3.vdi`) will be based on `D2.vdi`, not on `B.vdi` itself, since `D2.vdi` is the most recent view of `B.vdi` existing for this snapshot branch of the given virtual machine.

Note that if there is more than one descendant hard disk of the given base hard disk found in a snapshot, and there is an exact device, channel and bus match, then this exact match will be used. Otherwise, the youngest descendant will be picked up.

There is one more important aspect of the smart attachment procedure which is not related to snapshots at all. Before walking through the snapshots as described above, the backup copy of the current list of hard disk attachment is searched for descendants. This backup copy is created when the hard disk configuration is changed for the first time after the last IMachine::saveSettings() call and used by IMachine::discardSettings() to undo the recent hard disk changes. When such a descendant is found in this backup copy, it will be simply re-attached back, without creating a new differencing hard disk for it. This optimization is necessary to make it possible to re-attach the base or immutable hard disk to a different bus, channel or device slot without losing the contents of the differencing hard disk actually attached to the machine in place of it.

6.144.1 Attributes

6.144.1.1 machine (read-only)

```
IMachine IMediumAttachment::machine
```

Machine object for this medium attachment.

6.144.1.2 medium (read-only)

```
IMedium IMediumAttachment::medium
```

Medium object associated with this attachment; it can be `null` for removable devices.

6.144.1.3 controller (read-only)

```
wstring IMediumAttachment::controller
```

Name of the storage controller of this attachment; this refers to one of the controllers in IMachine::storageControllers[] by name.

6.144.1.4 port (read-only)

```
long IMediumAttachment::port
```

Port number of this attachment. See IMachine::attachDevice() for the meaning of this value for the different controller types.

6.144.1.5 device (read-only)

```
long IMediumAttachment::device
```

Device slot number of this attachment. See IMachine::attachDevice() for the meaning of this value for the different controller types.

6.144.1.6 type (read-only)

```
DeviceType IMediumAttachment::type
```

Device type of this attachment.

6.144.1.7 passthrough (read-only)

```
boolean IMediumAttachment::passthrough
```

Pass I/O requests through to a device on the host.

6.144.1.8 temporaryEject (read-only)

```
boolean IMediumAttachment::temporaryEject
```

Whether guest-triggered eject results in unmounting the medium.

6.144.1.9 isEjected (read-only)

```
boolean IMediumAttachment::isEjected
```

Signals that the removable medium has been ejected. This is not necessarily equivalent to having a `null` medium association.

6.144.1.10 nonRotational (read-only)

```
boolean IMediumAttachment::nonRotational
```

Whether the associated medium is non-rotational.

6.144.1.11 discard (read-only)

```
boolean IMediumAttachment::discard
```

Whether the associated medium supports discarding unused blocks.

6.144.1.12 hotPluggable (read-only)

`boolean IMediumAttachment::hotPluggable`

Whether this attachment is hot pluggable or not.

6.144.1.13 bandwidthGroup (read-only)

`IBandwidthGroup IMediumAttachment::bandwidthGroup`

The bandwidth group this medium attachment is assigned to.

6.145 IMediumChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a medium attachment changes.

6.145.1 Attributes

6.145.1.1 mediumAttachment (read-only)

`IMediumAttachment IMediumChangedEvent::mediumAttachment`

Medium attachment that is subject to change.

6.146 IMediumConfigChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

The configuration of the given medium was changed (location, properties, child/parent or anything else).

6.146.1 Attributes

6.146.1.1 medium (read-only)

`IMedium IMediumConfigChangedEvent::medium`

ID of the medium this event relates to.

6.147 IMediumFormat

The IMediumFormat interface represents a medium format.

Each medium format has an associated backend which is used to handle media stored in this format. This interface provides information about the properties of the associated backend.

Each medium format is identified by a string represented by the id attribute. This string is used in calls like IVirtualBox::createMedium() to specify the desired format.

The list of all supported medium formats can be obtained using ISystemProperties::mediumFormats[].

See also: IMedium

6.147.1 Attributes

6.147.1.1 id (read-only)

```
wstring IMediumFormat::id
```

Identifier of this format.
The format identifier is a non-null non-empty ASCII string. Note that this string is case-insensitive. This means that, for example, all of the following strings:

```
"VDI"
"vdi"
"VdI"
```

refer to the same medium format.
This string is used in methods of other interfaces where it is necessary to specify a medium format, such as IVirtualBox::createMedium().

6.147.1.2 name (read-only)

```
wstring IMediumFormat::name
```

Human readable description of this format.
Mainly for use in file open dialogs.

6.147.1.3 capabilities (read-only)

```
MediumFormatCapabilities IMediumFormat::capabilities[]
```

Capabilities of the format as an array of the flags.
For the meaning of individual capability flags see MediumFormatCapabilities.

6.147.2 describeFileExtensions

```
void IMediumFormat::describeFileExtensions(
           [out] wstring extensions[],
           [out] DeviceType types[])
```

extensions The array of supported extensions.

types The array which indicates the device type for every given extension.

Returns two arrays describing the supported file extensions.
The first array contains the supported extensions and the seconds one the type each extension supports. Both have the same size.
Note that some backends do not work on files, so this array may be empty.
See also: capabilities[]

6.147.3 describeProperties

```
void IMediumFormat::describeProperties(
           [out] wstring names[],
           [out] wstring descriptions[],
           [out] DataType types[],
           [out] unsigned long flags[],
           [out] wstring defaults[])
```

names Array of property names.

descriptions Array of property descriptions.

types Array of property types.

flags Array of property flags.

defaults Array of default property values.

Returns several arrays describing the properties supported by this format.

An element with the given index in each array describes one property. Thus, the number of elements in each returned array is the same and corresponds to the number of supported properties.

The returned arrays are filled in only if the Properties flag is set. All arguments must be non-`null`.

See also: DataType, DataFlags

6.148 IMediumIO

The IMediumIO interface is used to access and modify the content of a medium. It is returned by IMedium::openForIO().

6.148.1 Attributes

6.148.1.1 medium (read-only)

```
IMedium IMediumIO::medium
```

The open medium.

6.148.1.2 writable (read-only)

```
boolean IMediumIO::writable
```

Whether the medium can be written to. (It can always be read from.)

6.148.1.3 explorer (read-only)

```
IVFSExplorer IMediumIO::explorer
```

Returns the virtual file system explorer for the medium.

This will attempt to recognize the format of the medium content and present it as a virtual directory structure to the API user.

A FAT floppy image will be represented will a single root subdir 'fat12' that gives access to the file system content.

A ISO-9660 image will have one subdir in the root for each format present in the image, so the API user can select which data view to access (iso9660, rockridge, joliet, udf, hfs, ...).

A partitioned harddisk image will have subdirs for each partition. The filesystem content of each partition can be accessed thru the subdirs if we have a file system interpreter for it. There will also be raw files for each subdirectory, to provide a simple way of accessing raw partition data from an API client.

Please note that the explorer may show inconsistent information if the API user modifies the raw image content after it was opened.

6.148.2 close

```
void IMediumIO::close()
```

Explicitly close the medium I/O rather than waiting for garbage collection and the destructor.
This will wait for any pending reads and writes to complete and then close down the I/O access without regard for open explorer instances or anything like that.

6.148.3 convertToStream

```
IProgress IMediumIO::convertToStream(
            [in] wstring format,
            [in] MediumVariant variant[],
            [in] unsigned long bufferSize,
            [out] IDataStream stream)
```

format Identifier of the storage format to use for output.

variant The partition table format.

bufferSize Requested buffer size (in bytes) for efficient conversion. Sizes which are too small or too large are silently truncated to suitable values. Tens to hundreds of Megabytes are a good choice.

stream Data stream object for reading the target image.

Converts the currently opened image into a stream of the specified image type/variant. It is sufficient to open the image in read-only mode. Only few types and variants are supported due to the inherent restrictions of the output style.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: The requested format/variant combination cannot handle stream output.
- `VBOX_E_FILE_ERROR`: An error occurred during the conversion.

6.148.4 formatFAT

```
void IMediumIO::formatFAT(
            [in] boolean quick)
```

quick Quick format it when set.

Formats the medium as FAT. Generally only useful for floppy images as no partition table will be created.

6.148.5 initializePartitionTable

```
void IMediumIO::initializePartitionTable(
            [in] PartitionTableType format,
            [in] boolean wholeDiskInOneEntry)
```

format The partition table format.

wholeDiskInOneEntry When `true` a partition table entry for the whole disk is created. Otherwise the partition table is empty.

Writes an empty partition table to the disk.

6.148.6 read

```
octet[] IMediumIO::read(
            [in] long long offset,
            [in] unsigned long size)
```

offset The byte offset into the medium to start reading at.

size How many bytes to try read.

Read data from the medium.

6.148.7 write

```
unsigned long IMediumIO::write(
            [in] long long offset,
            [in] octet data[])
```

offset The byte offset into the medium to start reading at.

data Array of data to write.

Write data to the medium.

6.149 IMediumRegisteredEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

The given medium was registered or unregistered within this VirtualBox installation.

6.149.1 Attributes

6.149.1.1 mediumId (read-only)

```
uuid IMediumRegisteredEvent::mediumId
```

ID of the medium this event relates to.

6.149.1.2 mediumType (read-only)

```
DeviceType IMediumRegisteredEvent::mediumType
```

Type of the medium this event relates to.

6.149.1.3 registered (read-only)

```
boolean IMediumRegisteredEvent::registered
```

If `true`, the medium was registered, otherwise it was unregistered.

6.150 IMouse

The IMouse interface represents the virtual machine's mouse. Used in IConsole::mouse.
Through this interface, the virtual machine's virtual mouse can be controlled.

6.150.1 Attributes

6.150.1.1 absoluteSupported (read-only)

`boolean IMouse::absoluteSupported`

Whether the guest OS supports absolute mouse pointer positioning or not.

> **Note:** You can use the IMouseCapabilityChangedEvent event to be instantly informed about changes of this attribute during virtual machine execution.

See also: putMouseEventAbsolute()

6.150.1.2 relativeSupported (read-only)

`boolean IMouse::relativeSupported`

Whether the guest OS supports relative mouse pointer positioning or not.

> **Note:** You can use the IMouseCapabilityChangedEvent event to be instantly informed about changes of this attribute during virtual machine execution.

See also: putMouseEvent()

6.150.1.3 touchScreenSupported (read-only)

`boolean IMouse::touchScreenSupported`

Whether the guest OS has enabled the multi-touch reporting device, touchscreen variant.

> **Note:** You can use the IMouseCapabilityChangedEvent event to be instantly informed about changes of this attribute during virtual machine execution.

See also: putMouseEvent()

6.150.1.4 touchPadSupported (read-only)

`boolean IMouse::touchPadSupported`

Whether the guest OS has enabled the multi-touch reporting device, touchpad variant.

> **Note:** You can use the IMouseCapabilityChangedEvent event to be instantly informed about changes of this attribute during virtual machine execution.

See also: putMouseEvent()

6.150.1.5 needsHostCursor (read-only)

`boolean IMouse::needsHostCursor`

Whether the guest OS can currently switch to drawing it's own mouse cursor on demand.

> **Note:** You can use the IMouseCapabilityChangedEvent event to be instantly informed about changes of this attribute during virtual machine execution.

See also: putMouseEvent()

6.150.1.6 pointerShape (read-only)

`IMousePointerShape IMouse::pointerShape`

The current mouse pointer used by the guest.

6.150.1.7 eventSource (read-only)

`IEventSource IMouse::eventSource`

Event source for mouse events.

6.150.2 putEventMultiTouch

```
void IMouse::putEventMultiTouch(
            [in] long count,
            [in] long long contacts[],
            [in] boolean isTouchScreen,
            [in] unsigned long scanTime)
```

count Number of contacts in the event.

contacts Each array element contains packed information about one contact. Bits 0..15: X coordinate in pixels or normalized X position. Bits 16..31: Y coordinate in pixels or normalized Y position. Bits 32..39: contact identifier. Bit 40: "in contact" flag, which indicates that there is a contact with the touch surface. Bit 41: "in range" flag, the contact is close enough to the touch surface. All other bits are reserved for future use and must be set to 0.

isTouchScreen Distinguishes between touchscreen and touchpad events.

scanTime Timestamp of the event in milliseconds. Only relative time between events is important.

Sends a multi-touch pointer event. For touchscreen events the coordinates are expressed in pixels and start from `[1,1]` which corresponds to the top left corner of the virtual display, for touchpad events the coordinates are normalized to the range 0..0xffff.

Note: The guest may not understand or may choose to ignore this event.

See also: touchScreenSupported and touchPadSupported
If this method fails, the following error codes may be reported:

- `E_ACCESSDENIED`: Console not powered up.
- `VBOX_E_IPRT_ERROR`: Could not send event to virtual device.

6.150.3 putEventMultiTouchString

```
void IMouse::putEventMultiTouchString(
            [in] long count,
            [in] wstring contacts,
            [in] boolean isTouchScreen,
            [in] unsigned long scanTime)
```

count See also: putEventMultiTouch()

contacts Contains information about all contacts: "id1,x1,y1,inContact1,inRange1;...;idN,xN,yN,inContactN,inRang
For example for two contacts: "0,10,20,1,1;1,30,40,1,1"

isTouchScreen Distinguishes between touchscreen and touchpad events.

scanTime See also: putEventMultiTouch()

See also: putEventMultiTouch()

6.150.4 putMouseEvent

```
void IMouse::putMouseEvent(
            [in] long dx,
            [in] long dy,
            [in] long dz,
            [in] long dw,
            [in] long buttonState)
```

dx Amount of pixels the mouse should move to the right. Negative values move the mouse to the left.

dy Amount of pixels the mouse should move downwards. Negative values move the mouse upwards.

dz Amount of mouse wheel moves. Positive values describe clockwise wheel rotations, negative values describe counterclockwise rotations.

dw Amount of horizontal mouse wheel moves. Positive values describe a movement to the left, negative values describe a movement to the right.

buttonState The current state of mouse buttons. Every bit represents a mouse button as follows: Bit 0 (0x01)left mouse buttonBit 1 (0x02)right mouse buttonBit 2 (0x04)middle mouse button A value of 1 means the corresponding button is pressed. otherwise it is released.

Initiates a mouse event using relative pointer movements along x and y axis.
If this method fails, the following error codes may be reported:

- E_ACCESSDENIED: Console not powered up.
- VBOX_E_IPRT_ERROR: Could not send mouse event to virtual mouse.

6.150.5 putMouseEventAbsolute

```
void IMouse::putMouseEventAbsolute(
            [in] long x,
            [in] long y,
            [in] long dz,
            [in] long dw,
            [in] long buttonState)
```

x X coordinate of the pointer in pixels, starting from 1.

y Y coordinate of the pointer in pixels, starting from 1.

dz Amount of mouse wheel moves. Positive values describe clockwise wheel rotations, negative values describe counterclockwise rotations.

dw Amount of horizontal mouse wheel moves. Positive values describe a movement to the left, negative values describe a movement to the right.

buttonState The current state of mouse buttons. Every bit represents a mouse button as follows: Bit 0 (0x01)left mouse buttonBit 1 (0x02)right mouse buttonBit 2 (0x04)middle mouse button A value of 1 means the corresponding button is pressed. otherwise it is released.

Positions the mouse pointer using absolute x and y coordinates. These coordinates are expressed in pixels and start from `[1,1]` which corresponds to the top left corner of the virtual display. The values `[-1,-1]` and `[0x7fffffff,0x7fffffff]` have special meanings as respectively "no data" (to signal that the host wishes to report absolute pointer data in future) and "out of range" (the host pointer is outside of all guest windows).

> **Note:** This method will have effect only if absolute mouse positioning is supported by the guest OS.

See also: absoluteSupported
If this method fails, the following error codes may be reported:

- `E_ACCESSDENIED:` Console not powered up.
- `VBOX_E_IPRT_ERROR:` Could not send mouse event to virtual mouse.

6.151 IMouseCapabilityChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the mouse capabilities reported by the guest have changed. The new capabilities are passed.

6.151.1 Attributes

6.151.1.1 supportsAbsolute (read-only)

```
boolean IMouseCapabilityChangedEvent::supportsAbsolute
```

Supports absolute coordinates.

6.151.1.2 supportsRelative (read-only)

```
boolean IMouseCapabilityChangedEvent::supportsRelative
```

Supports relative coordinates.

6.151.1.3 supportsTouchScreen (read-only)

```
boolean IMouseCapabilityChangedEvent::supportsTouchScreen
```

Supports multi-touch events, touchscreen variant.

6.151.1.4 supportsTouchPad (read-only)

```
boolean IMouseCapabilityChangedEvent::supportsTouchPad
```

Supports multi-touch events, touchpad variant.

6.151.1.5 needsHostCursor (read-only)

```
boolean IMouseCapabilityChangedEvent::needsHostCursor
```

If host cursor is needed.

6.152 IMousePointerShape

The guest mouse pointer description.

6.152.1 Attributes

6.152.1.1 visible (read-only)

```
boolean IMousePointerShape::visible
```

Flag whether the pointer is visible.

6.152.1.2 alpha (read-only)

```
boolean IMousePointerShape::alpha
```

Flag whether the pointer has an alpha channel.

6.152.1.3 hotX (read-only)

```
unsigned long IMousePointerShape::hotX
```

The pointer hot spot X coordinate.

6.152.1.4 hotY (read-only)

```
unsigned long IMousePointerShape::hotY
```

The pointer hot spot Y coordinate.

6.152.1.5 width (read-only)

```
unsigned long IMousePointerShape::width
```

Width of the pointer shape in pixels.

6.152.1.6 height (read-only)

```
unsigned long IMousePointerShape::height
```

Height of the pointer shape in pixels.

6.152.1.7 shape (read-only)

```
octet IMousePointerShape::shape[]
```

Shape bitmaps.

The `shape` buffer contains a 1bpp (bits per pixel) AND mask followed by a 32bpp XOR (color) mask.

For pointers without alpha channel the XOR mask pixels are 32 bit values: (lsb)BGR0(msb). For pointers with alpha channel the XOR mask consists of (lsb)BGRA(msb) 32 bit values.

An AND mask is provided for pointers with alpha channel, so if the client does not support alpha, the pointer could be displayed as a normal color pointer.

The AND mask is a 1bpp bitmap with byte aligned scanlines. The size of the AND mask therefore is `cbAnd = (width + 7) / 8 * height`. The padding bits at the end of each scanline are undefined.

The XOR mask follows the AND mask on the next 4-byte aligned offset: `uint8_t *pu8Xor = pu8And + (cbAnd + 3) & ~3`. Bytes in the gap between the AND and the XOR mask are undefined. The XOR mask scanlines have no gap between them and the size of the XOR mask is: `cbXor = width * 4 * height`.

> **Note:** If `shape` size is 0, then the shape is not known or did not change. This can happen if only the pointer visibility is changed.

6.153 IMousePointerShapeChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the guest mouse pointer shape has changed. The new shape data is given.

6.153.1 Attributes

6.153.1.1 visible (read-only)

```
boolean IMousePointerShapeChangedEvent::visible
```

Flag whether the pointer is visible.

6.153.1.2 alpha (read-only)

```
boolean IMousePointerShapeChangedEvent::alpha
```

Flag whether the pointer has an alpha channel.

6.153.1.3 xhot (read-only)

```
unsigned long IMousePointerShapeChangedEvent::xhot
```

The pointer hot spot X coordinate.

6.153.1.4 yhot (read-only)

```
unsigned long IMousePointerShapeChangedEvent::yhot
```

The pointer hot spot Y coordinate.

6.153.1.5 width (read-only)

```
unsigned long IMousePointerShapeChangedEvent::width
```

Width of the pointer shape in pixels.

6.153.1.6 height (read-only)

```
unsigned long IMousePointerShapeChangedEvent::height
```

Height of the pointer shape in pixels.

6.153.1.7 shape (read-only)

`octet IMousePointerShapeChangedEvent::shape[]`

Shape buffer arrays.

The `shape` buffer contains a 1-bpp (bits per pixel) AND mask followed by a 32-bpp XOR (color) mask.

For pointers without alpha channel the XOR mask pixels are 32-bit values: (lsb)BGR0(msb). For pointers with alpha channel the XOR mask consists of (lsb)BGRA(msb) 32-bit values.

An AND mask is used for pointers with alpha channel, so if the callback does not support alpha, the pointer could be displayed as a normal color pointer.

The AND mask is a 1-bpp bitmap with byte aligned scanlines. The size of the AND mask therefore is `cbAnd = (width + 7) / 8 * height`. The padding bits at the end of each scanline are undefined.

The XOR mask follows the AND mask on the next 4-byte aligned offset: `uint8_t *pXor = pAnd + (cbAnd + 3) & ~3`. Bytes in the gap between the AND and the XOR mask are undefined. The XOR mask scanlines have no gap between them and the size of the XOR mask is: `cXor = width * 4 * height`.

> **Note:** If `shape` is 0, only the pointer visibility is changed.

6.154 INATEngine

Interface for managing a NAT engine which is used with a virtual machine. This allows for changing NAT behavior such as port-forwarding rules. This interface is used in the INetworkAdapter::NATEngine attribute.

6.154.1 Attributes

6.154.1.1 network (read/write)

`wstring INATEngine::network`

The network attribute of the NAT engine (the same value is used with built-in DHCP server to fill corresponding fields of DHCP leases).

6.154.1.2 hostIP (read/write)

`wstring INATEngine::hostIP`

IP of host interface to bind all opened sockets to.

> **Note:** Changing this does not change binding of port forwarding.

6.154.1.3 TFTPPrefix (read/write)

`wstring INATEngine::TFTPPrefix`

TFTP prefix attribute which is used with the built-in DHCP server to fill the corresponding fields of DHCP leases.

> **Note:** Ignored if TFTP is disabled.

6.154.1.4 TFTPBootFile (read/write)

```
wstring INATEngine::TFTPBootFile
```

TFTP boot file attribute which is used with the built-in DHCP server to fill the corresponding fields of DHCP leases.

> **Note:** Ignored if TFTP is disabled.

6.154.1.5 TFTPNextServer (read/write)

```
wstring INATEngine::TFTPNextServer
```

TFTP server attribute which is used with the built-in DHCP server to fill the corresponding fields of DHCP leases.

> **Note:** The preferred form is IPv4 addresses.

> **Note:** Ignored if TFTP is disabled.

6.154.1.6 aliasMode (read/write)

```
unsigned long INATEngine::aliasMode
```

6.154.1.7 DNSPassDomain (read/write)

```
boolean INATEngine::DNSPassDomain
```

Whether the DHCP server should pass the DNS domain used by the host.

6.154.1.8 DNSProxy (read/write)

```
boolean INATEngine::DNSProxy
```

Whether the DHCP server (and the DNS traffic by NAT) should pass the address of the DNS proxy and process traffic using DNS servers registered on the host.

6.154.1.9 DNSUseHostResolver (read/write)

```
boolean INATEngine::DNSUseHostResolver
```

Whether the DHCP server (and the DNS traffic by NAT) should pass the address of the DNS proxy and process traffic using the host resolver mechanism.

6.154.1.10 redirects (read-only)

```
wstring INATEngine::redirects[]
```

Array of NAT port-forwarding rules in string representation, in the following format: “name,protocol id,host ip,host port,guest ip,guest port”.

6.154.1.11 localhostReachable (read/write)

```
boolean INATEngine::localhostReachable
```

Whether traffic from the guest directed to 10.0.2.2 will reach the host's loopback interface, i.e. localhost or 127.0.0.1.

6.154.1.12 forwardBroadcast (read/write)

```
boolean INATEngine::forwardBroadcast
```

Forwards broadcast packets sent by the guest to the host machine as such. IP address will show to host as 255.255.255.255.

6.154.1.13 enableTFTP (read/write)

```
boolean INATEngine::enableTFTP
```

Enables TFTP functionality, and related configuration options.

6.154.1.14 natMRU (read/write)

```
unsigned long INATEngine::natMRU
```

MRU (maximum receive unit) of the NAT engine in bytes.

6.154.1.15 IPv6Enabled (read/write)

```
boolean INATEngine::IPv6Enabled
```

This attribute define whether gateway will support IPv6 or not.

6.154.1.16 IPv6Prefix (read/write)

```
wstring INATEngine::IPv6Prefix
```

This a CIDR IPv6 defining prefix for link-local addresses autoconfiguration within network. Note: ignored if attribute IPv6Enabled is false.

6.154.2 addRedirect

```
void INATEngine::addRedirect(
            [in] wstring name,
            [in] NATProtocol proto,
            [in] wstring hostIP,
            [in] unsigned short hostPort,
            [in] wstring guestIP,
            [in] unsigned short guestPort)
```

name The name of the rule. An empty name is acceptable, in which case the NAT engine auto-generates one using the other parameters.

proto Protocol handled with the rule.

hostIP IP of the host interface to which the rule should apply. An empty ip address is acceptable, in which case the NAT engine binds the handling socket to any interface.

hostPort The port number to listen on.

guestIP The IP address of the guest which the NAT engine will forward matching packets to. An empty IP address is acceptable, in which case the NAT engine will forward packets to the first DHCP lease (x.x.x.15).

guestPort The port number to forward.

Adds a new NAT port-forwarding rule.

6.154.3 getNetworkSettings

```
void INATEngine::getNetworkSettings(
            [out] unsigned long mtu,
            [out] unsigned long sockSnd,
            [out] unsigned long sockRcv,
            [out] unsigned long TcpWndSnd,
            [out] unsigned long TcpWndRcv)
```

mtu

sockSnd

sockRcv

TcpWndSnd

TcpWndRcv

Returns network configuration of NAT engine. See setNetworkSettings() for parameter descriptions.

6.154.4 removeRedirect

```
void INATEngine::removeRedirect(
            [in] wstring name)
```

name The name of the rule to delete.

Removes a port-forwarding rule that was previously registered.

6.154.5 setNetworkSettings

```
void INATEngine::setNetworkSettings(
            [in] unsigned long mtu,
            [in] unsigned long sockSnd,
            [in] unsigned long sockRcv,
            [in] unsigned long TcpWndSnd,
            [in] unsigned long TcpWndRcv)
```

mtu MTU (maximum transmission unit) of the NAT engine in bytes.

sockSnd Capacity of the socket send buffer in bytes when creating a new socket.

sockRcv Capacity of the socket receive buffer in bytes when creating a new socket.

TcpWndSnd Initial size of the NAT engine's sending TCP window in bytes when establishing a new TCP connection.

TcpWndRcv Initial size of the NAT engine's receiving TCP window in bytes when establishing a new TCP connection.

Sets network configuration of the NAT engine.

6.155 INATNetwork

6.155.1 Attributes

6.155.1.1 networkName (read/write)

```
wstring INATNetwork::networkName
```

TBD: the idea, technically we can start any number of the NAT networks, but we should expect that at some point we will get collisions because of port-forwanding rules. so perhaps we should support only single instance of NAT network.

6.155.1.2 enabled (read/write)

```
boolean INATNetwork::enabled
```

6.155.1.3 network (read/write)

```
wstring INATNetwork::network
```

This is CIDR IPv4 string. Specifying it user defines IPv4 addresses of gateway (low address + 1) and DHCP server (= low address + 2). Note: If there are defined IPv4 port-forward rules update of network will be ignored (because new assignment could break existing rules).

6.155.1.4 gateway (read-only)

```
wstring INATNetwork::gateway
```

This attribute is read-only. It's recalculated on changing network attribute (low address of network + 1).

6.155.1.5 IPv6Enabled (read/write)

```
boolean INATNetwork::IPv6Enabled
```

This attribute define whether gateway will support IPv6 or not.

6.155.1.6 IPv6Prefix (read/write)

```
wstring INATNetwork::IPv6Prefix
```

This a CIDR IPv6 defining prefix for link-local addresses autoconfiguration within network. Note: ignored if attribute IPv6Enabled is false.

6.155.1.7 advertiseDefaultIPv6RouteEnabled (read/write)

```
boolean INATNetwork::advertiseDefaultIPv6RouteEnabled
```

6.155.1.8 needDhcpServer (read/write)

```
boolean INATNetwork::needDhcpServer
```

6.155.1.9 eventSource (read-only)

```
IEventSource INATNetwork::eventSource
```

6.155.1.10 portForwardRules4 (read-only)

```
wstring INATNetwork::portForwardRules4[]
```

Array of NAT port-forwarding rules in string representation, in the following format: “name:protocolid:[host ip]:host port:[guest ip]:guest port”.

6.155.1.11 localMappings (read-only)

```
wstring INATNetwork::localMappings[]
```

Array of mappings (address,offset),e.g. (“127.0.1.1=4”) maps 127.0.1.1 to networkid + 4.

6.155.1.12 localhostReachable (read/write)

```
boolean INATNetwork::localhostReachable
```

Whether traffic from the guest directed to 10.0.2.2 will reach the host’s loopback interface, i.e. localhost or 127.0.0.1.

6.155.1.13 forwardBroadcast (read/write)

```
boolean INATNetwork::forwardBroadcast
```

Forwards broadcast packets sent by the guest to the host machine as such. IP address will show to host as 255.255.255.255.

6.155.1.14 natMTU (read/write)

```
unsigned long INATNetwork::natMTU
```

MTU (maximum transmission unit) of the NAT engine in bytes.

6.155.1.15 natMRU (read/write)

```
unsigned long INATNetwork::natMRU
```

MRU (maximum receive unit) of the NAT engine in bytes.

6.155.1.16 loopbackIp6 (read/write)

```
long INATNetwork::loopbackIp6
```

Offset in ipv6 network from network id for address mapped into loopback6 interface of the host.

6.155.1.17 portForwardRules6 (read-only)

```
wstring INATNetwork::portForwardRules6[]
```

Array of NAT port-forwarding rules in string representation, in the following format: “name:protocolid:[host ip]:host port:[guest ip]:guest port”.

6.155.2 addLocalMapping

```
void INATNetwork::addLocalMapping(
            [in] wstring hostid,
            [in] long offset)
```

hostid

offset

6.155.3 addPortForwardRule

```
void INATNetwork::addPortForwardRule(
            [in] boolean isIpv6,
            [in] wstring ruleName,
            [in] NATProtocol proto,
            [in] wstring hostIP,
            [in] unsigned short hostPort,
            [in] wstring guestIP,
            [in] unsigned short guestPort)
```

isIpv6

ruleName

proto Protocol handled with the rule.

hostIP IP of the host interface to which the rule should apply. An empty IP address is acceptable, in which case the NAT engine binds the handling socket to any interface.

hostPort The port number to listen on.

guestIP The IP address of the guest which the NAT engine will forward matching packets to. An empty IP address is not acceptable.

guestPort The port number to forward.

6.155.4 removePortForwardRule

```
void INATNetwork::removePortForwardRule(
            [in] boolean isIpv6,
            [in] wstring ruleName)
```

isIpv6

ruleName

6.155.5 start

```
void INATNetwork::start()
```

6.155.6 stop

```
void INATNetwork::stop()
```

6.156 INATNetworkAlterEvent (INATNetworkChangedEvent)

Note: This interface extends INATNetworkChangedEvent and therefore supports all its methods and attributes as well.

6.156.1 Attributes

6.156.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean INATNetworkAlterEvent::midlDoesNotLikeEmptyInterfaces
```

6.157 INATNetworkChangedEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

6.157.1 Attributes

6.157.1.1 networkName (read-only)

```
wstring INATNetworkChangedEvent::networkName
```

6.158 INATNetworkCreationDeletionEvent (INATNetworkAlterEvent)

Note: This interface extends INATNetworkAlterEvent and therefore supports all its methods and attributes as well.

6.158.1 Attributes

6.158.1.1 creationEvent (read-only)

```
boolean INATNetworkCreationDeletionEvent::creationEvent
```

6.159 INATNetworkPortForwardEvent (INATNetworkAlterEvent)

Note: This interface extends INATNetworkAlterEvent and therefore supports all its methods and attributes as well.

6.159.1 Attributes

6.159.1.1 create (read-only)

```
boolean INATNetworkPortForwardEvent::create
```

6.159.1.2 ipv6 (read-only)

```
boolean INATNetworkPortForwardEvent::ipv6
```

6.159.1.3 name (read-only)

```
wstring INATNetworkPortForwardEvent::name
```

6.159.1.4 proto (read-only)

```
NATProtocol INATNetworkPortForwardEvent::proto
```

6.159.1.5 hostIp (read-only)

```
wstring INATNetworkPortForwardEvent::hostIp
```

6.159.1.6 hostPort (read-only)

```
long INATNetworkPortForwardEvent::hostPort
```

6.159.1.7 guestIp (read-only)

```
wstring INATNetworkPortForwardEvent::guestIp
```

6.159.1.8 guestPort (read-only)

```
long INATNetworkPortForwardEvent::guestPort
```

6.160 INATNetworkSettingEvent (INATNetworkAlterEvent)

> **Note:** This interface extends INATNetworkAlterEvent and therefore supports all its methods and attributes as well.

6.160.1 Attributes

6.160.1.1 enabled (read-only)

```
boolean INATNetworkSettingEvent::enabled
```

6.160.1.2 network (read-only)

```
wstring INATNetworkSettingEvent::network
```

6.160.1.3 gateway (read-only)

```
wstring INATNetworkSettingEvent::gateway
```

6.160.1.4 advertiseDefaultIPv6RouteEnabled (read-only)

```
boolean INATNetworkSettingEvent::advertiseDefaultIPv6RouteEnabled
```

6.160.1.5 needDhcpServer (read-only)

```
boolean INATNetworkSettingEvent::needDhcpServer
```

6.161 INATNetworkStartStopEvent (INATNetworkChangedEvent)

> **Note:** This interface extends INATNetworkChangedEvent and therefore supports all its methods and attributes as well.

6.161.1 Attributes

6.161.1.1 startEvent (read-only)

```
boolean INATNetworkStartStopEvent::startEvent
```

IsStartEvent is true when NAT network is started and false on stopping.

6.162 INATRedirectEvent (IMachineEvent)

> **Note:** This interface extends IMachineEvent and therefore supports all its methods and attributes as well.

Notification when NAT redirect rule added or removed.

6.162.1 Attributes

6.162.1.1 slot (read-only)

```
unsigned long INATRedirectEvent::slot
```

Adapter which NAT attached to.

6.162.1.2 remove (read-only)

```
boolean INATRedirectEvent::remove
```

Whether rule remove or add.

6.162.1.3 name (read-only)

```
wstring INATRedirectEvent::name
```

Name of the rule.

6.162.1.4 proto (read-only)

```
NATProtocol INATRedirectEvent::proto
```

Protocol (TCP or UDP) of the redirect rule.

6.162.1.5 hostIP (read-only)

```
wstring INATRedirectEvent::hostIP
```

Host IP address to bind socket on.

6.162.1.6 hostPort (read-only)

```
long INATRedirectEvent::hostPort
```

Host port to bind socket on.

6.162.1.7 guestIP (read-only)

```
wstring INATRedirectEvent::guestIP
```

Guest IP address to redirect to.

6.162.1.8 guestPort (read-only)

```
long INATRedirectEvent::guestPort
```

Guest port to redirect to.

6.163 INetworkAdapter

Represents a virtual network adapter that is attached to a virtual machine. Each virtual machine has a fixed number of network adapter slots with one instance of this attached to each of them. Call IMachine::getNetworkAdapter() to get the network adapter that is attached to a given slot in a given machine.

Each network adapter can be in one of five attachment modes, which are represented by the NetworkAttachmentType enumeration; see the attachmentType attribute.

6.163.1 Attributes

6.163.1.1 adapterType (read/write)

```
NetworkAdapterType INetworkAdapter::adapterType
```

Type of the virtual network adapter. Depending on this value, VirtualBox will provide a different virtual network hardware to the guest.

6.163.1.2 slot (read-only)

```
unsigned long INetworkAdapter::slot
```

Slot number this adapter is plugged into. Corresponds to the value you pass to IMachine::getNetworkAdapter() to obtain this instance.

6.163.1.3 enabled (read/write)

```
boolean INetworkAdapter::enabled
```

Flag whether the network adapter is present in the guest system. If disabled, the virtual guest hardware will not contain this network adapter. Can only be changed when the VM is not running.

6.163.1.4 MACAddress (read/write)

```
wstring INetworkAdapter::MACAddress
```

Ethernet MAC address of the adapter, 12 hexadecimal characters. When setting it to `null` or an empty string for an enabled adapter, VirtualBox will generate a unique MAC address. Disabled adapters can have an empty MAC address.

6.163.1.5 attachmentType (read/write)

`NetworkAttachmentType INetworkAdapter::attachmentType`

Sets/Gets network attachment type of this network adapter.

6.163.1.6 bridgedInterface (read/write)

`wstring INetworkAdapter::bridgedInterface`

Name of the network interface the VM should be bridged to.

6.163.1.7 hostOnlyInterface (read/write)

`wstring INetworkAdapter::hostOnlyInterface`

Name of the host only network interface the VM is attached to.

6.163.1.8 hostOnlyNetwork (read/write)

`wstring INetworkAdapter::hostOnlyNetwork`

Name of the host only network the VM is attached to.

6.163.1.9 internalNetwork (read/write)

`wstring INetworkAdapter::internalNetwork`

Name of the internal network the VM is attached to.

6.163.1.10 NATNetwork (read/write)

`wstring INetworkAdapter::NATNetwork`

Name of the NAT network the VM is attached to.

6.163.1.11 genericDriver (read/write)

`wstring INetworkAdapter::genericDriver`

Name of the driver to use for the “Generic” network attachment type.

6.163.1.12 cloudNetwork (read/write)

`wstring INetworkAdapter::cloudNetwork`

Name of the cloud network the VM is attached to.

6.163.1.13 cableConnected (read/write)

`boolean INetworkAdapter::cableConnected`

Flag whether the adapter reports the cable as connected or not. It can be used to report offline situations to a VM.

6.163.1.14 lineSpeed (read/write)

```
unsigned long INetworkAdapter::lineSpeed
```

Line speed reported by custom drivers, in units of 1 kbps.

6.163.1.15 promiscModePolicy (read/write)

```
NetworkAdapterPromiscModePolicy INetworkAdapter::promiscModePolicy
```

The promiscuous mode policy of the network adapter when attached to an internal network, host only network or a bridge.

6.163.1.16 traceEnabled (read/write)

```
boolean INetworkAdapter::traceEnabled
```

Flag whether network traffic from/to the network card should be traced. Can only be toggled when the VM is turned off.

6.163.1.17 traceFile (read/write)

```
wstring INetworkAdapter::traceFile
```

Filename where a network trace will be stored. If not set, VBox-pid.pcap will be used.

6.163.1.18 NATEngine (read-only)

```
INATEngine INetworkAdapter::NATEngine
```

Points to the NAT engine which handles the network address translation for this interface. This is active only when the interface actually uses NAT.

6.163.1.19 bootPriority (read/write)

```
unsigned long INetworkAdapter::bootPriority
```

Network boot priority of the adapter. Priority 1 is highest. If not set, the priority is considered to be at the lowest possible setting.

6.163.1.20 bandwidthGroup (read/write)

```
IBandwidthGroup INetworkAdapter::bandwidthGroup
```

The bandwidth group this network adapter is assigned to.

6.163.2 getProperties

```
wstring[] INetworkAdapter::getProperties(
            [in] wstring names,
            [out] wstring returnNames[])
```

names Names of properties to get.

returnNames Names of returned properties.

Returns values for a group of properties in one call.
The names of the properties to get are specified using the `names` argument which is a list of comma-separated property names or an empty string if all properties are to be returned.

> **Note:** Currently the value of this argument is ignored and the method always returns all existing properties.

The method returns two arrays, the array of property names corresponding to the `names` argument and the current values of these properties. Both arrays have the same number of elements with each element at the given index in the first array corresponds to an element at the same index in the second array.

6.163.3 getProperty

```
wstring INetworkAdapter::getProperty(
            [in] wstring key)
```

key Name of the property to get.

Returns the value of the network attachment property with the given name.
If the requested data `key` does not exist, this function will succeed and return an empty string in the `value` argument.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG: name` is `null` or empty.

6.163.4 setProperty

```
void INetworkAdapter::setProperty(
            [in] wstring key,
            [in] wstring value)
```

key Name of the property to set.

value Property value to set.

Sets the value of the network attachment property with the given name.
Setting the property value to `null` or an empty string is equivalent to deleting the existing value.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG: name` is `null` or empty.

6.164 INetworkAdapterChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a property of one of the virtual network adapters changes. Interested callees should use INetworkAdapter methods and attributes to find out what has changed.

6.164.1 Attributes

6.164.1.1 networkAdapter (read-only)

```
INetworkAdapter INetworkAdapterChangedEvent::networkAdapter
```

Network adapter that is subject to change.

6.165 INvramStore

Provides access to the NVRAM store collecting all permanent states from different sources (UEFI, TPM, etc.).

6.165.1 Attributes

6.165.1.1 nonVolatileStorageFile (read-only)

```
wstring INvramStore::nonVolatileStorageFile
```

The location of the file storing the non-volatile memory content when the VM is powered off. The file does not always exist.

6.165.1.2 uefiVariableStore (read-only)

```
IUefiVariableStore INvramStore::uefiVariableStore
```

Object to manipulate the data in an existing UEFI variable store.

6.165.1.3 keyId (read-only)

```
wstring INvramStore::keyId
```

Key Id of the password used for encrypting the NVRAM file. Internal use only for now.

6.165.1.4 keyStore (read-only)

```
wstring INvramStore::keyStore
```

Key store used for encrypting the NVRAM file. Internal use only for now.

6.165.2 initUefiVariableStore

```
void INvramStore::initUefiVariableStore(
           [in] unsigned long size)
```

size Size in bytes of the UEFI variable store. Must be 0 for now to initialize to the default size.

Initializes the UEFI variable store.

6.166 IPCIAddress

Address on the PCI bus.

6.166.1 Attributes

6.166.1.1 bus (read/write)

```
short IPCIAddress::bus
```

Bus number.

6.166.1.2 device (read/write)

```
short IPCIAddress::device
```

Device number.

6.166.1.3 devFunction (read/write)

```
short IPCIAddress::devFunction
```

Device function number.

6.166.2 asLong

```
long IPCIAddress::asLong()
```

Convert PCI address into long.

6.166.3 fromLong

```
void IPCIAddress::fromLong(
           [in] long number)
```

number

Make PCI address from long.

6.167 IPCIDeviceAttachment

> **Note:** With the web service, this interface is mapped to a structure. Attributes that return this interface will not return an object, but a complete structure containing the attributes listed below as structure members.

Information about PCI attachments.

6.167.1 Attributes

6.167.1.1 name (read-only)

```
wstring IPCIDeviceAttachment::name
```

Device name.

6.167.1.2 isPhysicalDevice (read-only)

```
boolean IPCIDeviceAttachment::isPhysicalDevice
```

If this is physical or virtual device.

6.167.1.3 hostAddress (read-only)

`long IPCIDeviceAttachment::hostAddress`

Address of device on the host, applicable only to host devices.

6.167.1.4 guestAddress (read-only)

`long IPCIDeviceAttachment::guestAddress`

Address of device in the guest.

6.168 IParallelPort

The IParallelPort interface represents the virtual parallel port device.

The virtual parallel port device acts like an ordinary parallel port inside the virtual machine. This device communicates to the real parallel port hardware using the name of the parallel device on the host computer specified in the #path attribute.

Each virtual parallel port device is assigned a base I/O address and an IRQ number that will be reported to the guest operating system and used to operate the given parallel port from within the virtual machine.

See also: IMachine::getParallelPort()

6.168.1 Attributes

6.168.1.1 slot (read-only)

`unsigned long IParallelPort::slot`

Slot number this parallel port is plugged into. Corresponds to the value you pass to IMachine::getParallelPort() to obtain this instance.

6.168.1.2 enabled (read/write)

`boolean IParallelPort::enabled`

Flag whether the parallel port is enabled. If disabled, the parallel port will not be reported to the guest OS.

6.168.1.3 IOBase (read/write)

`unsigned long IParallelPort::IOBase`

Base I/O address of the parallel port. Only for X86-based platforms.

6.168.1.4 IRQ (read/write)

`unsigned long IParallelPort::IRQ`

IRQ number of the parallel port. Only for X86-based platforms.

6.168.1.5 path (read/write)

`wstring IParallelPort::path`

Host parallel device name. If this parallel port is enabled, setting a `null` or an empty string as this attribute's value will result in the parallel port behaving as if not connected to any device.

6.169 IParallelPortChangedEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a property of one of the virtual parallel ports changes. Interested callees should use ISerialPort methods and attributes to find out what has changed.

6.169.1 Attributes

6.169.1.1 parallelPort (read-only)

```
IParallelPort IParallelPortChangedEvent::parallelPort
```

Parallel port that is subject to change.

6.170 IPerformanceCollector

The IPerformanceCollector interface represents a service that collects and stores performance metrics data.

Performance metrics are associated with objects of interfaces like IHost and IMachine. Each object has a distinct set of performance metrics. The set can be obtained with getMetrics().

Metric data is collected at the specified intervals and is retained internally. The interval and the number of retained samples can be set with setupMetrics(). Both metric data and collection settings are not persistent, they are discarded as soon as VBoxSVC process terminates. Moreover, metric settings and data associated with a particular VM only exist while VM is running. They disappear as soon as VM shuts down. It is not possible to set up metrics for machines that are powered off. One needs to start VM first, then set up metric collection parameters.

Metrics are organized hierarchically, with each level separated by a slash (/) character. Generally, the scheme for metric names is like this:

`Category/Metric[/SubMetric][:aggregation]`

"Category/Metric" together form the base metric name. A base metric is the smallest unit for which a sampling interval and the number of retained samples can be set. Only base metrics can be enabled and disabled. All sub-metrics are collected when their base metric is collected. Collected values for any set of sub-metrics can be queried with queryMetricsData().

For example "CPU/Load/User:avg" metric name stands for the "CPU" category, "Load" metric, "User" submetric, "average" aggregate. An aggregate function is computed over all retained data. Valid aggregate functions are:

- avg – average
- min – minimum
- max – maximum

When setting up metric parameters, querying metric data, enabling or disabling metrics wildcards can be used in metric names to specify a subset of metrics. For example, to select all CPU-related metrics use `CPU/*`, all averages can be queried using `*:avg` and so on. To query metric values without aggregates `*:` can be used.

The valid names for base metrics are:

- CPU/Load
- CPU/MHz

- RAM/Usage
- RAM/VMM

The general sequence for collecting and retrieving the metrics is:

- Obtain an instance of IPerformanceCollector with IVirtualBox::performanceCollector
- Allocate and populate an array with references to objects the metrics will be collected for. Use references to IHost and IMachine objects.
- Allocate and populate an array with base metric names the data will be collected for.
- Call setupMetrics(). From now on the metric data will be collected and stored.
- Wait for the data to get collected.
- Allocate and populate an array with references to objects the metric values will be queried for. You can re-use the object array used for setting base metrics.
- Allocate and populate an array with metric names the data will be collected for. Note that metric names differ from base metric names.
- Call queryMetricsData(). The data that have been collected so far are returned. Note that the values are still retained internally and data collection continues.

For an example of usage refer to the following files in VirtualBox SDK:

- Java: `bindings/webservice/java/jax-ws/samples/metrictest.java`
- Python: `bindings/xpcom/python/sample/shellcommon.py`

6.170.1 Attributes

6.170.1.1 metricNames (read-only)

```
wstring IPerformanceCollector::metricNames[]
```

Array of unique names of metrics.

This array represents all metrics supported by the performance collector. Individual objects do not necessarily support all of them. getMetrics() can be used to get the list of supported metrics for a particular object.

6.170.2 disableMetrics

```
IPerformanceMetric[] IPerformanceCollector::disableMetrics(
            [in] wstring metricNames[],
            [in] $unknown objects[])
```

metricNames Metric name filter. Comma-separated list of metrics with wildcard support.

objects Set of objects to disable metrics for.

Turns off collecting specified base metrics. Returns an array of IPerformanceMetric describing the metrics have been affected.

> **Note:** `Null` or empty metric name array means all metrics. `Null` or empty object array means all existing objects. If metric name array contains a single element and object array contains many, the single metric name array element is applied to each object array element to form metric/object pairs.

6.170.3 enableMetrics

```
IPerformanceMetric[] IPerformanceCollector::enableMetrics(
            [in] wstring metricNames[],
            [in] $unknown objects[])
```

metricNames Metric name filter. Comma-separated list of metrics with wildcard support.

objects Set of objects to enable metrics for.

Turns on collecting specified base metrics. Returns an array of IPerformanceMetric describing the metrics have been affected.

> **Note:** `Null` or empty metric name array means all metrics. `Null` or empty object array means all existing objects. If metric name array contains a single element and object array contains many, the single metric name array element is applied to each object array element to form metric/object pairs.

6.170.4 getMetrics

```
IPerformanceMetric[] IPerformanceCollector::getMetrics(
            [in] wstring metricNames[],
            [in] $unknown objects[])
```

metricNames Metric name filter. Currently, only a comma-separated list of metrics is supported.

objects Set of objects to return metric parameters for.

Returns parameters of specified metrics for a set of objects.

> **Note:** `Null` metrics array means all metrics. `Null` object array means all existing objects.

6.170.5 queryMetricsData

```
long[] IPerformanceCollector::queryMetricsData(
            [in] wstring metricNames[],
            [in] $unknown objects[],
            [out] wstring returnMetricNames[],
            [out] $unknown returnObjects[],
            [out] wstring returnUnits[],
            [out] unsigned long returnScales[],
            [out] unsigned long returnSequenceNumbers[],
            [out] unsigned long returnDataIndices[],
            [out] unsigned long returnDataLengths[])
```

metricNames Metric name filter. Comma-separated list of metrics with wildcard support.

objects Set of objects to query metrics for.

returnMetricNames Names of metrics returned in `returnData`.

returnObjects Objects associated with metrics returned in `returnData`.

returnUnits Units of measurement for each returned metric.

returnScales Divisor that should be applied to return values in order to get floating point values. For example: `(double)returnData[returnDataIndices[0]+i] / returnScales[0]` will retrieve the floating point value of i-th sample of the first metric.

returnSequenceNumbers Sequence numbers of the first elements of value sequences of particular metrics returned in `returnData`. For aggregate metrics it is the sequence number of the sample the aggregate started calculation from.

returnDataIndices Indices of the first elements of value sequences of particular metrics returned in `returnData`.

returnDataLengths Lengths of value sequences of particular metrics.

Queries collected metrics data for a set of objects.

The data itself and related metric information are returned in seven parallel and one flattened array of arrays. Elements of `returnMetricNames, returnObjects, returnUnits, returnScales, returnSequenceNumbers, returnDataIndices` and `returnDataLengths` with the same index describe one set of values corresponding to a single metric.

The `returnData` parameter is a flattened array of arrays. Each start and length of a subarray is indicated by `returnDataIndices` and `returnDataLengths`. The first value for metric `metricNames[i]` is at `returnData[returnIndices[i]]`.

> **Note:** `Null` or empty metric name array means all metrics. `Null` or empty object array means all existing objects. If metric name array contains a single element and object array contains many, the single metric name array element is applied to each object array element to form metric/object pairs.

> **Note:** Data collection continues behind the scenes after call to `queryMetricsData`. The return data can be seen as the snapshot of the current state at the time of `queryMetricsData` call. The internally kept metric values are not cleared by the call. This allows querying different subsets of metrics or aggregates with subsequent calls. If periodic querying is needed it is highly suggested to query the values with `interval`*count period to avoid confusion. This way a completely new set of data values will be provided by each query.

6.170.6 setupMetrics

```
IPerformanceMetric[] IPerformanceCollector::setupMetrics(
           [in] wstring metricNames[],
           [in] $unknown objects[],
           [in] unsigned long period,
           [in] unsigned long count)
```

metricNames Metric name filter. Comma-separated list of metrics with wildcard support.

objects Set of objects to setup metric parameters for.

period Time interval in seconds between two consecutive samples of performance data.

count Number of samples to retain in performance data history. Older samples get discarded.

Sets parameters of specified base metrics for a set of objects. Returns an array of IPerformanceMetric describing the metrics have been affected.

Note: `Null` or empty metric name array means all metrics. `Null` or empty object array means all existing objects. If metric name array contains a single element and object array contains many, the single metric name array element is applied to each object array element to form metric/object pairs.

6.171 IPerformanceMetric

The IPerformanceMetric interface represents parameters of the given performance metric.

6.171.1 Attributes

6.171.1.1 metricName (read-only)

```
wstring IPerformanceMetric::metricName
```

Name of the metric.

6.171.1.2 object (read-only)

```
$unknown IPerformanceMetric::object
```

Object this metric belongs to.

6.171.1.3 description (read-only)

```
wstring IPerformanceMetric::description
```

Textual description of the metric.

6.171.1.4 period (read-only)

```
unsigned long IPerformanceMetric::period
```

Time interval between samples, measured in seconds.

6.171.1.5 count (read-only)

```
unsigned long IPerformanceMetric::count
```

Number of recent samples retained by the performance collector for this metric.
When the collected sample count exceeds this number, older samples are discarded.

6.171.1.6 unit (read-only)

```
wstring IPerformanceMetric::unit
```

Unit of measurement.

6.171.1.7 minimumValue (read-only)

```
long IPerformanceMetric::minimumValue
```

Minimum possible value of this metric.

6.171.1.8 maximumValue (read-only)

`long IPerformanceMetric::maximumValue`

Maximum possible value of this metric.

6.172 IPlatform

Platform object for a virtual machine.
Accessed via IMachine::platform.

6.172.1 Attributes

6.172.1.1 architecture (read/write)

`PlatformArchitecture IPlatform::architecture`

Returns the platform architecture.
Changing this means settings for the previously selected architecture will be discarded and replaced by defaults.

> **Note:** Setting the platform architecture currently is not implemented yet.

6.172.1.2 properties (read-only)

`IPlatformProperties IPlatform::properties`

Returns the platform properties.

6.172.1.3 x86 (read-only)

`IPlatformX86 IPlatform::x86`

Platform object for x86-specific platform properties.
Changes to this object will not necessarily be preserved unless architecture is set to x86.

6.172.1.4 ARM (read-only)

`IPlatformARM IPlatform::ARM`

Platform object for ARM-specific platform properties.
Changes to this object will not necessarily be preserved unless architecture is set to ARM.

6.172.1.5 chipsetType (read/write)

`ChipsetType IPlatform::chipsetType`

Chipset type used in this VM.

6.172.1.6 iommuType (read/write)

`IommuType IPlatform::iommuType`

IOMMU type used in this VM.

6.172.1.7 RTCUseUTC (read/write)

`boolean IPlatform::RTCUseUTC`

When set to `true`, the RTC device of the virtual machine will run in UTC time, otherwise in local time. Especially Unix guests prefer the time in UTC.

6.173 IPlatformARM

The ARM-specific platform properties for a virtual machine.
Accessed via IMachine::platform and IPlatform::ARM.

6.173.1 getCPUProperty

```
boolean IPlatformARM::getCPUProperty(
            [in] CPUPropertyTypeARM property)
```

property Property type to query.

Returns the virtual CPU boolean value of the specified property.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: Invalid property.

6.173.2 setCPUProperty

```
void IPlatformARM::setCPUProperty(
            [in] CPUPropertyTypeARM property,
            [in] boolean value)
```

property Property type to query.

value Property value.

Sets the virtual CPU boolean value of the specified property.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: Invalid property.

6.174 IPlatformProperties

Properties of a specific virtualization platform.

6.174.1 Attributes

6.174.1.1 rawModeSupported (read-only)

`boolean IPlatformProperties::rawModeSupported`

Indicates whether VirtualBox was built with raw-mode support.
When this reads as False, the Enabled setting will be ignored and assumed to be True.

> **Note:** For guest platform properties, this attribute is being ignored.

6.174.1.2 exclusiveHwVirt (read/write)

`boolean IPlatformProperties::exclusiveHwVirt`

Exclusive use of hardware virtualization by VirtualBox. When enabled, VirtualBox assumes it can obtain full and exclusive access to the VT-x or AMD-V feature of the host. To share hardware virtualization with other hypervisors, this property must be disabled.

> **Note:** For hosts running on macOS this is being ignored, as the kernel mediates hardware access there.

> **Note:** For guest platform properties, this attribute is being ignored.

6.174.1.3 serialPortCount (read-only)

`unsigned long IPlatformProperties::serialPortCount`

Maximum number of serial ports associated with every IMachine instance.

6.174.1.4 parallelPortCount (read-only)

`unsigned long IPlatformProperties::parallelPortCount`

Maximum number of parallel ports associated with every IMachine instance.

6.174.1.5 maxBootPosition (read-only)

`unsigned long IPlatformProperties::maxBootPosition`

Maximum device position in the boot order. This value corresponds to the total number of devices a machine can boot from, to make it possible to include all possible devices to the boot list. See also: IMachine::setBootOrder()

6.174.1.6 supportedParavirtProviders (read-only)

`ParavirtProvider IPlatformProperties::supportedParavirtProviders[]`

Returns an array of officially supported values for enum ParavirtProvider, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.7 supportedFirmwareTypes (read-only)

`FirmwareType IPlatformProperties::supportedFirmwareTypes[]`

Returns an array of officially supported values for enum FirmwareType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.8 supportedGuestOSTypes (read-only)

`IGuestOSType IPlatformProperties::supportedGuestOSTypes[]`

Returns an array of officially supported values for enum IGuestOSType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.9 supportedGfxControllerTypes (read-only)

`GraphicsControllerType IPlatformProperties::supportedGfxControllerTypes[]`

Returns an array of officially supported values for enum GraphicsControllerType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.10 supportedNetAdpPromiscModePols (read-only)

`NetworkAdapterPromiscModePolicy IPlatformProperties::supportedNetAdpPromiscModePols[]`

Returns an array of officially supported values for enum NetworkAdapterPromiscModePolicy, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.11 supportedNetworkAdapterTypes (read-only)

`NetworkAdapterType IPlatformProperties::supportedNetworkAdapterTypes[]`

Returns an array of officially supported values for enum NetworkAdapterType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.12 supportedUartTypes (read-only)

`UartType IPlatformProperties::supportedUartTypes[]`

Returns an array of officially supported values for enum UartType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.13 supportedUSBControllerTypes (read-only)

`USBControllerType IPlatformProperties::supportedUSBControllerTypes[]`

Returns an array of officially supported values for enum USBControllerType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.14 supportedAudioControllerTypes (read-only)

`AudioControllerType IPlatformProperties::supportedAudioControllerTypes[]`

Returns an array of officially supported values for enum AudioControllerType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.15 supportedBootDevices (read-only)

`DeviceType IPlatformProperties::supportedBootDevices[]`

Returns an array of officially supported boot devices for enum DeviceType, in the sense of what is e.g. worth offering in the VirtualBox GUI.
The returned array also defines the default boot device order for all supported devices.

6.174.1.16 supportedStorageBuses (read-only)

`StorageBus IPlatformProperties::supportedStorageBuses[]`

Returns an array of officially supported values for enum StorageBus, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.17 supportedStorageControllerTypes (read-only)

```
StorageControllerType IPlatformProperties::supportedStorageControllerTypes[]
```

Returns an array of officially supported values for enum StorageControllerType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.18 supportedChipsetTypes (read-only)

```
ChipsetType IPlatformProperties::supportedChipsetTypes[]
```

Returns an array of officially supported values for enum ChipsetType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.19 supportedIommuTypes (read-only)

```
IommuType IPlatformProperties::supportedIommuTypes[]
```

Returns an array of officially supported values for enum IommuType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.1.20 supportedTpmTypes (read-only)

```
TpmType IPlatformProperties::supportedTpmTypes[]
```

Returns an array of officially supported values for enum TpmType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.174.2 getDeviceTypesForStorageBus

```
DeviceType[] IPlatformProperties::getDeviceTypesForStorageBus(
            [in] StorageBus bus)
```

bus The storage bus type to get the value for.

Returns list of all the supported device types (DeviceType) for the given type of storage bus.

6.174.3 getMaxDevicesPerPortForStorageBus

```
unsigned long IPlatformProperties::getMaxDevicesPerPortForStorageBus(
            [in] StorageBus bus)
```

bus The storage bus type to get the value for.

Returns the maximum number of devices which can be attached to a port for the given storage bus.

6.174.4 getMaxInstancesOfStorageBus

```
unsigned long IPlatformProperties::getMaxInstancesOfStorageBus(
            [in] ChipsetType chipset,
            [in] StorageBus bus)
```

chipset The chipset type to get the value for.

bus The storage bus type to get the value for.

Returns the maximum number of storage bus instances which can be configured for each VM. This corresponds to the number of storage controllers one can have. Value may depend on chipset type used.

6.174.5 getMaxInstancesOfUSBControllerType

```
unsigned long IPlatformProperties::getMaxInstancesOfUSBControllerType(
            [in] ChipsetType chipset,
            [in] USBControllerType type)
```

chipset The chipset type to get the value for.

type The USB controller type to get the value for.

Returns the maximum number of USB controller instances which can be configured for each VM. This corresponds to the number of USB controllers one can have. Value may depend on chipset type used.

6.174.6 getMaxNetworkAdapters

```
unsigned long IPlatformProperties::getMaxNetworkAdapters(
            [in] ChipsetType chipset)
```

chipset The chipset type to get the value for.

Maximum total number of network adapters associated with every IMachine instance.

6.174.7 getMaxNetworkAdaptersOfType

```
unsigned long IPlatformProperties::getMaxNetworkAdaptersOfType(
            [in] ChipsetType chipset,
            [in] NetworkAttachmentType type)
```

chipset The chipset type to get the value for.

type Type of attachment.

Maximum number of network adapters of a given attachment type, associated with every IMachine instance.

6.174.8 getMaxPortCountForStorageBus

```
unsigned long IPlatformProperties::getMaxPortCountForStorageBus(
            [in] StorageBus bus)
```

bus The storage bus type to get the value for.

Returns the maximum number of ports the given storage bus supports.

6.174.9 getMinGuestRAM

```
unsigned long IPlatformProperties::getMinGuestRAM(
            [in] FirmwareType firmware)
```

firmware The chipset type to get the value for.

Returns minimum guest system memory size in Megabytes for the given firmware choice.

The firmwares have built-in expectations about how much memory must be present in the system for them to work. These limits are not enforced anywhere, so you can still configure a VM with as little as ISystemProperties::minGuestRAM specifies, it just is not very likely to boot up.

6.174.10 getMinPortCountForStorageBus

```
unsigned long IPlatformProperties::getMinPortCountForStorageBus(
            [in] StorageBus bus)
```

bus The storage bus type to get the value for.

Returns the minimum number of ports the given storage bus supports.

6.174.11 getStorageBusForControllerType

```
StorageBus IPlatformProperties::getStorageBusForControllerType(
            [in] StorageControllerType storageControllerType)
```

storageControllerType The storage controller type to get the value for.

Returns the StorageBus enum value for a given storage controller type.

6.174.12 getStorageControllerHotplugCapable

```
boolean IPlatformProperties::getStorageControllerHotplugCapable(
            [in] StorageControllerType controllerType)
```

controllerType The storage controller to check the setting for.

Returns whether the given storage controller supports hot-plugging devices.

6.174.13 getStorageControllerTypesForBus

```
StorageControllerType[] IPlatformProperties::getStorageControllerTypesForBus(
            [in] StorageBus storageBus)
```

storageBus The storage bus type to get the values for.

Returns the possible StorageControllerType enum values for a given storage bus.

6.174.14 getSupportedGfxFeaturesForType

```
GraphicsFeature[] IPlatformProperties::getSupportedGfxFeaturesForType(
            [in] GraphicsControllerType graphicsControllerType)
```

graphicsControllerType Graphics controller type to return features for.

Returns the supported features of a given graphics controller.

6.174.15 getSupportedVRAMRange

```
unsigned long IPlatformProperties::getSupportedVRAMRange(
            [in] GraphicsControllerType graphicsControllerType,
            [in] boolean accelerate3DEnabled,
            [out] unsigned long minMB,
            [out] unsigned long maxMB)
```

graphicsControllerType Graphics controller type to return VRAM range for.

accelerate3DEnabled Whether 3D acceleration for the graphics controller type is enabled or not. Ignored if the given graphics controller does not support 3D acceleration.

minMB The minimum size (in MB) the graphics controller supports.

maxMB The maximum size (in MB) the graphics controller supports.

Returns the VRAM range of a given graphics controller.

6.175 IPlatformX86

The x86 specific platform properties for a virtual machine.
Accessed via IMachine::platform and IPlatform::x86.

6.175.1 Attributes

6.175.1.1 HPETEnabled (read/write)

```
boolean IPlatformX86::HPETEnabled
```

This attribute controls if High Precision Event Timer (HPET) is enabled in this VM. Use this property if you want to provide guests with additional time source, or if guest requires HPET to function correctly. Default is false.

6.175.2 getCPUIDLeaf

```
void IPlatformX86::getCPUIDLeaf(
           [in] unsigned long idx,
           [in] unsigned long idxSub,
           [out] unsigned long valEax,
           [out] unsigned long valEbx,
           [out] unsigned long valEcx,
           [out] unsigned long valEdx)
```

idx CPUID leaf index.

idxSub CPUID leaf sub-index (ECX). Set to 0xffffffff (or 0) if not applicable.

valEax CPUID leaf value for register eax.

valEbx CPUID leaf value for register ebx.

valEcx CPUID leaf value for register ecx.

valEdx CPUID leaf value for register edx.

Returns the virtual CPU cpuid information for the specified leaf.
Currently supported index values for cpuid: Standard CPUID leaves: 0 - 0x1f Extended CPUID leaves: 0x80000000 - 0x8000001f VIA CPUID leaves: 0xc0000000 - 0xc000000f
See the Intel, AMD and VIA programmer's manuals for detailed information about the CPUID instruction and its leaves.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: Invalid index.

6.175.3 getCPUIDLeafByOrdinal

```
void IPlatformX86::getCPUIDLeafByOrdinal(
           [in] unsigned long ordinal,
           [out] unsigned long idx,
           [out] unsigned long idxSub,
           [out] unsigned long valEax,
           [out] unsigned long valEbx,
           [out] unsigned long valEcx,
           [out] unsigned long valEdx)
```

ordinal The ordinal number of the leaf to get.

idx CPUID leaf index.

idxSub CPUID leaf sub-index.

valEax CPUID leaf value for register eax.

valEbx CPUID leaf value for register ebx.

valEcx CPUID leaf value for register ecx.

valEdx CPUID leaf value for register edx.

Used to enumerate CPUID information override values.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: Invalid ordinal number is out of range.

6.175.4 getCPUProperty

```
boolean IPlatformX86::getCPUProperty(
            [in] CPUPropertyTypeX86 property)
```

property Property type to query.

Returns the virtual CPU boolean value of the specified property.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: Invalid property.

6.175.5 getHWVirtExProperty

```
boolean IPlatformX86::getHWVirtExProperty(
            [in] HWVirtExPropertyType property)
```

property Property type to query.

Returns the value of the specified hardware virtualization boolean property.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: Invalid property.

6.175.6 removeAllCPUIDLeaves

```
void IPlatformX86::removeAllCPUIDLeaves()
```

Removes all the virtual CPU cpuid leaves

6.175.7 removeCPUIDLeaf

```
void IPlatformX86::removeCPUIDLeaf(
            [in] unsigned long idx,
            [in] unsigned long idxSub)
```

idx CPUID leaf index.

idxSub CPUID leaf sub-index (ECX). Set to 0xffffffff (or 0) if not applicable. The 0xffffffff value works like a wildcard.

Removes the virtual CPU cpuid leaf for the specified index
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: Invalid index.

6.175.8 setCPUIDLeaf

```
void IPlatformX86::setCPUIDLeaf(
            [in] unsigned long idx,
            [in] unsigned long idxSub,
            [in] unsigned long valEax,
            [in] unsigned long valEbx,
            [in] unsigned long valEcx,
            [in] unsigned long valEdx)
```

idx CPUID leaf index.

idxSub CPUID leaf sub-index (ECX). Set to 0xffffffff (or 0) if not applicable. The 0xffffffff causes it to remove all other subleaves before adding one with sub-index 0.

valEax CPUID leaf value for register eax.

valEbx CPUID leaf value for register ebx.

valEcx CPUID leaf value for register ecx.

valEdx CPUID leaf value for register edx.

Sets the virtual CPU cpuid information for the specified leaf. Note that these values are not passed unmodified. VirtualBox clears features that it doesn't support.

Currently supported index values for cpuid: Standard CPUID leaves: 0 - 0x1f Extended CPUID leaves: 0x80000000 - 0x8000001f VIA CPUID leaves: 0xc0000000 - 0xc000000f

The subleaf index is only applicable to certain leaves (see manuals as this is subject to change).

See the Intel, AMD and VIA programmer's manuals for detailed information about the cpuid instruction and its leaves.

Do not use this method unless you know exactly what you're doing. Misuse can lead to random crashes inside VMs.

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` Invalid index.

6.175.9 setCPUProperty

```
void IPlatformX86::setCPUProperty(
            [in] CPUPropertyTypeX86 property,
            [in] boolean value)
```

property Property type to query.

value Property value.

Sets the virtual CPU boolean value of the specified property.

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` Invalid property.

6.175.10 setHWVirtExProperty

```
void IPlatformX86::setHWVirtExProperty(
            [in] HWVirtExPropertyType property,
            [in] boolean value)
```

property Property type to set.

value New property value.

Sets a new value for the specified hardware virtualization boolean property.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: Invalid property.

6.176 IProcess

Abstract parent interface for processes handled by VirtualBox.

6.176.1 Attributes

6.176.1.1 arguments (read-only)

```
wstring IProcess::arguments[]
```

The arguments this process is using for execution.

6.176.1.2 environment (read-only)

```
wstring IProcess::environment[]
```

The initial process environment. Not yet implemented.

6.176.1.3 eventSource (read-only)

```
IEventSource IProcess::eventSource
```

Event source for process events.

6.176.1.4 executablePath (read-only)

```
wstring IProcess::executablePath
```

Full path of the actual executable image.

6.176.1.5 exitCode (read-only)

```
long IProcess::exitCode
```

The exit code. Only available when the process has been terminated normally.

6.176.1.6 name (read-only)

```
wstring IProcess::name
```

The friendly name of this process.

6.176.1.7 PID (read-only)

```
unsigned long IProcess::PID
```

The process ID (PID).

6.176.1.8 status (read-only)

```
ProcessStatus IProcess::status
```

The current process status; see ProcessStatus for more information.

6.176.2 read

```
octet[] IProcess::read(
            [in] unsigned long handle,
            [in] unsigned long toRead,
            [in] unsigned long timeoutMS)
```

handle Handle to read from. Usually 0 is stdin.

toRead Number of bytes to read.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Reads data from a running process.

6.176.3 terminate

```
void IProcess::terminate()
```

Terminates (kills) a running process.

> **Note:** It can take up to 30 seconds to get a guest process killed. In case a guest process could not be killed an appropriate error is returned.

6.176.4 waitFor

```
ProcessWaitResult IProcess::waitFor(
            [in] unsigned long waitFor,
            [in] unsigned long timeoutMS)
```

waitFor Specifies what to wait for; see ProcessWaitForFlag for more information.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Waits for one or more events to happen.

> **Note:** Deprecated and will be removed. Use waitForArray() instead.

6.176.5 waitForArray

```
ProcessWaitResult IProcess::waitForArray(
            [in] ProcessWaitForFlag waitFor[],
            [in] unsigned long timeoutMS)
```

waitFor Specifies what to wait for; see ProcessWaitForFlag for more information.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Waits for one or more events to happen. Scriptable version of waitFor().

6.176.6 write

```
unsigned long IProcess::write(
            [in] unsigned long handle,
            [in] unsigned long flags,
            [in] octet data[],
            [in] unsigned long timeoutMS)
```

handle Handle to write to. Usually 0 is stdin, 1 is stdout and 2 is stderr.

flags A combination of ProcessInputFlag flags.

data Array of bytes to write. The size of the array also specifies how much to write.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Writes data to a running process.

6.176.7 writeArray

```
unsigned long IProcess::writeArray(
            [in] unsigned long handle,
            [in] ProcessInputFlag flags[],
            [in] octet data[],
            [in] unsigned long timeoutMS)
```

handle Handle to write to. Usually 0 is stdin, 1 is stdout and 2 is stderr.

flags A combination of ProcessInputFlag flags.

data Array of bytes to write. The size of the array also specifies how much to write.

timeoutMS Timeout (in ms) to wait for the operation to complete. Pass 0 for an infinite timeout.

Writes data to a running process. Scriptable version of write().

6.177 IProgress

The IProgress interface is used to track and control asynchronous tasks within VirtualBox.

An instance of this is returned every time VirtualBox starts an asynchronous task (in other words, a separate thread) which continues to run after a method call returns. For example, IMachine::saveState(), which saves the state of a running virtual machine, can take a long time to complete. To be able to display a progress bar, a user interface such as the VirtualBox graphical user interface can use the IProgress object returned by that method.

Note that IProgress is a "read-only" interface in the sense that only the VirtualBox internals behind the Main API can create and manipulate progress objects, whereas client code can only use the IProgress object to monitor a task's progress and, if cancelable is `true`, cancel the task by calling cancel().

A task represented by IProgress consists of either one or several sub-operations that run sequentially, one by one (see operation and operationCount). Every operation is identified by a number (starting from 0) and has a separate description.

You can find the individual percentage of completion of the current operation in operationPercent and the percentage of completion of the task as a whole in percent.

Similarly, you can wait for the completion of a particular operation via waitForOperationCompletion() or for the completion of the whole task via waitForCompletion().

6.177.1 Attributes

6.177.1.1 id (read-only)

```
uuid IProgress::id
```

ID of the task.

6.177.1.2 description (read-only)

```
wstring IProgress::description
```

Description of the task.

6.177.1.3 initiator (read-only)

```
$unknown IProgress::initiator
```

Initiator of the task.

6.177.1.4 cancelable (read-only)

```
boolean IProgress::cancelable
```

Whether the task can be interrupted.

6.177.1.5 percent (read-only)

```
unsigned long IProgress::percent
```

Current progress value of the task as a whole, in percent. This value depends on how many operations are already complete. Returns 100 if completed is `true`.

6.177.1.6 timeRemaining (read-only)

```
long IProgress::timeRemaining
```

Estimated remaining time until the task completes, in seconds. Returns 0 once the task has completed; returns -1 if the remaining time cannot be computed, in particular if the current progress is 0.

Even if a value is returned, the estimate will be unreliable for low progress values. It will become more reliable as the task progresses; it is not recommended to display an ETA before at least 20% of a task have completed.

6.177.1.7 completed (read-only)

```
boolean IProgress::completed
```

Whether the task has been completed.

6.177.1.8 canceled (read-only)

```
boolean IProgress::canceled
```

Whether the task has been canceled.

6.177.1.9 resultCode (read-only)

```
long IProgress::resultCode
```

Result code of the progress task. Valid only if completed is `true`.

6.177.1.10 errorInfo (read-only)

```
IVirtualBoxErrorInfo IProgress::errorInfo
```

Extended information about the unsuccessful result of the progress operation. May be `null` if no extended information is available. Valid only if completed is `true` and resultCode indicates a failure.

6.177.1.11 operationCount (read-only)

```
unsigned long IProgress::operationCount
```

Number of sub-operations this task is divided into. Every task consists of at least one suboperation.

6.177.1.12 operation (read-only)

```
unsigned long IProgress::operation
```

Number of the sub-operation being currently executed.

6.177.1.13 operationDescription (read-only)

```
wstring IProgress::operationDescription
```

Description of the sub-operation being currently executed.

6.177.1.14 operationPercent (read-only)

```
unsigned long IProgress::operationPercent
```

Progress value of the current sub-operation only, in percent.

6.177.1.15 operationWeight (read-only)

```
unsigned long IProgress::operationWeight
```

Weight value of the current sub-operation only.

6.177.1.16 timeout (read/write)

```
unsigned long IProgress::timeout
```

When non-zero, this specifies the number of milliseconds after which the operation will automatically be canceled. This can only be set on cancelable objects.

6.177.1.17 eventSource (read-only)

```
IEventSource IProgress::eventSource
```

6.177.2 cancel

```
void IProgress::cancel()
```

Cancels the task.

> **Note:** If cancelable is `false`, then this method will fail.

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_OBJECT_STATE`: Operation cannot be canceled.

6.177.3 waitForCompletion

```
void IProgress::waitForCompletion(
            [in] long timeout)
```

timeout Maximum time in milliseconds to wait or -1 to wait indefinitely.

Waits until the task is done (including all sub-operations) with a given timeout in milliseconds; specify -1 for an indefinite wait.

Note that the VirtualBox/XPCOM/COM/native event queues of the calling thread are not processed while waiting. Neglecting event queues may have dire consequences (degrade performance, resource hogs, deadlocks, etc.), this is specially so for the main thread on platforms using XPCOM. Callers are advised wait for short periods and service their event queues between calls, or to create a worker thread to do the waiting.

If this method fails, the following error codes may be reported:

- `VBOX_E_IPRT_ERROR`: Failed to wait for task completion.

6.177.4 waitForOperationCompletion

```
void IProgress::waitForOperationCompletion(
            [in] unsigned long operation,
            [in] long timeout)
```

operation Number of the operation to wait for. Must be less than operationCount.

timeout Maximum time in milliseconds to wait or -1 to wait indefinitely.

Waits until the given operation is done with a given timeout in milliseconds; specify -1 for an indefinite wait.

See for event queue considerations.

If this method fails, the following error codes may be reported:

- `VBOX_E_IPRT_ERROR`: Failed to wait for operation completion.

6.178 IProgressCreatedEvent (IProgressEvent)

> **Note:** This interface extends IProgressEvent and therefore supports all its methods and attributes as well.

Notification of a new progress object creation/deletion. Covers purely progress objects in VBoxSVC. This event is signaled on the event source of IVirtualBox, unlike the other progress events.

6.178.1 Attributes

6.178.1.1 create (read-only)

```
boolean IProgressCreatedEvent::create
```

If `true`, the progress object was created, otherwise it was deleted.

6.179 IProgressEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Base abstract interface for all progress events.

6.179.1 Attributes

6.179.1.1 progressId (read-only)

```
uuid IProgressEvent::progressId
```

GUID of the progress this event relates to.

6.180 IProgressPercentageChangedEvent (IProgressEvent)

> **Note:** This interface extends IProgressEvent and therefore supports all its methods and attributes as well.

Progress state change event.

6.180.1 Attributes

6.180.1.1 percent (read-only)

```
long IProgressPercentageChangedEvent::percent
```

New percent

6.181 IProgressTaskCompletedEvent (IProgressEvent)

> **Note:** This interface extends IProgressEvent and therefore supports all its methods and attributes as well.

Progress task completion event.

6.181.1 Attributes

6.181.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IProgressTaskCompletedEvent::midlDoesNotLikeEmptyInterfaces
```

6.182 IRangedInteger64FormValue (IFormValue)

> **Note:** This interface extends IFormValue and therefore supports all its methods and attributes as well.

6.182.1 Attributes

6.182.1.1 suffix (read-only)

```
wstring IRangedInteger64FormValue::suffix
```

Counterpart of the IFormValue::label attribute. May be null or empty. Usually used for units.

6.182.1.2 minimum (read-only)

```
long long IRangedInteger64FormValue::minimum
```

6.182.1.3 maximum (read-only)

```
long long IRangedInteger64FormValue::maximum
```

6.182.2 getInteger

```
long long IRangedInteger64FormValue::getInteger()
```

6.182.3 setInteger

```
IProgress IRangedInteger64FormValue::setInteger(
          [in] long long value)
```

value

6.183 IRangedIntegerFormValue (IFormValue)

> **Note:** This interface extends IFormValue and therefore supports all its methods and attributes as well.

6.183.1 Attributes

6.183.1.1 suffix (read-only)

```
wstring IRangedIntegerFormValue::suffix
```

Counterpart of the IFormValue::label attribute. May be null or empty. Usually used for units.

6.183.1.2 minimum (read-only)

```
long IRangedIntegerFormValue::minimum
```

6.183.1.3 maximum (read-only)

```
long IRangedIntegerFormValue::maximum
```

6.183.2 getInteger

```
long IRangedIntegerFormValue::getInteger()
```

6.183.3 setInteger

```
IProgress IRangedIntegerFormValue::setInteger(
          [in] long value)
```

value

6.184 IRecordingScreenSettings

The IRecordingScreenSettings interface represents recording settings of a single virtual screen. This is used only in the IRecordingSettings interface.

6.184.1 Attributes

6.184.1.1 id (read-only)

```
unsigned long IRecordingScreenSettings::id
```

This attribute contains the screen ID bound to these settings.

6.184.1.2 enabled (read/write)

```
boolean IRecordingScreenSettings::enabled
```

This setting determines whether this screen is enabled while recording.

6.184.1.3 features (read/write)

```
RecordingFeature IRecordingScreenSettings::features[]
```

This setting determines all enabled recording features for this screen.

6.184.1.4 destination (read/write)

```
RecordingDestination IRecordingScreenSettings::destination
```

This setting determines the recording destination for this screen.

6.184.1.5 filename (read/write)

```
wstring IRecordingScreenSettings::filename
```

This setting determines the filename VirtualBox uses to save the recorded content. This setting cannot be changed while video recording is enabled.

> **Note:** When setting this attribute, the specified path has to be absolute (full path). When reading this attribute, a full path is always returned.

6.184.1.6 maxTime (read/write)

`unsigned long IRecordingScreenSettings::maxTime`

This setting defines the maximum amount of time in seconds to record. Recording will stop as soon as the defined time interval has elapsed. If this value is zero, recording will not be limited by time. This setting cannot be changed while recording is enabled.

6.184.1.7 maxFileSize (read/write)

`unsigned long IRecordingScreenSettings::maxFileSize`

This setting determines the maximal number of recording file size in MB. Recording will stop as soon as the file size has reached the defined value. If this value is zero, recording will not be limited by the file size. This setting cannot be changed while recording is enabled.

6.184.1.8 options (read/write)

`wstring IRecordingScreenSettings::options`

This setting contains any additional recording options required in comma-separated key=value format, which are currently not represented via own attributes. Consider these options as experimental and mostly for codec-specific settings, and are subject to change. This setting cannot be changed while recording is enabled.

6.184.1.9 audioCodec (read/write)

`RecordingAudioCodec IRecordingScreenSettings::audioCodec`

Determines the audio codec to use for encoding the recorded audio data. This setting cannot be changed while recording is enabled.

> **Note:** Only the Opus codec is supported currently.

6.184.1.10 audioRateControlMode (read/write)

`RecordingRateControlMode IRecordingScreenSettings::audioRateControlMode`

Determines the audio rate control mode. This setting cannot be changed while recording is enabled.

6.184.1.11 audioDeadline (read/write)

`RecordingCodecDeadline IRecordingScreenSettings::audioDeadline`

Determines the audio deadline to use. This setting cannot be changed while recording is enabled.

6.184.1.12 audioHz (read/write)

`unsigned long IRecordingScreenSettings::audioHz`

Determines the Hertz (Hz) rate of the recorded audio data. This setting cannot be changed while recording is enabled.

6.184.1.13 audioBits (read/write)

```
unsigned long IRecordingScreenSettings::audioBits
```

Determines the bits per sample of the recorded audio data. This setting cannot be changed while recording is enabled.

6.184.1.14 audioChannels (read/write)

```
unsigned long IRecordingScreenSettings::audioChannels
```

Determines the audio channels of the recorded audio data. Specify 2 for stereo or 1 for mono. More than stereo (2) channels are not supported at the moment. This setting cannot be changed while recording is enabled.

6.184.1.15 videoCodec (read/write)

```
RecordingVideoCodec IRecordingScreenSettings::videoCodec
```

Determines the video codec to use for encoding the recorded video data. This setting cannot be changed while recording is enabled.

> **Note:** Only the VP8 codec is supported currently.

6.184.1.16 videoDeadline (read/write)

```
RecordingCodecDeadline IRecordingScreenSettings::videoDeadline
```

Determines the video deadline to use. This setting cannot be changed while recording is enabled.

6.184.1.17 videoWidth (read/write)

```
unsigned long IRecordingScreenSettings::videoWidth
```

Determines the horizontal resolution of the recorded video data. This setting cannot be changed while recording is enabled.

6.184.1.18 videoHeight (read/write)

```
unsigned long IRecordingScreenSettings::videoHeight
```

Determines the vertical resolution of the recorded video data. This setting cannot be changed while recording is enabled.

6.184.1.19 videoRate (read/write)

```
unsigned long IRecordingScreenSettings::videoRate
```

Determines the bitrate in kilobits per second. Increasing this value makes the video look better for the cost of an increased file size or transfer rate. This setting cannot be changed while recording is enabled.

6.184.1.20 videoRateControlMode (read/write)

```
RecordingRateControlMode IRecordingScreenSettings::videoRateControlMode
```

Determines the video rate control mode. This setting cannot be changed while recording is enabled.

6.184.1.21 videoFPS (read/write)

```
unsigned long IRecordingScreenSettings::videoFPS
```

Determines the maximum number of frames per second (FPS). Frames with a higher frequency will be skipped. Reducing this value increases the number of skipped frames and reduces the file size or transfer rate. This setting cannot be changed while recording is enabled.

6.184.1.22 videoScalingMode (read/write)

```
RecordingVideoScalingMode IRecordingScreenSettings::videoScalingMode
```

Determines the video scaling mode to use. This setting cannot be changed while recording is enabled.

6.184.2 isFeatureEnabled

```
boolean IRecordingScreenSettings::isFeatureEnabled(
            [in] RecordingFeature feature)
```

feature Feature to check for.

Returns whether a particular recording feature is enabled for this screen or not.

6.185 IRecordingScreenStateChangedEvent (IRecordingStateChangedEvent)

> **Note:** This interface extends IRecordingStateChangedEvent and therefore supports all its methods and attributes as well.

Notification when a screen recording state has changed.

6.185.1 Attributes

6.185.1.1 screen (read-only)

```
unsigned long IRecordingScreenStateChangedEvent::screen
```

Number of screen for which the event has happened.

6.186 IRecordingSettings

The IRecordingSettings interface represents recording settings of the virtual machine. This is used only in the IMachine::recordingSettings attribute.

6.186.1 Attributes

6.186.1.1 paused (read/write)

```
boolean IRecordingSettings::paused
```

Pauses the recording or returns the current paused status.

> **Note:** Currently not implemented.

> **Note:** This will not affect the recording progress object.

If recording already has been paused, this attribute does nothing.

6.186.1.2 enabled (read/write)

```
boolean IRecordingSettings::enabled
```

This setting determines whether recording is enabled for this VM.

Note that this attribute does not actually start or stop recording. For starting recording, use start(), for stopping, use the progress object via progress.

If a VM is being started with this setting enabled, recording automatically will be started with the currently configured recording settings.

6.186.1.3 screens (read-only)

```
IRecordingScreenSettings IRecordingSettings::screens[]
```

This setting returns an array for recording settings of all configured virtual screens.

6.186.1.4 progress (read-only)

```
IProgress IRecordingSettings::progress
```

Returns the recording progress object. Also valid when recording has been paused.

Cancelling the progress will stop the recording.

6.186.2 getScreenSettings

```
IRecordingScreenSettings IRecordingSettings::getScreenSettings(
            [in] unsigned long screenId)
```

screenId Screen ID to retrieve recording screen settings for.

Returns the recording settings for a particular screen.

6.186.3 resume

```
void IRecordingSettings::resume()
```

Resumes the recording.

> **Note:** Currently not implemented.

> **Note:** This will not affect the recording progress object.

6.186.4 start

```
IProgress IRecordingSettings::start()
```

Starts recording and returns a progress object.

6.187 IRecordingStateChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the machine's recording state has changed.

6.187.1 Attributes

6.187.1.1 state (read-only)

```
RecordingState IRecordingStateChangedEvent::state
```

New recording state.

6.187.1.2 error (read-only)

```
IVirtualBoxErrorInfo IRecordingStateChangedEvent::error
```

Error information in case the event is indicating an error.

6.188 IResourceStore

Provides access to resources accessed by various device implementations and created when the VM is started (only the FDT currently). This is not accessible from anywhere right now but just for convenience if required later on.

6.188.1 Attributes

6.188.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IResourceStore::midlDoesNotLikeEmptyInterfaces
```

6.189 IReusableEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Base abstract interface for all reusable events.

6.189.1 Attributes

6.189.1.1 generation (read-only)

```
unsigned long IReusableEvent::generation
```

Current generation of event, incremented on reuse.

6.189.2 reuse

```
void IReusableEvent::reuse()
```

Marks an event as reused, increments 'generation', fields shall no longer be considered valid.

6.190 IRuntimeErrorEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when an error happens during the virtual machine execution.
There are three kinds of runtime errors:

- *fatal*
- *non-fatal with retry*
- *non-fatal warnings*

Fatal errors are indicated by the `fatal` parameter set to `true`. In case of fatal errors, the virtual machine execution is always paused before calling this notification, and the notification handler is supposed either to immediately save the virtual machine state using IMachine::saveState() or power it off using IConsole::powerDown(). Resuming the execution can lead to unpredictable results.

Non-fatal errors and warnings are indicated by the `fatal` parameter set to `false`. If the virtual machine is in the Paused state by the time the error notification is received, it means that the user can *try to resume* the machine execution after attempting to solve the problem that caused the error. In this case, the notification handler is supposed to show an appropriate message to the user (depending on the value of the `id` parameter) that offers several actions such as *Retry*, *Save* or *Power Off*. If the user wants to retry, the notification handler should continue the machine execution using the IConsole::resume() call. If the machine execution is not Paused during this notification, then it means this notification is a *warning* (for example, about a fatal condition that can happen very soon); no immediate action is required from the user, the machine continues its normal execution.

Note that in either case the notification handler **must not** perform any action directly on a thread where this notification is called. Everything it is allowed to do is to post a message to another thread that will then talk to the user and take the corresponding action.

Currently, the following error identifiers are known:

- `"HostMemoryLow"`
- `"HostAudioNotResponding"`
- `"VDIStorageFull"`
- `"3DSupportIncompatibleAdditions"`

6.190.1 Attributes

6.190.1.1 fatal (read-only)

```
boolean IRuntimeErrorEvent::fatal
```

Whether the error is fatal or not.

6.190.1.2 id (read-only)

```
wstring IRuntimeErrorEvent::id
```

Error identifier.

6.190.1.3 message (read-only)

```
wstring IRuntimeErrorEvent::message
```

Optional error message.

6.191 ISerialPort

The ISerialPort interface represents the virtual serial port device.

The virtual serial port device acts like an ordinary serial port inside the virtual machine. This device communicates to the real serial port hardware in one of two modes: host pipe or host device.

In host pipe mode, the #path attribute specifies the path to the pipe on the host computer that represents a serial port. The #server attribute determines if this pipe is created by the virtual machine process at machine startup or it must already exist before starting machine execution.

In host device mode, the #path attribute specifies the name of the serial port device on the host computer.

There is also a third communication mode: the disconnected mode. In this mode, the guest OS running inside the virtual machine will be able to detect the serial port, but all port write operations will be discarded and all port read operations will return no data.

See also: IMachine::getSerialPort()

6.191.1 Attributes

6.191.1.1 slot (read-only)

```
unsigned long ISerialPort::slot
```

Slot number this serial port is plugged into. Corresponds to the value you pass to IMachine::getSerialPort() to obtain this instance.

6.191.1.2 enabled (read/write)

```
boolean ISerialPort::enabled
```

Flag whether the serial port is enabled. If disabled, the serial port will not be reported to the guest OS.

6.191.1.3 IOAddress (read/write)

```
unsigned long ISerialPort::IOAddress
```

Base I/O address of the serial port. For x86, this is an I/O port, for ARM this is an MMIO address.

6.191.1.4 IRQ (read/write)

```
unsigned long ISerialPort::IRQ
```

IRQ number of the serial port.

6.191.1.5 hostMode (read/write)

`PortMode ISerialPort::hostMode`

How is this port connected to the host.

> **Note:** Changing this attribute may fail if the conditions for path are not met.

6.191.1.6 server (read/write)

`boolean ISerialPort::server`

Flag whether this serial port acts as a server (creates a new pipe on the host) or as a client (uses the existing pipe). This attribute is used only when hostMode is PortMode_HostPipe or PortMode_TCP.

6.191.1.7 path (read/write)

`wstring ISerialPort::path`

Path to the serial port's pipe on the host when hostMode is PortMode_HostPipe, the host serial device name when hostMode is PortMode_HostDevice or the TCP **port** (server) or **hostname:port** (client) when hostMode is PortMode_TCP. For those cases, setting a `null` or empty string as the attribute's value is an error. Otherwise, the value of this property is ignored.

6.191.1.8 uartType (read/write)

`UartType ISerialPort::uartType`

Selects the emulated UART implementation.

6.192 ISerialPortChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a property of one of the virtual serial ports changes. Interested callees should use ISerialPort methods and attributes to find out what has changed.

6.192.1 Attributes

6.192.1.1 serialPort (read-only)

`ISerialPort ISerialPortChangedEvent::serialPort`

Serial port that is subject to change.

6.193 ISession

The ISession interface represents a client process and allows for locking virtual machines (represented by IMachine objects) to prevent conflicting changes to the machine.

Any caller wishing to manipulate a virtual machine needs to create a session object first, which lives in its own process space. Such session objects are then associated with IMachine objects living in the VirtualBox server process to coordinate such changes.

There are two typical scenarios in which sessions are used:

- To alter machine settings or control a running virtual machine, one needs to lock a machine for a given session (client process) by calling IMachine::lockMachine().

  Whereas multiple sessions may control a running virtual machine, only one process can obtain a write lock on the machine to prevent conflicting changes. A write lock is also needed if a process wants to actually run a virtual machine in its own context, such as the VirtualBox GUI or VBoxHeadless front-ends. They must also lock a machine for their own sessions before they are allowed to power up the virtual machine.

  As a result, no machine settings can be altered while another process is already using it, either because that process is modifying machine settings or because the machine is running.

- To start a VM using one of the existing VirtualBox front-ends (e.g. the VirtualBox GUI or VBoxHeadless), one would use IMachine::launchVMProcess(), which also takes a session object as its first parameter. This session then identifies the caller and lets the caller control the started machine (for example, pause machine execution or power it down) as well as be notified about machine execution state changes.

How sessions objects are created in a client process depends on whether you use the Main API via COM or via the webservice:

- When using the COM API directly, an object of the Session class from the VirtualBox type library needs to be created. In regular COM C++ client code, this can be done by calling `createLocalObject()`, a standard COM API. This object will then act as a local session object in further calls to open a session.

- In the webservice, the session manager (IWebsessionManager) instead creates a session object automatically whenever IWebsessionManager::logon() is called. A managed object reference to that session object can be retrieved by calling IWebsessionManager::getSessionObject().

6.193.1 Attributes

6.193.1.1 state (read-only)

`SessionState ISession::state`

Current state of this session.

6.193.1.2 type (read-only)

`SessionType ISession::type`

Type of this session. The value of this attribute is valid only if the session currently has a machine locked (i.e. its state is Locked), otherwise an error will be returned.

6.193.1.3 name (read/write)

`wstring ISession::name`

Name of this session. Important only for VM sessions, otherwise it it will be remembered, but not used for anything significant (and can be left at the empty string which is the default). The value can only be changed when the session state is SessionState_Unlocked. Make sure that you use a descriptive name which does not conflict with the VM process session names: “GUI/Qt”, “GUI/SDL” and “headless”.

6.193.1.4 machine (read-only)

`IMachine ISession::machine`

Machine object associated with this session.

6.193.1.5 console (read-only)

`IConsole ISession::console`

Console object associated with this session. Only sessions which locked the machine for a VM process have a non-null console.

6.193.2 unlockMachine

`void ISession::unlockMachine()`

Unlocks a machine that was previously locked for the current session.

Calling this method is required every time a machine has been locked for a particular session using the IMachine::launchVMProcess() or IMachine::lockMachine() calls. Otherwise the state of the machine will be set to Aborted on the server, and changes made to the machine settings will be lost.

Generally, it is recommended to unlock all machines explicitly before terminating the application (regardless of the reason for the termination).

> **Note:** Do not expect the session state (state to return to “Unlocked” immediately after you invoke this method, particularly if you have started a new VM process. The session state will automatically return to “Unlocked” once the VM is no longer executing, which can of course take a very long time.

If this method fails, the following error codes may be reported:

- `E_UNEXPECTED:` Session is not locked.

6.194 ISessionStateChangedEvent (IMachineEvent)

> **Note:** This interface extends IMachineEvent and therefore supports all its methods and attributes as well.

The state of the session for the given machine was changed. See also: IMachine::sessionState

6.194.1 Attributes

6.194.1.1 state (read-only)

`SessionState ISessionStateChangedEvent::state`

New session state.

6.195 ISharedFolder

The ISharedFolder interface represents a folder in the host computer's file system accessible from the guest OS running inside a virtual machine using an associated logical name.

There are three types of shared folders:

- *Global* (IVirtualBox::sharedFolders[]), shared folders available to all virtual machines.
- *Permanent* (IMachine::sharedFolders[]), VM-specific shared folders available to the given virtual machine at startup.
- *Transient* (IConsole::sharedFolders[]), VM-specific shared folders created in the session context (for example, when the virtual machine is running) and automatically discarded when the session is closed (the VM is powered off).

Logical names of shared folders must be unique within the given scope (global, permanent or transient). However, they do not need to be unique across scopes. In this case, the definition of the shared folder in a more specific scope takes precedence over definitions in all other scopes. The order of precedence is (more specific to more general):

1. Transient definitions
2. Permanent definitions
3. Global definitions

For example, if MyMachine has a shared folder named `C_DRIVE` (that points to `C:\\`), then creating a transient shared folder named `C_DRIVE` (that points to `C:\\\\WINDOWS`) will change the definition of `C_DRIVE` in the guest OS so that `\\\\VBOXSVR\\C_DRIVE` will give access to `C:\\WINDOWS` instead of `C:\\` on the host PC. Removing the transient shared folder `C_DRIVE` will restore the previous (permanent) definition of `C_DRIVE` that points to `C:\\` if it still exists.

Note that permanent and transient shared folders of different machines are in different name spaces, so they don't overlap and don't need to have unique logical names.

6.195.1 Attributes

6.195.1.1 name (read-only)

`wstring ISharedFolder::name`

Logical name of the shared folder.

6.195.1.2 hostPath (read-only)

`wstring ISharedFolder::hostPath`

Full path to the shared folder in the host file system.

6.195.1.3 accessible (read-only)

`boolean ISharedFolder::accessible`

Whether the folder defined by the host path is currently accessible or not.
For example, the folder can be inaccessible if it is placed on the network share that is not available by the time this property is read.

6.195.1.4 writable (read/write)

`boolean ISharedFolder::writable`

Whether the folder defined by the host path is writable or not.

6.195.1.5 autoMount (read/write)

`boolean ISharedFolder::autoMount`

Whether the folder gets automatically mounted by the guest or not.

6.195.1.6 autoMountPoint (read/write)

`wstring ISharedFolder::autoMountPoint`

Desired mount point in the guest for automatically mounting the folder when autoMount is set. For Windows and OS/2 guests this should be a drive letter, while other guests it should be a absolute directory. It is possible to combine the two, e.g. "T:/mnt/testrsrc" will be attached to "T:" by windows and OS/2 while the unixy guests will mount it at "/mnt/testrsrc".
When empty the guest will choose a mount point. The guest may do so too should the specified mount point be in use or otherwise unusable.

6.195.1.7 lastAccessError (read-only)

`wstring ISharedFolder::lastAccessError`

Text message that represents the result of the last accessibility check.
Accessibility checks are performed each time the accessible attribute is read. An empty string is returned if the last accessibility check was successful. A non-empty string indicates a failure and should normally describe a reason of the failure (for example, a file read error).

6.195.1.8 symlinkPolicy (read/write)

`SymlinkPolicy ISharedFolder::symlinkPolicy`

The security policy for allowing guest VMs to create symbolic links within a Shared Folder.

6.196 ISharedFolderChangedEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a shared folder is added or removed. The `scope` argument defines one of three scopes: global shared folders (Global), permanent shared folders of the machine (Machine) or transient shared folders of the machine (Session). Interested callees should use query the corresponding collections to find out what has changed.

6.196.1 Attributes

6.196.1.1 scope (read-only)

`Scope ISharedFolderChangedEvent::scope`

Scope of the notification.

6.197 IShowWindowEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a call to IMachine::showConsoleWindow() requests the console window to be activated and brought to foreground on the desktop of the host PC.

This notification should cause the VM console process to perform the requested action as described above. If it is impossible to do it at a time of this notification, this method should return a failure.

Note that many modern window managers on many platforms implement some sort of focus stealing prevention logic, so that it may be impossible to activate a window without the help of the currently active application (which is supposedly an initiator of this notification). In this case, this method must return a non-zero identifier that represents the top-level window of the VM console process. The caller, if it represents a currently active process, is responsible to use this identifier (in a platform-dependent manner) to perform actual window activation.

This method must set `winId` to zero if it has performed all actions necessary to complete the request and the console window is now active and in foreground, to indicate that no further action is required on the caller's side.

6.197.1 Attributes

6.197.1.1 winId (read/write)

`long long IShowWindowEvent::winId`

Platform-dependent identifier of the top-level VM console window, or zero if this method has performed all actions necessary to implement the *show window* semantics for the given platform and/or this VirtualBox front-end.

6.198 ISnapshot

The ISnapshot interface represents a snapshot of the virtual machine.

Together with the differencing media that are created when a snapshot is taken, a machine can be brought back to the exact state it was in when the snapshot was taken.

The ISnapshot interface has no methods, only attributes; snapshots are controlled through methods of the IMachine interface which also manage the media associated with the snapshot. The following operations exist:

- IMachine::takeSnapshot() creates a new snapshot by creating new, empty differencing images for the machine's media and saving the VM settings and (if the VM is running) the current VM state in the snapshot.

  The differencing images will then receive all data written to the machine's media, while their parent (base) images remain unmodified after the snapshot has been taken (see

IMedium for details about differencing images). This simplifies restoring a machine to the state of a snapshot: only the differencing images need to be deleted.

The current machine state is not changed by taking a snapshot except that IMachine::currentSnapshot is set to the newly created snapshot, which is also added to the machine's snapshots tree.

- IMachine::restoreSnapshot() resets a machine to the state of a previous snapshot by deleting the differencing image of each of the machine's media and setting the machine's settings and state to the state that was saved in the snapshot (if any).

  This destroys the machine's current state. After calling this, IMachine::currentSnapshot points to the snapshot that was restored.

- IMachine::deleteSnapshot() deletes a snapshot without affecting the current machine state.

  This does not change the current machine state, but instead frees the resources allocated when the snapshot was taken: the settings and machine state file are deleted (if any), and the snapshot's differencing image for each of the machine's media gets merged with its parent image.

  Neither the current machine state nor other snapshots are affected by this operation, except that parent media will be modified to contain the disk data associated with the snapshot being deleted.

  When deleting the current snapshot, the IMachine::currentSnapshot attribute is set to the current snapshot's parent or `null` if it has no parent. Otherwise the attribute is unchanged.

Each snapshot contains a copy of virtual machine's settings (hardware configuration etc.). This copy is contained in an immutable (read-only) instance of IMachine which is available from the snapshot's machine attribute. When restoring the snapshot, these settings are copied back to the original machine.

In addition, if the machine was running when the snapshot was taken (IMachine::state is Running), the current VM state is saved in the snapshot (similarly to what happens when a VM's state is saved). The snapshot is then said to be *online* because when restoring it, the VM will be running.

If the machine was in the Saved or AbortedSaved state, the snapshot receives a copy of the execution state file (IMachine::stateFilePath).

Otherwise, if the machine was not running (PoweredOff or Aborted), the snapshot is *offline*; it then contains a so-called "zero execution state", representing a machine that is powered off.

6.198.1 Attributes

6.198.1.1 id (read-only)

```
uuid ISnapshot::id
```

UUID of the snapshot.

6.198.1.2 name (read/write)

```
wstring ISnapshot::name
```

Short name of the snapshot.

Note: Setting this attribute causes IMachine::saveSettings() to be called implicitly.

6.198.1.3 description (read/write)

```
wstring ISnapshot::description
```

Optional description of the snapshot.

> **Note:** Setting this attribute causes IMachine::saveSettings() to be called implicitly.

6.198.1.4 timeStamp (read-only)

```
long long ISnapshot::timeStamp
```

Timestamp of the snapshot, in milliseconds since 1970-01-01 UTC.

6.198.1.5 online (read-only)

```
boolean ISnapshot::online
```

`true` if this snapshot is an online snapshot and `false` otherwise.

When this attribute is `true`, the IMachine::stateFilePath attribute of the machine object associated with this snapshot will point to the saved state file. Otherwise, it will be an empty string.

6.198.1.6 machine (read-only)

```
IMachine ISnapshot::machine
```

Virtual machine this snapshot is taken on. This object stores all settings the machine had when taking this snapshot.

> **Note:** The returned machine object is immutable, i.e. no any settings can be changed.

6.198.1.7 parent (read-only)

```
ISnapshot ISnapshot::parent
```

Parent snapshot (a snapshot this one is based on), or `null` if the snapshot has no parent (i.e. is the first snapshot).

6.198.1.8 children (read-only)

```
ISnapshot ISnapshot::children[]
```

Child snapshots (all snapshots having this one as a parent). By inspecting this attribute starting with a machine's root snapshot (which can be obtained by calling IMachine::findSnapshot() with a `null` UUID), a machine's snapshots tree can be iterated over.

6.198.1.9 childrenCount (read-only)

```
unsigned long ISnapshot::childrenCount
```

Returns the number of direct children of this snapshot.

6.199 ISnapshotChangedEvent (ISnapshotEvent)

> **Note:** This interface extends ISnapshotEvent and therefore supports all its methods and attributes as well.

Snapshot properties (name and/or description) have been changed. See also: ISnapshot

6.199.1 Attributes

6.199.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean ISnapshotChangedEvent::midlDoesNotLikeEmptyInterfaces
```

6.200 ISnapshotDeletedEvent (ISnapshotEvent)

> **Note:** This interface extends ISnapshotEvent and therefore supports all its methods and attributes as well.

Snapshot of the given machine has been deleted.

> **Note:** This notification is delivered **after** the snapshot object has been uninitialized on the server (so that any attempt to call its methods will return an error).

See also: ISnapshot

6.200.1 Attributes

6.200.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean ISnapshotDeletedEvent::midlDoesNotLikeEmptyInterfaces
```

6.201 ISnapshotEvent (IMachineEvent)

> **Note:** This interface extends IMachineEvent and therefore supports all its methods and attributes as well.

Base interface for all snapshot events.

6.201.1 Attributes

6.201.1.1 snapshotId (read-only)

```
uuid ISnapshotEvent::snapshotId
```

ID of the snapshot this event relates to.

6.202 ISnapshotRestoredEvent (ISnapshotEvent)

> **Note:** This interface extends ISnapshotEvent and therefore supports all its methods and attributes as well.

Snapshot of the given machine has been restored. See also: ISnapshot

6.202.1 Attributes

6.202.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean ISnapshotRestoredEvent::midlDoesNotLikeEmptyInterfaces
```

6.203 ISnapshotTakenEvent (ISnapshotEvent)

> **Note:** This interface extends ISnapshotEvent and therefore supports all its methods and attributes as well.

A new snapshot of the machine has been taken. See also: ISnapshot

6.203.1 Attributes

6.203.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean ISnapshotTakenEvent::midlDoesNotLikeEmptyInterfaces
```

6.204 IStateChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the execution state of the machine has changed. The new state is given.

6.204.1 Attributes

6.204.1.1 state (read-only)

```
MachineState IStateChangedEvent::state
```

New machine state.

6.205 IStorageController

Represents a storage controller that is attached to a virtual machine (IMachine). Just as drives (hard disks, DVDs, FDs) are attached to storage controllers in a real computer, virtual drives (represented by IMediumAttachment) are attached to virtual storage controllers, represented by this interface.

As opposed to physical hardware, VirtualBox has a very generic concept of a storage controller, and for purposes of the Main API, all virtual storage is attached to virtual machines via instances of this interface. There are five types of such virtual storage controllers: IDE, SCSI, SATA, SAS

and Floppy (see bus). Depending on which of these four is used, certain sub-types may be available and can be selected in controllerType.

Depending on these settings, the guest operating system might see significantly different virtual hardware.

6.205.1 Attributes

6.205.1.1 name (read/write)

```
wstring IStorageController::name
```

Name of the storage controller, as originally specified with IMachine::addStorageController(). This then uniquely identifies this controller with other method calls such as IMachine::attachDevice() and IMachine::mountMedium().

6.205.1.2 maxDevicesPerPortCount (read-only)

```
unsigned long IStorageController::maxDevicesPerPortCount
```

Maximum number of devices which can be attached to one port.

6.205.1.3 minPortCount (read-only)

```
unsigned long IStorageController::minPortCount
```

Minimum number of ports that portCount can be set to.

6.205.1.4 maxPortCount (read-only)

```
unsigned long IStorageController::maxPortCount
```

Maximum number of ports that portCount can be set to.

6.205.1.5 instance (read/write)

```
unsigned long IStorageController::instance
```

The instance number of the device in the running VM.

6.205.1.6 portCount (read/write)

```
unsigned long IStorageController::portCount
```

The number of currently usable ports on the controller. The minimum and maximum number of ports for one controller are stored in minPortCount and maxPortCount.

6.205.1.7 bus (read-only)

```
StorageBus IStorageController::bus
```

The bus type of the storage controller (IDE, SATA, SCSI, SAS or Floppy).

6.205.1.8 controllerType (read/write)

`StorageControllerType IStorageController::controllerType`

The exact variant of storage controller hardware presented to the guest. Depending on this value, VirtualBox will provide a different virtual storage controller hardware to the guest. For SATA, SAS and floppy controllers, only one variant is available, but for IDE and SCSI, there are several.

For SCSI controllers, the default type is LsiLogic.

6.205.1.9 useHostIOCache (read/write)

`boolean IStorageController::useHostIOCache`

If true, the storage controller emulation will use a dedicated I/O thread, enable the host I/O caches and use synchronous file APIs on the host. This was the only option in the API before VirtualBox 3.2 and is still the default for IDE controllers.

If false, the host I/O cache will be disabled for image files attached to this storage controller. Instead, the storage controller emulation will use asynchronous I/O APIs on the host. This makes it possible to turn off the host I/O caches because the emulation can handle unaligned access to the file. This should be used on OS X and Linux hosts if a high I/O load is expected or many virtual machines are running at the same time to prevent I/O cache related hangs. This option new with the API of VirtualBox 3.2 and is now the default for non-IDE storage controllers.

6.205.1.10 bootable (read-only)

`boolean IStorageController::bootable`

Returns whether it is possible to boot from disks attached to this controller.

6.206 IStorageControllerChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a storage controllers changes.

6.206.1 Attributes

6.206.1.1 machinId (read-only)

`uuid IStorageControllerChangedEvent::machinId`

The id of the machine containing the storage controller.

6.206.1.2 controllerName (read-only)

`wstring IStorageControllerChangedEvent::controllerName`

The name of the storage controller.

6.207 IStorageDeviceChangedEvent (IEvent)

Note: This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a storage device is attached or removed.

6.207.1 Attributes

6.207.1.1 storageDevice (read-only)

`IMediumAttachment IStorageDeviceChangedEvent::storageDevice`

Storage device that is subject to change.

6.207.1.2 removed (read-only)

`boolean IStorageDeviceChangedEvent::removed`

Flag whether the device was removed or added to the VM.

6.207.1.3 silent (read-only)

`boolean IStorageDeviceChangedEvent::silent`

Flag whether the guest should be notified about the change.

6.208 IStringArray

When you need to return an array of strings asynchronously (under a progress) you cannot use by-value out parameter `type="wstring" safearray="yes" dir="out"`, hence this wrapper.

6.208.1 Attributes

6.208.1.1 values (read-only)

`wstring IStringArray::values[]`

6.209 IStringFormValue (IFormValue)

Note: This interface extends IFormValue and therefore supports all its methods and attributes as well.

6.209.1 Attributes

6.209.1.1 multiline (read-only)

`boolean IStringFormValue::multiline`

6.209.1.2 clipboardString (read-only)

```
wstring IStringFormValue::clipboardString
```

Intended for cases when a read-only string value is used to display information and different string is to be used when copying to the clipboard.

6.209.2 getString

```
wstring IStringFormValue::getString()
```

6.209.3 setString

```
IProgress IStringFormValue::setString(
          [in] wstring text)
```

text

6.210 ISystemProperties

The ISystemProperties interface represents global properties of the given VirtualBox installation.

These properties define limits and default values for various attributes and parameters. Most of the properties are read-only, but some can be changed by a user.

6.210.1 Attributes

6.210.1.1 platform (read-only)

```
IPlatformProperties ISystemProperties::platform
```

Platform properties of the VirtualBox installation.

6.210.1.2 minGuestRAM (read-only)

```
unsigned long ISystemProperties::minGuestRAM
```

Minimum guest system memory in Megabytes.
See also IPlatformProperties::getMinGuestRAM(), IGuestOSType::recommendedRAM.

6.210.1.3 maxGuestRAM (read-only)

```
unsigned long ISystemProperties::maxGuestRAM
```

Maximum guest system memory in Megabytes.

6.210.1.4 minGuestVRAM (read-only)

```
unsigned long ISystemProperties::minGuestVRAM
```

Minimum guest video memory in Megabytes.

6.210.1.5 maxGuestVRAM (read-only)

```
unsigned long ISystemProperties::maxGuestVRAM
```

Maximum guest video memory in Megabytes.

6.210.1.6 minGuestCPUCount (read-only)

`unsigned long ISystemProperties::minGuestCPUCount`

Minimum CPU count.

6.210.1.7 maxGuestCPUCount (read-only)

`unsigned long ISystemProperties::maxGuestCPUCount`

Maximum CPU count.

6.210.1.8 maxGuestMonitors (read-only)

`unsigned long ISystemProperties::maxGuestMonitors`

Maximum of monitors which could be connected.

6.210.1.9 infoVDSize (read-only)

`long long ISystemProperties::infoVDSize`

Maximum size of a virtual disk image in bytes. Informational value, does not reflect the limits of any virtual disk image format.

6.210.1.10 defaultMachineFolder (read/write)

`wstring ISystemProperties::defaultMachineFolder`

Full path to the default directory used to create new or open existing machines when a machine settings file name contains no path.

Starting with VirtualBox 4.0, by default, this attribute contains the full path of folder named "VirtualBox VMs" in the user's home directory, which depends on the host platform.

When setting this attribute, a full path must be specified. Setting this property to `null` or an empty string or the special value "Machines" (for compatibility reasons) will restore that default value.

If the folder specified herein does not exist, it will be created automatically as needed.

See also: IVirtualBox::createMachine(), IVirtualBox::openMachine()

6.210.1.11 loggingLevel (read/write)

`wstring ISystemProperties::loggingLevel`

Specifies the logging level in current use by VirtualBox.

6.210.1.12 mediumFormats (read-only)

`IMediumFormat ISystemProperties::mediumFormats[]`

List of all medium storage formats supported by this VirtualBox installation.

Keep in mind that the medium format identifier (IMediumFormat::id) used in other API calls like IVirtualBox::createMedium() to refer to a particular medium format is a case-insensitive string. This means that, for example, all of the following strings:

```
"VDI"
"vdi"
"VdI"
```

refer to the same medium format.

Note that the virtual medium framework is backend-based, therefore the list of supported formats depends on what backends are currently installed.

See also: IMediumFormat

6.210.1.13 defaultHardDiskFormat (read/write)

```
wstring ISystemProperties::defaultHardDiskFormat
```

Identifier of the default medium format used by VirtualBox.

The medium format set by this attribute is used by VirtualBox when the medium format was not specified explicitly. One example is IVirtualBox::createMedium() with the empty format argument. A more complex example is implicit creation of differencing media when taking a snapshot of a virtual machine: this operation will try to use a format of the parent medium first and if this format does not support differencing media the default format specified by this argument will be used.

The list of supported medium formats may be obtained by the mediumFormats[] call. Note that the default medium format must have a capability to create differencing media; otherwise operations that create media implicitly may fail unexpectedly.

The initial value of this property is `"VDI"` in the current version of the VirtualBox product, but may change in the future.

> **Note:** Setting this property to `null` or empty string will restore the initial value.

See also: mediumFormats[], IMediumFormat::id, IVirtualBox::createMedium()

6.210.1.14 freeDiskSpaceWarning (read/write)

```
long long ISystemProperties::freeDiskSpaceWarning
```

Issue a warning if the free disk space is below (or in some disk intensive operation is expected to go below) the given size in bytes.

6.210.1.15 freeDiskSpacePercentWarning (read/write)

```
unsigned long ISystemProperties::freeDiskSpacePercentWarning
```

Issue a warning if the free disk space is below (or in some disk intensive operation is expected to go below) the given percentage.

6.210.1.16 freeDiskSpaceError (read/write)

```
long long ISystemProperties::freeDiskSpaceError
```

Issue an error if the free disk space is below (or in some disk intensive operation is expected to go below) the given size in bytes.

6.210.1.17 freeDiskSpacePercentError (read/write)

```
unsigned long ISystemProperties::freeDiskSpacePercentError
```

Issue an error if the free disk space is below (or in some disk intensive operation is expected to go below) the given percentage.

6.210.1.18 VRDEAuthLibrary (read/write)

`wstring ISystemProperties::VRDEAuthLibrary`

Library that provides authentication for Remote Desktop clients. The library is used if a virtual machine's authentication type is set to "external" in the VM RemoteDisplay configuration.

The system library extension (".DLL" or ".so") must be omitted. A full path can be specified; if not, then the library must reside on the system's default library path.

The default value of this property is `"VBoxAuth"`. There is a library of that name in one of the default VirtualBox library directories.

For details about VirtualBox authentication libraries and how to implement them, please refer to the VirtualBox manual.

> **Note:** Setting this property to `null` or empty string will restore the initial value.

6.210.1.19 webServiceAuthLibrary (read/write)

`wstring ISystemProperties::webServiceAuthLibrary`

Library that provides authentication for webservice clients. The library is used if a virtual machine's authentication type is set to "external" in the VM RemoteDisplay configuration and will be called from within the IWebsessionManager::logon() implementation.

As opposed to VRDEAuthLibrary, there is no per-VM setting for this, as the webservice is a global resource (if it is running). Only for this setting (for the webservice), setting this value to a literal `"null"` string disables authentication, meaning that IWebsessionManager::logon() will always succeed, no matter what user name and password are supplied.

The initial value of this property is `"VBoxAuth"`, meaning that the webservice will use the same authentication library that is used by default for VRDE (again, see VRDEAuthLibrary). The format and calling convention of authentication libraries is the same for the webservice as it is for VRDE.

> **Note:** Setting this property to `null` or empty string will restore the initial value.

6.210.1.20 defaultVRDEExtPack (read/write)

`wstring ISystemProperties::defaultVRDEExtPack`

The name of the extension pack providing the default VRDE.

This attribute is for choosing between multiple extension packs providing VRDE. If only one is installed, it will automatically be the default one. The attribute value can be empty if no VRDE extension pack is installed.

For details about VirtualBox Remote Desktop Extension and how to implement one, please refer to the VirtualBox SDK.

6.210.1.21 defaultCryptoExtPack (read/write)

`wstring ISystemProperties::defaultCryptoExtPack`

The name of the extension pack providing the default cryptographic support for full VM encryption.

This attribute is for choosing between multiple extension packs providing cryptographic support. If only one is installed, it will automatically be the default one. The attribute value can be empty if no cryptographic extension pack is installed.

6.210.1.22 logHistoryCount (read/write)

```
unsigned long ISystemProperties::logHistoryCount
```

This value specifies how many old release log files are kept.

6.210.1.23 defaultAudioDriver (read-only)

```
AudioDriverType ISystemProperties::defaultAudioDriver
```

This value hold the default audio driver for the current system.

6.210.1.24 autostartDatabasePath (read/write)

```
wstring ISystemProperties::autostartDatabasePath
```

The path to the autostart database. Depending on the host this might be a filesystem path or something else.

6.210.1.25 defaultAdditionsISO (read/write)

```
wstring ISystemProperties::defaultAdditionsISO
```

The path to the default Guest Additions ISO image. Can be empty if the location is not known in this installation.

6.210.1.26 defaultFrontend (read/write)

```
wstring ISystemProperties::defaultFrontend
```

Selects which VM frontend should be used by default when launching a VM through the IMachine::launchVMProcess() method. Empty or `null` strings do not define a particular default, it is up to IMachine::launchVMProcess() to select one. See the description of IMachine::launchVMProcess() for the valid frontend types.

This global setting is overridden by the per-VM attribute IMachine::defaultFrontend or a frontend type passed to IMachine::launchVMProcess().

6.210.1.27 screenShotFormats (read-only)

```
BitmapFormat ISystemProperties::screenShotFormats[]
```

Supported bitmap formats which can be used with takeScreenShot and takeScreenShotToArray methods.

6.210.1.28 proxyMode (read/write)

```
ProxyMode ISystemProperties::proxyMode
```

The proxy mode setting: System, NoProxy or Manual.

6.210.1.29 proxyURL (read/write)

```
wstring ISystemProperties::proxyURL
```

Proxy server URL for the Manual proxy mode.
The format is: [{type}“://“][{userid}[:{password}]@]{server}[“:“{port}]
Valid types are: http (default), https, socks4, socks4a, socks5, socks5h and direct. Please note that these are proxy types defining how the proxy operates rather than how to proxy any similarly named protocol (i.e. don’t confuse a http-proxy with proxying the http protocol, as a http-proxy usually can proxy https and other protocols too).
The port number defaults to 80 for http, 443 for https and 1080 for the socks ones.

> **Note:** The password is currently stored as plain text! Use the System mode if you consider the proxy password to be sensitive.

An empty string will cause the behavior to be identical to System. For compatibility with libproxy, an URL starting with “direct://“ will cause NoProxy behavior.

6.210.1.30 supportedPlatformArchitectures (read-only)

```
PlatformArchitecture ISystemProperties::supportedPlatformArchitectures[]
```

Returns an array of officially supported values for enum PlatformArchitecture, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.31 supportedClipboardModes (read-only)

```
ClipboardMode ISystemProperties::supportedClipboardModes[]
```

Returns an array of officially supported values for enum ClipboardMode, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.32 supportedDnDModes (read-only)

```
DnDMode ISystemProperties::supportedDnDModes[]
```

Returns an array of officially supported values for enum DnDMode, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.33 supportedPointingHIDTypes (read-only)

```
PointingHIDType ISystemProperties::supportedPointingHIDTypes[]
```

Returns an array of officially supported values for enum PointingHIDType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.34 supportedKeyboardHIDTypes (read-only)

```
KeyboardHIDType ISystemProperties::supportedKeyboardHIDTypes[]
```

Returns an array of officially supported values for enum KeyboardHIDType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.35 supportedVFSTypes (read-only)

`VFSType ISystemProperties::supportedVFSTypes[]`

Returns an array of officially supported values for enum VFSType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.36 supportedImportOptions (read-only)

`ImportOptions ISystemProperties::supportedImportOptions[]`

Returns an array of officially supported values for enum ImportOptions, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.37 supportedExportOptions (read-only)

`ExportOptions ISystemProperties::supportedExportOptions[]`

Returns an array of officially supported values for enum ExportOptions, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.38 supportedGraphicsFeatures (read-only)

`GraphicsFeature ISystemProperties::supportedGraphicsFeatures[]`

Returns an array of officially supported values for enum GraphicsFeature on the host side. For querying supported features of a specific graphics controller, use IPlatformProperties::getSupportedGfxFeaturesForType().

6.210.1.39 supportedRecordingFeatures (read-only)

`RecordingFeature ISystemProperties::supportedRecordingFeatures[]`

Returns an array of officially supported values for enum RecordingFeature, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.40 supportedRecordingAudioCodecs (read-only)

`RecordingAudioCodec ISystemProperties::supportedRecordingAudioCodecs[]`

Returns an array of officially supported values for enum RecordingAudioCodec, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.41 supportedRecordingVideoCodecs (read-only)

`RecordingVideoCodec ISystemProperties::supportedRecordingVideoCodecs[]`

Returns an array of officially supported values for enum RecordingVideoCodec, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.42 supportedRecordingVSModes (read-only)

`RecordingVideoScalingMode ISystemProperties::supportedRecordingVSModes[]`

Returns an array of officially supported values for enum RecordingVideoScalingMode, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.43 supportedRecordingARCModes (read-only)

`RecordingRateControlMode ISystemProperties::supportedRecordingARCModes[]`

Returns an array of officially supported audio codec values for enum RecordingRateControlMode, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.44 supportedRecordingVRCModes (read-only)

`RecordingRateControlMode ISystemProperties::supportedRecordingVRCModes[]`

Returns an array of officially supported video codec values for enum RecordingRateControlMode, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.45 supportedCloneOptions (read-only)

`CloneOptions ISystemProperties::supportedCloneOptions[]`

Returns an array of officially supported values for enum CloneOptions, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.46 supportedAutostopTypes (read-only)

`AutostopType ISystemProperties::supportedAutostopTypes[]`

Returns an array of officially supported values for enum AutostopType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.47 supportedVMProcPriorities (read-only)

`VMProcPriority ISystemProperties::supportedVMProcPriorities[]`

Returns an array of officially supported values for enum VMProcPriority, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.48 supportedNetworkAttachmentTypes (read-only)

`NetworkAttachmentType ISystemProperties::supportedNetworkAttachmentTypes[]`

Returns an array of officially supported values for enum NetworkAttachmentType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.49 supportedPortModes (read-only)

`PortMode ISystemProperties::supportedPortModes[]`

Returns an array of officially supported values for enum PortMode, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.50 supportedAudioDriverTypes (read-only)

`AudioDriverType ISystemProperties::supportedAudioDriverTypes[]`

Returns an array of officially supported values for enum AudioDriverType, in the sense of what is e.g. worth offering in the VirtualBox GUI.

6.210.1.51 languageId (read/write)

```
wstring ISystemProperties::languageId
```

The API language ID used to translate messages to client.

6.210.2 getCPUProfiles

```
ICPUProfile[] ISystemProperties::getCPUProfiles(
            [in] CPUArchitecture architecture,
            [in] wstring namePattern)
```

architecture The architecture to get profiles for. Required.

namePattern Name pattern. Simple wildcard matching using asterisk (*) and question mark (?).

Returns CPU profiles matching the given criteria.

6.210.3 getDefaultIoCacheSettingForStorageController

```
boolean ISystemProperties::getDefaultIoCacheSettingForStorageController(
            [in] StorageControllerType controllerType)
```

controllerType The storage controller type to get the setting for.

Returns the default I/O cache setting for the given storage controller

6.210.4 getExecutionEnginesForVmCpuArchitecture

```
VMExecutionEngine[] ISystemProperties::getExecutionEnginesForVmCpuArchitecture(
            [in] CPUArchitecture cpuArchitecture)
```

cpuArchitecture The CPU architecture type to get the values for.

Returns the possible VMExecutionEngine enum values for a given VM CPU architecture.

6.211 IToken

The IToken interface represents a token passed to an API client, which triggers cleanup actions when it is explicitly released by calling the abandon() method (preferred, as it is accurately defined when the release happens), or when the object reference count drops to 0. The latter way is implicitly used when an API client crashes, however the discovery that there was a crash can take rather long, depending on the platform (COM needs 6 minutes). So better don't rely on the crash behavior too much.

6.211.1 abandon

```
void IToken::abandon()
```

Releases this token. Cannot be undone in any way, and makes the token object unusable (even the dummy() method will return an error), ready for releasing. It is a more defined way than just letting the reference count drop to 0, because the latter (depending on the platform) can trigger asynchronous cleanup activity.

6.211.2 dummy

```
void IToken::dummy()
```

Purely a NOOP. Useful when using proxy type API bindings (e.g. the webservice) which manage objects on behalf of the actual client, using an object reference expiration time based garbage collector.

6.212 ITrustedPlatformModule

The ITrustedPlatformModule interface represents the settings of the virtual machine's trusted platform module. This is used only in the IMachine::trustedPlatformModule attribute.

6.212.1 Attributes

6.212.1.1 type (read/write)

```
TpmType ITrustedPlatformModule::type
```

Type of TPM configured for the virtual machine.

6.212.1.2 location (read/write)

```
wstring ITrustedPlatformModule::location
```

Location of the TPM. This is only used for the TpmType_Swtpm type of TPM where the location denotes a **hostname:port** where to connect to.

6.213 IUSBController

6.213.1 Attributes

6.213.1.1 name (read/write)

```
wstring IUSBController::name
```

The USB Controller name.

6.213.1.2 type (read/write)

```
USBControllerType IUSBController::type
```

The USB Controller type.

6.213.1.3 USBStandard (read-only)

```
unsigned short IUSBController::USBStandard
```

USB standard version which the controller implements. This is a BCD which means that the major version is in the high byte and minor version is in the low byte.

6.214 IUSBControllerChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a property of the virtual USB controllers changes. Interested callees should use IUSBController methods and attributes to find out what has changed.

6.214.1 Attributes

6.214.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IUSBControllerChangedEvent::midlDoesNotLikeEmptyInterfaces
```

6.215 IUSBDevice

The IUSBDevice interface represents a virtual USB device attached to the virtual machine.

A collection of objects implementing this interface is stored in the IConsole::USBDevices[] attribute which lists all USB devices attached to a running virtual machine's USB controller.

6.215.1 Attributes

6.215.1.1 id (read-only)

```
uuid IUSBDevice::id
```

Unique USB device ID. This ID is built from #vendorId, #productId, #revision and #serialNumber.

6.215.1.2 vendorId (read-only)

```
unsigned short IUSBDevice::vendorId
```

Vendor ID.

6.215.1.3 productId (read-only)

```
unsigned short IUSBDevice::productId
```

Product ID.

6.215.1.4 revision (read-only)

```
unsigned short IUSBDevice::revision
```

Product revision number. This is a packed BCD represented as unsigned short. The high byte is the integer part and the low byte is the decimal.

6.215.1.5 manufacturer (read-only)

```
wstring IUSBDevice::manufacturer
```

Manufacturer string.

6.215.1.6 product (read-only)

```
wstring IUSBDevice::product
```

Product string.

6.215.1.7 serialNumber (read-only)

```
wstring IUSBDevice::serialNumber
```

Serial number string.

6.215.1.8 address (read-only)

```
wstring IUSBDevice::address
```

Host-specific address of the device, uniquely identifying a physically connected device in the system. Note that the address of a USB device may change across device re-plugs and host suspend/resume cycles or reboots.

6.215.1.9 port (read-only)

```
unsigned short IUSBDevice::port
```

Host USB port number on the hub the device is physically connected to.

6.215.1.10 portPath (read-only)

```
wstring IUSBDevice::portPath
```

Host-specific identifier of the port (including hub) the USB device is physically connected to. Note that hubs may be dynamically added and removed, and that hub enumeration may not be consistent across host reboots.

6.215.1.11 version (read-only)

```
unsigned short IUSBDevice::version
```

The major USB version of the device - 1, 2 or 3.

6.215.1.12 speed (read-only)

```
USBConnectionSpeed IUSBDevice::speed
```

The speed at which the device is currently communicating.

6.215.1.13 remote (read-only)

```
boolean IUSBDevice::remote
```

Whether the device is physically connected to a remote VRDE client or to a local host machine.

6.215.1.14 deviceInfo (read-only)

```
wstring IUSBDevice::deviceInfo[]
```

Array of device attributes as single strings.

So far the following are used: 0: The manufacturer string, if the device doesn't expose the ID one is taken from an internal database or an empty string if none is found. 1: The product string, if the device doesn't expose the ID one is taken from an internal database or an empty string if none is found.

6.215.1.15 backend (read-only)

```
wstring IUSBDevice::backend
```

The backend which will be used to communicate with this device.

6.216 IUSBDeviceFilter

The IUSBDeviceFilter interface represents a USB device filter used to perform actions on one or more USB devices.

USB device filters are used by running virtual machines to automatically capture selected USB devices once they are physically attached to the host computer.

A USB device is matched to a specific device filter if and only if all attributes of the device filter match the corresponding attributes of the device (that is, attributes are joined together using a logical AND operation). On the other hand filter attributes in separate USB device filters are applied as a logical OR operation. So if one would like to create a match such as "this vendor id OR this product id", then one would need to create two filters and specify "any match" (see below) for the other unused attributes.

All filter attributes used for matching are strings. The following filtering expressions are currently supported:

- *Interval filters*. Used to specify valid intervals for integer device attributes (Vendor ID, Product ID, Revision and Port). The format of the string is:

  `int:((m)|([m]-[n]))(,(m)|([m]-[n]))*`

  where `m` and `n` are integer numbers, either in octal (starting with `0`), hexadecimal (starting with `0x`) or decimal (otherwise), so that `m < n`. If `m` is omitted before a dash (`-`), the minimum possible integer is assumed; if `n` is omitted after a dash, the maximum possible integer is assumed.

- *Boolean filters*. Used to specify acceptable values for boolean device attributes. The format of the string is:

  `true|false|yes|no|0|1`

- *Exact match*. Used to specify a single value for the given device attribute. Any string that doesn't start with `int:` represents an exact match. String device attributes are compared to this string in a case-sensitive manner. Integer attributes are first converted to a string (see individual filter attributes) and then compared in a case-insensitive manner.

- *Any match*. Any value of the corresponding device attribute will match the given filter. An empty or `null` string is used to construct this type of filtering expressions.

> **Note:** On Windows hosts, interval filters are not currently available. Also all string filter attributes (manufacturer, product, serialNumber) are ignored, so they behave as *any match* no matter what string expression is specified.

See also: IUSBDeviceFilters::deviceFilters[], IHostUSBDeviceFilter

6.216.1 Attributes

6.216.1.1 name (read/write)

`wstring IUSBDeviceFilter::name`

Visible name for this filter. This name is used to visually distinguish one filter from another, so it can neither be `null` nor an empty string.

6.216.1.2 active (read/write)

`boolean IUSBDeviceFilter::active`

Whether this filter active or has been temporarily disabled.

6.216.1.3 vendorId (read/write)

`wstring IUSBDeviceFilter::vendorId`

Vendor ID filter. The string representation for the *exact matching* has the form `XXXX`, where `X` is the hex digit (including leading zeroes).

6.216.1.4 productId (read/write)

`wstring IUSBDeviceFilter::productId`

Product ID filter. The string representation for the *exact matching* has the form `XXXX`, where `X` is the hex digit (including leading zeroes).

6.216.1.5 revision (read/write)

`wstring IUSBDeviceFilter::revision`

Product revision number filter. The string representation for the *exact matching* has the form `IIFF`, where `I` is the decimal digit of the integer part of the revision, and `F` is the decimal digit of its fractional part (including leading and trailing zeros). Note that for interval filters, it's best to use the hexadecimal form, because the revision is stored as a 16 bit packed BCD value; so the expression `int:0x0100-0x0199` will match any revision from `1.0` to `1.99`.

6.216.1.6 manufacturer (read/write)

`wstring IUSBDeviceFilter::manufacturer`

Manufacturer filter.

6.216.1.7 product (read/write)

`wstring IUSBDeviceFilter::product`

Product filter.

6.216.1.8 serialNumber (read/write)

`wstring IUSBDeviceFilter::serialNumber`

Serial number filter.

6.216.1.9 port (read/write)

```
wstring IUSBDeviceFilter::port
```

Host USB port filter.

6.216.1.10 remote (read/write)

```
wstring IUSBDeviceFilter::remote
```

Remote state filter.

> **Note:** This filter makes sense only for machine USB filters, i.e. it is ignored by IHostUSBDeviceFilter objects.

6.216.1.11 maskedInterfaces (read/write)

```
unsigned long IUSBDeviceFilter::maskedInterfaces
```

This is an advanced option for hiding one or more USB interfaces from the guest. The value is a bit mask where the bits that are set means the corresponding USB interface should be hidden, masked off if you like. This feature only works on Linux hosts.

6.217 IUSBDeviceFilters

6.217.1 Attributes

6.217.1.1 deviceFilters (read-only)

```
IUSBDeviceFilter IUSBDeviceFilters::deviceFilters[]
```

List of USB device filters associated with the machine.

If the machine is currently running, these filters are activated every time a new (supported) USB device is attached to the host computer that was not ignored by global filters (IHost::USBDeviceFilters[]).

These filters are also activated when the machine is powered up. They are run against a list of all currently available USB devices (in states Available, Busy, Held) that were not previously ignored by global filters.

If at least one filter matches the USB device in question, this device is automatically captured (attached to) the virtual USB controller of this machine.

See also: IUSBDeviceFilter, IUSBController

6.217.2 createDeviceFilter

```
IUSBDeviceFilter IUSBDeviceFilters::createDeviceFilter(
          [in] wstring name)
```

name Filter name. See IUSBDeviceFilter::name for more info.

Creates a new USB device filter. All attributes except the filter name are set to empty (any match), *active* is `false` (the filter is not active).

The created filter can then be added to the list of filters using insertDeviceFilter().

See also: deviceFilters[]

If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE`: The virtual machine is not mutable.

6.217.3 insertDeviceFilter

```
void IUSBDeviceFilters::insertDeviceFilter(
           [in] unsigned long position,
           [in] IUSBDeviceFilter filter)
```

position Position to insert the filter to.

filter USB device filter to insert.

Inserts the given USB device to the specified position in the list of filters.
Positions are numbered starting from `0`. If the specified position is equal to or greater than the number of elements in the list, the filter is added to the end of the collection.

> **Note:** Duplicates are not allowed, so an attempt to insert a filter that is already in the collection, will return an error.

See also: deviceFilters[]
If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` Virtual machine is not mutable.
- `E_INVALIDARG:` USB device filter not created within this VirtualBox instance.
- `VBOX_E_INVALID_OBJECT_STATE:` USB device filter already in list.

6.217.4 removeDeviceFilter

```
IUSBDeviceFilter IUSBDeviceFilters::removeDeviceFilter(
           [in] unsigned long position)
```

position Position to remove the filter from.

Removes a USB device filter from the specified position in the list of filters.
Positions are numbered starting from `0`. Specifying a position equal to or greater than the number of elements in the list will produce an error.
See also: deviceFilters[]
If this method fails, the following error codes may be reported:

- `VBOX_E_INVALID_VM_STATE:` Virtual machine is not mutable.
- `E_INVALIDARG:` USB device filter list empty or invalid `position`.

6.218 IUSBDeviceStateChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a USB device is attached to or detached from the virtual USB controller.
This notification is sent as a result of the indirect request to attach the device because it matches one of the machine USB filters, or as a result of the direct request issued by IConsole::attachUSBDevice() or IConsole::detachUSBDevice().
This notification is sent in case of both a succeeded and a failed request completion. When the request succeeds, the `error` parameter is `null`, and the given device has been already added to (when `attached` is `true`) or removed from (when `attached` is `false`) the collection represented by IConsole::USBDevices[]. On failure, the collection doesn't change and the `error` parameter represents the error message describing the failure.

6.218.1 Attributes

6.218.1.1 device (read-only)

`IUSBDevice IUSBDeviceStateChangedEvent::device`

Device that is subject to state change.

6.218.1.2 attached (read-only)

`boolean IUSBDeviceStateChangedEvent::attached`

`true` if the device was attached and `false` otherwise.

6.218.1.3 error (read-only)

`IVirtualBoxErrorInfo IUSBDeviceStateChangedEvent::error`

`null` on success or an error message object on failure.

6.219 IUSBProxyBackend

The USBProxyBackend interface represents a source for USB devices available to the host for attaching to the VM.

6.219.1 Attributes

6.219.1.1 name (read-only)

`wstring IUSBProxyBackend::name`

The unique name of the proxy backend.

6.219.1.2 type (read-only)

`wstring IUSBProxyBackend::type`

The type of the backend.

6.220 IUefiVariableStore

The IUefiVariableStore interface allows inspecting and manipulating the content of an existing UEFI variable store in a NVRAM file. This is used only in the INvramStore::uefiVariableStore attribute.

6.220.1 Attributes

6.220.1.1 secureBootEnabled (read/write)

`boolean IUefiVariableStore::secureBootEnabled`

Flag whether secure boot is currently enabled for the VM.

6.220.2 addKek

```
void IUefiVariableStore::addKek(
            [in] octet keyEncryptionKey[],
            [in] uuid owner,
            [in] SignatureType signatureType)
```

keyEncryptionKey The Key Encryption Key (KEK) to add.

owner UUID of the KEK owner.

signatureType Type of the signature.

Convenience method to add a new Key Encryption Key (KEK) for Secure Boot.

6.220.3 addSignatureToDb

```
void IUefiVariableStore::addSignatureToDb(
            [in] octet signature[],
            [in] uuid owner,
            [in] SignatureType signatureType)
```

signature The signature to add.

owner UUID of the signature owner.

signatureType Type of the signature.

Convenience method to add a new entry to the signature database.

6.220.4 addSignatureToDbx

```
void IUefiVariableStore::addSignatureToDbx(
            [in] octet signature[],
            [in] uuid owner,
            [in] SignatureType signatureType)
```

signature The signature to add.

owner UUID of the signature owner.

signatureType Type of the signature.

Convenience method to add a new entry to the forbidden signature database.

6.220.5 addSignatureToMok

```
void IUefiVariableStore::addSignatureToMok(
            [in] octet signature[],
            [in] uuid owner,
            [in] SignatureType signatureType)
```

signature The signature to add.

owner UUID of the signature owner.

signatureType Type of the signature.

Convenience method to add a new entry to the MOK (Machine Owner Key) signature database.

6.220.6 addVariable

```
void IUefiVariableStore::addVariable(
            [in] wstring name,
            [in] uuid owner,
            [in] UefiVariableAttributes attributes[],
            [in] octet data[])
```

name Name of the variable.

owner UUID of the variable owner.

attributes Attributes of the variable.

data The variable data.

Adds a new variable to the non volatile storage area.

6.220.7 changeVariable

```
void IUefiVariableStore::changeVariable(
            [in] wstring name,
            [in] octet data[])
```

name Name of the variable.

data The new variable data.

Changes the data of the given variable.

6.220.8 deleteVariable

```
void IUefiVariableStore::deleteVariable(
            [in] wstring name,
            [in] uuid owner)
```

name Name of the variable.

owner UUID of the variable owner.

Deletes the given variable from the non volatile storage area.

6.220.9 enrollDefaultMsSignatures

```
void IUefiVariableStore::enrollDefaultMsSignatures()
```

Convenience method to enroll the standard Microsoft KEK and signatures in the signature databases.

6.220.10 enrollOraclePlatformKey

```
void IUefiVariableStore::enrollOraclePlatformKey()
```

Enroll the default platform key from Oracle for enabling Secure Boot.

6.220.11 enrollPlatformKey

```
void IUefiVariableStore::enrollPlatformKey(
            [in] octet platformKey[],
            [in] uuid owner)
```

platformKey The platform key (PK) to enroll.

owner UUID of the PK owner.

Convenience method to enroll a new platform key (PK) for enabling Secure Boot.

6.220.12 queryVariableByName

```
void IUefiVariableStore::queryVariableByName(
            [in] wstring name,
            [out] uuid owner,
            [out] UefiVariableAttributes attributes[],
            [out] octet data[])
```

name Name of the variable to look for.

owner UUID of the variable owner returned on success.

attributes Attributes of the variable.

data The variable data returned on success.

Queries the variable content variable by the given name.

6.220.13 queryVariables

```
void IUefiVariableStore::queryVariables(
            [out] wstring names[],
            [out] uuid owners[])
```

names The variable names returned on success.

owners UUID of the variable owners returned on success.

Queries all variables in the non volatile storage and returns their names.

6.221 IUnattended

The IUnattended interface represents the pipeline for preparing the Guest OS for fully automated install.

The typical workflow is:

1. Call IVirtualBox::createUnattendedInstaller() to create the object
2. Set isoPath and call detectIsoOS()
3. Create, configure and register a machine according to detectedOSTypeId and the other detectedOS* attributes.
4. Set machine to the new IMachine instance.
5. Set the other IUnattended attributes as desired.

6. Call prepare() for the object to check the attribute values and create an internal installer instance.
7. Call constructMedia() to create additional media files (ISO/floppy) needed.
8. Call reconfigureVM() to reconfigure the VM with the installation ISO, additional media files and whatnot
9. Optionally call done() to destroy the internal installer and allow restarting from the second step.

Note! Step two is currently not implemented.

6.221.1 Attributes

6.221.1.1 isoPath (read/write)

```
wstring IUnattended::isoPath
```

Guest operating system ISO image

6.221.1.2 machine (read/write)

```
IMachine IUnattended::machine
```

The associated machine object.
This must be set before prepare() is called. The VM must be registered.

6.221.1.3 user (read/write)

```
wstring IUnattended::user
```

Assigns an user login name.

6.221.1.4 userPassword (read/write)

```
wstring IUnattended::userPassword
```

Assigns a password to the user.

6.221.1.5 adminPassword (read/write)

```
wstring IUnattended::adminPassword
```

Assigns a password to the Administrator / 'root' accounts.
If no Administrator / 'root' password is being set explicitly, the password set via userPassword will be used instead.

6.221.1.6 fullUserName (read/write)

```
wstring IUnattended::fullUserName
```

The full name of the user. This is optional and defaults to user. Please note that not all guests picks up this attribute.

6.221.1.7 productKey (read/write)

```
wstring IUnattended::productKey
```

Any key which is used as authorization of access to install genuine OS

6.221.1.8 additionsIsoPath (read/write)

```
wstring IUnattended::additionsIsoPath
```

Guest Additions ISO image path. This defaults to ISystemProperties::defaultAdditionsISO when the Unattended object is instantiated.
This property is ignored when installGuestAdditions is false.

6.221.1.9 installGuestAdditions (read/write)

```
boolean IUnattended::installGuestAdditions
```

Indicates whether the Guest Additions should be installed or not.
Setting this to false does not affect guest additions shipped with Linux distributions, only the installation of additions pointed to by additionsIsoPath.

6.221.1.10 productKeyRequired (read-only)

```
boolean IUnattended::productKeyRequired
```

Indicates whether the product key is required for the installation.

6.221.1.11 validationKitIsoPath (read/write)

```
wstring IUnattended::validationKitIsoPath
```

VirtualBox ValidationKit ISO image path. This is used when installTestExecService is set to true.

6.221.1.12 installTestExecService (read/write)

```
boolean IUnattended::installTestExecService
```

Indicates whether the test execution service (TXS) from the VBox ValidationKit should be installed.
The TXS binary will be taken from the ISO indicated by validationKitIsoPath.

6.221.1.13 userPayloadIsoPath (read/write)

```
wstring IUnattended::userPayloadIsoPath
```

User Payload ISO image path. This is used when installUserPayload is set to true.

6.221.1.14 installUserPayload (read/write)

```
boolean IUnattended::installUserPayload
```

Indicates whether the User Payload should be installed or not.
The User Payload will be taken from the ISO indicated by userPayloadIsoPath.

6.221.1.15 timeZone (read/write)

`wstring IUnattended::timeZone`

The guest time zone specifier.
This is unfortunately guest OS specific.
Windows XP and earlier takes the index number from this table: https://support.microsoft.com/en-gb/help/973627/microsoft-time-zone-index-values
Windows Vista and later takes the time zone string from this table: https://technet.microsoft.com/en-us/library/cc749073(v=ws.10).aspx
Linux usually takes the TZ string from this table: https://en.wikipedia.org/wiki/List_of_tz_database_time_zones
The default is currently UTC/GMT, but this may change to be same as the host later.
TODO: Investigate automatic mapping between linux and the two windows time zone formats.
TODO: Take default from host (this requires mapping).

6.221.1.16 keyboardLayout (read/write)

`wstring IUnattended::keyboardLayout`

Sets the keyboard layout of the guest, if available.
The 2 letter locale identifier, no codesets or such.
The format is two lower case country letter (ISO 3166-1 alpha-2). For instance 'us', 'de', or 'no'.
The default is taken from the host if possible, with 'us' as fallback.

6.221.1.17 keyboardVariant (read/write)

`wstring IUnattended::keyboardVariant`

Sets the keyboard variant of the guest, if available. Leave empty to use the default variant for the current keyboard layout set.
TODO: Define + document a keyboard variants table (e.g. by using 'localectl list-x11-keymap-variants [layout]' as a base), plus a mapping to Windows-based guests for that.
This depends on the current keyboard layout being set.

6.221.1.18 locale (read/write)

`wstring IUnattended::locale`

The 5 letter locale identifier, no codesets or such.
The format is two lower case language letters (ISO 639-1), underscore ('_'), and two upper case country letters (ISO 3166-1 alpha-2). For instance 'en_US', 'de_DE', or 'ny_NO'.
The default is taken from the host if possible, with 'en_US' as fallback.

6.221.1.19 language (read/write)

`wstring IUnattended::language`

This is more or less a Windows specific setting for choosing the UI language setting of the installer.
The value should be from the list available via detectedOSLanguages. The typical format is {language-code}-{COUNTRY} but windows may also use {16-bit code}:{32-bit code} or insert another component between the language and country codes. We consider the format guest OS specific.
Note that it is crucial that this is correctly specified for Windows installations. If an unsupported value is given the installer will ask for an installation language and wait for user input. Best to leave it to the default value.
The default is the first one from detectedOSLanguages.

6.221.1.20 country (read/write)

```
wstring IUnattended::country
```

The 2 upper case letter country identifier, ISO 3166-1 alpha-2.
This is used for mirrors and such.
The default is taken from the host when possible, falling back on locale.

6.221.1.21 proxy (read/write)

```
wstring IUnattended::proxy
```

Proxy incantation to pass on to the guest OS installer.
This is important to get right if the guest OS installer is of the type that goes online to fetch the packages (e.g. debian-*-netinst.iso) or to fetch updates during the install process.
Format: [schema=]schema://[login[:password]@]proxy[:port][;...]
The default is taken from the host proxy configuration (once implemented).

6.221.1.22 packageSelectionAdjustments (read/write)

```
wstring IUnattended::packageSelectionAdjustments
```

Guest OS specific package selection adjustments.
This is a semicolon separated list of keywords, and later maybe guest OS package specifiers. Currently the 'minimal' is the only recognized value, and this only works with a selection of linux installers.

6.221.1.23 hostname (read/write)

```
wstring IUnattended::hostname
```

The fully qualified guest hostname.
This defaults to machine-name + ".myguest.virtualbox.org", though it may change to the host domain name later.

6.221.1.24 auxiliaryBasePath (read/write)

```
wstring IUnattended::auxiliaryBasePath
```

The path + basename for auxiliary files generated by the unattended installation. This defaults to the VM folder + Unattended + VM UUID.
The files which get generated depend on the OS being installed. When installing Windows there is currently only an auxiliaryBasePath + "floppy.img" being created. But for Linux, a "cdrom.viso" and one or more configuration files are generated.

6.221.1.25 imageIndex (read/write)

```
unsigned long IUnattended::imageIndex
```

The image index on installation CD/DVD used to install. This index should be one of the indices of detectedImageIndices[].
Changing this after ISO detection may cause detectedOSTypeId, detectedOSVersion, detectedOSFlavor, detectedOSLanguages, and detectedOSHints to be updated with values specific to the selected image.
Used only with Windows installation CD/DVD: https://technet.microsoft.com/en-us/library/cc766022%28v=ws.10%29.aspx

6.221.1.26 scriptTemplatePath (read/write)

```
wstring IUnattended::scriptTemplatePath
```

The unattended installation script template file.
The template default is based on the guest OS type and is determined by the internal installer when when prepare() is invoked. Most users will want the defaults.
After prepare() is called, it can be read to see which file is being used.

6.221.1.27 postInstallScriptTemplatePath (read/write)

```
wstring IUnattended::postInstallScriptTemplatePath
```

The post installation (shell/batch) script template file.
The template default is based on the guest OS type and is determined by the internal installer when when prepare() is invoked. Most users will want the defaults.
After prepare() is called, it can be read to see which file is being used.

6.221.1.28 postInstallCommand (read/write)

```
wstring IUnattended::postInstallCommand
```

Custom post installation command.
Exactly what is expected as input here depends on the guest OS installer and the post installation script template (see postInstallScriptTemplatePath). Most users will not need to set this attribute.

6.221.1.29 extraInstallKernelParameters (read/write)

```
wstring IUnattended::extraInstallKernelParameters
```

Extra kernel arguments passed to the install kernel of some guests.
This is currently only picked up by Linux guests. The exact parameters are specific to the guest OS being installed of course.
After prepare() is called, it can be read to see which parameters are being used.

6.221.1.30 detectedOSTypeId (read-only)

```
wstring IUnattended::detectedOSTypeId
```

The detected OS type ID (IGuestOSType::id).
Set by detectIsoOS() or prepare().
Not yet implemented.

6.221.1.31 detectedOSVersion (read-only)

```
wstring IUnattended::detectedOSVersion
```

The detected OS version string.
Set by detectIsoOS() or prepare().
Not yet implemented.

6.221.1.32 detectedOSFlavor (read-only)

```
wstring IUnattended::detectedOSFlavor
```

The detected OS flavor (e.g. server, desktop, etc)
Set by detectIsoOS() or prepare().
Not yet implemented.

6.221.1.33 detectedOSLanguages (read-only)

```
wstring IUnattended::detectedOSLanguages
```

The space separated list of (Windows) installation UI languages we detected (lang.ini). The language specifier format is specific to the guest OS. They are used to set language. Set by detectIsoOS() or prepare(). Partially implemented.

6.221.1.34 detectedOSHints (read-only)

```
wstring IUnattended::detectedOSHints
```

Space separated list of other stuff detected about the OS and the installation ISO. Set by detectIsoOS() or prepare(). Not yet implemented.

6.221.1.35 detectedImageNames (read-only)

```
wstring IUnattended::detectedImageNames[]
```

A list of names of the images detected from install.wim file of a Windows Vista or later ISO. This array is parallel to detectedImageIndices[]. This will be empty for older Windows ISOs and non-Windows ISOs.

6.221.1.36 detectedImageIndices (read-only)

```
unsigned long IUnattended::detectedImageIndices[]
```

A list of image indexes detected from install.wim file of a Windows ISO. This array is parallel to detectedImageNames[]. imageIndex should be set to one of these indices for Vista and later Windows ISOs.

6.221.1.37 isUnattendedInstallSupported (read-only)

```
boolean IUnattended::isUnattendedInstallSupported
```

Checks the detected OS type and version against a set of rules and returns whether unattended install is supported or not. Note that failing detecting OS type from the ISO causes this attribute to be false by default.

6.221.1.38 avoidUpdatesOverNetwork (read/write)

```
boolean IUnattended::avoidUpdatesOverNetwork
```

When set to true installation is configured to abstain from using network to update/get data. Especially useful when network is not available (as in our test boxes).

6.221.2 constructMedia

```
void IUnattended::constructMedia()
```

Constructors the necessary ISO/VISO/Floppy images, with unattended scripts and all necessary bits on them.

6.221.3 detectIsoOS

```
void IUnattended::detectIsoOS()
```

Detects the OS on the ISO given by isoPath and sets detectedOSTypeId, detectedOSVersiondetectedOSFlavor, detectedOSLanguages, detectedOSHints, detectedImageNames[], and detectedImageIndices[],
Not really yet implemented.

6.221.4 done

```
void IUnattended::done()
```

Done - time to start the VM.
This deletes the internal installer instance that prepare() created. Before done() is called, it is not possible to start over again from prepare().

6.221.5 prepare

```
void IUnattended::prepare()
```

Prepare for running the unattended process of installation.
This will perform detectIsoOS() if not yet called on the current isoPath value. It may then may cause detectedOSTypeId, detectedOSVersion, detectedOSFlavor, detectedOSLanguages, and detectedOSHints to be updated with values specific to the image selected by imageIndex if the ISO contains images.
This method will then instantiate the installer based on the detected guest type (see detectedOSTypeId).

6.221.6 reconfigureVM

```
void IUnattended::reconfigureVM()
```

Reconfigures the machine to start the installation.
This involves mounting the ISOs and floppy images created by constructMedia(), attaching new DVD and floppy drives as necessary, and possibly modifying the boot order.

6.222 IUpdateAgent

Abstract parent interface for handling updatable software components.

6.222.1 Attributes

6.222.1.1 name (read-only)

```
wstring IUpdateAgent::name
```

Name of the update component.

6.222.1.2 eventSource (read-only)

```
IEventSource IUpdateAgent::eventSource
```

Event source for update agent events.

6.222.1.3 order (read-only)

```
unsigned long IUpdateAgent::order
```

Order hint the update component needs to run at, in conjunction with other update components.

> **Note:** Lower numbers mean higher priority. 0 means no order or disabled.

6.222.1.4 dependsOn (read-only)

```
wstring IUpdateAgent::dependsOn[]
```

Array of other update component names this component depends on before being able to get installed.

> **Note:** Ordered entries, highest priority first. No dependencies when array is empty. Dependency entries also can contain a minimum version number after the update component name, separated by an "@", e.g. "Guest Additions@7.1.0_BETA2".

6.222.1.5 version (read-only)

```
wstring IUpdateAgent::version
```

Version the update contains.

6.222.1.6 downloadUrl (read-only)

```
wstring IUpdateAgent::downloadUrl
```

Download URL of the update.

6.222.1.7 webUrl (read-only)

```
wstring IUpdateAgent::webUrl
```

Web URL of the update.

6.222.1.8 releaseNotes (read-only)

```
wstring IUpdateAgent::releaseNotes
```

Release notes of the update.

6.222.1.9 enabled (read/write)

```
boolean IUpdateAgent::enabled
```

Enables or disables the update component.

6.222.1.10 hidden (read-only)

```
boolean IUpdateAgent::hidden
```

Whether the update component shall be hidden from the user or not.

6.222.1.11 state (read-only)

```
UpdateState IUpdateAgent::state
```

Returns the current update state.

6.222.1.12 checkFrequency (read/write)

```
unsigned long IUpdateAgent::checkFrequency
```

The update check frequency (in seconds).

6.222.1.13 channel (read/write)

```
UpdateChannel IUpdateAgent::channel
```

Update channel to use for checking for updates.

6.222.1.14 repositoryURL (read/write)

```
wstring IUpdateAgent::repositoryURL
```

Update repository URL to use for retrieving the update.

> **Note:** Only repositories with the https:// URL scheme are allowed.

6.222.1.15 lastCheckDate (read-only)

```
wstring IUpdateAgent::lastCheckDate
```

Date of last update check.

> **Note:** In ISO 8601 format (e.g. 2020-05-11T21:13:39.348416000Z).

6.222.1.16 checkCount (read-only)

```
unsigned long IUpdateAgent::checkCount
```

How many times the update check has happened already.

6.222.1.17 isCheckNeeded (read-only)

```
boolean IUpdateAgent::isCheckNeeded
```

Returns `TRUE` if an update check is needed, or `FALSE` if not.

> **Note:** Compares the system's current date with the last update check date and currently set check frequency.

6.222.1.18 supportedChannels (read-only)

`UpdateChannel IUpdateAgent::supportedChannels[]`

Returns a safe array of all supported update channels this agents offers.

6.222.2 checkFor

`IProgress IUpdateAgent::checkFor()`

Checks for an update.

6.222.3 download

`IProgress IUpdateAgent::download()`

Downloads the update.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: Downloading update not supported.

6.222.4 install

`IProgress IUpdateAgent::install()`

Installs the update.
If this method fails, the following error codes may be reported:

- `VBOX_E_NOT_SUPPORTED`: Installing update not supported.

6.222.5 rollback

`void IUpdateAgent::rollback()`

Rolls back installing the update.

6.223 IUpdateAgentAvailableEvent (IUpdateAgentEvent)

> **Note:** This interface extends IUpdateAgentEvent and therefore supports all its methods and attributes as well.

Notification when an update is available.

6.223.1 Attributes

6.223.1.1 version (read-only)

`wstring IUpdateAgentAvailableEvent::version`

Version of the update.

6.223.1.2 channel (read-only)

`UpdateChannel IUpdateAgentAvailableEvent::channel`

Channel containing the update.

6.223.1.3 severity (read-only)

`UpdateSeverity IUpdateAgentAvailableEvent::severity`

Severity of the update.

6.223.1.4 downloadURL (read-only)

`wstring IUpdateAgentAvailableEvent::downloadURL`

Download URL of the update.

6.223.1.5 webURL (read-only)

`wstring IUpdateAgentAvailableEvent::webURL`

Web URL of the update.

6.223.1.6 releaseNotes (read-only)

`wstring IUpdateAgentAvailableEvent::releaseNotes`

Release notes of the update.

6.224 IUpdateAgentErrorEvent (IUpdateAgentEvent)

> **Note:** This interface extends IUpdateAgentEvent and therefore supports all its methods and attributes as well.

Notification when an update agent error occurred.

6.224.1 Attributes

6.224.1.1 msg (read-only)

`wstring IUpdateAgentErrorEvent::msg`

Error message in human readable format.

6.224.1.2 rcError (read-only)

`long IUpdateAgentErrorEvent::rcError`

IPRT-style error code.

6.225 IUpdateAgentEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Abstract base interface for update agent events.

6.225.1 Attributes

6.225.1.1 agent (read-only)

`IUpdateAgent IUpdateAgentEvent::agent`

Update agent this event belongs to.

6.226 IUpdateAgentSettingsChangedEvent (IUpdateAgentEvent)

> **Note:** This interface extends IUpdateAgentEvent and therefore supports all its methods and attributes as well.

Notification when update agent settings have been changed.

6.226.1 Attributes

6.226.1.1 attributeHint (read-only)

`wstring IUpdateAgentSettingsChangedEvent::attributeHint`

6.227 IUpdateAgentStateChangedEvent (IUpdateAgentEvent)

> **Note:** This interface extends IUpdateAgentEvent and therefore supports all its methods and attributes as well.

Notification when an update agent state has been changed.

6.227.1 Attributes

6.227.1.1 state (read-only)

`UpdateState IUpdateAgentStateChangedEvent::state`

New update agent state.

6.228 IVBoxSVCAvailabilityChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when VBoxSVC becomes unavailable (due to a crash or similar unexpected circumstances) or available again.

6.228.1 Attributes

6.228.1.1 available (read-only)

```
boolean IVBoxSVCAvailabilityChangedEvent::available
```

Whether VBoxSVC is available now.

6.229 IVBoxSVCRegistration

Note: This interface is not supported in the web service.

Implemented by the VirtualBox class factory and registered with VBoxSDS so it can retrieve IVirtualBox on behalf of other VBoxSVCs.

6.229.1 getVirtualBox

```
$unknown IVBoxSVCRegistration::getVirtualBox()
```

Gets an IUnknown interface to the VirtualBox object in the VBoxSVC process.

6.230 IVFSExplorer

The VFSExplorer interface unifies access to different file system types. This includes local file systems as well remote file systems like S3. For a list of supported types see VFSType. An instance of this is returned by IAppliance::createVFSExplorer().

6.230.1 Attributes

6.230.1.1 path (read-only)

```
wstring IVFSExplorer::path
```

Returns the current path in the virtual file system.

6.230.1.2 type (read-only)

```
VFSType IVFSExplorer::type
```

Returns the file system type which is currently in use.

6.230.2 cd

```
IProgress IVFSExplorer::cd(
            [in] wstring dir)
```

dir The name of the directory to go in.

Change the current directory level.

6.230.3 cdUp

```
IProgress IVFSExplorer::cdUp()
```

Go one directory upwards from the current directory level.

6.230.4 entryList

```
void IVFSExplorer::entryList(
           [out] wstring names[],
           [out] unsigned long types[],
           [out] long long sizes[],
           [out] unsigned long modes[])
```

names The list of names for the entries.

types The list of types for the entries. FsObjType

sizes The list of sizes (in bytes) for the entries.

modes The list of file modes (in octal form) for the entries.

Returns a list of files/directories after a call to update(). The user is responsible for keeping this internal list up do date.

6.230.5 exists

```
wstring[] IVFSExplorer::exists(
           [in] wstring names[])
```

names The names to check.

Checks if the given file list exists in the current directory level.

6.230.6 remove

```
IProgress IVFSExplorer::remove(
           [in] wstring names[])
```

names The names to remove.

Deletes the given files in the current directory level.

6.230.7 update

```
IProgress IVFSExplorer::update()
```

Updates the internal list of files/directories from the current directory level. Use entryList() to get the full list after a call to this method.

6.231 IVRDEServer

6.231.1 Attributes

6.231.1.1 enabled (read/write)

```
boolean IVRDEServer::enabled
```

Flag if VRDE server is enabled.

6.231.1.2 authType (read/write)

```
AuthType IVRDEServer::authType
```

VRDE authentication method.

6.231.1.3 authTimeout (read/write)

```
unsigned long IVRDEServer::authTimeout
```

Timeout for guest authentication. Milliseconds.

6.231.1.4 allowMultiConnection (read/write)

```
boolean IVRDEServer::allowMultiConnection
```

Flag whether multiple simultaneous connections to the VM are permitted. Note that this will be replaced by a more powerful mechanism in the future.

6.231.1.5 reuseSingleConnection (read/write)

```
boolean IVRDEServer::reuseSingleConnection
```

Flag whether the existing connection must be dropped and a new connection must be established by the VRDE server, when a new client connects in single connection mode.

6.231.1.6 VRDEExtPack (read/write)

```
wstring IVRDEServer::VRDEExtPack
```

The name of Extension Pack providing VRDE for this VM. Overrides ISystemProperties::defaultVRDEExtPack. Querying this attribute returns the effective value directly, that means if the per-VM setting is empty then the global default is directly returned.

6.231.1.7 authLibrary (read/write)

```
wstring IVRDEServer::authLibrary
```

Library used for authentication of RDP clients by this VM. Overrides ISystemProperties::VRDEAuthLibrary.

6.231.1.8 VRDEProperties (read-only)

```
wstring IVRDEServer::VRDEProperties[]
```

Array of names of properties, which are supported by this VRDE server.

6.231.2 getVRDEProperty

```
wstring IVRDEServer::getVRDEProperty(
           [in] wstring key)
```

key Name of the key to get.

Returns a VRDE specific property string.
If the requested data `key` does not exist, this function will succeed and return an empty string in the `value` argument.

6.231.3 setVRDEProperty

```
void IVRDEServer::setVRDEProperty(
            [in] wstring key,
            [in] wstring value)
```

key Name of the key to set.

value Value to assign to the key.

Sets a VRDE specific property string.
If you pass `null` or empty string as a key `value`, the given `key` will be deleted.

6.232 IVRDEServerChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when a property of the VRDE server changes. Interested callees should use IVRDEServer methods and attributes to find out what has changed.

6.232.1 Attributes

6.232.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IVRDEServerChangedEvent::midlDoesNotLikeEmptyInterfaces
```

6.233 IVRDEServerInfo

> **Note:** With the web service, this interface is mapped to a structure. Attributes that return this interface will not return an object, but a complete structure containing the attributes listed below as structure members.

Contains information about the remote desktop (VRDE) server capabilities and status. This is used in the IConsole::VRDEServerInfo attribute.

6.233.1 Attributes

6.233.1.1 active (read-only)

```
boolean IVRDEServerInfo::active
```

Whether the remote desktop connection is active.

6.233.1.2 port (read-only)

```
long IVRDEServerInfo::port
```

VRDE server port number. If this property is equal to `0`, then the VRDE server failed to start, usually because there are no free IP ports to bind to. If this property is equal to `-1`, then the VRDE server has not yet been started.

6.233.1.3 numberOfClients (read-only)

```
unsigned long IVRDEServerInfo::numberOfClients
```

How many times a client connected.

6.233.1.4 beginTime (read-only)

```
long long IVRDEServerInfo::beginTime
```

When the last connection was established, in milliseconds since 1970-01-01 UTC.

6.233.1.5 endTime (read-only)

```
long long IVRDEServerInfo::endTime
```

When the last connection was terminated or the current time, if connection is still active, in milliseconds since 1970-01-01 UTC.

6.233.1.6 bytesSent (read-only)

```
long long IVRDEServerInfo::bytesSent
```

How many bytes were sent in last or current, if still active, connection.

6.233.1.7 bytesSentTotal (read-only)

```
long long IVRDEServerInfo::bytesSentTotal
```

How many bytes were sent in all connections.

6.233.1.8 bytesReceived (read-only)

```
long long IVRDEServerInfo::bytesReceived
```

How many bytes were received in last or current, if still active, connection.

6.233.1.9 bytesReceivedTotal (read-only)

```
long long IVRDEServerInfo::bytesReceivedTotal
```

How many bytes were received in all connections.

6.233.1.10 user (read-only)

```
wstring IVRDEServerInfo::user
```

Login user name supplied by the client.

6.233.1.11 domain (read-only)

```
wstring IVRDEServerInfo::domain
```

Login domain name supplied by the client.

6.233.1.12 clientName (read-only)

```
wstring IVRDEServerInfo::clientName
```

The client name supplied by the client.

6.233.1.13 clientIP (read-only)

```
wstring IVRDEServerInfo::clientIP
```

The IP address of the client.

6.233.1.14 clientVersion (read-only)

```
unsigned long IVRDEServerInfo::clientVersion
```

The client software version number.

6.233.1.15 encryptionStyle (read-only)

```
unsigned long IVRDEServerInfo::encryptionStyle
```

Public key exchange method used when connection was established. Values: 0 - RDP4 public key exchange scheme. 1 - X509 certificates were sent to client.

6.234 IVRDEServerInfoChangedEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Notification when the status of the VRDE server changes. Interested callees should use IVRDEServerInfo attributes to find out what is the current status.

6.234.1 Attributes

6.234.1.1 midlDoesNotLikeEmptyInterfaces (read-only)

```
boolean IVRDEServerInfoChangedEvent::midlDoesNotLikeEmptyInterfaces
```

6.235 IVetoEvent (IEvent)

> **Note:** This interface extends IEvent and therefore supports all its methods and attributes as well.

Base abstract interface for veto events.

6.235.1 addApproval

```
void IVetoEvent::addApproval(
           [in] wstring reason)
```

reason Reason for approval, could be null or empty string.

Adds an approval on this event.

6.235.2 addVeto

```
void IVetoEvent::addVeto(
            [in] wstring reason)
```

reason Reason for veto, could be null or empty string.

Adds a veto on this event.

6.235.3 getApprovals

```
wstring[] IVetoEvent::getApprovals()
```

Current approval reason list, if size is 0 - no approvals.

6.235.4 getVetos

```
wstring[] IVetoEvent::getVetos()
```

Current veto reason list, if size is 0 - no veto.

6.235.5 isApproved

```
boolean IVetoEvent::isApproved()
```

If this event was approved.

6.235.6 isVetoed

```
boolean IVetoEvent::isVetoed()
```

If this event was vetoed.

6.236 IVirtualBox

The IVirtualBox interface represents the main interface exposed by the product that provides virtual machine management.

An instance of IVirtualBox is required for the product to do anything useful. Even though the interface does not expose this, internally, IVirtualBox is implemented as a singleton and actually lives in the process of the VirtualBox server (VBoxSVC.exe). This makes sure that IVirtualBox can track the state of all virtual machines on a particular host, regardless of which frontend started them.

To enumerate all the virtual machines on the host, use the machines[] attribute.

Error information handling is a bit special with IVirtualBox: creating an instance will always succeed. The return of the actual error code/information is postponed to any attribute or method call. The reason for this is that COM likes to mutilate the error code and lose the detailed error information returned by instance creation.

6.236.1 Attributes

6.236.1.1 version (read-only)

```
wstring IVirtualBox::version
```

A string representing the version number of the product. The format is 3 integer numbers divided by dots (e.g. 1.0.1). The last number represents the build number and will frequently change.

This may be followed by a _ALPHA[0-9]*, _BETA[0-9]* or _RC[0-9]* tag in prerelease builds. Non-Oracle builds may (/shall) also have a publisher tag, at the end. The publisher tag starts with an underscore just like the prerelease build type tag.

6.236.1.2 versionNormalized (read-only)

```
wstring IVirtualBox::versionNormalized
```

A string representing the version number of the product, without the publisher information (but still with other tags). See version.

6.236.1.3 revision (read-only)

```
unsigned long IVirtualBox::revision
```

The internal build revision number of the product.

6.236.1.4 packageType (read-only)

```
wstring IVirtualBox::packageType
```

A string representing the package type of this product. The format is OS_ARCH_DIST where OS is one of: WINDOWS, LINUX, SOLARIS, or DARWIN. ARCH is 32BITS, 64BITS, or ARM64. DIST is GENERIC, UBUNTU_606, UBUNTU_710, or similar.

6.236.1.5 APIVersion (read-only)

```
wstring IVirtualBox::APIVersion
```

A string representing the VirtualBox API version number. The format is 2 integer numbers divided by an underscore (e.g. 1_0). After the first public release of packages with a particular API version the API will not be changed in an incompatible way. Note that this guarantee does not apply to development builds, and also there is no guarantee that this version is identical to the first two integer numbers of the package version.

6.236.1.6 APIRevision (read-only)

```
long long IVirtualBox::APIRevision
```

This is mainly intended for the VBox Validation Kit so it can fluently deal with incompatible API changes and new functionality during development (i.e. on trunk).

The high 7 bits (62:56) is the major version number, the next 8 bits (55:48) are the minor version number, the next 8 bits (47:40) are the build number, and the rest (39:0) is the API revision number.

The API revision number is manually increased on trunk when making incompatible changes that the validation kit or others needs to be able to detect and cope with dynamically. It can also be used to indicate the presence of new features on both trunk and branches.

6.236.1.7 homeFolder (read-only)

`wstring IVirtualBox::homeFolder`

Full path to the directory where the global settings file, `VirtualBox.xml`, is stored.

In this version of VirtualBox, the value of this property is always `<user_dir>/.VirtualBox` (where `<user_dir>` is the path to the user directory, as determined by the host OS), and cannot be changed.

This path is also used as the base to resolve relative paths in places where relative paths are allowed (unless otherwise expressly indicated).

6.236.1.8 settingsFilePath (read-only)

`wstring IVirtualBox::settingsFilePath`

Full name of the global settings file. The value of this property corresponds to the value of homeFolder plus `/VirtualBox.xml`.

6.236.1.9 host (read-only)

`IHost IVirtualBox::host`

Associated host object.

6.236.1.10 systemProperties (read-only)

`ISystemProperties IVirtualBox::systemProperties`

Associated system information object.

6.236.1.11 machines (read-only)

`IMachine IVirtualBox::machines[]`

Array of machine objects registered within this VirtualBox instance.

6.236.1.12 machineGroups (read-only)

`wstring IVirtualBox::machineGroups[]`

Array of all machine group names which are used by the machines which are accessible. Each group is only listed once, however they are listed in no particular order and there is no guarantee that there are no gaps in the group hierarchy (i.e. `"/"`, `"/group/subgroup"` is a valid result).

6.236.1.13 hardDisks (read-only)

`IMedium IVirtualBox::hardDisks[]`

Array of medium objects known to this VirtualBox installation.

This array contains only base media. All differencing media of the given base medium can be enumerated using IMedium::children[].

6.236.1.14 DVDImages (read-only)

`IMedium IVirtualBox::DVDImages[]`

Array of CD/DVD image objects currently in use by this VirtualBox instance.

6.236.1.15 floppyImages (read-only)

`IMedium IVirtualBox::floppyImages[]`

Array of floppy image objects currently in use by this VirtualBox instance.

6.236.1.16 progressOperations (read-only)

`IProgress IVirtualBox::progressOperations[]`

6.236.1.17 guestOSTypes (read-only)

`IGuestOSType IVirtualBox::guestOSTypes[]`

Array of all guest OS type objects known to this VirtualBox installation.

6.236.1.18 guestOSFamilies (read-only)

`wstring IVirtualBox::guestOSFamilies[]`

Array of guest OS family identifiers known to this VirtualBox installation.
The guest OS family identifier corrresponds to the IGuestOSType::familyId attribute.

6.236.1.19 sharedFolders (read-only)

`ISharedFolder IVirtualBox::sharedFolders[]`

Collection of global shared folders. Global shared folders are available to all virtual machines.
New shared folders are added to the collection using createSharedFolder(). Existing shared folders can be removed using removeSharedFolder().

6.236.1.20 performanceCollector (read-only)

`IPerformanceCollector IVirtualBox::performanceCollector`

Associated performance collector object.

6.236.1.21 DHCPServers (read-only)

`IDHCPServer IVirtualBox::DHCPServers[]`

DHCP servers.

6.236.1.22 NATNetworks (read-only)

`INATNetwork IVirtualBox::NATNetworks[]`

6.236.1.23 eventSource (read-only)

`IEventSource IVirtualBox::eventSource`

Event source for VirtualBox events.

6.236.1.24 extensionPackManager (read-only)

```
IExtPackManager IVirtualBox::extensionPackManager
```

> **Note:** This attribute is not supported in the web service.

The extension pack manager.

6.236.1.25 internalNetworks (read-only)

```
wstring IVirtualBox::internalNetworks[]
```

Names of all internal networks.

6.236.1.26 hostOnlyNetworks (read-only)

```
IHostOnlyNetwork IVirtualBox::hostOnlyNetworks[]
```

Names of all host-only networks.

6.236.1.27 genericNetworkDrivers (read-only)

```
wstring IVirtualBox::genericNetworkDrivers[]
```

Names of all generic network drivers.

6.236.1.28 cloudNetworks (read-only)

```
ICloudNetwork IVirtualBox::cloudNetworks[]
```

Names of all configured cloud networks.

6.236.1.29 cloudProviderManager (read-only)

```
ICloudProviderManager IVirtualBox::cloudProviderManager
```

The cloud provider manager (singleton).

6.236.2 checkFirmwarePresent

```
boolean IVirtualBox::checkFirmwarePresent(
            [in] PlatformArchitecture platformArchitecture,
            [in] FirmwareType firmwareType,
            [in] wstring version,
            [out] wstring url,
            [out] wstring file)
```

platformArchitecture Architecture of firmware to check.

firmwareType Type of firmware to check.

version Expected version number, usually empty string (presently ignored).

url Suggested URL to download this firmware from.

file Filename of firmware, only valid if result == TRUE.

Check if this VirtualBox installation has a firmware of the given type available, either system-wide or per-user. Optionally, this may return a hint where this firmware can be downloaded from.

6.236.3 composeMachineFilename

```
wstring IVirtualBox::composeMachineFilename(
            [in] wstring name,
            [in] wstring group,
            [in] wstring createFlags,
            [in] wstring baseFolder)
```

name Suggested machine name.

group Machine group name for the new machine or machine group. It is used to determine the right subdirectory.

createFlags Machine creation flags, see createMachine() (optional).

baseFolder Base machine folder (optional).

Returns a recommended full path of the settings file name for a new virtual machine.
This API serves two purposes:

- It gets called by createMachine() if `null` or empty string (which is recommended) is specified for the `settingsFile` argument there, which means that API should use a recommended default file name.
- It can be called manually by a client software before creating a machine, e.g. if that client wants to pre-create the machine directory to create virtual hard disks in that directory together with the new machine settings file. In that case, the file name should be stripped from the full settings file path returned by this function to obtain the machine directory.

See IMachine::name and createMachine() for more details about the machine name.
`groupName` defines which additional subdirectory levels should be included. It must be either a valid group name or `null` or empty string which designates that the machine will not be related to a machine group.
If `baseFolder` is a `null` or empty string (which is recommended), the default machine settings folder (see ISystemProperties::defaultMachineFolder) will be used as a base folder for the created machine, resulting in a file name like "/home/user/VirtualBox VMs/name/name.vbox". Otherwise the given base folder will be used.
This method does not access the host disks. In particular, it does not check for whether a machine with this name already exists.

6.236.4 createAppliance

```
IAppliance IVirtualBox::createAppliance()
```

Creates a new appliance object, which represents an appliance in the Open Virtual Machine Format (OVF). This can then be used to import an OVF appliance into VirtualBox or to export machines as an OVF appliance; see the documentation for IAppliance for details.

6.236.5 createCloudNetwork

```
ICloudNetwork IVirtualBox::createCloudNetwork(
            [in] wstring networkName)
```

networkName

6.236.6 createDHCPServer

```
IDHCPServer IVirtualBox::createDHCPServer(
            [in] wstring name)
```

name server name

Creates a DHCP server settings to be used for the given internal network name
If this method fails, the following error codes may be reported:

- E_INVALIDARG: Host network interface name already exists.

6.236.7 createHostOnlyNetwork

```
IHostOnlyNetwork IVirtualBox::createHostOnlyNetwork(
            [in] wstring networkName)
```

networkName

6.236.8 createMachine

```
IMachine IVirtualBox::createMachine(
            [in] wstring settingsFile,
            [in] wstring name,
            [in] PlatformArchitecture platform,
            [in] wstring groups[],
            [in] wstring osTypeId,
            [in] wstring flags,
            [in] wstring cipher,
            [in] wstring passwordId,
            [in] wstring password)
```

settingsFile Fully qualified path where the settings file should be created, empty string or null for a default folder and file based on the name argument and the primary group. (see composeMachineFilename()).

name Machine name.

platform The platform architecture for the new VM.

groups Array of group names. null or an empty array have the same meaning as an array with just the empty string or "/", i.e. create a machine without group association.

osTypeId Guest OS Type ID.

flags Additional property parameters, passed as a comma-separated list of "name=value" type entries. The following ones are recognized: forceOverwrite=1 to overwrite an existing machine settings file, UUID=<uuid> to specify a machine UUID and directoryIncludesUUID=1 to switch to a special VM directory naming scheme which should not be used unless necessary.

cipher The cipher. It should be empty if encryption is not required.

passwordId The password id. It should be empty if encryption is not required.

password The password. It should be empty if encryption is not required.

Creates a new virtual machine by creating a machine settings file at the given location.

VirtualBox machine settings files use a custom XML dialect. Starting with VirtualBox 4.0, a ".vbox" extension is recommended, but not enforced, and machine files can be created at arbitrary locations.

However, it is recommended that machines are created in the default machine folder (e.g. "/home/user/VirtualBox VMs/name/name.vbox"; see ISystemProperties::defaultMachineFolder). If you specify `null` or empty string (which is recommended) for the `settingsFile` argument, composeMachineFilename() is called automatically to have such a recommended name composed based on the machine name given in the `name` argument and the primary group.

If the resulting settings file already exists, this method will fail, unless the forceOverwrite flag is set.

The new machine is created unregistered, with the initial configuration set according to the specified guest OS type. A typical sequence of actions to create a new virtual machine is as follows:

1. Call this method to have a new machine created. The returned machine object will be "mutable" allowing to change any machine property.
2. Configure the machine using the appropriate attributes and methods.
3. Call IMachine::saveSettings() to write the settings to the machine's XML settings file. The configuration of the newly created machine will not be saved to disk until this method is called.
4. Call registerMachine() to add the machine to the list of machines known to VirtualBox.

The specified guest OS type identifier must match an ID of one of known guest OS types listed in the guestOSTypes[] array.

> **Note:** IMachine::settingsModified will return `false` for the created machine, until any of machine settings are changed.

> **Note:** There is no way to change the name of the settings file or subfolder of the created machine directly.

If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND: osTypeId` is invalid.
- `VBOX_E_FILE_ERROR:` Resulting settings file name is invalid or the settings file already exists or could not be created due to an I/O error.
- `E_INVALIDARG: name` is empty or `null`.

6.236.9 createMedium

```
IMedium IVirtualBox::createMedium(
           [in] wstring format,
           [in] wstring location,
           [in] AccessMode accessMode,
           [in] DeviceType aDeviceTypeType)
```

format Identifier of the storage format to use for the new medium.

location Location of the storage unit for the new medium.

accessMode Whether to open the image in read/write or read-only mode. For a "DVD" device type, this is ignored and read-only mode is always assumed.

aDeviceTypeType Must be one of "HardDisk", "DVD" or "Floppy".

Creates a new base medium object that will use the given storage format and location for medium data.

The actual storage unit is not created by this method. In order to do it, and before you are able to attach the created medium to virtual machines, you must call one of the following methods to allocate a format-specific storage unit at the specified location:

- IMedium::createBaseStorage()
- IMedium::createDiffStorage()

Some medium attributes, such as IMedium::id, may remain uninitialized until the medium storage unit is successfully created by one of the above methods.

Depending on the given device type, the file at the storage location must be in one of the media formats understood by VirtualBox:

- With a "HardDisk" device type, the file must be a hard disk image in one of the formats supported by VirtualBox (see ISystemProperties::mediumFormats[]). After the storage unit is successfully created and this method succeeds, if the medium is a base medium, it will be added to the hardDisks[] array attribute.
- With a "DVD" device type, the file must be an ISO 9960 CD/DVD image. After this method succeeds, the medium will be added to the DVDImages[] array attribute.
- With a "Floppy" device type, the file must be an RAW floppy image. After this method succeeds, the medium will be added to the floppyImages[] array attribute.

The list of all storage formats supported by this VirtualBox installation can be obtained using ISystemProperties::mediumFormats[]. If the `format` attribute is empty or `null` then the default storage format specified by ISystemProperties::defaultHardDiskFormat will be used for disks r creating a storage unit of the medium.

Note that the format of the location string is storage format specific. See IMedium::location and IMedium for more details.

If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND: format` identifier is invalid. See ISystemProperties::mediumFormats[].
- `VBOX_E_FILE_ERROR: location` is a not valid file name (for file-based formats only).

6.236.10 createNATNetwork

```
INATNetwork IVirtualBox::createNATNetwork(
           [in] wstring networkName)
```

networkName

6.236.11 createSharedFolder

```
void IVirtualBox::createSharedFolder(
           [in] wstring name,
           [in] wstring hostPath,
           [in] boolean writable,
           [in] boolean automount,
           [in] wstring autoMountPoint)
```

name Unique logical name of the shared folder.

hostPath Full path to the shared folder in the host file system.

writable Whether the share is writable or readonly

automount Whether the share gets automatically mounted by the guest or not.

autoMountPoint Where the guest should automatically mount the folder, if possible. For Windows and OS/2 guests this should be a drive letter, while other guests it should be a absolute directory.

Creates a new global shared folder by associating the given logical name with the given host path, adds it to the collection of shared folders and starts sharing it. Refer to the description of ISharedFolder to read more about logical names.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_IN_USE: Shared folder already exists.
- VBOX_E_FILE_ERROR: Shared folder hostPath not accessible.

6.236.12 createUnattendedInstaller

```
IUnattended IVirtualBox::createUnattendedInstaller()
```

Creates a new IUnattended guest installation object. This can be used to analyze an installation ISO to create and configure a new machine for it to be installed on. It can also be used to (re)install an existing machine.

6.236.13 findCloudNetworkByName

```
ICloudNetwork IVirtualBox::findCloudNetworkByName(
           [in] wstring networkName)
```

networkName

6.236.14 findDHCPServerByNetworkName

```
IDHCPServer IVirtualBox::findDHCPServerByNetworkName(
           [in] wstring name)
```

name server name

Searches a DHCP server settings to be used for the given internal network name

If this method fails, the following error codes may be reported:

- E_INVALIDARG: Host network interface name already exists.

6.236.15 findHostOnlyNetworkById

```
IHostOnlyNetwork IVirtualBox::findHostOnlyNetworkById(
            [in] uuid id)
```

id GUID of the host-only network to search for.

Searches through all host networks for one with the given GUID.

> **Note:** The method returns an error if the given GUID does not correspond to any host network.

6.236.16 findHostOnlyNetworkByName

```
IHostOnlyNetwork IVirtualBox::findHostOnlyNetworkByName(
            [in] wstring networkName)
```

networkName

6.236.17 findMachine

```
IMachine IVirtualBox::findMachine(
            [in] wstring nameOrId)
```

nameOrId What to search for. This can either be the UUID or the name of a virtual machine.

Attempts to find a virtual machine given its name or UUID.

> **Note:** Inaccessible machines cannot be found by name, only by UUID, because their name cannot safely be determined.

If this method fails, the following error codes may be reported:

- `VBOX_E_OBJECT_NOT_FOUND`: Could not find registered machine matching `nameOrId`.

6.236.18 findNATNetworkByName

```
INATNetwork IVirtualBox::findNATNetworkByName(
            [in] wstring networkName)
```

networkName

6.236.19 findProgressById

```
IProgress IVirtualBox::findProgressById(
            [in] uuid id)
```

id GUID of the progress object to search for.

Searches through all progress objects known to VBoxSVC for an instance with the given GUID.

> **Note:** The method returns an error if the given GUID does not correspond to any currently known progress object.

6.236.20 getExtraData

```
wstring IVirtualBox::getExtraData(
           [in] wstring key)
```

key Name of the data key to get.

Returns associated global extra data.
If the requested data `key` does not exist, this function will succeed and return an empty string in the `value` argument.
If this method fails, the following error codes may be reported:

- `VBOX_E_FILE_ERROR`: Settings file not accessible.
- `VBOX_E_XML_ERROR`: Could not parse the settings file.

6.236.21 getExtraDataKeys

```
wstring[] IVirtualBox::getExtraDataKeys()
```

Returns an array representing the global extra data keys which currently have values defined.

6.236.22 getGuestOSDescsBySubtype

```
wstring[] IVirtualBox::getGuestOSDescsBySubtype(
           [in] wstring OSSubtype)
```

OSSubtype Guest OS subtype.

Returns a list of guest OS family descriptions for a corresponding guest OS family subtype.
The guestOSTypes[] collection contains all available guest OS type objects. Each object has an IGuestOSType::subtype attribute which contains a human readable description of the guest OS subtype this object describes. The guest OS family description is contained in the IGuestOSType::familyDescription attribute.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: `OSSubtype` is not a valid guest OS subtype.

6.236.23 getGuestOSSubtypesByFamilyId

```
wstring[] IVirtualBox::getGuestOSSubtypesByFamilyId(
           [in] wstring family)
```

family Guest OS type ID string.

Returns a list of guest OS family subtypes for a corresponding guest OS family ID type.
The guestOSTypes[] collection contains all available guest OS type objects. Each object has an IGuestOSType::familyId attribute which contains the guest OS family this object describes. The guest OS family subtype is contained in the IGuestOSType::subtype attribute.
If this method fails, the following error codes may be reported:

- `E_INVALIDARG`: `family` is not a valid guest OS family identifier.

6.236.24 getGuestOSType

```
IGuestOSType IVirtualBox::getGuestOSType(
            [in] wstring id)
```

id Guest OS type ID string.

Returns an object describing the specified guest OS type.

The requested guest OS type is specified using a string which is a mnemonic identifier of the guest operating system, such as `"win31"` or `"ubuntu"`. The guest OS type ID of a particular virtual machine can be read or set using the IMachine::OSTypeId attribute.

The guestOSTypes[] collection contains all available guest OS type objects. Each object has an IGuestOSType::id attribute which contains an identifier of the guest OS this object describes.

While this function returns an error for unknown guest OS types, they can be still used without serious problems (if one accepts the fact that there is no default VM config information).

If this method fails, the following error codes may be reported:

- `E_INVALIDARG: id` is not a valid Guest OS type.

6.236.25 getMachineStates

```
MachineState[] IVirtualBox::getMachineStates(
            [in] IMachine machines[])
```

machines Array with the machine references.

Gets the state of several machines in a single operation.

6.236.26 getMachinesByGroups

```
IMachine[] IVirtualBox::getMachinesByGroups(
            [in] wstring groups[])
```

groups What groups to match. The usual group list rules apply, i.e. passing an empty list will match VMs in the toplevel group, likewise the empty string.

Gets all machine references which are in one of the specified groups.

6.236.27 getPlatformProperties

```
IPlatformProperties IVirtualBox::getPlatformProperties(
            [in] PlatformArchitecture architecture)
```

architecture The platform architecture to return the platform properties for.

Gets the associated properties object for the given VM platform architecture.

6.236.28 getTrackedObject

```
void IVirtualBox::getTrackedObject(
            [in] wstring trObjId,
            [out] $unknown pIface,
            [out] TrackedObjectState state,
            [out] long long creationTime,
            [out] long long deletionTime)
```

trObjId The object Id.

pIface The found interface.

state The object state.

creationTime Creation time.

deletionTime Deletion time.

Queries the IUnknown interface to an object from the tracked object list.

6.236.29 getTrackedObjectIds

```
wstring[] IVirtualBox::getTrackedObjectIds(
            [in] wstring name)
```

name The interface name.

Returns the tracked object Ids list associated with the passed interface name

6.236.30 openMachine

```
IMachine IVirtualBox::openMachine(
            [in] wstring settingsFile,
            [in] wstring password)
```

settingsFile Name of the machine settings file.

password The password. If the machine is not encrypted the parameter is ignored.

Opens a virtual machine from the existing settings file. The opened machine remains unregistered until you call registerMachine().

The specified settings file name must be fully qualified. The file must exist and be a valid machine XML settings file whose contents will be used to construct the machine object.

> **Note:** If the VM is encrypted and password is incorrect the method returns success allowing you to register the encrypted machine but it remains in inaccessible state. You can check IMachine::accessible and IMachine::accessError properties to determine the real machine state.

> **Note:** IMachine::settingsModified will return `false` for the opened machine, until any of machine settings are changed.

If this method fails, the following error codes may be reported:

- `VBOX_E_FILE_ERROR`: Settings file name invalid, not found or sharing violation.

6.236.31 openMedium

```
IMedium IVirtualBox::openMedium(
            [in] wstring location,
            [in] DeviceType deviceType,
            [in] AccessMode accessMode,
            [in] boolean forceNewUuid)
```

location Location of the storage unit that contains medium data in one of the supported storage formats.

deviceType Must be one of "HardDisk", "DVD" or "Floppy".

accessMode Whether to open the image in read/write or read-only mode. For a "DVD" device type, this is ignored and read-only mode is always assumed.

forceNewUuid Allows the caller to request a completely new medium UUID for the image which is to be opened. Useful if one intends to open an exact copy of a previously opened image, as this would normally fail due to the duplicate UUID.

Finds existing media or opens a medium from an existing storage location.

Once a medium has been opened, it can be passed to other VirtualBox methods, in particular to IMachine::attachDevice().

Depending on the given device type, the file at the storage location must be in one of the media formats understood by VirtualBox:

- With a "HardDisk" device type, the file must be a hard disk image in one of the formats supported by VirtualBox (see ISystemProperties::mediumFormats[]). After this method succeeds, if the medium is a base medium, it will be added to the hardDisks[] array attribute.
- With a "DVD" device type, the file must be an ISO 9960 CD/DVD image. After this method succeeds, the medium will be added to the DVDImages[] array attribute.
- With a "Floppy" device type, the file must be an RAW floppy image. After this method succeeds, the medium will be added to the floppyImages[] array attribute.

After having been opened, the medium can be re-found by this method and can be attached to virtual machines. See IMedium for more details.

The UUID of the newly opened medium will either be retrieved from the storage location, if the format supports it (e.g. for hard disk images), or a new UUID will be randomly generated (e.g. for ISO and RAW files). If for some reason you need to change the medium's UUID, use IMedium::setIds().

If a differencing hard disk medium is to be opened by this method, the operation will succeed only if its parent medium and all ancestors, if any, are already known to this VirtualBox installation (for example, were opened by this method before).

This method attempts to guess the storage format of the specified medium by reading medium data at the specified location.

If `accessMode` is ReadWrite (which it should be for hard disks and floppies), the image is opened for read/write access and must have according permissions, as VirtualBox may actually write status information into the disk's metadata sections.

Note that write access is required for all typical hard disk usage in VirtualBox, since VirtualBox may need to write metadata such as a UUID into the image. The only exception is opening a source image temporarily for copying and cloning (see IMedium::cloneTo() when the image will be closed again soon.

The format of the location string is storage format specific. See IMedium::location and IMedium for more details.

If this method fails, the following error codes may be reported:

- `VBOX_E_FILE_ERROR:` Invalid medium storage file location or could not find the medium at the specified location.
- `VBOX_E_IPRT_ERROR:` Could not get medium storage format.
- `E_INVALIDARG:` Invalid medium storage format.
- `VBOX_E_INVALID_OBJECT_STATE:` Medium has already been added to a media registry.

6.236.32 registerMachine

```
void IVirtualBox::registerMachine(
            [in] IMachine machine)
```

machine

Registers the machine previously created using createMachine() or opened using openMachine() within this VirtualBox installation. After successful method invocation, the IMachineRegisteredEvent event is fired.

> **Note:** This method implicitly calls IMachine::saveSettings() to save all current machine settings before registering it.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: No matching virtual machine found.
- VBOX_E_INVALID_OBJECT_STATE: Virtual machine was not created within this VirtualBox instance.

6.236.33 removeCloudNetwork

```
void IVirtualBox::removeCloudNetwork(
            [in] ICloudNetwork network)
```

network

6.236.34 removeDHCPServer

```
void IVirtualBox::removeDHCPServer(
            [in] IDHCPServer server)
```

server DHCP server settings to be removed

Removes the DHCP server settings
If this method fails, the following error codes may be reported:

- E_INVALIDARG: Host network interface name already exists.

6.236.35 removeHostOnlyNetwork

```
void IVirtualBox::removeHostOnlyNetwork(
            [in] IHostOnlyNetwork network)
```

network

6.236.36 removeNATNetwork

```
void IVirtualBox::removeNATNetwork(
            [in] INATNetwork network)
```

network

6.236.37 removeSharedFolder

```
void IVirtualBox::removeSharedFolder(
           [in] wstring name)
```

name Logical name of the shared folder to remove.

Removes the global shared folder with the given name previously created by createSharedFolder() from the collection of shared folders and stops sharing it.

If this method fails, the following error codes may be reported:

- VBOX_E_OBJECT_NOT_FOUND: Shared folder name does not exist.

6.236.38 setExtraData

```
void IVirtualBox::setExtraData(
           [in] wstring key,
           [in] wstring value)
```

key Name of the data key to set.

value Value to assign to the key.

Sets associated global extra data.

If you pass null or an empty string as a key value, the given key will be deleted.

> **Note:** Key must contain printable (non-control) UTF-8 characters only.

> **Note:** Before performing the actual data change, this method will ask all registered event listeners using the IExtraDataCanChangeEvent notification for a permission. If one of the listeners refuses the new value, the change will not be performed.

> **Note:** On success, the IExtraDataChangedEvent notification is called to inform all registered listeners about a successful data change.

If this method fails, the following error codes may be reported:

- VBOX_E_FILE_ERROR: Settings file not accessible.
- VBOX_E_XML_ERROR: Could not parse the settings file.
- E_ACCESSDENIED: Modification request refused.
- E_INVALIDARG: Key contains invalid characters.

6.236.39 setSettingsSecret

```
void IVirtualBox::setSettingsSecret(
           [in] wstring password)
```

password The cipher key.

Unlocks the secret data by passing the unlock password to the server. The server will cache the password for that machine.

If this method fails, the following error codes may be reported:

- VBOX_E_INVALID_VM_STATE: Virtual machine is not mutable.

6.237 IVirtualBoxClient

Note: This interface is not supported in the web service.

Convenience interface for client applications. Treat this as a singleton, i.e. never create more than one instance of this interface.

At the moment only available for clients of the local API (not usable via the webservice). Once the session logic is redesigned this might change.

Error information handling is a bit special with IVirtualBoxClient: creating an instance will always succeed. The return of the actual error code/information is postponed to any attribute or method call. The reason for this is that COM likes to mutilate the error code and lose the detailed error information returned by instance creation.

6.237.1 Attributes

6.237.1.1 virtualBox (read-only)

```
IVirtualBox IVirtualBoxClient::virtualBox
```

Reference to the server-side API root object.

6.237.1.2 session (read-only)

```
ISession IVirtualBoxClient::session
```

Create a new session object and return the reference to it.

6.237.1.3 eventSource (read-only)

```
IEventSource IVirtualBoxClient::eventSource
```

Event source for VirtualBoxClient events.

6.237.2 checkMachineError

```
void IVirtualBoxClient::checkMachineError(
           [in] IMachine machine)
```

machine The machine object to check.

Perform error checking before using an IMachine object. Generally useful before starting a VM and all other uses. If anything is not as it should be then this method will return an appropriate error.

6.238 IVirtualBoxErrorInfo

The IVirtualBoxErrorInfo interface represents extended error information.

Extended error information can be set by VirtualBox components after unsuccessful or partially successful method invocation. This information can be retrieved by the calling party as an IVirtualBoxErrorInfo object and then shown to the client in addition to the plain 32-bit result code.

In MS COM, this interface extends the IErrorInfo interface, in XPCOM, it extends the nsIException interface. In both cases, it provides a set of common attributes to retrieve error information.

Sometimes invocation of some component's method may involve methods of other components that may also fail (independently of this method's failure), or a series of non-fatal errors may precede a fatal error that causes method failure. In cases like that, it may be desirable to preserve information about all errors happened during method invocation and deliver it to the caller. The next attribute is intended specifically for this purpose and allows to represent a chain of errors through a single IVirtualBoxErrorInfo object set after method invocation.

> **Note:** errors are stored to a chain in the reverse order, i.e. the initial error object you query right after method invocation is the last error set by the callee, the object it points to in the `next` attribute is the previous error and so on, up to the first error (which is the last in the chain).

6.238.1 Attributes

6.238.1.1 resultCode (read-only)

`long IVirtualBoxErrorInfo::resultCode`

Result code of the error. Usually, it will be the same as the result code returned by the method that provided this error information, but not always. For example, on Win32, CoCreateInstance() will most likely return E_NOINTERFACE upon unsuccessful component instantiation attempt, but not the value the component factory returned. Value is typed 'long', not 'result', to make interface usable from scripting languages.

> **Note:** In MS COM, there is no equivalent. In XPCOM, it is the same as nsIException::result.

6.238.1.2 resultDetail (read-only)

`long IVirtualBoxErrorInfo::resultDetail`

Optional result data of this error. This will vary depending on the actual error usage. By default this attribute is not being used.

6.238.1.3 interfaceID (read-only)

`uuid IVirtualBoxErrorInfo::interfaceID`

UUID of the interface that defined the error.

> **Note:** In MS COM, it is the same as IErrorInfo::GetGUID, except for the data type. In XPCOM, there is no equivalent.

6.238.1.4 component (read-only)

`wstring IVirtualBoxErrorInfo::component`

Name of the component that generated the error.

> **Note:** In MS COM, it is the same as IErrorInfo::GetSource. In XPCOM, there is no equivalent.

6.238.1.5 text (read-only)

```
wstring IVirtualBoxErrorInfo::text
```

Text description of the error.

> **Note:** In MS COM, it is the same as IErrorInfo::GetDescription. In XPCOM, it is the same as nsIException::message.

6.238.1.6 next (read-only)

```
IVirtualBoxErrorInfo IVirtualBoxErrorInfo::next
```

Next error object if there is any, or `null` otherwise.

> **Note:** In MS COM, there is no equivalent. In XPCOM, it is the same as nsIException::inner.

6.239 IVirtualBoxSDS

> **Note:** This interface is not supported in the web service.

The IVirtualBoxSDS interface represents the system-wide directory service helper.
It exists only on Windows host, and its purpose is to work around design flaws in Microsoft's (D)COM, in particular the local server instantiation behavior.

6.239.1 deregisterVBoxSVC

> **Note:** This method is not supported in the web service.

```
void IVirtualBoxSDS::deregisterVBoxSVC(
            [in] IVBoxSVCRegistration vboxSVC,
            [in] long pid)
```

vboxSVC Same as specified during registration.

pid The process ID of the VBoxSVC instance (same as during registration).

Registers a VBoxSVC instance with the SDS.

6.239.2 launchVMProcess

```
unsigned long IVirtualBoxSDS::launchVMProcess(
            [in] wstring machine,
            [in] wstring comment,
            [in] wstring frontend,
            [in] wstring environmentChanges[],
            [in] wstring cmdOptions,
            [in] unsigned long sessionId)
```

machine The name or id of the machine the VM will start for.

comment The comment for VM.

frontend Front-end to use for the new VM process. The following are currently supported:

- `"gui"`: VirtualBox Qt GUI front-end
- `"headless"`: VBoxHeadless (VRDE Server) front-end
- `"sdl"`: VirtualBox SDL front-end
- `""`: use the per-VM default frontend if set, otherwise the global default defined in the system properties. If neither are set, the API will launch a `"gui"` session, which may fail if there is no windowing environment available. See IMachine::defaultFrontend and ISystemProperties::defaultFrontend.

environmentChanges The list of putenv-style changes to the VM process environment. See IMachine::launchVMProcess() for details.

cmdOptions Additional command line options to pass to the VM process.

sessionId Windows session where the VM process should be launched.

Spawns a new process that will execute the virtual machine in the interactive windows session of calling user. Any additional checks are performed by created process itself

If launching the VM succeeds, the new VM process will create its own session and write-lock the machine for it, preventing conflicting changes from other processes. If the machine is already locked (because it is already running or because another session has a write lock), launching the VM process will therefore fail. Reversely, future attempts to obtain a write lock will also fail while the machine is running.

Launching a VM process can take some time (a new VM is started in a new process, for which memory and other resources need to be set up) but the method does not wait for completion and just returns the PID of created process.

If this method fails, the following error codes may be reported:

- `E_INVALIDARG:` Some of the parameters are invalid.
- `VBOX_E_IPRT_ERROR:` Launching process for machine failed.

6.239.3 registerVBoxSVC

> **Note:** This method is not supported in the web service.

```
$unknown IVirtualBoxSDS::registerVBoxSVC(
           [in] IVBoxSVCRegistration vboxSVC,
           [in] long pid)
```

vboxSVC Interface implemented by the VirtualBox class factory.

pid The process ID of the VBoxSVC instance.

Registers a VBoxSVC instance with VBoxSDS. If the caller is not running in a Windows 0 session, the method attempts to run VBoxSVC in that session.

If this method fails, the following error codes may be reported:

- `E_PENDING:` The caller is not running in a Windows session 0 and no VBoxSVC is registered. VBoxSVC registration begins in Windows session 0. You should call this method again later.

6.240 IVirtualSystemDescription

Represents one virtual system (machine) in an appliance. This interface is used in the IAppliance::virtualSystemDescriptions[] array. After IAppliance::interpret() has been called, that array contains information about how the virtual systems described in the OVF should best be imported into VirtualBox virtual machines. See IAppliance for the steps required to import an OVF into VirtualBox.

6.240.1 Attributes

6.240.1.1 count (read-only)

```
unsigned long IVirtualSystemDescription::count
```

Return the number of virtual system description entries.

6.240.2 addDescription

```
void IVirtualSystemDescription::addDescription(
           [in] VirtualSystemDescriptionType type,
           [in] wstring VBoxValue,
           [in] wstring extraConfigValue)
```

type

VBoxValue

extraConfigValue

This method adds an additional description entry to the stack of already available descriptions for this virtual system. This is handy for writing values which aren't directly supported by VirtualBox. One example would be the License type of VirtualSystemDescriptionType.

6.240.3 getDescription

```
void IVirtualSystemDescription::getDescription(
           [out] VirtualSystemDescriptionType types[],
           [out] wstring refs[],
           [out] wstring OVFValues[],
           [out] wstring VBoxValues[],
           [out] wstring extraConfigValues[])
```

types

refs

OVFValues

VBoxValues

extraConfigValues

Returns information about the virtual system as arrays of instruction items. In each array, the items with the same indices correspond and jointly represent an import instruction for VirtualBox.

The list below identifies the value sets that are possible depending on the VirtualSystemDescriptionType enum value in the array item in aTypes[]. In each case, the array item with the same index in OVFValues[] will contain the original value as contained

in the OVF file (just for informational purposes), and the corresponding item in `aVBoxValues[]` will contain a suggested value to be used for VirtualBox. Depending on the description type, the `aExtraConfigValues[]` array item may also be used.

- "OS": the guest operating system type. There must be exactly one such array item on import. The corresponding item in `aVBoxValues[]` contains the suggested guest operating system for VirtualBox. This will be one of the values listed in IVirtualBox::guestOSTypes[]. The corresponding item in `OVFValues[]` will contain a numerical value that described the operating system in the OVF.

- "Name": the name to give to the new virtual machine. There can be at most one such array item; if none is present on import, then an automatic name will be created from the operating system type. The corresponding item im `OVFValues[]` will contain the suggested virtual machine name from the OVF file, and `aVBoxValues[]` will contain a suggestion for a unique VirtualBox IMachine name that does not exist yet.

- "Description": an arbitrary description.

- "License": the EULA section from the OVF, if present. It is the responsibility of the calling code to display such a license for agreement; the Main API does not enforce any such policy.

- Miscellaneous: reserved for future use.

- "CPU": the number of CPUs. There can be at most one such item, which will presently be ignored.

- "Memory": the amount of guest RAM, in bytes. There can be at most one such array item; if none is present on import, then VirtualBox will set a meaningful default based on the operating system type.

- "HardDiskControllerIDE": an IDE hard disk controller. There can be at most two such items. An optional value in `OVFValues[]` and `aVBoxValues[]` can be "PIIX3" or "PIIX4" to specify the type of IDE controller; this corresponds to the ResourceSubType element which VirtualBox writes into the OVF. The matching item in the `aRefs[]` array will contain an integer that items of the "Harddisk" type can use to specify which hard disk controller a virtual disk should be connected to. Note that in OVF, an IDE controller has two channels, corresponding to "master" and "slave" in traditional terminology, whereas the IDE storage controller that VirtualBox supports in its virtual machines supports four channels (primary master, primary slave, secondary master, secondary slave) and thus maps to two IDE controllers in the OVF sense.

- "HardDiskControllerSATA": an SATA hard disk controller. There can be at most one such item. This has no value in `OVFValues[]` or `aVBoxValues[]`. The matching item in the `aRefs[]` array will be used as with IDE controllers (see above).

- "HardDiskControllerSCSI": a SCSI hard disk controller. There can be at most one such item. The items in `OVFValues[]` and `aVBoxValues[]` will either be "LsiLogic", "BusLogic" or "LsiLogicSas". (Note that in OVF, the LsiLogicSas controller is treated as a SCSI controller whereas VirtualBox considers it a class of storage controllers of its own; see StorageControllerType). The matching item in the `aRefs[]` array will be used as with IDE controllers (see above).

- "HardDiskImage": a virtual hard disk, most probably as a reference to an image file. There can be an arbitrary number of these items, one for each virtual disk image that accompanies the OVF.

  The array item in `OVFValues[]` will contain the file specification from the OVF file (without a path since the image file should be in the same location as the OVF file itself), whereas the

item in aVBoxValues[] will contain a qualified path specification to where VirtualBox uses the hard disk image. This means that on import the image will be copied and converted from the "ovf" location to the "vbox" location; on export, this will be handled the other way round.

The matching item in the aExtraConfigValues[] array must contain a string of the following format: "controller=<index>;channel=<c>" In this string, <index> must be an integer specifying the hard disk controller to connect the image to. That number must be the index of an array item with one of the hard disk controller types (HardDiskControllerSCSI, HardDiskControllerSATA, HardDiskControllerIDE). In addition, <c> must specify the channel to use on that controller. For IDE controllers, this can be 0 or 1 for master or slave, respectively. For compatibility with VirtualBox versions before 3.2, the values 2 and 3 (for secondary master and secondary slave) are also supported, but no longer exported. For SATA and SCSI controllers, the channel can range from 0-29.

- "CDROM": a virtual CD-ROM drive. The matching item in aExtraConfigValue[] contains the same attachment information as with "HardDiskImage" items.
- "CDROM": a virtual floppy drive. The matching item in aExtraConfigValue[] contains the same attachment information as with "HardDiskImage" items.
- "NetworkAdapter": a network adapter. The array item in aVBoxValues[] will specify the hardware for the network adapter, whereas the array item in aExtraConfigValues[] will have a string of the "type=<X>" format, where <X> must be either "NAT" or "Bridged".
- "USBController": a USB controller. There can be at most one such item. If, and only if, such an item is present, USB support will be enabled for the new virtual machine.
- "SoundCard": a sound card. There can be at most one such item. If and only if such an item is present, sound support will be enabled for the new virtual machine. Note that the virtual machine in VirtualBox will always be presented with the standard VirtualBox soundcard, which may be different from the virtual soundcard expected by the appliance.

6.240.4 getDescriptionByType

```
void IVirtualSystemDescription::getDescriptionByType(
            [in] VirtualSystemDescriptionType type,
            [out] VirtualSystemDescriptionType types[],
            [out] wstring refs[],
            [out] wstring OVFValues[],
            [out] wstring VBoxValues[],
            [out] wstring extraConfigValues[])
```

type

types

refs

OVFValues

VBoxValues

extraConfigValues

This is the same as getDescription() except that you can specify which types should be returned.

6.240.5 getValuesByType

```
wstring[] IVirtualSystemDescription::getValuesByType(
            [in] VirtualSystemDescriptionType type,
            [in] VirtualSystemDescriptionValueType which)
```

type

which

This is the same as getDescriptionByType() except that you can specify which value types should be returned. See VirtualSystemDescriptionValueType for possible values.

6.240.6 removeDescriptionByType

```
void IVirtualSystemDescription::removeDescriptionByType(
            [in] VirtualSystemDescriptionType type)
```

type

Delete all records which are equal to the passed type from the list

6.240.7 setFinalValues

```
void IVirtualSystemDescription::setFinalValues(
            [in] boolean enabled[],
            [in] wstring VBoxValues[],
            [in] wstring extraConfigValues[])
```

enabled

VBoxValues

extraConfigValues

This method allows the appliance's user to change the configuration for the virtual system descriptions. For each array item returned from getDescription(), you must pass in one boolean value and one configuration value.

Each item in the boolean array determines whether the particular configuration item should be enabled. You can only disable items of the types HardDiskControllerIDE, HardDiskControllerSATA, HardDiskControllerSCSI, HardDiskImage, CDROM, Floppy, NetworkAdapter, USBController and SoundCard.

For the "vbox" and "extra configuration" values, if you pass in the same arrays as returned in the aVBoxValues and aExtraConfigValues arrays from getDescription(), the configuration remains unchanged. Please see the documentation for getDescription() for valid configuration values for the individual array item types. If the corresponding item in the aEnabled array is `false`, the configuration value is ignored.

6.241 IVirtualSystemDescriptionForm (IForm)

Note: This interface extends IForm and therefore supports all its methods and attributes as well.

6.241.1 getVirtualSystemDescription

```
IVirtualSystemDescription IVirtualSystemDescriptionForm::getVirtualSystemDescription()
```

6.242 IWebsessionManager

Note: This interface is supported in the web service only, not in COM/XPCOM.

Websession manager. This provides essential services to webservice clients.

6.242.1 getSessionObject

```
ISession IWebsessionManager::getSessionObject(
            [in] IVirtualBox refIVirtualBox)
```

refIVirtualBox

Returns a managed object reference to a new ISession object for every call to this method.
See also: ISession

6.242.2 logoff

```
void IWebsessionManager::logoff(
            [in] IVirtualBox refIVirtualBox)
```

refIVirtualBox

Logs off the client who has previously logged on with logon() and destroys all resources associated with the websession (most importantly, all managed objects created in the server while the websession was active).

6.242.3 logon

```
IVirtualBox IWebsessionManager::logon(
            [in] wstring username,
            [in] wstring password)
```

username

password

Logs a new client onto the webservice and returns a managed object reference to the IVirtualBox instance, which the client can then use as a basis to further queries, since all calls to the VirtualBox API are based on the IVirtualBox interface, in one way or the other.

7 Enumerations (enums)

7.1 APICMode

BIOS APIC initialization mode. If the hardware does not support the mode then the code falls back to a lower mode.

Disabled

APIC

X2APIC

7.2 AccessMode

Access mode for opening files.

ReadOnly

ReadWrite

7.3 AdditionsFacilityClass

Guest Additions facility classes.

None No/invalid class.

Driver Driver.

Service System service.

Program Program.

Feature Feature.

ThirdParty Third party.

All All facility classes selected.

7.4 AdditionsFacilityStatus

Guest Additions facility states.

Inactive Facility is not active.

Paused Facility has been paused.

PreInit Facility is preparing to initialize.

Init Facility is initializing.

Active Facility is up and running.

Terminating Facility is shutting down.

Terminated Facility successfully shut down.

Failed Facility failed to start.

Unknown Facility status is unknown.

7.5 AdditionsFacilityType

Guest Additions facility IDs.

None No/invalid facility.

VBoxGuestDriver VirtualBox base driver (VBoxGuest).

AutoLogon Auto-logon modules (VBoxGINA, VBoxCredProv, pam_vbox).

VBoxService VirtualBox system service (VBoxService).

VBoxTrayClient VirtualBox desktop integration (VBoxTray on Windows, VBoxClient on non-Windows).

Seamless Seamless guest desktop integration.

Graphics Guest graphics mode. If not enabled, seamless rendering will not work, resize hints are not immediately acted on and guest display resizes are probably not initiated by the Guest Additions.

MonitorAttach Guest supports monitor hotplug.

All All facilities selected.

7.6 AdditionsRunLevelType

Guest Additions run level type.

None Guest Additions are not loaded.

System Guest drivers are loaded.

Userland Common components (such as application services) are loaded.

Desktop Per-user desktop components are loaded.

7.7 AdditionsUpdateFlag

Guest Additions update flags.

None No flag set.

WaitForUpdateStartOnly Starts the regular updating process and waits until the actual Guest Additions update inside the guest was started. This can be necessary due to needed interaction with the guest OS during the installation phase.

7.8 AudioCodecType

The exact variant of audio codec hardware presented to the guest; see IAudioAdapter::audioCodec.

Null `null` value. Never used by the API.

SB16 SB16; this is the only option for the SB16 device.

STAC9700 A STAC9700 AC'97 codec.

AD1980 An AD1980 AC'97 codec. Recommended for Linux guests.

STAC9221 A STAC9221 HDA codec.

7.9 AudioControllerType

Virtual audio controller type.

AC97

SB16

HDA

VirtioSound

7.10 AudioDeviceState

Audio device state enumeration.

Unknown Device state is unknown / cannot be determined

Active Device is active and can be used.

Disabled Device is in a disabled state.

NotPresent Device is marked as not being present.

Unplugged Device has been unplugged.

7.11 AudioDeviceType

Audio device type enumeration.

Unknown Device type is unknown / cannot be determined

BuiltLin Built-in device (cannot be removed).

ExternalUSB External device, connected via USB.

ExternalOther External device, connected via some other method.

7.12 AudioDirection

Audio direction enumeration.

Unknown Direction cannot be determined.

In Input (Recording).

Out Output (Playback).

Duplex Duplex (Recording + Playback).

7.13 AudioDriverType

Host audio driver type.

Default Use the default audio driver automatically determined for the host that this VirtualBox instance is running on. Useful for VMs which need to run on different host OSes.

Null Null value, also means “dummy audio driver”.

OSS Open Sound System (Linux / Unix hosts only).

ALSA Advanced Linux Sound Architecture (Linux hosts only).

Pulse PulseAudio (Linux hosts only).

WinMM Reserved for historical reasons.

DirectSound DirectSound (Windows hosts only).

WAS Windows Audio Session (Windows hosts only).

CoreAudio CoreAudio (Mac hosts only).

MMPM Reserved for historical reasons.

SolAudio Reserved for historical reasons.

7.14 AuthType

VirtualBox authentication type.

Null Null value, also means “no authentication”.

External

Guest

7.15 AutostopType

Autostop types, used with IMachine::autostopType.

Disabled Stopping the VM during system shutdown is disabled.

SaveState The state of the VM will be saved when the system shuts down.

PowerOff The VM is powered off when the system shuts down.

AcpiShutdown An ACPI shutdown event is generated.

7.16 BandwidthGroupType

Type of a bandwidth control group.

Null Null type, must be first.

Disk The bandwidth group controls disk I/O.

Network The bandwidth group controls network I/O.

7.17 BitmapFormat

Format of a bitmap. Generic values for formats used by the source bitmap, the screen shot or image update APIs.

Opaque Unknown buffer format (the user may not assume any particular format of the buffer).

BGR Generic BGR format without alpha channel. Pixel layout depends on the number of bits per pixel:

- **32** - bits 31:24 undefined, bits 23:16 R, bits 15:8 G, bits 7:0 B.
- **16** - bits 15:11 R, bits 10:5 G, bits 4:0 B.

BGR0 4 bytes per pixel: B, G, R, 0.

BGRA 4 bytes per pixel: B, G, R, A.

RGBA 4 bytes per pixel: R, G, B, A.

PNG PNG image.

JPEG JPEG image.

7.18 CPUArchitecture

Basic CPU architecture types.

Any Matches any CPU architecture.

x86 32-bit (and 16-bit) x86.

AMD64 64-bit x86. (Also known as x86-64 or x64.)

ARMv8_32 32-bit only ARMv8. (Also known as AArch32 or ARM32.)

ARMv8_64 64-bit only ARMv8. (Also known as AArch64 or ARM64.)

7.19 CPUPropertyTypeARM

Virtual CPU property type for ARM-based CPUs. This enumeration represents possible values of the IPlatformARM get- and setCPUProperty methods.

Null Null value (never used by the API).

HWVirt Enables the hardware virtualization feature on the guest CPU. This requires the nested virtualization feature on the host CPU.

GICITS Enables the Interrupt Translation Service (ITS) for the GIC.

7.20 CPUPropertyTypeX86

Virtual CPU property type for x86-based CPUs. This enumeration represents possible values of the IPlatformX86 get- and setCPUProperty methods.

Null Null value (never used by the API).

PAE This setting determines whether VirtualBox will expose the Physical Address Extension (PAE) feature of the host CPU to the guest. Note that in case PAE is not available, it will not be reported.

LongMode This setting determines whether VirtualBox will advertise long mode (i.e. 64-bit guest support) and let the guest enter it.

TripleFaultReset This setting determines whether a triple fault within a guest will trigger an internal error condition and stop the VM (default) or reset the virtual CPU/VM and continue execution.

APIC This setting determines whether an APIC is part of the virtual CPU. This feature can only be turned off when the X2APIC feature is off.

X2APIC This setting determines whether an x2APIC is part of the virtual CPU. Since this feature implies that the APIC feature is present, it automatically enables the APIC feature when set.

IBPBOnVMExit If set, force an indirect branch prediction barrier on VM exits if the host CPU supports it. This setting will significantly slow down workloads causing many VM exits, so it is only recommended for situation where there is a real need to be paranoid.

IBPBOnVMEntry If set, force an indirect branch prediction barrier on VM entry if the host CPU supports it. This setting will significantly slow down workloads causing many VM exits, so it is only recommended for situation where there is a real need to be paranoid.

HWVirt Enabled the hardware virtualization (AMD-V/VT-x) feature on the guest CPU. This requires hardware virtualization on the host CPU.

SpecCtrl If set, the speculation control CPUID bits and MSRs, when available on the host, are exposed to the guest. Depending on the host CPU and operating system, this may significantly slow down workloads causing many VM exits.

SpecCtrlByHost If set, the speculation controls are managed by the host. This is intended for guests which do not set the speculation controls themselves. Note! This has not yet been implemented beyond leaving everything to the host OS.

L1DFlushOnEMTScheduling If set and the host is affected by CVE-2018-3646, flushes the level 1 data cache when the EMT is scheduled to do ring-0 guest execution. There could be a small performance penalty for certain typs of workloads. For security reasons this setting will be enabled by default.

L1DFlushOnVMEntry If set and the host is affected by CVE-2018-3646, flushes the level 1 data on every VM entry. This setting may significantly slow down workloads causing many VM exits, so it is only recommended for situation where there is a real need to be paranoid.

MDSClearOnEMTScheduling If set and the host is affected by CVE-2018-12126, CVE-2018-12127, or CVE-2018-12130, clears the relevant MDS buffers when the EMT is scheduled to do ring-0 guest execution. There could be a small performance penalty for certain typs of workloads. For security reasons this setting will be enabled by default.

MDSClearOnVMEntry If set and the host is affected by CVE-2018-12126, CVE-2018-12127, or CVE-2018-12130, clears the relevant MDS buffers on every VM entry. This setting may slow down workloads causing many VM exits, so it is only recommended for situation where there is a real need to be paranoid.

7.21 CertificateVersion

X.509 certificate version numbers.

V1

V2

V3

Unknown

7.22 ChipsetType

Type of emulated chipset (think x86 southbridge or ARM SoC/board).

Null `null` value. Never used by the API.

PIIX3 x86: A PIIX3 (PCI IDE ISA Xcelerator) chipset.

ICH9 x86: A ICH9 (I/O Controller Hub) chipset.

ARMv8Virtual ARM: Virtual ARMv8 base platform (board).

7.23 CleanupMode

Cleanup mode, used with IMachine::unregister().

UnregisterOnly Unregister only the machine, but neither delete snapshots nor detach media.

DetachAllReturnNone Delete all snapshots and detach all media but return none; this will keep all media registered.

DetachAllReturnHardDisksOnly Delete all snapshots, detach all media and return hard disks for closing, but not removable media.

Full Delete all snapshots, detach all media and return all media for closing.

DetachAllReturnHardDisksAndVMRemovable Delete all snapshots, detach all media and return hard disks and removable media located in the folder of the machine to be deleted and referenced only by this machine, for closing.

7.24 ClipboardMode

Host-Guest clipboard interchange mode.

Disabled

HostToGuest

GuestToHost

Bidirectional

7.25 CloneMode

Clone mode, used with IMachine::cloneTo().

MachineState Clone the state of the selected machine.

MachineAndChildStates Clone the state of the selected machine and its child snapshots if present.

AllStates Clone all states (including all snapshots) of the machine, regardless of the machine object used.

7.26 CloneOptions

Clone options, used with IMachine::cloneTo().

Link Create a clone VM where all virtual disks are linked to the original VM.

KeepAllMACs Don't generate new MAC addresses of the attached network adapters.

KeepNATMACs Don't generate new MAC addresses of the attached network adapters when they are using NAT.

KeepDiskNames Don't change the disk names.

KeepHwUUIDs Don't change UUID of the machine hardware.

7.27 CloudImageState

Cloud image state

Invalid Invalid state

Provisioning The image is in the process of provisioning

Importing The image is in the process of importing

Available The image is avalable

Exporting The image is in the process of exporting

Disabled The image is disabled

Deleted The image was deleted

7.28 CloudMachineState

Cloud instance execution state

Invalid Invalid state

Provisioning The machine is in the process of provisioning

Running The machine runs

Starting The machine is in the process of starting

Stopping The machine is in the process of stopping

Stopped The machine was stopped

CreatingImage The machine is in the process of creating image

Terminating The machine is in the process of terminating

Terminated The machine was terminated

7.29 DHCPConfigScope

Global IDHCPServer::globalConfig

Group IDHCPServer::groupConfigs[]

MachineNIC IDHCPServer::individualConfigs[]

MAC IDHCPServer::individualConfigs[]

7.30 DHCPGroupConditionType

MAC MAC address

MACWildcard MAC address wildcard pattern.

vendorClassID Vendor class ID

vendorClassIDWildcard Vendor class ID wildcard pattern.

userClassID User class ID

userClassIDWildcard User class ID wildcard pattern.

7.31 DHCPOption

SubnetMask IPv4 netmask. Set to IDHCPServer::networkMask by default.

TimeOffset UTC offset in seconds (32-bit decimal value).

Routers Space separated list of IPv4 router addresses.

TimeServers Space separated list of IPv4 time server (RFC 868) addresses.

NameServers Space separated list of IPv4 name server (IEN 116) addresses.

DomainNameServers Space separated list of IPv4 DNS addresses.

LogServers Space separated list of IPv4 log server addresses.

CookieServers Space separated list of IPv4 cookie server (RFC 865) addresses.

LPRServers Space separated list of IPv4 line printer server (RFC 1179) addresses.

ImpressServers Space separated list of IPv4 imagen impress server addresses.

ResourseLocationServers Space separated list of IPv4 resource location (RFC 887) addresses.

HostName The client name. See RFC 1035 for character limits.

BootFileSize Number of 512 byte blocks making up the boot file (16-bit decimal value).

MeritDumpFile Client core file.

DomainName Domain name for the client.

SwapServer IPv4 address of the swap server that the client should use.

RootPath The path to the root disk the client should use.

ExtensionPath Path to a file containing additional DHCP options (RFC2123).

IPForwarding Whether IP forwarding should be enabled by the client (boolean).

OptNonLocalSourceRouting Whether non-local datagrams should be forwarded by the client (boolean)

PolicyFilter List of IPv4 addresses and masks pairs controlling non-local source routing.

MaxDgramReassemblySize The maximum datagram size the client should reassemble (16-bit decimal value).

DefaultIPTTL The default time-to-leave on outgoing (IP) datagrams (8-bit decimal value).

PathMTUAgingTimeout RFC1191 path MTU discovery timeout value in seconds (32-bit decimal value).

PathMTUPlateauTable RFC1191 path MTU discovery size table, sorted in ascending order (list of 16-bit decimal values).

InterfaceMTU The MTU size for the interface (16-bit decimal value).

AllSubnetsAreLocal Indicates whether the MTU size is the same for all subnets (boolean).

BroadcastAddress Broadcast address (RFC1122) for the client to use (IPv4 address).

PerformMaskDiscovery Whether to perform subnet mask discovery via ICMP (boolean).

MaskSupplier Whether to respond to subnet mask requests via ICMP (boolean).

PerformRouterDiscovery Whether to perform router discovery (RFC1256) (boolean).

RouterSolicitationAddress Where to send router solicitation requests (RFC1256) (IPv4 address).

StaticRoute List of network and router address pairs addresses.

TrailerEncapsulation Whether to negotiate the use of trailers for ARP (RTF893) (boolean).

ARPCacheTimeout The timeout in seconds for ARP cache entries (32-bit decimal value).

EthernetEncapsulation Whether to use IEEE 802.3 (RTF1042) rather than of v2 (RFC894) ethernet encapsulation (boolean).

TCPDefaultTTL Default time-to-live for TCP sends (non-zero 8-bit decimal value).

TCPKeepaliveInterval The interface in seconds between TCP keepalive messages (32-bit decimal value).

TCPKeepaliveGarbage Whether to include a byte of garbage in TCP keepalive messages for backward compatibility (boolean).

NISDomain The NIS (Sun Network Information Services) domain name (string).

NISServers Space separated list of IPv4 NIS server addresses.

NTPServers Space separated list of IPv4 NTP (RFC1035) server addresses.

VendorSpecificInfo Vendor specific information. Only accessible using Hex.

NetBIOSNameServers Space separated list of IPv4 NetBIOS name server (NBNS) addresses (RFC1001,RFC1002).

NetBIOSDatagramServers Space separated list of IPv4 NetBIOS datagram distribution server (NBDD) addresses (RFC1001,RFC1002).

NetBIOSNodeType NetBIOS node type (RFC1001,RFC1002): 1=B-node, 2=P-node, 4=M-node, and 8=H-node (8-bit decimal value).

NetBIOSScope NetBIOS scope (RFC1001,RFC1002). Only accessible using Hex.

XWindowsFontServers Space separated list of IPv4 X windows font server addresses.

XWindowsDisplayManager Space separated list of IPv4 X windows display manager addresses.

NetWareIPDomainName Netware IP domain name (RFC2242) (string).

NetWareIPInformation Netware IP information (RFC2242). Only accessible using Hex.

NISPlusDomain The NIS+ domain name (string).

NISPlusServers Space separated list of IPv4 NIS+ server addresses.

TFTPServerName TFTP server name (string).

BootfileName Bootfile name (string).

MobileIPHomeAgents Space separated list of IPv4 mobile IP agent addresses.

SMTPServers Space separated list of IPv4 simple mail transport protocol (SMPT) server addresses.

POP3Servers Space separated list of IPv4 post office protocol 3 (POP3) server addresses.

NNTPServers Space separated list of IPv4 network news transport protocol (NTTP) server addresses.

WWWServers Space separated list of default IPv4 world wide web (WWW) server addresses.

FingerServers Space separated list of default IPv4 finger server addresses.

IRCServers Space separated list of default IPv4 internet relay chat (IRC) server addresses.

StreetTalkServers Space separated list of IPv4 StreetTalk server addresses.

STDAServers Space separated list of IPv4 StreetTalk directory assistance (STDA) server addresses.

SLPDirectoryAgent Address of one or more service location protocol (SLP) directory agent, and an indicator of whether their use is mandatory. Only accessible using Hex.

SLPServiceScope List of service scopes for the service location protocol (SLP) and whether using the list is mandatory. Only accessible using Hex.

DomainSearch Domain search list, see RFC3397 and section 4.1.4 in RFC1035 for encoding. Only accessible using Hex.

7.32 DHCPOptionEncoding

Normal Value format is specific to the option and generally user friendly.

Hex Value format is a series of hex bytes (`09314f3200fe`), optionally colons as byte separators (`9:31:4f:32::fe`).

7.33 DataFlags

None

Mandatory

Expert

Array

FlagMask

7.34 DataType

Int32

Int8

String

7.35 DeviceActivity

Device activity for IConsole::getDeviceActivity().

Null

Idle

Reading

Writing

7.36 DeviceType

Device type.

Null Null value, may also mean “no device” (not allowed for IConsole::getDeviceActivity()).

Floppy Floppy device.

DVD CD/DVD-ROM device.

HardDisk Hard disk device.

Network Network device.

USB USB device.

SharedFolder Shared folder device.

Graphics3D Graphics device 3D activity.

End End of valid device types (exclusive). Used for invalid validation and such.

7.37 DirectoryCopyFlag

Directory copying flags.

None No flag set.

CopyIntoExisting Allow copying into an existing destination directory.

Recursive Copy directories recursively.

FollowLinks Follow symbolic links.

7.38 DirectoryCreateFlag

Directory creation flags.

None No flag set.

Parents No error if existing, make parent directories as needed.

7.39 DirectoryOpenFlag

Directory open flags.

None No flag set.

NoSymlinks Don't allow symbolic links as part of the path.

DenyAscent Deny relative opening of anything above this directory.
Not implemented yet.

NoFollowSymlinks Don't follow symbolic links in the final component.
Not implemented yet.

7.40 DirectoryRemoveRecFlag

Directory recursive remove flags.

None No flag set.

ContentAndDir Delete the content of the directory and the directory itself.

ContentOnly Only delete the content of the directory, omit the directory it self.

7.41 DirectoryStatus

Directory statuses.

Undefined Directory is in an undefined state.

Open Guest directory has been successfully opened.

Close Guest directory has been closed.

Rewind Guest directory reading was rewound.

Down Service/OS is stopping, guest directory was closed.

Error Something went wrong.

7.42 DnDAction

Possible actions of a drag'n drop operation.

Ignore Do nothing.

Copy Copy the item to the target.

Move Move the item to the target.

Link Link the item from within the target.

7.43 DnDMode

Drag and drop interchange mode.

Disabled

HostToGuest

GuestToHost

Bidirectional

7.44 ExportOptions

Export options, used with IAppliance::write().

CreateManifest Write the optional manifest file (.mf) which is used for integrity checks prior import.

ExportDVDImages Export DVD images. Default is not to export them as it is rarely needed for typical VMs.

StripAllMACs Do not export any MAC address information. Default is to keep them to avoid losing information which can cause trouble after import, at the price of risking duplicate MAC addresses, if the import options are used to keep them.

StripAllNonNATMACs Do not export any MAC address information, except for adapters using NAT. Default is to keep them to avoid losing information which can cause trouble after import, at the price of risking duplicate MAC addresses, if the import options are used to keep them.

7.45 FileAccessMode

File open access mode for use with IGuestSession::fileOpen() and IGuestSession::fileOpenEx().

ReadOnly Open the file only with read access.

WriteOnly Open the file only with write access.

ReadWrite Open the file with both read and write access.

AppendOnly Open the file for appending only, no read or seek access.

Note: Not yet implemented.

AppendRead Open the file for appending and read. Writes always goes to the end of the file while reads are done at the current or specified file position.

> **Note:** Not yet implemented.

7.46 FileCopyFlag

File copying flags.

None No flag set.

NoReplace Do not replace the destination file if it exists.

FollowLinks Follow symbolic links.

Update Only copy when the source file is newer than the destination file or when the destination file is missing.

7.47 FileOpenAction

What action IGuestSession::fileOpen() and IGuestSession::fileOpenEx() should take whether the file being opened exists or not.

OpenExisting Opens an existing file, fails if no file exists. (Was “oe”.)

OpenOrCreate Opens an existing file, creates a new one if no file exists. (Was “oc”.)

CreateNew Creates a new file is no file exists, fails if there is a file there already. (Was “ce”.)

CreateOrReplace Creates a new file, replace any existing file. (Was “ca”.)

> **Note:** Currently undefined whether we will inherit mode and ACLs from the existing file or replace them.

OpenExistingTruncated Opens and truncate an existing file, fails if no file exists. (Was “ot”.)

AppendOrCreate Opens an existing file and places the file pointer at the end of the file, creates the file if it does not exist. This action implies write access. (Was “oa”.)

> **Note:** Deprecated. Only here for historical reasons. Do not use!

7.48 FileOpenExFlag

Open flags for IGuestSession::fileOpenEx().

None No flag set.

7.49 FileSeekOrigin

What a file seek (IFile::seek()) is relative to.

Begin Seek from the beginning of the file.

Current Seek from the current file position.

End Seek relative to the end of the file. To seek to the position two bytes from the end of the file, specify -2 as the seek offset.

7.50 FileSharingMode

File sharing mode for IGuestSession::fileOpenEx().

Read Only share read access to the file.

Write Only share write access to the file.

ReadWrite Share both read and write access to the file, but deny deletion.

Delete Only share delete access, denying read and write.

ReadDelete Share read and delete access to the file, denying writing.

WriteDelete Share write and delete access to the file, denying reading.

All Share all access, i.e. read, write and delete, to the file.

7.51 FileStatus

File statuses.

Undefined File is in an undefined state.

Opening Guest file is opening.

Open Guest file has been successfully opened.

Closing Guest file closing.

Closed Guest file has been closed.

Down Service/OS is stopping, guest file was closed.

Error Something went wrong.

7.52 FirmwareBootMenuMode

Firmware boot menu mode.

Disabled

MenuOnly

MessageAndMenu

7.53 FirmwareType

Firmware type.

BIOS BIOS Firmware. Only applicable to x86 platforms.

EFI EFI Firmware, bitness detected basing on OS type.

EFI32 EFI firmware, 32-bit.

EFI64 EFI firmware, 64-bit.

EFIDUAL EFI firmware, combined 32 and 64-bit.

7.54 FormValueType

Boolean

String

Choice

RangedInteger

RangedInteger64

7.55 FramebufferCapabilities

Framebuffer capability flags.

UpdateImage Requires NotifyUpdateImage. NotifyUpdate must not be called.

VHWA Supports VHWA interface. If set, then IFramebuffer::processVHWACommand can be called.

VisibleRegion Supports visible region. If set, then IFramebuffer::setVisibleRegion can be called.

RenderCursor This framebuffer implementation can render a pointer cursor itself. Unless the MoveCursor capability is also set the cursor will always be rendered at the location of (and usually using) the host pointer.

MoveCursor Supports rendering a pointer cursor anywhere within the guest screen. Implies RenderCursor.

7.56 FsObjMoveFlag

File moving flags.

None No flag set.

Replace Replace the destination file, symlink, etc if it exists, however this does not allow replacing any directories.

FollowLinks Follow symbolic links in the final components or not (only applied to the given source and target paths, not to anything else).

AllowDirectoryMoves Allow moving directories across file system boundaries. Because it is could be a big undertaking, we require extra assurance that we should do it when requested.

7.57 FsObjRenameFlag

Flags for use when renaming file system objects (files, directories, symlink, etc), see IGuestSession::fsObjRename().

NoReplace Do not replace any destination object.

Replace This will attempt to replace any destination object other except directories. (The default is to fail if the destination exists.)

7.58 FsObjType

File system object (file) types.

Unknown Used either if the object has type that is not in this enum, or if the type has not yet been determined or set.

Fifo FIFO or named pipe, depending on the platform/terminology.

DevChar Character device.

Directory Directory.

DevBlock Block device.

File Regular file.

Symlink Symbolic link.

Socket Socket.

WhiteOut A white-out file. Found in union mounts where it is used for hiding files after deletion, I think.

7.59 GraphicsControllerType

Graphics controller type, used with IGraphicsAdapter::graphicsControllerType.

Null Reserved value, invalid.

VBoxVGA VirtualBox VGA device.

VMSVGA VMware SVGA II device.

VBoxSVGA VirtualBox VGA device with VMware SVGA II extensions.

QemuRamFB QEMU RAM based framebuffer based device.

7.60 GraphicsFeature

Graphics features.

None No feature specified, invalid.

Acceleration3D 3D acceleration.

7.61 GuestDebugIoProvider

The enabled guest debug I/O provider. This enumeration represents possible values for the IGuestDebugControl::debugIoProvider attribute.

None No connection available (only useful with None).

TCP The remote stub is available through a TCP connection.

UDP The remote stub is available through a UDP connection.

IPC The remote stub is available through a IPC connection, namely a named pipe on Windows or a Unix socket on other hosts.

7.62 GuestDebugProvider

The enabled guest debug provider. This enumeration represents possible values for the IGuestDebugControl::debugProvider attribute.

None Guest debugging is disabled.

Native The native debugger console is enabled.

GDB The GDB remote stub is enabled.

KD The WinDbg/KD remote stub is enabled.

7.63 GuestMonitorChangedEventType

How the guest monitor has been changed.

Enabled The guest monitor has been enabled by the guest.

Disabled The guest monitor has been disabled by the guest.

NewOrigin The guest monitor origin has changed in the guest.

7.64 GuestMonitorStatus

The current status of the guest display.

Disabled The guest monitor is disabled in the guest.

Enabled The guest monitor is enabled in the guest.

Blank The guest monitor is enabled in the guest but should display nothing.

7.65 GuestMouseEventMode

The mode (relative, absolute, multi-touch) of a pointer event.
@todo A clear pattern seems to be emerging that we should usually have multiple input devices active for different types of reporting, so we should really have different event types for relative (including wheel), absolute (not including wheel) and multi-touch events.

Relative Relative event.

Absolute Absolute event.

7.66 GuestSessionStatus

Guest session status. This enumeration represents possible values of the IGuestSession::status attribute.

Undefined Guest session is in an undefined state.

Starting Guest session is being started.

Started Guest session has been started.

Terminating Guest session is being terminated.

Terminated Guest session terminated normally.

TimedOutKilled Guest session timed out and was killed.

TimedOutAbnormally Guest session timed out and was not killed successfully.

Down Service/OS is stopping, guest session was killed.

Error Something went wrong.

7.67 GuestSessionWaitForFlag

Guest session waiting flags.

None No waiting flags specified. Do not use this.

Start Wait for the guest session being started.

Terminate Wait for the guest session being terminated.

Status Wait for the next guest session status change.

7.68 GuestSessionWaitResult

Guest session waiting results. Depending on the session waiting flags (for more information see GuestSessionWaitForFlag) the waiting result can vary based on the session's current status.

To wait for a guest session to terminate after it has been created by IGuest::createSession() one would specify GuestSessionWaitResult_Terminate.

None No result was returned. Not being used.

Start The guest session has been started.

Terminate The guest session has been terminated.

Status The guest session has changed its status. The status then can be retrieved via IGuestSession::status.

Error Error while executing the process.

Timeout The waiting operation timed out. This also will happen when no event has been occurred matching the current waiting flags in a IGuestSession::waitFor() call.

WaitFlagNotSupported A waiting flag specified in the IGuestSession::waitFor() call is not supported by the guest.

7.69 GuestShutdownFlag

Guest shutdown flags.

None No flag set.

PowerOff Performs a reboot after shutdown.

Reboot Performs a reboot after shutdown.

Force Force the system to shutdown/reboot regardless of objecting application or other stuff. This flag might not be realized on all systems.

7.70 GuestUserState

State a guest user has been changed to.

Unknown Unknown state. Not being used.

LoggedIn A guest user has been successfully logged into the guest OS.

> **Note:** This property is not implemented yet!

LoggedOut A guest user has been successfully logged out of the guest OS.

> **Note:** This property is not implemented yet!

Locked A guest user has locked its account. This might include running a password-protected screensaver in the guest.

> **Note:** This property is not implemented yet!

Unlocked A guest user has unlocked its account.

> **Note:** This property is not implemented yet!

Disabled A guest user has been disabled by the guest OS.

> **Note:** This property is not implemented yet!

Idle A guest user currently is not using the guest OS.

> **Note:** Currently only available for Windows guests since Windows 2000 SP2.

Note: On Windows guests this function currently only supports reporting contiguous idle times up to 49.7 days per user.

The event will be triggered if a guest user is not active for at least 5 seconds. This threshold can be adjusted by either altering VBoxService's command line in the guest to

```
--vminfo-user-idle-threshold <ms>
```

, or by setting the per-VM guest property

```
/VirtualBox/GuestAdd/VBoxService/--vminfo-user-idle-threshold <ms>
```

with the RDONLYGUEST flag on the host. In both cases VBoxService needs to be restarted in order to get the changes applied.

InUse A guest user continued using the guest OS after being idle.

Created A guest user has been successfully created.

Note: This property is not implemented yet!

Deleted A guest user has been successfully deleted.

Note: This property is not implemented yet!

SessionChanged To guest OS has changed the session of a user.

Note: This property is not implemented yet!

CredentialsChanged To guest OS has changed the authentication credentials of a user. This might include changed passwords and authentication types.

Note: This property is not implemented yet!

RoleChanged To guest OS has changed the role of a user permanently, e.g. granting / denying administrative rights.

Note: This property is not implemented yet!

GroupAdded To guest OS has added a user to a specific user group.

Note: This property is not implemented yet!

GroupRemoved To guest OS has removed a user from a specific user group.

> **Note:** This property is not implemented yet!

Elevated To guest OS temporarily has elevated a user to perform a certain task.

> **Note:** This property is not implemented yet!

7.71 HWVirtExPropertyType

Hardware virtualization property type. This enumeration represents possible values for the IPlatformX86::getHWVirtExProperty() and IPlatformX86::setHWVirtExProperty() methods.

Null Null value (never used by the API).

Enabled Whether hardware virtualization (VT-x/AMD-V) is enabled at all. If such extensions are not available, they will not be used.

VPID Whether VT-x VPID is enabled. If this extension is not available, it will not be used.

NestedPaging Whether Nested Paging is enabled. If this extension is not available, it will not be used.

UnrestrictedExecution Whether VT-x unrestricted execution is enabled. If this feature is not available, it will not be used.

LargePages Whether large page allocation is enabled; requires nested paging and a 64-bit host.

Force Whether the VM should fail to start if hardware virtualization (VT-x/AMD-V) cannot be used. If not set, there will be an automatic fallback to software virtualization.

UseNativeApi Use the native hypervisor API instead of the VirtualBox one (HM) for VT-X/AMD-V. This is ignored if Enabled isn't set.

VirtVmsaveVmload Whether AMD-V Virtualized VMSAVE/VMLOAD is enabled. If this feature is not available, it will not be used.

7.72 HostNetworkInterfaceMediumType

Type of encapsulation. Ethernet encapsulation includes both wired and wireless Ethernet connections. See also: IHostNetworkInterface

Unknown The type of interface cannot be determined.

Ethernet Ethernet frame encapsulation.

PPP Point-to-point protocol encapsulation.

SLIP Serial line IP encapsulation.

7.73 HostNetworkInterfaceStatus

Current status of the interface. See also: IHostNetworkInterface

Unknown The state of interface cannot be determined.

Up The interface is fully operational.

Down The interface is not functioning.

7.74 HostNetworkInterfaceType

Network interface type.

Invalid

Bridged

HostOnly

7.75 ImportOptions

Import options, used with IAppliance::importMachines().

KeepAllMACs Don't generate new MAC addresses of the attached network adapters.

KeepNATMACs Don't generate new MAC addresses of the attached network adapters when they are using NAT.

ImportToVDI Import all disks to VDI format

KeepExtraData Import machine extra-data from the appliance.

7.76 IommuType

The IOMMU type. This enumeration represents possible values for the IPlatform::iommuType attribute.

None No IOMMU is present.

Automatic No IOMMU is present.

AMD An AMD IOMMU.

Intel An Intel IOMMU.

7.77 KeyboardHIDType

Type of keyboard device used in a virtual machine.

None No keyboard.

PS2Keyboard PS/2 keyboard.

USBKeyboard USB keyboard.

ComboKeyboard Combined device, working as PS/2 or USB keyboard, depending on guest behavior. Using of such device can have negative performance implications.

7.78 KeyboardLED

Keyboard LED indicators.

NumLock

CapsLock

ScrollLock

7.79 LockType

Used with IMachine::lockMachine().

Null Placeholder value, do not use when obtaining a lock.

Shared Request only a shared lock for remote-controlling the machine. Such a lock allows changing certain VM settings which can be safely modified for a running VM.

Write Lock the machine for writing. This requests an exclusive lock, i.e. there cannot be any other API client holding any type of lock for this VM concurrently. Remember that a VM process counts as an API client which implicitly holds the equivalent of a shared lock during the entire VM runtime.

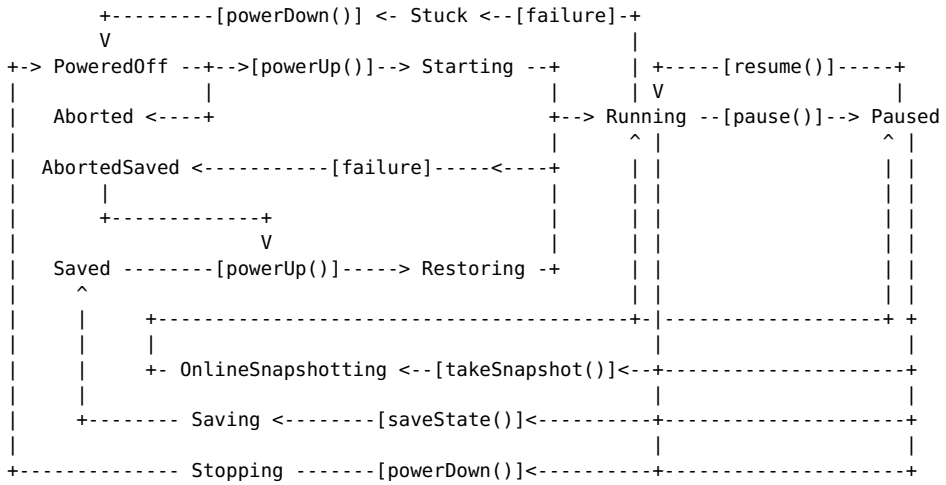
VM Lock the machine for writing, and create objects necessary for running a VM in this process.

7.80 MachineState

Virtual machine execution state.

This enumeration represents possible values of the IMachine::state attribute.

Below is the basic virtual machine state diagram. It shows how the state changes during virtual machine execution. The text in square braces shows a method of the IConsole or IMachine interface that performs the given state transition.



Note that states to the right of PoweredOff, Aborted, AbortedSaved, and Saved in the above diagram are called *online VM states*. These states represent the virtual machine which is being executed in a dedicated process (usually with a GUI window attached to it where you can see

the activity of the virtual machine and interact with it). There are two special pseudo-states, FirstOnline and LastOnline, that can be used in relational expressions to detect if the given machine state is online or not:

```
if (machine.GetState() >= MachineState_FirstOnline &&
    machine.GetState() <= MachineState_LastOnline)
{
    ...the machine is being executed...
}
```

When the virtual machine is in one of the online VM states (that is, being executed), only a few machine settings can be modified. Methods working with such settings contain an explicit note about that. An attempt to change any other setting or perform a modifying operation during this time will result in the `VBOX_E_INVALID_VM_STATE` error.

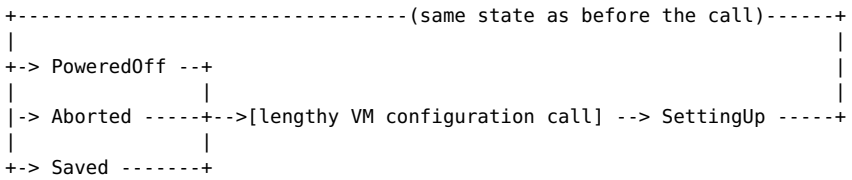
All online states except Running, Paused and Stuck are transitional: they represent temporary conditions of the virtual machine that will last as long as the operation that initiated such a condition.

The Stuck state is a special case. It means that execution of the machine has reached the "Guru Meditation" condition. This condition indicates an internal VMM (virtual machine manager) failure which may happen as a result of either an unhandled low-level virtual hardware exception or one of the recompiler exceptions (such as the *too-many-traps* condition).

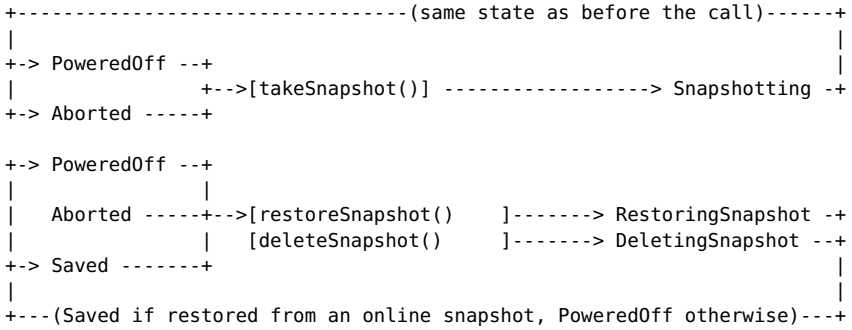
Note also that any online VM state may transit to the Aborted state. This happens if the process that is executing the virtual machine terminates unexpectedly (for example, crashes). Other than that, the Aborted state is equivalent to PoweredOff.

There are also a few additional state diagrams that do not deal with virtual machine execution and therefore are shown separately. The states shown on these diagrams are called *offline VM states* (this includes PoweredOff, Aborted, AbortedSaved and Saved too).

The first diagram shows what happens when a lengthy setup operation is being executed (such as IMachine::attachDevice()).



The next two diagrams demonstrate the process of taking a snapshot of a powered off virtual machine, restoring the state to that as of a snapshot or deleting a snapshot, respectively.



Null Null value (never used by the API).

PoweredOff The machine is not running and has no saved execution state; it has either never been started or been shut down successfully.

Saved The machine is not currently running, but the execution state of the machine has been saved to an external file when it was running, from where it can be resumed.

Teleported The machine was teleported to a different host (or process) and then powered off. Take care when powering it on again may corrupt resources it shares with the teleportation target (e.g. disk and network).

Aborted The process running the machine has terminated abnormally. This may indicate a crash of the VM process in host execution context, or the VM process has been terminated externally.

AbortedSaved A machine in the `Saved` stated has terminated abnormally before reaching the `Running` state. Similar to the `Aborted` state, this may indicate a crash of the VM process in host execution context, or the VM process has been terminated externally.

Running The machine is currently being executed.

Paused Execution of the machine has been paused.

Stuck Execution of the machine has reached the "Guru Meditation" condition. This indicates a severe error in the hypervisor itself.

Teleporting The machine is about to be teleported to a different host or process. It is possible to pause a machine in this state, but it will go to the `TeleportingPausedVM` state and it will not be possible to resume it again unless the teleportation fails.

LiveSnapshotting A live snapshot is being taken. The machine is running normally, but some of the runtime configuration options are inaccessible. Also, if paused while in this state it will transition to `OnlineSnapshotting` and it will not be resume the execution until the snapshot operation has completed.

Starting Machine is being started after powering it on from a zero execution state.

Stopping Machine is being normally stopped powering it off, or after the guest OS has initiated a shutdown sequence.

Saving Machine is saving its execution state to a file.

Restoring Execution state of the machine is being restored from a file after powering it on from either the `Saved` or `AbortedSaved` execution state.

TeleportingPausedVM The machine is being teleported to another host or process, but it is not running. This is the paused variant of the `Teleporting` state.

TeleportingIn Teleporting the machine state in from another host or process.

DeletingSnapshotOnline Like `DeletingSnapshot`, but the merging of media is ongoing in the background while the machine is running.

DeletingSnapshotPaused Like `DeletingSnapshotOnline`, but the machine was paused when the merging of differencing media was started.

OnlineSnapshotting Like `LiveSnapshotting`, but the machine was paused when the merging of differencing media was started.

RestoringSnapshot A machine snapshot is being restored; this typically does not take long.

DeletingSnapshot A machine snapshot is being deleted; this can take a long time since this may require merging differencing media. This value indicates that the machine is not running while the snapshot is being deleted.

SettingUp Lengthy setup operation is in progress.

Snapshotting Taking an (offline) snapshot.

FirstOnline Pseudo-state: first online state (for use in relational expressions).

LastOnline Pseudo-state: last online state (for use in relational expressions).

FirstTransient Pseudo-state: first transient state (for use in relational expressions).

LastTransient Pseudo-state: last transient state (for use in relational expressions).

7.81 MediumFormatCapabilities

Medium format capability flags.

Uuid Supports UUIDs as expected by VirtualBox code.

CreateFixed Supports creating fixed size images, allocating all space instantly.

CreateDynamic Supports creating dynamically growing images, allocating space on demand.

CreateSplit2G Supports creating images split in chunks of a bit less than 2 GBytes.

Differencing Supports being used as a format for differencing media (see IMedium::createDiffStorage()).

Asynchronous Supports asynchronous I/O operations for at least some configurations.

File The format backend operates on files (the IMedium::location attribute of the medium specifies a file used to store medium data; for a list of supported file extensions see IMediumFormat::describeFileExtensions()).

Properties The format backend uses the property interface to configure the storage location and properties (the IMediumFormat::describeProperties() method is used to get access to properties supported by the given medium format).

TcpNetworking The format backend uses the TCP networking interface for network access.

VFS The format backend supports virtual filesystem functionality.

Discard The format backend supports discarding blocks.

Preferred Indicates that this is a frequently used format backend.

CapabilityMask

7.82 MediumState

Virtual medium state. See also: IMedium

NotCreated Associated medium storage does not exist (either was not created yet or was deleted).

Created Associated storage exists and accessible; this gets set if the accessibility check performed by IMedium::refreshState() was successful.

LockedRead Medium is locked for reading (see IMedium::lockRead()), no data modification is possible.

LockedWrite Medium is locked for writing (see IMedium::lockWrite()), no concurrent data reading or modification is possible.

Inaccessible Medium accessibility check (see IMedium::refreshState()) has not yet been performed, or else, associated medium storage is not accessible. In the first case, IMedium::lastAccessError is empty, in the second case, it describes the error that occurred.

Creating Associated medium storage is being created.

Deleting Associated medium storage is being deleted.

7.83 MediumType

Virtual medium type. For each IMedium, this defines how the medium is attached to a virtual machine (see IMediumAttachment) and what happens when a snapshot (see ISnapshot) is taken of a virtual machine which has the medium attached. At the moment DVD and floppy media are always of type "writethrough".

Normal Normal medium (attached directly or indirectly, preserved when taking snapshots).

Immutable Immutable medium (attached indirectly, changes are wiped out the next time the virtual machine is started).

Writethrough Write through medium (attached directly, ignored when taking snapshots).

Shareable Allow using this medium concurrently by several machines.

> **Note:** Present since VirtualBox 3.2.0, and accepted since 3.2.8.

Readonly A readonly medium, which can of course be used by several machines.

> **Note:** Present and accepted since VirtualBox 4.0.

MultiAttach A medium which is indirectly attached, so that one base medium can be used for several VMs which have their own differencing medium to store their modifications. In some sense a variant of Immutable with unset AutoReset flag in each differencing medium.

> **Note:** Present and accepted since VirtualBox 4.0.

7.84 MediumVariant

Virtual medium image variant. More than one flag may be set. See also: IMedium

Standard No particular variant requested, results in using the backend default.

VmdkSplit2G VMDK image split in chunks of less than 2GByte.

VmdkRawDisk VMDK image representing a raw disk.

VmdkStreamOptimized VMDK streamOptimized image. Special import/export format which is read-only/append-only.

VmdkESX VMDK format variant used on ESX products.

VdiZeroExpand Fill new blocks with zeroes while expanding image file.

Fixed Fixed image. Only allowed for base images.

Diff Differencing image. Only allowed for child images.

Formatted Special flag which requests formatting the disk image. Right now supported for floppy images only.

NoCreateDir Special flag which suppresses automatic creation of the subdirectory. Only used when passing the medium variant as an input parameter.

7.85 MetricType

Metric names

Invalid Invalid name

CpuUtilization Cpu Utilization

MemoryUtilization Memory Utilization

DiskBytesRead Disk Bytes Read

DiskBytesWritten Disk Bytes Written

NetworksBytesIn Networks Bytes In

NetworksBytesOut Networks Bytes Out

7.86 MouseButtonState

Mouse button state.

LeftButton

RightButton

MiddleButton

WheelUp

WheelDown

XButton1

XButton2

MouseStateMask

7.87 NATAliasMode

AliasLog

AliasProxyOnly

AliasUseSamePorts

7.88 NATProtocol

Protocol definitions used with NAT port-forwarding rules.

UDP Port-forwarding uses UDP protocol.

TCP Port-forwarding uses TCP protocol.

7.89 NetworkAdapterPromiscModePolicy

The promiscuous mode policy of an interface.

Deny Deny promiscuous mode requests.

AllowNetwork Allow promiscuous mode, but restrict the scope it to the internal network so that it only applies to other VMs.

AllowAll Allow promiscuous mode, include unrelated traffic going over the wire and internally on the host.

7.90 NetworkAdapterType

Network adapter type.

Null Null value (never used by the API).

Am79C970A AMD PCNet-PCI II network card (Am79C970A).

Am79C973 AMD PCNet-FAST III network card (Am79C973).

I82540EM Intel PRO/1000 MT Desktop network card (82540EM).

I82543GC Intel PRO/1000 T Server network card (82543GC).

I82545EM Intel PRO/1000 MT Server network card (82545EM).

Virtio Virtio network device.

Am79C960 AMD PCnet-ISA/NE2100 network card (Am79C960).

NE2000 Novell NE2000 network card (NE2000).

NE1000 Novell NE1000 network card (NE1000).

WD8013 WD/SMC EtherCard Plus 16 network card (WD8013EBT).

WD8003 WD/SMC EtherCard Plus network card (WD8003E).

ELNK2 3Com EtherLink II network card (3C503).

ELNK1 3Com EtherLink network card (3C501/3C500).

UsbNet Ethernet over USB device (usbnet).

I82583V Intel 82583V Gigabit Network Connection [EXPERIMENTAL].

7.91 NetworkAttachmentType

Network attachment type.

Null Null value, also means “not attached”.

NAT

Bridged

Internal

HostOnly

Generic

NATNetwork

Cloud

HostOnlyNetwork

7.92 ParavirtProvider

The paravirtualized guest interface provider. This enumeration represents possible values for the IMachine::paravirtProvider attribute.

None No provider is used.

Default A default provider is automatically chosen according to the guest OS type.

Legacy Used for VMs which didn’t used to have any provider settings. Usually interpreted as `None` for most VMs.

Minimal A minimal set of features to expose to the paravirtualized guest.

HyperV Microsoft Hyper-V.

KVM Linux KVM.

7.93 PartitionTableType

Partition table types.

MBR

GPT

7.94 PartitionType

Empty Empty partition entry

FAT12 FAT12 if partition size less than 65536 sectors

FAT16 FAT16 if partition size less than 65536 sectors

FAT FAT12 or FAT16 if partition size greater or equal than 65536 sectors

IFS NT and OS/2 installable file system, e.g. NTFS, exFAT, HPFS.

FAT32CHS The FAT32 with CHS addressing.

FAT32LBA The FAT32 with LBA addressing.

FAT16B The FAT16 with LBA addressing.

Extended The extended partition with LBA addressing.

WindowsRE Windows Recovery Environment (RE) partition (hidden NTFS partition).

LinuxSwapOld The linux swap partition (old versions).

LinuxOld The linux native partition (old versions).

DragonFlyBSDSlice The BSD slice.

LinuxSwap The linux swap partition.

Linux The linux native partition.

LinuxExtended The linux extended partition.

LinuxLVM The linux LVM partition.

BSDSlice The BSD slice.

AppleUFS The Apple UFS partition.

AppleHFS The Apple HFS partition.

Solaris The Apple HFS partition.

GPT The GPT protective MBR partition.

EFI The EFI system partition.

Unknown Unknown partition type.

MBR MBR partition scheme.

iFFS Intel Fast Flash (iFFS) partition.

SonyBoot Sony boot partition.

LenovoBoot Lenovo boot partition.

WindowsMSR Microsoft Reserved Partition (MSR).

WindowsBasicData Windows Basic data partition

WindowsLDMMeta Windows Logical Disk Manager (LDM) metadata partition.

WindowsLDMData Windows Logical Disk Manager data partition.

WindowsRecovery Windows Recovery Environment.

WindowsStorageSpaces Windows Storage Spaces partition.

WindowsStorageReplica Windows Storage Replica partition.

IBMGPFS IBM General Parallel File System (GPFS) partition.

LinuxData Linux filesystem data.

LinuxRAID Linux RAID partition.

LinuxRootX86 Linux root partition for x86.

LinuxRootAMD64 Linux root partition for AMD64.

LinuxRootARM32 Linux root partition for ARM32.

LinuxRootARM64 Linux root partition for ARM64 / AArch64.

LinuxHome Linux /home partition.

LinuxSrv Linux /srv partition.

LinuxPlainDmCrypt Linux plain dm-crypt partition.

LinuxLUKS Linux unitified key setup (LUKS) partition.

LinuxReserved Linux reserved partition.

FreeBSDBoot FreeBSD boot partition.

FreeBSDData FreeBSD data partition.

FreeBSDSwap FreeBSD swap partition.

FreeBSDUFS FreeBSD unix file system (UFS) partition.

FreeBSDVinum FreeBSD Vinum volume manager partition.

FreeBSDZFS FreeBSD ZFS partition.

FreeBSDUnknown Unknown FreeBSD partition.

AppleHFSPlus Apple hierarchical file system plus (HFS+) partition.

AppleAPFS Apple APFS/FileFault container partition.

AppleRAID Apple RAID partition.

AppleRAIDOffline Apple RAID partition, offline.

AppleBoot Apple boot partition.

AppleLabel Apple label.

AppleTvRecovery Apple TV recovery partition.

AppleCoreStorage Apple Core Storage Containe.

SoftRAIDStatus SoftRAID status.

SoftRAIDScratch SoftRAID scratch.

SoftRAIDVolume SoftRAID volume.

SoftRAIDCache SoftRAID cache.

AppleUnknown Unknown Apple partition.

SolarisBoot Solaris boot partition.

SolarisRoot Solaris root partition.

SolarisSwap Solaris swap partition.

SolarisBackup Solaris backup partition.

SolarisUsr Solaris /usr partition.

SolarisVar Solaris /var partition.

SolarisHome Solaris /home partition.

SolarisAltSector Solaris alternate sector.

SolarisReserved Solaris reserved partition.

SolarisUnknown Unknown Solaris partition.

NetBSDSwap NetBSD swap partition.

NetBSDFFS NetBSD fast file system (FFS) partition.

NetBSDLFS NetBSD log structured file system (LFS) partition.

NetBSDRAID NetBSD RAID partition.

NetBSDConcatenated NetBSD concatenated partition.

NetBSDEncrypted NetBSD encrypted partition.

NetBSDUnknown Unknown NetBSD partition.

ChromeOSKernel Chrome OS kernel partition.

ChromeOSRootFS Chrome OS root file system partition.

ChromeOSFuture Chrome OS partition reserved for future.

ContLnxUsr Container Linux /usr partition.

ContLnxRoot Container Linux resizable root filesystem partition.

ContLnxReserved Container Linux OEM customization partition.

ContLnxRootRAID Container Linux root filesystem on RAID partition.

HaikuBFS Haiku BFS

MidntBSDBoot MidnightBSD boot partition.

MidntBSDData MidnightBSD data partition.

MidntBSDSwap MidnightBSD swap partition.

MidntBSDUFS MidnightBSD unix file system (UFS) partition.

MidntBSDVium MidnightBSD Vinum volume manager partition.

MidntBSDZFS MidnightBSD ZFS partition.

MidntBSDUnknown Unknown MidnightBSD partition.

OpenBSDData OpenBSD data partition.

QNXPowerSafeFS QNX power-safe file system partition.

Plan9 Plan 9 partition.

VMWareVMKCore VMWare ESX coredump partition.

VMWareVMFS VMWare ESX virtual machine file system (VMFS) partition.

VMWareReserved VMWare ESX reserved partition.

VMWareUnknown Unknown VMWare partition.

AndroidX86Bootloader Android x86 bootloader partition.

AndroidX86Bootloader2 Android x86 bootloader2 partition.

AndroidX86Boot Android x86 boot partition.

AndroidX86Recovery Android x86 recovery partition.

AndroidX86Misc Android x86 misc partition.

AndroidX86Metadata Android x86 metadata partition.

AndroidX86System Android x86 system partition.

AndroidX86Cache Android x86 cache partition.

AndroidX86Data Android x86 data partition.

AndroidX86Persistent Android x86 persistent data partition.

AndroidX86Vendor Android x86 vendor partition.

AndroidX86Config Android x86 config partition.

AndroidX86Factory Android x86 factory partition.

AndroidX86FactoryAlt Android x86 alternative factory partition.

AndroidX86Fastboot Android x86 fastboot partition.

AndroidX86OEM Android x86 OEM partition.

AndroidARMMeta Android ARM meta partition.

AndroidARMExt Android ARM EXT partition.

ONIEBoot Open Network Install Environment (ONIE) boot partition.

ONIEConfig Open Network Install Environment (ONIE) config partition.

PowerPCPrep PowerPC PReP boot partition.

XDGShrBootConfig freedesktop.org shared boot loader configuration partition.

CephBlock Ceph block partition.

CephBlockDB Ceph block DB partition.

CephBlockDBDmc Ceph dm-crypt block DB partition.

CephBlockDBDmcLUKS Ceph dm-crypt Linux unitified key setup (LUKS) block DB partition.

CephBlockDmc Ceph dm-crypt block partition.

CephBlockDmcLUKS Ceph dm-crypt Linux unitified key setup (LUKS) block partition.

CephBlockWALog Ceph block write-ahead log partition.

CephBlockWALogDmc Ceph dm-crypt block write-ahead log partition.

CephBlockWALogDmcLUKS Ceph dm-crypt Linux unitified key setup (LUKS) block write-ahead log partition.

CephDisk Ceph disk in creation partition.

CephDiskDmc Ceph dm-crypt disk in creation partition.

CephJournal Ceph Journal partition.

CephJournalDmc Ceph dm-crypt journal partition.

CephJournalDmcLUKS Ceph dm-crypt Linux unitified key setup (LUKS) journal partition.

CephLockbox Ceph Lockbox for dm-crypt keys partition.

CephMultipathBlock1 Ceph multipath block 1 partition.

CephMultipathBlock2 Ceph multipath block 2 partition.

CephMultipathBlockDB Ceph multipath block DB partition.

CephMultipathBLockWALog Ceph multipath block write-ahead log partition.

CephMultipathJournal Ceph multipath journal partition.

CephMultipathOSD Ceph multipath object storage deamon (OSD) partition.

CephOSD Ceph object storage deamon (OSD) partition.

CephOSDDmc Ceph dm-crypt object storage deamon (OSD) partition.

CephOSDDmcLUKS Ceph dm-crypt Linux unitified key setup (LUKS) object storage deamon (OSD) partition.

7.95 PartitioningType

The type of the disk partition.

MBR

GPT

7.96 PathStyle

The path style of a system. (Values matches the RTPATH_STR_F_STYLE_XXX defines in iprt/path.h!)

DOS DOS-style paths with forward and backward slashes, drive letters and UNC. Known from DOS, OS/2 and Windows.

UNIX UNIX-style paths with forward slashes only.

Unknown The path style is not known, most likely because the Guest Additions aren't active yet.

7.97 PlatformArchitecture

Platform architecture.

> **Note:** This does not specify the CPU type. See CPUArchitecture for more information.

None No platform selected. Never used by the API.

x86 x86-based platform (AMD64 / x86). Valid CPUArchitecture choices: AMD64, x86

ARM ARM-based platform (AArch32, AArch64). Valid CPUArchitecture choices: ARMv8_32, ARMv8_64

7.98 PointingHIDType

Type of pointing device used in a virtual machine.

None No mouse.

PS2Mouse PS/2 auxiliary device, a.k.a. mouse.

USBMouse USB mouse (relative pointer).

USBTablet USB tablet (absolute pointer). Also enables a relative USB mouse in addition.

ComboMouse Combined device, working as PS/2 or USB mouse, depending on guest behavior. Using this device can have negative performance implications.

USBMultiTouch USB multi-touch device, just touchscreen. It is a specific mode of the USB tablet and also enables the mouse device.

USBMultiTouchScreenPlusPad USB multi-touch device, touchscreen plus touchpad. It also enables the mouse device.

7.99 PortMode

The PortMode enumeration represents possible communication modes for the virtual serial port device.

Disconnected Virtual device is not attached to any real host device.

HostPipe Virtual device is attached to a host pipe.

HostDevice Virtual device is attached to a host device.

RawFile Virtual device is attached to a raw file.

TCP Virtual device is attached to a TCP socket.

7.100 ProcessCreateFlag

Guest process execution flags.

> **Note:** The values are passed to the Guest Additions, so its not possible to change (move) or reuse values.here. See GUEST_PROC_CREATE_FLAG_XXX in GuestControlSvc.h.

None No flag set.

WaitForProcessStartOnly Only use the specified timeout value to wait for starting the guest process - the guest process itself then uses an infinite timeout.

IgnoreOrphanedProcesses Do not report an error when executed processes are still alive when VBoxService or the guest OS is shutting down.

Hidden Do not show the started process according to the guest OS guidelines.

Profile Utilize the user's profile data when exeuting a process. Only available for Windows guests at the moment.

WaitForStdOut The guest process waits until all data from stdout is read out.

WaitForStdErr The guest process waits until all data from stderr is read out.

ExpandArguments Expands environment variables in process arguments.

> **Note:** This is not yet implemented and is currently silently ignored. We will document the protocolVersion number for this feature once it appears, so don't use it till then.

UnquotedArguments Work around for Windows and OS/2 applications not following normal argument quoting and escaping rules. The arguments are passed to the application without any extra quoting, just a single space between each.

> **Note:** Present since VirtualBox 4.3.28 and 5.0 beta 3.

7.101 ProcessInputFlag

Guest process input flags.

None No flag set.

EndOfFile End of file (input) reached.

7.102 ProcessInputStatus

Process input statuses.

Undefined Undefined state.

Broken Input pipe is broken.

Available Input pipe became available for writing.

Written Data has been successfully written.

Overflow Too much input data supplied, data overflow.

7.103 ProcessOutputFlag

Guest process output flags for specifying which type of output to retrieve.

None No flags set. Get output from stdout.

StdErr Get output from stderr.

7.104 ProcessPriority

Process priorities.

Invalid Invalid priority, do not use.

Default Default process priority determined by the OS.

7.105 ProcessStatus

Process execution statuses.

Undefined Process is in an undefined state.

Starting Process is being started.

Started Process has been started.

Paused Process has been paused.

Terminating Process is being terminated.

TerminatedNormally Process terminated normally.

TerminatedSignal Process terminated via signal.

TerminatedAbnormally Process terminated abnormally.

TimedOutKilled Process timed out and was killed.

TimedOutAbnormally Process timed out and was not killed successfully.

Down Service/OS is stopping, process was killed.

Error Something went wrong.

7.106 ProcessWaitForFlag

Process waiting flags.

None No waiting flags specified. Do not use this.

Start Wait for the process being started.

Terminate Wait for the process being terminated.

StdIn Wait for stdin becoming available. Not implemented; will currently return WaitFlagNotSupported.

Writing process input can be performed via IProcess::write() or waiting for the IGuestProcessInputNotifyEvent event.

StdOut Wait for data becoming available on stdout. Not implemented; will currently return WaitFlagNotSupported.

Reading process stdout output can be performed via IProcess::read() or waiting for the IGuestProcessOutputEvent event.

StdErr Wait for data becoming available on stderr. Not implemented; will currently return WaitFlagNotSupported.

Reading process stderr output can be performed via IProcess::read() or waiting for the IGuestProcessOutputEvent event.

7.107 ProcessWaitResult

Process waiting results. Depending on the process waiting flags (for more information see ProcessWaitForFlag) the waiting result can vary based on the processes' current status.

To wait for a guest process to terminate after it has been created by IGuestSession::processCreate() or IGuestSession::processCreateEx() one would specify ProcessWaitFor_Terminate.

If a guest process has been started with ProcessCreateFlag_WaitForStdOut a client can wait with ProcessWaitResult_StdOut for new data to arrive on stdout; same applies for ProcessCreateFlag_WaitForStdErr and ProcessWaitResult_StdErr.

None No result was returned. Not being used.

Start The process has been started.

Terminate The process has been terminated.

Status The process has changed its status. The status then can be retrieved via IProcess::status.

Error Error while executing the process.

Timeout The waiting operation timed out. Also use if the guest process has timed out in the guest side (kill attempted).

StdIn The process signalled that stdin became available for writing.

StdOut Data on stdout became available for reading.

StdErr Data on stderr became available for reading.

WaitFlagNotSupported A waiting flag specified in the IProcess::waitFor() call is not supported by the guest.

7.108 ProcessorFeature

CPU features.

HWVirtEx

PAE

LongMode

NestedPaging

UnrestrictedGuest

NestedHWVirt

VirtVmsaveVmload

7.109 ProxyMode

Proxy setting: System (default), NoProxy and Manual. ISystemProperties::proxyMode

System Use the system proxy settings as far as possible.

NoProxy Direct connection to the Internet.

Manual Use the manual proxy from ISystemProperties::proxyURL.

7.110 Reason

Internal event reason type.

Unspecified Null value, means “no known reason”.

HostSuspend Host is being suspended (power management event).

HostResume Host is being resumed (power management event).

HostBatteryLow Host is running low on battery (power management event).

Snapshot A snapshot of the VM is being taken.

7.111 RecordingAudioCodec

Recording audio codec enumeration.

None No codec set.

WavPCM WAV format, linear PCM, uncompressed.

MP3 MPEG-2 Audio Layer III.

OggVorbis Ogg Vorbis.

Opus Opus Audio.

Other Other codec.

7.112 RecordingCodecDeadline

Recording codec deadline.
How (and if at all) this deadline is being implemented depends on the codec being used.

Default Default deadline.

Realtime Realtime quality, often resulting in poor / basic quality.

Good Balance between realtime and best deadline.

Best Best quality, slowest.

7.113 RecordingDestination

Recording destination enumeration.

None No destination.

File Destination is a regular file.

7.114 RecordingFeature

Recording feature flags. More than one flag may be set.

None No feature set.

Video Video recording.

Audio Audio recording.

7.115 RecordingRateControlMode

Recording video rate control mode enumeration.

> **Note:** Not all codecs may support all rate control modes.

ABR Average bit rate (ABR). Constrained to a certain size.

CBR Constant bit rate (CBR). Constrained to a certain size.

VBR Variable bit rate (VBR). Constrained to a certain quality.

7.116 RecordingState

Recording state enumeration.

> **Note:** Not all states might be available / implemented for all codecs.

Unknown Recording state is unknown currently, i.e. if no recording has been ever started yet.

Initializing Recording has been requested to start (enabled) and is initializing.

Started Recording has been started and is recording.

Paused Recording has been paused.

Resumed Recording has been resumed from the paused state.

Finalizing Recording has been requested to stop and currently is in its finalizing stage.

Stopped Recording has been stopped.

Canceled Recording has been canceled.

LimitReached A recording limit has been reached (time / size).

Error Recording has reached an unrecoverable error state and can't continue.

7.117 RecordingVideoCodec

Recording video codec enumeration.

None No codec set.

MJPEG Motion JPEG.

H262 MPEG-2, Part 2.

H264 Advanced Video Coding (AVC), MPEG-4 Part 10.

H265 High Efficiency Video Coding (HEVC), MPEG-H Part 2.

H266 Versatile Video Coding (VVC), MPEG-I Part 3.

VP8 VP8 codec.

VP9 VP9 codec.

AV1 AOMedia Video 1 codec.

Other Other codec.

7.118 RecordingVideoScalingMode

Recording video scaling method enumeration.

None No scaling performed. The image will be cropped when the source is bigger than the target size.

NearestNeighbor Performs scaling via nearest-neighbor interpolation. Not yet implemented.

Bilinear Performs scaling via bilinear interpolation. Not yet implemented.

Bicubic Performs scaling via bicubic interpolation. Not yet implemented.

7.119 Scope

Scope of the operation.
A generic enumeration used in various methods to define the action or argument scope.

Global

Machine

Session

7.120 ScreenLayoutMode

How IDisplay::setScreenLayout() method should work.

Apply If the guest is already at desired mode then the API might avoid setting the mode.

Reset Always set the new mode even if the guest is already at desired mode.

Attach Attach new screens and always set the new mode for existing screens.

Silent Do not notify the guest of the change. Normally this is wished, but it might not be when re-setting monitor information from the last session (no hotplug happened, as it is still the same virtual monitor).

7.121 SessionState

Session state. This enumeration represents possible values of IMachine::sessionState and ISession::state attributes.

Null Null value (never used by the API).

Unlocked In IMachine::sessionState, this means that the machine is not locked for any sessions.

In ISession::state, this means that no machine is currently locked for this session.

Locked In IMachine::sessionState, this means that the machine is currently locked for a session, whose process identifier can then be found in the IMachine::sessionPID attribute.

In ISession::state, this means that a machine is currently locked for this session, and the mutable machine object can be found in the ISession::machine attribute (see IMachine::lockMachine() for details).

Spawning A new process is being spawned for the machine as a result of IMachine::launchVMProcess() call. This state also occurs as a short transient state during an IMachine::lockMachine() call.

Unlocking The session is being unlocked.

7.122 SessionType

Session type. This enumeration represents possible values of the ISession::type attribute.

Null Null value (never used by the API).

WriteLock Session has acquired an exclusive write lock on a machine using IMachine::lockMachine().

Remote Session has launched a VM process using IMachine::launchVMProcess()

Shared Session has obtained a link to another session using IMachine::lockMachine()

7.123 SettingsVersion

Settings version of VirtualBox settings files. This is written to the “version” attribute of the root “VirtualBox” element in the settings file XML and indicates which VirtualBox version wrote the file.

Null Null value, indicates invalid version.

v1_0 Legacy settings version, not currently supported.

v1_1 Legacy settings version, not currently supported.

v1_2 Legacy settings version, not currently supported.

v1_3pre Legacy settings version, not currently supported.

v1_3 Settings version “1.3”, written by VirtualBox 2.0.12.

v1_4 Intermediate settings version, understood by VirtualBox 2.1.x.

v1_5 Intermediate settings version, understood by VirtualBox 2.1.x.

v1_6 Settings version “1.6”, written by VirtualBox 2.1.4 (at least).

v1_7 Settings version “1.7”, written by VirtualBox 2.2.x and 3.0.x.

v1_8 Intermediate settings version “1.8”, understood by VirtualBox 3.1.x.

v1_9 Settings version “1.9”, written by VirtualBox 3.1.x.

v1_10 Settings version “1.10”, written by VirtualBox 3.2.x.

v1_11 Settings version “1.11”, written by VirtualBox 4.0.x.

v1_12 Settings version “1.12”, written by VirtualBox 4.1.x.

v1_13 Settings version “1.13”, written by VirtualBox 4.2.x.

v1_14 Settings version “1.14”, written by VirtualBox 4.3.x.

v1_15 Settings version “1.15”, written by VirtualBox 5.0.x.

v1_16 Settings version “1.16”, written by VirtualBox 5.1.x.

v1_17 Settings version “1.17”, written by VirtualBox 6.0.x.

v1_18 Settings version “1.18”, written by VirtualBox 6.1.x.

v1_19 Settings version “1.19”, written by VirtualBox 7.0.x.

v1_20 Settings version “1.20”, written by VirtualBox 7.1.x.

v1_21 Settings version “1.21”, written by VirtualBox 7.2.x.

Future Settings version greater than “1.21”, written by a future VirtualBox version.

7.124 SignatureType

UEFI signature type enumeration.

X509 X.509 certificate.

Sha256 SHA256 hash.

7.125 StorageBus

The bus type of the storage controller (IDE, SATA, SCSI, SAS or Floppy); see IStorageController::bus.

Null `null` value. Never used by the API.

IDE

SATA

SCSI

Floppy

SAS

USB

PCIe

VirtioSCSI

7.126 StorageControllerType

The exact variant of storage controller hardware presented to the guest; see IStorageController::controllerType.

Null `null` value. Never used by the API.

LsiLogic A SCSI controller of the LsiLogic variant.

BusLogic A SCSI controller of the BusLogic variant.

IntelAhci An Intel AHCI SATA controller; this is the only variant for SATA.

PIIX3 An IDE controller of the PIIX3 variant.

PIIX4 An IDE controller of the PIIX4 variant.

ICH6 An IDE controller of the ICH6 variant.

I82078 A floppy disk controller; this is the only variant for floppy drives.

LsiLogicSas A variant of the LsiLogic controller using SAS.

USB Special USB based storage controller.

NVMe An NVMe storage controller.

VirtioSCSI Virtio SCSI storage controller.

7.127 SymlinkPolicy

Shared Folder Symbolic Link Security Policies

None No symlink policy set (never used by the API).

Forbidden Users in the guest VM are not able to create symbolic links inside the Shared Folder.

AllowedInShareSubtree Users in the guest VM are allowed to create symbolic links within the Shared Folder so long as the target remains in the Shared Folder or within the subtree of the Shared Folder.

AllowedToRelativeTargets Users in the guest VM are allowed to create symbolic links within the Shared Folder to targets within the Shared Folder and its subtree as well as to relative targets (../) outside of the Shared Folder.

AllowedToAnyTarget Users in the guest VM are allowed to create symbolic links within the Shared Folder and its subtree as well as to other targets via either relative pathnames or absolute pathnames.

7.128 SymlinkReadFlag

Symbolic link reading flags.

None No flags set.

NoSymlinks Don't allow symbolic links as part of the path.

7.129 SymlinkType

Symbolic link types. This is significant when creating links on the Windows platform, ignored elsewhere.

Unknown It is not known what is being targeted.

Directory The link targets a directory.

File The link targets a file (or whatever else except directories).

7.130 TouchContactState

Touch event contact state.

None The touch has finished.

InContact Whether the touch is really touching the device.

InRange Whether the touch is close enough to the device to be detected.

ContactStateMask

7.131 TpmType

TPM type enumeration.

None No TPM present in the virtual machine.

v1_2 A TPM compliant to the 1.2 TCG specification is emulated.

v2_0 A TPM compliant to the 2.0 TCG specification is emulated.

Host The host TPM is passed through, might not be available on all host platforms.

Swtpm The VM will attach to an external software TPM emulation compliant to swtpm.

7.132 TrackedObjectState

Tracked Object State.

None

Alive

Deleted

Invalid

7.133 USBConnectionSpeed

USB device/port speed state. This enumeration represents speeds at which a USB device can communicate with the host.

The speed is a function of both the device itself and the port which it is attached to, including hubs and cables in the path.

> **Note:** Due to differences in USB stack implementations on various hosts, the reported speed may not exactly match the actual speed.

See also: IHostUSBDevice

Null `null` value. Never returned by the API.

Low Low speed, 1.5 Mbps.

Full Full speed, 12 Mbps.

High High speed, 480 Mbps.

Super SuperSpeed, 5 Gbps.

SuperPlus SuperSpeedPlus, 10 Gbps.

7.134 USBControllerType

The USB controller type. IUSBController::type.

Null `null` value. Never used by the API.

OHCI

EHCI

XHCI

Last Last element (invalid). Used for parameter checks.

7.135 USBDeviceFilterAction

Actions for host USB device filters. See also: IHostUSBDeviceFilter, USBDeviceState

Null Null value (never used by the API).

Ignore Ignore the matched USB device.

Hold Hold the matched USB device.

7.136 USBDeviceState

USB device state. This enumeration represents all possible states of the USB device physically attached to the host computer regarding its state on the host computer and availability to guest computers (all currently running virtual machines).

Once a supported USB device is attached to the host, global USB filters (IHost::USBDeviceFilters[]) are activated. They can either ignore the device, or put it to USBDeviceState_Held state, or do nothing. Unless the device is ignored by global filters, filters of all currently running guests (IUSBDeviceFilters::deviceFilters[]) are activated that can put it to USBDeviceState_Captured state.

If the device was ignored by global filters, or didn't match any filters at all (including guest ones), it is handled by the host in a normal way. In this case, the device state is determined by the host and can be one of USBDeviceState_Unavailable, USBDeviceState_Busy or USBDeviceState_Available, depending on the current device usage.

Besides auto-capturing based on filters, the device can be manually captured by guests (IConsole::attachUSBDevice()) if its state is USBDeviceState_Busy, USBDeviceState_Available or USBDeviceState_Held.

> **Note:** Due to differences in USB stack implementations in Linux and Win32, states USBDeviceState_Busy and USBDeviceState_Unavailable are applicable only to the Linux version of the product. This also means that (IConsole::attachUSBDevice()) can only succeed on Win32 if the device state is USBDeviceState_Held.

See also: IHostUSBDevice, IHostUSBDeviceFilter

NotSupported Not supported by the VirtualBox server, not available to guests.

Unavailable Being used by the host computer exclusively, not available to guests.

Busy Being used by the host computer, potentially available to guests.

Available Not used by the host computer, available to guests (the host computer can also start using the device at any time).

Held Held by the VirtualBox server (ignored by the host computer), available to guests.

Captured Captured by one of the guest computers, not available to anybody else.

7.137 UartType

The UART type represents the emulated UART chip for the serial port device.

U16450 The most basic emulated UART which doesn't support FIFO operation.

U16550A The successor of the 16450 UART introducing a 16 byte FIFO to reduce operational overhead.

U16750 This UART developed by Texas Instruments introduced a 64 byte FIFO and hardware flow control.

7.138 UefiVariableAttributes

Possible variable attributes. More than one flag may be set. See also: IUefiVariableStore

None No attributes set.

NonVolatile Variable is stored in non volatile storage.

BootServiceAccess Variable can be accessed from the boot services.

RuntimeAccess Variable can be accessed from the runtime.

HwErrorRecord Variable contains a hardware error record.

AuthWriteAccess Writing to this variable requires an authenticated source.

AuthTimeBasedWriteAccess Variable was written with a time based authentication.

AuthAppendWrite The variable can be appended only.

7.139 UpdateChannel

Invalid No channel specified. Do not use this.

Stable All stable releases (maintenance and minor releases within the same major release).

All All stable releases, including major versions.

WithBetas All stable and major releases, including beta versions.

WithTesting All stable, major and beta releases, including testing versions.

7.140 UpdateSeverity

Invalid No severity specified. Do not use this.

Critical Update contains critical patches.

Major Update contains a major version.

Minor Update contains a minor version.

Testing Update contains a testing version. Use with caution!

7.141 UpdateState

Invalid Invalid / not set update state.

Available An uppdate is available.

NotAvailable No update available.

Downloading Update is being downloaded currently.

Downloaded Update has been successfully downloaded.

Installing Update is being installed currently.

Installed Update has been successfully installed.

UserInteraction Update requires user interaction to continue.

Canceled Update has been canceled.

Maintenance Update service currently is in maintenance mode.

Error An error occurred while updating.

7.142 VBoxEventType

Type of an event. See IEvent for an introduction to VirtualBox event handling.

Invalid Invalid event, must be first.

Any Wildcard for all events. Events of this type are never delivered, and only used in IEventSource::registerListener() call to simplify registration.

Vetoable Wildcard for all vetoable events. Events of this type are never delivered, and only used in IEventSource::registerListener() call to simplify registration.

MachineEvent Wildcard for all machine events. Events of this type are never delivered, and only used in IEventSource::registerListener() call to simplify registration.

SnapshotEvent Wildcard for all snapshot events. Events of this type are never delivered, and only used in IEventSource::registerListener() call to simplify registration.

InputEvent Wildcard for all input device (keyboard, mouse) events. Events of this type are never delivered, and only used in IEventSource::registerListener() call to simplify registration.

LastWildcard Last wildcard.

OnMachineStateChanged See IMachineStateChangedEvent.

OnMachineDataChanged See IMachineDataChangedEvent.

OnExtraDataChanged See IExtraDataChangedEvent.

OnExtraDataCanChange See IExtraDataCanChangeEvent.

OnMediumRegistered See IMediumRegisteredEvent.

OnMachineRegistered See IMachineRegisteredEvent.

OnSessionStateChanged See ISessionStateChangedEvent.

OnSnapshotTaken See ISnapshotTakenEvent.

OnSnapshotDeleted See ISnapshotDeletedEvent.

OnSnapshotChanged See ISnapshotChangedEvent.

OnGuestPropertyChanged See IGuestPropertyChangedEvent.

OnMousePointerShapeChanged See IMousePointerShapeChangedEvent.

OnMouseCapabilityChanged See IMouseCapabilityChangedEvent.

OnKeyboardLedsChanged See IKeyboardLedsChangedEvent.

OnStateChanged See IStateChangedEvent.

OnAdditionsStateChanged See IAdditionsStateChangedEvent.

OnNetworkAdapterChanged See INetworkAdapterChangedEvent.

OnSerialPortChanged See ISerialPortChangedEvent.

OnParallelPortChanged See IParallelPortChangedEvent.

OnStorageControllerChanged See IStorageControllerChangedEvent.

OnMediumChanged See IMediumChangedEvent.

OnVRDEServerChanged See IVRDEServerChangedEvent.

OnUSBControllerChanged See IUSBControllerChangedEvent.

OnUSBDeviceStateChanged See IUSBDeviceStateChangedEvent.

OnSharedFolderChanged See ISharedFolderChangedEvent.

OnRuntimeError See IRuntimeErrorEvent.

OnCanShowWindow See ICanShowWindowEvent.

OnShowWindow See IShowWindowEvent.

OnCPUChanged See ICPUChangedEvent.

OnVRDEServerInfoChanged See IVRDEServerInfoChangedEvent.

OnEventSourceChanged See IEventSourceChangedEvent.

OnCPUExecutionCapChanged See ICPUExecutionCapChangedEvent.

OnGuestKeyboard See IGuestKeyboardEvent.

OnGuestMouse See IGuestMouseEvent.

OnNATRedirect See INATRedirectEvent.

OnHostPCIDevicePlug See IHostPCIDevicePlugEvent.

OnVBoxSVCAvailabilityChanged See IVBoxSVCAvailabilityChangedEvent.

OnBandwidthGroupChanged See IBandwidthGroupChangedEvent.

OnGuestMonitorChanged See IGuestMonitorChangedEvent.

OnStorageDeviceChanged See IStorageDeviceChangedEvent.

OnClipboardModeChanged See IClipboardModeChangedEvent.

OnDnDModeChanged See IDnDModeChangedEvent.

OnNATNetworkChanged See INATNetworkChangedEvent.

OnNATNetworkStartStop See INATNetworkStartStopEvent.

OnNATNetworkAlter See INATNetworkAlterEvent.

OnNATNetworkCreationDeletion See INATNetworkCreationDeletionEvent.

OnNATNetworkSetting See INATNetworkSettingEvent.

OnNATNetworkPortForward See INATNetworkPortForwardEvent.

OnGuestSessionStateChanged See IGuestSessionStateChangedEvent.

OnGuestSessionRegistered See IGuestSessionRegisteredEvent.

OnGuestProcessRegistered See IGuestProcessRegisteredEvent.

OnGuestProcessStateChanged See IGuestProcessStateChangedEvent.

OnGuestProcessInputNotify See IGuestProcessInputNotifyEvent.

OnGuestProcessOutput See IGuestProcessOutputEvent.

OnGuestFileRegistered See IGuestFileRegisteredEvent.

OnGuestFileStateChanged See IGuestFileStateChangedEvent.

OnGuestFileOffsetChanged See IGuestFileOffsetChangedEvent.

OnGuestFileRead See IGuestFileReadEvent.

OnGuestFileWrite See IGuestFileWriteEvent.

OnRecordingStateChanged See IRecordingStateChangedEvent.

OnGuestUserStateChanged See IGuestUserStateChangedEvent.

OnGuestMultiTouch See IGuestMultiTouchEvent.

OnHostNameResolutionConfigurationChange See IHostNameResolutionConfigurationChangeEvent.

OnSnapshotRestored See ISnapshotRestoredEvent.

OnMediumConfigChanged See IMediumConfigChangedEvent.

OnAudioAdapterChanged See IAudioAdapterChangedEvent.

OnProgressPercentageChanged See IProgressPercentageChangedEvent.

OnProgressTaskCompleted See IProgressTaskCompletedEvent.

OnCursorPositionChanged See ICursorPositionChangedEvent.

OnGuestAdditionsStatusChanged See IGuestAdditionsStatusChangedEvent.

OnGuestMonitorInfoChanged See IGuestMonitorInfoChangedEvent.

OnGuestFileSizeChanged See IGuestFileSizeChangedEvent.

OnClipboardFileTransferModeChanged See IClipboardFileTransferModeChangedEvent.

OnCloudProviderListChanged See ICloudProviderListChangedEvent.

OnCloudProviderRegistered See ICloudProviderRegisteredEvent.

OnCloudProviderUninstall See ICloudProviderUninstallEvent.

OnCloudProfileRegistered See ICloudProfileRegisteredEvent.

OnCloudProfileChanged See ICloudProfileChangedEvent.

OnProgressCreated See IProgressCreatedEvent.

OnLanguageChanged See ILanguageChangedEvent.

OnUpdateAgentAvailable See IUpdateAgentAvailableEvent.

OnUpdateAgentError See IUpdateAgentErrorEvent.

OnUpdateAgentSettingsChanged See IUpdateAgentSettingsChangedEvent.

OnUpdateAgentStateChanged See IUpdateAgentStateChangedEvent.

OnHostAudioDeviceChanged See IHostAudioDeviceChangedEvent.

OnGuestDebugControlChanged See IGuestDebugControlChangedEvent.

OnMachineGroupsChanged See IMachineGroupsChangedEvent.

OnGuestDirectoryRegistered See IGuestDirectoryRegisteredEvent.

OnGuestDirectoryStateChanged See IGuestDirectoryStateChangedEvent.

OnGuestDirectoryRead See IGuestDirectoryReadEvent.

OnClipboardError See IClipboardErrorEvent.

OnExtPackInstalled See IExtPackInstalledEvent.

OnExtPackUninstalled See IExtPackUninstalledEvent.

OnRecordingScreenStateChanged See IRecordingScreenStateChangedEvent.

End Must be last event, used for iterations and structures relying on numerical event values.

7.143 VFSType

Virtual file systems supported by VFSExplorer.

File

Cloud

S3

WebDav

7.144 VMExecutionEngine

The main execution engine of a VM.

NotSet Has not yet been set (try again later), for reading the actual used execution engine from IMachineDebugger::executionEngine. This can not be set through IMachine::VMExecutionEngine.

Default Use the default/best available option for the host. This will never be returned by IMachineDebugger::executionEngine.

HwVirt Hardware assisted virtualization through HM. Only for x86-based hosts.

NativeApi Hardware assisted virtualization thru native API (NEM).

Interpreter CPU emulation using the interpreter.

Recompiler CPU emulation using the recompiler.

7.145 VMProcPriority

Virtual machine process priorities.

Invalid Invalid priority, do not use.

Default Default process priority determined by the OS.

Flat Assumes a scheduling policy which puts the process at the default priority and with all threads at the same priority

Low Assumes a scheduling policy which puts the process mostly below the default priority of the host OS.

Normal Assume a scheduling policy which shares the CPU resources fairly with other processes running with the default priority of the host OS.

High Assumes a scheduling policy which puts the task above the default priority of the host OS. This policy might easily cause other tasks in the system to starve.

7.146 VirtualSystemDescriptionType

Used with IVirtualSystemDescription to describe the type of a configuration value.

Ignore

OS

Name

Product

Vendor

Version

ProductUrl

VendorUrl

Description

License

Miscellaneous

CPU

Memory

HardDiskControllerIDE

HardDiskControllerSATA

HardDiskControllerSCSI

HardDiskControllerSAS

HardDiskImage

Floppy

CDROM

NetworkAdapter

USBController

SoundCard

SettingsFile Optional, may be unset by the API caller. If this is changed by the API caller it defines the absolute path of the VM settings file and therefore also the VM folder with highest priority.

BaseFolder Optional, may be unset by the API caller. If set (and SettingsFile is not changed), defines the VM base folder (taking the primary group into account if also set).

PrimaryGroup Optional, empty by default and may be unset by the API caller. Defines the primary group of the VM after import. May influence the selection of the VM folder. Additional groups may be configured later using IMachine::groups[], after importing.

CloudInstanceShape

CloudDomain

CloudBootDiskSize

CloudBucket

CloudOCIVCN

CloudPublicIP

CloudProfileName

CloudOCISubnet

CloudKeepObject

CloudLaunchInstance

CloudInstanceId

CloudImageId

CloudInstanceState

CloudImageState

CloudInstanceDisplayName

CloudImageDisplayName

CloudOCILaunchMode

CloudPrivateIP

CloudBootVolumeId

CloudOCIVCNCompartment

CloudOCISubnetCompartment

CloudPublicSSHKey

BootingFirmware

CloudInitScriptPath

CloudCompartmentId

CloudShapeCpus

CloudShapeMemory

CloudInstanceMetadata

CloudInstanceFreeFormTags

CloudImageFreeFormTags

HardDiskControllerVirtioSCSI

HardDiskControllerNVMe

NVRAM

7.147 VirtualSystemDescriptionValueType

Used with IVirtualSystemDescription::getValuesByType() to describe the value type to fetch.

Reference

Original

Auto

ExtraConfig

8 Working with the Cloud

VirtualBox supports and goes towards the Oracle tendencies like “move to the Cloud”.

8.1 OCI features

VirtualBox supports the Oracle Cloud Infrastructure (OCI). See the interfaces: ICloudClient, ICloudProvider, ICloudProfile, ICloudProviderManager.

Each cloud interface has own implementation to support OCI features. There are everal functions in the implementation which should be explained in details because OCI requires some special data or settings.

Also see the enumeration VirtualSystemDescriptionType for the possible values.

8.2 Function ICloudClient::exportVM

See the ICloudClient::exportVM. The function exports an existing virtual machine into OCI. The final result of this operation is creation a custom image from the bootable image of VM. The Id of created image is returned in the parameter “description” (which is IVirtualSystemDescription) as an entry with the type VirtualSystemDescriptionType::CloudImageId. The standard steps here are:

- Upload VBox image to OCI Object Storage.
- Create OCI custom image from the uploaded object.

Parameter “description” must contain all information and settings needed for creation a custom image in OCI. At least next entries must be presented there before the call:

- VirtualSystemDescriptionType::Name - Name of new instance in OCI.
- VirtualSystemDescriptionType::HardDiskImage - The local path or id of bootable VM image.
- VirtualSystemDescriptionType::CloudBucket - A cloud bucket name where the exported image is uploaded.
- VirtualSystemDescriptionType::CloudImageDisplayName - A name which is assigned to a new custom image in the OCI.
- VirtualSystemDescriptionType::CloudKeepObject - Whether keep or delete an uploaded object after its usage.
- VirtualSystemDescriptionType::CloudLaunchInstance - Whether launch or not a new instance.

8.3 Function ICloudClient::launchVM

See the ICloudClient::launchVM. The function launches a new instance in OCI with a bootable volume previously created from a custom image in OCI or as the source may be used an existing bootable volume which shouldn't be attached to any instance. For launching instance from a custom image use the parameter VirtualSystemDescriptionType::CloudImageId. For launching instance from a bootable volume use the parameter VirtualSystemDescriptionType::CloudBootVolumeId. Only one of them must be presented otherwise the error will occur. The final result of this operation is a running instance. The id of created instance is returned in the parameter "description" (which is IVirtualSystemDescription) as an entry with the type VirtualSystemDescriptionType::CloudInstanceId. Parameter "description" must contain all information and settings needed for creation a new instance in OCI. At least next entries must be presented there before the call:

- VirtualSystemDescriptionType::Name - Name of new instance in OCI.
- VirtualSystemDescriptionType::CloudOCISubnet - OCID of existing subnet in OCI which will be used by the instance.
- Use VirtualSystemDescriptionType::CloudImageId - OCID of custom image used as a bootable image for the instance or VirtualSystemDescriptionType::CloudBootVolumeId - OCID of existing and non-attached bootable volume used as a bootable volume for the instance.
- Add VirtualSystemDescriptionType::CloudBootDiskSize - The size of instance bootable volume in GB, If you use VirtualSystemDescriptionType::CloudImageId.
- VirtualSystemDescriptionType::CloudInstanceShape - The shape of instance according to OCI documentation, defines the number of CPUs and RAM memory.
- VirtualSystemDescriptionType::CloudLaunchInstance - Whether launch or not a new instance.
- VirtualSystemDescriptionType::CloudDomain - Availability domain in OCI where new instance is created.
- VirtualSystemDescriptionType::CloudPublicIP - Whether the instance will have a public IP or not.
- VirtualSystemDescriptionType::CloudPublicSSHKey - Public SSH key which is used to connect to an instance via SSH. It may be one or more records with the type VirtualSystemDescriptionType::CloudPublicSSHKey in the VirtualSystemDescription. But at least one should be presented otherwise user won't be able to connect to the instance via SSH.

8.4 Function ICloudClient::getInstanceInfo

See the ICloudClient::getInstanceInfo. The function takes an instance id (parameter "uid"), finds the requested instance in OCI and gets back information about the found instance in the parameter "description" (which is IVirtualSystemDescription) The entries with next types will be presented in the object:

- VirtualSystemDescriptionType::Name - Displayed name of the instance.
- VirtualSystemDescriptionType::CloudDomain - Availability domain in OCI.
- VirtualSystemDescriptionType::CloudImageId - Name of custom image used for creation this instance.

- VirtualSystemDescriptionType::CloudInstanceId - The OCID of the instance.
- VirtualSystemDescriptionType::OS - Guest OS type of the instance.
- VirtualSystemDescriptionType::CloudBootDiskSize - Size of instance bootable image.
- VirtualSystemDescriptionType::CloudInstanceState - The instance state according to OCI documentation.
- VirtualSystemDescriptionType::CloudInstanceShape - The instance shape according to OCI documentation
- VirtualSystemDescriptionType::Memory - RAM memory in GB allocated for the instance.
- VirtualSystemDescriptionType::CPU - Number of virtual CPUs allocated for the instance.

8.5 Function ICloudClient::importInstance

See the ICloudClient::importInstance. The API function imports an existing instance from the OCI to the local host. The standard steps here are:

- Create a custom image from an existing OCI instance.
- Export the custom image to OCI object (the object is created in the OCI Object Storage).
- Download the OCI object to the local host.

As the result of operation user will have a file with the suffix ".oci" on the local host. This file is a TAR archive which contains a bootable instance image in QCOW2 format and a JSON file with some metadata related to the imported instance. The function takes the parameter "description" (which is IVirtualSystemDescription) Parameter "description" must contain all information and settings needed for successful operation result. At least next entries must be presented there before the call:

- VirtualSystemDescriptionType::Name is used for the several purposes:
  - As a custom image name. A custom image is created from an instance.
  - As OCI object name. An object is a file in OCI Object Storage. The object is created from the custom image.
  - Name of imported instance on the local host. Because the result of import is a file, the file will have this name and extension ".oci".
- VirtualSystemDescriptionType::CloudInstanceId - The OCID of the existing instance.
- VirtualSystemDescriptionType::CloudBucket - a cloud bucket name in OCI Object Storage where created an OCI object from a custom image.

9 Host-Guest Communication Manager

The VirtualBox Host-Guest Communication Manager (HGCM) allows a guest application or a guest driver to call a host shared library. The following features of VirtualBox are implemented using HGCM:

- Shared Folders
- Shared Clipboard
- Guest configuration interface

The shared library on the host contains a so called HGCM service. The guest HGCM client establishes a connection to the HGCM service on the host in order to call that HGCM service's entry point. When calling an HGCM service the client supplies a function code, the number of parameters for the function, and an array of the parameter arguments.

9.1 Virtual hardware implementation

HGCM uses the Virtual Machine Monitor (VMM) virtual PCI device to exchange data between the guest and the host. The guest always acts as an initiator of requests. A request is constructed in the guest's physical memory which must be locked by the guest. The physical address is passed to the VMM device using a 32-bit `out edx, eax` instruction. The physical memory must be allocated below 4GB on 64-bit guests.

The host parses the request header and data and queues the request for the corresponding host HGCM service type. The guest continues execution and usually waits on an HGCM event semaphore.

When the request has been processed by the HGCM service, the VMM device sets the completion flag in the request header, sets the HGCM event and raises an IRQ for the guest. The IRQ handler signals the HGCM event semaphore and all HGCM callers check the completion flag in the corresponding request header. If the flag is set, the request is considered completed.

9.2 Protocol specification

The HGCM protocol definitions are contained in the `VBox/VBoxGuest.h` header file.

9.2.1 Request header

HGCM request structures contains a generic header (VMMDevHGCMRequestHeader):

Name	Description
size	Size of the entire request.
version	Version of the header, must be set to `0x10001`.
type	Type of the request.
rc	HGCM return code, which will be set by the VMM device.
reserved1	A reserved field 1.
reserved2	A reserved field 2.
flags	HGCM flags, set by the VMM device.
result	The HGCM result code, set by the VMM device.

Note:

- All fields are 32-bit.
- Fields from `size` to `reserved2` are a standard VMM device request header, which is used for other interfaces as well.

The **type** field indicates the type of the HGCM request:

Name (decimal value)	Description
VMMDevReq_HGCMConnect (60)	Connect to an HGCM service.
VMMDevReq_HGCMDisconnect (61)	Disconnect from the service.
VMMDevReq_HGCMCall32 (62)	Call an HGCM function using the 32-bit interface.
VMMDevReq_HGCMCall64 (63)	Call an HGCM function using the 64-bit interface.
VMMDevReq_HGCMCancel (64)	Cancel an HGCM request currently being processed by a host HGCM service.

The **flags** field may contain:

Name (hexadecimal value)	Description
VBOX_HGCM_REQ_DONE (0x00000001)	The request has been processed by the host service.
VBOX_HGCM_REQ_CANCELLED (0x00000002)	This request was cancelled.

9.2.2 Connect

The connection request must be issued by the guest HGCM client before it can call the HGCM service (VMMDevHGCMConnect):

Name	Description
header	The generic HGCM request header with type equal to VMMDevReq_HGCMConnect (60).
type	The type of the service location information (32 bit).
location	The service location information (128 bytes).
clientId	The client identifier assigned to the connecting client by the HGCM subsystem (32-bit).

The **type** field tells the HGCM how to look for the requested service:

Name (hexadecimal value)	Description
VMMDevHGCMLoc_LocalHost (0x1)	The requested service is a shared library located on the host and the location information contains the library name.
VMMDevHGCMLoc_LocalHost_Existing (0x2)	The requested service is a preloaded one and the location information contains the service name.

Note: Currently preloaded HGCM services are hard-coded in VirtualBox:

- VBoxSharedFolders
- VBoxSharedClipboard
- VBoxGuestPropSvc
- VBoxSharedOpenGL

There is no difference between both types of HGCM services, only the location mechanism is different.

The client identifier is returned by the host and must be used in all subsequent requests by the client.

9.2.3 Disconnect

This request disconnects the client and makes the client identifier invalid (VMMDevHGCMDisconnect):

Name	Description
header	The generic HGCM request header with type equal to VMMDevReq_HGCMDisconnect (61).
clientId	The client identifier previously returned by the connect request (32-bit).

9.2.4 Call32 and Call64

Calls the HGCM service entry point (VMMDevHGCMCall) using 32-bit or 64-bit addresses:

Name	Description
header	The generic HGCM request header with type equal to either VMMDevReq_HGCMCall32 (62) or VMMDevReq_HGCMCall64 (63).
clientId	The client identifier previously returned by the connect request (32-bit).
function	The function code to be processed by the service (32 bit).
cParms	The number of following parameters (32-bit). This value is 0 if the function requires no parameters.
parms	An array of parameter description structures (HGCMFunctionParameter32 or HGCMFunctionParameter64).

The 32-bit parameter description (HGCMFunctionParameter32) consists of a 32-bit type field and 8 bytes of an opaque value, so 12 bytes in total. The 64-bit variant (HGCMFunctionParameter64) consists of the 32-bit type and 12 bytes of a value, so 16 bytes in total.

Type	Format of the value
VMMDevHGCMParm-Type_32bit (1)	A 32-bit value.
VMMDevHGCMParm-Type_64bit (2)	A 64-bit value.
VMMDevHGCMParm-Type_PhysAddr (3)	A 32-bit size followed by a 32-bit or 64-bit guest physical address.
VMMDevHGCMParm-Type_LinAddr (4)	A 32-bit size followed by a 32-bit or 64-bit guest linear address. The buffer is used both for guest to host and for host to guest data.
VMMDevHGCMParm-Type_LinAddr_In (5)	Same as VMMDevHGCMParmType_LinAddr but the buffer is used only for host to guest data.
VMMDevHGCMParm-Type_LinAddr_Out (6)	Same as VMMDevHGCMParmType_LinAddr but the buffer is used only for guest to host data.
VMMDevHGCMParm-Type_LinAddr_Locked (7)	Same as VMMDevHGCMParmType_LinAddr but the buffer is already locked by the guest.
VMMDevHGCMParm-Type_LinAddr_Locked_In (1)	Same as VMMDevHGCMParmType_LinAddr_In but the buffer is already locked by the guest.
VMMDevHGCMParm-Type_LinAddr_Locked_Out (1)	Same as VMMDevHGCMParmType_LinAddr_Out but the buffer is already locked by the guest.

9.2.5 Cancel

This request cancels a call request (VMMDevHGCMCancel):

Name	Description
header	The generic HGCM request header with type equal to VMMDevReq_HGCMCancel (64).

9.3 Guest software interface

Guest HGCM clients can call HGCM services from both drivers and applications.

9.3.1 The guest driver interface

The driver interface is implemented in the VirtualBox guest additions driver (VBoxGuest) which works with the VMM virtual device. Drivers must use the VBox Guest Library (VBGL) which provides an API for HGCM clients (`VBox/VBoxGuestLib.h` and `VBox/VBoxGuest.h`). The key VBGL API routines are as follows:

```
DECLR0VBGL(int) VbglR0HGCMConnect(VBGLHGCMHANDLE *pHandle, const char *pszServiceName, HGCMCLIENTID *pidClient);
```

VbglR0HGCMConnect() connects to the HGCM service on the host. An example of a guest driver connecting to the VirtualBox Shared Folder HGCM service follows:

```
VBoxGuestHGCMConnectInfo data;
```

```
    memset(&data, sizeof(VBoxGuestHGCMConnectInfo));

    data.result   = VINF_SUCCESS;
    data.Loc.type = VMMDevHGCMLoc_LocalHost_Existing;
    strcpy(data.Loc.u.host.achName, "VBoxSharedFolders");

    rc = VbglR0HGCMConnect(&handle, "VBoxSharedFolders"&, data);

    if (RT_SUCCESS(rc))
    {
        rc = data.result;
    }

    if (RT_SUCCESS(rc))
    {
        /* Get the assigned client identifier. */
        ulClientID = data.u32ClientID;
    }
```

```
DECLVBGL(int) VbglR0HGCMDisconnect(VBGLHGCMHANDLE handle, VBoxGuestHGCMDisconnectInfo *pData);
```

VbglR0HGCMDisconnect() disconnects from the HGCM service on the host. An example of a guest driver disconnecting from an HGCM service using the client identifier from an earlier call to VbglR0HGCMConnect() follows:

```
    VBoxGuestHGCMDisconnectInfo data;

    RtlZeroMemory(&data, sizeof (VBoxGuestHGCMDisconnectInfo));

    data.result      = VINF_SUCCESS;
    data.u32ClientID = ulClientID;

    rc = VbglR0HGCMDisconnect(handle, &data);
```

```
DECLVBGL(int) VbglR0HGCMCall(VBGLHGCMHANDLE handle, VBoxGuestHGCMCallInfo *pData, uint32_t cbData);
```

VbglR0HGCMCall() calls a function specified in the VBoxGuestHGCMCallInfo argument in the HGCM service on the host. An example of a guest driver calling the Shared Folders HGCM service to issue a read of an object in a shared folder follows:

```
typedef struct _VBoxSFRead
{
    VBoxGuestHGCMCallInfo callInfo;

    /** pointer, in: SHFLROOT
     * Root handle of the mapping which name is queried.
     */
    HGCMFunctionParameter root;

    /** value64, in:
     * SHFLHANDLE of object to read from.
     */
    HGCMFunctionParameter handle;

    /** value64, in:
     * Offset to read from.
     */
```

```
    HGCMFunctionParameter offset;

    /** value64, in/out:
     * Bytes to read/How many were read.
     */
    HGCMFunctionParameter cb;

    /** pointer, out:
     * Buffer to place data to.
     */
    HGCMFunctionParameter buffer;

} VBoxSFRead;

/** Number of parameters */
#define SHFL_CPARMS_READ (5)

...

    VBoxSFRead data;

    /* The call information. */
    data.callInfo.result      = VINF_SUCCESS;     /* Will be returned by HGCM. */
    data.callInfo.u32ClientID = ulClientID;       /* Client identifier. */
    data.callInfo.u32Function = SHFL_FN_READ;     /* The function code. */
    data.callInfo.cParms      = SHFL_CPARMS_READ; /* Number of parameters. */

    /* Initialize parameters. */
    data.root.type                      = VMMDevHGCMParmType_32bit;
    data.root.u.value32                 = pMap->root;

    data.handle.type                    = VMMDevHGCMParmType_64bit;
    data.handle.u.value64               = hFile;

    data.offset.type                    = VMMDevHGCMParmType_64bit;
    data.offset.u.value64               = offset;

    data.cb.type                        = VMMDevHGCMParmType_32bit;
    data.cb.u.value32                   = *pcbBuffer;

    data.buffer.type                    = VMMDevHGCMParmType_LinAddr_Out;
    data.buffer.u.Pointer.size          = *pcbBuffer;
    data.buffer.u.Pointer.u.linearAddr  = (uintptr_t)pBuffer;

    rc = VbglR0HGCMCall(handle, &data.callInfo, sizeof (data));

    if (RT_SUCCESS(rc))
    {
        rc = data.callInfo.result;
        *pcbBuffer = data.cb.u.value32; /* This is returned by the HGCM service. */
    }
```

9.3.2 Guest application interface

Applications call the VirtualBox Guest Additions driver to utilize the HGCM interface. The following IOCTLs correspond to the `VbglR0HGCM*` functions listed above:

- `VBGL_IOCTL_HGCM_CONNECT => VbglR0HGCMConnect()`
- `VBGL_IOCTL_HGCM_DISCONNECT => VbglR0HGCMDisconnect()`
- `VBGL_IOCTL_HGCM_CALL => VbglR0HGCMCall()`

These IOCTLs get the same input buffer as the VbglR0HGCM* functions and the output buffer has the same format as the input buffer. The same address can be used for both the input and output buffers.

9.4 HGCM Service Implementation

The HGCM service is a shared library with a specific set of entry points. The library must export the VBoxHGCMSvcLoad entry point:

```
extern "C" DECLCALLBACK(DECLEXPORT(int)) VBoxHGCMSvcLoad(VBOXHGCMSVCFNTABLE *ptable)
```

The service must check the ptable->cbSize and ptable->u32Version fields of the input structure and fill in the remaining fields with function pointers of service entry points (listed below) and the size of the required client buffer size.

The HGCM service gets a dedicated thread which calls service entry points synchronously, thus the service entry point will only be called again once a previous call has returned. However, the guest calls can be processed asynchronously. Therefore the service must call a completion callback when the operation is actually completed. The callback can be issued from another thread as well.

Service entry points are listed in the VBox/hgcmsvc.h in the VBOXHGCMSVCFNTABLE structure.

Entry Point	Description
int pfnUnload(void *pvService)	The service is being unloaded.
int pfnConnect(void *pvService, uint32_t u32ClientID, void *pvClient, uint32_t fRequestor, bool fRestoring)	A client u32ClientID is connected to the service. The pvClient parameter points to an allocated memory buffer which can be used by the service to store the client information.
int pfnDisconnect(void *pvService, uint32_t u32ClientID, void *pvClient)	A client is being disconnected.
void pfnCall(void *pvService, VBOXHGCMCALLHANDLE callHandle, uint32_t u32ClientID, void *pvClient, uint32_t function, uint32_t cParms, VBOXHGCMSVCPARM paParms[], uint64_t tsArrival)	A guest client calls a service function. The callHandle must be used in the VBOXHGCMSVCHELPERS::pfnCallComplete callback when the call has been processed.
int pfnHostCall(void *pvService, uint32_t function, uint32_t cParms, VBOXHGCMSVCPARM paParms[])	Called by the VirtualBox host components to perform functions which should be not accessible by the guest. Usually this entry point is used by VirtualBox to configure the service.
int pfnSaveState(void *pvService, uint32_t u32ClientID, void *pvClient, PSSMHANDLE pSSM, PCVMMR3VTABLE pVMM)	The VM state is being saved and the service must save relevant information using the SSM API (VBox/ssm.h).
int pfnLoadState(void *pvService, uint32_t u32ClientID, void *pvClient, PSSMHANDLE pSSM, PCVMMR3VTABLE pVMM, uint32_t uVersion)	The VM is being restored from the saved state and the service must load the saved information and be able to continue operations from the saved state.

10 RDP Web Control

The VirtualBox *RDP Web Control* (RDPWeb) provides remote access to a running VM. RDPWeb is an RDP (Remote Desktop Protocol) client based on Flash technology and can be used from a Web browser with a Flash plugin.

10.1 RDPWeb features

RDPWeb is embedded into a Web page and connects to a VRDP server in order to display the remote VM screen and pass keyboard and mouse events to the VM.

10.2 RDPWeb reference

RDPWeb consists of two required components:

- Flash movie file `RDPClientUI.swf`
- JavaScript helpers contained in `webclient.js`

The VirtualBox SDK contains sample HTML code including:

- A JavaScript library for embedding Flash content: `SWFObject.js`
- A sample HTML page: `webclient3.html`

10.2.1 RDPWeb functions

`RDPClientUI.swf` and `webclient.js` work together to provide the RDP Web Control functionality. The JavaScript code is responsible for proper Flash initialization, delivering mouse events to the Flash object, and processing resize requests from the Flash object. On the other hand, the SWF file contains a few JavaScript callable methods, which are used both from `webclient.js` and the user HTML page.

10.2.1.1 JavaScript functions

The `webclient.js` file contains several helper JavaScript functions. In the following table ElementId refers to an HTML element name or attribute, and Element to the HTML element itself. The HTML code

```
<div id="FlashRDP">
</div>
```

would have ElementId equal to FlashRDP and Element equal to the div element.

- `RDPWebClient.embedSWF(SWFFileName, ElementId)`

  Uses the open-source SWFObject library to replace the HTML element with the Flash movie.

- `RDPWebClient.isRDPWebControlById(ElementId)`

  Returns true if the given ElementId refers to an RDPWeb Flash element.

- `RDPWebClient.isRDPWebControlByElement(Element)`

  Returns true if the given Element is an RDPWeb Flash element.

- `RDPWebClient.getFlashById(ElementId)`

  Returns an element, which is referenced by the given ElementId. This function will try to resolve any element, even if it is not a Flash movie.

10.2.1.2 Flash methods callable from JavaScript

The `RDPWebClienUI.swf` methods can be called directly from JavaScript code on an HTML page:

- getProperty(Name)
- setProperty(Name)
- connect()
- disconnect()
- keyboardSendCAD()

10.2.1.3 Flash JavaScript callbacks

`RDPWebClienUI.swf` calls JavaScript functions provided by the HTML page.

10.2.2 Embedding RDPWeb in an HTML page

It is necessary to include the `webclient.js` helper script. If the SWFObject library is used, the `swfobject.js` must also be included. Using the SWFObject library allows RDPWeb flash content to be embedded in a Web page using dynamic HTML. The HTML must include a "placeholder", which consists of 2 `div` elements.

10.3 RDPWeb change log

10.3.1 Version 1.2.28

- `keyboardLayout`, `keyboardLayouts`, `UUID` properties.
- Support for German keyboard layout on the client.
- Rebranding to Oracle.

10.3.2 Version 1.1.26

- `webclient.js` is a part of the distribution package.
- `lastError` property.
- `keyboardSendScancodes` and `keyboardSendCAD` methods.

10.3.3 Version 1.0.24

- Initial release.

11 Drag and Drop

As of VirtualBox 4.2 it's possible to transfer files from the host to Linux, Solaris, and macOS guests by dragging files, directories, or text from the host into the guest's screen. This is called *drag and drop (DnD)*.

VirtualBox 5.0 added support for Windows guests as well as the ability to transfer data in the opposite direction, that is, from the guest to the host.

Note: Currently only the VirtualBox Manager front-end supports drag and drop.

This chapter will show how to use the required interfaces provided by VirtualBox for adding drag and drop functionality to third-party front-ends.

11.1 Basic concepts

In order to use the interfaces provided by VirtualBox, some basic concepts need to be understood first: a drag and drop operation logically contains both a *source* and a *target*:

The *source* provides the data, i.e., it is the origin of data. This data can be stored within the source directly or it can be retrieved on-demand by the source itself.

The *target* on the other hand provides a visual representation to the source where the user can drop the data the source offers. This representation can be a window (or just a certain part of it), for example.

The source and the target have abstract interfaces called IDnDSource and IDnDTarget. VirtualBox also provides implementations of both interfaces, called IGuestDnDSource and IGuestDnDTarget. Both implementations are used in the VirtualBox Manager front-end.

11.2 Supported formats

As the target needs to perform specific actions depending on the data the source provided, the target first needs to know what type of data it is actually going to retrieve. It might be that the source offers data the target cannot (or intentionally does not want to) support.

VirtualBox describes the data types using *MIME types* – which were originally defined in RFC 2046 and are also called *Content-types* or *media types*. IGuestDnDSource and IGuestDnDTarget support the following MIME types by default:

- **text/uri-list** - A list of URIs (Uniform Resource Identifier, see RFC 3986) pointing to the file and/or directory paths already transferred from the source to the target.
- **text/plain;charset=utf-8** and **UTF8_STRING** - text in UTF-8 format.
- **text/plain, TEXT** and **STRING** - plain ASCII text, depending on the source's active ANSI page (if any).

If, for whatever reason, a certain default format should not be supported or a new format should be registered, IDnDSource and IDnDTarget have methods derived from IDnDBase which provide adding, removing and enumerating specific formats.

Note: Registering new or removing default formats on the guest side is not currently implemented.

12 VirtualBox external authentication modules

VirtualBox supports arbitrary external modules to perform authentication. External authentication modules are used for remote desktop access to a VM when the VRDE authentication type is set to “external”. VRDE authentication will then use the authentication module which was specified with `VBoxManage setproperty vrdeauthlibrary`. The web service will use the external authentication module specified with `VBoxManage setproperty websrvauthlibrary`.

This library will be loaded by the VM or web service process on demand, i.e. when the first remote desktop connection is made by a client or when a client that wants to use the web service logs on.

External authentication is the most flexible authentication type since the external handler can choose to both grant access to everyone (like the “null” authentication method) as well as delegate the request to the guest authentication component. When delegating the request to the guest component the external handler will still be called afterwards with the option to override the result.

An authentication library is required to implement exactly one entry point:

```
#include "VBoxAuth.h"

/**
 * Authentication library entry point.
 *
 * Parameters:
 *
 *   szCaller         The name of the component which calls the library (UTF8).
 *   pUuid            Pointer to the UUID of the accessed virtual machine. Can be NULL.
 *   guestJudgement   Result of the guest authentication.
 *   szUser           User name passed in by the client (UTF8).
 *   szPassword       Password passed in by the client (UTF8).
 *   szDomain         Domain passed in by the client (UTF8).
 *   fLogon           Boolean flag. Indicates whether the entry point is called
 *                    for a client logon or the client disconnect.
 *   clientId         Server side unique identifier of the client.
 *
 * Return code:
 *
 *   AuthResultAccessDenied    Client access has been denied.
 *   AuthResultAccessGranted   Client has the right to use the
 *                             virtual machine.
 *   AuthResultDelegateToGuest Guest operating system must
 *                             authenticate the client and the
 *                             library must be called again with
 *                             the result of the guest
 *                             authentication.
 *
 * Note: When 'fLogon' is 0, only pszCaller, pUuid and clientId are valid and the return
 *       code is ignored.
 */
AuthResult AUTHCALL AuthEntry(
    const char *szCaller,
    PAUTHUUID pUuid,
    AuthGuestJudgement guestJudgement,
    const char *szUser,
    const char *szPassword
    const char *szDomain
```

```
    int fLogon,
    unsigned clientId)
{
    /* Process request against your authentication source of choice. */
    // if (authSucceeded(...))
    //     return AuthResultAccessGranted;
    return AuthResultAccessDenied;
}
```

A note regarding the UUID implementation of the `pUuid` argument: VirtualBox uses a consistent binary representation of UUIDs on all platforms. For this reason the integer fields comprising the UUID are stored as little endian values. If you want to pass such UUIDs to code which assumes that integer fields are big endian (often also called network byte order), you need to adjust the contents of the UUID to achieve the same string representation. The required changes are:

- reverse the order of bytes 0, 1, 2 and 3
- reverse the order of bytes 4 and 5
- reverse the order of bytes 6 and 7.

Using this conversion you will get identical results when converting the binary UUID to the string representation.

The `guestJudgement` argument contains information about the guest authentication status. For the first call, it is always set to `AuthGuestNotAsked`. If the `AuthEntry` function returns `AuthResultDelegateToGuest`, a guest authentication will be attempted and another call to the `AuthEntry` is made with its result. The guest authentication can return either granted, denied, or no judgement (the guest component chose for whatever reason to not make a decision). In case there is a problem with the guest authentication module (e.g. the Guest Additions are not installed or not running or the guest did not respond within a timeout), the “not reacted” status will be returned.

13 Using the Java API

13.1 Introduction

VirtualBox can be controlled by a Java API, both locally using COM/XPCOM and remotely using SOAP. As with the Python bindings, a generic glue layer tries to hide all platform differences, allowing for source and binary compatibility on different platforms.

13.2 Requirements

To use the Java bindings, there are certain requirements depending on the platform. First of all, you need JDK 1.5 (Java 5) or later. Also please make sure that the version of the VirtualBox API .jar file exactly matches the version of VirtualBox in use. To avoid confusion, the VirtualBox API provides versioning in the Java package name, e.g. the package is named `org.virtualbox_3_2` for VirtualBox version 3.2.

- **XPCOM** - for all platforms except Microsoft Windows. A Java bridge based on JavaXPCOM is shipped with VirtualBox. The classpath must contain `vboxjxpcom.jar` and the `vbox.home` property must be set to the location where the VirtualBox binaries are. Please make sure that the JVM bitness matches the bitness of VirtualBox in use as the XPCOM bridge relies on native libraries.

  Start your application like this:

  ```
  java -cp vboxjxpcom.jar -Dvbox.home=/opt/virtualbox MyProgram
  ```

- **COM** - for Microsoft Windows. We rely on `Jacob` - a generic Java to COM bridge - which has to be installed separately. See http://sourceforge.net/projects/jacob-project/ for installation instructions. Also, the VirtualBox provided `vboxjmscom.jar` file must be in the class path.

  Start your application like this:

  ```
  java -cp vboxjmscom.jar;c:\jacob\jacob.jar -Djava.library.path=c:\jacob MyProgram
  ```

- **SOAP** - all platforms. Java 6 is required as it comes with built-in support for SOAP via the JAX-WS library. Also, the VirtualBox provided `vbojws.jar` must be in the class path. In the SOAP case it's possible to create several VirtualBoxManager instances to communicate with multiple VirtualBox hosts.

  Start your application like this:

  ```
  java -cp vboxjws.jar MyProgram
  ```

Exception handling is also generalized by the generic glue layer, so that all methods can throw `VBoxException` containing a human-readable text message (see `getMessage()` method) along with the wrapped original exception (see `getWrapped()` method).

13.3 Example

This example shows a simple use case of the Java API. Differences for SOAP vs. local execution are minimal and limited to the connection setup phase (see `ws` variable). In the SOAP case it's possible to create several VirtualBoxManager instances to communicate with multiple VirtualBox hosts.

```
import org.virtualbox_4_3.*;
....
VirtualBoxManager mgr = VirtualBoxManager.createInstance(null);
boolean ws = false; // or true, if we need the SOAP version
if (ws)
{
    String url = "http://myhost:18034";
    String user = "test";
    String passwd = "test";
    mgr.connect(url, user, passwd);
}
IVirtualBox vbox = mgr.getVBox();
System.out.println("VirtualBox version: " + vbox.getVersion() + "\n");
// get first VM name
String m =  vbox.getMachines().get(0).getName();
System.out.println("\nAttempting to start VM '" + m + "'");
// start it
mgr.startVm(m, null, 7000);

if (ws)
    mgr.disconnect();

mgr.cleanup();
```

For a more complete example, see `TestVBox.java`, shipped with the SDK. It contains exception handling and error printing code which is important for reliable larger scale projects.

It is good practice in long-running API clients to process the system events every now and then in the main thread (this does not work in other threads). As a rule of thumb it makes sense to process them every few 100msec to every few seconds). This is done by calling

```
mgr.waitForEvents(0);
```

This helps prevent a large number of system events from accumulating which can need a significant amount of memory, and as they also play a role in object cleanup it helps freeing additional memory in a timely manner which is used by the API implementation itself. Java's garbage collection approach already needs more memory due to the delayed freeing of memory used by no longer accessible objects and not processing the system events exacerbates the memory usage. The `TestVBox.java` example code sprinkles such lines over the code to achieve the desired effect. In multi-threaded applications it can be called from the main thread periodically. Sometimes it's possible to use the non-zero timeout variant of the method, which then waits the specified number of milliseconds for events, processing them immediately as they arrive. It achieves better runtime behavior than separate sleeping/processing.

14 License information

The sample code files shipped with the SDK are generally licensed liberally to make it easy for anyone to use this code for their own application code.

The Java files under `sdk/bindings/webservice/java/jax-ws/` (library files for the object-oriented web service) are, by contrast, licensed under the GNU Lesser General Public License (LGPL) V2.1.

See `sdk/bindings/webservice/java/jax-ws/src/COPYING.LIB` for the full text of the LGPL 2.1.

When in doubt, please refer to the individual source code files shipped with this SDK.

15 Main API change log

Generally, VirtualBox will maintain API compatibility within a major release; a major release occurs when the first or the second of the three version components of VirtualBox change (that is, in the x.y.z scheme, a major release is one where x or y change, but not when only z changes).

In other words, updates like those from 2.0.0 to 2.0.2 will not come with API breakages.

Migration between major releases most likely will lead to API breakage, so please make sure you updated code accordingly. The OOWS Java wrappers enforce that mechanism by putting VirtualBox classes into version-specific packages such as `org.virtualbox_2_2`. This approach allows for connecting to multiple VirtualBox versions simultaneously from the same Java application.

The following sections list incompatible changes that the Main API underwent since the original release of this SDK Reference with VirtualBox 2.0. A change is deemed "incompatible" only if it breaks existing client code (e.g. changes in method parameter lists, renamed or removed interfaces and similar). In other words, the list does not contain new interfaces, methods or attributes or other changes that do not affect existing client code.

15.1 Incompatible API changes with version 7.2

- The attribute GraphicsFeature::Acceleration2DVideo has been removed.
- The attribute GuestOSType::recommended2DVideoAcceleration has been removed.
- The RecordingState enumeration has been added and is being used for the IRecordingStateChanged and IRecordingScreenStateChanged events.

15.2 Incompatible API changes with version 7.1

- Review the Main API changes made for the ARM64 CPU architecture here for incompatible changes to interfaces, methods, and attributes.
- The Python API bindings for Python 2.x is now marked as being deprecated and will be removed in a future version. Please upgrade your code to use Python 3.
- The Python API bindings now live in a separate `python` sub directory and also now support installing via the pip package manager (e.g. `pip -v install ./vboxapi`). This also allows installing into Python virtual environments.
- The Windows host installer now uses the sub directory `installer` instead of `install` for housing the bindings installers. This now matches the directory layout for the other platforms. Please adapt your scripts if needed.
- Guest process creation requires a new parameter for specifying the current working directory for the new guest process. This is optional and can be empty. See IGuestSession::processCreate and IGuestSession::processCreateEx for more information.
- The APIs IGuestSession::fsQueryInfo and IGuestSession::fsQueryFreeSpace are now implemented. See IGuestSession::fsQueryInfo and IGuestSession::fsQueryFreeSpace for more information.

- The APIs IGuestSession::waitFor and IProcess::waitFor are now marked as being deprecated. Use IGuestSession::waitForArray and IProcess::waitForArray instead.
- The attribute IGuestSession::mountPoints has been added. This requires 7.1 (or newer) Guest Additions to be installed on the guest.
- The attribute IHost::acceleration3DAvailable has been removed. Use ISystempProperties::supportedGraphicsFeatures instead.
- Starting a recording now has to be done via the newly introduced method IRecordingSettings::start. This method also will return a recording progress object on success, which also can be retrieved at a later time via the new attribute IRecordingSettings::progress.
- The recording settings attribute IRecordingSettings::enabled does not implicitly start or stop a recording anymore, but only will indicate that recording for a VM is enabled or disabled. This attribute also will be used to determine if recording will be started automatically on VM start or restore.
- The recording settings now expose a progress object attribute via IRecordingSettings::progress, which can be used to stop an ongoing recording, track the recording progress or getting notified of recording errors.

15.3 Incompatible API changes with version 7.0

- The machine's audio adapter has been moved into the new IAudioSettings interface, which in turn takes care of of all audio settings of a virtual machine. See IMachine::audioSettings and IAudioSettings for more information.
- The IVirtualBox::openMachine call now requires an additional password parameter. If the machine is not encrypted the parameter is ignored.
- When a new VM is being created, the default audio driver will be now AudioDriverType_Default. This driver type will automatically choose the best audio driver (backend) for the host OS Oracle® VirtualBox® currently is running on.
- The host update functionality at IHost::update has been refactored into IHost::updateHost, which in turn uses the new IHostUpdateAgent interface, derived from the new IUpdateAgent interface.
- IGuestSession::directoryCopyFromGuest() and IGuestSession::directoryCopyToGuest() no longer implicitly copy recursively and follow symbolic links – for this to continue working, the newly introduced flags DirectoryCopyFlag::Recursive and/or DirectoryCopyFlag::FollowLinks have to be used.
- VBoxEventType_Last has been renamed to VBoxEventType_End for consistency.

15.4 Incompatible API changes with version 6.1

- Split off the graphics adapter part of IMachine into IGraphicsAdapter. This moved 5 attributes.

15.5 Incompatible API changes with version 6.0

- Video recording APIs were changed as follows:
  - All attributes which were living in IMachine before have been moved to an own, dedicated interface named IRecordingSettings. This new interface can be accessed via the new IMachine::recordingSettings attribute. This should emphasize that recording is not limited to video capturing as such.
  - For further flexibility all specific per-VM-screen settings have been moved to a new interface called IRecordingScreenSettings. Such settings now exist per configured VM display and can be retrieved via the IRecordingSettings::screens attribute or the IRecordingSettings::getScreenSettings method.

    Note: For now all screen settings will share the same settings, e.g. different settings on a per-screen basis is not implemented yet.

  - The event `IVideoCaptureChangedEvent` was renamed into `IRecordingChangedEvent`.
- Guest Control APIs were changed as follows:
  - IGuest::createSession(), IGuestSession::processCreate(), IGuestSession::processCreateEx(), IGuestSession::directoryOpen() and IGuestSession::fileOpen() now will return the new error code VBOX_E_MAXIMUM_REACHED if the limit for the according object group has been reached.
  - The enumerations FileOpenExFlags, FsObjMoveFlags and DirectoryCopyFlags have been renamed to FileOpenExFlag, FsObjMoveFlag and DirectoryCopyFlag accordingly to match the rest of the API.
  - The following methods have been implemented: `IGuestSession::directoryCopyFromGuest()` and `IGuestSession::directoryCopyToGuest()`.

    The following attributes have been implemented: `IGuestFsObjInfo::accessTime`, `IGuestFsObjInfo::birthTime`, `IGuestFsObjInfo::changeTime` and `IGuestFsObjInfo::modificationTime`.
- The webservice version of the ISharedFolder interface was changed from a struct to a managed object. This causes incompatibilities on the protocol level as the shared folder attributes are not returned in the responses of IVirtualBox::getSharedFolders and IMachine::getSharedFolders anymore. They return object UUIDs instead which need be wrapped by stub objects. The change is not visible when using the appropriate client bindings from the most recent VirtualBox SDK.

15.6 Incompatible API changes with version 5.x

- ProcessCreateFlag::NoProfile has been renamed to ProcessCreateFlag::Profile, and the semantics have also been changed: ProcessCreateFlag::NoProfile explicitly **did not** utilize the guest user's profile data whereas ProcessCreateFlag::Profile explicitly **does** utilize the guest userâĂŹs profile data.

15.7 Incompatible API changes with version 5.1.28

- The Host-Guest Communication Manager (HGCM) guest driver interfaces were renamed:
  - VbglHGCMConnect() was renamed to VbglR0HGCMConnect()
  - VbglHGCMDisconnect() was renamed to VbglR0HGCMDisconnect()
  - VbglHGCMCall() was renamed to VbglR0HGCMCall()
- The Host-Guest Communication Manager (HGCM) guest application interface IOCTLs were renamed:
  - VBOXGUEST_IOCTL_HGCM_CONNECT was renamed to VBGL_IOCTL_HGCM_CONNECT
  - VBOXGUEST_IOCTL_HGCM_DISCONNECT was renamed to VBGL_IOCTL_HGCM_DISCONNECT
  - VBOXGUEST_IOCTL_HGCM_CALL was renamed to VBGL_IOCTL_HGCM_CALL

15.8 Incompatible API changes with version 5.0

- The methods for saving state, adopting a saved state file, discarding a saved state, taking a snapshot, restoring a snapshot and deleting a snapshot have been moved from `IConsole` to `IMachine`. This straightens out the logical placement of methods and was necessary to resolve a long-standing issue, preventing 32-bit API clients from invoking those operations in the case where no VM is running.
  - IMachine::saveState() replaces `IConsole::saveState()`
  - IMachine::adoptSavedState() replaces `IConsole::adoptSavedState()`
  - IMachine::discardSavedState() replaces `IConsole::discardSavedState()`
  - IMachine::takeSnapshot() replaces `IConsole::takeSnapshot()`
  - IMachine::deleteSnapshot() replaces `IConsole::deleteSnapshot()`
  - IMachine::deleteSnapshotAndAllChildren() replaces `IConsole::deleteSnapshotAndAllChildren()`
  - IMachine::deleteSnapshotRange() replaces `IConsole::deleteSnapshotRange()`
  - IMachine::restoreSnapshot() replaces `IConsole::restoreSnapshot()`

  Small adjustments to the parameter lists have been made to reduce the number of API calls when taking online snapshots (no longer needs explicit pausing), and taking a snapshot also returns now the snapshot id (useful for finding the right snapshot if there are non-unique snapshot names).
- Two new machine states have been introduced to allow proper distinction between saving state and taking a snapshot. MachineState::Saving now is used exclusively while the VM's state is being saved, without any overlaps with snapshot functionality. The new state MachineState::Snapshotting is used when an offline snapshot is taken and likewise the new state MachineState::OnlineSnapshotting is used when an online snapshot is taken.
- A new event has been introduced, which signals when a snapshot has been restored: ISnapshotRestoredEvent. Previously the event ISnapshotDeletedEvent was signalled, which isn't logical (but could be distinguished from actual deletion by the fact that the snapshot was still there).
- The method IVirtualBox::createMedium() replaces `VirtualBox::createHardDisk()`. Adjusting existing code needs adding two parameters with value `AccessMode_ReadWrite` and `DeviceType_HardDisk` respectively. The new method supports creating floppy and DVD images, and (less obviously) further API functionality such as cloning floppy images.

- The method IMachine::getStorageControllerByInstance() now has an additional parameter (first parameter), for specifying the storage bus which the storage controller must be using. The method was not useful before, as the instance numbers are only unique for a specfic storage bus.
- The attribute `IMachine::sessionType` has been renamed to IMachine::sessionName(). This cleans up the confusing terminology (as the session type is something different).
- The attribute `IMachine::guestPropertyNotificationPatterns` has been removed. In practice it was not usable because it is too global and didn't distinguish between API clients.
- Drag and drop APIs were changed as follows:
  - Methods for providing host to guest drag and drop functionality, such as `IGuest::dragHGEnter`, `IGuest::dragHGMove()`, `IGuest::dragHGLeave()`, `IGuest::dragHGDrop()` and `IGuest::dragHGPutData()`, have been moved to an abstract base class called IDnDTarget. VirtualBox implements this base class in the IGuestDnDTarget interface. The implementation can be used by using the IGuest::dnDTarget() method.

    Methods for providing guest to host drag and drop functionality, such as `IGuest::dragGHPending()`, `IGuest::dragGHDropped()` and `IGuest::dragGHGetData()`, have been moved to an abstract base class called IDnDSource. VirtualBox implements this base class in the IGuestDnDSource interface. The implementation can be used by using the IGuest::dnDSource() method.
  - The `DragAndDropAction` enumeration has been renamed to DnDAction.
  - The `DragAndDropMode` enumeration has been renamed to DnDMode.
  - The attribute `IMachine::dragAndDropMode` has been renamed to IMachine::dnDMode().
  - The event `IDragAndDropModeChangedEvent` has been renamed to IDnDModeChangedEvent.
- IDisplay and IFramebuffer interfaces were changed to allow IFramebuffer object to reside in a separate front-end process:
  - IDisplay::ResizeCompleted() has been removed, because the IFramebuffer object does not provide the screen memory anymore.
  - IDisplay::SetFramebuffer() has been replaced with IDisplay::AttachFramebuffer() and IDisplay::DetachFramebuffer().
  - IDisplay::GetFramebuffer() has been replaced with IDisplay::QueryFramebuffer().
  - IDisplay::GetScreenResolution() has a new output parameter `guestMonitorStatus` which tells whether the monitor is enabled in the guest.
  - IDisplay::TakeScreenShot() and IDisplay::TakeScreenShotToArray() have a new parameter `bitmapFormat`. As a consequence of this, IDisplay::TakeScreenShotPNGToArray() has been removed.
  - IFramebuffer::RequestResize() has been replaced with IFramebuffer::NotifyChange().
  - IFramebuffer::NotifyUpdateImage() added to support IFramebuffer objects in a different process.
  - IFramebuffer::Lock(), IFramebuffer::Unlock(), IFramebuffer::Address(), IFramebuffer::UsesGuestVRAM() have been removed because the IFramebuffer object does not provide the screen memory anymore.
- IGuestSession, IGuestFile and IGuestProcess interfaces were changed as follows:

- Replaced IGuestSession::directoryQueryInfo and IGuestSession::fileQueryInfo with a new IGuestSession::fsObjQueryInfo method that works on any type of file system object.
- Replaced IGuestSession::fileRemove, IGuestSession::symlinkRemoveDirectory and IGuestSession::symlinkRemoveFile with a new IGuestSession::fsObjRemove method that works on any type of file system object except directories. (fileRemove also worked on any type of object too, though that was not the intent of the method.)
- Replaced IGuestSession::directoryRename and IGuestSession::directoryRename with a new IGuestSession::fsObjRename method that works on any type of file system object. (directoryRename and fileRename may already have worked for any kind of object, but that was never the intent of the methods.)
- Replaced the unimplemented IGuestSession::directorySetACL and IGuestSession::fileSetACL with a new IGuestSession::fsObjSetACL method that works on all type of file system object. Also added a UNIX-style mode parameter as an alternative to the ACL.
- Replaced IGuestSession::fileRemove, IGuestSession::symlinkRemoveDirectory and IGuestSession::symlinkRemoveFile with a new IGuestSession::fsObjRemove method that works on any type of file system object except directories (fileRemove also worked on any type of object, though that was not the intent of the method.)
- Renamed IGuestSession::copyTo to IGuestSession::fileCopyToGuest.
- Renamed IGuestSession::copyFrom to IGuestSession::fileCopyFromGuest.
- Renamed the CopyFileFlag enum to FileCopyFlag.
- Renamed the IGuestSession::environment attribute to IGuestSession::environmentChanges to better reflect what it does.
- Changed the IGuestProcess::environment to a stub returning E_NOTIMPL since it wasn't doing what was advertised (returned changes, not the actual environment).
- Renamed IGuestSession::environmentSet to IGuestSession::environmentScheduleSet to better reflect what it does.
- Renamed IGuestSession::environmentUnset to IGuestSession::environmentScheduleUnset to better reflect what it does.
- Removed IGuestSession::environmentGet it was only getting changes while giving the impression it was actual environment variables, and it did not represent scheduled unset operations.
- Removed IGuestSession::environmentClear as it duplicates assigning an empty array to the IGuestSession::environmentChanges (formerly known as IGuestSession::environment).
- Changed the IGuestSession::processCreate and IGuestSession::processCreateEx methods to accept arguments starting with argument zero (argv[0]) instead of argument one (argv[1]). (Not yet implemented on the guest additions side, so argv[0] will probably be ignored for a short while.)
- Added a followSymlink parameter to the following methods:
  - IGuestSession::directoryExists
  - IGuestSession::fileExists
  - IGuestSession::fileQuerySize
- The parameters to the IGuestSession::fileOpen and IGuestSession::fileOpenEx methods were altered:

        * The openMode string parameter was replaced by the enum FileAccessMode and renamed to accessMode.
        * The disposition string parameter was replaced by the enum FileOpenAction and renamed to openAction.
        * The unimplemented sharingMode string parameter was replaced by the enum FileSharingMode (fileOpenEx only).
        * Added a flags parameter taking a list of FileOpenExFlag values (fileOpenEx only).
        * Removed the offset parameter (fileOpenEx only).
      - IGuestFile::seek now returns the new offset.
      - Renamed the FileSeekType enum used by IGuestFile::seek to FileSeekOrigin and added the missing End value and renaming the Set to Begin.
      - Extended the unimplemented IGuestFile::setACL method with a UNIX-style mode parameter as an alternative to the ACL.
      - Renamed the IFile::openMode attribute to IFile::accessMode and change the type from string to FileAccessMode to reflect the changes to the fileOpen methods.
      - Renamed the IGuestFile::disposition attribute to IFile::openAction and change the type from string to FileOpenAction to reflect the changes to the fileOpen methods.
      - Added IGuestSession::pathStyle attribute.
      - Added IGuestSession::fsObjExists attribute.
- IConsole::GetDeviceActivity() returns information about multiple devices.
- IMachine::ReadSavedThumbnailToArray() has a new parameter `bitmapFormat`. As a consequence of this, IMachine::ReadSavedThumbnailPNGToArray() has been removed.
- IMachine::QuerySavedScreenshotPNGSize() has been renamed to IMachine::QuerySavedScreenshotInfo() which also returns an array of available screenshot formats.
- IMachine::ReadSavedScreenshotPNGToArray() has been renamed to IMachine::ReadSavedScreenshotToArray() which has a new parameter `bitmapFormat`.
- IMachine::QuerySavedThumbnailSize() has been removed.
- The method IWebsessionManager::getSessionObject() now returns a new ISession instance for every invocation. This puts the behavior in line with other binding styles, which never forced the equivalent of establishing another connection and logging in again to get another instance.

15.9 Incompatible API changes with version 4.3

- The explicit medium locking methods IMedium::lockRead() and IMedium::lockWrite() have been redesigned. They return a lock token object reference now, and calling the IToken::abandon() method (or letting the reference count to this object drop to 0) will unlock it. This eliminates the rather common problem that an API client crash left behind locks, and also improves the safety (API clients can't release locks they didn't obtain).
- The parameter list of IAppliance::write() has been changed slightly, to allow multiple flags to be passed.
- `IMachine::delete` has been renamed to IMachine::deleteConfig(), to improve API client binding compatibility.

- `IMachine::export` has been renamed to IMachine::exportTo(), to improve API client binding compatibility.
- For IMachine::launchVMProcess() the meaning of the `type` parameter has changed slightly. Empty string now means that the per-VM or global default front-end is launched. Most callers of this method should use the empty string now, unless they really want to override the default and launch a particular front-end.
- Medium management APIs were changed as follows:
  - The type of attribute IMedium::variant() changed from `unsigned long` to `safe-array MediumVariant`. It is an array of flags instead of a set of flags which were stored inside one variable.
  - The parameter list for IMedium::cloneTo() was modified. The type of parameter variant was changed from unsigned long to safe-array MediumVariant.
  - The parameter list for IMedium::createBaseStorage() was modified. The type of parameter variant was changed from unsigned long to safe-array MediumVariant.
  - The parameter list for IMedium::createDiffStorage() was modified. The type of parameter variant was changed from unsigned long to safe-array MediumVariant.
  - The parameter list for IMedium::cloneToBase() was modified. The type of parameter variant was changed from unsigned long to safe-array MediumVariant.
- The type of attribute IMediumFormat::capabilities() changed from `unsigned long` to `safe-array MediumFormatCapabilities`. It is an array of flags instead of a set of flags which were stored inside one variable.
- The attribute IMedium::logicalSize() now returns the logical size of exactly this medium object (whether it is a base or diff image). The old behavior was no longer acceptable, as each image can have a different capacity.
- Guest control APIs - such as IGuest, IGuestSession, IGuestProcess and so on - now emit own events to provide clients much finer control and the ability to write own front-ends for guest operations. The event IGuestSessionEvent acts as an abstract base class for all guest control events. Certain guest events contain a IVirtualBoxErrorInfo member to provide more information in case of an error happened on the guest side.
- Guest control sessions on the guest started by IGuest::createSession() now are dedicated guest processes to provide more safety and performance for certain operations. Also, the IGuest::createSession() call does not wait for the guest session being created anymore due to the dedicated guest session processes just mentioned. This also will enable webservice clients to handle guest session creation more gracefully. To wait for a guest session being started, use the newly added attribute IGuestSession::status() to query the current guest session status.
- The IGuestFile APIs are now implemented to provide native guest file access from the host.
- The parameter list for IMedium::updateGuestAdditions() was modified. It now supports specifying optional command line arguments for the Guest Additions installer performing the actual update on the guest.
- A new event IGuestUserStateChangedEvent was introduced to provide guest user status updates to the host via event listeners. To use this event there needs to be at least the 4.3 Guest Additions installed on the guest. At the moment only the states “Idle” and “InUse” of the GuestUserState enumeration arei supported on Windows guests, starting at Windows 2000 SP2.

- The attribute IGuestSession::protocolVersion was added to provide a convenient way to lookup the guest session's protocol version it uses to communicate with the installed Guest Additions on the guest. Older Guest Additions will set the protocol version to 1, whereas Guest Additions 4.3 will set the protocol version to 2. This might change in the future as new features arise.
- `IDisplay::getScreenResolution` has been extended to return the display position in the guest.
- The IUSBController class is not a singleton of IMachine anymore but IMachine contains a list of USB controllers present in the VM. The USB device filter handling was moved to IUSBDeviceFilters.

15.10 Incompatible API changes with version 4.2

- Guest control APIs for executing guest processes, working with guest files or directories have been moved to the newly introduced IGuestSession interface which can be created by calling IGuest::createSession().

  A guest session will act as a guest user's impersonation so that the guest credentials only have to be provided when creating a new guest session. There can be up to 32 guest sessions at once per VM, each session serving up to 2048 guest processes running or files opened.

  Instead of working with process or directory handles before version 4.2, there now are the dedicated interfaces IGuestProcess, IGuestDirectory and IGuestFile. To retrieve more information of a file system object the new interface IGuestFsObjInfo has been introduced.

  Even though the guest control API was changed it is backwards compatible so that it can be used with older installed Guest Additions. However, to use upcoming features like process termination or waiting for input / output new Guest Additions must be installed when these features got implemented.

  The following limitations apply:

  - The IGuestFile interface is not fully implemented yet.
  - The symbolic link APIs IGuestSession::symlinkCreate(), IGuestSession::symlinkExists(), IGuestSession::symlinkRead(), IGuestSession::symlinkRemoveDirectory() and IGuestSession::symlinkRemoveFile() are not implemented yet.
  - The directory APIs IGuestSession::directoryRemove(), IGuestSession::directoryRemoveRecursive(), IGuestSession::directoryRename() and IGuestSession::directorySetACL() are not implemented yet.
  - The temporary file creation API IGuestSession::fileCreateTemp() is not implemented yet.
  - Guest process termination via IProcess::terminate() is not implemented yet.
  - Waiting for guest process output via ProcessWaitForFlag::StdOut and ProcessWaitForFlag::StdErr is not implemented yet.

    To wait for process output, IProcess::read() with appropriate flags still can be used to periodically check for new output data to arrive. Note that ProcessCreateFlag::WaitForStdOut and / or ProcessCreateFlag::WaitForStdErr need to be specified when creating a guest process via IGuestSession::processCreate() or IGuestSession::processCreateEx().
  - ACL (Access Control List) handling in general is not implemented yet.

- The LockType enumeration now has an additional value `VM` which tells IMachine::lockMachine() to create a full-blown object structure for running a VM. This was the previous behavior with `Write`, which now only creates the minimal object structure to save time and resources (at the moment the Console object is still created, but all sub-objects such as Display, Keyboard, Mouse, Guest are not.
- Machines can be put in groups (actually an array of groups). The primary group affects the default placement of files belonging to a VM. IVirtualBox::createMachine() and IVirtualBox::composeMachineFilename() have been adjusted accordingly, the former taking an array of groups as an additional parameter and the latter taking a group as an additional parameter. The create option handling has been changed for those two methods, too.
- The method IVirtualBox::findMedium() has been removed, since it provides a subset of the functionality of IVirtualBox::openMedium().
- The use of acronyms in API enumeration, interface, attribute and method names has been made much more consistent, previously they sometimes were lowercase and sometimes mixed case. They are now consistently all caps:

Old name	New name
PointingHidType	PointingHIDType
KeyboardHidType	KeyboardHIDType
IPciAddress	IPCIAddress
IPciDeviceAttachment	IPCIDeviceAttachment
IMachine::pointingHidType	IMachine::pointingHIDType
IMachine::keyboardHidType	IMachine::keyboardHIDType
IMachine::hpetEnabled	IMachine::HPETEnabled
IMachine::sessionPid	IMachine::sessionPID
IMachine::ioCacheEnabled	IMachine::IOCacheEnabled
IMachine::ioCacheSize	IMachine::IOCacheSize
IMachine::pciDeviceAssignments	IMachine::PCIDeviceAssignments
IMachine::attachHostPciDevice()	IMachine::attachHostPCIDevice
IMachine::detachHostPciDevice()	IMachine::detachHostPCIDevice()
IConsole::attachedPciDevices	IConsole::attachedPCIDevices
IHostNetworkInterface::dhcpEnabled	IHostNetworkInterface::DHCPEnabled
IHostNetworkInterface::enableStaticIpConfig()	IHostNetworkInterface::enableStaticIPConfig()
IHostNetworkInterface::enableStaticIpConfigV6()	IHostNetworkInterface::enableStaticIPConfigV6()
IHostNetworkInterface::enableDynamicIpConfig()	IHostNetworkInterface::enableDynamicIPConfig()
IHostNetworkInterface::dhcpRediscover()	IHostNetworkInterface::DHCPRediscover()
IHost::acceleration3DAvailable	
IGuestOSType::recommendedPae	IGuestOSType::recommendedPAE
IGuestOSType::recommendedDvdStorageController	IGuestOSType::recommendedDVDStorageController
IGuestOSType::recommendedDvdStorageBus	IGuestOSType::recommendedDVDStorageBus
IGuestOSType::recommendedHdStorageController	IGuestOSType::recommendedHDStorageController
IGuestOSType::recommendedHdStorageBus	IGuestOSType::recommendedHDStorageBus
IGuestOSType::recommendedUsbHid	IGuestOSType::recommendedUSBHID
IGuestOSType::recommendedHpet	IGuestOSType::recommendedHPET
IGuestOSType::recommendedUsbTablet	IGuestOSType::recommendedUSBTablet
IGuestOSType::recommendedRtcUseUtc	IGuestOSType::recommendedRTCUseUTC
IGuestOSType::recommendedUsb	IGuestOSType::recommendedUSB
INetworkAdapter::natDriver	INetworkAdapter::NATEngine
IUSBController::enabledEhci	IUSBController::enabledEHCI"
INATEngine::tftpPrefix	INATEngine::TFTPPrefix
INATEngine::tftpBootFile	INATEngine::TFTPBootFile
INATEngine::tftpNextServer	INATEngine::TFTPNextServer
INATEngine::dnsPassDomain	INATEngine::DNSPassDomain
INATEngine::dnsProxy	INATEngine::DNSProxy
INATEngine::dnsUseHostResolver	INATEngine::DNSUseHostResolver
VBoxEventType::OnHostPciDevicePlug	VBoxEventType::OnHostPCIDevicePlug
ICPUChangedEvent::cpu	ICPUChangedEvent::CPU
INATRedirectEvent::hostIp	INATRedirectEvent::hostIP
INATRedirectEvent::guestIp	INATRedirectEvent::guestIP
IHostPciDevicePlugEvent	IHostPCIDevicePlugEvent

15.11 Incompatible API changes with version 4.1

- The method IAppliance::importMachines() has one more parameter now, which allows to configure the import process in more detail.
- The method IVirtualBox::openMedium() has one more parameter now, which allows resolving duplicate medium UUIDs without the need for external tools.
- The INetworkAdapter interface has been cleaned up. The various methods to activate an attachment type have been replaced by the INetworkAdapter::attachmentType setter.

Additionally each attachment mode now has its own attribute, which means that host only networks no longer share the settings with bridged interfaces.

To allow introducing new network attachment implementations without making API changes, the concept of a generic network attachment driver has been introduced, which is configurable through key/value properties.

- This version introduces the guest facilities concept. A guest facility either represents a module or feature the guest is running or offering, which is defined by AdditionsFacilityType. Each facility is member of a AdditionsFacilityClass and has a current status indicated by AdditionsFacilityStatus, together with a timestamp (in ms) of the last status update.

  To address the above concept, the following changes were made:

  - In the IGuest interface, the following were removed:
    - the `supportsSeamless` attribute;
    - the `supportsGraphics` attribute;
  - The function IGuest::getFacilityStatus() was added. It quickly provides a facility's status without the need to get the facility collection with IGuest::facilities.
  - The attribute IGuest::facilities was added to provide an easy to access collection of all currently known guest facilities, that is, it contains all facilies where at least one status update was made since the guest was started.
  - The interface IAdditionsFacility was added to represent a single facility returned by IGuest::facilities.
  - AdditionsFacilityStatus was added to represent a facility's overall status.
  - AdditionsFacilityType and AdditionsFacilityClass were added to represent the facility's type and class.

15.12 Incompatible API changes with version 4.0

- A new Java glue layer replacing the previous OOWS JAX-WS bindings was introduced. The new library allows for uniform code targeting both local (COM/XPCOM) and remote (SOAP) transports. Now, instead of `IWebsessionManager`, the new class `VirtualBoxManager` must be used. See chapter 13, *Using the Java API*, page 499 for details.
- The confusingly named and impractical session APIs were changed. In existing client code, the following changes need to be made:
  - Replace any `IVirtualBox::openSession(uuidMachine, ...)` API call with the machine's IMachine::lockMachine() call and a `LockType.Write` argument. The functionality is unchanged, but instead of "opening a direct session on a machine" all documentation now refers to "obtaining a write lock on a machine for the client session".
  - Similarly, replace any `IVirtualBox::openExistingSession(uuidMachine, ...)` call with the machine's IMachine::lockMachine() call and a `LockType.Shared` argument. Whereas it was previously impossible to connect a client session to a running VM process in a race-free manner, the new API will atomically either write-lock the machine for the current session or establish a remote link to an existing session. Existing client code which tried calling both `openSession()` and `openExistingSession()` can now use this one call instead.
  - Third, replace any `IVirtualBox::openRemoteSession(uuidMachine, ...)` call with the machine's IMachine::launchVMProcess() call. The functionality is unchanged.

  - The SessionState enum was adjusted accordingly: “Open” is now “Locked”, “Closed” is now “Unlocked”, “Closing” is now “Unlocking”.

- Virtual machines created with VirtualBox 4.0 or later no longer register their media in the global media registry in the `VirtualBox.xml` file. Instead, such machines list all their media in their own machine XML files. As a result, a number of media-related APIs had to be modified again.
  - Neither `IVirtualBox::createHardDisk()` nor IVirtualBox::openMedium() register media automatically any more.
  - IMachine::attachDevice() and IMachine::mountMedium() now take an IMedium object instead of a UUID as an argument. It is these two calls which add media to a registry now (either a machine registry for machines created with VirtualBox 4.0 or later or the global registry otherwise). As a consequence, if a medium is opened but never attached to a machine, it is no longer added to any registry any more.
  - To reduce code duplication, the APIs IVirtualBox::findHardDisk(), getHardDisk(), findDVDImage(), getDVDImage(), findFloppyImage() and getFloppyImage() have all been merged into IVirtualBox::findMedium(), and IVirtualBox::openHardDisk(), openDVDImage() and openFloppyImage() have all been merged into IVirtualBox::openMedium().
  - The rare use case of changing the UUID and parent UUID of a medium previously handled by `openHardDisk()` is now in a separate IMedium::setIDs method.
  - `ISystemProperties::get/setDefaultHardDiskFolder()` have been removed since disk images are now by default placed in each machine’s folder.
  - The ISystemProperties::infoVDSize attribute replaces the `getMaxVDISize()` API call; this now uses bytes instead of megabytes.

- Machine management APIs were enhanced as follows:
  - IVirtualBox::createMachine() is no longer restricted to creating machines in the default “Machines” folder, but can now create machines at arbitrary locations. For this to work, the parameter list had to be changed.
  - The long-deprecated `IVirtualBox::createLegacyMachine()` API has been removed.
  - To reduce code duplication and for consistency with the aforementioned media APIs, `IVirtualBox::getMachine()` has been merged with IVirtualBox::findMachine(), and `IMachine::getSnapshot()` has been merged with IMachine::findSnapshot().
  - `IVirtualBox::unregisterMachine()` was replaced with IMachine::unregister() with additional functionality for cleaning up machine files.
  - `IMachine::deleteSettings` has been replaced by IMachine::delete, which allows specifying which disk images are to be deleted as part of the deletion, and because it can take a while it also returns a `IProgress` object reference, so that the completion of the asynchronous activities can be monitored.
  - `IConsole::forgetSavedState` has been renamed to `IConsole::discardSavedState()`.

- All event callbacks APIs were replaced with a new, generic event mechanism that can be used both locally (COM, XPCOM) and remotely (web services). Also, the new mechanism is usable from scripting languages and a local Java. See events for details. The new concept will require changes to all clients that used event callbacks.

- `additionsActive()` was replaced with additionsRunLevel() and getAdditionsStatus() in order to support a more detailed status of the current Guest Additions loading/readiness

state. IGuest::additionsVersion() no longer returns the Guest Additions interface version but the installed Guest Additions version and revision in form of `3.3.0r12345`.

- To address shared folders auto-mounting support, the following APIs were extended to require an additional `automount` parameter:
  - IVirtualBox::createSharedFolder()
  - IMachine::createSharedFolder()
  - IConsole::createSharedFolder()

  Also, a new property named `autoMount` was added to the ISharedFolder interface.

- The appliance (OVF) APIs were enhanced as follows:
  - `IMachine::export` received an extra parameter `location`, which is used to decide for the disk naming.
  - IAppliance::write() received an extra parameter `manifest`, which can suppress creating the manifest file on export.
  - IVFSExplorer::entryList() received two extra parameters `sizes` and `modes`, which contains the sizes (in bytes) and the file access modes (in octal form) of the returned files.

- Support for remote desktop access to virtual machines has been cleaned up to allow third party implementations of the remote desktop server. This is called the VirtualBox Remote Desktop Extension (VRDE) and can be added to VirtualBox by installing the corresponding extension package; see the VirtualBox User Guide for details.

  The following API changes were made to support the VRDE interface:
  - `IVRDPServer` has been renamed to IVRDEServer.
  - `IRemoteDisplayInfo` has been renamed to IVRDEServerInfo.
  - IMachine::VRDEServer replaces `VRDPServer`.
  - IConsole::VRDEServerInfo replaces `RemoteDisplayInfo`.
  - ISystemProperties::VRDEAuthLibrary replaces `RemoteDisplayAuthLibrary`.
  - The following methods have been implemented in `IVRDEServer` to support generic VRDE properties:
    * IVRDEServer::setVRDEProperty
    * IVRDEServer::getVRDEProperty
    * IVRDEServer::VRDEProperties

    A few implementation-specific attributes of the old `IVRDPServer` interface have been removed and replaced with properties:
    * `IVRDPServer::Ports` has been replaced with the `"TCP/Ports"` property. The property value is a string, which contains a comma-separated list of ports or ranges of ports. Use a dash between two port numbers to specify a range. Example: `"5000,5010-5012"`
    * `IVRDPServer::NetAddress` has been replaced with the `"TCP/Address"` property. The property value is an IP address string. Example: `"127.0.0.1"`
    * `IVRDPServer::VideoChannel` has been replaced with the `"VideoChannel/Enabled"` property. The property value is either `"true"` or `"false"`

    * `IVRDPServer::VideoChannelQuality` has been replaced with the `"VideoChannel/Quality"` property. The property value is a string which contain a decimal number in range 10..100. Invalid values are ignored and the quality is set to the default value 75. Example: `"50"`

- The VirtualBox external authentication module interface has been updated and made more generic. Because of that, `VRDPAuthType` enumeration has been renamed to AuthType.

15.13 Incompatible API changes with version 3.2

- The following interfaces were renamed for consistency:
  - IMachine::getCpuProperty() is now IMachine::getCPUProperty();
  - IMachine::setCpuProperty() is now IMachine::setCPUProperty();
  - IMachine::getCpuIdLeaf() is now IMachine::getCPUIDLeaf();
  - IMachine::setCpuIdLeaf() is now IMachine::setCPUIDLeaf();
  - IMachine::removeCpuIdLeaf() is now IMachine::removeCPUIDLeaf();
  - IMachine::removeAllCpuIdLeafs() is now IMachine::removeAllCPUIDLeaves();
  - the CpuPropertyType enum is now CPUPropertyType.
  - IVirtualBoxCallback::onSnapshotDiscarded() is now IVirtualBoxCallback::onSnapshotDeleted.
- When creating a VM configuration with IVirtualBox::createMachine() it is now possible to ignore existing configuration files which would previously have caused a failure. For this the `override` parameter was added.
- Deleting snapshots via `IConsole::deleteSnapshot()` is now possible while the associated VM is running in almost all cases. The API is unchanged, but client code that verifies machine states to determine whether snapshots can be deleted may need to be adjusted.
- The IoBackendType enumeration was replaced with a boolean flag (see IStorageController::useHostIOCache).
- To address multi-monitor support, the following APIs were extended to require an additional `screenId` parameter:
  - IMachine::querySavedThumbnailSize()
  - IMachine::readSavedThumbnailToArray()
  - IMachine::querySavedScreenshotPNGSize()
  - IMachine::readSavedScreenshotPNGToArray()
- The `shape` parameter of IConsoleCallback::onMousePointerShapeChange was changed from a implementation-specific pointer to a safearray, enabling scripting languages to process pointer shapes.

15.14 Incompatible API changes with version 3.1

- Due to the new flexibility in medium attachments that was introduced with version 3.1 (in particular, full flexibility with attaching CD/DVD drives to arbitrary controllers), we seized the opportunity to rework all interfaces dealing with storage media to make the API more flexible as well as logical. The IStorageController, IMedium, IMediumAttachment

and IMachine interfaces were affected the most. Existing code using them to configure storage and media needs to be carefully checked.

All media (hard disks, floppies and CDs/DVDs) are now uniformly handled through the IMedium interface. The device-specific interfaces (`IHardDisk`, `IDVDImage`, `IHostDVDDrive`, `IFloppyImage` and `IHostFloppyDrive`) have been merged into IMedium; CD/DVD and floppy media no longer need special treatment. The device type of a medium determines in which context it can be used. Some functionality was moved to the other storage-related interfaces.

`IMachine::attachHardDisk` and similar methods have been renamed and generalized to deal with any type of drive and medium. IMachine::attachDevice() is the API method for adding any drive to a storage controller. The floppy and DVD/CD drives are no longer handled specially, and that means you can have more than one of them. As before, drives can only be changed while the VM is powered off. Mounting (or unmounting) removable media at runtime is possible with IMachine::mountMedium().

Newly created virtual machines have no storage controllers associated with them. Even the IDE Controller needs to be created explicitly. The floppy controller is now visible as a separate controller, with a new storage bus type. For each storage bus type you can query the device types which can be attached, so that it is not necessary to hardcode any attachment rules.

This required matching changes e.g. in the callback interfaces (the medium specific change notification was replaced by a generic medium change notification) and removing associated enums (e.g. `DriveState`). In many places the incorrect use of the plural form "media" was replaced by "medium", to improve consistency.

- Reading the IMedium::state attribute no longer automatically performs an accessibility check; a new method IMedium::refreshState() does this. The attribute only returns the state now.

- There were substantial changes related to snapshots, triggered by the "branched snapshots" functionality introduced with version 3.1. IConsole::discardSnapshot was renamed to `IConsole::deleteSnapshot()`. IConsole::discardCurrentState and IConsole::discardCurrentSnapshotAndState were removed; corresponding new functionality is in `IConsole::restoreSnapshot()`. Also, when `IConsole::takeSnapshot()` is called on a running virtual machine, a live snapshot will be created. The old behavior was to temporarily pause the virtual machine while creating an online snapshot.

- The `IVRDPServer`, `IRemoteDisplayInfo"` and `IConsoleCallback` interfaces were changed to reflect VRDP server ability to bind to one of available ports from a list of ports.

  The `IVRDPServer::port` attribute has been replaced with `IVRDPServer::ports`, which is a comma-separated list of ports or ranges of ports.

  An `IRemoteDisplayInfo::port"` attribute has been added for querying the actual port VRDP server listens on.

  An IConsoleCallback::onRemoteDisplayInfoChange() notification callback has been added.

- The parameter lists for the following functions were modified:

  - IHost::removeHostOnlyNetworkInterface()
  - IHost::removeUSBDeviceFilter()

- In the OOWS bindings for JAX-WS, the behavior of structures changed: for one, we implemented natural structures field access so you can just call a "get" method to obtain a field. Secondly, setters in structures were disabled as they have no expected effect and were at best misleading.

15.15 Incompatible API changes with version 3.0

- In the object-oriented web service bindings for JAX-WS, proper inheritance has been introduced for some classes, so explicit casting is no longer needed to call methods from a parent class. In particular, IHardDisk and other classes now properly derive from IMedium.
- All object identifiers (machines, snapshots, disks, etc) switched from GUIDs to strings (now still having string representation of GUIDs inside). As a result, no particular internal structure can be assumed for object identifiers; instead, they should be treated as opaque unique handles. This change mostly affects Java and C++ programs; for other languages, GUIDs are transparently converted to strings.
- The uses of NULL strings have been changed greatly. All out parameters now use empty strings to signal a null value. For in parameters both the old NULL and empty string is allowed. This change was necessary to support more client bindings, especially using the web service API. Many of them either have no special NULL value or have trouble dealing with it correctly in the respective library code.
- Accidentally, the `TSBool` interface still appeared in 3.0.0, and was removed in 3.0.2. This is an SDK bug, do not use the SDK for VirtualBox 3.0.0 for developing clients.
- The type of IVirtualBoxErrorInfo::resultCode changed from `result` to `long`.
- The parameter list of IVirtualBox::openHardDisk was changed.
- The method IConsole::discardSavedState was renamed to IConsole::forgetSavedState, and a parameter was added.
- The method IConsole::powerDownAsync was renamed to IConsole::powerDown, and the previous method with that name was deleted. So effectively a parameter was added.
- In the IFramebuffer interface, the following were removed:
  - the `operationSupported` attribute;

    (as a result, the `FramebufferAccelerationOperation` enum was no longer needed and removed as well);
  - the `solidFill()` method;
  - the `copyScreenBits()` method.
- In the IDisplay interface, the following were removed:
  - the `setupInternalFramebuffer()` method;
  - the `lockFramebuffer()` method;
  - the `unlockFramebuffer()` method;
  - the `registerExternalFramebuffer()` method.

15.16 Incompatible API changes with version 2.2

- Added explicit version number into JAX-WS Java package names, such as `org.virtualbox_2_2`, allowing connect to multiple VirtualBox clients from single Java application.

- The interfaces having a “2” suffix attached to them with version 2.1 were renamed again to have that suffix removed. This time around, this change involves only the name, there are no functional differences.

  As a result, IDVDImage2 is now IDVDImage; IHardDisk2 is now IHardDisk; IHardDisk2Attachment is now IHardDiskAttachment.

  Consequentially, all related methods and attributes that had a “2” suffix have been renamed; for example, IMachine::attachHardDisk2 now becomes IMachine::attachHardDisk().

- IVirtualBox::openHardDisk has an extra parameter for opening a disk read/write or read-only.

- The remaining collections were replaced by more performant safe-arrays. This affects the following collections:
  - IGuestOSTypeCollection
  - IHostDVDDriveCollection
  - IHostFloppyDriveCollection
  - IHostUSBDeviceCollection
  - IHostUSBDeviceFilterCollection
  - IProgressCollection
  - ISharedFolderCollection
  - ISnapshotCollection
  - IUSBDeviceCollection
  - IUSBDeviceFilterCollection

- Since “Host Interface Networking” was renamed to “bridged networking” and host-only networking was introduced, all associated interfaces needed renaming as well. In detail:
  - The HostNetworkInterfaceType enum has been renamed to HostNetworkInterfaceMediumType
  - The IHostNetworkInterface::type attribute has been renamed to IHostNetworkInterface::mediumType
  - INetworkAdapter::attachToHostInterface() has been renamed to INetworkAdapter::attachToBridgedInterface
  - In the IHost interface, createHostNetworkInterface() has been renamed to createHostOnlyNetworkInterface()
  - Similarly, removeHostNetworkInterface() has been renamed to removeHostOnlyNetworkInterface()

15.17 Incompatible API changes with version 2.1

- With VirtualBox 2.1, error codes were added to many error infos that give the caller a machine-readable (numeric) feedback in addition to the error string that has always been available. This is an ongoing process, and future versions of this SDK reference will document the error codes for each method call.

- The hard disk and other media interfaces were completely redesigned. This was necessary to account for the support of VMDK, VHD and other image types; since backwards compatibility had to be broken anyway, we seized the moment to redesign the interfaces in a more logical way.

  - Previously, the old IHardDisk interface had several derivatives called IVirtualDiskImage, IVMDKImage, IVHDImage, IISCSIHardDisk and ICustomHardDisk for the various disk formats supported by VirtualBox. The new IHardDisk2 interface that comes with version 2.1 now supports all hard disk image formats itself.
  - IHardDiskFormat is a new interface to describe the available back-ends for hard disk images (e.g. VDI, VMDK, VHD or iSCSI). The IHardDisk2::format attribute can be used to find out the back-end that is in use for a particular hard disk image. ISystemProperties::hardDiskFormats[] contains a list of all back-ends supported by the system. ISystemProperties::defaultHardDiskFormat contains the default system format.
  - In addition, the new IMedium interface is a generic interface for hard disk, DVD and floppy images that contains the attributes and methods shared between them. It can be considered a parent class of the more specific interfaces for those images, which are now IHardDisk2, IDVDImage2 and IFloppyImage2.

    In each case, the "2" versions of these interfaces replace the earlier versions that did not have the "2" suffix. Previously, the IDVDImage and IFloppyImage interfaces were entirely unrelated to IHardDisk.
  - As a result, all parts of the API that previously referenced IHardDisk, IDVDImage or IFloppyImage or any of the old subclasses are gone and will have replacements that use IHardDisk2, IDVDImage2 and IFloppyImage2; see, for example, IMachine::attachHardDisk2.
  - In particular, the IVirtualBox::hardDisks2 array replaces the earlier IVirtualBox::hardDisks collection.

- IGuestOSType was extended to group operating systems into families and for 64-bit support.
- The IHostNetworkInterface interface was completely rewritten to account for the changes in how Host Interface Networking is now implemented in VirtualBox 2.1.
- The IVirtualBox::machines2[] array replaces the former IVirtualBox::machines collection.
- Added IHost::getProcessorFeature() and ProcessorFeature enumeration.
- The parameter list for IVirtualBox::createMachine() was modified.
- Added IMachine::pushGuestProperty.
- New attributes in IMachine: accelerate3DEnabled, HWVirtExVPIDEnabled, `IMachine::guestPropertyNotificationPatterns`, CPUCount.
- Added IConsole::powerUpPaused() and IConsole::getGuestEnteredACPIMode().
- Removed ResourceUsage enumeration.